BITCOIN

V0.1.0

ANNOTATED EDITION – V0.3.9

Copyright (c) 2009 Satoshi Nakamoto

Annotations by Claude Sonnet 4.6 (Revised by Opus 4.6, GPT 5.6 Sol, Grok 4.5, Kimi K3, Opus 5.5)



To all bitcoiners

ACKNOWLEDGMENTS

This book was made possible by the following sources and resources:

ORIGINAL BITCOIN SOURCE CODE

`satoshi.nakamotoinstitute.org/code`

Archive of Bitcoin v0.1.0 source code, preserved by the Satoshi Nakamoto Institute

BITCOIN WHITEPAPER

`bitcoin.org/bitcoin.pdf`

Satoshi Nakamoto's original paper describing the Bitcoin protocol

BITCOIN WIKI

`en.bitcoin.it`

Community-maintained documentation on Bitcoin internals

LEARN ME A BITCOIN

`learnmeabitcoin.com`

Greg Walker's comprehensive Bitcoin technical guides

BITCOIN STACK EXCHANGE

`bitcoin.stackexchange.com`

Q&A archive of Bitcoin technical discussions

MASTERING BITCOIN

`github.com/bitcoinbook/bitcoinbook`

Andreas Antonopoulos's open source Bitcoin reference

THE GENESIS BOOK

`thegenesisbook.com`

Aaron van Wirdum's history of the people and projects that inspired Bitcoin

PROGRAMMING BITCOIN

`github.com/jimmysong/programmingbitcoin`

Jimmy Song's hands-on guide to Bitcoin internals (O'Reilly)

Special thanks to Satoshi Nakamoto for creating Bitcoin and releasing the source code to the world.

FUNDAMENTALS

Introduction 15

The Road to Bitcoin 19

Computer Concepts 29

Cryptography Basics 35

C++ Primer 51

The Bitcoin Protocol 63

Reading Paths 81

THE SOURCE CODE

rc/ (Resources) 87

base58.h 91

bignum.h 101

db.cpp 115

db.h 137

headers.h 151

irc.cpp 161

irc.h 175

key.h 181

license.txt 191

main.cpp 199

main.h 255

makefile 285

makefile.vc 293

market.cpp 301

market.h 313

net.cpp 323

net.h 349

readme.txt 371

script.h 407
serialize.h 423
sha.cpp 447
sha.h 465
ui.cpp 475
ui.h 523
ui.rc 535
uibase.cpp 541
uibase.h 573
uint256.h 589
uiproject.fbp 609
util.cpp 673
util.h 687

REFERENCE

What Came After 701
Appendix: OpenSSL 711
Glossary 731
Index 746
Colophon 749

FUNDAMENTALS

The history, cryptography, and computer science concepts behind Bitcoin

INTRODUCTION

For three decades before Bitcoin, cryptographers tried to build digital money. Every attempt failed. DigiCash went bankrupt.[1] e-gold was shut down after federal prosecution.[2] Hashcash never became currency.[3] b-money[4] and Bit Gold[5] were never implemented. The deployed systems all failed the same way: someone had to be in charge, and whoever was in charge could be pressured, arrested, or simply lose interest.

On October 31, 2008,[6] a pseudonymous figure calling themselves Satoshi Nakamoto posted a nine-page paper to the Cryptography Mailing List. The paper described "a purely peer-to-peer version of electronic cash"[7] requiring no trusted third party. On January 3, 2009,[8] Nakamoto mined the first block, and six days later, on January 9, released the software to the public.[9]

Embedded in that block was a message: "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks."[10] The headline from a British newspaper served as both a timestamp proof—the block could not have been created before that date—and a declaration of intent. The 2008 financial crisis, the bank bailouts, the revelation that the global economy depended on institutions that had been gambling with depositors' money: these were the conditions that made Bitcoin necessary. You will meet the headline again in this book: it sits in the source code itself, as the `pszTimestamp` string in `main.cpp`.

This book contains the complete source code of Bitcoin version 0.1.0, the first public release.[11][12] Roughly 20,000 lines of C++ that solved a problem widely considered unsolvable: how to prevent double-spending in a digital currency without a central authority. The code has been running continuously since that January day—producing a block approximately every ten minutes for over seventeen years.

WHY THIS CODE MATTERS

Every previous digital currency required trust. Trust that the operator wouldn't inflate the supply. Trust that they wouldn't freeze accounts. Trust that they wouldn't disappear. When trust was violated—or when governments decided the experiment had gone far enough—the money ceased to exist.

Bitcoin replaced trust with verification. Every node independently validates every transaction against the same rules encoded in this source code. No company operates the network. No server can be raided to halt it. The rules are enforced by mathematics and by the economic self-interest of thousands of participants who have no reason to trust each other and don't need to.

Satoshi Nakamoto's identity remains unknown. The code is public, and it speaks for itself.

HOW BITCOIN WORKS

The system rests on five interlocking mechanisms:

- **Transactions** transfer value between addresses, authorized by digital signatures that prove ownership of private keys without revealing them.
- **Blocks** bundle transactions into ordered batches. Each block includes the hash of the previous block, forming an append-only chain that cannot be altered without redoing the work of every subsequent block.
- **Proof-of-work** requires miners to find a hash below a target value—a process that demands real energy expenditure. This makes rewriting history prohibitively expensive.
- **Peer-to-peer network** connects nodes directly, with no central server. Transactions and blocks propagate through gossip: each node relays what it receives to its neighbors, reaching the entire network in seconds.
- **Consensus** emerges from a simple rule: every node follows the longest valid chain—in this first release, simply the one with the most blocks (later versions count total work instead). No voting, no coordinator, no authority.

The key files that implement this system are:

- **main.cpp** — Block validation, mining, consensus rules, and the genesis block
- **script.cpp** — A stack-based virtual machine for programmable spending conditions
- **net.cpp** — Peer-to-peer networking, message relay, and node discovery
- **key.h** — A thin wrapper around OpenSSL's elliptic-curve signatures on secp256k1
- **sha.cpp** — A fast SHA-256 core, borrowed from Wei Dai's Crypto++ library, that powers the mining loop

READING THE CODE

You do not need to be a programmer to read this book. Each source file begins with an introduction explaining its role in the system, and margin annotations throughout highlight what matters and why. The syntax highlighting makes the structure of the code visible at a glance:

ELEMENT	EXAMPLE
Keywords	`if` `int` `return` `class`
Functions	`GetHash()` `printf()`
Strings	`"hello"` `'A'`
Numbers	`42` `0x1F` `3.14`
Comments	`// Satoshi's notes`
Operators	`+` `==` `&&` `->`

Pay particular attention to the comments—the gray italic text beginning with `//`. These are Satoshi's own explanations, written as the code was developed. They reveal the reasoning behind design decisions that are not always obvious from the code alone.

WHAT YOU WILL FIND

This book is organized to build understanding progressively:

- **The Road to Bitcoin** traces the cryptographic ideas and failed projects that preceded Satoshi's synthesis—from Diffie-Hellman[13] to DigiCash[1] to Hal Finney's RPOW.[14]
- **Computer Concepts** and **Cryptography Basics** provide the technical foundation needed to follow the code: binary arithmetic, hash functions, elliptic curves, digital signatures.
- **C++ Primer** explains the programming language, file structure, and idioms you will encounter.
- **The Source Code** presents every file from Bitcoin v0.1.0, annotated line by line with explanations of what the code does, why Satoshi designed it that way, and what happened to it afterward.

Version 0.1.0 is remarkably complete. The fundamental architecture—the UTXO model, proof-of-work consensus, peer-to-peer gossip, the 21-million-coin halving schedule, and the basic shape of the scripting system—remains recognizable after

seventeen years and over a trillion dollars in value secured. Understanding this code means understanding Bitcoin at its deepest level: not what it does, but how and why.

SOURCES

1. David Chaum, *Blind Signatures for Untraceable Payments* (1983). Foundational DigiCash cryptography; Advances in Cryptology. https://sceweb.sce.uhcl.edu/yang/teaching/csci5234WebSecurityFall2011/Chaum-blind-signatures.PDF
2. U.S. Department of Justice, *e-gold indictment and seizure* (2007). https://www.justice.gov/archive/opa/pr/2007/April/07_crm_301.html
3. Adam Back, *Hashcash — A Denial of Service Counter-Measure* (August 2002). http://www.hashcash.org/papers/hashcash.pdf
4. Wei Dai, *b-money* (1998). http://www.weidai.com/bmoney.txt
5. Nick Szabo, *Bit gold* (December 2008). Earlier drafts circulated among cypherpunks; public post dated 2008. https://unenumerated.blogspot.com/2005/12/bit-gold.html
6. Satoshi Nakamoto, *Bitcoin P2P e-cash paper (Cryptography Mailing List)* (October 31, 2008). Initial announcement of the Bitcoin whitepaper. https://satoshi.nakamotoinstitute.org/emails/cryptography/1/
7. Satoshi Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (October 2008). https://bitcoin.org/bitcoin.pdf
8. Satoshi Nakamoto, *Bitcoin genesis block (block 0)* (January 3, 2009). Coinbase embeds The Times headline as timestamp proof. https://mempool.space/block/000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
9. Satoshi Nakamoto, *Bitcoin v0.1 released (Cryptography Mailing List / BitcoinTalk)* (January 9, 2009). Announced Windows-only for the first release. https://satoshi.nakamotoinstitute.org/emails/cryptography/15/
10. The Times, *Chancellor on brink of second bailout for banks* (January 3, 2009). Headline embedded in the Bitcoin genesis block. https://www.thetimes.com/article/chancellor-alistair-darling-on-brink-of-second-bailout-for-banks-n9l382mn62h
11. Satoshi Nakamoto Institute, *Bitcoin v0.1.0 Source Code Archive* (2009). https://satoshi.nakamotoinstitute.org/code/
12. Satoshi Nakamoto, *readme.txt (BitCoin v0.01 ALPHA)* (January 2009). Primary source — distributed with Bitcoin v0.1.0.
13. Whitfield Diffie and Martin Hellman, *New Directions in Cryptography* (1976). IEEE Transactions on Information Theory. https://ee.stanford.edu/~hellman/publications/24.pdf
14. Hal Finney, *RPOW — Reusable Proofs of Work* (2004). https://nakamotoinstitute.org/finney/rpow/

THE ROAD TO BITCOIN

Bitcoin did not emerge from nothing. It stands on three decades of cryptographic research and a graveyard of failed digital currencies. Each predecessor solved part of the puzzle and failed on the rest. Understanding what came before reveals what made Satoshi's synthesis different from everything that preceded it.

THE CYPHERPUNK DREAM

In 1992, a group of cryptographers, programmers, and privacy advocates began meeting in the San Francisco Bay Area and communicating over an unmoderated mailing list. They called themselves the **cypherpunks**—a portmanteau of "cipher" and "cyberpunk." Eric Hughes wrote in "A Cypherpunk's Manifesto" (1993): "Privacy is necessary for an open society in the electronic age... Cypherpunks write code."[1]

Among their goals was anonymous digital cash—money that couldn't be traced, frozen, or confiscated. For the next decade and a half the list's members designed, debated, and sometimes built systems to achieve this. Every attempt failed. But their failures mapped the problem space that Satoshi would eventually solve.

PUBLIC KEY CRYPTOGRAPHY (1976)

In 1976, Whitfield Diffie and Martin Hellman published "New Directions in Cryptography,"[2] introducing the concept of **public key cryptography**. Before this, sharing encrypted messages required both parties to have the same secret key—but how do you securely share that key in the first place?

Diffie-Hellman solved this with an elegant mathematical trick: each party has a private key they never share and a public key they broadcast to the world. The math allows two parties to compute a shared secret without ever transmitting it:

```
Public: prime p, generator g

Alice (private a):          Bob (private b):
Computes A = g^a mod p      Computes B = g^b mod p
Sends A to Bob              Sends B to Alice
Computes B^a mod p          Computes A^b mod p

Shared secret: g^(ab) mod p (same for both!)
```

An eavesdropper sees g, A, and B, but cannot compute g^(ab) mod p without knowing either a or b. This insight—that you can publish part of your key without compromising your secrets—is the foundation of all cryptocurrency.

The Bitcoin whitepaper doesn't cite Diffie and Hellman, but every Bitcoin signature descends from their idea. Your Bitcoin public key can be shared with the world, yet no one can work backward to your private key: Bitcoin's elliptic-curve signatures rest on the same kind of hard problem—the discrete logarithm—that protects Diffie-Hellman key exchange.

DAVID CHAUM'S DIGICASH (1989)

Cryptographer David Chaum invented **blind signatures** and founded DigiCash in 1989—the first attempt at cryptographic digital cash.[3] His "eCash" system used clever math to make transactions untraceable while preventing double-spending.

The problem: DigiCash required a central server to validate transactions. When the company went bankrupt in 1998, the money system died with it.[3] **Lesson learned: centralization is a single point of failure.**

HASHCASH (1997)

Adam Back, a British cryptographer, invented **Hashcash** to combat email spam.[4] The idea: require senders to compute a partial hash collision before sending email. Finding such a collision takes CPU work— trivial for one email, prohibitive for millions of spam messages.

```
Hashcash header: 1:20:060408:adam@cypherspace.org::1QTjaYd7niiQA/sc:ePa

The sender must find a value where SHA1(header) starts with 20 zero bits.
This requires ~2^20 hash computations—about a second of CPU time.

Check it yourself: SHA1 of this exact header begins 00000a4a…
—five zero hex digits, or 20 zero bits.
```

The Bitcoin whitepaper explicitly cites Hashcash in its references.[5][4] Satoshi adapted the concept: instead of proving work to send email, miners prove work to add blocks. The difficulty is adjusted so the network collectively finds one valid proof every 10 minutes on average. Every 2,016 blocks—about two weeks at ten minutes each—v0.1.0 measures how long they actually took and raises or lowers the target to match (`nTargetSpacing = 10 * 60` in `main.cpp`).

B-MONEY (1998)

Wei Dai, a cypherpunk and computer scientist, proposed **b-money**— a theoretical system for anonymous digital cash.[6] His paper describes:

- Money created through proof-of-work
- Transactions broadcast to all participants
- A collective ledger maintained by the network
- Contracts enforced by deposits and arbitration

Sound familiar? Wei Dai's b-money was never implemented, but Satoshi cited it first in the Bitcoin whitepaper's references.[5][6] In early emails, Satoshi told Wei Dai: "I was very interested to read your b-money page. I'm getting ready to release a paper that expands on your ideas into a complete working system."[7]

Fittingly, Wei Dai's own code ships inside Bitcoin v0.1.0: the SHA-256 routine in `sha.cpp` , which Bitcoin's miner runs over and over in its search for a valid block, comes from his Crypto++ library.

BIT GOLD (1998-2005)

Nick Szabo was the son of a Hungarian refugee who fled after Soviet troops crushed the 1956 revolt. This shaped his distrust of government overreach and drew him to Hayek's writings on monetary economics. After studying computer science and working at NASA's Jet Propulsion Laboratory, he joined the cypherpunks and briefly worked at DigiCash in Amsterdam.[8][3]

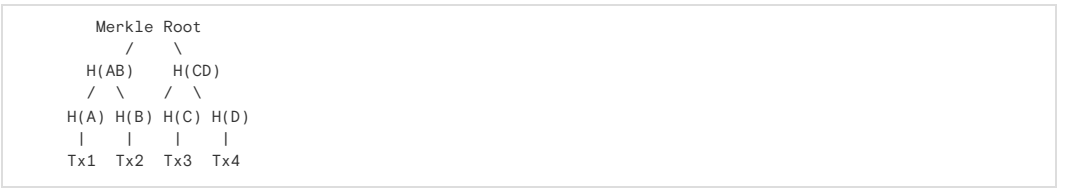
At DigiCash, Szabo saw firsthand how easy it was to manipulate balances in a centralized system. This led to his influential essay "Trusted Third Parties Are Security Holes"[9]—the insight that any system depending on a central party inherits that

party's vulnerabilities, whether from hackers, rogue employees, or government pressure.

Szabo designed **Bit Gold** to eliminate trusted third parties entirely.[10] His system used proof-of-work to create tokens, chained hashes together (each valid hash becoming the input for the next), and proposed a distributed "property club" to track ownership. It was never implemented, but its architecture clearly influenced Bitcoin.[10]

MERKLE TREES (1979)

Ralph Merkle invented the **Merkle tree**—a data structure that efficiently summarizes large amounts of data into a single hash.[11] His original patent was for digital signatures, but the concept proved far more versatile.



Bitcoin uses Merkle trees to commit to all transactions in a block while enabling "light clients" that can verify transaction inclusion without downloading the entire blockchain. The whitepaper's Section 7 explicitly credits "Protocols for Public Key Cryptosystems" (Merkle, 1980).[5][11]

TIMESTAMPING (1991)

Stuart Haber and W. Scott Stornetta published "How to Time-Stamp a Digital Document"[12]—a system that proves when a document existed by linking each timestamp to the hash of the one before it. Their follow-up work (1993) added Merkle trees, so that many documents could be summarized in a single hash. Both ideas directly influenced Bitcoin's blockchain structure.

The Bitcoin whitepaper cites three Haber-Stornetta papers.[5][12] Their idea of chaining hashes together—where each timestamp includes the hash of the previous one—is exactly how Bitcoin blocks link together.

RPOW: REUSABLE PROOF OF WORK (2004)

Hal Finney graduated top of his high-school class and studied engineering at Caltech, then spent years developing video games for Intellivision and Atari before becoming a core contributor to PGP.[8] Unlike some cypherpunks, Finney was a pragmatist. He didn't believe cryptography could create an anarchist utopia: "... there is no such place as cyberspace. I am not in cyberspace now; I am in California. ... It is the government in physical space I fear. Its agents carry physical guns which shoot real bullets."[13]

Still, Finney wanted electronic cash to exist. In 2004, he launched **RPOW** (Reusable Proofs of Work).[14] Users would generate a hashcash-style proof-of-work and exchange it for a token. That token could then be transferred to others, who would redeem it for a fresh token—making the proof of work reusable.

To prevent double-spending without trusting the server operator (himself), Finney ran RPOW on tamper-proof IBM hardware that could cryptographically prove it was running unmodified open-source code.[14] It was clever, but RPOW failed to gain users. The tokens had no reason to hold value—Moore's Law would make them cheaper to produce over time—and without users, there was nothing to buy.

When Satoshi announced Bitcoin on the cryptography mailing list in 2008,[15] Hal Finney was among the first to respond positively. He missed the January 9 launch; his node joined around block 49 on January 10 (as recorded in a `debug.log` file he later shared when reporting a crash), and on January 12 he received the first Bitcoin transaction from one person to another: 10 BTC from Satoshi in block 170.[16][17][18] Finney ran one of the first Bitcoin nodes and contributed code fixes before his death in 2014.

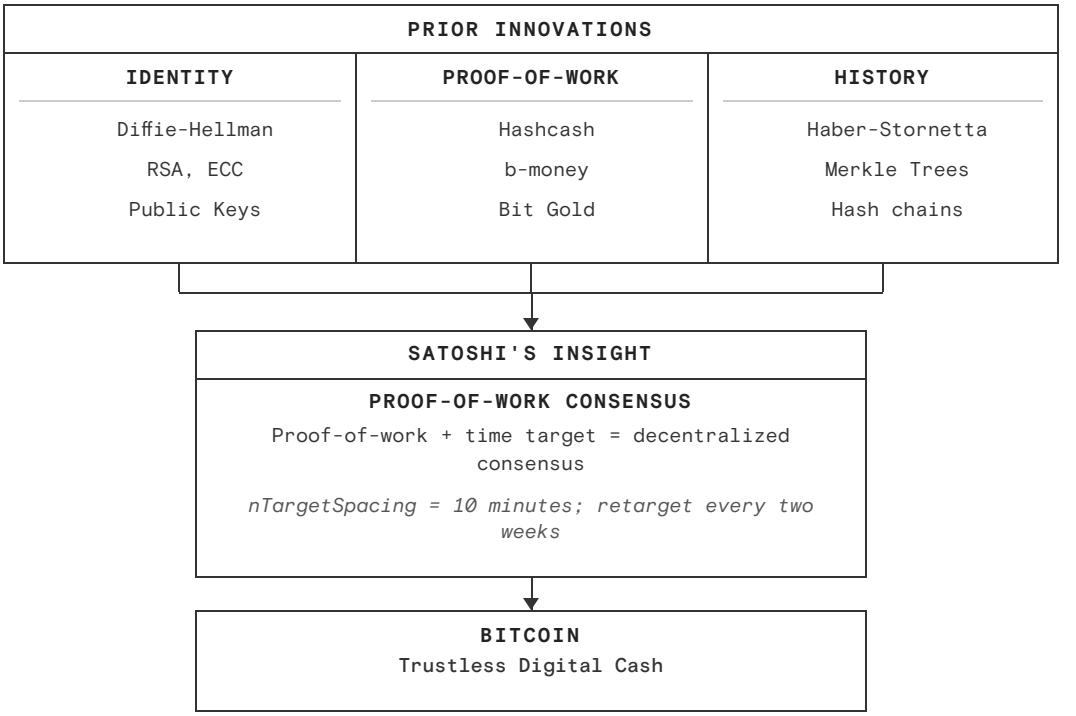
SATOSHI'S SYNTHESIS

Every predecessor solved part of the puzzle:

INNOVATION	CONTRIBUTION
Diffie-Hellman / RSA	Cryptographic keys for identity
DigiCash	Blind signatures (but centralized)
Hashcash	Proof-of-work as rate limiting

b-money	Decentralized ledger concept
Bit Gold	Chained proof-of-work tokens
Merkle trees	Efficient data commitment
Haber-Stornetta	Hash-linked timestamping
RPOW	Transferable proof-of-work

Satoshi's breakthrough was not any single invention but a synthesis—combining these existing pieces with one novel insight:



Use proof-of-work not just to create money, but to reach **consensus** about which transactions are valid. The longest valid chain is the true history, and a difficulty target adjusted every two weeks keeps new blocks arriving at a steady pace. No trusted party needed—just math and economic incentives.

As Satoshi wrote: "A lot of people automatically dismiss e-currency as a lost cause because of all the companies that failed since the 1990's. I hope it's obvious it was only the centrally controlled nature of those systems that doomed them."[19]

SOURCES

1. Eric Hughes, *A Cypherpunk's Manifesto* (March 9, 1993). https://www.activism.net/cypherpunk/manifesto.html
2. Whitfield Diffie and Martin Hellman, *New Directions in Cryptography* (1976). IEEE Transactions on Information Theory. https://ee.stanford.edu/~hellman/publications/24.pdf
3. David Chaum, *Blind Signatures for Untraceable Payments* (1983). Foundational DigiCash cryptography; Advances in Cryptology. https://sceweb.sce.uhcl.edu/yang/teaching/csci5234WebSecurityFall2011/Chaum-blind-signatures.PDF
4. Adam Back, *Hashcash — A Denial of Service Counter-Measure* (August 2002). http://www.hashcash.org/papers/hashcash.pdf
5. Satoshi Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (October 2008). https://bitcoin.org/bitcoin.pdf
6. Wei Dai, *b-money* (1998). http://www.weidai.com/bmoney.txt
7. Satoshi Nakamoto, *Email to Wei Dai* (2009). On expanding on b-money ideas. https://satoshi.nakamotoinstitute.org/emails/other/1/
8. Aaron van Wirdum, *The Genesis Book* (2024). History of the people and projects that inspired Bitcoin. https://thegenesisbook.com
9. Nick Szabo, *Trusted Third Parties Are Security Holes* (2001). https://nakamotoinstitute.org/trusted-third-parties/
10. Nick Szabo, *Bit gold* (December 2008). Earlier drafts circulated among cypherpunks; public post dated 2008. https://unenumerated.blogspot.com/2005/12/bit-gold.html
11. Ralph C. Merkle, *Protocols for Public Key Cryptosystems* (1980). Introduced Merkle trees (hash trees). https://www.merkle.com/papers/Protocols.pdf
12. Stuart Haber and W. Scott Stornetta, *How to Time-Stamp a Digital Document* (1991). Journal of Cryptology; cited in the Bitcoin whitepaper. https://www.anf.es/pdf/Haber_Stornetta.pdf
13. Hal Finney, *Post to the Cypherpunks mailing list* (August 1994). "There is no such place as cyberspace". https://cypherpunks.venona.com/date/1994/08/msg00239.html
14. Hal Finney, *RPOW — Reusable Proofs of Work* (2004). https://nakamotoinstitute.org/finney/rpow/
15. Satoshi Nakamoto, *Bitcoin P2P e-cash paper (Cryptography Mailing List)* (October 31, 2008). Initial announcement of the Bitcoin whitepaper. https://satoshi.nakamotoinstitute.org/emails/cryptography/1/
16. Hal Finney, *[bitcoin-list] Crash in bitcoin 0.1.0 (with debug.log)* (January 10, 2009). SourceForge post attaching Finney's debug.log; forensic evidence he joined near block 49. https://web.archive.org/web/20141130200234/https://sourceforge.net/p/bitcoin/mailman/message/21295694/
17. Alex Waltz, *Who was the 1st Bitcoin Miner?* (2026). Forensic reconstruction of Bitcoin's first days from Finney's debug.log, emails, and ExtraNonce patterns. https://1stbitcoinminer.com/
18. Hal Finney, *Bitcoin transaction — first received coins from Satoshi* (January 12, 2009). Block 170; Finney was the first person other than Satoshi to receive bitcoin. https://bitcointalk.org/index.php?topic=13.msg46#msg46

19. Satoshi Nakamoto, *Bitcoin open source implementation of P2P currency (P2P Foundation)* (February 15, 2009). On centrally controlled e-currency systems being doomed. https://satoshi.nakamotoinstitute.org/posts/p2pfoundation/2/

COMPUTER CONCEPTS

Before diving into the code, it helps to understand the fundamental concepts that appear throughout Bitcoin's implementation. None of this is difficult, but the code will make much more sense with these building blocks in place.

BITS AND BYTES

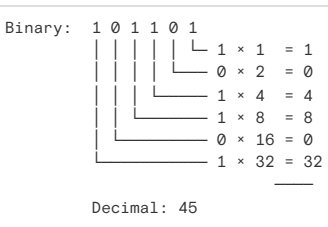
A **bit** is the smallest unit of data—a single 0 or 1. Eight bits make a **byte**, which can represent values from 0 to 255. All computer data, from text to images to Bitcoin transactions, is ultimately stored as bytes.

Common sizes you'll see in Bitcoin:

SIZE	USAGE
32 bits (4 bytes)	Standard integers, timestamps
64 bits (8 bytes)	Large numbers, Bitcoin amounts (in the smallest unit, 10^{-8} BTC—later nicknamed a satoshi)
160 bits (20 bytes)	Bitcoin addresses (RIPEMD-160 hash)
256 bits (32 bytes)	SHA-256 hashes, private keys

BINARY (BASE-2)

Computers think in **binary**—base-2, using only 0 and 1. Each position represents a power of 2:



HEXADECIMAL (BASE-16)

Binary is verbose, so programmers use **hexadecimal** (hex)—base-16. It uses digits 0-9 and letters A-F (where A=10, B=11, … F=15). Each hex digit represents exactly 4 bits:

```
Hex:  0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
Dec:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
```

Bitcoin hashes are typically shown in hex. A 256-bit hash becomes 64 hex characters:

```
000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
```

This is the hash of Bitcoin's genesis block—the first block ever mined.[1]

Text is just bytes too. The genesis block coinbase embeds a newspaper headline as ASCII—each character is one byte, which you can also write as hex or binary. Here is the opening of that message:[2]

```
ASCII:  T  h  e     T  i  m  e  s     0  3  /  J  a  n  /  2  0  0  9
Hex:   54 68 65 20 54 69 6d 65 73 20 30 33 2f 4a 61 6e 2f 32 30 30 39

Binary (first four bytes — "The "):
  'T'  = 01010100 = 0x54
  'h'  = 01101000 = 0x68
  'e'  = 01100101 = 0x65
  ' '  = 00100000 = 0x20

Full string: "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks"
```

Same data, three notations: readable text (ASCII), compact hex, and the bits the machine actually stores. The rest of the book flips between these views constantly—especially once byte *order* enters the picture.

LITTLE ENDIAN VS BIG ENDIAN

When storing multi-byte numbers, computers must decide which byte comes first. **Big endian** stores the most significant byte first (like how we write numbers). **Little endian** stores the least significant byte first.

```
Number: 0x12345678 (305,419,896 in decimal)

Big Endian:    [12] [34] [56] [78]  (most significant first)
Little Endian: [78] [56] [34] [12]  (least significant first)
```

Bitcoin uses little endian for most internal data but big endian for displaying hashes. This can be confusing—the genesis block hash shown above is actually stored in reverse order internally.

MEMORY AND POINTERS

Computer memory is like a vast array of numbered boxes, each holding one byte. The box number is called an **address**. A **pointer** is a variable that stores an address—it "points to" data elsewhere in memory.

In C++ code, you'll see:

- `*ptr` — The value that ptr points to
- `&var` — The address of var
- `ptr->field` — Access a field through a pointer

DATA TYPES

Bitcoin uses several C++ data types:

- `int` — Integer (usually 32 bits)
- `int64` — 64-bit integer (for Bitcoin amounts)
- `unsigned` — Non-negative integers only
- `char` — Single byte, often used for raw data
- `string` — Text or arbitrary byte sequences
- `vector` — Resizable array
- `map` — Key-value dictionary
- `bool` — True or false

CRYPTOGRAPHIC HASHING

A **hash function** takes any input and produces a fixed-size output (the "hash" or "digest"). Bitcoin primarily uses SHA-256,[3] which always produces 256 bits.

Key properties:

- **Deterministic** — Same input always gives same output
- **One-way** — Cannot reverse a hash to find the input
- **Avalanche effect** — Tiny input change completely changes output
- **Collision resistant** — Practically impossible to find two inputs with same hash

```
SHA256("Bitcoin") =
  b4056df6691f8dc72e56302ddad345d65fead3ead9299609a826e2344eb63aa4

SHA256("bitcoin") =  (just lowercase 'b')
  6b88c087247aa2f07ee1c5956b8e1a9f4c7f892a70e324f1bb3d161e05ca107b
```

PUBLIC KEY CRYPTOGRAPHY

Bitcoin uses **elliptic curve cryptography** for digital signatures. You have two related keys:

- **Private key** — A secret 256-bit number. Never share this!
- **Public key** — Derived from the private key. Safe to share.

The private key can sign messages; anyone with the public key can verify the signature is authentic. This is how Bitcoin proves ownership—only the holder of the private key can sign transactions spending their coins.

READING C++ SYNTAX

Some patterns you'll encounter:

- `// comment` — Single-line comment (explanatory text)
- `/* comment */` — Multi-line comment
- `if (condition) { ... }` — Conditional execution
- `for (init; condition; increment) { ... }` — Loop
- `class Name { ... };` — Define a data structure
- `return value;` — Exit function with result
- `#include` — Import another file
- `#define` — Create a constant or macro

SOURCES

1. Satoshi Nakamoto, *Bitcoin genesis block (block 0)* (January 3, 2009). Coinbase embeds The Times headline as timestamp proof. https://mempool.space/block/000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
2. The Times, *Chancellor on brink of second bailout for banks* (January 3, 2009). Headline embedded in the Bitcoin genesis block. https://www.thetimes.com/article/chancellor-alistair-darling-on-brink-of-second-bailout-for-banks-n9l382mn62h
3. NIST, *FIPS PUB 180-2 — Secure Hash Standard (SHA-256)* (2002). SHA-256 published August 2002 (draft 2001). https://csrc.nist.gov/publications/detail/fips/180/2/archive/2002-08-01

CRYPTOGRAPHY BASICS

Bitcoin's security rests on mathematical foundations developed over decades of cryptographic research. The two pillars—digital signatures and hash functions—are not merely implementation details but the reason Bitcoin works at all. Understanding them reveals why the system is secure and what would have to break for it to fail.

TWO PILLARS OF BITCOIN CRYPTOGRAPHY

	ECDSA	SHA-256
Purpose	Proves ownership	Creates fingerprints
Used for	Signing transactions	Mining, block hashes, Merkle trees
Property	Asymmetric (public/private keys)	One-way (irreversible)

ECDSA lets you prove you own coins without revealing your private key. **SHA-256** creates a practically unique 256-bit fingerprint of any data—change one bit and the hash changes completely.

THE DISCRETE LOGARITHM PROBLEM

Many cryptographic systems rely on operations that are easy in one direction but practically impossible to reverse. Consider this:

```
Easy:    3^15 mod 17 = 6      (compute in milliseconds)
Hard:    3^? mod 17 = 6       (which exponent gives 6?)
```

For small numbers, you can try all possibilities. But with 256-bit numbers, there are roughly as many possibilities as atoms in the observable universe. This asymmetry—easy forward, impossible backward—is the foundation of public key cryptography.

ELLIPTIC CURVE CRYPTOGRAPHY (ECC)

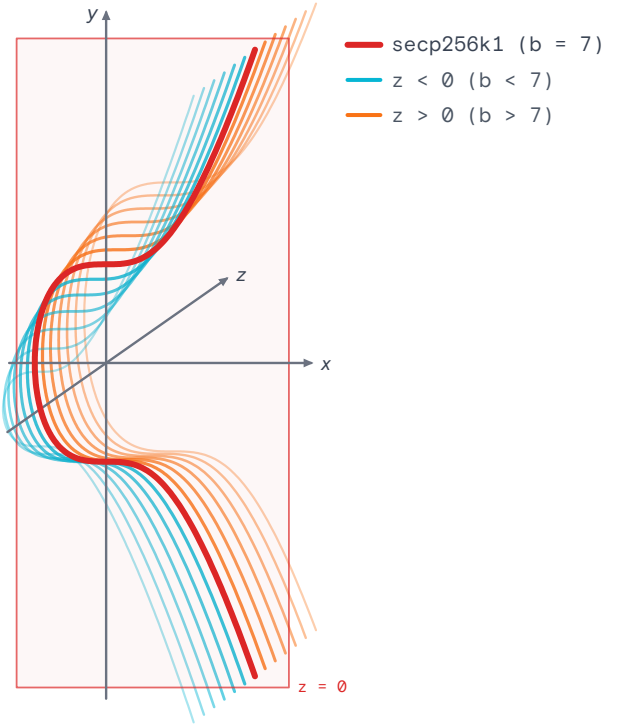
Bitcoin uses a specific type of discrete logarithm problem based on **elliptic curves**. An elliptic curve is defined by an equation like:

```
y² = x³ + ax + b

Bitcoin (secp256k1): y² = x³ + 7   (a=0, b=7)
```

The curve has a distinctive shape—symmetric about the x-axis, with a rounded "nose" on the left and extending to infinity on the right. The figure below stacks neighboring curves $y^2 = x^3 + b$ to show how secp256k1 sits in its family.

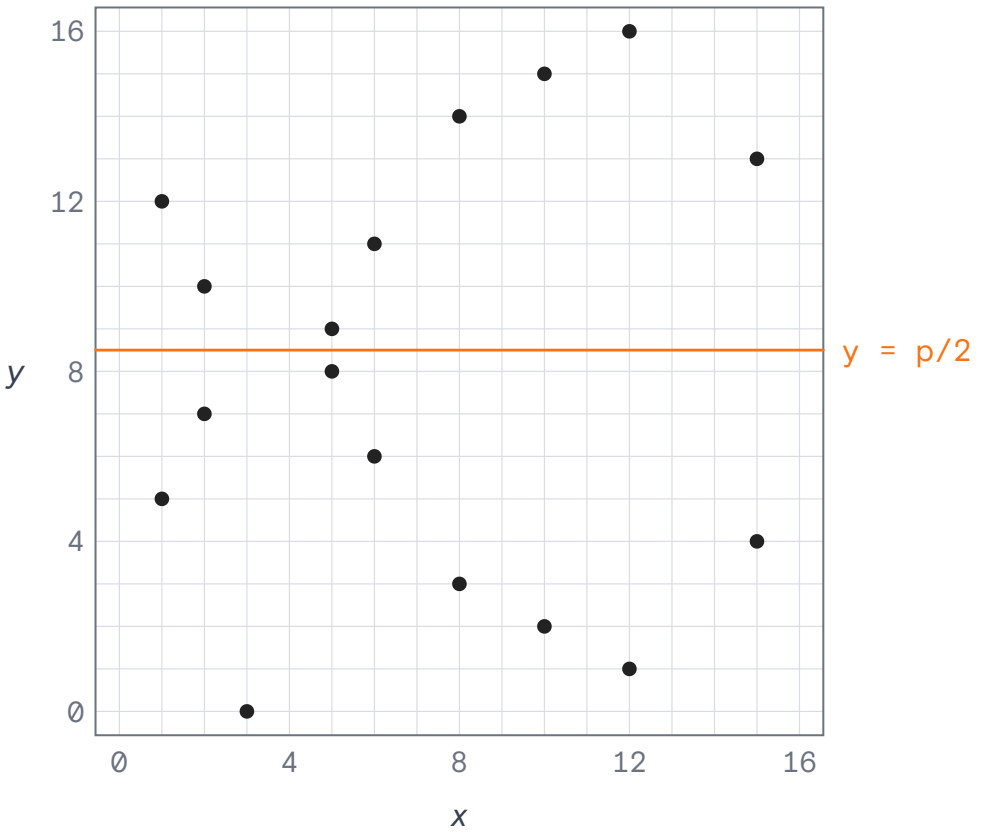
Curve family along z: each slice is $y^2 = x^3 + b$ for $b = 7 + z$. Red = secp256k1 at $z = 0$ · cyan = smaller b · orange = larger b.




```
Weierstrass family
y² = x³ + (7 + z)   ·  slices Δz = 1
z = 0  →  secp256k1: y² = x³ + 7
z < 0  →  smaller b (cyan)   ·  z > 0  →  larger b (orange)
```

In real cryptography the coordinates are not arbitrary-precision reals: they live in a **finite field** $\mathbb{F}_p$ (for secp256k1, a huge prime p). Addition still follows chord-and-tangent, but values wrap modulo p. A tiny field makes the pattern visible—each valid (x, y) pair below satisfies $y^2 \equiv x^3 + 7 \pmod{17}$:

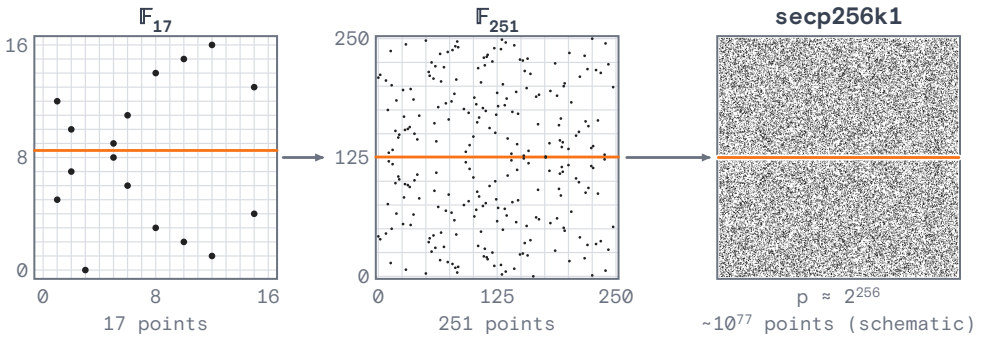
$\mathbb{F}_{17}$ schematic grid: each ● is an integer pair (x, y) with $y^2 \equiv x^3 + 7 \pmod{17}$. Orange midline = y = p/2 (negation mirror).




```
Modular picture (toy field)
Prime p = 17. Integer pairs (x, y) with y² ≡ x³ + 7 (mod p).
Negation (x, y) ↦ (x, −y) reflects across y = p/2 (orange).
```

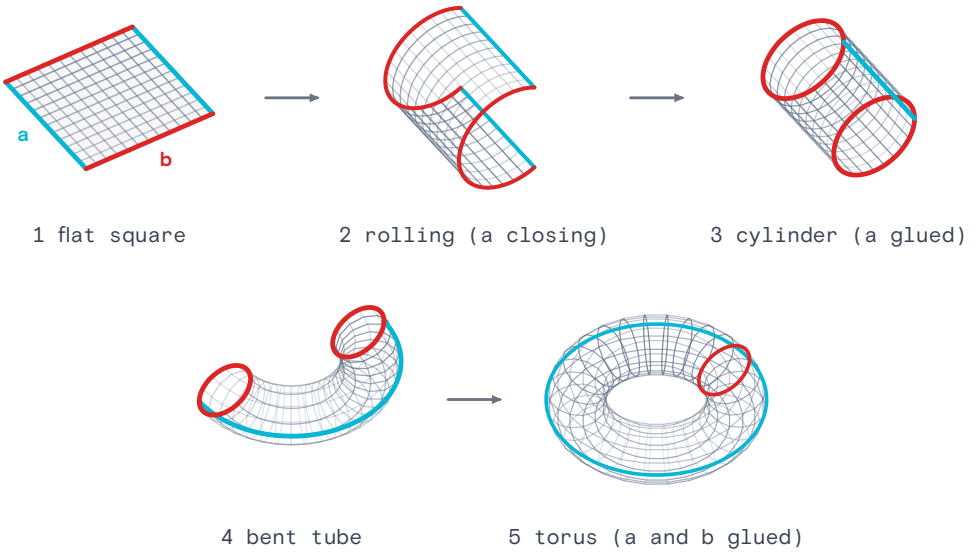
Raise the prime and the same equation just plants more dots on a finer lattice— still discrete, still mirrored across y = p/2. Bitcoin's field is so large that a literal plot is impossible; the right panel is only a density schematic.

Same curve, larger fields: $\mathbb{F}_{17}$ → $\mathbb{F}_{251}$ (real points) → secp256k1 schematic ($p \approx 2^{256}$, ~10^{77} points — faked density so the scale jump is visible). Orange midline = negation in every panel.

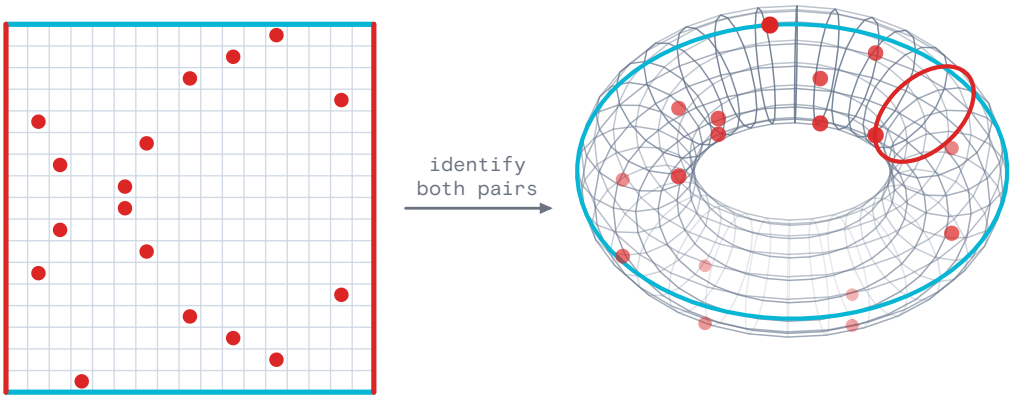


The scatter plot looks nothing like the smooth curve over $\mathbb{R}$ — and that is the point. Because both x and y wrap at p (0 and p are the same value), it helps to picture the grid with its top edge glued to its bottom and its left edge glued to its right—like a Pac-Man screen. Gluing one pair of opposite edges gives a cylinder; gluing both pairs gives a **torus**. This is a visualization aid, not literal geometry—though, fittingly, over the complex numbers an elliptic curve really is a torus.

From flat square to torus: identifying opposite edges. a = cyan · b = red — roll the sheet until the a-edges meet (cylinder), then bend and glue b.



From flat grid to torus: modular wrapping in both axes. $\mathbb{F}_{17}$ (p = 17); ● = $y^2 \equiv x^3 + 7 \pmod{17}$. a = top/bottom (cyan) · b = left/right (red). Torus maps the same curve points — secp256k1 uses $p \approx 2^{256}$ (same topology, vastly larger).



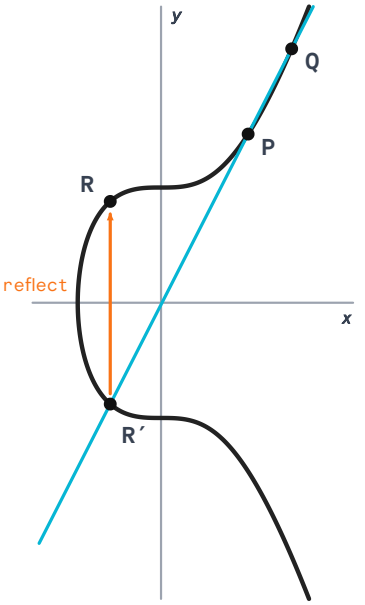
For any point P = (x, y) on the curve, negation is reflection across the horizontal midline: −P = (x, −y mod p). On the torus this flips each cross-section circle top-to-bottom. Addition still follows chord-and-tangent — but now the "line" wraps modulo p.

The points on an elliptic curve form a group under a geometric operation called **point addition** (chord-and-tangent).[1] Over $\mathbb{R}$ the picture is easy to draw; over $\mathbb{F}_p$ the same algebra holds with modular arithmetic. The rules in brief:

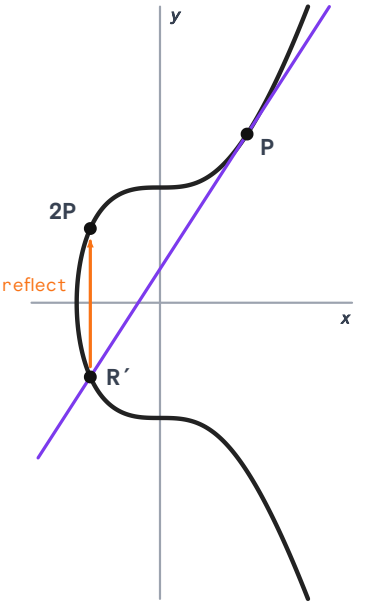
- **Identity** $\mathcal{O}$ (the point at infinity): $P \oplus \mathcal{O} = P$.
- **Negation**: −P = (x, −y). The vertical line through P and −P meets the curve at $\mathcal{O}$, so $P \oplus (-P) = \mathcal{O}$.
- **Addition** $P \oplus Q$ ($P \neq Q$): draw the line through both points; it hits the curve again at R′; reflect across the x-axis to get $R = P \oplus Q$.
- **Doubling** $2P = P \oplus P$: same idea with the *tangent* at P.
- **Subtraction**: $P - Q := P \oplus (-Q)$.

Group law on $y^2 = x^3 + 7$: (a) chord through P, Q → reflect R′ → R; (b) tangent at P → 2P; (c) vertical line → identity $\mathcal{O}$; (d) subtract by adding −Q.

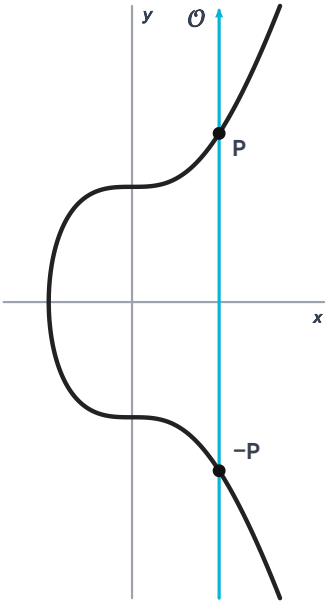
(a) Addition P ⊕ Q



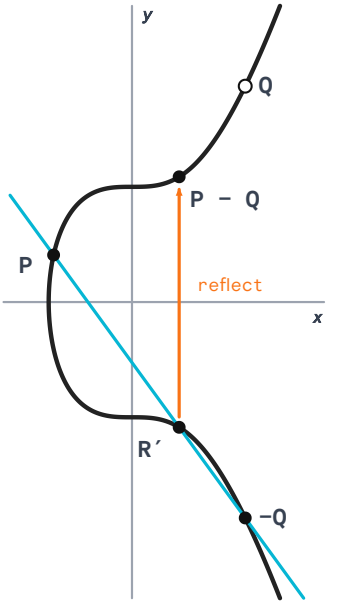
(b) Doubling 2P = P ⊕ P



(c) Inverse P ⊕ (−P) = $\mathcal{O}$



(d) Subtraction P − Q = P ⊕ (−Q)



Over $\mathbb{F}_{17}$ the same recipe is pure arithmetic. Take two dots from the toy grid, P = (5, 8) and Q = (6, 6). The usual chord formulas apply, with every result reduced mod 17:

```
slope  s  = (6 − 8) / (6 − 5) = −2 ≡ 15          (mod 17)
x₃ = s² − x₁ − x₂ = 225 − 5 − 6 = 214 ≡ 10       (mod 17)
y₃ = s(x₁ − x₃) − y₁ = 15·(5 − 10) − 8 = −83 ≡ 2 (mod 17)

P ⊕ Q = (10, 2)     check: 2² = 4,  10³ + 7 = 1007 ≡ 4 (mod 17) ✓
```

Using these operations, you can compute kG (adding G to itself k times):

```
G + G = 2G
2G + G = 3G
...
kG = P    (add G to itself k times)
```

Given G and P, finding k is the **elliptic curve discrete logarithm problem**—believed to be computationally infeasible for large k.

Bitcoin uses the **secp256k1** curve, where k is your private key and P is your public key. Anyone can verify that P = kG, but nobody can determine k from P alone.

THE GENERATOR POINT G

The generator point G is a specific point on the secp256k1 curve that serves as the starting point for all key derivations. It's a publicly known constant— everyone uses the same G. The secp256k1 generator point coordinates are:

```
G.x = 0x79BE667EF9DCBBAC55A06295CE870B07
      029BFCDB2DCE28D959F2815B16F81798

G.y = 0x483ADA7726A3C4655DA4FBFC0E1108A8
      FD17B448A68554199C47D08FFB10D4B8
```

These coordinates look random, and their derivation was never publicly explained—but the curve's other parameters are deliberately simple: the prime $p = 2^{256} - 2^{32} - 977$, and the coefficients a = 0, b = 7. (The choice of G doesn't affect security—any generator of the group is equally hard to attack.) Contrast NIST's P-256 curve, whose coefficients come from an unexplained random seed; after the 2013 Dual_EC_DRBG revelations, secp256k1's transparent, "nothing up my sleeve" parameters looked prescient.[2] Satoshi didn't define G in Bitcoin's code; he used OpenSSL's built-in `NID_secp256k1` curve identifier, which includes G and all other curve parameters.[3]

When you generate a Bitcoin key pair:

```
// Your private key: a random 256-bit number
k = 0x1 to 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFE
    BAAEDCE6AF48A03BBFD25E8CD0364140

// Your public key: k times G (point multiplication)
P = k × G

// Computing P from k: ~0.001 seconds
// Computing k from P: longer than the age of the universe (all computers on Earth)
```

This asymmetry—trivial to compute forward, impossible to reverse—is why your public key (and Bitcoin address) can be safely shared while your private key must remain secret.

A NOTE ON THE SOURCE CODE

You won't find elliptic curve math in Bitcoin's source code. Satoshi delegated all the elliptic-curve cryptography to **OpenSSL**,[3] a battle-tested library. (The one exception elsewhere: the miner's inner loop uses a public-domain SHA-256 routine borrowed from Wei Dai's Crypto++, in `sha.cpp`.) In `key.h`, the entire CKey class is a thin wrapper around OpenSSL functions:

```
// Condensed from key.h - Satoshi's entire ECC implementation:
EC_KEY* pkey = EC_KEY_new_by_curve_name(NID_secp256k1);  // Create key
EC_KEY_generate_key(pkey);                               // Generate keypair
ECDSA_sign(...);                                         // Sign data
ECDSA_verify(...);                                       // Verify signature
```

The point addition, scalar multiplication, and all the complex curve arithmetic live inside OpenSSL's compiled libraries. This was wise— cryptographic code is notoriously difficult to implement securely, and using a well-audited library avoided potential vulnerabilities. The appendix at the back of the book, *OpenSSL as Bitcoin Used It*, follows these calls into the library's own source.

DIGITAL SIGNATURES (ECDSA)

Bitcoin uses the **Elliptic Curve Digital Signature Algorithm** (ECDSA) to prove ownership. When you spend bitcoins, you create a signature that proves you know the private key without revealing it.

```
Signing (private key k, message m):
1. Pick a fresh secret random nonce j
2. Compute R = jG; let r = R.x mod n
3. Compute s = (hash(m) + r × k) / j   (all mod n)
4. Signature is (r, s)

Verification (public key P, message m, signature (r, s)):
1. u1 = hash(m) / s,  u2 = r / s   (mod n)
2. Compute the point u1×G + u2×P
3. Valid if its x-coordinate equals r
```

Here n is the number of points in the group generated by G, and "dividing" means multiplying by a modular inverse. The signature proves you know k (private key) because only someone who knows k could produce an s that satisfies the verification equation.

The nonce j must be secret and never reused: sign two messages with the same nonce and anyone can solve for your private key. Signing is only as strong as its randomness—in August 2013, a weakness in Android's secure random number generator forced an urgent security alert for every Bitcoin wallet app on that platform.[4]

HASH FUNCTIONS IN BITCOIN

Bitcoin uses several hash functions for different purposes:

SHA-256 (SECURE HASH ALGORITHM)

The primary hash function.[5] Takes any input and produces a 256-bit output. Used for:

- Block hashes (proof-of-work mining)
- Transaction IDs
- Merkle tree construction

How SHA-256 Works (Step by Step)

SHA-256 processes input through four main steps:

Step 1: Padding

The message is padded to a multiple of 512 bits: append a '1' bit, then '0' bits, then the original length as a 64-bit integer.

Step 2: Initialize Hash Values

Eight 32-bit words are initialized from the square roots of the first 8 primes:

```
H0 = 0x6a09e667   H4 = 0x510e527f
H1 = 0xbb67ae85   H5 = 0x9b05688c
H2 = 0x3c6ef372   H6 = 0x1f83d9ab
H3 = 0xa54ff53a   H7 = 0x5be0cd19
```

Step 3: Process Blocks (64 Rounds)

For each 512-bit block, run 64 rounds of compression. Each round:

```
T1 = h + S1(e) + Ch(e,f,g) + K[t] + W[t]
T2 = S0(a) + Maj(a,b,c)

Then rotate: h=g, g=f, f=e, e=d+T1, d=c, c=b, b=a, a=T1+T2
```

The operations mix bits thoroughly:

- **Ch** (Choose): if e then f else g
- **Maj** (Majority): majority vote of a,b,c
- **S0, S1**: rotate and XOR operations
- **K[t]**: 64 constants from cube roots of primes
- **W[t]**: message schedule derived from input

Step 4: Output

Concatenate the final H0-H7 values to produce 256 bits (64 hex chars):

```
SHA256("Bitcoin") = b4056df6691f8dc72e56302ddad345d65...
```

The key insight: every bit of input affects every bit of output through 64 rounds of mixing. The constants K[0..63] are derived from cube roots of the first 64 primes—nothing up anyone's sleeve.

Bitcoin often uses **double SHA-256** (SHA256(SHA256(x))). Satoshi never explained why; a common reading is that it guards against length-extension attacks.

THE AVALANCHE EFFECT

A critical property of cryptographic hash functions is the **avalanche effect**: changing even a single character in the input produces a completely different hash. This makes it impossible to predict or reverse-engineer the output:

```
Input:  "The Times 03/Jan/2009"
SHA256: 99fc2183119bc964034d6da7acdee6b4a0c379da75e076edb8f66a78a345afc8

Input:  "The Times 03/Jan/2008"  // Changed 2009 → 2008
SHA256: 55a253ca0bca02e1e5c2f1924b59fa626aecda10ae2d844937fa9868bc9cea5c

Input:  "the Times 03/Jan/2009"  // Changed T → t
SHA256: d91f6a7a6095c6c936748d7fbd31171f8831538947e782ac1eeb6f648129afc5
```

Notice how changing just one digit (2009→2008) or one letter's case (T→t) produces a hash that shares virtually nothing with the original. Because the hash commits to every byte, the headline can't be altered after the fact without producing a completely different block hash. That is what makes Satoshi's embedded Times headline a timestamp: the block couldn't have been created before that headline existed.[6][7]

RIPEMD-160

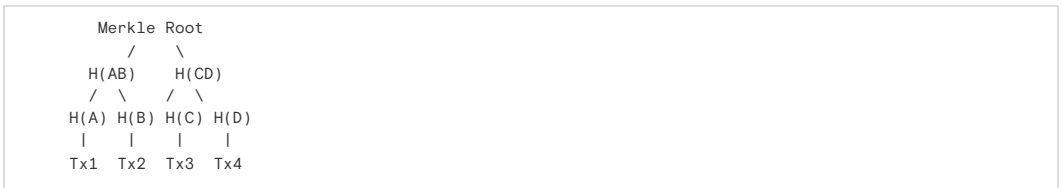
A 160-bit hash function used in combination with SHA-256 to create Bitcoin addresses:

```
Public Key (65 bytes)
    ↓ SHA-256
256-bit hash
    ↓ RIPEMD-160
160-bit hash (20 bytes)
    ↓ Base58Check encoding
Bitcoin Address (e.g., 1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa)
```

RIPEMD-160 shortens the result to 20 bytes, keeping addresses compact. Satoshi never explained why he paired two different hash functions; a common reading is defense in depth, since an attacker would have to defeat both to forge a matching public key.

MERKLE TREES

A **Merkle tree** is a data structure that efficiently summarizes many items into a single hash.[8] Each Bitcoin block contains a Merkle root that commits to all transactions in the block.



To prove Tx2 is in the block, you need only Tx2 itself, H(A), and H(CD): hash upward and compare with the Merkle root—not all four transactions. This enables "light clients" that verify transactions without downloading the entire blockchain.

PROOF OF WORK

Mining is a cryptographic lottery.[9] Miners must find a number (nonce) such that the block's hash starts with many zeros:

```
Target (v0.1.0 minimum difficulty): hash must be less than
        00000000fff0000000000000000000000000000000000000000000000000000

Miner tries (hash = double SHA-256 of the block header):
  nonce=0: hash = 7f3a2b... (too high)
  nonce=1: hash = 9c4d5e... (too high)
  nonce=2: hash = 1b8f7a... (too high)
  ...billions of attempts...
  nonce=2083236893: hash =
    000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f (valid!)
```

That last line is real: it is the nonce and hash of the genesis block.[7] Because SHA-256 is unpredictable, the only way to find a valid nonce is trial and error. The difficulty adjusts so that the network finds a block approximately every 10 minutes, regardless of total computing power. In v0.1.0 this is `nTargetSpacing = 10 * 60` seconds, and the target is recalculated every two weeks' worth of blocks (14 days ÷ 10 minutes = 2,016 blocks), not continuously.

WHY THIS MATTERS

These cryptographic primitives work together to create Bitcoin's security model:

- **Private keys** prove ownership (discrete log is hard)
- **Signatures** authorize transactions (ECDSA)
- **Hashes** create unique identifiers and commitments
- **Merkle trees** efficiently verify inclusion
- **Proof of work** makes history tamper-proof

No trusted authority is needed—mathematics provides the guarantees.

SOURCES

1. Jimmy Song, *Programming Bitcoin* (2019). O'Reilly. https://github.com/jimmysong/programmingbitcoin
2. NIST / security research community, *Dual_EC_DRBG controversy* (2007–2013). Concerns about NIST curve parameter selection; relevant to secp256k1 vs NIST curves. https://projectbullrun.org/dual-ec/
3. OpenSSL Project, *OpenSSL License / SSLeay License* (1998–2009). https://www.openssl.org/source/license-openssl-ssleay.txt
4. Bitcoin developers / Google, *Android SecureRandom ECDSA nonce reuse* (August 2013). Non-deterministic RNG led to private-key recovery from repeated r values. https://bitcoin.org/en/alert/2013-08-11-android
5. NIST, *FIPS PUB 180-2 — Secure Hash Standard (SHA-256)* (2002). SHA-256 published August 2002 (draft 2001). https://csrc.nist.gov/publications/detail/fips/180/2/archive/2002-08-01
6. The Times, *Chancellor on brink of second bailout for banks* (January 3, 2009). Headline embedded in the Bitcoin genesis block. https://www.thetimes.com/article/chancellor-alistair-darling-on-brink-of-second-bailout-for-banks-n9l382mn62h
7. Satoshi Nakamoto, *Bitcoin genesis block (block 0)* (January 3, 2009). Coinbase embeds The Times headline as timestamp proof. https://mempool.space/block/000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
8. Ralph C. Merkle, *Protocols for Public Key Cryptosystems* (1980). Introduced Merkle trees (hash trees). https://www.merkle.com/papers/Protocols.pdf
9. Adam Back, *Hashcash — A Denial of Service Counter-Measure* (August 2002). http://www.hashcash.org/papers/hashcash.pdf

C++ PRIMER

Bitcoin is written in C++, a language that gives the programmer direct control over memory, performance, and system resources—at the cost of considerable complexity. This primer covers the syntax, patterns, and file organization you will encounter in the source code. You do not need to understand every line to follow the annotations, but knowing these basics will make the architecture visible.

FILE AND DIRECTORY STRUCTURE

Bitcoin v0.1.0 follows traditional C++ project conventions. Understanding the file organization helps you navigate the codebase and understand how the pieces fit together.

HEADER FILES (.H) VS SOURCE FILES (.CPP)

C++ splits code into two file types for a crucial reason: **separate compilation**. A large project like Bitcoin would take forever to compile if every change required recompiling everything.

- **Header files (.h)** — Contain *declarations*: class definitions, function signatures, constants, and type definitions. They describe *what* exists without providing implementation details. Headers are "included" by other files that need to use those declarations.
- **Source files (.cpp)** — Contain *implementations*: the actual code for functions declared in headers. Each .cpp file is compiled independently into an "object file," then all object files are linked together into the final executable.

```
// main.h (header - declarations)
class CBlock {
public:
    int nVersion;
    uint256 GetHash();  // declared but not implemented
};

// main.cpp (source - implementations)
#include "main.h"

uint256 CBlock::GetHash() {
    // actual implementation here
    return Hash(BEGIN(nVersion), END(nNonce));
}
```

This separation means changing a function's implementation (in .cpp) only requires recompiling that one file. Changing a header requires recompiling everything that includes it—which is why headers change less frequently.

THE #INCLUDE DIRECTIVE

The `#include` directive literally copies the contents of another file into the current file before compilation:

```
#include "main.h"      // Local file (in same project)
#include <vector>      // System/library file
#include <openssl/sha.h>  // External library
```

Quotes ("") search the local directory first; angle brackets (<>) search system include paths. Bitcoin's `headers.h` is a "precompiled header" that includes all commonly needed files in one place—a pattern that speeds up compilation.

BITCOIN V0.1.0 FILE ORGANIZATION

FILE	PURPOSE
Core Logic	
main.cpp / main.h	Blockchain, consensus, mining, transactions
script.cpp / script.h	Script interpreter for spending conditions
net.cpp / net.h	P2P networking and message handling
market.cpp / market.h	Peer-to-peer marketplace: signed product listings and reviews (unfinished)
db.cpp / db.h	Berkeley DB persistence layer[1]
Cryptography	
key.h	Elliptic curve key management (secp256k1)
sha.cpp / sha.h	SHA-256 hash implementation
bignum.h	Arbitrary-precision arithmetic (wraps OpenSSL)
uint256.h	256-bit unsigned integer for hashes
base58.h	Base58Check encoding for addresses
Infrastructure	
serialize.h	Binary serialization for network/disk
util.cpp / util.h	Utility functions, logging, argument parsing

irc.cpp / irc.h	IRC-based peer discovery (bootstrap)
headers.h	Precompiled header (includes everything)
User Interface	
ui.cpp / ui.h	wxWidgets GUI application code[2]
uibase.cpp / uibase.h	Auto-generated GUI base classes
Build & Config	
makefile	MinGW GCC build instructions (Windows)
makefile.vc	Windows Visual C++ build instructions
ui.rc	Windows resource file (icons, metadata)
rc/	Icon and toolbar bitmap files referenced by ui.rc
uiproject.fbp	wxFormBuilder project (GUI designer)

WHY HEADER-ONLY FILES?

Some Bitcoin files are "header-only" (no corresponding .cpp file): `key.h`, `bignum.h`, `uint256.h`, `base58.h`, `serialize.h`. This pattern is used when:

- The code is mostly **template** code (must be in headers due to C++ rules)
- The code is **small enough** that inline compilation is efficient
- The code **wraps external libraries** (like OpenSSL) with thin adapters

Header-only code is simpler to use (just #include it) but increases compile time since it's recompiled in every file that includes it.

THE MAKEFILE

The `makefile` tells the compiler how to build the project. It specifies which files to compile, what compiler flags to use, and what libraries to link:

```
# Abridged from the v0.1.0 makefile

# Precompile the shared header once
headers.h.gch: headers.h $(HEADERS) net.h irc.h market.h uibase.h ui.h
    g++ -c $(CFLAGS) -o $@ $<

# Compile main.cpp into obj/main.o (object file)
obj/main.o: main.cpp $(HEADERS) net.h market.h sha.h
    g++ -c $(CFLAGS) -o $@ $<

# Link all object files into the final executable
bitcoin.exe: headers.h.gch $(OBJS)
    g++ $(CFLAGS) -mwindows ... -o $@ $(LIBPATHS) $(OBJS) $(LIBS)

# $(LIBS) includes -l db_cxx (Berkeley DB), -l eay32 (OpenSSL), wxWidgets ...
```

Bitcoin v0.1.0 includes two makefiles: `makefile` for MinGW GCC and `makefile.vc` for Microsoft's Visual C++. Both target Windows—the only platform v0.1.0 supported.[3]

RESOURCE AND PROJECT FILES

- **ui.rc** — Windows resource file. Defines the application icon, version info, and other Windows-specific metadata that gets embedded in the .exe file.
- **uiproject.fbp** — wxFormBuilder project file (XML format). Satoshi used this visual designer to create the GUI, then generated `uibase.cpp` and `uibase.h` from it. The .fbp file is the "source" for the GUI layout.

BASIC SYNTAX

Every C++ statement ends with a semicolon. Blocks of code are grouped with curly braces:

```
int x = 5;              // statement ends with ;
if (x > 0) {            // block starts with {
    DoSomething();
    DoSomethingElse();
}                       // block ends with }
```

Comments explain code without affecting execution:

```
// This is a single-line comment

/* This is a
   multi-line comment */
```

VARIABLES AND DATA TYPES

Variables store data. You must declare their type:

```
int count = 10;           // integer (whole number)
double price = 19.99;     // floating-point (decimal)
char letter = 'A';        // single character
bool isValid = true;      // boolean (true/false)
string name = "Satoshi";  // text string
```

SIGNED VS UNSIGNED

Integers can be **signed** (positive or negative) or **unsigned** (non-negative only). Unsigned integers can store larger positive values:

```
int balance = -100;            // signed: -2B to +2B
unsigned int amount = 4000000000;  // unsigned: 0 to 4B

// Bitcoin uses specific-width types:
int64 nValue;        // 64-bit signed (-9e18 to +9e18)
uint64 nAmount;      // 64-bit unsigned (0 to 18e18)
unsigned char byte;  // 8-bit unsigned (0 to 255)
```

Bitcoin amounts are stored as `int64` in satoshis (1 BTC = 100,000,000 satoshis), allowing for 8 decimal places of precision.

TYPE CASTING

Converting between types is called **casting**:

```
int a = 5;
int b = 2;
double result = (double)a / b;  // Cast a to double: 2.5
                                // Without cast: 2 (integer division)

// C++ style casts (more explicit):
int64 n = static_cast<int64>(value);

// Pointer casts (reinterpret raw bytes):
unsigned char* pch = (unsigned char*)&nValue;
```

OPERATORS

```
// Arithmetic
a + b    // addition
a - b    // subtraction
a * b    // multiplication
a / b    // division
a % b    // modulo (remainder)

// Comparison (return true/false)
a == b   // equal
a != b   // not equal
a < b    // less than
a > b    // greater than
a <= b   // less than or equal
a >= b   // greater than or equal

// Logical
a && b   // AND (both true)
a || b   // OR (either true)
!a       // NOT (invert)

// Bitwise (operate on individual bits)
a & b    // AND
a | b    // OR
a ^ b    // XOR
~a       // NOT (complement)
a << n   // left shift (multiply by 2^n)
a >> n   // right shift (divide by 2^n)

// Assignment
a = b    // assign
a += b   // add and assign (a = a + b)
a++      // increment (a = a + 1)
a--      // decrement (a = a - 1)
```

CONTROL FLOW

CONDITIONALS

```
if (condition) {
    // executed if condition is true
} else if (other_condition) {
    // executed if other_condition is true
} else {
    // executed if all conditions are false
}

// Ternary operator (compact if-else):
int max = (a > b) ? a : b;  // if a>b then a, else b
```

LOOPS

```
// For loop: init; condition; increment
for (int i = 0; i < 10; i++) {
    printf("%d\n", i);  // prints 0 through 9
}

// While loop: repeat while condition is true
while (count > 0) {
    count--;
}

// Do-while: execute at least once
do {
    TryConnect();
} while (!connected);

// Loop control:
break;     // exit loop immediately
continue;  // skip to next iteration
```

SWITCH

```
switch (opcode) {
    case OP_ADD:
        result = a + b;
        break;
    case OP_SUB:
        result = a - b;
        break;
    default:
        error = true;
        break;
}
```

FUNCTIONS

Functions encapsulate reusable code:

```
// Declaration: return_type name(parameters)
int Add(int a, int b) {
    return a + b;
}

// Calling the function:
int sum = Add(3, 5);  // sum = 8

// Void functions return nothing:
void PrintMessage(string msg) {
    printf("%s\n", msg.c_str());
}

// Default parameters:
void Connect(string host, int port = 8333) {
    // port defaults to 8333 if not specified
}
```

CLASSES AND STRUCTS

Classes bundle data and functions together:

```
class CTransaction {
public:                    // accessible from outside
    int nVersion;
    vector<CTxIn> vin;     // inputs
    vector<CTxOut> vout;   // outputs
    int nLockTime;

    uint256 GetHash() const {  // member function
        return SerializeHash(*this);  // hash the serialized bytes
    }

protected:                 // only accessible inside class (and subclasses)
    bool AddToMemoryPool();
};

// Creating and using objects:
CTransaction tx;
tx.nVersion = 1;
uint256 hash = tx.GetHash();
```

Structs are like classes but default to public access. Bitcoin v0.1.0 almost always uses `class` with an explicit `public:` section; `struct` appears only a handful of times.

POINTERS AND REFERENCES

Pointers store memory addresses:

```
int value = 42;
int* ptr = &value;   // ptr points to value's address
*ptr = 100;          // dereference: now value = 100

// Null pointer (points to nothing):
int* p = NULL;
if (p != NULL) { ... }

// Arrow operator for pointer to struct/class:
CBlock* pblock = new CBlock();
pblock->nVersion = 1;   // same as (*pblock).nVersion
```

References are aliases for existing variables:

```
int value = 42;
int& ref = value;    // ref IS value (same memory)
ref = 100;           // now value = 100

// Common use: efficient function parameters
void ProcessBlock(CBlock& block) {  // no copy made
    block.nNonce++;
}
```

ARRAYS AND VECTORS

```
// Fixed-size array:
int numbers[5] = {1, 2, 3, 4, 5};
numbers[0] = 10;     // access by index (0-based)

// Vector (dynamic array from STL):
vector<int> v;
v.push_back(1);      // add element
v.push_back(2);
int size = v.size(); // number of elements
int first = v[0];    // access by index

// Iterating:
for (int i = 0; i < v.size(); i++) {
    printf("%d\n", v[i]);
}

// Range-based (C++11 — not in v0.1.0, which predates it
// and uses the foreach macro instead):
for (int n : v) {
    printf("%d\n", n);
}
```

MAPS (DICTIONARIES)

```
// Key-value storage:
map<string, int> balances;
balances["Alice"] = 100;
balances["Bob"] = 50;

// Lookup:
int aliceBalance = balances["Alice"];

// Check if key exists:
if (balances.count("Charlie") > 0) {
    // Charlie exists in map
}

// Iterate over all entries:
for (map<string, int>::iterator it = balances.begin();
     it != balances.end(); ++it) {
    printf("%s: %d\n", it->first.c_str(), it->second);
}
```

THE PREPROCESSOR

Lines starting with `#` are processed before compilation:

```
// Include another file's contents:
#include "main.h"        // local file
#include <vector>        // system file

// Define constants (v0.1.0 prefers typed constants to #define):
static const int64 COIN = 100000000;            // 1 BTC in satoshis
static const unsigned int MAX_SIZE = 0x02000000;  // 32 MB size cap

// Conditional compilation:
#ifdef WIN32
    // Windows-specific code
#else
    // Unix/Linux code
#endif

// Macros (inline code substitution), from v0.1.0's util.h:
#define foreach  BOOST_FOREACH          // shorter name
#define loop     for (;;)              // loop forever
#define BEGIN(a) ((char*)&(a))
#define END(a)   ((char*)&((&(a))[1]))
```

`BEGIN(a)` and `END(a)` give the address of a variable's first byte and of the byte just past its last. That is how `CBlock::GetHash()`, shown earlier, hashes `BEGIN(nVersion)` through `END(nNonce)`: the six header fields sit next to each other in memory, so the range covers exactly the 80-byte block header. The trick only works for fields laid out contiguously—a transaction, whose inputs and outputs live in separately allocated vectors, is hashed by serializing it first (`SerializeHash(*this)`).

COMMON BITCOIN PATTERNS

Patterns you'll see throughout the code[4]:

```
// CRITICAL_BLOCK: Thread-safe access
CRITICAL_BLOCK(cs_main) {
    // Only one thread can execute this at a time
}

// foreach (alias for BOOST_FOREACH): Iterate over collections
foreach(const CTransaction& tx, vtx)     // from CBlock::CheckBlock()
    if (!tx.CheckTransaction())
        return error("CheckBlock() : CheckTransaction failed");

// Serialization: IMPLEMENT_SERIALIZE
IMPLEMENT_SERIALIZE(
    READWRITE(nVersion);
    READWRITE(vin);
    READWRITE(vout);
)

// Error handling:
if (!CheckTransaction())
    return error("Invalid transaction");

// Logging:
printf("ProcessBlock: %s\n", hash.ToString().c_str());
```

MEMORY MANAGEMENT

```
// Allocate on heap (must manually free):
CBlock* pblock = new CBlock();
// ... use pblock ...
delete pblock;  // free memory

// Automatic cleanup with smart pointers:
auto_ptr<CBlock> pblock(new CBlock());
// automatically deleted when out of scope
// (v0.1.0 does this in main.cpp; C++11 later deprecated auto_ptr)

// Stack allocation (automatic cleanup):
CBlock block;  // created on stack
// deleted automatically when function returns
```

SOURCES

1. Oracle Corporation, *Berkeley DB License* (2009). Sleepycat-era open-source linking requirements referenced in readme.txt.
2. wxWidgets Project, *wxWidgets Licence* (2005). https://www.wxwidgets.org/about/licence/
3. Satoshi Nakamoto, *Bitcoin v0.1 released (Cryptography Mailing List / BitcoinTalk)* (January 9, 2009). Announced Windows-only for the first release. https://satoshi.nakamotoinstitute.org/emails/cryptography/15/
4. Boost.org, *Boost Software License 1.0* (2003). https://www.boost.org/LICENSE_1_0.txt

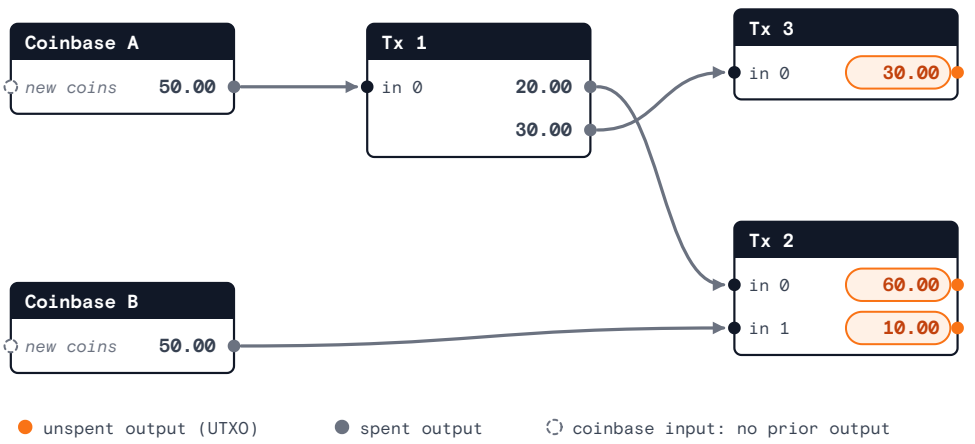
THE BITCOIN PROTOCOL

A protocol is the set of rules every participant enforces. Bitcoin's rules are not written down anywhere except in the code: v0.1.0 *is* the specification. This chapter describes those rules before you read them line by line—how value is represented, how a spend is authorized, how blocks are chained and mined, how new coins enter circulation, and how nodes find each other and agree on one history. Every figure is drawn from the v0.1.0 source and names the function that implements it. Much of this has since changed; "What Came After" traces the changes.

COINS, NOT ACCOUNTS

Nowhere in Bitcoin is there a table of balances. The ledger is made of **transaction outputs**: each output carries an amount and a condition that must be met to spend it. A transaction spends earlier outputs by naming them as its **inputs** and proving it meets their conditions, then creates new outputs of its own. The whitepaper puts it in one line: "We define an electronic coin as a chain of digital signatures."[1]

A transaction graph: amounts in BTC. Coinbase transactions create coins from nothing; every other input consumes exactly one earlier output, whole. The outputs nothing has consumed yet (orange) are the spendable coins, the *unspent transaction outputs*.



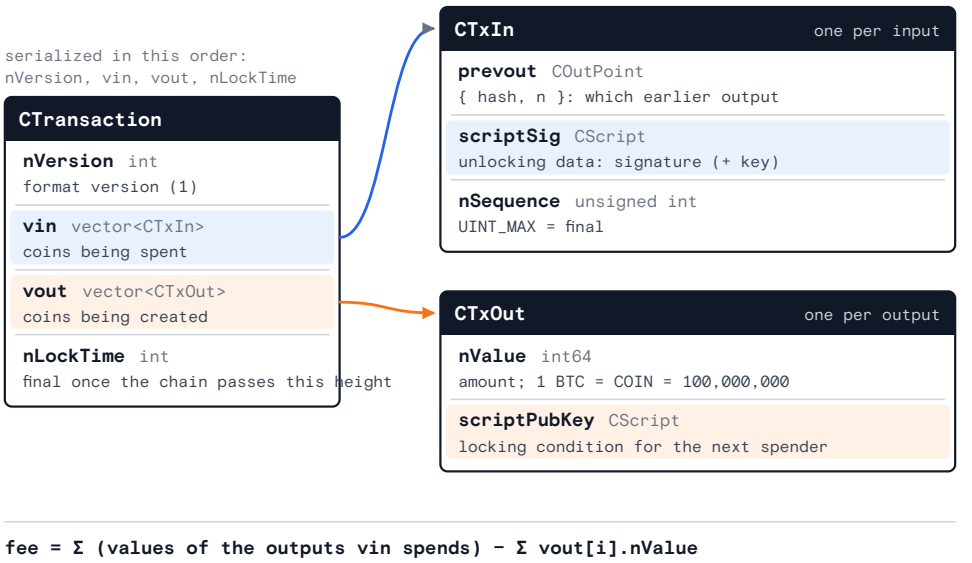
An output is always spent in full. To pay 20 out of a 50-coin output, Tx 1 above creates two outputs: 20 to the recipient and 30 back to the sender as **change**. What a wallet shows as a balance is simply the sum of the unspent outputs it holds keys for (`GetBalance()` walks the wallet and skips anything marked spent).

Each node keeps, for every transaction in the chain, a `CTxIndex` whose `vSpent` vector has one slot per output. A slot is null until a later transaction in the chain spends that output; then it records where that spender sits on disk. This is how double-spending is stopped: `ConnectInputs()` refuses any input whose slot is already filled ("prev tx already used").

ANATOMY OF A TRANSACTION

In code, a transaction is a `CTransaction` with four fields, serialized in the order shown. Inputs and outputs are small classes of their own, all in `main.h`:

The transaction classes: an input names the output it spends by `COutPoint`: the hash of the earlier transaction and an index into its `vout`. Nothing records the fee; it is whatever the inputs bring in that the outputs do not pay out.



fee = Σ (values of the outputs vin spends) − Σ vout[i].nValue
no field records it: the miner claims the difference in the coinbase

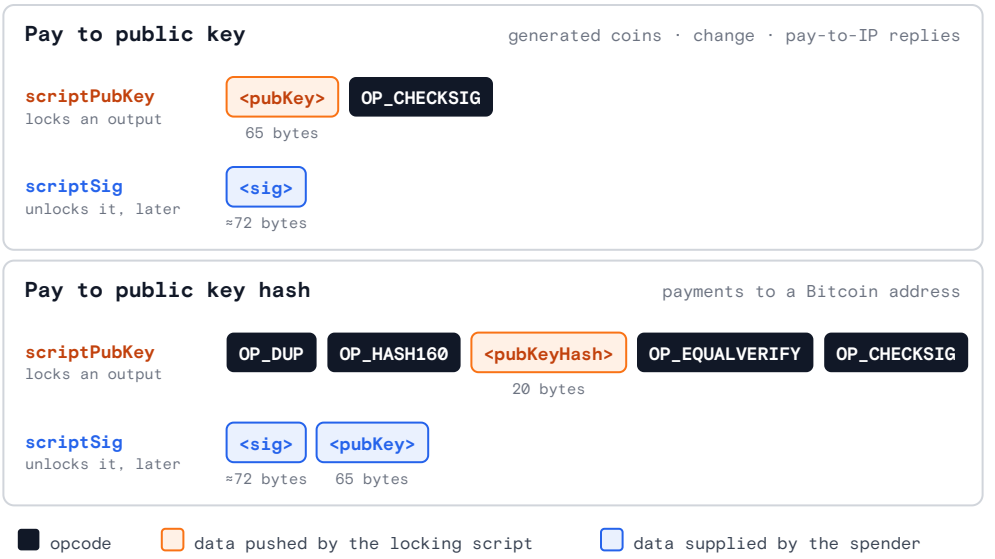
Amounts are 64-bit integers in the smallest unit: `COIN` is 100,000,000 and `CENT` is 1,000,000, so 0.01 BTC is one `CENT`. (The word "satoshi" for the unit came later.) Fees were mostly optional: a transaction under 10,000 bytes is free while the block being built holds fewer than 100 transactions. Otherwise `GetMinFee()` asks for 0.01 BTC per started kilobyte. Miners collect fees by paying themselves the subsidy plus the fees in the block's first transaction.

The two remaining fields sketch a feature that was later abandoned. A transaction whose `nLockTime` is a block height the chain has not yet passed is not *final*. While it waits in the memory pool, a version with the same inputs and higher `nSequence` numbers may replace it (`IsNewerThan()`). An input with `nSequence == UINT_MAX` is final. The wallet shows such a transaction as "Open for *n* blocks".

LOCKING AND UNLOCKING: SCRIPT

An output's condition is not a public-key field but a tiny program, the `scriptPubKey`. The input that later spends it supplies a second program, the `scriptSig`, which pushes the data the first one needs. The language, Script, is stack-based like Forth. Its only control flow is `OP_IF`-style branching, with no loops or backward jumps, so every script finishes. Satoshi later said this design allowed a great variety of transaction types beyond simple payments.[2] In practice v0.1.0 writes and recognizes only two forms. `Solver()` in `script.cpp` matches outputs against these templates when the wallet signs or looks for its own coins:

The two standard templates: orange data is fixed when the output is created; blue data is supplied by whoever spends it. `OP_PUBKEY` and `OP_PUBKEYHASH` in `Solver()`'s template list are placeholders that match these slots.



Pay to public key locks coins directly to a 65-byte public key. v0.1.0 uses it for coins it generates (`BitcoinMiner`), for change (`CreateTransaction` sends change to a key that funded the payment), and for pay-to-IP transfers. In those, the payee's node answers a `checkorder` message with a fresh key.

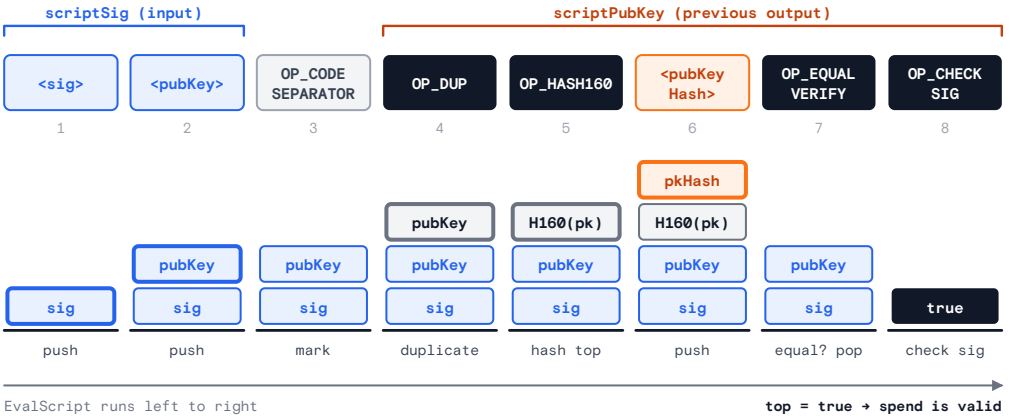
Pay to public key hash is what a Bitcoin address encodes. The address is a version byte (0) plus the 20-byte `Hash160` of a public key, which is RIPEMD-160 of SHA-256, all in Base58Check (`Hash160ToAddress()` in `base58.h`). When you paste an address into the Send dialog, `ui.cpp` builds this script around the decoded hash before calling `SendMoney()`. The spender must then reveal the full public key along with the signature.

RUNNING A SCRIPT

To check an input, `VerifySignature()` does not run two programs. It concatenates them into one: the input's `scriptSig`, then an `OP_CODESEPARATOR`, then the previous output's `scriptPubKey`. It hands the result to `EvalScript()`. The spend

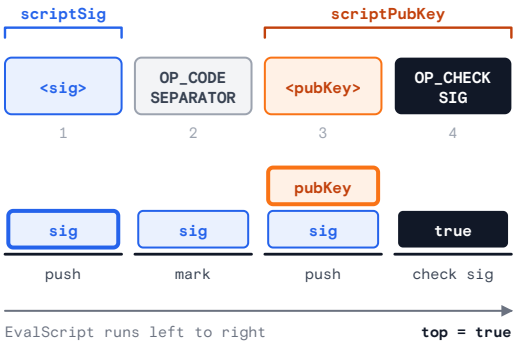
is valid if the program finishes with a true value on top of the stack: `return (stack.empty() ? false : CastToBool(stack.back()));`

Pay to public key hash, step by step: each column is the stack after one token executes, bottom to top; a heavy border marks the item that step produced. `OP_EQUALVERIFY` aborts the whole script unless the two hashes match; `OP_CHECKSIG` pops the key and signature and pushes the verdict.



The separator is there for the signature. When `OP_CHECKSIG` runs, it hashes the transaction together with the script from the last `OP_CODESEPARATOR` onward, so the signature covers the locking script but not the unlocking data (a signature cannot sign itself). The simpler template takes four steps:

Pay to public key: the key comes from the output, the signature from the input.



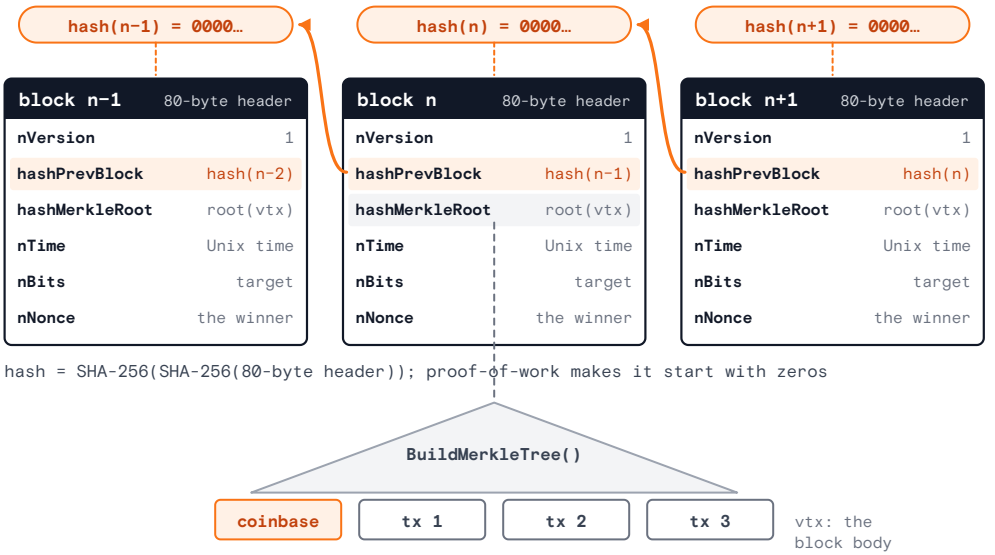
Running the spender's code and the owner's code as one program was a mistake. In v0.1.0, `OP_RETURN` simply jumps to the end of the script (`pc = pend`). A `scriptSig` of `OP_TRUE OP_RETURN` therefore ends execution with true on top before the

owner's conditions ever run, and it could spend anyone's coins. The flaw went unnoticed until July 2010; v0.3.5 fixed it by running the two scripts separately and disabled several risky opcodes at the same time.[34]

BLOCKS AND THE CHAIN

Transactions are batched into blocks. A `CBlock` is a list of transactions, `vtx`, plus a header of six fields that serialize to exactly 80 bytes. A block's identity is the double SHA-256 of that header, and each header carries the hash of the one before it:

> **Linked headers:** `hashPrevBlock` commits to the entire previous header, and `hashMerkleRoot` commits to every transaction in this block (see "Merkle Trees" in Cryptography Basics). Altering any old transaction changes its block's hash, which breaks the link from every block after it.



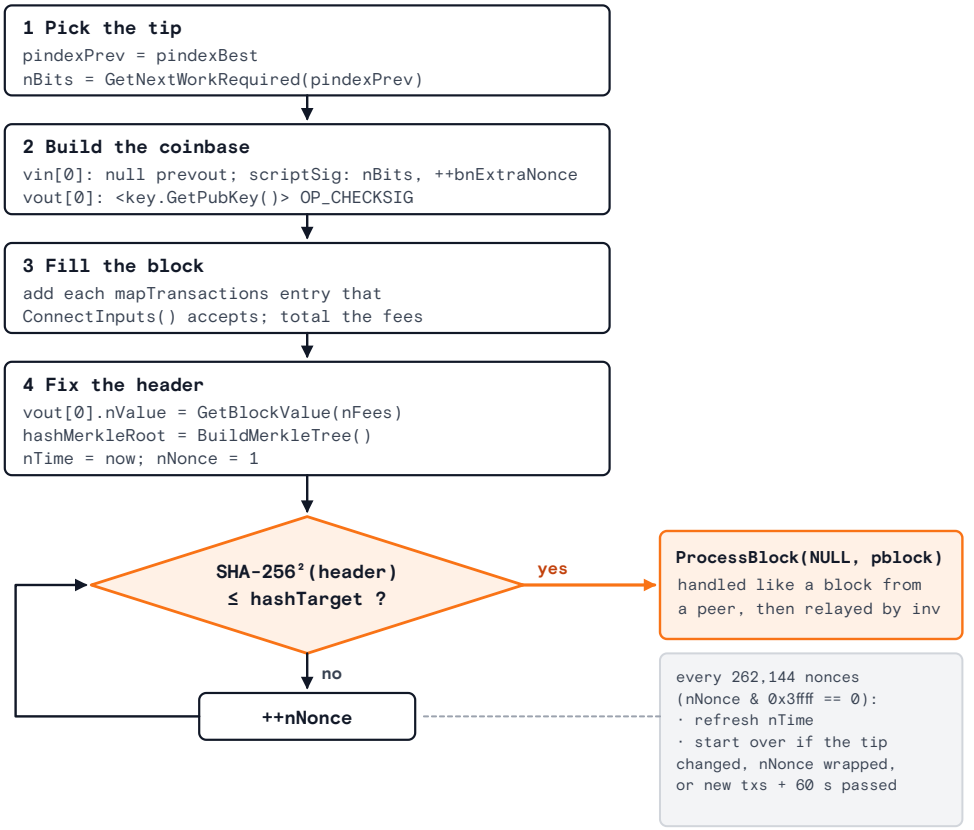
The first transaction in every block is the **coinbase**, the only transaction with no real inputs. Its single input has a null `prevout`, and its output pays the miner. The first block of all, the genesis block, is written directly into `main.cpp`. Its coinbase input carries a newspaper headline as proof it was not mined before January 3, 2009: *The Times 03/Jan/2009 Chancellor on brink of second bailout for banks.*[5]

A node checks a new block in two passes. `CheckBlock()` tests what can be checked in isolation: size limits, a timestamp no more than two hours in the future, exactly one coinbase and it comes first, a header hash that meets its own `nBits` target, and a merkle root that matches the transactions. `AcceptBlock()` then checks the block against its parent: the timestamp must be later than the median of the previous eleven blocks, and `nBits` must be exactly what `GetNextWorkRequired()` demands.

MINING

Anyone may propose the next block, but a valid one is expensive to produce: its header hash, read as a 256-bit number, must not exceed the **target** encoded in `nBits`. SHA-256 cannot be steered, so the only way to find such a header is to try one variation after another. That is the proof of work. In v0.1.0 the search runs in `BitcoinMiner()`, a background thread that "Generate Coins" switches on:

The mining loop: steps 1–4 build a candidate block; the diamond is the proof-of-work test, hashed with the prebuilt buffers `BlockSHA256()` uses for speed. A found block goes through `ProcessBlock()` exactly as if a peer had sent it, and `AcceptBlock()` relays it if it extends the best chain.

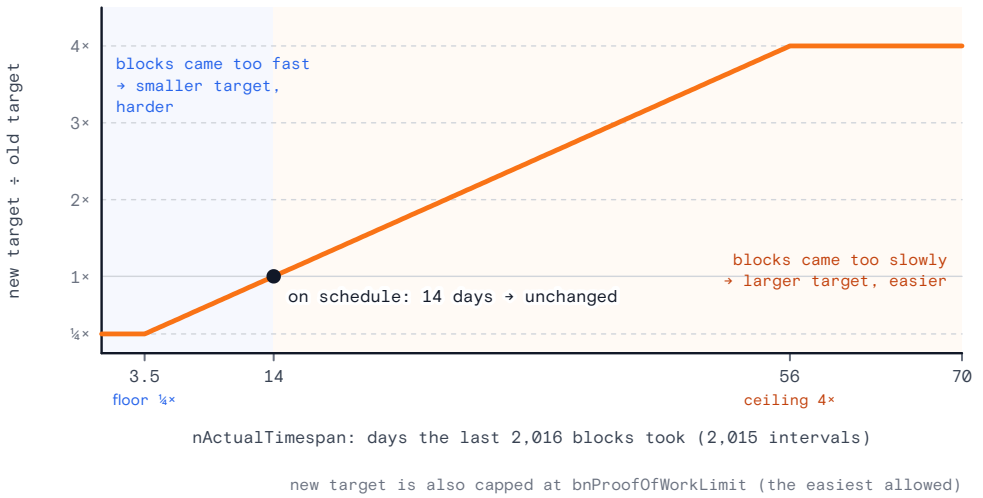


The 32-bit `nNonce` runs out quickly, so the miner has a second knob: the `bnExtraNonce` pushed into the coinbase's `scriptSig`. Each pass of the outer loop increments it, which changes the coinbase, so the merkle root changes, and the miner gets a fresh nonce space. The coinbase's output is worth `GetBlockValue(nFees)`, the subsidy plus all fees, and pays a newly generated key. Before any of that, the loop waits until the node has at least one peer: mining alone would only build a private chain.

DIFFICULTY

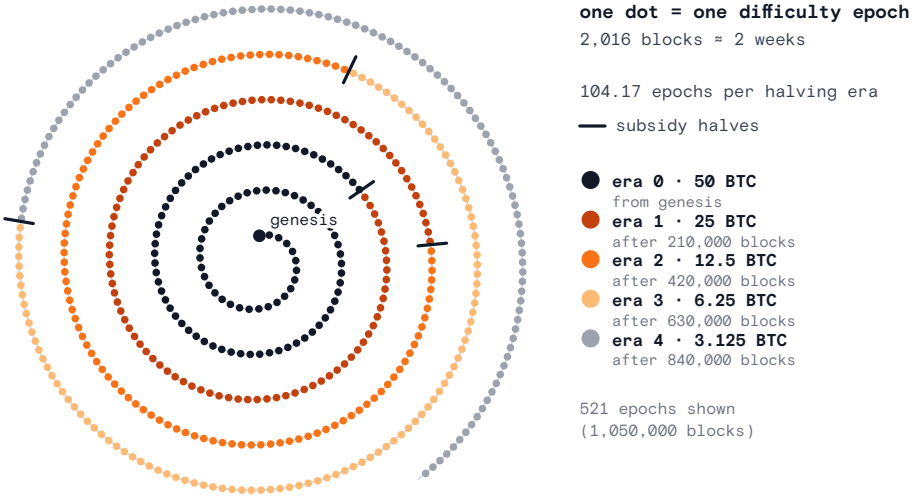
Blocks are meant to arrive every ten minutes (`nTargetSpacing`). Satoshi called this a balance between fast confirmation and network latency: "The algorithm aims for an average of 6 blocks per hour."[6] Total hash power changes, though, so the target must move with it. Every 2,016 blocks, two weeks' worth at the intended pace, `GetNextWorkRequired()` measures how long the last stretch actually took and scales the target by that ratio:

> **The retarget rule:** new target = old target × `nActualTimespan` ÷ two weeks. The measured span is clamped to between 3.5 and 56 days first, so one adjustment can change difficulty by at most a factor of four in either direction.



A smaller target means fewer acceptable hashes, so it is harder to meet. The code walks back `nInterval - 1` blocks to find the start of the period. It therefore times 2,015 block intervals and compares them against a two-week budget sized for 2,016, a small off-by-one that every later version keeps for compatibility. The target is stored in the header as a compact `nBits` value: one byte of exponent and three of mantissa. The genesis block's `1d00ffff` means $0x00ffff \times 256^{(0x1d - 3)}$. It is also `bnProofOfWorkLimit`, the easiest target the code ever allows.

Epochs and eras: each dot is one 2,016-block difficulty period, laid along a spiral from genesis in the spirit of the Anatomy of Bitcoin explorer's helix. Shading changes at every halving, roughly every 104 periods.



MONEY SUPPLY AND HALVINGS

New coins enter circulation only through coinbase transactions, which the whitepaper likens to mining gold: "The steady addition of a constant of amount of new coins is analogous to gold miners expending resources to add gold to circulation."[1] The whole monetary policy is three lines of `CBlock::GetBlockValue()`:

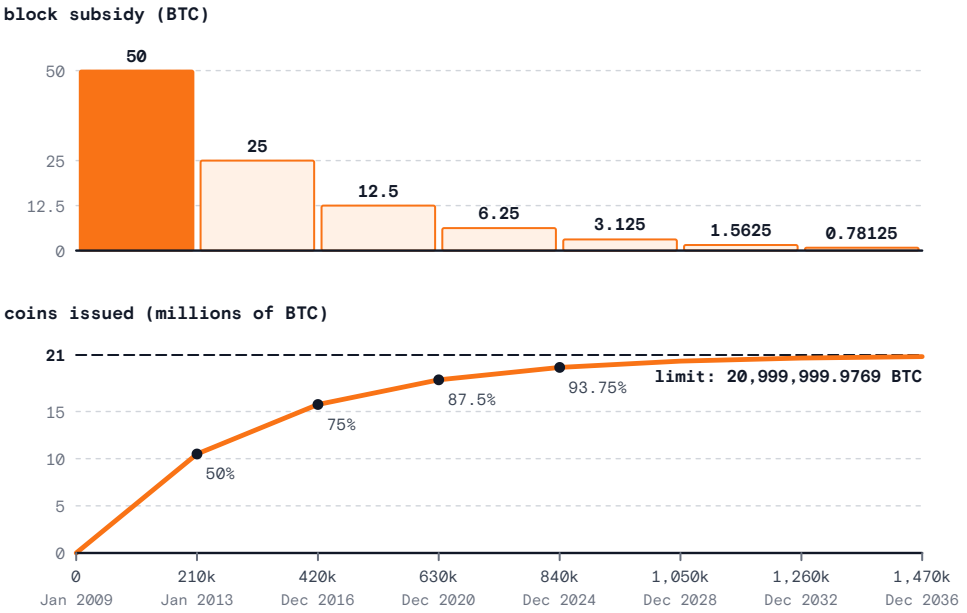
```
int64 nSubsidy = 50 * COIN;

// Subsidy is cut in half every 4 years
nSubsidy >>= (nBestHeight / 210000);

return nSubsidy + nFees;
```

Shifting right by one bit halves a number. After every 210,000 blocks, about four years at ten minutes each, the shift count goes up by one. The total is a geometric series: 210,000 × 50 × (1 + ½ + ¼ + …) approaches 21 million. Because the shift truncates to whole units, the real total falls just short, and the reward reaches zero after 33 halvings. The table lists each shift count with the tip heights that produce it:

Subsidy and supply: computed by applying the shift above to every block. Half of all coins are issued in the first era, three quarters by the end of the second.



SHIFT	TIP HEIGHT	SUBSIDY (BTC)	ISSUED BY ERA END
0	0–209,999	`50.00000000`	10,500,000 (50%)
1	210,000–419,999	`25.00000000`	15,750,000 (75%)
2	420,000–629,999	`12.50000000`	18,375,000 (87.5%)
3	630,000–839,999	`6.25000000`	19,687,500 (93.75%)
4	840,000–1,049,999	`3.12500000`	20,343,750 (96.875%)
…	…	…	…
32	6,720,000–6,929,999	`0.00000001`	20,999,999.9769 (100%)
33	6,930,000+	`0.00000000`	20,999,999.9769 (100%)

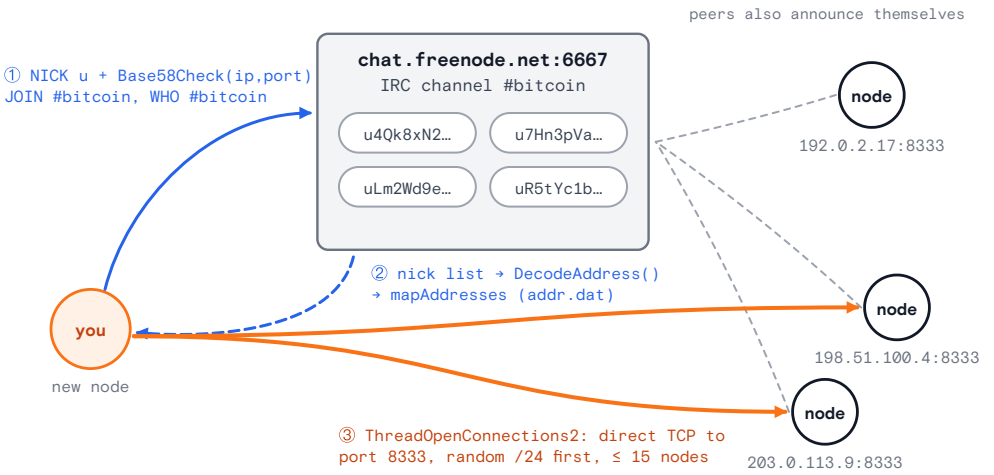
Two details in that line deserve a second look. First, it reads `nBestHeight`, the height of the current tip, not the height of the block being valued. A miner building block 210,001 on a tip at 210,000 is the first to get the reduced reward. By v0.3.10 the function takes the block's own height instead. Second, shifting a 64-bit integer

by 64 or more places is undefined in C++. On common hardware the count wraps, and the subsidy would jump back to 50 coins at block 13,440,000, centuries from now. BIP 42 closed that hole in 2014.[7]

THE NETWORK

There is no server. Each node listens on TCP port 8333 (`DEFAULT_PORT`) and connects directly to others. The hard part is the first contact: how does a fresh install learn a single address? v0.1.0 borrows a public meeting place, an IRC channel:

> **Bootstrapping through IRC:** `ThreadIRCSeed()` joins `#bitcoin` under a nickname that *is* its address: "u" followed by the Base58Check encoding of its IP and port. Every other nickname in the channel decodes to a peer. The IPs shown are documentation placeholders.

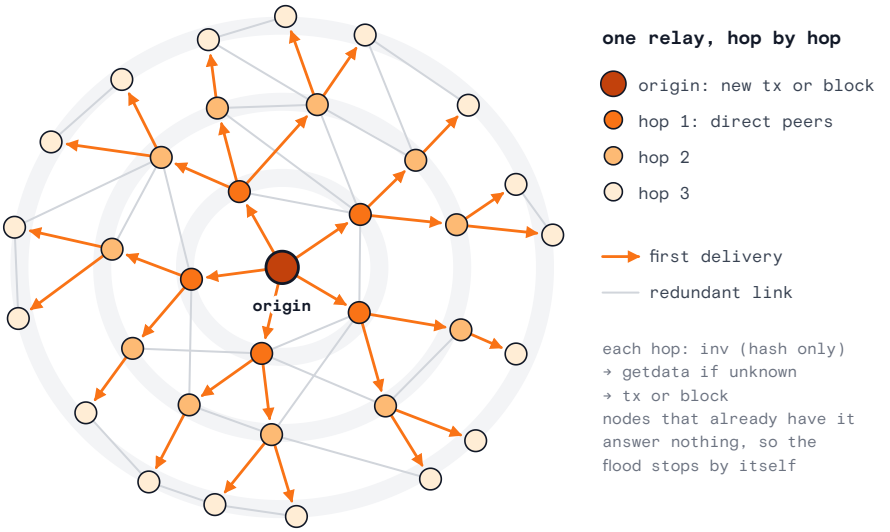


IRC was scaffolding. Once connected, a node asks its peers for more addresses (`getaddr`), advertises its own (`addr`), and stores everything in `addr.dat` for next time. As Satoshi later put it: "Bitcoin can get the list of IPs from any bitcoin node. In that sense, every node serves as a directory server."[8]

Three threads run the network. `ThreadSocketHandler` accepts connections and moves bytes, `ThreadOpenConnections2` dials out, and `ThreadMessageHandler` processes what arrives. The dialer stops once the node has 15 connections, inbound

and outbound together. It first picks a random /24 network (a "class C") and only then an address inside it, so someone holding a whole block of addresses gets no more attention than a lone node elsewhere. Information then spreads by **gossip**:

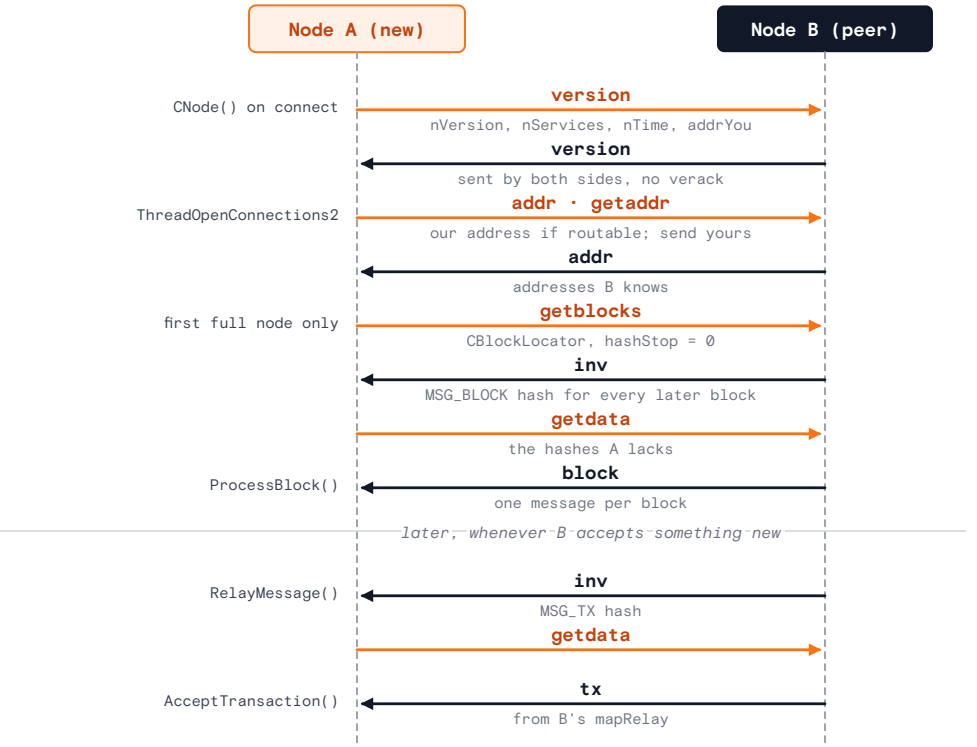
> **Flooding by inventory:** a node announces only a hash (`inv`). A neighbor that lacks the item asks for it (`getdata`), validates it, and announces it in turn. Duplicate announcements cost a few bytes and are ignored.



MESSAGES

Each message on the wire starts with the four magic bytes `f9 be b4 d9` (`pchMessageStart`), a 12-byte command name, and a payload length. There is no checksum yet. `ProcessMessage()` in `main.cpp` handles twelve commands, and a node must send `version` before anything else. A new connection unfolds like this:

A first conversation: the version exchange, address swap, and initial block download, followed later by ordinary transaction relay. The left margin names the code that sends or handles each step.

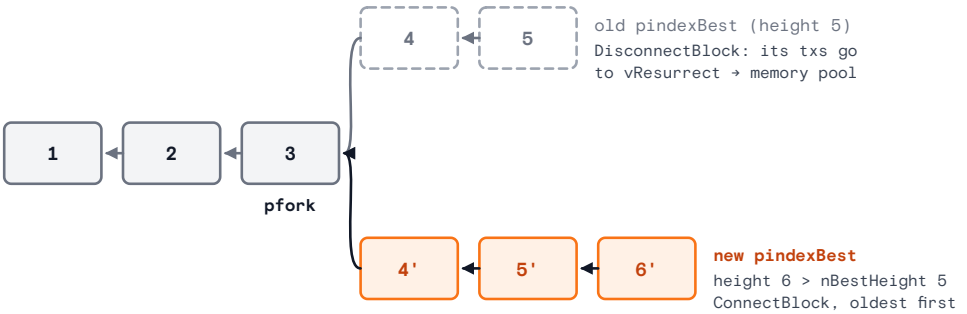


The rest of the command set reflects plans that never took hold. `checkorder`, `submitorder`, and `reply` carry the pay-to-IP protocol. You connect to the payee's address, it hands you a public key, and you pay that key. `review` belongs to an unfinished marketplace for products and user reviews (`market.cpp`). Pay-to-IP was later removed, and the marketplace was never finished.

LONGEST CHAIN WINS

Two miners can find blocks at nearly the same moment, and the network briefly splits. The whitepaper's rule resolves it: "Nodes always consider the longest chain to be the correct one and will keep working on extending it."[1] In v0.1.0 "longest" is literal. `AddToBlockIndex()` makes a block the new tip if `pindexNew→nHeight > nBestHeight`. A tie keeps whichever branch arrived first.

A reorganization: blocks 4 and 5 were the best chain until a competing branch reached height 6. Transactions from the abandoned blocks are not lost: `Reorganize()` collects them in `vResurrect` and hands them back to the memory pool to be mined again.



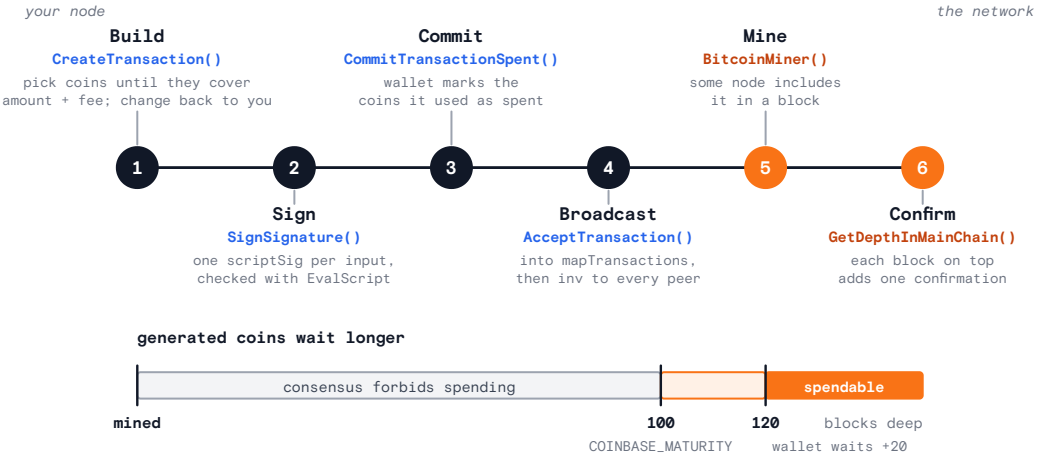

```
Reorganize(txdb, pindexNew): walk both tips back to the fork, disconnect the
old branch newest-first, connect the new branch oldest-first, then set the best chain.
```

Counting blocks rather than work has a flaw: a long branch of easy blocks could beat a shorter branch that took more total work. Later versions compare accumulated work (`bnChainWork`) instead. A block whose parent is unknown is kept in `mapOrphanBlocks` , and the node asks the sender for the missing ancestors with `getblocks` .

LIFE OF A PAYMENT

Here is a payment from one end to the other, with the function that does each step. Steps 1–4 happen on the sender's machine inside `SendMoney()` . The rest happens on the network.

From click to confirmation: the wallet shows "*n*/unconfirmed" until a transaction is six blocks deep. Coins a miner generates wait longer: consensus refuses to spend them until they have matured for 100 blocks (`COINBASE_MATURITY`), and the wallet waits 20 more before counting them.



A confirmation is simply depth: how many blocks sit on top of the one holding your transaction. Each block added makes reversing the payment more expensive, because an attacker would have to outpace the honest network from that point forward. The whitepaper works out how quickly that chance falls with depth.[1]

WHERE IT LIVES IN THE CODE

Every mechanism in this chapter is a function you can read. The Reading Paths that follow string them together into guided tours. This table is the index:

MECHANISM	DEFINITION	WHERE
Spending rules and double-spend check	`CTransaction::ConnectInputs`	main.cpp:772 p. 215
Script interpreter	`EvalScript`	script.cpp:44 p. 383
Standard script templates	`Solver`	script.cpp:913 p. 399
Signing an input	`SignSignature`	script.cpp:1090 p. 402
Checking an input	`VerifySignature`	script.cpp:1115 p. 402

Block subsidy and halving	`CBlock::GetBlockValue`	main.cpp:675 p. 213
Difficulty retarget	`GetNextWorkRequired`	main.cpp:685 p. 213
Block sanity checks	`CBlock::CheckBlock`	main.cpp:1154 p. 222
Block contextual checks	`CBlock::AcceptBlock`	main.cpp:1192 p. 222
Choosing the best chain	`CBlock::AddToBlockIndex`	main.cpp:1072 p. 220
Switching branches	`Reorganize`	main.cpp:974 p. 219
Mining loop	`BitcoinMiner`	main.cpp:2183 p. 240
IRC peer discovery	`ThreadIRCSeed`	irc.cpp:144 p. 165
Opening outbound connections	`ThreadOpenConnections2`	net.cpp:673 p. 336
Message dispatch	`ProcessMessage`	main.cpp:1690 p. 231
Building a payment	`CreateTransaction`	main.cpp:2514 p. 247
Sending a payment	`SendMoney`	main.cpp:2625 p. 249

SOURCES

1. Satoshi Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (October 2008). https://bitcoin.org/bitcoin.pdf
2. Satoshi Nakamoto, *BitcoinTalk — Re: Transactions and Scripts* (June 2010). On supporting a tremendous variety of transaction types. https://bitcointalk.org/index.php?topic=195.0
3. Bitcoin Wiki, *Common Vulnerabilities and Exposures — CVE-2010-5141 (OP_RETURN) and CVE-2010-5137* (July 28, 2010). Announced 2010-07-28; fixed in Bitcoin 0.3.5. https://en.bitcoin.it/wiki/Common_Vulnerabilities_and_Exposures
4. Satoshi Nakamoto / Bitcoin developers, *OP_RETURN and dangerous opcodes disabled* (July 2010). Response to early Script vulnerabilities. https://en.bitcoin.it/wiki/Script#Disabled_opcodes
5. Satoshi Nakamoto, *Bitcoin genesis block (block 0)* (January 3, 2009). Coinbase embeds The Times headline as timestamp proof. https://mempool.space/block/000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
6. Satoshi Nakamoto, *BitcoinTalk — block interval and difficulty retarget rationale* (2010). Thread "Proof-of-work difficulty increasing" (Feb–Jul 2010): 10-minute rationale (msg376), retarget formula (msg540), hashmeter (msg1648), ExtraNonce (msg1323), 120-block maturity (msg3537). https://bitcointalk.org/index.php?topic=43.0
7. Pieter Wuille, *BIP 42 — A finite monetary supply for Bitcoin* (2014). Right-shifting nSubsidy past bit width would reinflate; soft-fork capped halvings. https://github.com/bitcoin/bips/blob/master/bip-0042.mediawiki

8. Satoshi Nakamoto, *BitcoinTalk — IRC channel for Bitcoin* (March 16 and June 14, 2010). IRC seeding described as a temporary solution. https://bitcointalk.org/index.php?topic=84.0

READING PATHS

The chapters that follow present the source file by file, in alphabetical order, which is how the code is organized but not how it runs. These tours follow the execution instead: each step names a function, where it lives, and the page where it begins. Read a path straight through for the big picture, or keep a finger here and jump.

THE LIFE OF A TRANSACTION

Follow one payment from the moment its sender clicks *Send* to the moment another node accepts it and passes it on.

1. **`CSendDialog::OnButtonSend`** `ui.cpp:1337` p. 496
   The Send button. Parses the amount and the Bitcoin address, then builds a standard pay-to-public-key-hash `scriptPubKey` for the recipient and calls `SendMoney`.
2. **`SendMoney`** `main.cpp:2625` p. 249
   The orchestrator: create and sign the transaction, mark the coins it spends, accept it locally, and broadcast it.
3. **`CreateTransaction`** `main.cpp:2514` p. 247
   Selects coins from the wallet, adds a change output, signs every input, and loops until the fee is sufficient.
4. **`SignSignature`** `script.cpp:1090` p. 402
   Hashes the transaction, solves the previous output's `scriptPubKey` to produce the `scriptSig`, then test-runs both with `EvalScript`.
5. **`CommitTransactionSpent`** `main.cpp:2595` p. 248
   Adds the new transaction to the wallet and marks every coin it consumed as spent on disk.
6. **`CWalletTx::RelayWalletTransaction`** `main.cpp:599` p. 212
   Relays the transaction, preceded by any unconfirmed transactions it depends on, to every connected peer.
7. **`main.cpp:1868`** `main.cpp:1868` p. 234
   On the receiving node: the `tx` message handler accepts the transaction, adds it to the wallet if it pays us, relays it onward, and retries any orphans that were waiting for it.
8. **`CTransaction::AcceptTransaction`** `main.cpp:406` p. 209
   Mempool admission: reject loose coinbases and duplicates, check for conflicts, connect the inputs, then add to the memory pool.
9. **`CTransaction::ConnectInputs`** `main.cpp:772` p. 215
   Looks up each previous output, enforces coinbase maturity, verifies the signature, and refuses outputs that are already spent.

10. **VerifySignature** script.cpp:1115 p. 402
    Joins the input's `scriptSig` and the previous output's `scriptPubKey` into one script and runs it.
11. **EvalScript** script.cpp:44 p. 383
    The script interpreter itself: a stack machine whose `OP_CHECKSIG` finally checks the ECDSA signature.

MINING A BLOCK

Follow a block from an empty template on the miner's machine to the tip of every node's best chain.

1. **BitcoinMiner** main.cpp:2183 p. 240
   The mining loop. Builds a coinbase paying a fresh key, gathers transactions from the memory pool, and fills in the block header.
2. **GetNextWorkRequired** main.cpp:685 p. 213
   Computes the difficulty target for the next block, retargeting every 2016 blocks.
3. **CBlock::GetBlockValue** main.cpp:675 p. 213
   The block reward: the subsidy, halving every 210,000 blocks, plus the fees of the included transactions.
4. **BlockSHA256** main.cpp:2156 p. 239
   The hot loop's hash. The miner increments the nonce and double-hashes the prebuilt header until the result falls below the target.
5. **ProcessBlock** main.cpp:1236 p. 223
   A found block is handled exactly as if it had arrived from a peer. Checks it, accepts it, then adopts any orphans that were waiting for it.
6. **CBlock::CheckBlock** main.cpp:1154 p. 222
   Context-free checks: size, timestamp, a single coinbase first, valid transactions, proof of work, and merkle root.
7. **CBlock::AcceptBlock** main.cpp:1192 p. 222
   Contextual checks against the previous block, then writes the block to disk, indexes it, and relays it if it is now the best chain.
8. **CBlock::AddToBlockIndex** main.cpp:1072 p. 220
   Extends the best chain directly, or triggers a reorganization when a side branch has grown longer.
9. **CBlock::ConnectBlock** main.cpp:937 p. 218
   Connects every transaction's inputs, marking their previous outputs spent in the transaction index.
10. **Reorganize** main.cpp:974 p. 219
    Disconnects blocks back to the fork point and connects the longer branch, returning orphaned transactions to the memory pool.

A NODE JOINS THE NETWORK

Follow a freshly started node from loading its files to its first conversation with a peer.

1. **`CMyApp::OnInit2`** `ui.cpp:2885` p. 515
   Startup. Loads the address book, the block index, and the wallet, then starts networking and, if enabled, the miner.
2. **`LoadBlockIndex`** `main.cpp:1420` p. 226
   Reads the block index from disk; on first run, builds the genesis block with its *Times* headline and writes it out.
3. **`StartNode`** `net.cpp:881` p. 340
   Binds the listening socket on port 8333, discovers the external IP, and launches the IRC seeding, socket, connection, and message threads.
4. **`ThreadIRCSeed`** `irc.cpp:144` p. 165
   Joins `#bitcoin` on `chat.freenode.net`; nicknames beginning with `u` encode peers' IP addresses, which are decoded and stored.
5. **`AddAddress`** `net.cpp:136` p. 327
   Records a newly learned peer address in memory and in the address database.
6. **`ThreadOpenConnections2`** `net.cpp:673` p. 336
   Picks addresses to dial and opens outbound connections up to the connection limit.
7. **`ConnectNode`** `net.cpp:299` p. 330
   Opens a TCP socket to a peer and wraps it in a new `CNode`, whose constructor immediately queues a `version` message.
8. **`ThreadSocketHandler2`** `net.cpp:395` p. 331
   The `select()` loop that accepts inbound connections and moves bytes between sockets and each node's buffers.
9. **`ThreadMessageHandler2`** `net.cpp:816` p. 338
   Polls every node in turn, processing received messages and sending queued ones.
10. **`main.cpp:1705`** `main.cpp:1705` p. 231
    The `version` handler: negotiates the protocol version and, from a full node, requests blocks with `getblocks`. Synchronization begins.

THE SOURCE CODE

Bitcoin v0.1.0 — annotated and explained

rc/ (Resources)

11 files ICONS & BITMAPS

THE ORIGINAL BITCOIN LOGO

Before there was the orange "₿" symbol recognized worldwide, there was this—Satoshi's original Bitcoin logo, buried in a Windows resource directory. These icons and bitmaps shipped with v0.1.0, appearing in the Windows GUI that most early users experienced. The design is notably different from the stylized "B" that would later become synonymous with Bitcoin.

The main `bitcoin.ico` is a Windows multi-resolution icon containing 16×16, 32×32, and 48×48 pixel versions—standard practice for Windows applications of the era. The icon shows a gold coin with "BC" lettering, reflecting the "digital gold" concept Satoshi frequently invoked. This logo appears to be Satoshi's own creation.

The toolbar bitmap files (`.bmp`) were used in the wxWidgets GUI: address book icons for managing contacts, and send arrows for the payment interface. These may have been sourced from standard icon sets common in Windows development at the time. The "mask" files define transparency—a technique from early Windows GUI development before PNG transparency became standard.

In November 2010, a forum user named "bitboy" shared the now-iconic orange ₿ logo: the tilted "B" with two vertical strokes, in white on a flat orange circle.[1] It built on Satoshi's own February 2010 refresh, which had traded the "BC" initials for the stroked "B" while keeping the gold coin. Satoshi had welcomed such contributions: "I'm happy if someone with artistic skill wants to contribute alternatives."[2] The community adopted bitboy's design, but these original icons remain as artifacts of Bitcoin's earliest visual identity— assembled by its pseudonymous creator before the world knew what Bitcoin would become.



bitcoin.ico
Main application icon (multi-resolution: 16×16, 32×32, 48×48)



check.ico
Checkmark icon for confirmations



addressbook16.bmp
Address book toolbar icon (16×16)



addressbook16mask.bmp
Address book icon transparency mask



addressbook20.bmp
Address book toolbar icon (20×20)



addressbook20mask.bmp
Address book icon transparency mask



send16.bmp
Send payment toolbar icon (16×16)



send16mask.bmp
Send icon transparency mask



send16masknoshadow.bmp
Send icon mask without shadow



send20.bmp
Send payment toolbar icon (20×20)



send20mask.bmp
Send icon transparency mask

SUMMARY

These resource files represent Satoshi's complete vision: not just the cryptographic protocol, but a usable application with icons, buttons, and visual identity. Whether custom-made or sourced from icon libraries, they show Satoshi was building software for real people, not just publishing academic theory. The gold "BC" coin—Bitcoin's first logo—remains an artifact of the project's bootstrapped origins.

SOURCES

1. bitboy (BitcoinTalk user), *Orange ₿ logo first shared on BitcoinTalk* (November 1, 2010). Community design that became Bitcoin's standard mark; built on Satoshi's February 2010 gold-coin refresh. https://www.coindesk.com/markets/2019/05/18/about-that-orange-b-the-history-of-bitcoins-logos
2. Satoshi Nakamoto, *BitcoinTalk — Re: New icon/logo* (November 13, 2010). In the February 2010 icon thread, Satoshi invited artists to contribute alternative logos. https://bitcointalk.org/index.php?topic=64.msg21766#msg21766

base58.h

202 lines CPP

HUMAN-READABLE MONEY

Every legacy Bitcoin address—those strings starting with "1" (and, from 2012, "3")—uses the Base58Check encoding defined in this file. Base58 is a binary-to-text encoding designed specifically for human readability: it uses all alphanumeric characters except those easily confused with each other.

The excluded characters are: **0** (zero, confused with O), **O** (confused with 0), **I** (confused with l or 1), and **l** (lowercase L, confused with I or 1). This careful character selection means you can read a Bitcoin address aloud over the phone with minimal ambiguity.

Base58Check adds a crucial feature: a 4-byte checksum appended to the data. If you mistype a single character, the checksum almost certainly won't match, and the address will be rejected as invalid. The probability of a typo accidentally producing a valid address is approximately 1 in 4 billion. This protects users from losing funds to typos.

The encoding process: (1) prepend version byte (0×00 for mainnet addresses), (2) compute SHA256(SHA256(data)) and take first 4 bytes as checksum, (3) append checksum to data, (4) convert to Base58. The version byte is why mainnet addresses start with "1"—0×00 in Base58 is represented as leading "1"s.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.


//
```

Satoshi's original comment explaining Base58's design rationale. This is one of the best-documented design decisions in v0.1. The four reasons: (1) avoid confusable characters 0/O and I/l, (2) alphanumeric-only is friendlier for account numbers, (3) email clients won't line-break pure alphanumeric strings, (4) double-click selects the entire address. These practical considerations show Satoshi thinking about real-world usability, not just cryptographic elegance.

```
// Why base-58 instead of standard base-64 encoding?
// - Don't want 0OIl characters that look the same in some fonts and
//      could be used to create visually identical looking account numbers.
// - A string with non-alphanumeric characters is not as easily accepted as an account number.
// - E-mail usually won't line-break if there's no punctuation to break at.
// - Doubleclicking selects the whole number as one word if it's all alphanumeric.
//


```

The Base58 alphabet: `123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz`. Note the missing characters: no `0` (zero), `O` (oh), `I` (eye), or `l` (ell). This 58-character set maps directly to values 0-57. The character `1` represents value 0, `2` represents 1, etc. Every leading zero byte in the decoded payload becomes one `1`—the genesis address `1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa` starts with `1` because the 0×00 version byte encodes to it.

```
static const char* pszBase58 = "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz";


```

`EncodeBase58()`—converts arbitrary binary data to Base58 string. The algorithm is essentially base conversion: treat input bytes as a big-endian number, repeatedly divide by 58, and map remainders to characters. The use of `CBigNum` handles the arbitrary-precision arithmetic needed for 20+ byte inputs. The string is built in reverse (least significant digit first), then reversed at the end.

```
inline string EncodeBase58(const unsigned char* pbegin, const unsigned char* pend)
{
    CAutoBN_CTX pctx;
    CBigNum bn58 = 58;
    CBigNum bn0 = 0;

// Convert big endian data to little endian
// Extra zero at the end make sure bignum will interpret as a positive number
```

Extra zero byte ensures the bignum is interpreted as positive. Without this, a byte sequence starting with `0x80`+ would be treated as negative in MPI format. The `reverse_copy()` converts from the input's byte order to bignum's expected format.

```
    vector<unsigned char> vchTmp(pend-pbegin+1, 0);
reverse_copy(pbegin, pend, vchTmp.begin());

// Convert little endian data to bignum
    CBigNum bn;
    bn.setvch(vchTmp);

// Convert bignum to string
```

```
    string str;
```

Space reservation: `(pend - pbegin) * 138 / 100 + 1`. Base58 encoding expands data by factor log(256)/log(58) ≈ 1.37. The calculation 138/100 ≈ 1.38 provides a safe upper bound. Pre-allocating avoids repeated reallocations as the string grows.

```
    str.reserve((pend - pbegin) * 138 / 100 + 1);
    CBigNum dv;
    CBigNum rem;
```

The encoding loop: repeatedly divide by 58, taking remainders. Each remainder (0-57) maps to one Base58 character via the alphabet lookup. The loop continues until the number reaches zero. This produces digits from least to most significant, hence the later reversal.

```
while (bn > bn0)
{
if (!BN_div(&dv, &rem, &bn, &bn58, pctx))
throw bignum_error("EncodeBase58 : BN_div failed");
        bn = dv;
unsigned int c = rem.getulong();
        str += pszBase58[c];
}

```

Preserving leading zeros—a crucial detail. In Base58, leading `1` characters represent leading zero bytes in the original data. Without this loop, input `0x00 0x00 0xAB` would encode the same as `0xAB`, losing information. By counting leading zero bytes and prepending that many `1`s, the encoding becomes invertible. This is essential for addresses: the version byte `0x00` must be preserved.

```
// Leading zeroes encoded as base58 zeros
for (const unsigned char* p = pbegin; p < pend && *p == 0; p++)
        str += pszBase58[0];

// Convert little endian string to big endian
reverse(str.begin(), str.end());
return str;
}

inline string EncodeBase58(const vector<unsigned char>& vch)
{
return EncodeBase58(&vch[0], &vch[0] + vch.size());
}

```

`DecodeBase58()`—converts Base58 string back to binary. The inverse of EncodeBase58: treat the string as a base-58 number, convert to binary. Each character is looked up in the alphabet to get its value (0-57), then the running total is multiplied by 58 and the value added. The final result is converted to a byte vector.

```
inline bool DecodeBase58(const char* psz, vector<unsigned char>& vchRet)
{
    CAutoBN_CTX pctx;
    vchRet.clear();
    CBigNum bn58 = 58;
    CBigNum bn = 0;
    CBigNum bnChar;
while (isspace(*psz))
        psz++;

// Convert big endian string to bignum
```

```
for (const char* p = psz; *p; p++)
{
```

Character validation: `strchr(pszBase58, *p)` returns `NULL` if the character isn't in the Base58 alphabet. Invalid characters (including the excluded `0`, `O`, `I`, `l`) cause decoding to fail rather than silently producing wrong results. This is important for security—a corrupted address should be detected, not misinterpreted.

```
const char* p1 = strchr(pszBase58, *p);
if (p1 == NULL)
{
while (isspace(*p))
                p++;
if (*p != '\0')
return false;
break;
}
        bnChar.setulong(p1 - pszBase58);
if (!BN_mul(&bn, &bn, &bn58, pctx))
throw bignum_error("DecodeBase58 : BN_mul failed");
        bn += bnChar;
}

// Get bignum as little endian data
    vector<unsigned char> vchTmp = bn.getvch();

// Trim off sign byte if present
```

Sign byte trimming: if the high bit of the first data byte is set, MPI format prepends a `0x00` to indicate positive. This extra byte must be removed during decoding to recover the original data. The condition checks for this case: size ≥ 2, last byte is `0`, second-to-last has high bit set.

```
if (vchTmp.size() >= 2 && vchTmp.end()[-1] == 0 && vchTmp.end()[-2] >= 0x80)
        vchTmp.erase(vchTmp.end()-1);

// Restore leading zeros
```

Restoring leading zeros—the inverse of the encoding step. Count leading `1` characters (which represent zero bytes), then prepend that many zero bytes to the output. Combined with the character-by-character conversion, this ensures round-trip correctness: `DecodeBase58(EncodeBase58(data)) == data`.

```
int nLeadingZeros = 0;
for (const char* p = psz; *p == pszBase58[0]; p++)
        nLeadingZeros++;
    vchRet.assign(nLeadingZeros + vchTmp.size(), 0);

// Convert little endian data to big endian
reverse_copy(vchTmp.begin(), vchTmp.end(), vchRet.end() - vchTmp.size());
return true;
}

inline bool DecodeBase58(const string& str, vector<unsigned char>& vchRet)
{
return DecodeBase58(str.c_str(), vchRet);
}
```

`EncodeBase58Check()`—adds checksum for error detection. The checksum is the first 4 bytes of SHA256(SHA256(data)). Double-SHA256 was Satoshi's standard construction throughout Bitcoin—it provides extra security margin against length-extension attacks. The 4-byte checksum catches 99.99999998% of random errors (1 - 1/2^32).

```
    CALLED FROM  Hash160ToAddress p. 96 · EncodeAddress irc.cpp p. 162
117 inline string EncodeBase58Check(const vector<unsigned char>& vchIn)
118 {
119 // add 4-byte hash check to the end
120     vector<unsigned char> vch(vchIn);
```

Double-SHA256 checksum: `Hash(vch.begin(), vch.end())` computes `SHA256(SHA256(vch))`. Only the first 4 bytes are used as checksum—enough for error detection while keeping addresses reasonably short. The checksum is appended to the data, then the combined result is Base58-encoded.

```
121     uint256 hash = Hash(vch.begin(), vch.end());
122     vch.insert(vch.end(), (unsigned char*)&hash, (unsigned char*)&hash + 4);
123 return EncodeBase58(vch);
124 }
125
```

`DecodeBase58Check()`—verifies checksum and strips it. After Base58 decoding, the last 4 bytes are the checksum. The function recomputes the checksum over the data portion and compares. If they match, the checksum is removed and the original data returned. If not, the function returns false—the input was corrupted.

```
126 inline bool DecodeBase58Check(const char* psz, vector<unsigned char>& vchRet)
127 {
128 if (!DecodeBase58(psz, vchRet))
129 return false;
130 if (vchRet.size() < 4)
131 {
132         vchRet.clear();
133 return false;
134 }
135     uint256 hash = Hash(vchRet.begin(), vchRet.end()-4);
```

Checksum verification: `memcmp(&hash, &vchRet.end()[-4], 4)`. This comparison checks if the computed and stored checksums match. A mismatch means the address was corrupted (typo, OCR error, transmission corruption). By returning false rather than the corrupted data, users are protected from sending funds to wrong addresses.

```
136 if (memcmp(&hash, &vchRet.end()[-4], 4) != 0)
137 {
138         vchRet.clear();
139 return false;
140 }
141     vchRet.resize(vchRet.size()-4);
142 return true;
143 }
144
145 inline bool DecodeBase58Check(const string& str, vector<unsigned char>& vchRet)
146 {
147 return DecodeBase58Check(str.c_str(), vchRet);
148 }
149
150
```

```
151
152
153
154
```

`ADDRESSVERSION = 0`—the version byte for mainnet pay-to-pubkey-hash addresses. This byte is prepended to the hash160 before encoding, causing mainnet addresses to start with '1'. Testnet uses version 111, causing addresses to start with 'm' or 'n'. Version bytes enable the network to evolve—new address types (like P2SH, starting with '3') use different versions.

```
155 static const unsigned char ADDRESSVERSION = 0;
156
```

`Hash160ToAddress()`—converts a 160-bit hash to a Bitcoin address. This is the final step of address generation: (1) SHA256 the public key, (2) RIPEMD160 the result (giving 'hash160'), (3) prepend version byte, (4) Base58Check encode. The 160-bit hash provides 2^80 security against collision attacks while keeping addresses a manageable length.

```
CALLED FROM PubKeyToAddress p. 97 · CMainFrame::InsertTransaction ui.cpp p. 485 ·
CTxDetailsDialog::CTxDetailsDialog ui.cpp p. 493
157 inline string Hash160ToAddress(uint160 hash160)
158 {
159 // add 1-byte version number to the front
160     vector<unsigned char> vch(1, ADDRESSVERSION);
161     vch.insert(vch.end(), UBEGIN(hash160), UEND(hash160));
162 return EncodeBase58Check(vch);
163 }
164
```

`AddressToHash160()`—extracts the hash160 from a Bitcoin address. Used when creating transactions: the recipient's address is decoded to get the hash160, which goes into the scriptPubKey. Version byte validation (line 176: `nVersion <= ADDRESSVERSION`) rejects addresses from incompatible networks—you can't accidentally send mainnet coins to a testnet address.

```
165 inline bool AddressToHash160(const char* psz, uint160& hash160Ret)
166 {
167     vector<unsigned char> vch;
168 if (!DecodeBase58Check(psz, vch))
169 return false;
170 if (vch.empty())
171 return false;
172 unsigned char nVersion = vch[0];
173 if (vch.size() != sizeof(hash160Ret) + 1)
174 return false;
175 memcpy(&hash160Ret, &vch[1], sizeof(hash160Ret));
176 return (nVersion <= ADDRESSVERSION);
177 }
178
179 inline bool AddressToHash160(const string& str, uint160& hash160Ret)
180 {
181 return AddressToHash160(str.c_str(), hash160Ret);
182 }
183
```

`IsValidBitcoinAddress()`—validates address format. Checks that the string Base58Check decodes successfully and has correct version byte. Note: this only validates format, not that anyone controls the address. An address could be valid format but have no known private key (e.g., 'burn addresses' like 1BitcoinEaterAddressDontSendf59kuE).

```
184 inline bool IsValidBitcoinAddress(const char* psz)
185 {
186     uint160 hash160;
187 return AddressToHash160(psz, hash160);
188 }
189
190 inline bool IsValidBitcoinAddress(const string& str)
191 {
192 return IsValidBitcoinAddress(str.c_str());
193 }
194
195
196
197
```

`PubKeyToAddress()`—the complete public key to address pipeline in one line. `Hash160(vchPubKey)` computes `RIPEMD160(SHA256(pubkey))`, then `Hash160ToAddress()` adds version byte and Base58Check encodes. This is how wallet software generates receiving addresses from keys.

```
CALLED FROM  LoadWallet db.cpp p. 130 · CMainFrame::CMainFrame ui.cpp p. 481 ·
CMainFrame::InsertTransaction ui.cpp p. 484 · CTxDetailsDialog::CTxDetailsDialog ui.cpp p. 492 ·
CYourAddressDialog::OnButtonNew ui.cpp p. 503
198 inline string PubKeyToAddress(const vector<unsigned char>& vchPubKey)
199 {
200 return Hash160ToAddress(Hash160(vchPubKey));
201 }
202
```

SUMMARY

Base58Check encoding is a masterclass in practical cryptographic design. The checksum catches typos before funds are lost. The careful character selection prevents visual confusion. The version byte enables network evolution. And the encoding is compact enough for human use while being unambiguous enough for computer parsing.

The 4-byte checksum provides strong error detection: the probability of a random typo producing a valid address is 1 in 4.3 billion. For comparison, a credit card number's Luhn checksum catches all single-digit errors but misses some adjacent-digit transpositions. Bitcoin's double-SHA256 checksum is overkill for error detection alone—but Satoshi consistently chose conservative security margins.

The hash160 construction—RIPEMD160(SHA256(pubkey))—deserves attention. Using two different hash functions from two different design families provides defense in depth. If SHA-256 were broken, RIPEMD-160 still protects addresses from key recovery. If RIPEMD-160 were broken, SHA-256 still stands. This belt-and-suspenders approach is characteristic of Satoshi's security philosophy throughout Bitcoin: never depend on a single cryptographic assumption when you can depend on two.

KEY TERMS

Base58

The binary-to-text encoding whose 58-character alphabet drops 0, O, I, and l, so addresses can be read aloud or typed without ambiguity.

Base58Check

Base58 with a four-byte checksum appended, so a mistyped address fails validation with only about a one-in-four-billion chance of slipping through.

address

The string you share to receive payment: a hash160 plus a version byte, Base58Check-encoded into something like 1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa.

version byte

The leading byte (0×00 on mainnet) prepended before encoding that names the address type and produces the familiar leading 1.

checksum

The first four bytes of the double-SHA256 of the payload, appended so any corruption in copying the address is caught on decode.

hash160

The 160-bit RIPEMD160(SHA256(pubkey)) digest of a public key at the heart of every address.

burn address

A syntactically valid address with no known private key, so anything sent to it can never be spent.

bignum.h

499 lines CPP

WHEN 64 BITS AREN'T ENOUGH

Bitcoin deals with astronomically large numbers—256-bit integers that dwarf anything standard CPUs can handle natively. A 64-bit processor maxes out at 18 quintillion; Bitcoin routinely works with numbers that have 77+ digits. This file wraps OpenSSL's BIGNUM library to perform arithmetic on these titans.

Every cryptographic operation in Bitcoin relies on big number math: generating private keys (random 256-bit numbers), deriving public keys (elliptic curve point multiplication), creating signatures (modular arithmetic), verifying signatures (more modular arithmetic), and comparing hashes to difficulty targets. The elliptic curve work happens inside OpenSSL's own EC code (key.h); this file is Satoshi's direct handle on the same machinery, used for difficulty targets, Script arithmetic, and Base58 encoding.

OpenSSL's BIGNUM library has been battle-tested for decades in SSL/TLS—the encryption protecting billions of web connections.[1] Satoshi wisely reused this proven code rather than implementing big number arithmetic from scratch. Rolling your own crypto is how security vulnerabilities are born.

The CBigNum class wraps OpenSSL's C interface with C++ conveniences: operator overloading lets you write `a + b * c` instead of calling verbose functions. It also handles memory management automatically—BIGNUM requires manual allocation/deallocation, which the destructor handles safely.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

#include <stdexcept>
#include <vector>
#include <openssl/bn.h>
```

`bignum_error`—exception class for arithmetic failures. OpenSSL's BIGNUM functions return error codes rather than throwing exceptions. This wrapper converts errors to C++ exceptions, making error handling cleaner. Thrown when operations fail due to memory allocation errors or invalid operations.

```
class bignum_error : public std::runtime_error
{
public:
explicit bignum_error(const std::string& str) : std::runtime_error(str) {}
};
```

`CAutoBN_CTX`—RAII wrapper for OpenSSL's BN_CTX. A BN_CTX is a 'context' that provides temporary storage for intermediate values during multi-step calculations (like modular exponentiation). Without this, you'd need to allocate/free temporary BIGNUMs manually. The constructor allocates, destructor frees—C++ guarantees cleanup even if exceptions are thrown. This pattern appears throughout security-critical code.

```
class CAutoBN_CTX
{
protected:
    BN_CTX* pctx;
    BN_CTX* operator=(BN_CTX* pnew) { return pctx = pnew; }

public:
CAutoBN_CTX()
{
        pctx = BN_CTX_new();
if (pctx == NULL)
throw bignum_error("CAutoBN_CTX : BN_CTX_new() returned NULL");
}

~CAutoBN_CTX()
{
if (pctx != NULL)
BN_CTX_free(pctx);
}

operator BN_CTX*() { return pctx; }
    BN_CTX& operator*() { return *pctx; }
    BN_CTX** operator&() { return &pctx; }
bool operator!() { return (pctx == NULL); }
};
```

`CBigNum`—the heart of Bitcoin's big number support. Inherits from OpenSSL's `BIGNUM` struct (unusual but valid C++) to add C++ semantics. The inheritance means `CBigNum` can be passed directly to OpenSSL functions expecting `BIGNUM*`. Difficulty targets, Script arithmetic, and Base58 encoding flow through this class; ECDSA itself happens inside OpenSSL's EC code (key.h), which does its own BIGNUM math internally.

```
class CBigNum : public BIGNUM
{
public:
CBigNum()
{
```

`BN_init()`—initializes BIGNUM to zero. OpenSSL BIGNUMs must be initialized before use; uninitialized memory could cause crashes or security vulnerabilities. This is called in every constructor to ensure valid state.

```
BN_init(this);
}

CBigNum(const CBigNum& b)
{
BN_init(this);
if (!BN_copy(this, &b))
{
BN_clear_free(this);
throw bignum_error("CBigNum::CBigNum(const CBigNum&) : BN_copy failed");
}
}

explicit CBigNum(const std::string& str)
{
BN_init(this);
SetHex(str);
}

    CBigNum& operator=(const CBigNum& b)
{
if (!BN_copy(this, &b))
throw bignum_error("CBigNum::operator= : BN_copy failed");
return (*this);
}

```

`BN_clear_free()`—secure memory wiping. Unlike regular `free()`, this overwrites memory with zeros before releasing it. CBigNum rarely holds secrets in v0.1.0, but the habit is sound: a number wiped on release cannot leak through later memory inspection. The same paranoid cleanup appears throughout Bitcoin's key-handling code.

```
~CBigNum()
{
BN_clear_free(this);
}

```

Type conversion constructors—convert native integers to big numbers. The pattern `if (n >= 0) setulong(n); else setint64(n)` handles signed vs unsigned carefully. OpenSSL's `BN_set_word()` only handles unsigned values; negative numbers need the more complex MPI encoding path. These conversions let you write natural code like `CBigNum total = 50 * COIN`.

```
85 CBigNum(char n) { BN_init(this); if (n >= 0) setulong(n); else setint64(n); }
86 CBigNum(short n) { BN_init(this); if (n >= 0) setulong(n); else setint64(n); }
87 CBigNum(int n) { BN_init(this); if (n >= 0) setulong(n); else setint64(n); }
88 CBigNum(long n) { BN_init(this); if (n >= 0) setulong(n); else setint64(n); }
89 CBigNum(int64 n) { BN_init(this); setint64(n); }
90 CBigNum(unsigned char n) { BN_init(this); setulong(n); }
91 CBigNum(unsigned short n) { BN_init(this); setulong(n); }
92 CBigNum(unsigned int n) { BN_init(this); setulong(n); }
93 CBigNum(unsigned long n) { BN_init(this); setulong(n); }
94 CBigNum(uint64 n) { BN_init(this); setuint64(n); }
95 explicit CBigNum(uint256 n) { BN_init(this); setuint256(n); }
96
97 explicit CBigNum(const std::vector<unsigned char>& vch)
98 {
99 BN_init(this);
100 setvch(vch);
101 }
102
103 void setulong(unsigned long n)
104 {
105 if (!BN_set_word(this, n))
106 throw bignum_error("CBigNum conversion from unsigned long : BN_set_word failed");
107 }
108
109 unsigned long getulong() const
110 {
111 return BN_get_word(this);
112 }
113
114 unsigned int getuint() const
115 {
116 return BN_get_word(this);
117 }
118
119 int getint() const
120 {
121 unsigned long n = BN_get_word(this);
122 if (!BN_is_negative(this))
123 return (n > INT_MAX ? INT_MAX : n);
124 else
125 return (n > INT_MAX ? INT_MIN : -(int)n);
126 }
127
```

`setint64()`—converts 64-bit signed integer to BIGNUM via MPI format. MPI (Multi-Precision Integer) is a big-endian byte format with a 4-byte length prefix. The sign bit is encoded in the high bit of the first data byte. This complexity exists because OpenSSL's native functions don't handle 64-bit signed values directly. The careful handling of leading zeros and sign bits is essential for correct encoding.

```
128 void setint64(int64 n)
129 {
130 unsigned char pch[sizeof(n) + 6];
131 unsigned char* p = pch + 4;
132 bool fNegative = false;
133 if (n < (int64)0)
134 {
135             n = -n;
136             fNegative = true;
```

```
}
bool fLeadingZeroes = true;
for (int i = 0; i < 8; i++)
{
unsigned char c = (n >> 56) & 0xff;
            n <<= 8;
if (fLeadingZeroes)
{
if (c == 0)
continue;
if (c & 0x80)
*p++ = (fNegative ? 0x80 : 0);
else if (fNegative)
                    c |= 0x80;
                fLeadingZeroes = false;
}
*p++ = c;
}
unsigned int nSize = p - (pch + 4);
        pch[0] = (nSize >> 24) & 0xff;
        pch[1] = (nSize >> 16) & 0xff;
        pch[2] = (nSize >> 8) & 0xff;
        pch[3] = (nSize) & 0xff;
BN_mpi2bn(pch, p - pch, this);
}

void setuint64(uint64 n)
{
unsigned char pch[sizeof(n) + 6];
unsigned char* p = pch + 4;
bool fLeadingZeroes = true;
for (int i = 0; i < 8; i++)
{
unsigned char c = (n >> 56) & 0xff;
            n <<= 8;
if (fLeadingZeroes)
{
if (c == 0)
continue;
if (c & 0x80)
*p++ = 0;
                fLeadingZeroes = false;
}
*p++ = c;
}
unsigned int nSize = p - (pch + 4);
        pch[0] = (nSize >> 24) & 0xff;
        pch[1] = (nSize >> 16) & 0xff;
        pch[2] = (nSize >> 8) & 0xff;
        pch[3] = (nSize) & 0xff;
BN_mpi2bn(pch, p - pch, this);
}

```

`setuint256()`—converts Bitcoin's `uint256` type to `BIGNUM`. This bridges Bitcoin's hash/key representation with OpenSSL's arithmetic. The loop walks backwards through the `uint256`'s bytes (little-endian to big-endian conversion), skipping leading zeros for compactness. Used to build `bnProofOfWorkLimit` from `~uint256(0) >> 32` (main.h:22). The proof-of-work check itself goes the other way: it converts the target to a `uint256` with `getuint256()` and compares there (main.cpp:1182).

```
void setuint256(uint256 n)
{
unsigned char pch[sizeof(n) + 6];
unsigned char* p = pch + 4;
bool fLeadingZeroes = true;
unsigned char* pbegin = (unsigned char*)&n;
unsigned char* psrc = pbegin + sizeof(n);
```

```
while (psrc != pbegin)
{
unsigned char c = *(--psrc);
if (fLeadingZeroes)
{
if (c == 0)
continue;
if (c & 0x80)
*p++ = 0;
                fLeadingZeroes = false;
}
*p++ = c;
}
unsigned int nSize = p - (pch + 4);
        pch[0] = (nSize >> 24) & 0xff;
        pch[1] = (nSize >> 16) & 0xff;
        pch[2] = (nSize >> 8) & 0xff;
        pch[3] = (nSize >> 0) & 0xff;
BN_mpi2bn(pch, p - pch, this);
}

    uint256 getuint256()
{
unsigned int nSize = BN_bn2mpi(this, NULL);
if (nSize < 4)
return 0;
        std::vector<unsigned char> vch(nSize);
BN_bn2mpi(this, &vch[0]);
if (vch.size() > 4)
            vch[4] &= 0x7f;
        uint256 n = 0;
for (int i = 0, j = vch.size()-1; i < sizeof(n) && j >= 4; i++, j--)
((unsigned char*)&n)[i] = vch[j];
return n;
}

void setvch(const std::vector<unsigned char>& vch)
{
        std::vector<unsigned char> vch2(vch.size() + 4);
unsigned int nSize = vch.size();
        vch2[0] = (nSize >> 24) & 0xff;
        vch2[1] = (nSize >> 16) & 0xff;
        vch2[2] = (nSize >> 8) & 0xff;
        vch2[3] = (nSize >> 0) & 0xff;
reverse_copy(vch.begin(), vch.end(), vch2.begin() + 4);
BN_mpi2bn(&vch2[0], vch2.size(), this);
}

    std::vector<unsigned char> getvch() const
{
unsigned int nSize = BN_bn2mpi(this, NULL);
if (nSize < 4)
return std::vector<unsigned char>();
        std::vector<unsigned char> vch(nSize);
BN_bn2mpi(this, &vch[0]);
        vch.erase(vch.begin(), vch.begin() + 4);
reverse(vch.begin(), vch.end());
return vch;
}

```

`SetCompact()`—decodes Bitcoin's compact target format. Block headers store the proof-of-work *target* as a 4-byte compact value (`nBits`) rather than a full 256-bit number. Format: first byte is exponent (number of bytes), remaining 3 bytes are mantissa. Example: `0x1d00ffff` means a `0x1d` (29) byte number with high bytes `00ffff`. This encoding saves space in headers while representing any 256-bit target.

```
    CBigNum& SetCompact(unsigned int nCompact)
```

```
{
unsigned int nSize = nCompact >> 24;
        std::vector<unsigned char> vch(4 + nSize);
        vch[3] = nSize;
if (nSize >= 1) vch[4] = (nCompact >> 16) & 0xff;
if (nSize >= 2) vch[5] = (nCompact >> 8) & 0xff;
if (nSize >= 3) vch[6] = (nCompact >> 0) & 0xff;
BN_mpi2bn(&vch[0], vch.size(), this);
return *this;
}

```

`GetCompact()`—encodes BIGNUM to compact format. The inverse of SetCompact(). Mining code uses this to store the current target in block headers as `nBits`. The format sacrifices precision (only 3 significant bytes) but this is acceptable—the target only needs to be approximately correct. The slight loss of precision is bounded and predictable.

```
unsigned int GetCompact() const
{
unsigned int nSize = BN_bn2mpi(this, NULL);
        std::vector<unsigned char> vch(nSize);
        nSize -= 4;
BN_bn2mpi(this, &vch[0]);
unsigned int nCompact = nSize << 24;
if (nSize >= 1) nCompact |= (vch[4] << 16);
if (nSize >= 2) nCompact |= (vch[5] << 8);
if (nSize >= 3) nCompact |= (vch[6] << 0);
return nCompact;
}

```

`SetHex()`—parses hexadecimal string to `BIGNUM`. Block hashes, transaction IDs, and keys are commonly displayed as hex strings. This function handles optional `0x` prefix, leading whitespace, and negative signs. The lookup table `phexdigit[256]` provides O(1) hex-to-value conversion. Useful for debugging; hardcoded hashes like the genesis block's use `uint256`'s own `SetHex()` instead.

```
void SetHex(const std::string& str)
{
// skip 0x
const char* psz = str.c_str();
while (isspace(*psz))
            psz++;
bool fNegative = false;
if (*psz == '-')
{
            fNegative = true;
            psz++;
}
if (psz[0] == '0' && tolower(psz[1]) == 'x')
            psz += 2;
while (isspace(*psz))
            psz++;

// hex string to bignum
static char phexdigit[256] = { 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0, 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0, 0,1,2,3,4,5,6,7,8,9,0,0,0,0,0,0,
0,0xa,0xb,0xc,0xd,0xe,0xf,0,0,0,0,0,0,0,0, 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0,0xa,0xb,0xc,0xd,0xe,0xf,0,0,0,0,0,0,0,0 };
*this = 0;
while (isxdigit(*psz))
{
*this <<= 4;
int n = phexdigit[*psz++];
*this += n;
```

```
}
if (fNegative)
*this = 0 - *this;
}

```

Serialization support—enables storing/transmitting `CBigNum` values. Bitcoin's custom serialization framework (see serialize.h) uses these methods. `getvch()` converts to byte vector for transmission; `setvch()` reconstructs from bytes. The MPI format ensures consistent encoding across different machines regardless of native integer representation.

```
unsigned int GetSerializeSize(int nType=0, int nVersion=VERSION) const
{
return ::GetSerializeSize(getvch(), nType, nVersion);
}

template<typename Stream>
void Serialize(Stream& s, int nType=0, int nVersion=VERSION) const
{
::Serialize(s, getvch(), nType, nVersion);
}

template<typename Stream>
void Unserialize(Stream& s, int nType=0, int nVersion=VERSION)
{
        vector<unsigned char> vch;
::Unserialize(s, vch, nType, nVersion);
setvch(vch);
}


bool operator!() const
{
return BN_is_zero(this);
}

```

Arithmetic operators—make big numbers feel like native types. `operator+=` calls OpenSSL's `BN_add()`; `operator*=` calls `BN_mul()`; etc. The `CAutoBN_CTX` provides temporary storage needed by multiplication and division. These operators enable readable code like `bnNew *= nActualTimespan` in difficulty retargeting (main.cpp), rather than verbose function calls.

```
    CBigNum& operator+=(const CBigNum& b)
{
if (!BN_add(this, this, &b))
throw bignum_error("CBigNum::operator+= : BN_add failed");
return *this;
}

    CBigNum& operator-=(const CBigNum& b)
{
*this = *this - b;
return *this;
}

    CBigNum& operator*=(const CBigNum& b)
{
        CAutoBN_CTX pctx;
if (!BN_mul(this, this, &b, pctx))
throw bignum_error("CBigNum::operator*= : BN_mul failed");
return *this;
}

    CBigNum& operator/=(const CBigNum& b)
{
```

```
*this = *this / b;
return *this;
}

    CBigNum& operator%=(const CBigNum& b)
{
*this = *this % b;
return *this;
}

```

Bit shift operators. In v0.1.0 their main users are Script's OP_2MUL, OP_2DIV, OP_LSHIFT, and OP_RSHIFT (script.cpp) and `SetHex()` above, which shifts in four bits per hex digit. (The halving's `nSubsidy >>= (nBestHeight / 210000)` is a plain 64-bit shift, `bnProofOfWorkLimit`'s `~uint256(0) >> 32` is a `uint256` shift, and retargeting uses `*=`/`/=`.) OpenSSL's `BN_lshift()` and `BN_rshift()` handle arbitrarily large shifts correctly—you can shift a 256-bit number by 200 bits without overflow issues.

```
    CBigNum& operator<<=(unsigned int shift)
{
if (!BN_lshift(this, this, shift))
throw bignum_error("CBigNum:operator<<= : BN_lshift failed");
return *this;
}

    CBigNum& operator>>=(unsigned int shift)
{
if (!BN_rshift(this, this, shift))
throw bignum_error("CBigNum:operator>>= : BN_rshift failed");
return *this;
}


    CBigNum& operator++()
{
// prefix operator
if (!BN_add(this, this, BN_value_one()))
throw bignum_error("CBigNum::operator++ : BN_add failed");
return *this;
}

const CBigNum operator++(int)
{
// postfix operator
const CBigNum ret = *this;
++(*this);
return ret;
}

    CBigNum& operator--()
{
// prefix operator
        CBigNum r;
if (!BN_sub(&r, this, BN_value_one()))
throw bignum_error("CBigNum::operator-- : BN_sub failed");
*this = r;
return *this;
}

const CBigNum operator--(int)
{
// postfix operator
const CBigNum ret = *this;
--(*this);
return ret;
}


```

```
420 friend inline const CBigNum operator-(const CBigNum& a, const CBigNum& b);
421 friend inline const CBigNum operator/(const CBigNum& a, const CBigNum& b);
422 friend inline const CBigNum operator%(const CBigNum& a, const CBigNum& b);
423 };
424
425
426
```

Free-standing arithmetic operators. C++ convention: compound operators (`+=`) are members, simple operators (`+`) are free functions built on compound operators. This minimizes code duplication while providing the full expected arithmetic interface. The pattern `CBigNum r; BN_add(&r, &a, &b); return r;` creates a new `BIGNUM` for the result, leaving operands unchanged.

```
427 inline const CBigNum operator+(const CBigNum& a, const CBigNum& b)
428 {
429     CBigNum r;
430 if (!BN_add(&r, &a, &b))
431 throw bignum_error("CBigNum::operator+ : BN_add failed");
432 return r;
433 }
434
435 inline const CBigNum operator-(const CBigNum& a, const CBigNum& b)
436 {
437     CBigNum r;
438 if (!BN_sub(&r, &a, &b))
439 throw bignum_error("CBigNum::operator- : BN_sub failed");
440 return r;
441 }
442
443 inline const CBigNum operator-(const CBigNum& a)
444 {
445     CBigNum r(a);
446 BN_set_negative(&r, !BN_is_negative(&r));
447 return r;
448 }
449
450 inline const CBigNum operator*(const CBigNum& a, const CBigNum& b)
451 {
452     CAutoBN_CTX pctx;
453     CBigNum r;
454 if (!BN_mul(&r, &a, &b, pctx))
455 throw bignum_error("CBigNum::operator* : BN_mul failed");
456 return r;
457 }
458
```

`operator/()` and `operator%()`—division and modulus. Modular arithmetic is the foundation of public-key cryptography. ECDSA signatures involve operations like `(z + r*privateKey) * k_inverse mod n` where `n` is the curve order. OpenSSL's `BN_div()` and `BN_mod()` handle arbitrary-precision division with full precision—no rounding errors that would break cryptographic operations.

```
459 inline const CBigNum operator/(const CBigNum& a, const CBigNum& b)
460 {
461     CAutoBN_CTX pctx;
462     CBigNum r;
463 if (!BN_div(&r, NULL, &a, &b, pctx))
464 throw bignum_error("CBigNum::operator/ : BN_div failed");
465 return r;
466 }
467
468 inline const CBigNum operator%(const CBigNum& a, const CBigNum& b)
469 {
470     CAutoBN_CTX pctx;
471     CBigNum r;
```

```
472 if (!BN_mod(&r, &a, &b, pctx))
473 throw bignum_error("CBigNum::operator% : BN_div failed");
474 return r;
475 }
476
477 inline const CBigNum operator<<(const CBigNum& a, unsigned int shift)
478 {
479     CBigNum r;
480 if (!BN_lshift(&r, &a, shift))
481 throw bignum_error("CBigNum:operator<< : BN_lshift failed");
482 return r;
483 }
484
485 inline const CBigNum operator>>(const CBigNum& a, unsigned int shift)
486 {
487     CBigNum r;
488 if (!BN_rshift(&r, &a, shift))
489 throw bignum_error("CBigNum:operator>> : BN_rshift failed");
490 return r;
491 }
492
```

Comparison operators—enable intuitive comparisons like `bnNew > bnProofOfWorkLimit`, which keeps retargeting from ever making mining easier than the minimum difficulty (main.cpp:719). The proof-of-work test itself—a block's hash must not exceed the target—is done on `uint256` values after converting the target with `getuint256()`. `BN_cmp()` returns -1, 0, or 1 like `strcmp()`. These operators wrap that into natural boolean expressions.

```
493 inline bool operator==(const CBigNum& a, const CBigNum& b) { return (BN_cmp(&a, &b) == 0); }
494 inline bool operator!=(const CBigNum& a, const CBigNum& b) { return (BN_cmp(&a, &b) != 0); }
495 inline bool operator<=(const CBigNum& a, const CBigNum& b) { return (BN_cmp(&a, &b) <= 0); }
496 inline bool operator>=(const CBigNum& a, const CBigNum& b) { return (BN_cmp(&a, &b) >= 0); }
497 inline bool operator<(const CBigNum& a, const CBigNum& b) { return (BN_cmp(&a, &b) < 0); }
498 inline bool operator>(const CBigNum& a, const CBigNum& b) { return (BN_cmp(&a, &b) > 0); }
499
```

SUMMARY

Big number arithmetic is the mathematical foundation of Bitcoin's security. Private keys are 256-bit random numbers; public keys are points on an elliptic curve computed via point multiplication (repeated addition of 256-bit coordinates); signatures involve modular inverse and multiplication. All of this requires numbers far larger than CPUs handle natively.

Satoshi's choice to wrap OpenSSL rather than implement custom big number code was wise. Cryptographic arithmetic is notoriously difficult to get right—subtle bugs can leak private keys through timing variations or incorrect carries. OpenSSL's BIGNUM library has been audited extensively and is used by software protecting critical infrastructure worldwide.

The compact difficulty format (SetCompact/GetCompact) deserves special recognition. By encoding 256-bit targets as 4 bytes, block headers stay fixed at 80 bytes, making lightweight clients practical. A phone can verify proof-of-work by checking just the header, without downloading the full block data. This compression was essential for Bitcoin's scalability vision—and for the SPV (Simplified Payment Verification) protocol Satoshi described in Section 8 of the whitepaper.[2]

KEY TERMS

CBigNum

Bitcoin's C++ wrapper around OpenSSL's BIGNUM, letting the code write a + b on numbers far larger than any CPU register holds.

arbitrary precision

Arithmetic on integers of any size instead of the fixed 32 or 64 bits a processor handles natively, required for 256-bit keys and targets.

compact target

The 4-byte nBits encoding of the 256-bit proof-of-work threshold, one exponent byte plus three mantissa bytes, decoded by SetCompact.

MPI

Multi-Precision Integer, OpenSSL's big-endian byte format with a length prefix and a sign bit, used to store and transmit big numbers.

modular arithmetic

Math in which results wrap around at a fixed modulus; ECDSA signing and verification are built from modular multiplication and inversion.

BN_CTX

OpenSSL's scratch context holding temporary values during multi-step calculations, managed here by the RAII wrapper CAutoBN_CTX.

RAII

The C++ pattern in which a constructor acquires a resource and the destructor releases it, guaranteeing cleanup even when an exception is thrown.

operator overloading

The C++ feature letting a class redefine arithmetic and comparison symbols, so big-number code reads like ordinary math.

SOURCES

1. OpenSSL Project, *OpenSSL License / SSLeay License* (1998–2009). https://www.openssl.org/source/license-openssl-ssleay.txt
2. Satoshi Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (October 2008). https://bitcoin.org/bitcoin.pdf

db.cpp

605 lines CPP

MEMORY THAT SURVIVES

Bitcoin needs persistent storage that survives restarts: your wallet (private keys and transaction history), the blockchain index (where each block lives on disk), and peer addresses (nodes to reconnect to after shutdown). Satoshi chose **Berkeley DB** (BDB), a mature embedded key-value database from Sleepycat Software—battle-tested since 1991 and used by OpenLDAP, MySQL, and Subversion.[1] It required no separate server process, supported ACID transactions, and shipped with solid C++ bindings.

Everything in Bitcoin is ultimately stored as a key-value pair. Transactions are keyed by their 256-bit hash. Blocks are indexed by their hash. Addresses live under a 160-bit key. Even user settings ("fGenerateBitcoins", "nTransactionFee") are persisted as string-keyed values. `db.cpp` is the single file that bridges Bitcoin's in-memory C++ objects and their durable on-disk representations.

The file defines five databases: `CTxDB` (blockchain index and transaction positions), `CAddrDB` (peer network addresses), `CReviewDB` (user reviews for a marketplace reputation system Satoshi never launched), `CMarketDB` (a marketplace listing stub that was never built out), and `CWalletDB` (your private keys, transactions, and address book). All five inherit from the base `CDB` class, which handles the BDB environment lifecycle.

BDB served Bitcoin well until v0.8 (February 2013), when an obscure configuration limit became a consensus bug. BDB imposes a ceiling on concurrent "database locks." Block 225,430—a large but perfectly valid block—exceeded that ceiling on BDB-backed nodes but not on the new LevelDB-backed ones, causing an unintentional chain split documented as **BIP 50**.[2] Resolution required miners to deliberately orphan the longer (LevelDB) chain and revert to the shorter (BDB-compatible) one — a coordinated rollback that orphaned the blocks mined on the LevelDB chain, wiping out the coinbases miners—including BTC Guild, which downgraded first—had earned there.[2] The incident proved that consensus rules extend beyond Bitcoin's source code to encompass the database engine, its configuration, and even the operating system. The blockchain index stayed on LevelDB; the wallet

kept BDB for years for compatibility with existing `wallet.dat` files, until descriptor wallets moved to SQLite and Bitcoin Core 30 (2025) dropped Berkeley DB entirely, keeping only a small reader to migrate old wallets.[3][4]

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

#include "headers.h"









//
// CDB
//

```

`cs_db` — the database mutex. All BDB environment operations acquire this lock first, preventing concurrent initialization or shutdown from two threads.

```
static CCriticalSection cs_db;
```

`fDbEnvInit` — lazy-init flag. The BDB environment is expensive to open; it is created only on the first `CDB` constructor call and shared by every database file thereafter.

```
static bool fDbEnvInit = false;
DbEnv dbenv(0);
```

`mapFileUseCount` — reference-counts open database handles by filename. `DBFlush` uses this to know which files can be safely flushed: a file with a non-zero count still has live cursors or transactions.

```
static map<string, int> mapFileUseCount;

```

`CDBInit` — RAII guard for the BDB environment. Its destructor calls `dbenv.close()` when the program exits, ensuring the write-ahead log is flushed and no database is left in an inconsistent state. The single global instance `instance_of_cdbinit` at line 36 makes this automatic.

```
class CDBInit
{
public:
CDBInit()
{
}
~CDBInit()
{
if (fDbEnvInit)
{
            dbenv.close(0);
            fDbEnvInit = false;
}
}
}
instance_of_cdbinit;


```

`CDB` constructor — opens (or creates) a named database file. `pszMode` mirrors `fopen` semantics: `"r"` = read-only, `"r+"` = read-write, `"cr+"` = create if absent then read-write. Every specialized database class passes a specific mode.

```
39 CDB::CDB(const char* pszFile, const char* pszMode, bool fTxn) : pdb(NULL)
40 {
41 int ret;
42 if (pszFile == NULL)
43 return;
44
```

Mode string parsing. `strchr` tests for individual characters: `'c'` means create if absent, absence of `'+'` and `'w'` means read-only. These flags are translated into BDB's `nFlags` bitmask below.

```
45 bool fCreate = strchr(pszMode, 'c');
46 bool fReadOnly = (!strchr(pszMode, '+') && !strchr(pszMode, 'w'));
47 unsigned int nFlags = DB_THREAD;
48 if (fCreate)
49         nFlags |= DB_CREATE;
50 else if (fReadOnly)
51         nFlags |= DB_RDONLY;
```

`DB_AUTO_COMMIT` — wraps every individual read or write in an implicit transaction when no explicit transaction is open. This gives single-operation atomicity for free: a write either commits fully or does nothing.

```
52 if (!fReadOnly || fTxn)
53         nFlags |= DB_AUTO_COMMIT;
54
```

First `CDB` construction triggers one-time environment setup, guarded by `cs_db`. Subsequent opens skip straight to line 85. The environment is shared process-wide: all five database files use the same log directory, lock manager, and memory pool.

```
55 CRITICAL_BLOCK(cs_db)
56 {
57 if (!fDbEnvInit)
58 {
59             string strAppDir = GetAppDir();
```

Transaction log directory (`\database` subdirectory). BDB's write-ahead log records every mutation here before applying it to the data file. On crash, BDB replays the log on `DB_RECOVER` to restore consistency—Bitcoin gets crash safety for free.

```
60             string strLogDir = strAppDir + "\\database";
61 _mkdir(strLogDir.c_str());
62 printf("dbenv.open strAppDir=%s\n", strAppDir.c_str());
63
64             dbenv.set_lg_dir(strLogDir.c_str());
65             dbenv.set_lg_max(10000000);
```

`set_lk_max_locks(10000)` — the configuration that became BIP 50.[2] BDB counts every page-level lock a transaction holds simultaneously. Large blocks with thousands of inputs can exceed this limit on old BDB builds. Nodes that hit the limit refused to validate the block; nodes on LevelDB accepted it — splitting the chain.

```
            dbenv.set_lk_max_locks(10000);
            dbenv.set_lk_max_objects(10000);
            dbenv.set_errfile(fopen("db.log", "a")); /// debug
///dbenv.log_set_config(DB_LOG_AUTO_REMOVE, 1); /// causes corruption
```

BDB environment flags. `DB_INIT_LOCK` enables the lock subsystem (for concurrent access). `DB_INIT_LOG` enables the write-ahead log (crash recovery). `DB_INIT_MPOOL` enables the shared buffer pool (read cache). `DB_INIT_TXN` enables transactions. `DB_PRIVATE` means no other process may share this environment.

```
            ret = dbenv.open(strAppDir.c_str(),
                             DB_CREATE     |
                             DB_INIT_LOCK  |
                             DB_INIT_LOG   |
                             DB_INIT_MPOOL |
                             DB_INIT_TXN   |
                             DB_THREAD     |
                             DB_PRIVATE    |
```

`DB_RECOVER` — on open, replay the write-ahead log and roll back any uncommitted transactions from a previous crash. Bitcoin inherits full ACID durability from BDB without writing any recovery code itself.

```
                             DB_RECOVER,
0);
if (ret > 0)
throw runtime_error(strprintf("CDB() : error %d opening database environment\n", ret));
            fDbEnvInit = true;
}

        strFile = pszFile;
++mapFileUseCount[strFile];
}

```

Open a `DB_BTREE` (B-tree) database. BDB supports hash tables and queues too, but B-tree gives efficient range scans — essential for `LoadBlockIndex`, which iterates every block record in key order. The logical name `"main"` allows multiple B-trees to share one physical file.

```
    pdb = new Db(&dbenv, 0);

    ret = pdb->open(NULL, // Txn pointer
                    pszFile, // Filename
"main", // Logical db name
                    DB_BTREE, // Database type
                    nFlags, // Flags
0);

if (ret > 0)
{
delete pdb;
        pdb = NULL;
CRITICAL_BLOCK(cs_db)
--mapFileUseCount[strFile];
```

```
        strFile = "";
throw runtime_error(strprintf("CDB() : can't open database file %s, error %d\n", pszFile, ret));
}

```

On first creation, write the software version. v0.1.0 writes it but never reads it back (`ReadVersion` has no callers)—a hook left for future versions to detect files written by older software.

```
if (fCreate && !Exists(string("version")))
WriteVersion(VERSION);

RandAddSeed();
}

```

`CDB::Close` — safe shutdown sequence. Aborts any open transaction (`vTxn.front()->abort()`), closes the `Db` handle, checkpoints the log, then decrements the file's use-count. Leaving this sequence incomplete — for example, by crashing — is safe because `DB_RECOVER` handles it on restart.

```
void CDB::Close()
{
if (!pdb)
return;
if (!vTxn.empty())
        vTxn.front()->abort();
    vTxn.clear();
    pdb->close(0);
delete pdb;
    pdb = NULL;

```

`txn_checkpoint(0, 0, 0)` — flushes the in-memory log buffer to the data file immediately. Without this, committed writes sit in the log until BDB decides to checkpoint. Called here and in `DBFlush` to guarantee that closed files are fully durable on disk.

```
    dbenv.txn_checkpoint(0, 0, 0);

CRITICAL_BLOCK(cs_db)
--mapFileUseCount[strFile];

RandAddSeed();
}

```

`DBFlush` — called periodically and on shutdown. Checkpoints the environment, then iterates `mapFileUseCount`: any file with zero open handles gets its log sequence numbers reset (`lsn_reset`), freeing log space. On shutdown, archives and removes stale log files, then closes the environment.

```
CALLED FROM  Shutdown ui.cpp p. 481
void DBFlush(bool fShutdown)
{
// Flush log data to the actual data file
//  on all files that are not in use
printf("DBFlush(%s)\n", fShutdown ? "true" : "false");
CRITICAL_BLOCK(cs_db)
{
        dbenv.txn_checkpoint(0, 0, 0);
        map<string, int>::iterator mi = mapFileUseCount.begin();
while (mi != mapFileUseCount.end())
{

```

```
            string strFile = (*mi).first;
int nRefCount = (*mi).second;
if (nRefCount == 0)
{
```

`lsn_reset` — resets log sequence numbers for a closed database file. Required before the file can be moved or copied safely (e.g., when a user backs up `wallet.dat` while Bitcoin is running). Without this reset, the file's LSN would be out of sync with the environment's log.

```
                dbenv.lsn_reset(strFile.c_str(), 0);
                mapFileUseCount.erase(mi++);
}
else
                mi++;
}
if (fShutdown)
{
char** listp;
if (mapFileUseCount.empty())
                dbenv.log_archive(&listp, DB_ARCH_REMOVE);
            dbenv.close(0);
            fDbEnvInit = false;
}
}
}




//
// CTxDB
//

```

`ReadTxIndex` — given a transaction hash, returns a `CTxIndex` containing the on-disk position (`CDiskTxPos`) and the number of outputs. This is the lookup that lets Bitcoin validate new transactions: before accepting a spend, the node checks here that the referenced output actually exists.

```
CALLED FROM CTxDB::ReadDiskTx p. 123 · CMerkleTx::SetMerkleBranch main.cpp p. 207 ·
CTransaction::DisconnectInputs main.cpp p. 215 · CTransaction::ConnectInputs main.cpp p. 215
bool CTxDB::ReadTxIndex(uint256 hash, CTxIndex& txindex)
{
assert(!fClient);
    txindex.SetNull();
return Read(make_pair(string("tx"), hash), txindex);
}

```

`UpdateTxIndex` — persists spent-output state. Connecting an input writes the spender's disk position into `vSpent`; disconnecting a block clears it. v0.1.0 does not delete a tx when one output is spent — it updates per-output markers.

```
CALLED FROM CTransaction::DisconnectInputs main.cpp p. 215 ·
CTransaction::ConnectInputs main.cpp p. 216
bool CTxDB::UpdateTxIndex(uint256 hash, const CTxIndex& txindex)
{
assert(!fClient);
return Write(make_pair(string("tx"), hash), txindex);
}
```

`AddTxIndex` — records where a newly-confirmed transaction lives on disk. The key is the transaction hash; the value is the file number, byte offset, and output count. This index is what makes it possible to locate any transaction in O(log n) time without scanning the entire blockchain.

```
    CALLED FROM  CTransaction::ConnectInputs main.cpp p. 217
186 bool CTxDB::AddTxIndex(const CTransaction& tx, const CDiskTxPos& pos, int nHeight)
187 {
188 assert(!fClient);
189
190 // Add to tx index
191     uint256 hash = tx.GetHash();
192     CTxIndex txindex(pos, tx.vout.size());
193 return Write(make_pair(string("tx"), hash), txindex);
194 }
195
    CALLED FROM  CTransaction::DisconnectInputs main.cpp p. 215
196 bool CTxDB::EraseTxIndex(const CTransaction& tx)
197 {
198 assert(!fClient);
199     uint256 hash = tx.GetHash();
200
201 return Erase(make_pair(string("tx"), hash));
202 }
203
    CALLED FROM  CTransaction::AcceptTransaction main.cpp p. 209 ·
    CWalletTx::AcceptWalletTransaction main.cpp p. 211 · ReacceptWalletTransactions main.cpp p. 212 ·
    CWalletTx::RelayWalletTransaction main.cpp p. 212 · AlreadyHave main.cpp p. 229
204 bool CTxDB::ContainsTx(uint256 hash)
205 {
206 assert(!fClient);
207 return Exists(make_pair(string("tx"), hash));
208 }
209
```

`ReadOwnerTxes` — scans for all transactions sent to an address. Uses a BDB cursor with `DB_SET_RANGE` to seek to the first `"owner"` record matching the 160-bit hash, then iterates with `DB_NEXT`. This is a range scan — the B-tree ordering makes it efficient.

```
210 bool CTxDB::ReadOwnerTxes(uint160 hash160, int nMinHeight, vector<CTransaction>& vtx)
211 {
212 assert(!fClient);
213     vtx.clear();
214
215 // Get cursor
216     Dbc* pcursor = GetCursor();
217 if (!pcursor)
218 return false;
219
```

`DB_SET_RANGE` positions the cursor at the first key ≥ the supplied key. On subsequent iterations `DB_NEXT` advances one record at a time. This cursor pattern appears in `LoadBlockIndex` and `LoadAddresses` as well — it is the idiomatic BDB way to iterate a subset of records.

```
220 unsigned int fFlags = DB_SET_RANGE;
221     loop
222 {
223 // Read next record
224         CDataStream ssKey;
225 if (fFlags == DB_SET_RANGE)
226             ssKey << string("owner") << hash160 << CDiskTxPos(0, 0, 0);
227         CDataStream ssValue;
```

```
        fFlags = DB_NEXT;
if (ret == DB_NOTFOUND)
break;
else if (ret != 0)
return false;

// Unserialize
        string strType;
        uint160 hashItem;
        CDiskTxPos pos;
        ssKey >> strType >> hashItem >> pos;
int nItemHeight;
        ssValue >> nItemHeight;

// Read transaction
if (strType != "owner" || hashItem != hash160)
break;
if (nItemHeight >= nMinHeight)
{
            vtx.resize(vtx.size()+1);
if (!vtx.back().ReadFromDisk(pos))
return false;
}
}
return true;
}

```

`ReadDiskTx` — two-step transaction retrieval: first fetch the `CTxIndex` (containing the disk position), then read the full serialized `CTransaction` from that position in `blk0001.dat`. The four overloads accept either a hash or an `COutPoint` (txid + output index), with and without returning the index.

```
bool CTxDB::ReadDiskTx(uint256 hash, CTransaction& tx, CTxIndex& txindex)
{
assert(!fClient);
    tx.SetNull();
if (!ReadTxIndex(hash, txindex))
return false;
return (tx.ReadFromDisk(txindex.pos));
}

bool CTxDB::ReadDiskTx(uint256 hash, CTransaction& tx)
{
    CTxIndex txindex;
return ReadDiskTx(hash, tx, txindex);
}

bool CTxDB::ReadDiskTx(COutPoint outpoint, CTransaction& tx, CTxIndex& txindex)
{
return ReadDiskTx(outpoint.hash, tx, txindex);
}

bool CTxDB::ReadDiskTx(COutPoint outpoint, CTransaction& tx)
{
    CTxIndex txindex;
return ReadDiskTx(outpoint.hash, tx, txindex);
}

```

`WriteBlockIndex` — persists one block's metadata to BDB. The key is the block hash; the value is a `CDiskBlockIndex` containing the previous hash, merkle root, timestamp, difficulty bits, nonce, and on-disk position. Everything needed to reconstruct the chain — without reading the raw block data.

```
    CALLED FROM  CBlock::DisconnectBlock main.cpp p. 218 · CBlock::ConnectBlock main.cpp p. 219 ·
    CBlock::AddToBlockIndex main.cpp p. 221
282 bool CTxDB::WriteBlockIndex(const CDiskBlockIndex& blockindex)
283 {
284 return Write(make_pair(string("blockindex"), blockindex.GetBlockHash()), blockindex);
285 }
286
    CALLED FROM  Reorganize main.cpp p. 220
287 bool CTxDB::EraseBlockIndex(uint256 hash)
288 {
289 return Erase(make_pair(string("blockindex"), hash));
290 }
291
    CALLED FROM  CTxDB::LoadBlockIndex p. 126
292 bool CTxDB::ReadHashBestChain(uint256& hashBestChain)
293 {
294 return Read(string("hashBestChain"), hashBestChain);
295 }
296
    CALLED FROM  Reorganize main.cpp p. 220 · CBlock::AddToBlockIndex main.cpp p. 221
297 bool CTxDB::WriteHashBestChain(uint256 hashBestChain)
298 {
299 return Write(string("hashBestChain"), hashBestChain);
300 }
301
```

`InsertBlockIndex` — interns a block hash into the global `mapBlockIndex`. Returns the existing pointer if the hash is already known, or allocates a new `CBlockIndex`. Because the pointer is stored back into the map's own key, `pindexNew->phashBlock` is stable for the lifetime of the map — a clean self-referential trick.

```
    CALLED FROM  CTxDB::LoadBlockIndex p. 125
302 CBlockIndex* InsertBlockIndex(uint256 hash)
303 {
304 if (hash == 0)
305 return NULL;
306
307 // Return existing
308     map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hash);
309 if (mi != mapBlockIndex.end())
310 return (*mi).second;
311
312 // Create new
313     CBlockIndex* pindexNew = new CBlockIndex();
314 if (!pindexNew)
315 throw runtime_error("LoadBlockIndex() : new CBlockIndex failed");
316     mi = mapBlockIndex.insert(make_pair(hash, pindexNew)).first;
317     pindexNew->phashBlock = &((*mi).first);
318
319 return pindexNew;
320 }
321
```

`LoadBlockIndex` — reconstructs the entire in-memory blockchain index from BDB on startup. Iterates every `"blockindex"` record, deserializes each `CDiskBlockIndex`, and re-links the `CBlockIndex` doubly-linked list (`pprev`, `pnext`). Without this, Bitcoin would need to re-scan every block file on each launch.

```
bool CTxDB::LoadBlockIndex()
{
// Get cursor
    Dbc* pcursor = GetCursor();
if (!pcursor)
return false;

unsigned int fFlags = DB_SET_RANGE;
    loop
{
// Read next record
        CDataStream ssKey;
if (fFlags == DB_SET_RANGE)
            ssKey << make_pair(string("blockindex"), uint256(0));
        CDataStream ssValue;
int ret = ReadAtCursor(pcursor, ssKey, ssValue, fFlags);
        fFlags = DB_NEXT;
if (ret == DB_NOTFOUND)
break;
else if (ret != 0)
return false;

// Unserialize
        string strType;
        ssKey >> strType;
if (strType == "blockindex")
{
            CDiskBlockIndex diskindex;
            ssValue >> diskindex;

// Construct block index object
            CBlockIndex* pindexNew = InsertBlockIndex(diskindex.GetBlockHash());
```

Re-linking the chain. Each `CDiskBlockIndex` stores `hashPrev` and `hashNext`. `InsertBlockIndex` converts those hashes back to pointers, rebuilding the doubly-linked list that lets the node walk the chain in either direction in O(1) per step.

```
            pindexNew->pprev          = InsertBlockIndex(diskindex.hashPrev);
            pindexNew->pnext          = InsertBlockIndex(diskindex.hashNext);
            pindexNew->nFile          = diskindex.nFile;
            pindexNew->nBlockPos      = diskindex.nBlockPos;
            pindexNew->nHeight        = diskindex.nHeight;
            pindexNew->nVersion       = diskindex.nVersion;
            pindexNew->hashMerkleRoot = diskindex.hashMerkleRoot;
            pindexNew->nTime          = diskindex.nTime;
            pindexNew->nBits          = diskindex.nBits;
            pindexNew->nNonce         = diskindex.nNonce;

// Watch for genesis block and best block
```

Genesis block detection. On first run, no genesis block exists yet; it will be added later. On subsequent starts, this line finds the genesis `CBlockIndex` and sets the global `pindexGenesisBlock` pointer — the anchor of the entire chain.

```
if (pindexGenesisBlock == NULL && diskindex.GetBlockHash() == hashGenesisBlock)
                pindexGenesisBlock = pindexNew;
}
```

```
369 else
370 {
371 break;
372 }
373 }
374
```

`ReadHashBestChain` — restores the chain tip. After iterating all block records, this line fetches the hash of the best (longest) chain as of last shutdown. Bitcoin then sets `pindexBest` and `nBestHeight`, giving the node an immediate starting point to request new blocks from peers.

```
375 if (!ReadHashBestChain(hashBestChain))
376 {
377 if (pindexGenesisBlock == NULL)
378 return true;
379 return error("CTxDB::LoadBlockIndex() : hashBestChain not found\n");
380 }
381
382 if (!mapBlockIndex.count(hashBestChain))
383 return error("CTxDB::LoadBlockIndex() : blockindex for hashBestChain not found\n");
384     pindexBest = mapBlockIndex[hashBestChain];
385     nBestHeight = pindexBest->nHeight;
386 printf("LoadBlockIndex(): hashBestChain=%s  height=%d\n", hashBestChain.ToString().substr(0,14).c_str(),
    nBestHeight);
387
388 return true;
389 }
390
391
392
393
394
395 //
396 // CAddrDB
397 //
398
```

`CAddrDB::WriteAddress` — persists a peer's IP, port, and service flags. The key is derived from the address itself (`addr.GetKey()`), so writing the same address twice is idempotent — it just updates the record.

```
    CALLED FROM  AddAddress net.cpp p. 327
399 bool CAddrDB::WriteAddress(const CAddress& addr)
400 {
401 return Write(make_pair(string("addr"), addr.GetKey()), addr);
402 }
403
```

`LoadAddresses` — peer discovery on startup. First reads `addr.txt` if it exists (user-supplied seed nodes), then iterates BDB for every previously-discovered peer. IRC bootstrapping (started in `net.cpp`) runs on every startup regardless.

```
404 bool CAddrDB::LoadAddresses()
405 {
406 CRITICAL_BLOCK(cs_mapAddresses)
407 {
408 // Load user provided addresses
```

`addr.txt` — a plain-text bootstrap file. One IP address per line. Satoshi included this escape hatch for users behind firewalls or on networks where IRC was blocked. It predates the `-connect` and `-addnode` command-line options added in later versions.

```
409         CAutoFile filein = fopen("addr.txt", "rt");
410 if (filein)
411 {
412 try
413 {
414 char psz[1000];
415 while (fgets(psz, sizeof(psz), filein))
416 {
417                     CAddress addr(psz, NODE_NETWORK);
418 if (addr.ip != 0)
419 AddAddress(*this, addr);
420 }
421 }
422 catch (...) { }
423 }
424
425 // Get cursor
426         Dbc* pcursor = GetCursor();
427 if (!pcursor)
428 return false;
429
430         loop
431 {
432 // Read next record
433             CDataStream ssKey;
434             CDataStream ssValue;
435 int ret = ReadAtCursor(pcursor, ssKey, ssValue);
436 if (ret == DB_NOTFOUND)
437 break;
438 else if (ret != 0)
439 return false;
440
441 // Unserialize
442             string strType;
443             ssKey >> strType;
444 if (strType == "addr")
445 {
446                 CAddress addr;
447                 ssValue >> addr;
448                 mapAddresses.insert(make_pair(addr.GetKey(), addr));
449 }
450 }
451
452 //// debug print
453 printf("mapAddresses:\n");
454 foreach(const PAIRTYPE(vector<unsigned char>, CAddress)& item, mapAddresses)
455             item.second.print();
456 printf("-----\n");
457 }
458
459 return true;
460 }
461
462 bool LoadAddresses()
463 {
464 return CAddrDB("cr+").LoadAddresses();
465 }
466
467
468
469
470 //
471 // CReviewDB
472 //
```

473

CReviewDB — a reputation database for Satoshi's never-launched marketplace. Records are keyed by a *user* hash: a `CUser` (with its reputation "atoms") and that user's list of signed `CReviews` (market.h). The idea: buyers could rate sellers, with reviews propagated across the network. The infrastructure exists — a `"review"` network message and the edit-review dialog both reach `WriteReviews` — but the feature never saw real use. It was quietly removed in later versions.

```
    CALLED FROM  CReview::AcceptReview market.cpp p. 307 ·
    CViewProductDialog::UpdateProductDisplay ui.cpp p. 511
474 bool CReviewDB::ReadReviews(uint256 hash, vector<CReview>& vReviews)
475 {
476     vReviews.size(); // msvc workaround, just need to do anything with vReviews
477 return Read(make_pair(string("reviews"), hash), vReviews);
478 }
479
    CALLED FROM  CReview::AcceptReview market.cpp p. 307
480 bool CReviewDB::WriteReviews(uint256 hash, const vector<CReview>& vReviews)
481 {
482 return Write(make_pair(string("reviews"), hash), vReviews);
483 }
484
485
486
487
488
489
490
491 //
492 // CWalletDB
493 //
494
```

CWalletDB::LoadWallet — deserializes the entire wallet from `wallet.dat` at startup. Uses a BDB cursor to iterate every record, switching on the string type tag stored at the front of each key. The wallet schema — name, tx, key, defaultkey, setting — is a simple tagged union encoded in BDB.

```
495 bool CWalletDB::LoadWallet(vector<unsigned char>& vchDefaultKeyRet)
496 {
497     vchDefaultKeyRet.clear();
498
499 //// todo: shouldn't we catch exceptions and try to recover and continue?
```

Double lock: `cs_mapKeys` then `cs_mapWallet`. Both must be held simultaneously while populating the in-memory maps. Consistent lock ordering (always keys before wallet) prevents deadlock with any other thread that needs both locks.

```
500 CRITICAL_BLOCK(cs_mapKeys)
501 CRITICAL_BLOCK(cs_mapWallet)
502 {
503 // Get cursor
504         Dbc* pcursor = GetCursor();
505 if (!pcursor)
506 return false;
507
508         loop
509 {
510 // Read next record
511             CDataStream ssKey;
512             CDataStream ssValue;
513 int ret = ReadAtCursor(pcursor, ssKey, ssValue);
514 if (ret == DB_NOTFOUND)
```

```
516 else if (ret != 0)
517 return false;
518
519 // Unserialize
520 // Taking advantage of the fact that pair serialization
521 // is just the two items serialized one after the other
522             string strType;
523             ssKey >> strType;
```

`"name"` record — address book entry. Maps a Bitcoin address string to a human-readable label (e.g., `"Alice"`). These are the labels shown in the GUI's Sending Addresses and Receiving Addresses tabs. Purely local — never broadcast to the network.

```
524 if (strType == "name")
525 {
526                 string strAddress;
527                 ssKey >> strAddress;
528                 ssValue >> mapAddressBook[strAddress];
529 }
```

`"tx"` record — a wallet transaction. Stores the full `CWalletTx`, which wraps a `CTransaction` with extra metadata: the block it confirmed in, the time it was received, and whether it's marked as spent. The hash mismatch check at line 537 detects a corrupted wallet.

```
530 else if (strType == "tx")
531 {
532                 uint256 hash;
533                 ssKey >> hash;
534                 CWalletTx& wtx = mapWallet[hash];
535                 ssValue >> wtx;
536
537 if (wtx.GetHash() != hash)
538 printf("Error in wallet.dat, hash mismatch\n");
539
540 //// debug print
541 //printf("LoadWallet  %s\n", wtx.GetHash().ToString().c_str());
542 //printf(" %12I64d  %s  %s  %s\n",
543 //    wtx.vout[0].nValue,
544 //    DateTimeStr(wtx.nTime).c_str(),
545 //    wtx.hashBlock.ToString().substr(0,14).c_str(),
546 //    wtx.mapValue["message"].c_str());
547 }
```

`"key"` record — a keypair. The BDB key holds the 65-byte uncompressed public key; the value holds the private key. Both are loaded into `mapKeys` (pubkey → privkey) and `mapPubKeys` (hash160 → pubkey). Losing this record means losing the ability to spend any coins sent to that address.

```
548 else if (strType == "key")
549 {
550                 vector<unsigned char> vchPubKey;
551                 ssKey >> vchPubKey;
552                 CPrivKey vchPrivKey;
553                 ssValue >> vchPrivKey;
554
555                 mapKeys[vchPubKey] = vchPrivKey;
556                 mapPubKeys[Hash160(vchPubKey)] = vchPubKey;
557 }
```

`"defaultkey"` — your primary receiving address. This is the key shown in the GUI's *Your Bitcoin Address* field. On first run, no default key exists and a fresh one is generated (see `LoadWallet` at line 580). On subsequent runs, this record restores continuity — you get the same address.

```
558 else if (strType == "defaultkey")
559 {
560                 ssValue >> vchDefaultKeyRet;
561 }
```

`"setting"` records — persisted preferences. Three are saved here: `fGenerateBitcoins` (mining on/off), `nTransactionFee` (fee per transaction in satoshis), and `addrIncoming` (the listening address for incoming connections). Settings survive restarts without any configuration file.

```
562 else if (strType == "setting") /// or settings or option or options or config?
563 {
564                 string strKey;
565                 ssKey >> strKey;
566 if (strKey == "fGenerateBitcoins")  ssValue >> fGenerateBitcoins;
567 if (strKey == "nTransactionFee")    ssValue >> nTransactionFee;
568 if (strKey == "addrIncoming")       ssValue >> addrIncoming;
569 }
570 }
571 }
572
573 printf("fGenerateBitcoins = %d\n", fGenerateBitcoins);
574 printf("nTransactionFee = %I64d\n", nTransactionFee);
575 printf("addrIncoming = %s\n", addrIncoming.ToString().c_str());
576
577 return true;
578 }
579
```

`LoadWallet` wrapper — handles the first-run case. If the wallet loaded no default key (fresh install), it generates a new keypair, adds it to the wallet database, labels it `"Your Address"`, and writes it as the default. From this point on, the node has a Bitcoin identity: a private key that controls an address.

```
580 bool LoadWallet()
581 {
582     vector<unsigned char> vchDefaultKey;
583 if (!CWalletDB("cr").LoadWallet(vchDefaultKey))
584 return false;
585
586 if (mapKeys.count(vchDefaultKey))
587 {
588 // Set keyUser
589         keyUser.SetPubKey(vchDefaultKey);
590         keyUser.SetPrivKey(mapKeys[vchDefaultKey]);
591 }
592 else
593 {
594 // Create new keyUser and set as default key
595         keyUser.MakeNewKey();
596 if (!AddKey(keyUser))
597 return false;
598 if (!SetAddressBookName(PubKeyToAddress(keyUser.GetPubKey()), "Your Address"))
599 return false;
600 CWalletDB().WriteDefaultKey(keyUser.GetPubKey());
601 }
602
603 return true;
604 }
```

605

SUMMARY

`db.cpp` is Bitcoin's durability layer — the bridge between volatile in-memory state and a restart-safe disk representation. Every block validated, every transaction received, every key generated is ultimately written here. Satoshi's choice of Berkeley DB was pragmatic: a proven embedded database with ACID guarantees, requiring no infrastructure beyond the files in the application directory.

The BIP 50 incident is a reminder that consensus is deeper than code.[2] BDB's lock limit — a tunable constant set to 10,000 at line 66 — became an invisible consensus rule. Any node configured differently would diverge on the same block. Bitcoin's migration to LevelDB for the chain index, while retaining BDB for wallet files for another decade, reflects the cost of that lesson: backward compatibility with `wallet.dat` files kept BDB alive until Bitcoin Core 30 (2025) finally removed it, leaving only a built-in reader so old wallets can still be migrated.[3][4]

KEY TERMS

database environment

The single global DbEnv, opened lazily by the first CDB constructor in the application directory, that every database file shares for locking, logging, caching, and transactions.

write-ahead log

Berkeley DB's record of every change, kept in the database subdirectory and written before the data files are updated, so interrupted work can be replayed or rolled back after a crash.

DB_RECOVER

The flag passed when opening the database environment that makes Berkeley DB run recovery from its log first, restoring the files to a consistent state after an unclean shutdown.

checkpoint

A txn_checkpoint call that flushes logged changes into the data files; v0.1.0 checkpoints whenever a database closes and in DBFlush, which also resets the log sequence numbers of idle files.

lock limit

The ceiling of 10,000 Berkeley DB locks and lock objects set with set_lk_max_locks and set_lk_max_objects at startup, a tuning value later implicated in the chain split described in BIP 50.

CTxIndex

The record CTxDB stores under each confirmed transaction's hash: the transaction's position on disk plus one vSpent entry per output, noting where that output was later spent.

record type tag

The short string, such as "tx", "blockindex", "name", or "key", that begins every database key, grouping records by kind so cursors can range-scan them and LoadWallet can dispatch on them.

hashBestChain

The blkindex.dat record holding the hash of the current best-chain tip, rewritten as the chain advances and read by LoadBlockIndex at startup to restore pindexBest and nBestHeight.

SOURCES

1. Oracle Corporation, *Berkeley DB License* (2009). Sleepycat-era open-source linking requirements referenced in readme.txt.

2. Gavin Andresen, *BIP 50 — March 2013 Chain Fork Post-Mortem* (2013). Berkeley DB lock limit caused a consensus fork around block 225430. https://github.com/bitcoin/bips/blob/master/bip-0050.mediawiki
3. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
4. Bitcoin Core Contributors, *Bitcoin Core 30.0 release notes* (October 10, 2025). -datacarriersize default 100,000; legacy BDB wallets removed; Qt 6; bitcoin command. https://bitcoincore.org/en/releases/30.0/

db.h

421 lines CPP

THE PERSISTENCE LAYER

This header defines the database classes and interfaces used throughout Bitcoin. It declares how wallets, addresses, and blockchain data are structured and stored using Berkeley DB—a mature, embedded key-value database that provides ACID transactions, crash recovery, and concurrent access.

Satoshi chose Berkeley DB for its reliability and simplicity.[1] Unlike client-server databases (MySQL, PostgreSQL), Berkeley DB runs in-process with no network overhead. It's battle-tested in mission-critical systems and provides the transaction guarantees Bitcoin needs: if you crash mid-write, the database recovers to a consistent state on restart.

The database files created here are your Bitcoin node's permanent memory: `wallet.dat` holds your private keys and transaction history, `blkindex.dat` indexes the blockchain for fast lookup, and `addr.dat` remembers peer addresses for reconnection. Losing `wallet.dat` means losing your coins—this file IS your money.

The class hierarchy (CDB as base, specialized subclasses for each file) provides type-safe access patterns. You can't accidentally write a transaction to the address database—the type system prevents it. This defensive design has helped prevent data corruption bugs.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

#include <db_cxx.h>
class CTransaction;
class CTxIndex;
class CDiskBlockIndex;
class CDiskTxPos;
class COutPoint;
class CUser;
class CReview;
class CAddress;
class CWalletTx;

```

`mapAddressBook`—your address labels, mapping Bitcoin addresses to human-readable names. When you label an address as 'Coinbase', 'Salary', etc., this map stores it. Defined extern here, implemented elsewhere, and persisted via CWalletDB.

```
extern map<string, string> mapAddressBook;
extern bool fClient;


```

`dbenv`—the Berkeley DB environment, shared by all database files. The environment provides: transaction coordination across multiple databases, shared memory pool for caching, logging for crash recovery. All Bitcoin databases open within this single environment, enabling atomic operations that span multiple files.

```
extern DbEnv dbenv;
extern void DBFlush(bool fShutdown);




```

`CDB`—the base class for all database operations. Wraps a Berkeley DB handle (`Db* pdb`) with C++ conveniences: RAII-style open/close, template methods for type-safe read/write, transaction support. All the specialized database classes (`CTxDB`, `CWalletDB`, etc.) inherit from this, getting consistent error handling and resource management.

```
class CDB
{
protected:
    Db* pdb;
    string strFile;
    vector<DbTxn*> vTxn;

```

Constructor takes filename and mode (`'r'`, `'r+'`, `'w'`). The `fTxn` parameter enables transactional writes—slower but crash-safe. For reading, transactions aren't needed. For critical writes (new keys, spent coins), transactions ensure atomicity: either the entire write succeeds, or nothing changes.

```
explicit CDB(const char* pszFile, const char* pszMode="r+", bool fTxn=false);
~CDB() { Close(); }
public:
void Close();
```

```
37 private:
38 CDB(const CDB&);
39 void operator=(const CDB&);
40
41 protected:
```

`Read()`—generic key-value read. Takes any serializable key type `K` and value type `T`. The key is serialized to bytes, looked up in Berkeley DB, and the value is deserialized back into type `T`. The template magic means you can write `Read(txid, transaction)` for any type. The compiler checks that `K` and `T` are serializable, but it cannot know what bytes are actually stored; reading with the wrong type fails at runtime.

```
42 template<typename K, typename T>
43 bool Read(const K& key, T& value)
44 {
45 if (!pdb)
46 return false;
47
48 // Key
49         CDataStream ssKey(SER_DISK);
50         ssKey.reserve(1000);
51         ssKey << key;
52         Dbt datKey(&ssKey[0], ssKey.size());
53
54 // Read
55         Dbt datValue;
56         datValue.set_flags(DB_DBT_MALLOC);
57 int ret = pdb->get(GetTxn(), &datKey, &datValue, 0);
```

Memory security: `memset(datKey.get_data(), 0, datKey.get_size())`. After reading, the key and value buffers are zeroed. This prevents sensitive data (like private keys) from lingering in freed memory. An attacker with memory access couldn't find keys in deallocated buffers. This paranoid cleanup appears throughout Bitcoin's key-handling code.

```
58 memset(datKey.get_data(), 0, datKey.get_size());
59 if (datValue.get_data() == NULL)
60 return false;
61
62 // Unserialize value
63         CDataStream ssValue((char*)datValue.get_data(), (char*)datValue.get_data() +
   datValue.get_size(), SER_DISK);
64         ssValue >> value;
65
66 // Clear and free memory
67 memset(datValue.get_data(), 0, datValue.get_size());
68 free(datValue.get_data());
69 return (ret == 0);
70 }
71
```

`Write()`—generic key-value write. Serializes key and value, writes to database. The `fOverwrite` parameter controls behavior if key exists: `true` = replace, `false` = fail with `DB_NOOVERWRITE`. The comment 'Clear memory in case it was a private key' explains the final `memset()`—defense in depth against memory snooping.

```
72 template<typename K, typename T>
73 bool Write(const K& key, const T& value, bool fOverwrite=true)
74 {
75 if (!pdb)
76 return false;
77
78 // Key
```

```
        CDataStream ssKey(SER_DISK);
        ssKey.reserve(1000);
        ssKey << key;
        Dbt datKey(&ssKey[0], ssKey.size());

// Value
        CDataStream ssValue(SER_DISK);
        ssValue.reserve(10000);
        ssValue << value;
        Dbt datValue(&ssValue[0], ssValue.size());

// Write
```

Berkeley DB flags: `DB_NOOVERWRITE` causes writes to fail if the key exists. Used when adding new keys—you don't want to accidentally overwrite an existing key with a different value. The return code distinguishes 'success' from 'key existed', letting callers handle each case appropriately.

```
int ret = pdb->put(GetTxn(), &datKey, &datValue, (fOverwrite ? 0 : DB_NOOVERWRITE));

// Clear memory in case it was a private key
memset(datKey.get_data(), 0, datKey.get_size());
memset(datValue.get_data(), 0, datValue.get_size());
return (ret == 0);
}

```

`Erase()`—removes a key-value pair. Used, for example, when a reorganization disconnects a block's transactions, or when a wallet transaction is deleted. Returns `true` even if key didn't exist (`DB_NOTFOUND` is not an error for deletion). The key is zeroed after use for security.

```
template<typename K>
bool Erase(const K& key)
{
if (!pdb)
return false;

// Key
        CDataStream ssKey(SER_DISK);
        ssKey.reserve(1000);
        ssKey << key;
        Dbt datKey(&ssKey[0], ssKey.size());

// Erase
int ret = pdb->del(GetTxn(), &datKey, 0);

// Clear memory
memset(datKey.get_data(), 0, datKey.get_size());
return (ret == 0 || ret == DB_NOTFOUND);
}

template<typename K>
bool Exists(const K& key)
{
if (!pdb)
return false;

// Key
        CDataStream ssKey(SER_DISK);
        ssKey.reserve(1000);
        ssKey << key;
        Dbt datKey(&ssKey[0], ssKey.size());

// Exists
int ret = pdb->exists(GetTxn(), &datKey, 0);

// Clear memory
```

```
135 memset(datKey.get_data(), 0, datKey.get_size());
136 return (ret == 0);
137 }
138
```

`GetCursor()`—creates a database cursor for iteration. Cursors allow walking through all key-value pairs, essential for operations like 'load all wallet records' or 'find all transactions paying an address'. The cursor maintains position across multiple reads, enabling efficient sequential access.

```
139     Dbc* GetCursor()
140 {
141 if (!pdb)
142 return NULL;
143         Dbc* pcursor = NULL;
144 int ret = pdb->cursor(NULL, &pcursor, 0);
145 if (ret != 0)
146 return NULL;
147 return pcursor;
148 }
149
```

`ReadAtCursor()`—reads the next record via cursor. The flags parameter controls behavior: `DB_NEXT` for sequential scan, `DB_SET` for exact key lookup, `DB_SET_RANGE` for 'find first key >= this'. Returns key and value as `CDataStream`s for flexible deserialization. Used during wallet loading to iterate through all stored transactions.

```
150 int ReadAtCursor(Dbc* pcursor, CDataStream& ssKey, CDataStream& ssValue, unsigned int fFlags=DB_NEXT)
151 {
152 // Read at cursor
153         Dbt datKey;
154 if (fFlags == DB_SET || fFlags == DB_SET_RANGE || fFlags == DB_GET_BOTH || fFlags == DB_GET_BOTH_RANGE)
155 {
156             datKey.set_data(&ssKey[0]);
157             datKey.set_size(ssKey.size());
158 }
159         Dbt datValue;
160 if (fFlags == DB_GET_BOTH || fFlags == DB_GET_BOTH_RANGE)
161 {
162             datValue.set_data(&ssValue[0]);
163             datValue.set_size(ssValue.size());
164 }
165         datKey.set_flags(DB_DBT_MALLOC);
166         datValue.set_flags(DB_DBT_MALLOC);
167 int ret = pcursor->get(&datKey, &datValue, fFlags);
168 if (ret != 0)
169 return ret;
170 else if (datKey.get_data() == NULL || datValue.get_data() == NULL)
171 return 99999;
172
173 // Convert to streams
174         ssKey.SetType(SER_DISK);
175         ssKey.clear();
176         ssKey.write((char*)datKey.get_data(), datKey.get_size());
177         ssValue.SetType(SER_DISK);
178         ssValue.clear();
179         ssValue.write((char*)datValue.get_data(), datValue.get_size());
180
181 // Clear and free memory
182 memset(datKey.get_data(), 0, datKey.get_size());
183 memset(datValue.get_data(), 0, datValue.get_size());
184 free(datKey.get_data());
185 free(datValue.get_data());
186 return 0;
187 }
188
```

`GetTxn()`—returns current transaction, if any. Berkeley DB transactions provide atomicity: all writes within a transaction succeed or fail together. The `vTxn` vector supports nested transactions—you can start a sub-transaction within a transaction. Not heavily used in v0.1.0, but the infrastructure exists.

```
189     DbTxn* GetTxn()
190 {
191 if (!vTxn.empty())
192 return vTxn.back();
193 else
194 return NULL;
195 }
196
197 public:
```

Transaction methods: `TxnBegin()`, `TxnCommit()`, `TxnAbort()`. These wrap Berkeley DB's transaction API. Begin starts a transaction. Commit makes its changes permanent. Abort discards them, restoring the state from before Begin. If Bitcoin crashes between Begin and Commit, recovery automatically aborts the incomplete transaction—no corrupted state.

```
198 bool TxnBegin()
199 {
200 if (!pdb)
201 return false;
202         DbTxn* ptxn = NULL;
203 int ret = dbenv.txn_begin(GetTxn(), &ptxn, 0);
204 if (!ptxn || ret != 0)
205 return false;
206         vTxn.push_back(ptxn);
207 return true;
208 }
209
210 bool TxnCommit()
211 {
212 if (!pdb)
213 return false;
214 if (vTxn.empty())
215 return false;
216 int ret = vTxn.back()->commit(0);
217         vTxn.pop_back();
218 return (ret == 0);
219 }
220
221 bool TxnAbort()
222 {
223 if (!pdb)
224 return false;
225 if (vTxn.empty())
226 return false;
227 int ret = vTxn.back()->abort();
228         vTxn.pop_back();
229 return (ret == 0);
230 }
231
```

`ReadVersion()`/`WriteVersion()`—database schema versioning. As Bitcoin evolves, database formats may change. The version number lets future code detect old formats and migrate them. v0.1.0 writes it but never calls `ReadVersion()`—backward-compatibility planning from day one, left for later releases to use.

```
232 bool ReadVersion(int& nVersion)
```

```
{
        nVersion = 0;
return Read(string("version"), nVersion);
}

bool WriteVersion(int nVersion)
{
return Write(string("version"), nVersion);
}
};
```

`CTxDB`—the blockchain index database (`blkindex.dat`). Maps transaction hashes to their location on disk, block hashes to block metadata, and tracks the best chain. This index is what makes blockchain queries fast: rather than scanning every block file, look up the index. If `fClient` is `true` (lightweight mode), this database isn't used.

```
class CTxDB : public CDB
{
public:
CTxDB(const char* pszMode="r+", bool fTxn=false) : CDB(!fClient ? "blkindex.dat" : NULL, pszMode, fTxn)
{ }
private:
CTxDB(const CTxDB&);
void operator=(const CTxDB&);
public:
```

`CTxDB` methods: `ReadTxIndex()`, `UpdateTxIndex()`, `AddTxIndex()`, `EraseTxIndex()`. These maintain the transaction index: one record per confirmed transaction, holding its disk position and a `vSpent` slot per output. `AddTxIndex()` records a newly confirmed transaction with every slot empty (unspent); spending an output fills its slot (`UpdateTxIndex()`); `EraseTxIndex()` runs only when a block is disconnected during a reorganization. This is v0.1.0's notion of 'who owns what'—the separate UTXO-set database came with v0.8 (2013).

```
bool ReadTxIndex(uint256 hash, CTxIndex& txindex);
bool UpdateTxIndex(uint256 hash, const CTxIndex& txindex);
bool AddTxIndex(const CTransaction& tx, const CDiskTxPos& pos, int nHeight);
bool EraseTxIndex(const CTransaction& tx);
bool ContainsTx(uint256 hash);
bool ReadOwnerTxes(uint160 hash160, int nHeight, vector<CTransaction>& vtx);
bool ReadDiskTx(uint256 hash, CTransaction& tx, CTxIndex& txindex);
bool ReadDiskTx(uint256 hash, CTransaction& tx);
bool ReadDiskTx(COutPoint outpoint, CTransaction& tx, CTxIndex& txindex);
bool ReadDiskTx(COutPoint outpoint, CTransaction& tx);
```

`WriteBlockIndex()`/`EraseBlockIndex()`—maintain the block chain structure. Each `CDiskBlockIndex` records: block hash, previous block hash, height, timestamp, difficulty. This lets the node quickly determine if a block is in the best chain, find a block's ancestors, and identify chain reorganizations.

```
bool WriteBlockIndex(const CDiskBlockIndex& blockindex);
bool EraseBlockIndex(uint256 hash);
```

`ReadHashBestChain()`—retrieves the current chain tip. On startup, the node reads this to know which chain to continue building on. It's the single most important value in the database: 'what does this node consider the canonical blockchain history?'

```
271 bool ReadHashBestChain(uint256& hashBestChain);
272 bool WriteHashBestChain(uint256 hashBestChain);
273 bool LoadBlockIndex();
274 };
275
276
277
278
279
```

`CReviewDB`—Satoshi's marketplace feature. Stores user reviews and ratings in `reviews.dat`. This hints at a more ambitious vision where Bitcoin supported reputation systems for commerce. The feature was never fully developed and was removed in later versions,[2] but the infrastructure shows Satoshi thinking about trust in pseudonymous commerce.

```
280 class CReviewDB : public CDB
281 {
282 public:
283 CReviewDB(const char* pszMode="r+", bool fTxn=false) : CDB("reviews.dat", pszMode, fTxn) { }
284 private:
285 CReviewDB(const CReviewDB&);
286 void operator=(const CReviewDB&);
287 public:
288 bool ReadUser(uint256 hash, CUser& user)
289 {
290 return Read(make_pair(string("user"), hash), user);
291 }
292
293 bool WriteUser(uint256 hash, const CUser& user)
294 {
295 return Write(make_pair(string("user"), hash), user);
296 }
297
298 bool ReadReviews(uint256 hash, vector<CReview>& vReviews);
299 bool WriteReviews(uint256 hash, const vector<CReview>& vReviews);
300 };
301
302
303
304
305
```

`CMarketDB`—another marketplace remnant. Would have stored product listings in `market.dat`. Combined with CReviewDB, this would have enabled decentralized e-commerce: list products, accumulate reviews, transact in bitcoin. The vision never materialized, but these database stubs remain as archaeological evidence.

```
306 class CMarketDB : public CDB
307 {
308 public:
309 CMarketDB(const char* pszMode="r+", bool fTxn=false) : CDB("market.dat", pszMode, fTxn) { }
310 private:
311 CMarketDB(const CMarketDB&);
312 void operator=(const CMarketDB&);
313 };
314
315
316
```

```
317
318
```

`CAddrDB`—peer address storage (`addr.dat`). Remembers IP addresses of other Bitcoin nodes. On startup, the node loads these addresses and attempts reconnection. This provides persistence across restarts: you don't need to rediscover the network from scratch each time. Combined with IRC bootstrap and address gossip, this ensures network resilience.

```
319 class CAddrDB : public CDB
320 {
321 public:
322 CAddrDB(const char* pszMode="r+", bool fTxn=false) : CDB("addr.dat", pszMode, fTxn) { }
323 private:
324 CAddrDB(const CAddrDB&);
325 void operator=(const CAddrDB&);
326 public:
327 bool WriteAddress(const CAddress& addr);
328 bool LoadAddresses();
329 };
330
331 bool LoadAddresses();
332
333
334
335
336
```

`CWalletDB`—your money (`wallet.dat`). This is the most security-critical database. Stores: private keys (your ability to spend), transaction history (your record of what you own), address labels (your organizational metadata), and settings. Losing this file with no backup = permanent loss of all your bitcoin. There is no recovery mechanism.

```
337 class CWalletDB : public CDB
338 {
339 public:
340 CWalletDB(const char* pszMode="r+", bool fTxn=false) : CDB("wallet.dat", pszMode, fTxn) { }
341 private:
342 CWalletDB(const CWalletDB&);
343 void operator=(const CWalletDB&);
344 public:
```

`ReadName()`/`WriteName()`—address book operations. The key is the address string; the value is your label. `WriteName` also updates the in-memory `mapAddressBook` for fast access. These let you associate human meaning with cryptographic addresses: '1ABC...' becomes 'Mom's birthday gift.'

```
345 bool ReadName(const string& strAddress, string& strName)
346 {
347         strName = "";
348 return Read(make_pair(string("name"), strAddress), strName);
349 }
350
351 bool WriteName(const string& strAddress, const string& strName)
352 {
353         mapAddressBook[strAddress] = strName;
354 return Write(make_pair(string("name"), strAddress), strName);
355 }
356
357 bool EraseName(const string& strAddress)
358 {
359         mapAddressBook.erase(strAddress);
360 return Erase(make_pair(string("name"), strAddress));
361 }
362
```

`ReadTx()`/`WriteTx()`/`EraseTx()`—transaction storage. Every transaction involving your addresses is stored here: payments received, payments sent, change outputs. The key is the transaction hash (txid); the value is the full CWalletTx (transaction plus wallet-specific metadata like 'from address book entry').

```
363 bool ReadTx(uint256 hash, CWalletTx& wtx)
364 {
365 return Read(make_pair(string("tx"), hash), wtx);
366 }
367
368 bool WriteTx(uint256 hash, const CWalletTx& wtx)
369 {
370 return Write(make_pair(string("tx"), hash), wtx);
371 }
372
373 bool EraseTx(uint256 hash)
374 {
375 return Erase(make_pair(string("tx"), hash));
376 }
377
```

`ReadKey()`/`WriteKey()`—private key storage. This is where the actual secrets live. The key is the public key bytes; the value is the private key itself (a DER-encoded `CPrivKey`), unencrypted; wallet encryption arrived only in v0.4 (2011). Note `WriteKey()` uses `fOverwrite=false`—you never want to accidentally overwrite an existing key with a different value, as that would destroy the original key.

```
378 bool ReadKey(const vector<unsigned char>& vchPubKey, CPrivKey& vchPrivKey)
379 {
380         vchPrivKey.clear();
381 return Read(make_pair(string("key"), vchPubKey), vchPrivKey);
382 }
383
384 bool WriteKey(const vector<unsigned char>& vchPubKey, const CPrivKey& vchPrivKey)
385 {
386 return Write(make_pair(string("key"), vchPubKey), vchPrivKey, false);
387 }
388
```

`ReadDefaultKey()`/`WriteDefaultKey()`—the 'receiving address' shown in the UI. When someone wants to pay you, you give them this address. It's just one of your keys, designated as the default. You can change it anytime by calling WriteDefaultKey with a different key.

```
389 bool ReadDefaultKey(vector<unsigned char>& vchPubKey)
390 {
391         vchPubKey.clear();
392 return Read(string("defaultkey"), vchPubKey);
393 }
394
395 bool WriteDefaultKey(const vector<unsigned char>& vchPubKey)
396 {
397 return Write(string("defaultkey"), vchPubKey);
398 }
399
```

ReadSetting()/WriteSetting()—generic settings storage. User preferences (in v0.1.0: fGenerateBitcoins, nTransactionFee, and addrIncoming) are stored as key-value pairs with the 'setting' prefix. The template methods accept any serializable type, providing type-safe settings without proliferating ReadXxxSetting methods.

```
400 template<typename T>
401 bool ReadSetting(const string& strKey, T& value)
402 {
403 return Read(make_pair(string("setting"), strKey), value);
404 }
405
406 template<typename T>
407 bool WriteSetting(const string& strKey, const T& value)
408 {
409 return Write(make_pair(string("setting"), strKey), value);
410 }
411
```

LoadWallet()—the critical startup function. Reads wallet.dat, populating in-memory structures with all keys and transactions. Returns the default receiving address. If this function fails (a corrupted database, for example—wallet encryption did not yet exist), the node cannot access its funds. The vector<unsigned char> return is the default public key.

```
412 bool LoadWallet(vector<unsigned char>& vchDefaultKeyRet);
413 };
414
415 bool LoadWallet();
416
```

SetAddressBookName()—convenience wrapper for labeling addresses. Creates a temporary CWalletDB just to write one name, then closes it. This pattern appears throughout Bitcoin: open database, do one operation, close. It's simpler than managing long-lived database handles but has some performance overhead.

```
CALLED FROM LoadWallet db.cpp p. 130 · CSendDialog::OnButtonSend ui.cpp p. 497 ·
CYourAddressDialog::OnListEndLabelEdit ui.cpp p. 502 · CYourAddressDialog::OnButtonRename ui.cpp p. 502
· CYourAddressDialog::OnButtonNew ui.cpp p. 503 · CAddressBookDialog::OnListEndLabelEdit ui.cpp p. 504 ·
and 2 more
417 inline bool SetAddressBookName(const string& strAddress, const string& strName)
418 {
419 return CWalletDB().WriteName(strAddress, strName);
420 }
421
```

SUMMARY

The database layer is Bitcoin's long-term memory. Without it, every restart would mean losing your transaction history, forgetting peer addresses, and most critically, losing access to your private keys. The careful separation into multiple files (wallet.dat, blkindex.dat, addr.dat) isolates concerns: you can back up just wallet.dat to protect your funds, without including the multi-gigabyte blockchain index.

Berkeley DB's transactional guarantees are essential for financial software. If your computer crashes while Bitcoin is marking outputs as spent, you don't want to end up with inconsistent state where coins appear both spent and unspent. The transaction mechanism ensures atomicity: either the entire update succeeds, or the database rolls back to the previous consistent state.

The wallet.dat file deserves special attention. Unlike most application data, it's irreplaceable—there's no 'forgot password' recovery, no customer service to restore your account. The private keys stored here ARE your bitcoin. Satoshi's code treats this file with appropriate paranoia: memory wiping, careful writes, transaction protection. Modern wallets add encryption and hardware security modules, but the fundamental truth remains: guard your keys.

KEY TERMS

Berkeley DB

The embedded, in-process key-value database library that holds all of v0.1.0's persistent data, providing transactions, logging, and crash recovery without a separate database server.

CDB

The base class wrapping one Berkeley DB file, offering templated Read, Write, Erase, and Exists methods that serialize any key and value through a CDataStream before touching disk.

CTxDB

The CDB subclass for blkindex.dat, holding the transaction index, the block index, and the hash of the best chain; it opens no file at all when fClient is set.

CWalletDB

The CDB subclass for wallet.dat, storing address-book names, wallet transactions, unencrypted key pairs, the default key, and settings: the one file whose loss means lost coins.

CAddrDB

The CDB subclass for addr.dat, which saves every known peer address so a restarted node can reconnect without rediscovering the network from scratch.

database transaction

A Berkeley DB unit of work opened with TxnBegin and finished with TxnCommit or TxnAbort, so its writes either all take effect or none do; the vTxn stack lets them nest.

cursor

A Berkeley DB iterator obtained with GetCursor and advanced with ReadAtCursor, used to walk records in key order, such as every block-index entry or every wallet record at startup.

DB_NOOVERWRITE

The Berkeley DB flag Write passes when fOverwrite is false, making the write fail instead of replacing an existing key; WriteKey uses it so a stored private key is never overwritten.

SOURCES

1. Oracle Corporation, *Berkeley DB License* (2009). Sleepycat-era open-source linking requirements referenced in readme.txt.
2. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/

headers.h

72 lines CPP

THE GATHERING OF INCLUDES

Every C++ source file in Bitcoin v0.1.0 begins the same way: `#include "headers.h"`. This single file pulls in the entire dependency graph — the operating system, the cryptographic library, the GUI framework, the standard library, and every Bitcoin-specific header — in one carefully ordered block. It is the table of contents for everything Bitcoin needs to exist.

The technique is called a **precompiled header** (PCH). In 2009, compiling C++ was painfully slow — template-heavy libraries like Boost and wxWidgets could take minutes to parse on each compilation unit. By gathering all includes into a single file and compiling it once into a binary blob (`headers.h.gch` in the makefile), subsequent compilations skip re-parsing thousands of lines of library code. On Satoshi's machine, this likely cut build times from minutes to seconds. The `#pragma hdrstop` on line 52 marks the boundary for MSVC's precompiled header mechanism.

The include order is not arbitrary. It reveals Bitcoin's dependency layers from bottom to top: first the compiler workarounds and OS primitives, then the cryptographic foundation (OpenSSL), then the GUI (wxWidgets), then C standard libraries and C++ STL containers, then Boost utilities, and finally — after all external dependencies are established — Bitcoin's own headers in dependency order. Reading this file top to bottom is reading the architectural blueprint of the system.

This file also reveals that Bitcoin v0.1.0 was a *Windows-only* application.[1] The `windows.h`, `winsock2.h`, and `mswsock.h` includes are unconditional — there is no `#ifdef _WIN32` guard offering a Linux alternative. Satoshi developed and released Bitcoin exclusively for Windows. The Linux port came later, contributed by community developers. The entire network, for its first weeks of existence, ran on Windows machines.

The copyright line appears in every source file. Satoshi Nakamoto — a pseudonym never definitively linked to any real person — is the sole credited author. The MIT/X11 license reference points to `license.txt` in the same directory.[5]

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

```

MSVC pragma directives to suppress specific warnings. `4786` silences 'identifier truncated to 255 characters in debug info' — a notorious MSVC annoyance when using STL with long template names. `4804` suppresses 'unsafe use of type bool in operation'. `4717` suppresses 'recursive on all control paths, function will cause runtime stack overflow' — likely triggered by template metaprogramming in Boost. These pragmas reveal Satoshi was actively compiling with MSVC and dealing with its quirks.

```
#ifdef _MSC_VER
#pragma warning(disable:4786)
#pragma warning(disable:4804)
#pragma warning(disable:4717)
#endif
```

The `_WIN32_WINNT` macro is first undefined then set to `0x0400`, which corresponds to Windows NT 4.0. This sets the minimum Windows API version — Satoshi wanted Bitcoin to run on older Windows systems, not just the then-current Vista/XP. `WIN32_LEAN_AND_MEAN` excludes rarely-used Windows headers (Cryptography, DDE, RPC, Shell, Winsock 1.x) to speed up compilation.

```
#ifdef _WIN32_WINNT
#undef _WIN32_WINNT
#endif
#define _WIN32_WINNT 0x0400
#define WIN32_LEAN_AND_MEAN 1
```

wxWidgets — the cross-platform GUI framework.[6] `wx/wx.h` is the master include pulling in windows, dialogs, buttons, and text controls. `wx/clipbrd.h` adds clipboard support (for copying Bitcoin addresses). `wx/snglinst.h` provides single-instance checking to prevent running two Bitcoin nodes simultaneously on the same machine, which would corrupt the database.

```
#include <wx/wx.h>
#include <wx/clipbrd.h>
#include <wx/snglinst.h>
```

OpenSSL — the cryptographic foundation.[7] Five headers, five capabilities: `ecdsa.h` for elliptic curve signatures (transaction signing/verification), `evp.h` for the high-level digest interface, `rand.h` for cryptographically secure random number generation (key creation), `sha.h` for SHA-256 (block hashing, transaction IDs), and `ripemd.h` for RIPEMD-160 (Bitcoin address generation). These five algorithms are the entire cryptographic surface area of Bitcoin.

```
#include <openssl/ecdsa.h>
#include <openssl/evp.h>
#include <openssl/rand.h>
#include <openssl/sha.h>
#include <openssl/ripemd.h>
```

Windows-specific system headers. `windows.h` is the Win32 API master include. `winsock2.h` provides the Berkeley sockets API on Windows — every peer-to-peer network connection uses this. `mswsock.h` adds Microsoft socket extensions like `TransmitFile` and `ConnectEx`. The unconditional inclusion of these Windows headers — with no `#ifdef` for Unix alternatives — confirms this code was Windows-only.

```
#include <windows.h>
#include <winsock2.h>
#include <mswsock.h>
```

C standard library headers. A mix of I/O (`stdio.h`, `io.h`), memory management (`stdlib.h`, `malloc.h`), mathematics (`math.h`, `float.h`, `limits.h`), debugging (`assert.h`), and process management (`process.h` — Windows-specific, provides `_beginthread`). Satoshi used C headers rather than their C++ equivalents (`cstdio`, `cstdlib`) — a common practice in 2009-era C++ before modern style guides.

```
#include <stdio.h>
#include <stdlib.h>
#include <io.h>
#include <math.h>
#include <limits.h>
#include <float.h>
#include <assert.h>
#include <process.h>
#include <malloc.h>
#include <memory>
```

`BOUNDSCHECK` set to 1 was apparently meant to enable runtime bounds checking — but nothing anywhere in v0.1.0 references the macro. It is a dead switch: the safety feature it names was never wired in. Had it been live, it would trade performance for safety — out-of-bounds access throwing rather than silently corrupting memory. In a financial system, silent corruption is far worse than a crash.

```
#define BOUNDSCHECK 1
```

C++ Standard Library containers and algorithms. The STL provides the data structures Bitcoin is built on: `map` for the block index, the wallet, and the memory pool of unconfirmed transactions (v0.1.0 has no separate UTXO set), `vector` for transaction inputs/outputs, `set` for unique peer addresses, `deque` for message queues, `string` and `sstream` for serialization. These nine includes are the backbone of every data structure in the system.

```
#include <sstream>
#include <string>
#include <vector>
#include <list>
#include <deque>
#include <map>
#include <set>
#include <algorithm>
#include <numeric>
```

Boost — the 'standard library extension' of C++.[8] `BOOST_FOREACH` provides range-based for loops (years before C++11's range-for). `lexical_cast` converts between strings and numbers. `tuple` bundles multiple return values. `array` provides fixed-size arrays with bounds checking. Satoshi used Boost heavily — it was the idiomatic C++ choice in 2009, before C++11 absorbed many of its features into the standard.

```
#include <boost/foreach.hpp>
#include <boost/lexical_cast.hpp>
#include <boost/tuple/tuple.hpp>
#include <boost/tuple/tuple_comparison.hpp>
#include <boost/tuple/tuple_io.hpp>
#include <boost/array.hpp>
```

`#pragma hdrstop` — 'header stop'. This MSVC directive marks the end of the precompiled header region. Everything above this line is compiled once and cached; everything below is compiled fresh each time. GCC uses a different mechanism — it precompiles the entire `headers.h` into `headers.h.gch` (see the makefile). Either way, the result is the same: parse the expensive includes once, reuse the result across all compilation units.

```
#pragma hdrstop
```

`using namespace std` and `using namespace boost` — these directives import everything from both namespaces into global scope. Modern C++ style guides strongly discourage this (name collisions, ambiguous overloads), but in 2009 it was common practice. Every Bitcoin source file inherits these via the precompiled header, which means `vector`, `map`, `string` etc. are used without the `std::` prefix throughout the codebase.

```
using namespace std;
using namespace boost;


```

Bitcoin's own headers, included in strict dependency order. `serialize.h` first (binary serialization, needed by everything), then `uint256.h` (256-bit integers for hashes), `util.h` (logging, random numbers, time), `key.h` (ECDSA cryptography), `bignum.h` (arbitrary-precision arithmetic for scripts), `base58.h` (human-readable addresses), `script.h` (Bitcoin Script interpreter), `db.h` (Berkeley DB persistence), `net.h` (P2P networking), `irc.h` (IRC-based peer discovery), `main.h` (consensus logic), `market.h` (unused marketplace feature), and finally `uibase.h` / `ui.h` (GUI). This ordering is the dependency graph of the entire system.

```
#include "serialize.h"
#include "uint256.h"
#include "util.h"
#include "key.h"
#include "bignum.h"
#include "base58.h"
#include "script.h"
#include "db.h"
#include "net.h"
#include "irc.h"
#include "main.h"
```

`market.h` — a curious inclusion. This header declares an experimental peer-to-peer marketplace that Satoshi included but never completed. It allowed users to post buy/sell offers directly within the Bitcoin client. The feature was removed in later versions, but its presence here hints at Satoshi's broader vision: Bitcoin not just as money, but as a platform for decentralized commerce.

```
69 #include "market.h"
70 #include "uibase.h"
71 #include "ui.h"
72
```

SUMMARY

The precompiled header is a lens into Bitcoin's architecture. Four external dependencies — wxWidgets for the GUI, OpenSSL for cryptography, Berkeley DB for persistence, and Boost for C++ utilities — plus the Windows API form the foundation. On top of these, fourteen Bitcoin-specific headers implement everything from 256-bit arithmetic to peer-to-peer networking to the consensus rules themselves.

The Windows-only nature of this file is historically significant. When Satoshi released Bitcoin on January 9, 2009, it could only be compiled and run on Windows.[1] The Linux port required community effort — replacing `windows.h` with POSIX equivalents, `winsock2.h` with `sys/socket.h`, and `process.h` with `pthread.h`. The fact that Satoshi chose Windows as the initial platform reflects a pragmatic decision: most desktop users in 2009 ran Windows, and Satoshi wanted Bitcoin to be accessible to as many people as possible.

Today, Bitcoin Core has replaced nearly every dependency listed here — Boost, slimmed down but still present, is the holdout.[2] wxWidgets gave way to Qt.[3] Berkeley DB gave way to LevelDB for the block index (the wallet kept BDB for years).[4] The custom SHA-256 implementation was optimized with hardware intrinsics. Even the precompiled header pattern itself was eventually abandoned in favor of modern build systems. But the layered architecture visible in this include ordering — serialization, cryptography, scripting, storage, networking, consensus, UI — remains the fundamental structure of Bitcoin to this day.

KEY TERMS

precompiled header

A header whose parsed contents the compiler saves to disk once and reuses for every source file, sparing it from re-reading large library headers each time.

#pragma hdrstop

A compiler directive marking where the precompiled portion of a header ends; everything included above it can be compiled once and cached.

_WIN32_WINNT

A macro telling the Windows headers which minimum Windows version to target; headers.h sets it to 0×0400, the value for Windows NT 4.0.

Winsock

Microsoft's implementation of the Berkeley sockets interface, included through winsock2.h and started with WSAStartup before Bitcoin opens any network connection.

OpenSSL

The open-source cryptography library that supplies Bitcoin v0.1.0's elliptic-curve signatures, SHA-256, RIPEMD-160, and secure random numbers.

wxWidgets

The C++ GUI toolkit that draws Bitcoin's windows, dialogs, and controls; the makefiles link its 2.8 libraries, named wxmsw28 and wxbase28.

Boost

A widely used collection of C++ libraries; headers.h pulls in its foreach, lexical_cast, tuple, and array components.

SOURCES

1. Satoshi Nakamoto, *Bitcoin v0.1 released (Cryptography Mailing List / BitcoinTalk)* (January 9, 2009). Announced Windows-only for the first release. https://satoshi.nakamotoinstitute.org/emails/cryptography/15/
2. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
3. Qt Company / Nokia, *Qt licensed under LGPL* (January 2009). Enabled Bitcoin Core's later migration from wxWidgets to Qt. https://blog.qt.io/blog/2009/01/14/nokia-qt-lgpl/
4. Oracle Corporation, *Berkeley DB License* (2009). Sleepycat-era open-source linking requirements referenced in readme.txt.

5. Massachusetts Institute of Technology, *MIT License* (1988). License chosen for Bitcoin v0.1.0 (license.txt). https://opensource.org/licenses/MIT
6. wxWidgets Project, *wxWidgets Licence* (2005). https://www.wxwidgets.org/about/licence/
7. OpenSSL Project, *OpenSSL License / SSLeay License* (1998–2009). https://www.openssl.org/source/license-openssl-ssleay.txt
8. Boost.org, *Boost Software License 1.0* (2003). https://www.boost.org/LICENSE_1_0.txt

irc.cpp

266 lines CPP

FINDING FRIENDS IN THE DARK

How does a new Bitcoin node find other nodes to connect to? This is the "bootstrap problem" — a chicken-and-egg dilemma that every peer-to-peer network faces. Satoshi's clever solution: use IRC (Internet Relay Chat), a protocol from 1988 that was already running on well-known, stable servers worldwide. New nodes would connect to an IRC channel, announce themselves, and discover peers — all without any Bitcoin-specific infrastructure.

The mechanism is elegant in its simplicity: Bitcoin nodes connect to `chat.freenode.net`, join channel `#bitcoin`, and encode their IP address and port in their IRC nickname using Base58Check encoding. By requesting the WHO list, any node can instantly discover every other online node. No central Bitcoin server needed — just parasitic use of existing infrastructure. The entire bootstrap protocol is 265 lines of C++.

Satoshi on this design (BitcoinTalk, 2010): "We only needed IRC because nobody had a static IP. In the early days there were some steady supporters, but they all had pool-allocated IPs that change every few days. IRC was only intended as a temporary solution."[1] He added: "Bitcoin's built-in addr system is the main solution" and "We don't want to overstay our welcome. If we get too big...we'll get off."[1]

This code reveals Satoshi's pragmatism: rather than building custom infrastructure, he leveraged what already existed. IRC servers were free, reliable, and firewall-friendly (port 6667 was commonly allowed). The temporary solution worked perfectly for Bitcoin's first four years until the network was large enough that addr propagation alone sufficed for peer discovery. The IRC bootstrap was deprecated in v0.6.x and removed entirely in v0.8.2 (May 2013).[2]

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

#include "headers.h"
```

`#pragma pack(1)` and `struct ircaddr` — a tightly packed 6-byte structure holding an IP address (4 bytes) and port (2 bytes). The `#pragma pack(1)` directive eliminates padding between the `int` and `short` fields, ensuring the struct is exactly 6 bytes for binary serialization. Without packing, the compiler might insert 2 bytes of padding after `port`, corrupting the Base58 encoding.

```
#pragma pack(1)
struct ircaddr
{
int ip;
short port;
};
```

`EncodeAddress()` — converts a `CAddress` (Bitcoin's internal address representation) into an IRC-safe nickname. The IP and port are packed into the 6-byte `ircaddr` struct, then encoded with `EncodeBase58Check()` — the same encoding used for Bitcoin addresses. The result is prefixed with `'u'` to mark it as a valid node address. Six data bytes plus a four-byte checksum come out as about 14 Base58 characters, so a typical nickname is 15 characters long: the documentation address 203.0.113.5:8333, for example, becomes `uCQVSS4eS2kwNJj`. The 'u' prefix distinguishes real nodes from non-routable nodes (which get random 'x' prefixes).

```
CALLED FROM  ThreadIRCSeed p. 166
string EncodeAddress(const CAddress& addr)
{
struct ircaddr tmp;
    tmp.ip    = addr.ip;
    tmp.port  = addr.port;

    vector<unsigned char> vch(UBEGIN(tmp), UEND(tmp));
return string("u") + EncodeBase58Check(vch);
}
```

`DecodeAddress()` — the reverse operation. Strips the leading character, Base58Check-decodes the remainder, validates the length matches `ircaddr`, and reconstructs a `CAddress`. The `DecodeBase58Check()` call includes checksum validation — if someone joins `#bitcoin` with a malformed nickname, the decode fails gracefully and the node is ignored. This prevents corrupted addresses from entering the peer database.

```
CALLED FROM  ThreadIRCSeed p. 168
bool DecodeAddress(string str, CAddress& addr)
{
    vector<unsigned char> vch;
if (!DecodeBase58Check(str.substr(1), vch))
return false;

struct ircaddr tmp;
if (vch.size() != sizeof(tmp))
return false;
memcpy(&tmp, &vch[0], sizeof(tmp));
```

```
37
38      addr   = CAddress(tmp.ip, tmp.port);
39  return true;
40  }
41
42
43
44
45
46
```

Send() — raw IRC protocol sender. Writes a complete IRC command to the socket in a loop, handling partial sends (where send() may transmit fewer bytes than requested). The PONG filter on line 49 suppresses debug output for keepalive responses — without this, the log would be flooded with PING/PONG every 30 seconds. Note the absence of any encryption: all IRC traffic is plaintext. An eavesdropper on the network path could observe which IP addresses were running Bitcoin nodes.

CALLED FROM RecvLineIRC p. 164 · ThreadIRCSeed p. 166

```
47  static bool Send(SOCKET hSocket, const char* pszSend)
48  {
49  if (strstr(pszSend, "PONG") != pszSend)
50  printf("SENDING: %s\n", pszSend);
51  const char* psz = pszSend;
52  const char* pszEnd = psz + strlen(psz);
53  while (psz < pszEnd)
54  {
55  int ret = send(hSocket, psz, pszEnd - psz, 0);
56  if (ret < 0)
57  return false;
58          psz += ret;
59  }
60  return true;
61  }
62
```

RecvLine() — reads one line from the IRC socket, character by character. IRC uses \r\n line endings; this function returns on \r and skips \n. Reading one byte at a time with recv() is inefficient but simple — adequate for the low-bandwidth IRC protocol where messages arrive at human-conversation speed. The WSAGetLastError() call on line 89 reveals this was written for Windows first (Winsock API) — though that branch is dead code: the nBytes > 0 and nBytes <= 0 tests above it already cover every case, so a socket error is simply treated as a closed connection.

CALLED FROM RecvLineIRC p. 164 · GetMyExternalIP net.cpp p. 326

```
63  bool RecvLine(SOCKET hSocket, string& strLine)
64  {
65      strLine = "";
66      loop
67  {
68  char c;
69  int nBytes = recv(hSocket, &c, 1, 0);
70  if (nBytes > 0)
71  {
72  if (c == '\n')
73  continue;
74  if (c == '\r')
75  return true;
76              strLine += c;
77  }
78  else if (nBytes <= 0)
79  {
80  if (!strLine.empty())
81  return true;
```

```
// socket closed
printf("IRC socket closed\n");
return false;
}
else
{
// socket error
int nErr = WSAGetLastError();
if (nErr != WSAEMSGSIZE && nErr != WSAEINTR && nErr != WSAEINPROGRESS)
{
printf("IRC recv failed: %d\n", nErr);
return false;
}
}
}
}

```

`RecvLineIRC()` — IRC-aware line reader that automatically handles PING/PONG keepalive. When the IRC server sends `PING :server`, this function changes the 'I' to 'O' (line 112: `strLine[1] = 'O'`) and sends back `PONG :server`. This is the IRC keepalive protocol — if a client doesn't respond to PING within a timeout, the server disconnects it. The character substitution trick (`PING` to `PONG`) is a clever hack that avoids parsing the full PING message.

```
CALLED FROM RecvUntil p. 164 · ThreadIRCSeed p. 167
bool RecvLineIRC(SOCKET hSocket, string& strLine)
{
    loop
{
bool fRet = RecvLine(hSocket, strLine);
if (fRet)
{
if (fShutdown)
return false;
            vector<string> vWords;
ParseString(strLine, ' ', vWords);
if (vWords[0] == "PING")
{
                strLine[1] = 'O';
                strLine += '\r';
Send(hSocket, strLine.c_str());
continue;
}
}
return fRet;
}
}

```

`RecvUntil()` — waits for one of up to three target strings to appear in any IRC response line. Used during the connection handshake to wait for specific server responses. The `strLine.find()` comparison against `-1` (line 130) is technically comparing against `string::npos`, which is `(size_t)-1` — it works but is a style that modern C++ linters would flag.

```
CALLED FROM ThreadIRCSeed p. 165
bool RecvUntil(SOCKET hSocket, const char* psz1, const char* psz2=NULL, const char* psz3=NULL)
{
    loop
{
        string strLine;
if (!RecvLineIRC(hSocket, strLine))
return false;
printf("IRC %s\n", strLine.c_str());
if (psz1 && strLine.find(psz1) != -1)
```

```
131 return true;
132 if (psz2 && strLine.find(psz2) != -1)
133 return true;
134 if (psz3 && strLine.find(psz3) != -1)
135 return true;
136 }
137 }
138
139
140
141
```

`fRestartIRCSeed` — a global boolean flag used to signal the IRC thread to reconnect. When set to true, the inner `while` loop at line 182 exits and the outer `loop` restarts the entire IRC connection sequence — though nothing in v0.1.0 ever sets it; the mechanism exists for future use.

```
142 bool fRestartIRCSeed = false;
143
```

`ThreadIRCSeed()` — the main IRC bootstrap thread, running in a continuous outer loop. This is the entire peer discovery engine for Bitcoin v0.1.0. The function connects to `chat.freenode.net` on port 6667, authenticates with an encoded nickname, joins `#bitcoin`, requests the user list, and then monitors for new joins — all in a single 95-line function.

```
    CALLED FROM  main p. 169 · StartNode net.cpp p. 342
144 void ThreadIRCSeed(void* parg)
145 {
146     loop
147 {
```

`gethostbyname("chat.freenode.net")` — DNS resolution of the IRC server. In the v0.1.0 source code this reads `chat.freenode.net`, though later versions (from v0.3.0 in July 2010) added `irc.lfnet.org`.[2] Port 6667 is the standard IRC port — typically allowed through firewalls because IRC was widely used in 2009. Satoshi chose a well-known public IRC network rather than running a private server, so no server under his control was needed — though, as he later acknowledged, the IRC server itself was 'a single point of failure.'[1]

```
148 struct hostent* phostent = gethostbyname("chat.freenode.net");
149         CAddress addrConnect(*(u_long*)phostent->h_addr_list[0], htons(6667));
150
151         SOCKET hSocket;
```

`ConnectSocket()` — attempts a TCP connection to the IRC server. If this fails, the function simply prints `'IRC connect failed'` and returns — no retry, no fallback. Like every error path in this function, it exits the thread rather than restarting the outer loop. If IRC is completely unreachable, the node must fall back on the addr message protocol and any peer addresses saved from previous sessions.

```
152 if (!ConnectSocket(addrConnect, hSocket))
153 {
154 printf("IRC connect failed\n");
155 return;
156 }
157
158 if (!RecvUntil(hSocket, "Found your hostname", "using your IP address instead", "Couldn't look up your
    hostname"))
159 {
160 closesocket(hSocket);
```

```
161 return;
162 }
163
```

`EncodeAddress(addrLocalHost)` — encodes this node's own IP address into an IRC nickname. Line 166 checks `IsRoutable()`: if the node is behind NAT or has a private IP (10.x.x.x, 192.168.x.x), it cannot be reached by other nodes. In that case, line 167 generates a random nickname like `x749382561` — the 'x' prefix signals to other nodes that this address is not usable for incoming connections. The node can still *discover* peers via IRC even if it cannot be discovered itself.

```
164         string strMyName = EncodeAddress(addrLocalHost);
165
166 if (!addrLocalHost.IsRoutable())
167             strMyName = strprintf("x%u", GetRand(1000000000));
168
```

IRC authentication: `NICK` sets the nickname (the Base58-encoded address) and `USER` registers the connection. The USER command's `8` parameter sets the 'invisible' mode flag, reducing the node's visibility in global IRC queries. Satoshi was polite about IRC usage — minimizing the footprint on shared infrastructure.

```
169 Send(hSocket, strprintf("NICK %s\r", strMyName.c_str()).c_str());
170 Send(hSocket, strprintf("USER %s 8 * : %s\r", strMyName.c_str(), strMyName.c_str()).c_str());
171
```

`RecvUntil(hSocket, " 004 ")` — waits for IRC numeric reply 004 (`RPL_MYINFO`), which signals successful registration. IRC servers send a sequence of welcome messages (001-004) after a successful NICK/USER handshake. Waiting for 004 ensures the connection is fully established before proceeding. The `Sleep(500)` on line 177 adds a half-second delay — likely to avoid triggering IRC server flood protection that might disconnect clients sending commands too rapidly.

```
172 if (!RecvUntil(hSocket, " 004 "))
173 {
174 closesocket(hSocket);
175 return;
176 }
177 Sleep(500);
178
```

`JOIN #bitcoin` and `WHO #bitcoin` — the two commands that make the bootstrap work. `JOIN` subscribes to the channel (and announces this node's presence to everyone already there), while `WHO` requests the complete user list. Every user in the channel whose nickname starts with `'u'` is a Bitcoin node advertising its IP address. In the earliest days the entire network was visible here: Hal Finney's January 10, 2009 `debug.log` shows he discovered peers via this channel near block 49, when only two other nodes answered — both consistent with Satoshi.[35]

```
179 Send(hSocket, "JOIN #bitcoin\r");
180 Send(hSocket, "WHO #bitcoin\r");
181
```

The main receive loop: continuously reads IRC messages while the node is running. The loop exits on shutdown (`fShutdown`), socket error, or restart signal (`fRestartIRCSeed`). Lines beginning with `':'` are server or user messages; others are filtered out. The `ParseString()` call splits each line into words for command parsing.

```
182 while (!fRestartIRCSeed)
183 {
184             string strLine;
185 if (fShutdown || !RecvLineIRC(hSocket, strLine))
186 {
187 closesocket(hSocket);
188 return;
189 }
190 if (strLine.empty() || strLine[0] != ':')
191 continue;
192 printf("IRC %s\n", strLine.c_str());
193
194             vector<string> vWords;
195 ParseString(strLine, ' ', vWords);
196 if (vWords.size() < 2)
197 continue;
198
199 char pszName[10000];
200             pszName[0] = '\0';
201
```

WHO response parsing — IRC numeric reply 352 (`RPL_WHOREPLY`). The nickname is at index 7 in the response fields. The comment notes that this field is limited to 16 characters — and an encoded node address is 'u' plus about 14 Base58 characters, 15 in all, so it just fits. Satoshi chose index 7 over index 10 (the 'real name' field, which also carries the encoded name at full length) for consistency with JOIN messages.

```
202 if (vWords[1] == "352" && vWords.size() >= 8)
203 {
204 // index 7 is limited to 16 characters
205 // could get full length name at index 10, but would be different from join messages
206 strcpy(pszName, vWords[7].c_str());
207 printf("GOT WHO: [%s]  ", pszName);
208 }
209
```

JOIN message parsing — when a new node joins `#bitcoin`, its nickname is extracted from the IRC hostmask format `:username!username@host`. The `strchr()` call finds the `'!'` delimiter and truncates there to isolate the pure nickname. Every new JOIN is a real-time notification that a new Bitcoin node has come online.

```
210 if (vWords[1] == "JOIN")
211 {
212 // :username!username@50000007.F000000B.90000002.IP JOIN :#channelname
213 strcpy(pszName, vWords[0].c_str() + 1);
214 if (strchr(pszName, '!'))
215 *strchr(pszName, '!') = '\0';
216 printf("GOT JOIN: [%s]  ", pszName);
217 }
218
```

Address decoding and storage. If the nickname starts with `'u'` (a routable node), `DecodeAddress()` extracts the IP and port, then `AddAddress()` stores it in the persistent address database via `CAddrDB`. The `'new'` debug message distinguishes previously unknown peers from ones already in the database. Once stored, the address persists across restarts — the node only needs IRC for initial bootstrapping; subsequent runs can connect to remembered peers directly.

```
219 if (pszName[0] == 'u')
220 {
221                 CAddress addr;
222 if (DecodeAddress(pszName, addr))
223 {
224                     CAddrDB addrdb;
225 if (AddAddress(addrdb, addr))
226 printf("new  ");
227                     addr.print();
228 }
229 else
230 {
231 printf("decode failed\n");
232 }
233 }
234 }
235
```

`fRestartIRCSeed = false` — resets the restart flag before closing the socket and looping back to the top of `ThreadIRCSeed()`. The outer `loop` keyword (Satoshi's macro for `for(;;)`) would reconnect only if something set `fRestartIRCSeed` — and nothing in v0.1.0 does. A dropped connection, socket error, or failed handshake takes one of the `return` paths above instead, ending the thread; the node then relies on addr messages and its saved addresses.

```
236         fRestartIRCSeed = false;
237 closesocket(hSocket);
238 }
239 }
240
241
242
243
244
245
246
247
248
249
```

`#ifdef TEST` — a standalone test harness that initializes Winsock and calls `ThreadIRCSeed()` directly. This reveals Satoshi's testing methodology: he could compile `irc.cpp` as an independent program to debug the IRC bootstrap without running the full Bitcoin client. The `WSAStartup()` / `WSACleanup()` pair is the standard Windows socket initialization — further evidence of the Windows-first development approach.

```
250 #ifdef TEST
251 int main(int argc, char *argv[])
252 {
253     WSADATA wsadata;
254 if (WSAStartup(MAKEWORD(2,2), &wsadata) != NO_ERROR)
255 {
256 printf("Error at WSAStartup()\n");
257 return false;
258 }
```

```
259
260 ThreadIRCSeed(NULL);
261
262 WSACleanup();
263 return 0;
264 }
265 #endif
266
```

SUMMARY

The IRC bootstrap was deprecated in v0.6.x and removed entirely in v0.8.2 (May 2013).[2] When LFNet (one of the IRC servers used) eventually shut down, it temporarily broke peer discovery for old clients — validating Satoshi's foresight in building redundant discovery systems. The addr message protocol, which propagates peer addresses through the P2P network itself, proved to be the lasting solution. Modern Bitcoin Core uses DNS seeds (hardcoded domain names that resolve to known node IPs) as the primary bootstrap mechanism, with addr propagation for ongoing peer discovery.

Satoshi's choice of IRC reveals both pragmatism and humility. Rather than building a custom discovery protocol or running dedicated seed servers, he piggybacked on infrastructure that predated Bitcoin by two decades. The code is deliberately minimal — no IRC channel registration, no bot commands, no flood protection beyond the basic PING/PONG handler. When asked about alternatives on BitcoinTalk in June 2010, Satoshi weighed IRC (which "lists nodes that are currently online" but is "a single point of failure") against the addr system, and concluded: "The combination of the two gets us the best of both worlds and more total robustness."[1]

The security implications are worth noting. IRC traffic was unencrypted, meaning any network observer could identify Bitcoin node IP addresses simply by monitoring `#bitcoin` on Freenode. The Base58 encoding of nicknames was obfuscation, not encryption — anyone who understood the format could decode every address. This was acceptable in 2009 when Bitcoin had no monetary value and node operators had no reason to hide. As Bitcoin grew in value and attracted adversarial attention, the plaintext discovery mechanism became a liability — one of several reasons it was eventually removed.

That same transparency is why Hal Finney's crash report still matters: the IRC-derived peer list and block requests in his attached `debug.log` let later researchers reconstruct who was online when he first joined — contradicting the folklore that he was the second node after Satoshi at launch.[3][4]

KEY TERMS

ircaddr

The packed 6-byte struct, a 4-byte IPv4 address followed by a 2-byte port, whose Base58Check encoding becomes a node's IRC nickname.

encoded nickname

A nickname made of the letter u followed by the Base58Check encoding of an ircaddr, about 15 characters in all, from which any channel member can recover a node's IP address and port.

x nickname

The random name, x followed by a number below one billion, that a node uses when its own address is not routable, so it can listen in the channel without advertising an unreachable address.

PING/PONG

The IRC keepalive exchange: the server sends PING and expects PONG in reply, which RecvLineIRC produces by overwriting the I in the received line with an O and echoing it back.

WHO

The IRC command requesting a channel's member list; each answer arrives as numeric reply 352 (RPL_WHOREPLY), whose eighth word, index 7, carries a member's nickname.

JOIN

The IRC command for entering a channel; the server also relays each newcomer's JOIN to existing members, so a running node learns of newly started peers in real time.

RPL_MYINFO

IRC numeric reply 004, the last of the server's welcome messages; ThreadIRCSeed waits for it after sending NICK and USER before joining #bitcoin.

SOURCES

1. Satoshi Nakamoto, *BitcoinTalk — IRC channel for Bitcoin* (March 16 and June 14, 2010). IRC seeding described as a temporary solution. https://bitcointalk.org/index.php?topic=84.0
2. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
3. Hal Finney, *[bitcoin-list] Crash in bitcoin 0.1.0 (with debug.log)* (January 10, 2009). SourceForge post attaching Finney's debug.log; forensic evidence he joined near block 49. https://web.archive.org/web/20141130200234/https://sourceforge.net/p/bitcoin/mailman/message/21295694/

4. Alex Waltz, *Copy-Paste & LLM-friendly transcriptions* (2026). Plain-text transcriptions of primary sources used in the 1st Bitcoin Miner investigation. https://1stbitcoinminer.com/transcriptions/
5. Alex Waltz, *Who was the 1st Bitcoin Miner?* (2026). Forensic reconstruction of Bitcoin's first days from Finney's debug.log, emails, and ExtraNonce patterns. https://1stbitcoinminer.com/

irc.h

8 lines CPP

THE BOOTSTRAP PROTOCOL

Declares the IRC bootstrapping functionality—one of Bitcoin's most clever early design choices. When Bitcoin launches, it faces a chicken-and-egg problem: how do you find other Bitcoin nodes when you don't know any yet? Satoshi's solution: piggyback on existing infrastructure.

Bitcoin nodes join an IRC channel (`#bitcoin` on `chat.freenode.net` in this v0.1.0 tree; later builds also used LFNet) where their nicknames encode their IP addresses. By parsing the channel's user list, a new node discovers other participants. It's bootstrap via a communication medium that already had robust peer discovery: Internet Relay Chat.[1]

The elegance is in the simplicity. IRC servers handle the hard problem of maintaining user lists and handling churn. Bitcoin nodes just need to join a channel, advertise their presence via nickname, and harvest other nicknames. No central Bitcoin server needed—IRC provides the coordination layer for free.

This mechanism was deprecated and removed in later Bitcoin versions (replaced by DNS seeds and hardcoded peer lists),[2][3] but it served brilliantly in the early network when there were few nodes and no established infrastructure. The IRC bootstrap helped Bitcoin survive its fragile infancy.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

```

`RecvLine()`—reads one line from a socket. IRC is a line-based text protocol; each command ends with CRLF (\r\n). This function accumulates characters until it sees the carriage return (\r), skipping any \n, then returns the complete line. Used to parse IRC server responses — JOIN confirmations, NICK acknowledgments, WHO replies listing channel members — and, because it is exported here, also to read the HTTP reply in `GetMyExternalIP()` in net.cpp.

```
extern bool RecvLine(SOCKET hSocket, string& strLine);
```

`ThreadIRCSeed()`—the IRC bootstrap thread, runs continuously in background. Connects to `chat.freenode.net` port 6667, joins `#bitcoin`, sets a nickname encoding our IP address. Sends a WHO request to get the user list, decodes other users' nicknames to extract their IP addresses, and feeds those addresses into the peer discovery system. If the connection drops, the thread exits—there is no automatic reconnect in v0.1.0.

```
extern void ThreadIRCSeed(void* parg);
```

`fRestartIRCSeed`—restart flag for the IRC thread. Nothing in v0.1.0 ever sets it—the hook exists for future use. When set, the thread's inner receive loop exits and the outer loop reconnects. The bootstrap must be resilient—without peer discovery, a node is isolated and useless.

```
extern bool fRestartIRCSeed;

```

SUMMARY

The IRC bootstrap was a creative solution for the chicken-and-egg problem of peer discovery in a decentralized network. Rather than building custom infrastructure, Satoshi leveraged 20 years of IRC's evolution in handling dynamic user groups.

The nickname encoding scheme is particularly clever: each user's nick contains their IP address in an obfuscated format. Simply joining the channel and listing users reveals dozens of potential peers. No special protocol needed—just standard IRC commands that any IRC library supports.

The approach had real downsides: IRC servers could censor Bitcoin by banning the channel, and depending on specific IRC networks introduced centralization risk into an otherwise decentralized system. These concerns—along with LFNet's eventual shutdown—led to IRC bootstrap's removal. But in 2009, it was the pragmatic choice that got the network running quickly without custom infrastructure. Satoshi knew it was temporary, and he was right.[2]

KEY TERMS

IRC

Internet Relay Chat, the line-based text chat protocol whose public servers v0.1.0 borrows as a meeting point where Bitcoin nodes announce their addresses and find one another.

bootstrap problem

The puzzle every new peer-to-peer node faces on first launch: it must connect to existing peers, yet it knows no peer addresses until some peer tells it.

ThreadIRCSeed

The background thread StartNode launches to connect to chat.freenode.net on port 6667, join #bitcoin, advertise this node in its nickname, and harvest peer addresses from other nicknames.

#bitcoin

The IRC channel every v0.1.0 node joins at startup; its member list doubles as a live directory of nodes, since each reachable node's nickname encodes its IP address and port.

RecvLine

The helper that reads one line from a socket a byte at a time, stopping at the carriage return; exported here so net.cpp can reuse it for the HTTP reply in GetMyExternalIP.

fRestartIRCSeed

A global flag that, when set, makes ThreadIRCSeed drop its IRC connection and reconnect from the top; nothing in v0.1.0 ever sets it to true.

SOURCES

1. Alex Waltz, *Copy-Paste & LLM-friendly transcriptions* (2026). Plain-text transcriptions of primary sources used in the 1st Bitcoin Miner investigation. https://1stbitcoinminer.com/transcriptions/
2. Satoshi Nakamoto, *BitcoinTalk — IRC channel for Bitcoin* (March 16 and June 14, 2010). IRC seeding described as a temporary solution. https://bitcointalk.org/index.php?topic=84.0
3. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/

key.h

157 lines CPP

ELLIPTIC CURVE CRYPTOGRAPHY

This file is the mathematical core of Bitcoin ownership. Everything else in the system — transactions, blocks, the entire network — is ultimately protected by 156 lines of C++. If cryptography is the lock, `key.h` is the locksmith's shop.

Satoshi chose **elliptic curve cryptography (ECC)** on the `secp256k1` curve. In 2009 this was an unusual choice — nearly everyone used NIST's secp256r1. The Koblitz curve Satoshi picked was obscure, supported by few libraries, and lacked the institutional endorsement of the NIST alternatives. What it had instead was something more valuable: its parameters are *verifiably non-random*. The prime field and the curve coefficients `a=0, b=7` follow simple, predictable rules, leaving little room to hide a weakness. (The origin of its generator point was never documented, though the choice of generator does not affect the curve's security.)

The Snowden revelations of 2013 vindicated this paranoia. NSA documents showed the agency had deliberately weakened Dual_EC_DRBG, a NIST-blessed random number generator.[1] Security researchers immediately questioned whether NIST's elliptic curves had been similarly compromised. Bitcoin was immune — it had never used them.

A private key is a 256-bit random number. That is all it is. One of approximately 2^{256} possibilities — comparable to the estimated number of atoms in the observable universe. The public key and Bitcoin address are derived from it via one-way mathematics. Anyone who learns your private key can spend your coins instantly, irreversibly, from anywhere. There is no customer service. No chargeback. No recovery. This is the price of trustlessness.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.


```

Satoshi's rejected alternatives. These four commented-out blocks are a rare window into the design process. secp160k1 through secp224k1 were considered and rejected, presumably because their shorter key lengths offered insufficient security margins at the time. secp256k1 — the chosen curve — sits at the bottom of the list as the final answer.

```
// secp160k1
// const unsigned int PRIVATE_KEY_SIZE = 192;
// const unsigned int PUBLIC_KEY_SIZE  = 41;
// const unsigned int SIGNATURE_SIZE   = 48;
//
// secp192k1
// const unsigned int PRIVATE_KEY_SIZE = 222;
// const unsigned int PUBLIC_KEY_SIZE  = 49;
// const unsigned int SIGNATURE_SIZE   = 57;
//
// secp224k1
// const unsigned int PRIVATE_KEY_SIZE = 250;
// const unsigned int PUBLIC_KEY_SIZE  = 57;
// const unsigned int SIGNATURE_SIZE   = 66;
//
```

secp256k1 sizes: private key 279 bytes (OpenSSL's DER encoding), public key 65 bytes (uncompressed point: 04 prefix + 32-byte x + 32-byte y), signature up to 72 bytes (DER-encoded r and s). Like the other three blocks, these are comments — Satoshi's notes, not constants the code uses. The last line explains why he cared: Script's single-byte push opcodes carry at most 75 bytes, and both a 65-byte public key and a 72-byte signature plus its 1-byte hash type fit in one push. The key sizes were checked against the scripting language.

```
// secp256k1:
// const unsigned int PRIVATE_KEY_SIZE = 279;
// const unsigned int PUBLIC_KEY_SIZE  = 65;
// const unsigned int SIGNATURE_SIZE   = 72;
//
// see www.keylength.com
// script supports up to 75 for single byte push


```

key_error inherits std::runtime_error rather than returning error codes. In cryptographic code, an exception is safer than a return value that might be silently ignored. If a key operation fails and the caller doesn't check the return, Bitcoin might use an uninitialized key. An uncaught exception crashes the program loudly instead.

```
class key_error : public std::runtime_error
{
public:
explicit key_error(const std::string& str) : std::runtime_error(str) {}
};


// secure_allocator is defined is serialize.h
```

CPrivKey is a byte vector with a custom allocator. secure_allocator (defined in serialize.h) ensures that when the vector is destroyed, its memory is zeroed before being freed — preventing private key material from lingering in freed memory where it could be found later. (It does not lock pages in RAM, so a live key can still be swapped to disk.) This is why std::vector<unsigned char> is not used directly.

```
typedef vector<unsigned char, secure_allocator<unsigned char> > CPrivKey;


```

CKey is a thin RAII wrapper around OpenSSL's EC_KEY. The design is deliberate: Satoshi doesn't implement elliptic curve arithmetic directly — he delegates entirely to OpenSSL, inheriting both its correctness and its audited implementation. All key operations go through OpenSSL's API.

```
class CKey
{
protected:
    EC_KEY* pkey;

public:
CKey()
{
```

NID_secp256k1 is an OpenSSL numeric identifier. This single call pulls in everything: the prime field $p = 2^{256} - 2^{32} - 977$, the curve equation $y^2 = x^3 + 7$, the generator point G, and its order n. Satoshi didn't hardcode these constants — he relied on OpenSSL's built-in curve library, meaning Bitcoin's security depends on OpenSSL implementing the curve correctly.

```
        pkey = EC_KEY_new_by_curve_name(NID_secp256k1);
if (pkey == NULL)
throw key_error("CKey::CKey() : EC_KEY_new_by_curve_name failed");
}

```

Copy constructor uses EC_KEY_dup rather than copying the struct directly. EC_KEY contains internal OpenSSL state (reference counts, cached values) that can't be naively memcpy'd. Missing this distinction — using a raw copy instead of dup — would create two objects sharing the same memory, leading to a double-free when both are destroyed. A subtle bug that would be nearly impossible to detect in testing but catastrophic in production.

```
CKey(const CKey& b)
{
        pkey = EC_KEY_dup(b.pkey);
if (pkey == NULL)
throw key_error("CKey::CKey(const CKey&) : EC_KEY_dup failed");
}

    CKey& operator=(const CKey& b)
{
if (!EC_KEY_copy(pkey, b.pkey))
throw key_error("CKey::operator=(const CKey&) : EC_KEY_copy failed");
return (*this);
}

```

Destructor calls EC_KEY_free, which decrements OpenSSL's internal reference count and zeros key material. This is the RAII guarantee: the private key is securely erased the moment the CKey goes out of scope, even if an exception is thrown. Without this, key material would persist on the heap until the memory was reallocated and overwritten.

```
~CKey()
{
EC_KEY_free(pkey);
}

```

Key generation. EC_KEY_generate_key asks OpenSSL to generate a cryptographically secure random 256-bit private key and derive the corresponding public key. OpenSSL seeds its CSPRNG from the operating system entropy pool — /dev/urandom on Linux, CryptGenRandom on Windows — which aggregates CPU jitter, hardware interrupts, and other unpredictable sources. The entire security of every bitcoin created at this moment rests on these 256 bits being genuinely unpredictable. A 2013 Android flaw in SecureRandom caused thousands of wallets to generate weak keys; attackers systematically drained them.[4]

```
void MakeNewKey()
{
if (!EC_KEY_generate_key(pkey))
throw key_error("CKey::MakeNewKey() : EC_KEY_generate_key failed");
}

```

d2i_ECPrivateKey — 'd2i' is OpenSSL shorthand for 'DER to internal'. DER (Distinguished Encoding Rules) is the ASN.1 encoding OpenSSL uses for keys and for the signatures that appear inside scripts; the rest of Bitcoin's wire format is Satoshi's own (serialize.h). The private key is stored as an ASN.1 structure containing the raw key bytes plus curve parameters. This is why PRIVATE_KEY_SIZE is 279, not 32 — the DER wrapper adds metadata around the 32-byte key.

```
bool SetPrivKey(const CPrivKey& vchPrivKey)
{
const unsigned char* pbegin = &vchPrivKey[0];
if (!d2i_ECPrivateKey(&pkey, &pbegin, vchPrivKey.size()))
return false;
return true;
}

```

i2d_ECPrivateKey — 'i2d' is 'internal to DER'. Calling with a NULL output pointer is the OpenSSL idiom for querying the required buffer size before allocating. This two-pass pattern (size query, then serialize) avoids fixed-size buffers that might overflow or waste memory. The returned CPrivKey is what gets written to wallet.dat.

```
    CPrivKey GetPrivKey() const
{
unsigned int nSize = i2d_ECPrivateKey(pkey, NULL);
if (!nSize)
throw key_error("CKey::GetPrivKey() : i2d_ECPrivateKey failed");
        CPrivKey vchPrivKey(nSize, 0);
unsigned char* pbegin = &vchPrivKey[0];
if (i2d_ECPrivateKey(pkey, &pbegin) != nSize)
throw key_error("CKey::GetPrivKey() : i2d_ECPrivateKey returned unexpected size");
return vchPrivKey;
}
```

```
100
```

o2i_ECPublicKey — 'o2i' is 'octet string to internal'. Public keys in Bitcoin v0.1.0 use the uncompressed point format: a leading 0×04 byte followed by the 32-byte x-coordinate and 32-byte y-coordinate of the point P = privKey × G on the secp256k1 curve. Compressed keys (33 bytes, 0×02 or 0×03 prefix) were not yet standard — they arrived with Bitcoin Core 0.6 in 2012.

```
101 bool SetPubKey(const vector<unsigned char>& vchPubKey)
102 {
103 const unsigned char* pbegin = &vchPubKey[0];
104 if (!o2i_ECPublicKey(&pkey, &pbegin, vchPubKey.size()))
105 return false;
106 return true;
107 }
108
```

GetPubKey computes pubKey = privKey × G — scalar multiplication of the secp256k1 generator point. 'Multiplication' here is iterated point addition on the elliptic curve, not arithmetic multiplication. Computing it forward takes microseconds. The reverse — finding the private key scalar given the public point — is the elliptic curve discrete logarithm problem, believed to require approximately 2^{128} group operations. No algorithm better than this is known. The asymmetry is why Bitcoin works.

```
109     vector<unsigned char> GetPubKey() const
110 {
111 unsigned int nSize = i2o_ECPublicKey(pkey, NULL);
112 if (!nSize)
113 throw key_error("CKey::GetPubKey() : i2o_ECPublicKey failed");
114         vector<unsigned char> vchPubKey(nSize, 0);
115 unsigned char* pbegin = &vchPubKey[0];
116 if (i2o_ECPublicKey(pkey, &pbegin) != nSize)
117 throw key_error("CKey::GetPubKey() : i2o_ECPublicKey returned unexpected size");
118 return vchPubKey;
119 }
120
```

ECDSA_sign takes a 256-bit message hash (not the message itself) and produces a signature. Internally, OpenSSL generates a random nonce k, computes a curve point R = k × G, and sets r = R.x mod n. Then s = k^{-1}(hash + r·privKey) mod n. The signature is the pair (r, s), DER-encoded. The pchSig[10000] buffer is dramatically oversized for a max-72-byte signature — Satoshi was being defensive rather than querying the exact output length.

```
121 bool Sign(uint256 hash, vector<unsigned char>& vchSig)
122 {
123         vchSig.clear();
124 unsigned char pchSig[10000];
125 unsigned int nSize = 0;
```

Nonce reuse is catastrophic. If the same k is ever used to sign two different messages, the private key can be algebraically recovered from the two signatures: privKey = (s1·hash2 - s2·hash1) / (r·(s2-s1)) mod n. This is not theoretical — it has happened. Sony used a fixed k when signing PlayStation 3 firmware; fail0verflow exposed the flaw in December 2010,[5] and the PS3 signing key was published weeks later. Bitcoin's OpenSSL ECDSA always generates a fresh random k per signature.

```
126 if (!ECDSA_sign(0, (unsigned char*)&hash, sizeof(hash), pchSig, &nSize, pkey))
127 return false;
```

```
128         vchSig.resize(nSize);
129 memcpy(&vchSig[0], pchSig, nSize);
130 return true;
131 }
132
```

Verify takes the public key, message hash, and signature — and checks the math. The verifier computes u1 = s^{-1}·hash mod n and u2 = s^{-1}·r mod n, then confirms that (u1·G + u2·pubKey).x ≡ r mod n. This works because of the homomorphic structure of ECDSA. Crucially: the verifier never sees or needs the private key. Every node in the Bitcoin network independently verifies every signature, trusting mathematics instead of any authority.

```
133 bool Verify(uint256 hash, const vector<unsigned char>& vchSig)
134 {
135 // -1 = error, 0 = bad sig, 1 = good
136 if (ECDSA_verify(0, (unsigned char*)&hash, sizeof(hash), &vchSig[0], vchSig.size(), pkey) != 1)
137 return false;
138 return true;
139 }
140
```

Static convenience wrappers. The pattern — construct CKey, load key, call method, implicitly destroy — ensures the temporary key is securely erased when the stack unwinds. These are called during transaction validation (Verify) and wallet signing (Sign), where the caller only has a serialized key and doesn't need a persistent CKey object.

```
141 static bool Sign(const CPrivKey& vchPrivKey, uint256 hash, vector<unsigned char>& vchSig)
142 {
143         CKey key;
144 if (!key.SetPrivKey(vchPrivKey))
145 return false;
146 return key.Sign(hash, vchSig);
147 }
148
149 static bool Verify(const vector<unsigned char>& vchPubKey, uint256 hash, const vector<unsigned char>&
    vchSig)
150 {
151         CKey key;
152 if (!key.SetPubKey(vchPubKey))
153 return false;
154 return key.Verify(hash, vchSig);
155 }
156 };
157
```

SUMMARY

The CKey class is Satoshi's entire cryptographic interface—156 lines that abstract away elliptic curve group law, DER encoding, OpenSSL internals, and ECDSA arithmetic. The design principle is consistent delegation: do not implement cryptography yourself; use audited libraries and correct abstractions. This discipline—resisting the temptation to write clever code when a proven library exists—is arguably Satoshi's most important engineering decision.

The choice of secp256k1 has proven historically significant. Its verifiably non-random constants, combined with the post-Snowden distrust of NIST-blessed algorithms, meant that secp256k1 eventually became the most widely deployed elliptic curve on Earth—used by Ethereum, Litecoin, and hundreds of other cryptocurrencies, all following Satoshi's original selection. A curve nobody cared about in 2009 now protects more value than any other.

The one meaningful weakness in this implementation is the absence of deterministic nonce generation (RFC 6979, published in 2013).[2] Each signature requires a fresh random k value, and reusing k across two signatures leaks the private key algebraically. Deterministic ECDSA removes this dependency by deriving k from the private key and message hash, making signatures reproducible and eliminating the catastrophic nonce-reuse failure mode entirely. This was added to Bitcoin Core years after v0.1.0—one of the few genuine security improvements to the cryptographic layer.[3]

KEY TERMS

private key

A 256-bit random number that controls coins; anyone who learns it can spend the funds, so CKey zeroes it from memory on destruction.

public key

The 65-byte curve point (a 0×04 prefix plus x and y coordinates) derived from a private key, safe to publish and used to verify signatures.

ECDSA

The Elliptic Curve Digital Signature Algorithm, by which a private key signs a hash and anyone holding the public key can verify it without learning the key.

secp256k1

The particular elliptic curve Bitcoin signs on, chosen because its constants are verifiably non-random rather than taken from NIST.

DER

Distinguished Encoding Rules, the binary ASN.1 format wrapping private keys (279 bytes) and signatures (up to 72 bytes) for storage and transmission.

point multiplication

The one-way operation that derives a public key by adding the curve's generator point to itself private-key times; reversing it is the discrete logarithm problem.

nonce

The fresh random value k that each ECDSA signature consumes; reusing one across two signatures lets anyone algebraically recover the private key.

secure_allocator

The memory allocator used for key material that overwrites buffers with zeros before freeing them, so secrets do not linger in freed memory.

SOURCES

1. NIST / security research community, *Dual_EC_DRBG controversy* (2007–2013). Concerns about NIST curve parameter selection; relevant to secp256k1 vs NIST curves. https://projectbullrun.org/dual-ec/
2. Thomas Pornin, *RFC 6979 — Deterministic Usage of DSA and ECDSA* (August 2013). https://www.rfc-editor.org/rfc/rfc6979
3. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/

4. Bitcoin developers / Google, *Android SecureRandom ECDSA nonce reuse* (August 2013). Non-deterministic RNG led to private-key recovery from repeated r values. https://bitcoin.org/en/alert/2013-08-11-android
5. fail0verflow, *Console Hacking 2010 — PS3 ECDSA nonce reuse* (December 2010). Classic demonstration that reusing ECDSA nonces leaks private keys. https://events.ccc.de/congress/2010/Fahrplan/events/4087.en.html

license.txt

20 lines TEXT

FREEDOM IN CODE

Nineteen lines of legal text that shaped the future of money. Satoshi Nakamoto released Bitcoin under the **MIT License** — also known as the X11 License — one of the most permissive open-source licenses ever written.[1] This was not an accident. It was a deliberate architectural decision as important as any line of C++ in the system.

The MIT License grants virtually unlimited rights: anyone can use, copy, modify, merge, publish, distribute, sublicense, and sell copies of the software. The only requirement is preserving the copyright notice and license text. There is no copyleft obligation — unlike the GPL, derivative works can be made proprietary. This means companies can build closed-source products on top of Bitcoin without releasing their code. Exchanges like Coinbase, hardware wallet manufacturers like Ledger, and countless other businesses exist because this license allows it.

Satoshi could have chosen the GPL, which would have required all derivative works to also be open source. This might have slowed commercial adoption. He could have chosen no license at all, which would have made the legal status ambiguous and discouraged institutional use. Instead, the MIT License struck the ideal balance for a protocol that needed to be adopted as widely as possible, by as many types of actors as possible — individuals, corporations, governments — without legal friction. The protocol itself would prove hard to change — as Satoshi wrote on the BitcoinTalk forum: "The nature of Bitcoin is such that once version 0.1 was released, the core design was set in stone for the rest of its lifetime."[2] The code's legal status was just as durable: the MIT License ensured that even if Satoshi disappeared — as he eventually did[3] — no one could revoke the right to use, study, or modify the software.[1]

The copyright is assigned to "Satoshi Nakamoto" — a pseudonym. This raises an interesting legal question: can a pseudonymous entity hold copyright? In most jurisdictions, copyright vests in the creator regardless of the name used. But the

practical effect of using a pseudonym is that no one can enforce the copyright. Combined with the MIT License's permissiveness, Bitcoin is effectively in the public domain in all but name.

`Copyright (c) 2009 Satoshi Nakamoto` — the only attribution line. No company, no foundation, no institution. A single pseudonymous individual claiming authorship of a system that would grow to secure hundreds of billions of dollars. Under the Berne Convention, copyright exists automatically upon creation — no registration is required. The '(c)' is legally redundant but conventionally included.

```
Copyright (c) 2009 Satoshi Nakamoto

```

The grant clause — the heart of the MIT License. Eight verbs define what you may do: *use, copy, modify, merge, publish, distribute, sublicense*, and *sell*. The phrase 'without restriction' and 'without limitation' make this as broad as legally possible. 'Persons' includes corporations and other legal entities. 'Free of charge' means no royalties, no licensing fees, ever — the code is given away unconditionally.

```
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
```

The right to *sublicense* is particularly important. It means you can take Bitcoin's code, modify it, and pass your version on under terms of your own choosing — even proprietary ones — so long as Satoshi's notice and this permission text travel with the original portions. Litecoin, Dogecoin, and hundreds of other altcoins forked Bitcoin's code freely; most simply kept the MIT License. The GPL would have required every derivative to stay under the GPL.

```
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
```

'Subject to the following conditions' — there is exactly one condition in the MIT License, stated in the next paragraph: include the copyright notice and permission notice in all copies. That is it. No requirement to share modifications. No requirement to credit the author visibly. No restriction on commercial use. This single, minimal condition is what separates the MIT License from a pure public domain dedication.

```
furnished to do so, subject to the following conditions:

```

The attribution requirement. 'All copies or substantial portions' means you must include this license text if you redistribute the code or significant parts of it. In practice, most Bitcoin-derived projects include a `COPYING` or `LICENSE` file. This requirement is so lightweight that it is almost never a barrier to adoption — which is precisely why Satoshi chose it.

```
The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.

```

The warranty disclaimer — in ALL CAPS as required by the Uniform Commercial Code (UCC) in U.S. contract law. Conspicuous text is necessary for disclaimers to be legally enforceable. 'AS IS' means the software comes with no promises that it works correctly, is secure, or is fit for any particular purpose. Given that Bitcoin handles real money, this is a significant disclaimer. If a bug in the code causes you to lose your coins, you have no legal claim against the author.

```
13 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
14 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
```

'FITNESS FOR A PARTICULAR PURPOSE' — a legal term of art from the UCC. It means the author makes no promise that the software is suitable for any specific use case, including as a monetary system. 'NONINFRINGEMENT' disclaims liability if the code turns out to violate someone else's patents or copyrights. These are standard boilerplate, but their presence in the license of a financial system carries particular weight.

```
15 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
16 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
```

The liability limitation. 'IN NO EVENT SHALL THE AUTHORS... BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY' — this is a complete waiver of liability. Whether the theory is contract, tort, or otherwise, the author cannot be sued for damages arising from the software. Combined with the pseudonymous authorship, this creates a system where there is literally no one to sue if something goes wrong — a feature, not a bug, for a decentralized monetary system.

```
17 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
18 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
19 THE SOFTWARE.
20
```

SUMMARY

The MIT License is often overlooked in discussions of Bitcoin's design, but it is as fundamental as proof-of-work or the UTXO model.[1] By choosing the most permissive widely-used license, Satoshi ensured that Bitcoin could be freely adopted, forked, embedded, and commercialized by anyone. The entire cryptocurrency ecosystem — thousands of altcoins, hundreds of exchanges, countless wallets and tools — exists within the legal space created by these nineteen lines.

The combination of pseudonymous authorship and a permissive license created something unprecedented: software with no effective owner. No one can revoke the license. No one can demand royalties. No one can sue for copyright infringement in any meaningful way. Bitcoin's code belongs to everyone and no one — much like the protocol itself. This legal structure mirrors the technical architecture: just as no single node controls the network, no single entity controls the right to use the software.

It is worth noting what this license does *not* cover: the Bitcoin protocol itself. The license governs the code — this particular implementation. The protocol — the rules for valid blocks, transactions, and signatures — is an abstract specification that anyone can reimplement from scratch without any license at all. Multiple independent implementations (btcd in Go, bcoin in JavaScript, libbitcoin in C++) exist precisely because the protocol is unencumbered. Satoshi open-sourced not just the code, but the idea.

KEY TERMS

MIT License

A short permissive open-source license, also called the X11 license, that lets anyone use, modify, and sell the software if the notice is kept.

permissive license

An open-source license that imposes almost no conditions on reuse, allowing derived works to be distributed under any terms, including closed-source ones.

copyleft

A licensing approach, used by the GPL, that requires derived works to be distributed under the same open terms; the MIT License deliberately has none.

sublicense

The right to pass the software on to others under terms of one's own choosing, which the MIT License grants explicitly alongside copying and selling.

warranty disclaimer

The capitalized clause stating the software is provided "AS IS" and that its authors bear no liability for any damage arising from its use.

SOURCES

1. Massachusetts Institute of Technology, *MIT License* (1988). License chosen for Bitcoin v0.1.0 (license.txt). https://opensource.org/licenses/MIT
2. Satoshi Nakamoto, *BitcoinTalk — Re: Bitcoin 0.3.21* (June 17, 2010). On the core design being set in stone for the rest of its lifetime. https://bitcointalk.org/index.php?topic=195.msg1611#msg1611
3. Satoshi Nakamoto, *Last known email to Mike Hearn ("I've moved on to other things")* (April 23, 2011). "I've moved on to other things. It's in good hands with Gavin and everyone.". https://cointelegraph.com/news/satoshi-nakamoto-final-email-revisited-after-13-years

main.cpp

2661 lines CPP

WHERE CONSENSUS LIVES

This is the central nervous system of Bitcoin—2,660 lines of C++ that would eventually secure trillions of dollars. The `main.cpp` file contains the core logic that makes Bitcoin work: block creation, transaction processing, the proof-of-work consensus algorithm, and the rules that every node must follow.

Satoshi Nakamoto spent roughly two years developing this code in secrecy before the January 2009 release. As he put it on the cryptography mailing list in November 2008: "I had to write all the code before I could convince myself that I could solve every problem, then I wrote the paper."[1] Every design choice in this file reflects sustained thinking about game theory, cryptography, and distributed systems—the work of someone who understood that a single flaw would be fatal.

The file solves what cryptographers called the "Byzantine Generals Problem"—how can distributed nodes agree on truth without trusting each other?[2] Satoshi's insight was to make lying expensive: proof-of-work forces nodes to spend real energy, and the longest chain wins.[3] In this release that means greatest `nHeight`; selecting by cumulative work came later. As he wrote on the cryptography mailing list: "The proof-of-work chain is the solution to the synchronisation problem, and to knowing what the globally shared view is without having to trust anyone."[4]

What you are reading is essentially unchanged from Satoshi's original design. Later developers optimized performance and patched vulnerabilities, but the fundamental architecture—UTXO model, ten-minute blocks, difficulty adjustment, halving schedule, longest-chain tip selection—still follows this first release. Tip choice later moved from block height to cumulative work; the economic rules did not. More than seventeen years later, this is still the DNA of every block that gets mined.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

#include "headers.h"
#include "sha.h"




//
// Global state
//

```

cs_main — the global mutex. Bitcoin is heavily multi-threaded; this lock protects the blockchain state from race conditions during concurrent access by the network, mining, and UI threads.

```
CCriticalSection cs_main;

```

mapTransactions — the memory pool (mempool). Unconfirmed transactions wait here after passing validation, until a miner includes them in a block. nTransactionsUpdated is a generation counter; the mining thread polls it to detect when new transactions arrive and should be considered for the next block.

```
map<uint256, CTransaction> mapTransactions;
CCriticalSection cs_mapTransactions;
unsigned int nTransactionsUpdated = 0;
```

mapNextTx — the memory pool's spend index. For every output already spent by an unconfirmed transaction (COutPoint), it records which input spends it (CInPoint). It is not the UTXO set: confirmed spends are tracked on disk in each transaction's CTxIndex. Before a transaction enters the mempool, every input is checked here: if any input already appears in mapNextTx, another mempool transaction is already spending it — a mempool double-spend.

```
map<COutPoint, CInPoint> mapNextTx;

```

mapBlockIndex — the in-memory blockchain index. Maps every known block hash to its CBlockIndex (metadata + chain linkage). Loaded from BDB on startup by CTxDB::LoadBlockIndex(). Traversing the chain in either direction is O(1) per step because each CBlockIndex holds pointers to its predecessor (pprev) and successor (pnext).

```
map<uint256, CBlockIndex*> mapBlockIndex;
const uint256 hashGenesisBlock("0x000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f");
CBlockIndex* pindexGenesisBlock = NULL;
```

Chain tip state: nBestHeight is the height of the current best chain (0 = genesis), hashBestChain is its tip hash, and pindexBest is a direct pointer to the tip's CBlockIndex. These three values are updated together in AddToBlockIndex() (and set at startup when the index loads from disk). The mining thread reads pindexBest to build new block candidates.

```
int nBestHeight = -1;
uint256 hashBestChain = 0;
```

```
28 CBlockIndex* pindexBest = NULL;
29
```

Orphan blocks — blocks received before their parent. `mapOrphanBlocks` indexes them by their own hash; `mapOrphanBlocksByPrev` indexes them by their parent's hash. When a new block arrives and connects its parent, the node immediately checks `mapOrphanBlocksByPrev` to process any orphans that were waiting for it.

```
30 map<uint256, CBlock*> mapOrphanBlocks;
31 multimap<uint256, CBlock*> mapOrphanBlocksByPrev;
32
33 map<uint256, CDataStream*> mapOrphanTransactions;
34 multimap<uint256, CDataStream*> mapOrphanTransactionsByPrev;
35
```

`mapWallet` — your coins. Maps transaction hashes to `CWalletTx` (a `CTransaction` augmented with confirmation status, timestamps, and supporting transaction data). Every transaction you've sent or received lives here. `vWalletUpdated` signals the UI thread when the wallet changes.

```
36 map<uint256, CWalletTx> mapWallet;
37 vector<pair<uint256, bool> > vWalletUpdated;
38 CCriticalSection cs_mapWallet;
39
```

`mapKeys` — private key storage. Maps public key bytes to the matching private key. `mapPubKeys` maps the 160-bit hash of the public key (the Bitcoin address) back to the full public key — needed to construct spending scripts. Your entire Bitcoin wealth is controlled by what lives in these two maps.

```
40 map<vector<unsigned char>, CPrivKey> mapKeys;
41 map<uint160, vector<unsigned char> > mapPubKeys;
42 CCriticalSection cs_mapKeys;
```

`keyUser` — the default keypair. The public key is your main Bitcoin address; the private key lets you sign spending transactions. Set once during `LoadWallet()` from the `"defaultkey"` wallet record. Marketplace listings and reviews sign with this key's private key.

```
43 CKey keyUser;
44
45 string strSetDataDir;
46 int nDropMessagesTest = 0;
47
48 // Settings
```

`fGenerateBitcoins` — mining toggle. In v0.1.0, a typical consumer CPU could find a block in hours to days. Satoshi imagined 'one-CPU-one-vote' as the governance model before GPU mining (2010) and ASICs (2013) made CPU mining irrelevant.

```
49 int fGenerateBitcoins;
```

`nTransactionFee` — optional fee in satoshis. Initially zero. Satoshi treated fees as flood control rather than a toll: 'Free transactions are nice and we can keep it that way if people don't abuse them.'[14] The fee is deducted from the sending wallet's balance; miners collect accumulated fees via `GetBlockValue(nFees)`.

```
50 int64 nTransactionFee = 0;
51 CAddress addrIncoming;
52
53
54
55
56
57
58
59
60 //////////////////////////////////////////////////////////////////////////////
61 //
62 // mapKeys
63 //
64
```

`AddKey()` — updates the in-memory maps, then writes the keypair durably to `wallet.dat`. A crash between the two steps loses only a key that was never written — and so never handed out as an address — so restarting from disk is always safe.

```
   CALLED FROM  LoadWallet db.cpp p. 130 · GenerateNewKey p. 202 · BitcoinMiner p. 243
65 bool AddKey(const CKey& key)
66 {
67 CRITICAL_BLOCK(cs_mapKeys)
68 {
69         mapKeys[key.GetPubKey()] = key.GetPrivKey();
70         mapPubKeys[Hash160(key.GetPubKey())] = key.GetPubKey();
71 }
72 return CWalletDB().WriteKey(key.GetPubKey(), key.GetPrivKey());
73 }
74
```

`GenerateNewKey()` — creates a fresh ECDSA keypair on the secp256k1 curve using OpenSSL. The private key is 32 bytes of cryptographically secure random data. The security of every bitcoin stored at the resulting address depends entirely on this randomness — a weak RNG is catastrophic. Satoshi also called `RandAddSeed()` throughout the codebase to continuously feed entropy into OpenSSL's pool.

```
   CALLED FROM  ProcessMessage p. 236 · CYourAddressDialog::OnButtonNew ui.cpp p. 503
75 vector<unsigned char> GenerateNewKey()
76 {
77     CKey key;
78     key.MakeNewKey();
79 if (!AddKey(key))
80 throw runtime_error("GenerateNewKey() : AddKey failed\n");
81 return key.GetPubKey();
82 }
83
84
85
86
87 //////////////////////////////////////////////////////////////////////////////
88 //
89 // mapWallet
90 //
91
```

`AddToWallet()` — idempotent wallet update. If the transaction is new, records the receive time and writes to disk. If already present, merges in any new information: the confirming block hash, merkle branch position, `fFromMe` flag, or spent status. This merge logic handles the common case where a transaction is first seen in the mempool, then again when it confirms in a block — the second arrival updates confirmation data without duplicating the record.

CALLED FROM AddToWalletIfMine p. 204 · ProcessMessage p. 237 · CommitTransactionSpent p. 248

```
92 bool AddToWallet(const CWalletTx& wtxIn)
93 {
94     uint256 hash = wtxIn.GetHash();
95 CRITICAL_BLOCK(cs_mapWallet)
96 {
97 // Inserts only if not already there, returns tx inserted or tx found
98         pair<map<uint256, CWalletTx>::iterator, bool> ret = mapWallet.insert(make_pair(hash, wtxIn));
99         CWalletTx& wtx = (*ret.first).second;
100 bool fInsertedNew = ret.second;
101 if (fInsertedNew)
102             wtx.nTimeReceived = GetAdjustedTime();
103
104 //// debug print
105 printf("AddToWallet %s  %s\n", wtxIn.GetHash().ToString().substr(0,6).c_str(), fInsertedNew ? "new" :
    "update");
106
107 if (!fInsertedNew)
108 {
109 // Merge
110 bool fUpdated = false;
111 if (wtxIn.hashBlock != 0 && wtxIn.hashBlock != wtx.hashBlock)
112 {
113                 wtx.hashBlock = wtxIn.hashBlock;
114                 fUpdated = true;
115 }
116 if (wtxIn.nIndex != -1 && (wtxIn.vMerkleBranch != wtx.vMerkleBranch || wtxIn.nIndex != wtx.nIndex))
117 {
118                 wtx.vMerkleBranch = wtxIn.vMerkleBranch;
119                 wtx.nIndex = wtxIn.nIndex;
120                 fUpdated = true;
121 }
122 if (wtxIn.fFromMe && wtxIn.fFromMe != wtx.fFromMe)
123 {
124                 wtx.fFromMe = wtxIn.fFromMe;
125                 fUpdated = true;
126 }
127 if (wtxIn.fSpent && wtxIn.fSpent != wtx.fSpent)
128 {
129                 wtx.fSpent = wtxIn.fSpent;
130                 fUpdated = true;
131 }
132 if (!fUpdated)
133 return true;
134 }
135
136 // Write to disk
137 if (!wtx.WriteToDisk())
138 return false;
139
140 // Notify UI
141         vWalletUpdated.push_back(make_pair(hash, fInsertedNew));
142 }
143
144 // Refresh UI
145 MainFrameRepaint();
146 return true;
147 }
148
```

`AddToWalletIfMine()` — the filter between the network and your wallet. `tx.IsMine()` checks whether any output pays to a key you control. If yes, the transaction is wrapped in a `CWalletTx`, optionally given a merkle branch (if included in a block), and added to the wallet. All received funds flow through this function.

```
    CALLED FROM  CBlock::ConnectBlock p. 219 · ProcessMessage p. 234
149 bool AddToWalletIfMine(const CTransaction& tx, const CBlock* pblock)
150 {
151 if (tx.IsMine() || mapWallet.count(tx.GetHash()))
152 {
153         CWalletTx wtx(tx);
154 // Get merkle branch if transaction was found in a block
155 if (pblock)
156             wtx.SetMerkleBranch(pblock);
157 return AddToWallet(wtx);
158 }
159 return true;
160 }
161
    CALLED FROM  CTransaction::AcceptTransaction p. 210
162 bool EraseFromWallet(uint256 hash)
163 {
164 CRITICAL_BLOCK(cs_mapWallet)
165 {
166 if (mapWallet.erase(hash))
167 CWalletDB().EraseTx(hash);
168 }
169 return true;
170 }
171
172
173
174
175
176
177
178
179
180 //////////////////////////////////////////////////////////////////////////////
181 //
182 // mapOrphanTransactions
183 //
184
```

`AddOrphanTx()` - stores transactions whose parents haven't arrived yet. In a P2P network, messages arrive out of order. If transaction B spends transaction A's output, but B arrives first, B is an 'orphan'—valid in isolation but unverifiable without A. Satoshi's solution: store orphans indexed by their missing parent's hash. When A finally arrives, the node can immediately process waiting orphans. The dual maps (`mapOrphanTransactions` and `mapOrphanTransactionsByPrev`) enable both lookup by orphan hash and efficient retrieval when parents arrive. This pattern—holding incomplete data until dependencies resolve—appears throughout Bitcoin's networking code.

```
    CALLED FROM  ProcessMessage p. 235
185 void AddOrphanTx(const CDataStream& vMsg)
186 {
187     CTransaction tx;
188 CDataStream(vMsg) >> tx;
189     uint256 hash = tx.GetHash();
190 if (mapOrphanTransactions.count(hash))
191 return;
192     CDataStream* pvMsg = mapOrphanTransactions[hash] = new CDataStream(vMsg);
193 foreach(const CTxIn& txin, tx.vin)
```

```
194         mapOrphanTransactionsByPrev.insert(make_pair(txin.prevout.hash, pvMsg));
195 }
196
    CALLED FROM ProcessMessage p. 235
197 void EraseOrphanTx(uint256 hash)
198 {
199 if (!mapOrphanTransactions.count(hash))
200 return;
201 const CDataStream* pvMsg = mapOrphanTransactions[hash];
202     CTransaction tx;
203 CDataStream(*pvMsg) >> tx;
204 foreach(const CTxIn& txin, tx.vin)
205 {
206 for (multimap<uint256, CDataStream*>::iterator mi =
    mapOrphanTransactionsByPrev.lower_bound(txin.prevout.hash);
207              mi != mapOrphanTransactionsByPrev.upper_bound(txin.prevout.hash);)
208 {
209 if ((*mi).second == pvMsg)
210                 mapOrphanTransactionsByPrev.erase(mi++);
211 else
212                 mi++;
213 }
214 }
215 delete pvMsg;
216     mapOrphanTransactions.erase(hash);
217 }
218
219
220
221
222
223
224
225
226 //////////////////////////////////////////////////////////////////////////////
227 //
228 // CTransaction
229 //
230
```

`CTxIn::IsMine()` — is this input spending our own coin? Looks up the previous transaction in the wallet; if the referenced output paid to a key we control, this input is ours (it's a debit). Used to distinguish 'money I'm sending' from 'money passing through' when calculating wallet balance.

```
231 bool CTxIn::IsMine() const
232 {
233 CRITICAL_BLOCK(cs_mapWallet)
234 {
235         map<uint256, CWalletTx>::iterator mi = mapWallet.find(prevout.hash);
236 if (mi != mapWallet.end())
237 {
238 const CWalletTx& prev = (*mi).second;
239 if (prevout.n < prev.vout.size())
240 if (prev.vout[prevout.n].IsMine())
241 return true;
242 }
243 }
244 return false;
245 }
246
```

`CTxIn::GetDebit()` — the satoshi value being debited from our wallet by this input. Returns 0 for inputs not ours. `GetCredit()` summed across all outputs, minus `GetDebit()` summed across all inputs, gives the net change to your balance.

CALLED FROM CTransaction (inline) main.h p. 265 · CMainFrame::InsertTransaction ui.cpp p. 483 · CTxDetailsDialog::CTxDetailsDialog ui.cpp p. 491 · CViewOrderDialog::CViewOrderDialog ui.cpp p. 513

```
247 int64 CTxIn::GetDebit() const
248 {
249 CRITICAL_BLOCK(cs_mapWallet)
250 {
251         map<uint256, CWalletTx>::iterator mi = mapWallet.find(prevout.hash);
252 if (mi != mapWallet.end())
253 {
254 const CWalletTx& prev = (*mi).second;
255 if (prevout.n < prev.vout.size())
256 if (prev.vout[prevout.n].IsMine())
257 return prev.vout[prevout.n].nValue;
258 }
259 }
260 return 0;
261 }
262
```

`CWalletTx::GetTxTime()` — when did this transaction happen? For transactions received directly from the network, use the local arrival time. For transactions learned about through a block, use `GetMedianTime()`: the median timestamp of the 11 blocks centered on the containing block (five before, five after), which smooths out a single miner's inaccurate clock. Until five more blocks exist, it simply uses the block's own time. This is a display heuristic, not the consensus median-time-past check in `AcceptBlock()`.

CALLED FROM CMainFrame::InsertTransaction ui.cpp p. 483 · CMainFrame::RefreshStatus ui.cpp p. 486 · CMainFrame::OnIdle ui.cpp p. 487 · CTxDetailsDialog::CTxDetailsDialog ui.cpp p. 491

```
263 int64 CWalletTx::GetTxTime() const
264 {
265 if (!fTimeReceivedIsTxTime && hashBlock != 0)
266 {
267 // If we did not receive the transaction directly, we rely on the block's
268 // time to figure out when it happened.  We use the median over a range
269 // of blocks to try to filter out inaccurate block times.
270         map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hashBlock);
271 if (mi != mapBlockIndex.end())
272 {
273             CBlockIndex* pindex = (*mi).second;
274 if (pindex)
275 return pindex->GetMedianTime();
276 }
277 }
278 return nTimeReceived;
279 }
280
281
282
283
284
285
```

`CMerkleTx::SetMerkleBranch()` — links a transaction to its containing block via a merkle proof. Stores the block hash, the transaction's index within the block, and the sibling hashes needed to reconstruct the path from this transaction up to the block's merkle root. Returns the confirmation depth (how many blocks have been built on top). The SPV protocol Satoshi described in the whitepaper uses exactly these merkle branches — a lightweight client can verify a transaction is included in a block without downloading the full block.[3]

CALLED FROM AddToWalletIfMine p. 204 · CWalletTx::AddSupportingTransactions p. 208

```
286 int CMerkleTx::SetMerkleBranch(const CBlock* pblock)
287 {
288 if (fClient)
289 {
290 if (hashBlock == 0)
291 return 0;
292 }
293 else
294 {
295         CBlock blockTmp;
296 if (pblock == NULL)
297 {
298 // Load the block this tx is in
299             CTxIndex txindex;
300 if (!CTxDB("r").ReadTxIndex(GetHash(), txindex))
301 return 0;
302 if (!blockTmp.ReadFromDisk(txindex.pos.nFile, txindex.pos.nBlockPos, true))
303 return 0;
304             pblock = &blockTmp;
305 }
306
307 // Update the tx's hashBlock
308         hashBlock = pblock->GetHash();
309
310 // Locate the transaction
311 for (nIndex = 0; nIndex < pblock->vtx.size(); nIndex++)
312 if (pblock->vtx[nIndex] == *(CTransaction*)this)
313 break;
314 if (nIndex == pblock->vtx.size())
315 {
316             vMerkleBranch.clear();
317             nIndex = -1;
318 printf("ERROR: SetMerkleBranch() : couldn't find tx in block\n");
319 return 0;
320 }
321
322 // Fill in merkle branch
323         vMerkleBranch = pblock->GetMerkleBranch(nIndex);
324 }
325
326 // Is the tx in a block that's in the main chain
327     map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hashBlock);
328 if (mi == mapBlockIndex.end())
329 return 0;
330     CBlockIndex* pindex = (*mi).second;
331 if (!pindex || !pindex->IsInMainChain())
332 return 0;
333
334 return pindexBest->nHeight - pindex->nHeight + 1;
335 }
336
337
338
```

`AddSupportingTransactions()` — bundles ancestor transactions with a wallet transaction for relay to peers. When you broadcast a transaction, the recipient might not have the previous transactions your inputs reference. `COPY_DEPTH = 3` is a confirmation threshold, not a generation count: any ancestor with fewer than 3 confirmations is bundled, and the walk continues through its inputs; once an ancestor is 3 blocks deep, the receiver can be expected to have it already. This was Satoshi's pragmatic solution to the 'parent not found' problem.

CALLED FROM CreateTransaction p. 248

```
339 void CWalletTx::AddSupportingTransactions(CTxDB& txdb)
340 {
341     vtxPrev.clear();
342
343 const int COPY_DEPTH = 3;
344 if (SetMerkleBranch() < COPY_DEPTH)
345 {
346         vector<uint256> vWorkQueue;
347 foreach(const CTxIn& txin, vin)
348             vWorkQueue.push_back(txin.prevout.hash);
349
350 // This critsect is OK because txdb is already open
351 CRITICAL_BLOCK(cs_mapWallet)
352 {
353             map<uint256, const CMerkleTx*> mapWalletPrev;
354             set<uint256> setAlreadyDone;
355 for (int i = 0; i < vWorkQueue.size(); i++)
356 {
357                 uint256 hash = vWorkQueue[i];
358 if (setAlreadyDone.count(hash))
359 continue;
360                 setAlreadyDone.insert(hash);
361
362                 CMerkleTx tx;
363 if (mapWallet.count(hash))
364 {
365                     tx = mapWallet[hash];
366 foreach(const CMerkleTx& txWalletPrev, mapWallet[hash].vtxPrev)
367                         mapWalletPrev[txWalletPrev.GetHash()] = &txWalletPrev;
368 }
369 else if (mapWalletPrev.count(hash))
370 {
371                     tx = *mapWalletPrev[hash];
372 }
373 else if (!fClient && txdb.ReadDiskTx(hash, tx))
374 {
375 ;
376 }
377 else
378 {
379 printf("ERROR: AddSupportingTransactions() : unsupported transaction\n");
380 continue;
381 }
382
383 int nDepth = tx.SetMerkleBranch();
384                 vtxPrev.push_back(tx);
385
386 if (nDepth < COPY_DEPTH)
387 foreach(const CTxIn& txin, tx.vin)
388                         vWorkQueue.push_back(txin.prevout.hash);
389 }
390 }
391 }
392
393 reverse(vtxPrev.begin(), vtxPrev.end());
394 }
395
396
```

```
397
398
399
400
401
402
403
404
405
```

`AcceptTransaction()` - the gatekeeper. Validates syntax, checks inputs exist, verifies signatures. Rejects double-spends before they enter mempool.

```
406 bool CTransaction::AcceptTransaction(CTxDB& txdb, bool fCheckInputs, bool* pfMissingInputs)
407 {
408 if (pfMissingInputs)
409 *pfMissingInputs = false;
410
411 // Coinbase is only valid in a block, not as a loose transaction
```

`IsCoinBase()` check - coinbase transactions (mining rewards) can only exist in blocks, never broadcast alone. This prevents fake coin creation.

```
412 if (IsCoinBase())
413 return error("AcceptTransaction() : coinbase as individual tx");
414
415 if (!CheckTransaction())
416 return error("AcceptTransaction() : CheckTransaction failed");
417
418 // Do we already have it?
419     uint256 hash = GetHash();
420 CRITICAL_BLOCK(cs_mapTransactions)
421 if (mapTransactions.count(hash))
422 return false;
423 if (fCheckInputs)
424 if (txdb.ContainsTx(hash))
425 return false;
426
427 // Check for conflicts with in-memory transactions
428     CTransaction* ptxOld = NULL;
429 for (int i = 0; i < vin.size(); i++)
430 {
431         COutPoint outpoint = vin[i].prevout;
432 if (mapNextTx.count(outpoint))
433 {
434 // Allow replacing with a newer version of the same transaction
435 if (i != 0)
436 return false;
437             ptxOld = mapNextTx[outpoint].ptx;
438 if (!IsNewerThan(*ptxOld))
439 return false;
440 for (int i = 0; i < vin.size(); i++)
441 {
442                 COutPoint outpoint = vin[i].prevout;
443 if (!mapNextTx.count(outpoint) || mapNextTx[outpoint].ptx != ptxOld)
444 return false;
445 }
446 break;
447 }
448 }
449
450 // Check against previous transactions
451     map<uint256, CTxIndex> mapUnused;
452     int64 nFees = 0;
453 if (fCheckInputs && !ConnectInputs(txdb, mapUnused, CDiskTxPos(1,1,1), 0, nFees, false, false))
454 {
455 if (pfMissingInputs)
```

```
456 *pfMissingInputs = true;
457 return error("AcceptTransaction() : ConnectInputs failed %s", hash.ToString().substr(0,6).c_str());
458 }
459
460 // Store transaction in memory
461 CRITICAL_BLOCK(cs_mapTransactions)
462 {
463 if (ptxOld)
464 {
465 printf("mapTransaction.erase(%s) replacing with new version\n", ptxOld->GetHash().ToString().c_str());
466             mapTransactions.erase(ptxOld->GetHash());
467 }
468 AddToMemoryPool();
469 }
470
471 ///// are we sure this is ok when loading transactions or restoring block txes
472 // If updated, erase old tx from wallet
473 if (ptxOld)
474 EraseFromWallet(ptxOld->GetHash());
475
476 printf("AcceptTransaction(): accepted %s\n", hash.ToString().substr(0,6).c_str());
477 return true;
478 }
479
480
    CALLED FROM CTransaction::AcceptTransaction p. 210
481 bool CTransaction::AddToMemoryPool()
482 {
483 // Add to memory pool without checking anything.  Don't call this directly,
484 // call AcceptTransaction to properly check the transaction first.
485 CRITICAL_BLOCK(cs_mapTransactions)
486 {
487         uint256 hash = GetHash();
488         mapTransactions[hash] = *this;
489 for (int i = 0; i < vin.size(); i++)
490             mapNextTx[vin[i].prevout] = CInPoint(&mapTransactions[hash], i);
491         nTransactionsUpdated++;
492 }
493 return true;
494 }
495
496
    CALLED FROM Reorganize p. 220 · CBlock::AddToBlockIndex p. 221
497 bool CTransaction::RemoveFromMemoryPool()
498 {
499 // Remove transaction from memory pool
500 CRITICAL_BLOCK(cs_mapTransactions)
501 {
502 foreach(const CTxIn& txin, vin)
503             mapNextTx.erase(txin.prevout);
504         mapTransactions.erase(GetHash());
505         nTransactionsUpdated++;
506 }
507 return true;
508 }
509
510
511
512
513
514
    CALLED FROM CMerkleTx::GetBlocksToMaturity p. 211 · PrintBlockTree p. 229 ·
    CMerkleTx (inline) main.h p. 268 · FormatTxStatus ui.cpp p. 483
515 int CMerkleTx::GetDepthInMainChain() const
516 {
517 if (hashBlock == 0 || nIndex == -1)
518 return 0;
519
520 // Find the block it claims to be in
521     map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hashBlock);
522 if (mi == mapBlockIndex.end())
```

```
523 return 0;
524     CBlockIndex* pindex = (*mi).second;
525 if (!pindex || !pindex->IsInMainChain())
526 return 0;
527
528 // Make sure the merkle branch connects to this block
529 if (!fMerkleVerified)
530 {
531 if (CBlock::CheckMerkleBranch(GetHash(), vMerkleBranch, nIndex) != pindex->hashMerkleRoot)
532 return 0;
533         fMerkleVerified = true;
534 }
535
536 return pindexBest->nHeight - pindex->nHeight + 1;
537 }
538
539
```

CALLED FROM PrintBlockTree p. 229 · CMerkleTx (inline) main.h p. 268 · CMainFrame::InsertTransaction ui.cpp p. 484 · CTxDetailsDialog::CTxDetailsDialog ui.cpp p. 492

```
540 int CMerkleTx::GetBlocksToMaturity() const
541 {
542 if (!IsCoinBase())
543 return 0;
544 return max(0, (COINBASE_MATURITY+20) - GetDepthInMainChain());
545 }
546
547
548 bool CMerkleTx::AcceptTransaction(CTxDB& txdb, bool fCheckInputs)
549 {
550 if (fClient)
551 {
552 if (!IsInMainChain() && !ClientConnectInputs())
553 return false;
554 return CTransaction::AcceptTransaction(txdb, false);
555 }
556 else
557 {
558 return CTransaction::AcceptTransaction(txdb, fCheckInputs);
559 }
560 }
561
562
563
```

CALLED FROM ReacceptWalletTransactions p. 212 · ProcessMessage p. 237 · CWalletTx (inline) main.h p. 269

```
564 bool CWalletTx::AcceptWalletTransaction(CTxDB& txdb, bool fCheckInputs)
565 {
566 CRITICAL_BLOCK(cs_mapTransactions)
567 {
568 foreach(CMerkleTx& tx, vtxPrev)
569 {
570 if (!tx.IsCoinBase())
571 {
572                 uint256 hash = tx.GetHash();
573 if (!mapTransactions.count(hash) && !txdb.ContainsTx(hash))
574                     tx.AcceptTransaction(txdb, fCheckInputs);
575 }
576 }
577 if (!IsCoinBase())
578 return AcceptTransaction(txdb, fCheckInputs);
579 }
580 return true;
581 }
582
```

CALLED FROM CMyApp::OnInit2 ui.cpp p. 517

```
583 void ReacceptWalletTransactions()
584 {
585 // Reaccept any txes of ours that aren't already in a block
586     CTxDB txdb("r");
587 CRITICAL_BLOCK(cs_mapWallet)
588 {
589 foreach(PAIRTYPE(const uint256, CWalletTx)& item, mapWallet)
```

```
590 {
591             CWalletTx& wtx = item.second;
592 if (!wtx.IsCoinBase() && !txdb.ContainsTx(wtx.GetHash()))
593                 wtx.AcceptWalletTransaction(txdb, false);
594 }
595 }
596 }
597
598
    CALLED FROM  RelayWalletTransactions p. 212 · ProcessMessage p. 237 · SendMoney p. 249 ·
    CWalletTx (inline) main.h p. 269 · CSendingDialog::OnReply2 ui.cpp p. 501
599 void CWalletTx::RelayWalletTransaction(CTxDB& txdb)
600 {
601 foreach(const CMerkleTx& tx, vtxPrev)
602 {
603 if (!tx.IsCoinBase())
604 {
605             uint256 hash = tx.GetHash();
606 if (!txdb.ContainsTx(hash))
607 RelayMessage(CInv(MSG_TX, hash), (CTransaction)tx);
608 }
609 }
610 if (!IsCoinBase())
611 {
612         uint256 hash = GetHash();
613 if (!txdb.ContainsTx(hash))
614 {
615 printf("Relaying wtx %s\n", hash.ToString().substr(0,6).c_str());
616 RelayMessage(CInv(MSG_TX, hash), (CTransaction)*this);
617 }
618 }
619 }
620
    CALLED FROM  CBlock::AddToBlockIndex p. 221
621 void RelayWalletTransactions()
622 {
623 static int64 nLastTime;
624 if (GetTime() - nLastTime < 10 * 60)
625 return;
626     nLastTime = GetTime();
627
628 // Rebroadcast any of our txes that aren't in a block yet
629 printf("RelayWalletTransactions()\n");
630     CTxDB txdb("r");
631 CRITICAL_BLOCK(cs_mapWallet)
632 {
633 // Sort them in chronological order
634         multimap<unsigned int, CWalletTx*> mapSorted;
635 foreach(PAIRTYPE(const uint256, CWalletTx)& item, mapWallet)
636 {
637             CWalletTx& wtx = item.second;
638             mapSorted.insert(make_pair(wtx.nTimeReceived, &wtx));
639 }
640 foreach(PAIRTYPE(const unsigned int, CWalletTx*)& item, mapSorted)
641 {
642             CWalletTx& wtx = *item.second;
643             wtx.RelayWalletTransaction(txdb);
644 }
645 }
646 }
647
648
649
650
651
652
653
654
655
656
657 //////////////////////////////////////////////////////////////////////////////
```

```
658 //
659 // CBlock and CBlockIndex
660 //
661
    CALLED FROM CTxDB::ReadOwnerTxes db.cpp p. 123 · CTxDB::ReadDiskTx db.cpp p. 123 ·
    CMerkleTx::SetMerkleBranch p. 207 · CTransaction::ConnectInputs p. 216 · Reorganize p. 219 ·
    PrintBlockTree p. 228 · and 1 more
662 bool CBlock::ReadFromDisk(const CBlockIndex* pblockindex, bool fReadTransactions)
663 {
664 return ReadFromDisk(pblockindex->nFile, pblockindex->nBlockPos, fReadTransactions);
665 }
666
    CALLED FROM ProcessBlock p. 224 · ProcessMessage p. 233
667 uint256 GetOrphanRoot(const CBlock* pblock)
668 {
669 // Work back to the first block in the orphan chain
670 while (mapOrphanBlocks.count(pblock->hashPrevBlock))
671         pblock = mapOrphanBlocks[pblock->hashPrevBlock];
672 return pblock->GetHash();
673 }
674
    CALLED FROM CBlock::ConnectBlock p. 218 · BitcoinMiner p. 242
675 int64 CBlock::GetBlockValue(int64 nFees) const
676 {
```

50 BTC—the original block reward, equivalent to 5 billion satoshis. Satoshi explained to Mike Hearn in a 2009 email: 'The number of coins and distribution schedule was an educated guess. It was a difficult choice, because once the network is going it's locked in and we're stuck with it.'[15] He chose 50 as a round number that would produce approximately 21 million total coins via geometric series.

```
677     int64 nSubsidy = 50 * COIN;
678
679 // Subsidy is cut in half every 4 years
```

The halving formula: `nSubsidy >>= (nBestHeight / 210000)`. Every 210,000 blocks (~4 years), the reward halves — a geometric series converging on 21 million BTC. There is a subtle bug: once the shift count exceeds the integer width, C++ right-shift behavior is undefined and the subsidy could reinflate. BIP 42 documented this and soft-forked a finite number of halvings.[16][17]

```
680     nSubsidy >>= (nBestHeight / 210000);
681
682 return nSubsidy + nFees;
683 }
684
```

`GetNextWorkRequired()`—the difficulty adjustment algorithm, perhaps Satoshi's most elegant engineering. Without this, Bitcoin would be unstable: too many miners would create blocks too fast, too few would halt the network. This function automatically adjusts difficulty to maintain the 10-minute average regardless of how much hash power joins or leaves.

```
    CALLED FROM CBlock::AcceptBlock p. 223 · BitcoinMiner p. 240
685 unsigned int GetNextWorkRequired(const CBlockIndex* pindexLast)
686 {
```

`nTargetTimespan` = 14 days (1,209,600 seconds). Difficulty recalculates every 2016 blocks. A two-week window is long enough to smooth out variance (a lucky streak of fast blocks shouldn't swing the retarget too hard), yet short enough to track major hash power changes. Satoshi summarized the rule on BitcoinTalk: 'The difficulty is multiplied by 14/(actual days taken).'[18] The code retargets the *target*: `new_target = old_target × (actual_time / target_time)`. Difficulty is the inverse, so `new_difficulty = old_difficulty × (target_time / actual_time)`. If blocks came in 80% of the expected time, difficulty rises 25%.

```
687 const unsigned int nTargetTimespan = 14 * 24 * 60 * 60; // two weeks
```

`nTargetSpacing` = 10 minutes (600 seconds). Why 10 minutes? Satoshi explained the trade-off: 'The frequency of block generation is balanced between confirming transactions as fast as possible and the latency of the network.' Shorter blocks would multiply the chain's size, and 'the initial block download would take 10 times as long. It wouldn't work anyway because that would be only 1 minute average between blocks, too close to the broadcast latency when the network gets larger.'[18] Ten minutes leaves time for a block to propagate before the next is likely found.

```
688 const unsigned int nTargetSpacing = 10 * 60;
689 const unsigned int nInterval = nTargetTimespan / nTargetSpacing;
690
691 // Genesis block
692 if (pindexLast == NULL)
693 return bnProofOfWorkLimit.GetCompact();
694
695 // Only change once per interval
696 if ((pindexLast->nHeight+1) % nInterval != 0)
697 return pindexLast->nBits;
698
699 // Go back by what we want to be 14 days worth of blocks
700 const CBlockIndex* pindexFirst = pindexLast;
701 for (int i = 0; pindexFirst && i < nInterval-1; i++)
702         pindexFirst = pindexFirst->pprev;
703 assert(pindexFirst);
704
705 // Limit adjustment step
706 unsigned int nActualTimespan = pindexLast->nTime - pindexFirst->nTime;
707 printf("  nActualTimespan = %d  before bounds\n", nActualTimespan);
708 if (nActualTimespan < nTargetTimespan/4)
709         nActualTimespan = nTargetTimespan/4;
710 if (nActualTimespan > nTargetTimespan*4)
711         nActualTimespan = nTargetTimespan*4;
712
713 // Retarget
714     CBigNum bnNew;
715     bnNew.SetCompact(pindexLast->nBits);
716     bnNew *= nActualTimespan;
717     bnNew /= nTargetTimespan;
718
719 if (bnNew > bnProofOfWorkLimit)
720         bnNew = bnProofOfWorkLimit;
721
722 /// debug print
723 printf("\n\n\nGetNextWorkRequired RETARGET *****\n");
724 printf("nTargetTimespan = %d    nActualTimespan = %d\n", nTargetTimespan, nActualTimespan);
725 printf("Before: %08x  %s\n", pindexLast->nBits, CBigNum().SetCompact(pindexLast-
    >nBits).getuint256().ToString().c_str());
726 printf("After:  %08x  %s\n", bnNew.GetCompact(), bnNew.getuint256().ToString().c_str());
727
728 return bnNew.GetCompact();
729 }
730
731
732
```

```
733
734
735
736
737
738
    CALLED FROM CBlock::DisconnectBlock p. 218
739 bool CTransaction::DisconnectInputs(CTxDB& txdb)
740 {
741 // Relinquish previous transactions' spent pointers
742 if (!IsCoinBase())
743 {
744 foreach(const CTxIn& txin, vin)
745 {
746             COutPoint prevout = txin.prevout;
747
748 // Get prev txindex from disk
749             CTxIndex txindex;
750 if (!txdb.ReadTxIndex(prevout.hash, txindex))
751 return error("DisconnectInputs() : ReadTxIndex failed");
752
753 if (prevout.n >= txindex.vSpent.size())
754 return error("DisconnectInputs() : prevout.n out of range");
755
756 // Mark outpoint as not spent
757             txindex.vSpent[prevout.n].SetNull();
758
759 // Write back
760             txdb.UpdateTxIndex(prevout.hash, txindex);
761 }
762 }
763
764 // Remove transaction from index
765 if (!txdb.EraseTxIndex(*this))
766 return error("DisconnectInputs() : EraseTxPos failed");
767
768 return true;
769 }
770
771
```

`ConnectInputs()` - the double-spend prevention engine. Verifies each input references an unspent output. This is Bitcoin's core innovation.

```
    CALLED FROM CTransaction::AcceptTransaction p. 209 · CBlock::ConnectBlock p. 218 · BitcoinMiner p. 242
772 bool CTransaction::ConnectInputs(CTxDB& txdb, map<uint256, CTxIndex>& mapTestPool, CDiskTxPos posThisTx,
    int nHeight, int64& nFees, bool fBlock, bool fMiner, int64 nMinFee)
773 {
774 // Take over previous transactions' spent pointers
775 if (!IsCoinBase())
776 {
777         int64 nValueIn = 0;
778 for (int i = 0; i < vin.size(); i++)
779 {
780             COutPoint prevout = vin[i].prevout;
781
782 // Read txindex
783             CTxIndex txindex;
784 bool fFound = true;
785 if (fMiner && mapTestPool.count(prevout.hash))
786 {
787 // Get txindex from current proposed changes
788                 txindex = mapTestPool[prevout.hash];
789 }
790 else
791 {
792 // Read txindex from txdb
793                 fFound = txdb.ReadTxIndex(prevout.hash, txindex);
794 }
```

```
795 if (!fFound && (fBlock || fMiner))
796 return fMiner ? false : error("ConnectInputs() : %s prev tx %s index entry not found",
    GetHash().ToString().substr(0,6).c_str(),  prevout.hash.ToString().substr(0,6).c_str());
797
798 // Read txPrev
799             CTransaction txPrev;
800 if (!fFound || txindex.pos == CDiskTxPos(1,1,1))
801 {
802 // Get prev tx from single transactions in memory
803 CRITICAL_BLOCK(cs_mapTransactions)
804 {
805 if (!mapTransactions.count(prevout.hash))
806 return error("ConnectInputs() : %s mapTransactions prev not found %s",
    GetHash().ToString().substr(0,6).c_str(),  prevout.hash.ToString().substr(0,6).c_str());
807                     txPrev = mapTransactions[prevout.hash];
808 }
809 if (!fFound)
810                     txindex.vSpent.resize(txPrev.vout.size());
811 }
812 else
813 {
814 // Get prev tx from disk
815 if (!txPrev.ReadFromDisk(txindex.pos))
816 return error("ConnectInputs() : %s ReadFromDisk prev tx %s failed",
    GetHash().ToString().substr(0,6).c_str(),  prevout.hash.ToString().substr(0,6).c_str());
817 }
818
819 if (prevout.n >= txPrev.vout.size() || prevout.n >= txindex.vSpent.size())
820 return error("ConnectInputs() : %s prevout.n out of range %d %d %d",
    GetHash().ToString().substr(0,6).c_str(), prevout.n, txPrev.vout.size(), txindex.vSpent.size());
821
822 // If prev is coinbase, check that it's matured
823 if (txPrev.IsCoinBase())
824 for (CBlockIndex* pindex = pindexBest; pindex && nBestHeight - pindex->nHeight < COINBASE_MATURITY-1;
    pindex = pindex->pprev)
825 if (pindex->nBlockPos == txindex.pos.nBlockPos && pindex->nFile == txindex.pos.nFile)
826 return error("ConnectInputs() : tried to spend coinbase at depth %d", nBestHeight - pindex->nHeight);
827
828 // Verify signature
```

`VerifySignature()` — script validation call site inside `ConnectInputs`. After loading the previous tx and enforcing coinbase maturity, each input's `scriptSig` must satisfy the prior `scriptPubKey` or the block/tx is rejected.

```
829 if (!VerifySignature(txPrev, *this, i))
830 return error("ConnectInputs() : %s VerifySignature failed", GetHash().ToString().substr(0,6).c_str());
831
832 // Check for conflicts
833 if (!txindex.vSpent[prevout.n].IsNull())
```

`'prev tx already used'` - double-spend detected! The transaction tries to spend coins already spent elsewhere. Rejected.

```
834 return fMiner ? false : error("ConnectInputs() : %s prev tx already used at %s",
    GetHash().ToString().substr(0,6).c_str(), txindex.vSpent[prevout.n].ToString().c_str());
835
836 // Mark outpoints as spent
837             txindex.vSpent[prevout.n] = posThisTx;
838
839 // Write back
840 if (fBlock)
841                 txdb.UpdateTxIndex(prevout.hash, txindex);
842 else if (fMiner)
843                 mapTestPool[prevout.hash] = txindex;
844
845             nValueIn += txPrev.vout[prevout.n].nValue;
```

```
}

// Tally transaction fees
        int64 nTxFee = nValueIn - GetValueOut();
if (nTxFee < 0)
return error("ConnectInputs() : %s nTxFee < 0", GetHash().ToString().substr(0,6).c_str());
if (nTxFee < nMinFee)
return false;
        nFees += nTxFee;
}

if (fBlock)
{
// Add transaction to disk index
if (!txdb.AddTxIndex(*this, posThisTx, nHeight))
return error("ConnectInputs() : AddTxPos failed");
}
else if (fMiner)
{
// Add transaction to test pool
        mapTestPool[GetHash()] = CTxIndex(CDiskTxPos(1,1,1), vout.size());
}

return true;
}


```

CALLED FROM CMerkleTx::AcceptTransaction p. 211

```
bool CTransaction::ClientConnectInputs()
{
if (IsCoinBase())
return false;

// Take over previous transactions' spent pointers
CRITICAL_BLOCK(cs_mapTransactions)
{
        int64 nValueIn = 0;
for (int i = 0; i < vin.size(); i++)
{
// Get prev tx from single transactions in memory
            COutPoint prevout = vin[i].prevout;
if (!mapTransactions.count(prevout.hash))
return false;
            CTransaction& txPrev = mapTransactions[prevout.hash];

if (prevout.n >= txPrev.vout.size())
return false;

// Verify signature
if (!VerifySignature(txPrev, *this, i))
return error("ConnectInputs() : VerifySignature failed");

///// this is redundant with the mapNextTx stuff, not sure which I want to get rid of
///// this has to go away now that posNext is gone
// // Check for conflicts
// if (!txPrev.vout[prevout.n].posNext.IsNull())
//     return error("ConnectInputs() : prev tx already used");
//
// // Flag outpoints as used
// txPrev.vout[prevout.n].posNext = posThisTx;

            nValueIn += txPrev.vout[prevout.n].nValue;
}
if (GetValueOut() > nValueIn)
return false;
}

return true;
}


```

```
916
917
```

`DisconnectBlock()` — undo a block on reorg: disconnect inputs in reverse order, clear the previous index's `hashNext`. The chain can walk backward without rewriting history files.

```
    CALLED FROM  Reorganize p. 219
918 bool CBlock::DisconnectBlock(CTxDB& txdb, CBlockIndex* pindex)
919 {
920 // Disconnect in reverse order
921 for (int i = vtx.size()-1; i >= 0; i--)
922 if (!vtx[i].DisconnectInputs(txdb))
923 return false;
924
925 // Update block index on disk without changing it in memory.
926 // The memory index structure will be changed after the db commits.
927 if (pindex->pprev)
928 {
929         CDiskBlockIndex blockindexPrev(pindex->pprev);
930         blockindexPrev.hashNext = 0;
931         txdb.WriteBlockIndex(blockindexPrev);
932 }
933
934 return true;
935 }
936
```

`ConnectBlock()` — apply a block: assign each tx a disk position, call `ConnectInputs` (marks UTXOs spent, verifies scripts), enforce coinbase ≤ subsidy+fees. This is where a valid header becomes economic state.

```
    CALLED FROM  Reorganize p. 220 · CBlock::AddToBlockIndex p. 221
937 bool CBlock::ConnectBlock(CTxDB& txdb, CBlockIndex* pindex)
938 {
939 //// issue here: it doesn't know the version
940 unsigned int nTxPos = pindex->nBlockPos + ::GetSerializeSize(CBlock(), SER_DISK) - 1 +
    GetSizeOfCompactSize(vtx.size());
941
942     map<uint256, CTxIndex> mapUnused;
943     int64 nFees = 0;
944 foreach(CTransaction& tx, vtx)
945 {
946         CDiskTxPos posThisTx(pindex->nFile, pindex->nBlockPos, nTxPos);
947         nTxPos += ::GetSerializeSize(tx, SER_DISK);
948
949 if (!tx.ConnectInputs(txdb, mapUnused, posThisTx, pindex->nHeight, nFees, true, false))
950 return false;
951 }
952
```

Coinbase value check — miner reward cannot exceed `GetBlockValue(nFees)`. The overflow bug of August 2010 slipped past related value checks elsewhere; this line is the local subsidy cap for a connected block.[19]

```
953 if (vtx[0].GetValueOut() > GetBlockValue(nFees))
954 return false;
955
956 // Update block index on disk without changing it in memory.
957 // The memory index structure will be changed after the db commits.
958 if (pindex->pprev)
959 {
960         CDiskBlockIndex blockindexPrev(pindex->pprev);
961         blockindexPrev.hashNext = pindex->GetBlockHash();
```

```
962         txdb.WriteBlockIndex(blockindexPrev);
963 }
964
965 // Watch for transactions paying to me
966 foreach(CTransaction& tx, vtx)
967 AddToWalletIfMine(tx, this);
968
969 return true;
970 }
971
972
973
```

`Reorganize()` - handles chain splits. When a longer valid chain appears, Bitcoin switches to it. 'The majority decision is represented by the longest chain.'[3]

```
    CALLED FROM  CBlock::AddToBlockIndex p. 221
974 bool Reorganize(CTxDB& txdb, CBlockIndex* pindexNew)
975 {
976 printf("*** REORGANIZE ***\n");
977
978 // Find the fork
979     CBlockIndex* pfork = pindexBest;
980     CBlockIndex* plonger = pindexNew;
981 while (pfork != plonger)
982 {
983 if (!(pfork = pfork->pprev))
984 return error("Reorganize() : pfork->pprev is null");
985 while (plonger->nHeight > pfork->nHeight)
986 if (!(plonger = plonger->pprev))
987 return error("Reorganize() : plonger->pprev is null");
988 }
989
990 // List of what to disconnect
```

`vDisconnect` - blocks to undo. During reorg, transactions return to mempool. Users may see confirmations temporarily reverse.

```
991     vector<CBlockIndex*> vDisconnect;
992 for (CBlockIndex* pindex = pindexBest; pindex != pfork; pindex = pindex->pprev)
993         vDisconnect.push_back(pindex);
994
995 // List of what to connect
```

`vConnect` - blocks to apply. The new chain's transactions take precedence. Because a reorg can undo recent blocks, recipients wait for several confirmations; six became the convention, following the whitepaper's attacker-probability table.[3]

```
996     vector<CBlockIndex*> vConnect;
997 for (CBlockIndex* pindex = pindexNew; pindex != pfork; pindex = pindex->pprev)
998         vConnect.push_back(pindex);
999 reverse(vConnect.begin(), vConnect.end());
1000
1001 // Disconnect shorter branch
1002     vector<CTransaction> vResurrect;
1003 foreach(CBlockIndex* pindex, vDisconnect)
1004 {
1005         CBlock block;
1006 if (!block.ReadFromDisk(pindex->nFile, pindex->nBlockPos, true))
1007 return error("Reorganize() : ReadFromDisk for disconnect failed");
1008 if (!block.DisconnectBlock(txdb, pindex))
1009 return error("Reorganize() : DisconnectBlock failed");
1010
1011 // Queue memory transactions to resurrect
```

```
1012 foreach(const CTransaction& tx, block.vtx)
1013 if (!tx.IsCoinBase())
1014                 vResurrect.push_back(tx);
1015 }
1016
1017 // Connect longer branch
1018     vector<CTransaction> vDelete;
1019 for (int i = 0; i < vConnect.size(); i++)
1020 {
1021         CBlockIndex* pindex = vConnect[i];
1022         CBlock block;
1023 if (!block.ReadFromDisk(pindex->nFile, pindex->nBlockPos, true))
1024 return error("Reorganize() : ReadFromDisk for connect failed");
1025 if (!block.ConnectBlock(txdb, pindex))
1026 {
1027 // Invalid block, delete the rest of this branch
1028             txdb.TxnAbort();
1029 for (int j = i; j < vConnect.size(); j++)
1030 {
1031                 CBlockIndex* pindex = vConnect[j];
1032                 pindex->EraseBlockFromDisk();
1033                 txdb.EraseBlockIndex(pindex->GetBlockHash());
1034                 mapBlockIndex.erase(pindex->GetBlockHash());
1035 delete pindex;
1036 }
1037 return error("Reorganize() : ConnectBlock failed");
1038 }
1039
1040 // Queue memory transactions to delete
1041 foreach(const CTransaction& tx, block.vtx)
1042             vDelete.push_back(tx);
1043 }
1044 if (!txdb.WriteHashBestChain(pindexNew->GetBlockHash()))
1045 return error("Reorganize() : WriteHashBestChain failed");
1046
1047 // Commit now because resurrecting could take some time
1048     txdb.TxnCommit();
1049
1050 // Disconnect shorter branch
1051 foreach(CBlockIndex* pindex, vDisconnect)
1052 if (pindex->pprev)
1053             pindex->pprev->pnext = NULL;
1054
1055 // Connect longer branch
1056 foreach(CBlockIndex* pindex, vConnect)
1057 if (pindex->pprev)
1058             pindex->pprev->pnext = pindex;
1059
1060 // Resurrect memory transactions that were in the disconnected branch
1061 foreach(CTransaction& tx, vResurrect)
1062         tx.AcceptTransaction(txdb, false);
1063
1064 // Delete redundant memory transactions that are in the connected branch
1065 foreach(CTransaction& tx, vDelete)
1066         tx.RemoveFromMemoryPool();
1067
1068 return true;
1069 }
1070
1071
```

CALLED FROM CBlock::AcceptBlock p. 223 · LoadBlockIndex p. 228

```
1072 bool CBlock::AddToBlockIndex(unsigned int nFile, unsigned int nBlockPos)
1073 {
1074 // Check for duplicate
1075     uint256 hash = GetHash();
1076 if (mapBlockIndex.count(hash))
1077 return error("AddToBlockIndex() : %s already exists", hash.ToString().substr(0,14).c_str());
1078
1079 // Construct new block index object
1080     CBlockIndex* pindexNew = new CBlockIndex(nFile, nBlockPos, *this);
1081 if (!pindexNew)
```

```
return error("AddToBlockIndex() : new CBlockIndex failed");
    map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.insert(make_pair(hash, pindexNew)).first;
    pindexNew->phashBlock = &((*mi).first);
    map<uint256, CBlockIndex*>::iterator miPrev = mapBlockIndex.find(hashPrevBlock);
if (miPrev != mapBlockIndex.end())
{
        pindexNew->pprev = (*miPrev).second;
        pindexNew->nHeight = pindexNew->pprev->nHeight + 1;
}

    CTxDB txdb;
    txdb.TxnBegin();
    txdb.WriteBlockIndex(CDiskBlockIndex(pindexNew));

// New best
if (pindexNew->nHeight > nBestHeight)
{
if (pindexGenesisBlock == NULL && hash == hashGenesisBlock)
{
            pindexGenesisBlock = pindexNew;
            txdb.WriteHashBestChain(hash);
}
else if (hashPrevBlock == hashBestChain)
{
// Adding to current best branch
if (!ConnectBlock(txdb, pindexNew) || !txdb.WriteHashBestChain(hash))
{
                txdb.TxnAbort();
                pindexNew->EraseBlockFromDisk();
                mapBlockIndex.erase(pindexNew->GetBlockHash());
delete pindexNew;
return error("AddToBlockIndex() : ConnectBlock failed");
}
            txdb.TxnCommit();
            pindexNew->pprev->pnext = pindexNew;

// Delete redundant memory transactions
foreach(CTransaction& tx, vtx)
                tx.RemoveFromMemoryPool();
}
else
{
// New best branch
if (!Reorganize(txdb, pindexNew))
{
                txdb.TxnAbort();
return error("AddToBlockIndex() : Reorganize failed");
}
}

// New best link
        hashBestChain = hash;
        pindexBest = pindexNew;
        nBestHeight = pindexBest->nHeight;
        nTransactionsUpdated++;
printf("AddToBlockIndex: new best=%s  height=%d\n", hashBestChain.ToString().substr(0,14).c_str(),
nBestHeight);
}

    txdb.TxnCommit();
    txdb.Close();

// Relay wallet transactions that haven't gotten in yet
if (pindexNew == pindexBest)
RelayWalletTransactions();

MainFrameRepaint();
return true;
}
```

1153

`CheckBlock()` - structural validation. Verifies size limits, timestamp sanity, merkle root, and proof-of-work. Rejects malformed blocks.

CALLED FROM ProcessBlock p. 223

```
1154 bool CBlock::CheckBlock() const
1155 {
1156 // These are checks that are independent of context
1157 // that can be verified before saving an orphan block.
1158
1159 // Size limits
1160 if (vtx.empty() || vtx.size() > MAX_SIZE || ::GetSerializeSize(*this, SER_DISK) > MAX_SIZE)
1161 return error("CheckBlock() : size limits failed");
1162
1163 // Check timestamp
1164 if (nTime > GetAdjustedTime() + 2 * 60 * 60)
1165 return error("CheckBlock() : block timestamp too far in the future");
1166
1167 // First transaction must be coinbase, the rest must not be
1168 if (vtx.empty() || !vtx[0].IsCoinBase())
1169 return error("CheckBlock() : first tx is not coinbase");
1170 for (int i = 1; i < vtx.size(); i++)
1171 if (vtx[i].IsCoinBase())
1172 return error("CheckBlock() : more than one coinbase");
1173
1174 // Check transactions
1175 foreach(const CTransaction& tx, vtx)
1176 if (!tx.CheckTransaction())
1177 return error("CheckBlock() : CheckTransaction failed");
1178
1179 // Check proof of work matches claimed amount
```

`bnProofOfWorkLimit` check — rejects a claimed `nBits` easier than the minimum difficulty. The target must not exceed `~uint256(0) >> 32` (all-1s would make proof-of-work trivial).

```
1180 if (CBigNum().SetCompact(nBits) > bnProofOfWorkLimit)
1181 return error("CheckBlock() : nBits below minimum work");
1182 if (GetHash() > CBigNum().SetCompact(nBits).getuint256())
1183 return error("CheckBlock() : hash doesn't match nBits");
1184
1185 // Check merkleroot
1186 if (hashMerkleRoot != BuildMerkleTree())
1187 return error("CheckBlock() : hashMerkleRoot mismatch");
1188
1189 return true;
1190 }
1191
```

`AcceptBlock()` - adds a validated block to the index. Checks `nBits` matches `GetNextWorkRequired()`, then writes to disk. Reorgs can still disconnect it later.

CALLED FROM ProcessBlock p. 224

```
1192 bool CBlock::AcceptBlock()
1193 {
1194 // Check for duplicate
1195     uint256 hash = GetHash();
1196 if (mapBlockIndex.count(hash))
1197 return error("AcceptBlock() : block already in mapBlockIndex");
1198
1199 // Get prev block index
1200     map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hashPrevBlock);
1201 if (mi == mapBlockIndex.end())
1202 return error("AcceptBlock() : prev block not found");
```

```
    CBlockIndex* pindexPrev = (*mi).second;

// Check timestamp against prev
if (nTime <= pindexPrev->GetMedianTimePast())
return error("AcceptBlock() : block's timestamp is too early");

// Check proof of work
if (nBits != GetNextWorkRequired(pindexPrev))
return error("AcceptBlock() : incorrect proof of work");

// Write block to history file
unsigned int nFile;
unsigned int nBlockPos;
if (!WriteToDisk(!fClient, nFile, nBlockPos))
return error("AcceptBlock() : WriteToDisk failed");
if (!AddToBlockIndex(nFile, nBlockPos))
return error("AcceptBlock() : AddToBlockIndex failed");

if (hashBestChain == hash)
RelayInventory(CInv(MSG_BLOCK, hash));

```

Commented-out atom minting — before launch, AcceptBlock would extract the coinbase pubkey and grant a random reputation atom to the miner. Atom values were drawn at random from below `USHRT_MAX`, so collisions would have been likely. Disabled here, the marketplace reputation graph never auto-funded from mining.[17]

```
// // Add atoms to user reviews for coins created
// vector<unsigned char> vchPubKey;
// if (ExtractPubKey(vtx[0].vout[0].scriptPubKey, false, vchPubKey))
// {
//     unsigned short nAtom = GetRand(USHRT_MAX - 100) + 100;
//     vector<unsigned short> vAtoms(1, nAtom);
//     AddAtomsAndPropagate(Hash(vchPubKey.begin(), vchPubKey.end()), vAtoms, true);
// }

return true;
}

```

`ProcessBlock()` - main entry point for new blocks. First checks if we've seen it, validates with `CheckBlock()`, then `AcceptBlock()` or orphan it.

```
    CALLED FROM  ProcessMessage p. 236 · BitcoinMiner p. 243
bool ProcessBlock(CNode* pfrom, CBlock* pblock)
{
// Check for duplicate
    uint256 hash = pblock->GetHash();
if (mapBlockIndex.count(hash))
return error("ProcessBlock() : already have block %d %s", mapBlockIndex[hash]->nHeight,
hash.ToString().substr(0,14).c_str());
if (mapOrphanBlocks.count(hash))
return error("ProcessBlock() : already have block (orphan) %s", hash.ToString().substr(0,14).c_str());

// Preliminary checks
if (!pblock->CheckBlock())
{
delete pblock;
return error("ProcessBlock() : CheckBlock FAILED");
}

// If don't already have its previous block, shunt it off to holding area until we get it
if (!mapBlockIndex.count(pblock->hashPrevBlock))
{
printf("ProcessBlock: ORPHAN BLOCK, prev=%s\n", pblock->hashPrevBlock.ToString().substr(0,14).c_str());
```

Orphan storage — if the parent is unknown, park the block in `mapOrphanBlocks` / `mapOrphanBlocksByPrev` and ask for the missing ancestor. Out-of-order gossip is normal; once the parent arrives, `ProcessBlock()` connects the waiting orphans automatically.

```
1256         mapOrphanBlocks.insert(make_pair(hash, pblock));
1257         mapOrphanBlocksByPrev.insert(make_pair(pblock->hashPrevBlock, pblock));
1258
1259 // Ask this guy to fill in what we're missing
1260 if (pfrom)
1261             pfrom->PushMessage("getblocks", CBlockLocator(pindexBest), GetOrphanRoot(pblock));
1262 return true;
1263 }
1264
1265 // Store to disk
1266 if (!pblock->AcceptBlock())
1267 {
1268 delete pblock;
1269 return error("ProcessBlock() : AcceptBlock FAILED");
1270 }
1271 delete pblock;
1272
1273 // Recursively process any orphan blocks that depended on this one
1274     vector<uint256> vWorkQueue;
1275     vWorkQueue.push_back(hash);
1276 for (int i = 0; i < vWorkQueue.size(); i++)
1277 {
1278         uint256 hashPrev = vWorkQueue[i];
1279 for (multimap<uint256, CBlock*>::iterator mi = mapOrphanBlocksByPrev.lower_bound(hashPrev);
1280              mi != mapOrphanBlocksByPrev.upper_bound(hashPrev);
1281 ++mi)
1282 {
1283             CBlock* pblockOrphan = (*mi).second;
1284 if (pblockOrphan->AcceptBlock())
1285                 vWorkQueue.push_back(pblockOrphan->GetHash());
1286             mapOrphanBlocks.erase(pblockOrphan->GetHash());
1287 delete pblockOrphan;
1288 }
1289         mapOrphanBlocksByPrev.erase(hashPrev);
1290 }
1291
1292 printf("ProcessBlock: ACCEPTED\n");
1293 return true;
1294 }
1295
1296
1297
1298
1299
1300
1301
1302
1303 template<typename Stream>
1304 bool ScanMessageStart(Stream& s)
1305 {
1306 // Scan ahead to the next pchMessageStart, which should normally be immediately
1307 // at the file pointer.  Leaves file pointer at end of pchMessageStart.
1308     s.clear(0);
1309 short prevmask = s.exceptions(0);
1310 const char* p = BEGIN(pchMessageStart);
1311 try
1312 {
1313         loop
1314 {
1315 char c;
1316             s.read(&c, 1);
1317 if (s.fail())
1318 {
1319                 s.clear(0);
```

```
                s.exceptions(prevmask);
return false;
}
if (*p != c)
                p = BEGIN(pchMessageStart);
if (*p == c)
{
if (++p == END(pchMessageStart))
{
                    s.clear(0);
                    s.exceptions(prevmask);
return true;
}
}
}
}
catch (...)
{
        s.clear(0);
        s.exceptions(prevmask);
return false;
}
}

```

CALLED FROM CDB::CDB db.cpp p. 118 · OpenBlockFile p. 225

```
string GetAppDir()
{
    string strDir;
if (!strSetDataDir.empty())
{
        strDir = strSetDataDir;
}
else if (getenv("APPDATA"))
{
        strDir = strprintf("%s\\Bitcoin", getenv("APPDATA"));
}
else if (getenv("USERPROFILE"))
{
        string strAppData = strprintf("%s\\Application Data", getenv("USERPROFILE"));
static bool fMkdirDone;
if (!fMkdirDone)
{
            fMkdirDone = true;
_mkdir(strAppData.c_str());
}
        strDir = strprintf("%s\\Bitcoin", strAppData.c_str());
}
else
{
return ".";
}
static bool fMkdirDone;
if (!fMkdirDone)
{
        fMkdirDone = true;
_mkdir(strDir.c_str());
}
return strDir;
}

```

CALLED FROM AppendBlockFile p. 226 · CTransaction (inline) main.h p. 266 · CBlock (inline) main.h p. 273 · CBlockIndex (inline) main.h p. 275

```
FILE* OpenBlockFile(unsigned int nFile, unsigned int nBlockPos, const char* pszMode)
{
if (nFile == -1)
return NULL;
    FILE* file = fopen(strprintf("%s\\blk%04d.dat", GetAppDir().c_str(), nFile).c_str(), pszMode);
if (!file)
return NULL;
if (nBlockPos != 0 && !strchr(pszMode, 'a') && !strchr(pszMode, 'w'))
{
```

```
1388 if (fseek(file, nBlockPos, SEEK_SET) != 0)
1389 {
1390 fclose(file);
1391 return NULL;
1392 }
1393 }
1394 return file;
1395 }
1396
1397 static unsigned int nCurrentBlockFile = 1;
1398
```

CALLED FROM CBlock (inline) main.h p. 273

```
1399 FILE* AppendBlockFile(unsigned int& nFileRet)
1400 {
1401     nFileRet = 0;
1402     loop
1403 {
1404         FILE* file = OpenBlockFile(nCurrentBlockFile, 0, "ab");
1405 if (!file)
1406 return NULL;
1407 if (fseek(file, 0, SEEK_END) != 0)
1408 return NULL;
1409 // FAT32 filesize max 4GB, fseek and ftell max 2GB, so we must stay under 2GB
1410 if (ftell(file) < 0x7F000000 - MAX_SIZE)
1411 {
1412             nFileRet = nCurrentBlockFile;
1413 return file;
1414 }
1415 fclose(file);
1416         nCurrentBlockFile++;
1417 }
1418 }
1419
1420 bool LoadBlockIndex(bool fAllowNew)
1421 {
1422 //
1423 // Load block index
1424 //
1425     CTxDB txdb("cr");
1426 if (!txdb.LoadBlockIndex())
1427 return false;
1428     txdb.Close();
1429
1430 //
1431 // Init with genesis block
1432 //
1433 if (mapBlockIndex.empty())
1434 {
1435 if (!fAllowNew)
1436 return false;
1437
1438
```

Genesis Block creation. Satoshi hardcoded this first block. The hash was computed offline; this code just reconstructs it for verification.

```
1439 // Genesis Block:
1440 // GetHash()      = 0x000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
1441 // hashMerkleRoot = 0x4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b
1442 // txNew.vin[0].scriptSig     = 486604799 4
     0x736B6E616220726F662074756F6C69616220646E6F63657320666F206B6E697262206E6F20726F6C6C65636E61684320393030
     322F6E614A2F33302073656D695420656854
1443 // txNew.vout[0].nValue       = 5000000000
1444 // txNew.vout[0].scriptPubKey =
     0x5F1DF16B2B704C8A578D0BBAF74D385CDE12C11EE50455F3C438EF4C3FBCF649B6DE611FEAE06279A60939E028A8D65C10B730
     71A6F16719274855FEB0FD8A6704 OP_CHECKSIG
1445 // block.nVersion = 1
1446 // block.nTime    = 1231006505
1447 // block.nBits    = 0x1d00ffff
```

```
1448 // block.nNonce   = 2083236893
1449 // CBlock(hash=000000000019d6, ver=1, hashPrevBlock=0000000000000000, hashMerkleRoot=4a5e1e,
     nTime=1231006505, nBits=1d00ffff, nNonce=2083236893, vtx=1)
1450 //   CTransaction(hash=4a5e1e, ver=1, vin.size=1, vout.size=1, nLockTime=0)
1451 //     CTxIn(COutPoint(000000, -1), coinbase
     04ffff001d0104455468652054696d65732030332f4a616e2f32303039204368616e63656c6c6f72206f6e206272696e6b206f6620
     7365636f6e64206261696c6f757420666f722062616e6b73)
1452 //     CTxOut(nValue=50.00000000, scriptPubKey=0x5F1DF16B2B704C8A578D0B)
1453 //   vMerkleTree: 4a5e1e
1454
1455 // Genesis block
1456 char* pszTimestamp = "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks";
1457         CTransaction txNew;
1458         txNew.vin.resize(1);
1459         txNew.vout.resize(1);
1460         txNew.vin[0].scriptSig     = CScript() << 486604799 << CBigNum(4) << vector<unsigned char>
     ((unsigned char*)pszTimestamp, (unsigned char*)pszTimestamp + strlen(pszTimestamp));
```

`50 * COIN`—the first 50 bitcoins ever created, worth exactly $0 at the time (and now worth millions). Curiously, these specific coins are permanently unspendable due to a quirk: the genesis block's coinbase transaction isn't indexed in the UTXO database. Whether this was intentional or an oversight is unknown. Satoshi never commented on it, and no one has ever been able to ask.

```
1461         txNew.vout[0].nValue       = 50 * COIN;
1462         txNew.vout[0].scriptPubKey = CScript() <<
     CBigNum("0x5F1DF16B2B704C8A578D0BBAF74D385CDE12C11EE50455F3C438EF4C3FBCF649B6DE611FEAE06279A60939E028A8D
     65C10B73071A6F16719274855FEB0FD8A6704") << OP_CHECKSIG;
1463         CBlock block;
1464         block.vtx.push_back(txNew);
```

`hashPrevBlock = 0`—the genesis block's defining characteristic. Every other block must reference a valid previous block; this is the only block that points to nothing. The all-zeros hash is Bitcoin's 'Big Bang moment'—the chain has to start somewhere. Genesis is loaded specially in `LoadBlockIndex`; any later block with an unknown parent (including `hashPrevBlock = 0`) fails `AcceptBlock` with 'prev block not found.'

```
1465         block.hashPrevBlock = 0;
1466         block.hashMerkleRoot = block.BuildMerkleTree();
1467         block.nVersion = 1;
1468         block.nTime    = 1231006505;
```

`nBits = 0x1d00ffff`—the initial difficulty target in compact format. This expands to a 256-bit target where valid hashes must start with approximately 32 zero bits. At this difficulty, a single circa-2009 CPU hashing a few hundred thousand times per second needed hours, on average, to find a block alone.[18] For comparison: today's network runs at difficulty requiring ~80+ leading zero bits, approximately 10^14 times harder.

```
1469         block.nBits    = 0x1d00ffff;
```

`nNonce = 2083236893`—the winning nonce hardcoded into every client (nonce means 'number used once'). At this target, expected work is ~2^{32} hashes; the nonce value itself is not a proven attempt count. The resulting hash `000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f` starts with the required zeros, proving work was done.

```
1470         block.nNonce   = 2083236893;
1471
1472 //// debug print, delete this later
1473 printf("%s\n", block.GetHash().ToString().c_str());
```

```
1474 printf("%s\n", block.hashMerkleRoot.ToString().c_str());
1475 printf("%s\n", hashGenesisBlock.ToString().c_str());
1476             txNew.vout[0].scriptPubKey.print();
1477             block.print();
1478 assert(block.hashMerkleRoot ==
     uint256("0x4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b"));
1479
1480 assert(block.GetHash() == hashGenesisBlock);
1481
1482 // Start new block file
1483 unsigned int nFile;
1484 unsigned int nBlockPos;
1485 if (!block.WriteToDisk(!fClient, nFile, nBlockPos))
1486 return error("LoadBlockIndex() : writing genesis block to disk failed");
1487 if (!block.AddToBlockIndex(nFile, nBlockPos))
1488 return error("LoadBlockIndex() : genesis block not accepted");
1489 }
1490
1491 return true;
1492 }
1493
1494
1495
     CALLED FROM  CMyApp::OnInit2 ui.cpp p. 516
1496 void PrintBlockTree()
1497 {
1498 // precompute tree structure
1499     map<CBlockIndex*, vector<CBlockIndex*> > mapNext;
1500 for (map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.begin(); mi != mapBlockIndex.end(); ++mi)
1501 {
1502         CBlockIndex* pindex = (*mi).second;
1503         mapNext[pindex->pprev].push_back(pindex);
1504 // test
1505 //while (rand() % 3 == 0)
1506 //    mapNext[pindex->pprev].push_back(pindex);
1507 }
1508
1509     vector<pair<int, CBlockIndex*> > vStack;
1510     vStack.push_back(make_pair(0, pindexGenesisBlock));
1511
1512 int nPrevCol = 0;
1513 while (!vStack.empty())
1514 {
1515 int nCol = vStack.back().first;
1516         CBlockIndex* pindex = vStack.back().second;
1517         vStack.pop_back();
1518
1519 // print split or gap
1520 if (nCol > nPrevCol)
1521 {
1522 for (int i = 0; i < nCol-1; i++)
1523 printf("| ");
1524 printf("|\\\n");
1525 }
1526 else if (nCol < nPrevCol)
1527 {
1528 for (int i = 0; i < nCol; i++)
1529 printf("| ");
1530 printf("|\n");
1531 }
1532         nPrevCol = nCol;
1533
1534 // print columns
1535 for (int i = 0; i < nCol; i++)
1536 printf("| ");
1537
1538 // print item
1539         CBlock block;
1540         block.ReadFromDisk(pindex, true);
1541 printf("%d (%u,%u) %s  %s  tx %d",
1542             pindex->nHeight,
```

```
            pindex->nFile,
            pindex->nBlockPos,
            block.GetHash().ToString().substr(0,14).c_str(),
DateTimeStr(block.nTime).c_str(),
            block.vtx.size());

CRITICAL_BLOCK(cs_mapWallet)
{
if (mapWallet.count(block.vtx[0].GetHash()))
{
                CWalletTx& wtx = mapWallet[block.vtx[0].GetHash()];
printf("    mine:  %d  %d  %d", wtx.GetDepthInMainChain(), wtx.GetBlocksToMaturity(), wtx.GetCredit());
}
}
printf("\n");


// put the main timechain first
        vector<CBlockIndex*>& vNext = mapNext[pindex];
for (int i = 0; i < vNext.size(); i++)
{
if (vNext[i]->pnext)
{
swap(vNext[0], vNext[i]);
break;
}
}

// iterate children
for (int i = 0; i < vNext.size(); i++)
            vStack.push_back(make_pair(nCol+i, vNext[i]));
}
}









////////////////////////////////////////////////////////////////////////////////
//
// Messages
//


```

CALLED FROM ProcessMessage p. 233 · SendMessages p. 238

```
bool AlreadyHave(CTxDB& txdb, const CInv& inv)
{
switch (inv.type)
{
case MSG_TX: return mapTransactions.count(inv.hash) || txdb.ContainsTx(inv.hash);
case MSG_BLOCK: return mapBlockIndex.count(inv.hash) || mapOrphanBlocks.count(inv.hash);
case MSG_REVIEW: return true;
case MSG_PRODUCT: return mapProducts.count(inv.hash);
}
// Don't know what it is, just say we already got one
return true;
}









```

CALLED FROM ThreadMessageHandler2 net.cpp p. 339

```
bool ProcessMessages(CNode* pfrom)
```

```
1612 {
1613     CDataStream& vRecv = pfrom->vRecv;
1614 if (vRecv.empty())
1615 return true;
1616 printf("ProcessMessages(%d bytes)\n", vRecv.size());
1617
1618 //
1619 // Message format
1620 //  (4) message start
1621 //  (12) command
1622 //  (4) size
1623 //  (x) data
1624 //
1625
1626     loop
1627 {
1628 // Scan for message start
1629         CDataStream::iterator pstart = search(vRecv.begin(), vRecv.end(), BEGIN(pchMessageStart),
    END(pchMessageStart));
1630 if (vRecv.end() - pstart < sizeof(CMessageHeader))
1631 {
1632 if (vRecv.size() > sizeof(CMessageHeader))
1633 {
1634 printf("\n\nPROCESSMESSAGE MESSAGESTART NOT FOUND\n\n");
1635                 vRecv.erase(vRecv.begin(), vRecv.end() - sizeof(CMessageHeader));
1636 }
1637 break;
1638 }
1639 if (pstart - vRecv.begin() > 0)
1640 printf("\n\nPROCESSMESSAGE SKIPPED %d BYTES\n\n", pstart - vRecv.begin());
1641         vRecv.erase(vRecv.begin(), pstart);
1642
1643 // Read header
1644         CMessageHeader hdr;
1645         vRecv >> hdr;
1646 if (!hdr.IsValid())
1647 {
1648 printf("\n\nPROCESSMESSAGE: ERRORS IN HEADER %s\n\n\n", hdr.GetCommand().c_str());
1649 continue;
1650 }
1651         string strCommand = hdr.GetCommand();
1652
1653 // Message size
1654 unsigned int nMessageSize = hdr.nMessageSize;
```

Partial-message wait — if the declared payload is larger than bytes received, rewind the header and sleep. A peer can advertise a huge `nMessageSize` and trickle data, pinning memory in `vRecv`. Later versions harden this; v0.1.0 mostly trusts peers.[17]

```
1655 if (nMessageSize > vRecv.size())
1656 {
1657 // Rewind and wait for rest of message
1658 ///// need a mechanism to give up waiting for overlong message size error
1659 printf("MESSAGE-BREAK 2\n");
1660             vRecv.insert(vRecv.begin(), BEGIN(hdr), END(hdr));
1661 Sleep(100);
1662 break;
1663 }
1664
1665 // Copy message to its own buffer
1666         CDataStream vMsg(vRecv.begin(), vRecv.begin() + nMessageSize, vRecv.nType, vRecv.nVersion);
1667         vRecv.ignore(nMessageSize);
1668
1669 // Process message
1670 bool fRet = false;
1671 try
1672 {
1673 CheckForShutdown(2);
1674 CRITICAL_BLOCK(cs_main)
```

```
                fRet = ProcessMessage(pfrom, strCommand, vMsg);
CheckForShutdown(2);
}
CATCH_PRINT_EXCEPTION("ProcessMessage()")
if (!fRet)
printf("ProcessMessage(%s, %d bytes) from %s to %s FAILED\n", strCommand.c_str(), nMessageSize, pfrom->addr.ToString().c_str(), addrLocalHost.ToString().c_str());
}

    vRecv.Compact();
return true;
}
```

CALLED FROM ProcessMessages p. 231

```
bool ProcessMessage(CNode* pfrom, string strCommand, CDataStream& vRecv)
{
static map<unsigned int, vector<unsigned char> > mapReuseKey;
printf("received: %-12s (%d bytes)  ", strCommand.c_str(), vRecv.size());
for (int i = 0; i < min(vRecv.size(), (unsigned int)25); i++)
printf("%02x ", vRecv[i] & 0xff);
printf("\n");
if (nDropMessagesTest > 0 && GetRand(nDropMessagesTest) == 0)
{
printf("dropmessages DROPPING RECV MESSAGE\n");
return true;
}
```

`"version"` handshake — first message on a connection; anything else is rejected until this succeeds. Negotiates protocol version, services bits, and peer time.[17]

```
if (strCommand == "version")
{
// Can only do this once
if (pfrom->nVersion != 0)
return false;

        int64 nTime;
        CAddress addrMe;
        vRecv >> pfrom->nVersion >> pfrom->nServices >> nTime >> addrMe;
if (pfrom->nVersion == 0)
return false;

        pfrom->vSend.SetVersion(min(pfrom->nVersion, VERSION));
        pfrom->vRecv.SetVersion(min(pfrom->nVersion, VERSION));
```

`fClient` / `SER_BLOCKHEADERONLY` — peers without `NODE_NETWORK` are treated as lightweight clients and only receive block headers, not full blocks.[17]

```
        pfrom->fClient = !(pfrom->nServices & NODE_NETWORK);
if (pfrom->fClient)
{
            pfrom->vSend.nType |= SER_BLOCKHEADERONLY;
            pfrom->vRecv.nType |= SER_BLOCKHEADERONLY;
}
```

`AddTimeData()` — peer clocks influence our adjusted network time. A minority of dishonest peers cannot easily skew the median, but the design still trusts the peer set for time.[17]

```
1727 AddTimeData(pfrom->addr.ip, nTime);
1728
```

`fAskedForBlocks` — ask the *first* full node for `getblocks`, then never again. Jeremy Rubin's annotations point out that a node should cross-check other peers in case this one lies — v0.1.0 does not.[17]

```
1729 // Ask the first connected node for block updates
1730 static bool fAskedForBlocks;
1731 if (!fAskedForBlocks && !pfrom->fClient)
1732 {
1733             fAskedForBlocks = true;
1734             pfrom->PushMessage("getblocks", CBlockLocator(pindexBest), uint256(0));
1735 }
1736
1737 printf("version addrMe = %s\n", addrMe.ToString().c_str());
1738 }
1739
1740
1741 else if (pfrom->nVersion == 0)
1742 {
1743 // Must have a version message before anything else
1744 return false;
1745 }
1746
1747
```

`"addr"` — peer gossip. Incoming addresses are stored via `AddAddress` so the node can reconnect after IRC or restart without rediscovering the whole network.

```
1748 else if (strCommand == "addr")
1749 {
1750         vector<CAddress> vAddr;
1751         vRecv >> vAddr;
1752
1753 // Store the new addresses
1754         CAddrDB addrdb;
1755 foreach(const CAddress& addr, vAddr)
1756 {
1757 if (fShutdown)
1758 return true;
1759 if (AddAddress(addrdb, addr))
1760 {
1761 // Put on lists to send to other nodes
1762                 pfrom->setAddrKnown.insert(addr);
1763 CRITICAL_BLOCK(cs_vNodes)
1764 foreach(CNode* pnode, vNodes)
1765 if (!pnode->setAddrKnown.count(addr))
1766                             pnode->vAddrToSend.push_back(addr);
1767 }
1768 }
1769 }
1770
1771
```

`"inv"` — inventory advertisement. For each unknown `CInv`, call `AskFor`; if we already have an orphan block, request the parent chain with `getblocks`. This is how transactions and blocks flood the mesh without flooding full payloads first.

```
else if (strCommand == "inv")
{
        vector<CInv> vInv;
        vRecv >> vInv;

        CTxDB txdb("r");
foreach(const CInv& inv, vInv)
{
if (fShutdown)
return true;
            pfrom->AddInventoryKnown(inv);

bool fAlreadyHave = AlreadyHave(txdb, inv);
printf("  got inventory: %s  %s\n", inv.ToString().c_str(), fAlreadyHave ? "have" : "new");

if (!fAlreadyHave)
                pfrom->AskFor(inv);
else if (inv.type == MSG_BLOCK && mapOrphanBlocks.count(inv.hash))
                pfrom->PushMessage("getblocks", CBlockLocator(pindexBest),
    GetOrphanRoot(mapOrphanBlocks[inv.hash]));
}
}


```

`"getdata"` — fetch request. Serve blocks from disk (headers-only for lightweight clients) or relayed objects from `mapRelay`. Peers pull after `inv`; they are never pushed full blocks unsolicited in this design.

```
else if (strCommand == "getdata")
{
        vector<CInv> vInv;
        vRecv >> vInv;

foreach(const CInv& inv, vInv)
{
if (fShutdown)
return true;
printf("received getdata for: %s\n", inv.ToString().c_str());

if (inv.type == MSG_BLOCK)
{
// Send block from disk
                map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(inv.hash);
if (mi != mapBlockIndex.end())
{
//// could optimize this to send header straight from blockindex for client
                    CBlock block;
                    block.ReadFromDisk((*mi).second, !pfrom->fClient);
                    pfrom->PushMessage("block", block);
}
}
else if (inv.IsKnownType())
{
// Send stream from relay memory
CRITICAL_BLOCK(cs_mapRelay)
{
                    map<CInv, CDataStream>::iterator mi = mapRelay.find(inv);
if (mi != mapRelay.end())
                        pfrom->PushMessage(inv.GetCommand(), (*mi).second);
}
```

```
}
}
}


```

"getblocks" — sync handshake. Locator finds a common ancestor; we queue inv entries for subsequent best-chain blocks until hashStop. Exponential locator spacing keeps the message small even for long histories.

```
else if (strCommand == "getblocks")
{
        CBlockLocator locator;
        uint256 hashStop;
        vRecv >> locator >> hashStop;

// Find the first block the caller has in the main chain
        CBlockIndex* pindex = locator.GetBlockIndex();

// Send the rest of the chain
if (pindex)
            pindex = pindex->pnext;
printf("getblocks %d to %s\n", (pindex ? pindex->nHeight : -1),
hashStop.ToString().substr(0,14).c_str());
for (; pindex; pindex = pindex->pnext)
{
if (pindex->GetBlockHash() == hashStop)
{
printf("  getblocks stopping at %d %s\n", pindex->nHeight, pindex-
>GetBlockHash().ToString().substr(0,14).c_str());
break;
}

// Bypass setInventoryKnown in case an inventory message got lost
CRITICAL_BLOCK(pfrom->cs_inventory)
{
                CInv inv(MSG_BLOCK, pindex->GetBlockHash());
// returns true if wasn't already contained in the set
if (pfrom->setInventoryKnown2.insert(inv).second)
{
                    pfrom->setInventoryKnown.erase(inv);
                    pfrom->vInventoryToSend.push_back(inv);
}
}
}
}


```

"tx" — full transaction payload. Validate, accept into the memory pool / wallet paths, relay to peers, and recursively process any orphan txs that were waiting on this one's outputs.

```
else if (strCommand == "tx")
{
        vector<uint256> vWorkQueue;
        CDataStream vMsg(vRecv);
        CTransaction tx;
        vRecv >> tx;

        CInv inv(MSG_TX, tx.GetHash());
        pfrom->AddInventoryKnown(inv);

bool fMissingInputs = false;
if (tx.AcceptTransaction(true, &fMissingInputs))
{
AddToWalletIfMine(tx, NULL);
```

```
RelayMessage(inv, vMsg);
            mapAlreadyAskedFor.erase(inv);
            vWorkQueue.push_back(inv.hash);

// Recursively process any orphan transactions that depended on this one
for (int i = 0; i < vWorkQueue.size(); i++)
{
                uint256 hashPrev = vWorkQueue[i];
for (multimap<uint256, CDataStream*>::iterator mi = mapOrphanTransactionsByPrev.lower_bound(hashPrev);
                     mi != mapOrphanTransactionsByPrev.upper_bound(hashPrev);
++mi)
{
const CDataStream& vMsg = *((*mi).second);
                    CTransaction tx;
CDataStream(vMsg) >> tx;
                    CInv inv(MSG_TX, tx.GetHash());

if (tx.AcceptTransaction(true))
{
printf("   accepted orphan tx %s\n", inv.hash.ToString().substr(0,6).c_str());
AddToWalletIfMine(tx, NULL);
RelayMessage(inv, vMsg);
                        mapAlreadyAskedFor.erase(inv);
                        vWorkQueue.push_back(inv.hash);
}
}
}

foreach(uint256 hash, vWorkQueue)
EraseOrphanTx(hash);
}
else if (fMissingInputs)
{
printf("storing orphan tx %s\n", inv.hash.ToString().substr(0,6).c_str());
AddOrphanTx(vMsg);
}
}


```

"review" — marketplace reputation message. Signed attestations with stars and atoms for the unfinished eBay-style system; relayed if AcceptReview() succeeds.[17]

```
else if (strCommand == "review")
{
        CDataStream vMsg(vRecv);
        CReview review;
        vRecv >> review;

        CInv inv(MSG_REVIEW, review.GetHash());
        pfrom->AddInventoryKnown(inv);

if (review.AcceptReview())
{
// Relay the original message as-is in case it's a higher version than we know how to parse
RelayMessage(inv, vMsg);
            mapAlreadyAskedFor.erase(inv);
}
}


```

"block" — full block payload. Deserialized then handed to ProcessBlock. Debug print() of every received block is left in — early-network forensics gold.

```
else if (strCommand == "block")
{
```

```
        auto_ptr<CBlock> pblock(new CBlock);
        vRecv >> *pblock;

//// debug print
printf("received block:\n"); pblock->print();

        CInv inv(MSG_BLOCK, pblock->GetHash());
        pfrom->AddInventoryKnown(inv);

if (ProcessBlock(pfrom, pblock.release()))
            mapAlreadyAskedFor.erase(inv);
}


```

"getaddr" — request recent peer addresses (last hour). Keeps the addr database fresh without relying solely on IRC.

```
else if (strCommand == "getaddr")
{
        pfrom->vAddrToSend.clear();
//// need to expand the time range if not enough found
        int64 nSince = GetAdjustedTime() - 60 * 60; // in the last hour
CRITICAL_BLOCK(cs_mapAddresses)
{
foreach(const PAIRTYPE(vector<unsigned char>, CAddress)& item, mapAddresses)
{
if (fShutdown)
return true;
const CAddress& addr = item.second;
if (addr.nTime > nSince)
                    pfrom->vAddrToSend.push_back(addr);
}
}
}


```

"checkorder" — IP-to-IP payment handshake. Peer proposes an order; we return approval plus a fresh pubkey (reused per IP until spent). Deprecated with the marketplace/payment UX.[17]

```
else if (strCommand == "checkorder")
{
        uint256 hashReply;
        CWalletTx order;
        vRecv >> hashReply >> order;

/// we have a chance to check the order here

// Keep giving the same key to the same ip until they use it
if (!mapReuseKey.count(pfrom->addr.ip))
            mapReuseKey[pfrom->addr.ip] = GenerateNewKey();

// Send back approval of order and pubkey to use
        CScript scriptPubKey;
        scriptPubKey << mapReuseKey[pfrom->addr.ip] << OP_CHECKSIG;
        pfrom->PushMessage("reply", hashReply, (int)0, scriptPubKey);
}


```

`"submitorder"` — after checkorder, the payer sends the signed wallet tx. We accept into the wallet, relay, and `"reply"` success or failure.[17]

```
1993 else if (strCommand == "submitorder")
1994 {
1995         uint256 hashReply;
1996         CWalletTx wtxNew;
1997         vRecv >> hashReply >> wtxNew;
1998
1999 // Broadcast
2000 if (!wtxNew.AcceptWalletTransaction())
2001 {
2002             pfrom->PushMessage("reply", hashReply, (int)1);
2003 return error("submitorder AcceptWalletTransaction() failed, returning error 1");
2004 }
2005         wtxNew.fTimeReceivedIsTxTime = true;
2006 AddToWallet(wtxNew);
2007         wtxNew.RelayWalletTransaction();
2008         mapReuseKey.erase(pfrom->addr.ip);
2009
2010 // Send back confirmation
2011         pfrom->PushMessage("reply", hashReply, (int)0);
2012 }
2013
2014
```

`"reply"` — dynamic dispatch: a nonce routes the payload to a pending `CRequestTracker` callback. Request/response over the P2P socket without a separate RPC channel.[17]

```
2015 else if (strCommand == "reply")
2016 {
2017         uint256 hashReply;
2018         vRecv >> hashReply;
2019
2020         CRequestTracker tracker;
2021 CRITICAL_BLOCK(pfrom->cs_mapRequests)
2022 {
2023             map<uint256, CRequestTracker>::iterator mi = pfrom->mapRequests.find(hashReply);
2024 if (mi != pfrom->mapRequests.end())
2025 {
2026                 tracker = (*mi).second;
2027                 pfrom->mapRequests.erase(mi);
2028 }
2029 }
2030 if (!tracker.IsNull())
2031             tracker.fn(tracker.param1, vRecv);
2032 }
2033
2034
2035 else
2036 {
2037 // Ignore unknown commands for extensibility
2038 printf("ProcessMessage(%s) : Ignored unknown message\n", strCommand.c_str());
2039 }
2040
2041
2042 if (!vRecv.empty())
2043 printf("ProcessMessage(%s) : %d extra bytes\n", strCommand.c_str(), vRecv.size());
2044
2045 return true;
2046 }
2047
2048
2049
2050
2051
```

```
2052
2053
2054
2055
    CALLED FROM  ThreadMessageHandler2 net.cpp p. 339
2056 bool SendMessages(CNode* pto)
2057 {
2058 CheckForShutdown(2);
2059 CRITICAL_BLOCK(cs_main)
2060 {
2061 // Don't send anything until we get their version message
2062 if (pto->nVersion == 0)
2063 return true;
2064
2065
2066 //
2067 // Message: addr
2068 //
2069         vector<CAddress> vAddrToSend;
2070         vAddrToSend.reserve(pto->vAddrToSend.size());
2071 foreach(const CAddress& addr, pto->vAddrToSend)
2072 if (!pto->setAddrKnown.count(addr))
2073                 vAddrToSend.push_back(addr);
2074         pto->vAddrToSend.clear();
2075 if (!vAddrToSend.empty())
2076             pto->PushMessage("addr", vAddrToSend);
2077
2078
2079 //
2080 // Message: inventory
2081 //
2082         vector<CInv> vInventoryToSend;
2083 CRITICAL_BLOCK(pto->cs_inventory)
2084 {
2085             vInventoryToSend.reserve(pto->vInventoryToSend.size());
2086 foreach(const CInv& inv, pto->vInventoryToSend)
2087 {
2088 // returns true if wasn't already contained in the set
2089 if (pto->setInventoryKnown.insert(inv).second)
2090                     vInventoryToSend.push_back(inv);
2091 }
2092             pto->vInventoryToSend.clear();
2093             pto->setInventoryKnown2.clear();
2094 }
2095 if (!vInventoryToSend.empty())
2096             pto->PushMessage("inv", vInventoryToSend);
2097
2098
2099 //
2100 // Message: getdata
2101 //
2102         vector<CInv> vAskFor;
2103         int64 nNow = GetTime() * 1000000;
2104         CTxDB txdb("r");
2105 while (!pto->mapAskFor.empty() && (*pto->mapAskFor.begin()).first <= nNow)
2106 {
2107 const CInv& inv = (*pto->mapAskFor.begin()).second;
2108 printf("sending getdata: %s\n", inv.ToString().c_str());
2109 if (!AlreadyHave(txdb, inv))
2110                 vAskFor.push_back(inv);
2111             pto->mapAskFor.erase(pto->mapAskFor.begin());
2112 }
2113 if (!vAskFor.empty())
2114             pto->PushMessage("getdata", vAskFor);
2115
2116 }
2117 return true;
2118 }
2119
2120
2121
```

```
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133 //////////////////////////////////////////////////////////////////////////////
2134 //
2135 // BitcoinMiner
2136 //
2137
```

CALLED FROM BitcoinMiner p. 242

```
2138 int FormatHashBlocks(void* pbuffer, unsigned int len)
2139 {
2140 unsigned char* pdata = (unsigned char*)pbuffer;
2141 unsigned int blocks = 1 + ((len + 8) / 64);
2142 unsigned char* pend = pdata + 64 * blocks;
2143 memset(pdata + len, 0, 64 * blocks - len);
2144     pdata[len] = 0x80;
2145 unsigned int bits = len * 8;
2146     pend[-1] = (bits >> 0) & 0xff;
2147     pend[-2] = (bits >> 8) & 0xff;
2148     pend[-3] = (bits >> 16) & 0xff;
2149     pend[-4] = (bits >> 24) & 0xff;
2150 return blocks;
2151 }
2152
2153 using CryptoPP::ByteReverse;
2154 static int detectlittleendian = 1;
2155
```

CALLED FROM BitcoinMiner p. 243

```
2156 void BlockSHA256(const void* pin, unsigned int nBlocks, void* pout)
2157 {
2158 unsigned int* pinput = (unsigned int*)pin;
2159 unsigned int* pstate = (unsigned int*)pout;
2160
2161     CryptoPP::SHA256::InitState(pstate);
2162
2163 if (*(char*)&detectlittleendian != 0)
2164 {
2165 for (int n = 0; n < nBlocks; n++)
2166 {
2167 unsigned int pbuf[16];
2168 for (int i = 0; i < 16; i++)
2169                 pbuf[i] = ByteReverse(pinput[n * 16 + i]);
2170             CryptoPP::SHA256::Transform(pstate, pbuf);
2171 }
2172 for (int i = 0; i < 8; i++)
2173             pstate[i] = ByteReverse(pstate[i]);
2174 }
2175 else
2176 {
2177 for (int n = 0; n < nBlocks; n++)
2178             CryptoPP::SHA256::Transform(pstate, pinput + n * 16);
2179 }
2180 }
2181
2182
```

`BitcoinMiner()` - the mining function. Satoshi envisioned 'one-CPU-one-vote' democracy. In 2009, anyone's laptop could mine blocks. Forensics of the first ~170 blocks show how fragile that democracy was: long gaps between blocks imply the network sometimes halted entirely when Satoshi stopped mining.[1011]

```
    CALLED FROM  ThreadBitcoinMiner net.cpp p. 339
2183 bool BitcoinMiner()
2184 {
2185 printf("BitcoinMiner started\n");
```

`THREAD_PRIORITY_LOWEST` - mining runs in background. Satoshi wanted mining to not interfere with normal computer use.

```
2186 SetThreadPriority(GetCurrentThread(), THREAD_PRIORITY_LOWEST);
2187
2188     CKey key;
2189     key.MakeNewKey();
```

`bnExtraNonce = 0` — initialized *once*, outside the `while (fGenerateBitcoins)` loop. It does not reset after a block is found; only a client restart zeroes it. Lerner's Patoshi analysis treats that free-running counter as privacy flaw #1: consecutive coinbases from the same process form a rising ExtraNonce timeline that links ~22k early blocks to one miner.[201017]

```
2190     CBigNum bnExtraNonce = 0;
2191 while (fGenerateBitcoins)
2192 {
2193 Sleep(50);
2194 CheckForShutdown(3);
```

`while (vNodes.empty())` — the miner refuses to work until at least one peer is connected. Alone, a node would mine an orphan chain nobody else sees. In the first days that check mattered: Hal Finney's `debug.log` shows he joined near block 49 and saw only two other peers — both consistent with Satoshi running multiple nodes.[2110]

```
2195 while (vNodes.empty())
2196 {
2197 Sleep(1000);
2198 CheckForShutdown(3);
2199 }
2200
2201 unsigned int nTransactionsUpdatedLast = nTransactionsUpdated;
2202         CBlockIndex* pindexPrev = pindexBest;
```

`GetNextWorkRequired()` - fetches the current compact target (`nBits`). Miner must find `hash ≤ target`. Higher difficulty = smaller target = harder.

```
2203 unsigned int nBits = GetNextWorkRequired(pindexPrev);
2204
2205
2206 //
2207 // Create coinbase tx
2208 //
2209         CTransaction txNew;
2210         txNew.vin.resize(1);
```

`prevout.SetNull()`—the signature of money creation. Normal transactions must reference a previous output to spend; coinbase transactions reference nothing because they create coins from thin air. `SetNull()` sets `hash=0` and `index=-1`, a combination that can never occur naturally (no real transaction has hash 0). This null reference is how nodes identify coinbase transactions. It's the only way new bitcoin enters circulation—every satoshi in existence traces back through a chain of transactions to some block's coinbase with this null `prevout`.

```
2211         txNew.vin[0].prevout.SetNull();
```

`++bnExtraNonce` in the coinbase `scriptSig` — runs whenever this process builds a new template: after `nNonce` overflow, after the tip moves, or when the mempool changes enough to rebuild. Fresh ExtraNonce entropy changes the merkle root and expands the search space beyond 2^{32} header nonces. For stock v0.1.0 miners, each new network tip forces a rebuild, so ExtraNonce often advances roughly once per block seen (Lerner's privacy flaw #3)—which is why non-Patoshi slopes track the chain, while Patoshi's denser slope stands out.[2011]

```
2212         txNew.vin[0].scriptSig << nBits << ++bnExtraNonce;
2213         txNew.vout.resize(1);
2214         txNew.vout[0].scriptPubKey << key.GetPubKey() << OP_CHECKSIG;
2215
2216
2217 //
2218 // Create new block
2219 //
2220         auto_ptr<CBlock> pblock(new CBlock());
2221 if (!pblock.get())
2222 return false;
2223
2224 // Add our coinbase tx as first transaction
2225         pblock->vtx.push_back(txNew);
2226
2227 // Collect the latest transactions into the block
2228         int64 nFees = 0;
2229 CRITICAL_BLOCK(cs_main)
2230 CRITICAL_BLOCK(cs_mapTransactions)
2231 {
2232             CTxDB txdb("r");
2233             map<uint256, CTxIndex> mapTestPool;
2234             vector<char> vfAlreadyAdded(mapTransactions.size());
2235 bool fFoundSomething = true;
2236 unsigned int nBlockSize = 0;
```

Template packing loop stops around `MAX_SIZE/2` (~16 MB). But the size is only checked between full passes over the mempool: a single pass of the inner loop adds every eligible transaction with no size check, so `nBlockSize` can overshoot the limit, even past `MAX_SIZE` — a latent packing bug noted in annotated v0.1.0 walkthroughs.[17]

```
2237 while (fFoundSomething && nBlockSize < MAX_SIZE/2)
2238 {
2239                 fFoundSomething = false;
2240 unsigned int n = 0;
2241 for (map<uint256, CTransaction>::iterator mi = mapTransactions.begin(); mi != mapTransactions.end();
     ++mi, ++n)
2242 {
2243 if (vfAlreadyAdded[n])
2244 continue;
2245                     CTransaction& tx = (*mi).second;
2246 if (tx.IsCoinBase() || !tx.IsFinal())
2247 continue;
2248
```

```
// Transaction fee requirements, mainly only needed for flood control
// Under 10K (about 80 inputs) is free for first 100 transactions
// Base rate is 0.01 per KB
```

Satoshi's fee discount for early transactions. The boolean `pblock->vtx.size() < 100` grants a discount to the first 100 transactions in a block: if under 10KB, they're free. After that, every transaction pays 0.01 BTC plus 0.01 BTC per full KB. This was flood control—allow normal usage for free, but charge when blocks fill up. Satoshi later wrote: 'Free transactions are nice and we can keep it that way if people don't abuse them.'[14] The thresholds of 100 transactions and 10KB were educated guesses; modern Bitcoin uses more sophisticated fee markets where users bid for block space.

```
                    int64 nMinFee = tx.GetMinFee(pblock->vtx.size() < 100);

                    map<uint256, CTxIndex> mapTestPoolTmp(mapTestPool);
if (!tx.ConnectInputs(txdb, mapTestPoolTmp, CDiskTxPos(1,1,1), 0, nFees, false, true, nMinFee))
continue;
swap(mapTestPool, mapTestPoolTmp);

                    pblock->vtx.push_back(tx);
                    nBlockSize += ::GetSerializeSize(tx, SER_NETWORK);
                    vfAlreadyAdded[n] = true;
                    fFoundSomething = true;
}
}
}
        pblock->nBits = nBits;
```

`GetBlockValue(nFees)` - calculates reward. Block subsidy plus all transaction fees. Fees incentivize miners after subsidy halves to zero.

```
        pblock->vtx[0].vout[0].nValue = pblock->GetBlockValue(nFees);
printf("\n\nRunning BitcoinMiner with %d transactions in block\n", pblock->vtx.size());


//
// Prebuild hash buffer
//
struct unnamed1
{
struct unnamed2
{
int nVersion;
                uint256 hashPrevBlock;
                uint256 hashMerkleRoot;
unsigned int nTime;
unsigned int nBits;
unsigned int nNonce;
}
            block;
unsigned char pchPadding0[64];
            uint256 hash1;
unsigned char pchPadding1[64];
}
        tmp;

        tmp.block.nVersion       = pblock->nVersion;
        tmp.block.hashPrevBlock  = pblock->hashPrevBlock  = (pindexPrev ? pindexPrev->GetBlockHash() : 0);
        tmp.block.hashMerkleRoot = pblock->hashMerkleRoot = pblock->BuildMerkleTree();
        tmp.block.nTime          = pblock->nTime          = max((pindexPrev ? pindexPrev-
>GetMedianTimePast()+1 : 0), GetAdjustedTime());
        tmp.block.nBits          = pblock->nBits          = nBits;
        tmp.block.nNonce         = pblock->nNonce         = 1;

unsigned int nBlocks0 = FormatHashBlocks(&tmp.block, sizeof(tmp.block));
```

```
unsigned int nBlocks1 = FormatHashBlocks(&tmp.hash1, sizeof(tmp.hash1));


//
// Search
//
unsigned int nStart = GetTime();
```

`hashTarget` - the PoW threshold decoded from `nBits`. Hash must be ≤ this 256-bit number. Lower target = harder puzzle.

```
        uint256 hashTarget = CBigNum().SetCompact(pblock->nBits).getuint256();
        uint256 hash;
```

The mining loop. Increment `nNonce`, double-SHA256 the header, compare to target. Every `0x40000` nonces (roughly every second on a 2009 CPU) the inner loop also refreshes `nTime` and checks whether the tip moved—see below.

```
        loop
{
```

`BlockSHA256()` - optimized SHA-256 for mining. Called hundreds of thousands of times per second on a 2009 CPU; Satoshi's own hashmeter later logged '2 CPUs 799 khash/s.'[18] Performance here directly affects mining revenue.

```
BlockSHA256(&tmp.block, nBlocks0, &tmp.hash1);
BlockSHA256(&tmp.hash1, nBlocks1, &hash);


```

`hash <= hashTarget` - FOUND IT! This comparison is why mining works: finding a hash ≤ target is rare and proves work was done.

```
if (hash <= hashTarget)
{
                pblock->nNonce = tmp.block.nNonce;
assert(hash == pblock->GetHash());

//// debug print
printf("BitcoinMiner:\n");
```

'proof-of-work found' - victory message. Satoshi: 'The proof-of-work also solves the problem of determining representation in majority decision making.'

```
printf("proof-of-work found  \n  hash: %s  \ntarget: %s\n", hash.GetHex().c_str(),
hashTarget.GetHex().c_str());
                    pblock->print();

SetThreadPriority(GetCurrentThread(), THREAD_PRIORITY_NORMAL);
CRITICAL_BLOCK(cs_main)
{
// Save key
if (!AddKey(key))
return false;
                    key.MakeNewKey();

// Process this block the same as if we had received it from another node
if (!ProcessBlock(NULL, pblock.release()))
printf("ERROR in BitcoinMiner, ProcessBlock, block not accepted\n");
```

```
}
SetThreadPriority(GetCurrentThread(), THREAD_PRIORITY_LOWEST);

Sleep(500);
break;
}

// Update nTime every few seconds
```

`(++nNonce & 0x3fff) == 0` — throttle: only every 2^{18} nonces does the miner leave the hot path. That cadence (roughly once a second on period hardware) is where timestamps catch up and where tip/mempool abort checks run. Lerner uses these lagged timestamps in the clock-inversion argument: a single machine almost never publishes a child with an earlier `nTime` than its own parent—and across the first 50k blocks, Patoshi→Patoshi inversions are zero.[20]

```
if ((++tmp.block.nNonce & 0x3fff) == 0)
{
CheckForShutdown(3);
```

`nNonce == 0` after the increment — 32-bit overflow. Break, rebuild the template, and `++bnExtraNonce`. At the initial target (~2^{32} expected hashes per block), overflow was uncommon but real; ExtraNonce exists so mining can continue past a full nonce sweep. As Satoshi put it: 'Whenever the 32-bit nonce starts over at 1, bnExtraNonce gets incremented, which is an arbitrary precision integer.'[18]

```
if (tmp.block.nNonce == 0)
break;
```

`pindexPrev != pindexBest` — another miner's block (or our own, after `ProcessBlock`) moved the tip. Abort this template; the outer loop will mint a fresh coinbase and bump ExtraNonce. That tip-driven rebuild is how stock clients imprint 'blocks seen' onto ExtraNonce slopes.[20]

```
if (pindexPrev != pindexBest)
break;
if (nTransactionsUpdated != nTransactionsUpdatedLast && GetTime() - nStart > 60)
break;
if (!fGenerateBitcoins)
break;
```

Refresh `nTime` from `GetAdjustedTime()` (floored by median-time-past+1). Timestamps are *not* updated every hash—only on this throttle—so a slow parent can leave a child looking slightly 'early.' Combined with peer median-time adjustment, that explains most observed timestamp inversions between *different* miners.[20]

```
                tmp.block.nTime = pblock->nTime = max(pindexPrev->GetMedianTimePast()+1,
GetAdjustedTime());
}
}
}

return true;
}
```

```
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380 //////////////////////////////////////////////////////////////////////////////
2381 //
2382 // Actions
2383 //
2384
2385
     CALLED FROM  SendMoney p. 249 · CMainFrame::CMainFrame ui.cpp p. 480 ·
     CMainFrame::OnPaintListCtrl ui.cpp p. 488 · CSendDialog::OnButtonSend ui.cpp p. 496 ·
     CSendingDialog::StartTransfer ui.cpp p. 499 · CSendingDialog::OnReply2 ui.cpp p. 501 ·  and 2 more
2386 int64 GetBalance()
2387 {
2388     int64 nStart, nEnd;
2389 QueryPerformanceCounter((LARGE_INTEGER*)&nStart);
2390
2391     int64 nTotal = 0;
2392 CRITICAL_BLOCK(cs_mapWallet)
2393 {
2394 for (map<uint256, CWalletTx>::iterator it = mapWallet.begin(); it != mapWallet.end(); ++it)
2395 {
2396             CWalletTx* pcoin = &(*it).second;
2397 if (!pcoin->IsFinal() || pcoin->fSpent)
2398 continue;
2399             nTotal += pcoin->GetCredit();
2400 }
2401 }
2402
2403 QueryPerformanceCounter((LARGE_INTEGER*)&nEnd);
2404 ///printf(" GetBalance() time = %16I64d\n", nEnd - nStart);
2405 return nTotal;
2406 }
2407
2408
2409
```

`SelectCoins()` — coin selection, working on whole wallet transactions rather than individual outputs. An exact single match wins outright. If the smaller coins together can't reach the target, take the smallest larger coin. Otherwise, run 1,000 rounds of a randomized subset-sum search and keep whichever choice — best subset or single larger coin — overshoots less (ties go to the single coin). An early ancestor of Bitcoin Core's later coin-selection logic.

```
     CALLED FROM  CreateTransaction p. 247
2410 bool SelectCoins(int64 nTargetValue, set<CWalletTx*>& setCoinsRet)
2411 {
2412     setCoinsRet.clear();
2413
2414 // List of values less than target
2415     int64 nLowestLarger = _I64_MAX;
2416     CWalletTx* pcoinLowestLarger = NULL;
2417     vector<pair<int64, CWalletTx*> > vValue;
2418     int64 nTotalLower = 0;
2419
2420 CRITICAL_BLOCK(cs_mapWallet)
2421 {
2422 for (map<uint256, CWalletTx>::iterator it = mapWallet.begin(); it != mapWallet.end(); ++it)
2423 {
2424             CWalletTx* pcoin = &(*it).second;
```

```
2425 if (!pcoin→IsFinal() || pcoin→fSpent)
2426 continue;
2427             int64 n = pcoin→GetCredit();
2428 if (n <= 0)
2429 continue;
2430 if (n < nTargetValue)
2431 {
2432                 vValue.push_back(make_pair(n, pcoin));
2433                 nTotalLower += n;
2434 }
2435 else if (n == nTargetValue)
2436 {
2437                 setCoinsRet.insert(pcoin);
2438 return true;
2439 }
2440 else if (n < nLowestLarger)
2441 {
2442                 nLowestLarger = n;
2443                 pcoinLowestLarger = pcoin;
2444 }
2445 }
2446 }
2447
2448 if (nTotalLower < nTargetValue)
2449 {
2450 if (pcoinLowestLarger == NULL)
2451 return false;
2452         setCoinsRet.insert(pcoinLowestLarger);
2453 return true;
2454 }
2455
2456 // Solve subset sum by stochastic approximation
2457 sort(vValue.rbegin(), vValue.rend());
2458     vector<char> vfIncluded;
2459     vector<char> vfBest(vValue.size(), true);
2460     int64 nBest = nTotalLower;
2461
2462 for (int nRep = 0; nRep < 1000 && nBest != nTargetValue; nRep++)
2463 {
2464         vfIncluded.assign(vValue.size(), false);
2465         int64 nTotal = 0;
2466 bool fReachedTarget = false;
2467 for (int nPass = 0; nPass < 2 && !fReachedTarget; nPass++)
2468 {
2469 for (int i = 0; i < vValue.size(); i++)
2470 {
2471 if (nPass == 0 ? rand() % 2 : !vfIncluded[i])
2472 {
2473                     nTotal += vValue[i].first;
2474                     vfIncluded[i] = true;
2475 if (nTotal >= nTargetValue)
2476 {
2477                         fReachedTarget = true;
2478 if (nTotal < nBest)
2479 {
2480                             nBest = nTotal;
2481                             vfBest = vfIncluded;
2482 }
2483                         nTotal -= vValue[i].first;
2484                         vfIncluded[i] = false;
2485 }
2486 }
2487 }
2488 }
2489 }
2490
2491 // If the next larger is still closer, return it
2492 if (pcoinLowestLarger && nLowestLarger - nTargetValue <= nBest - nTargetValue)
2493         setCoinsRet.insert(pcoinLowestLarger);
2494 else
2495 {
2496 for (int i = 0; i < vValue.size(); i++)
```

```
2497 if (vfBest[i])
2498                 setCoinsRet.insert(vValue[i].second);
2499
2500 //// debug print
2501 printf("SelectCoins() best subset: ");
2502 for (int i = 0; i < vValue.size(); i++)
2503 if (vfBest[i])
2504 printf("%s ", FormatMoney(vValue[i].first).c_str());
2505 printf("total %s\n", FormatMoney(nBest).c_str());
2506 }
2507
2508 return true;
2509 }
2510
2511
2512
2513
```

`CreateTransaction()` — build a payment: select coins, add change back to the key of one of the selected coins, sign inputs, check the fee. If the fee is below `GetMinFee()`, it raises the fee and rebuilds the transaction itself; `nFeeRequiredRet` reports that fee so `SendMoney()` can explain a failure.

```
    CALLED FROM  SendMoney p. 249 · CSendingDialog::OnReply2 ui.cpp p. 501
2514 bool CreateTransaction(CScript scriptPubKey, int64 nValue, CWalletTx& wtxNew, int64& nFeeRequiredRet)
2515 {
2516     nFeeRequiredRet = 0;
2517 CRITICAL_BLOCK(cs_main)
2518 {
2519 // txdb must be opened before the mapWallet lock
2520         CTxDB txdb("r");
2521 CRITICAL_BLOCK(cs_mapWallet)
2522 {
2523             int64 nFee = nTransactionFee;
2524             loop
2525 {
2526                 wtxNew.vin.clear();
2527                 wtxNew.vout.clear();
2528 if (nValue < 0)
2529 return false;
2530                 int64 nValueOut = nValue;
2531                 nValue += nFee;
2532
2533 // Choose coins to use
2534                 set<CWalletTx*> setCoins;
2535 if (!SelectCoins(nValue, setCoins))
2536 return false;
2537                 int64 nValueIn = 0;
2538 foreach(CWalletTx* pcoin, setCoins)
2539                     nValueIn += pcoin->GetCredit();
2540
2541 // Fill vout[0] to the payee
2542                 wtxNew.vout.push_back(CTxOut(nValueOut, scriptPubKey));
2543
2544 // Fill vout[1] back to self with any change
2545 if (nValueIn > nValue)
2546 {
2547 // Use the same key as one of the coins
2548                     vector<unsigned char> vchPubKey;
2549                     CTransaction& txFirst = *(*setCoins.begin());
2550 foreach(const CTxOut& txout, txFirst.vout)
2551 if (txout.IsMine())
2552 if (ExtractPubKey(txout.scriptPubKey, true, vchPubKey))
2553 break;
2554 if (vchPubKey.empty())
2555 return false;
2556
2557 // Fill vout[1] to ourself
```

```
2558                    CScript scriptPubKey;
2559                    scriptPubKey << vchPubKey << OP_CHECKSIG;
2560                    wtxNew.vout.push_back(CTxOut(nValueIn - nValue, scriptPubKey));
2561 }
2562
2563 // Fill vin
2564 foreach(CWalletTx* pcoin, setCoins)
2565 for (int nOut = 0; nOut < pcoin->vout.size(); nOut++)
2566 if (pcoin->vout[nOut].IsMine())
2567                             wtxNew.vin.push_back(CTxIn(pcoin->GetHash(), nOut));
2568
2569 // Sign
2570 int nIn = 0;
2571 foreach(CWalletTx* pcoin, setCoins)
2572 for (int nOut = 0; nOut < pcoin->vout.size(); nOut++)
2573 if (pcoin->vout[nOut].IsMine())
2574 SignSignature(*pcoin, wtxNew, nIn++);
2575
2576 // Check that enough fee is included
2577 if (nFee < wtxNew.GetMinFee(true))
2578 {
2579                     nFee = nFeeRequiredRet = wtxNew.GetMinFee(true);
2580 continue;
2581 }
2582
2583 // Fill vtxPrev by copying from previous transactions vtxPrev
2584                 wtxNew.AddSupportingTransactions(txdb);
2585                 wtxNew.fTimeReceivedIsTxTime = true;
2586
2587 break;
2588 }
2589 }
2590 }
2591 return true;
2592 }
2593
2594 // Call after CreateTransaction unless you want to abort
    CALLED FROM  SendMoney p. 249 · CSendingDialog::OnReply2 ui.cpp p. 501
2595 bool CommitTransactionSpent(const CWalletTx& wtxNew)
2596 {
2597 CRITICAL_BLOCK(cs_main)
2598 CRITICAL_BLOCK(cs_mapWallet)
2599 {
2600 //// todo: make this transactional, never want to add a transaction
2601 ////  without marking spent transactions
2602
2603 // Add tx to wallet, because if it has change it's also ours,
2604 // otherwise just for transaction history.
2605 AddToWallet(wtxNew);
2606
2607 // Mark old coins as spent
2608         set<CWalletTx*> setCoins;
2609 foreach(const CTxIn& txin, wtxNew.vin)
2610             setCoins.insert(&mapWallet[txin.prevout.hash]);
2611 foreach(CWalletTx* pcoin, setCoins)
2612 {
2613             pcoin->fSpent = true;
2614             pcoin->WriteToDisk();
2615             vWalletUpdated.push_back(make_pair(pcoin->GetHash(), false));
2616 }
2617 }
2618 MainFrameRepaint();
2619 return true;
2620 }
2621
2622
2623
2624
```

`SendMoney()` — end-to-end spend: create, record in the wallet and mark spent coins, accept into the memory pool, relay. The function every "Send" button ultimately calls.

CALLED FROM `CSendDialog::OnButtonSend` ui.cpp p. 497 · `ThreadRandSendTest` ui.cpp p. 519

```
2625 bool SendMoney(CScript scriptPubKey, int64 nValue, CWalletTx& wtxNew)
2626 {
2627 CRITICAL_BLOCK(cs_main)
2628 {
2629         int64 nFeeRequired;
2630 if (!CreateTransaction(scriptPubKey, nValue, wtxNew, nFeeRequired))
2631 {
2632             string strError;
2633 if (nValue + nFeeRequired > GetBalance())
2634                 strError = strprintf("Error: This is an oversized transaction that requires a
     transaction fee of %s ", FormatMoney(nFeeRequired).c_str());
2635 else
2636                 strError = "Error: Transaction creation failed ";
2637 wxMessageBox(strError, "Sending...");
2638 return error("SendMoney() : %s\n", strError.c_str());
2639 }
2640 if (!CommitTransactionSpent(wtxNew))
2641 {
2642 wxMessageBox("Error finalizing transaction", "Sending...");
2643 return error("SendMoney() : Error finalizing transaction");
2644 }
2645
2646 printf("SendMoney: %s\n", wtxNew.GetHash().ToString().substr(0,6).c_str());
2647
2648 // Broadcast
2649 if (!wtxNew.AcceptTransaction())
2650 {
2651 // This must not fail. The transaction has already been signed and recorded.
2652 throw runtime_error("SendMoney() : wtxNew.AcceptTransaction() failed\n");
2653 wxMessageBox("Error: Transaction not valid", "Sending...");
2654 return error("SendMoney() : Error: Transaction not valid");
2655 }
2656         wtxNew.RelayWalletTransaction();
2657 }
2658 MainFrameRepaint();
2659 return true;
2660 }
2661
```

SUMMARY

This file embodies Satoshi's central insight: proof-of-work is not merely a rate-limiter (as in Adam Back's Hashcash)[5] but the foundation of distributed consensus itself. Every predecessor—DigiCash,[6] b-money,[7] Bit Gold,[8] RPOW[9]—stumbled on the same problem: how to prevent double-spending without a trusted third party. Satoshi solved it by making dishonesty more expensive than honesty.

The elegance is in the alignment of incentives. Miners profit by following the rules and extending the longest chain. Rewriting history requires outspending all honest miners combined—an attack that becomes exponentially harder with each confirmation. Satoshi calculated in the whitepaper that an attacker controlling 10% of network hashrate has only a 0.024% chance of catching up after six blocks.[3] This is why "six confirmations" became the industry standard for settlement finality.

This code has been stress-tested under the most adversarial conditions imaginable: nation-state pressure, hundreds of billions of dollars at stake, and relentless scrutiny from security researchers worldwide. Continuity was not automatic at birth: forensic work on early block times and ExtraNonce sequences shows the chain halted for many hours in the first ~170 blocks when nobody was mining—then recovered as Hal Finney, Dustin Trammell, and others joined.[10][11] That it still produces a block approximately every ten minutes is a testament to Satoshi's engineering—and to those first operators who kept the experiment alive.[12]

Satoshi's farewell, sent in a private email to Mike Hearn before vanishing (April 23, 2011): "I've moved on to other things. It's in good hands with Gavin and everyone."[13] These 2,660 lines of C++ launched a monetary system that has operated continuously for over seventeen years, securing over a trillion dollars in value, without a CEO, a board of directors, or a single hour of planned downtime.

KEY TERMS

mempool

The node's in-memory pool (mapTransactions) of valid but unconfirmed transactions, where they wait for a miner to include them in a block.

UTXO

An unspent transaction output: a coin that has been created but not yet spent. In v0.1.0, spends are recorded on disk in each transaction's CTxIndex (vSpent) when a block connects.

coinbase

The special first transaction of a block whose input references nothing (a null prevout) and whose outputs pay the miner the subsidy plus fees.

merkle root

The single hash stored in a block header that commits to every transaction in the block, built by repeatedly double-hashing pairs of transaction IDs.

nonce

The 32-bit block-header field miners increment while searching for a hash at or below the difficulty target.

difficulty target

The 256-bit threshold a block's hash must not exceed, stored compactly in the header as nBits and recalculated every 2016 blocks to hold the ten-minute average.

chain reorganization

The switch to a competing longer valid chain, in which the old tip's blocks are disconnected and their transactions return to the mempool.

halving

The event every 210,000 blocks that right-shifts the subsidy in GetBlockValue, cutting the block reward in half and converging the total supply toward 21 million.

orphan transaction

A transaction that arrives before the parent whose output it spends, parked in mapOrphanTransactions until the parent shows up.

SOURCES

1. Satoshi Nakamoto, *Re: Bitcoin P2P e-cash paper (Cryptography Mailing List)* (November 8, 2008). On writing all the code before writing the paper. https://www.metzdowd.com/pipermail/cryptography/2008-November/014832.html

2. Leslie Lamport, Robert Shostak, and Marshall Pease, *The Byzantine Generals Problem* (1982). ACM Transactions on Programming Languages and Systems, 4(3):382–401. https://lamport.azurewebsites.net/pubs/byz.pdf
3. Satoshi Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (October 2008). https://bitcoin.org/bitcoin.pdf
4. Satoshi Nakamoto, *Bitcoin P2P e-cash paper (Cryptography Mailing List)* (November 2008). On the proof-of-work chain as the solution to synchronisation. https://satoshi.nakamotoinstitute.org/emails/cryptography/3/
5. Adam Back, *Hashcash — A Denial of Service Counter-Measure* (August 2002). http://www.hashcash.org/papers/hashcash.pdf
6. David Chaum, *Blind Signatures for Untraceable Payments* (1983). Foundational DigiCash cryptography; Advances in Cryptology. https://sceweb.sce.uhcl.edu/yang/teaching/csci5234WebSecurityFall2011/Chaum-blind-signatures.PDF
7. Wei Dai, *b-money* (1998). http://www.weidai.com/bmoney.txt
8. Nick Szabo, *Bit gold* (December 2008). Earlier drafts circulated among cypherpunks; public post dated 2008. https://unenumerated.blogspot.com/2005/12/bit-gold.html
9. Hal Finney, *RPOW — Reusable Proofs of Work* (2004). https://nakamotoinstitute.org/finney/rpow/
10. Alex Waltz, *Who was the 1st Bitcoin Miner?* (2026). Forensic reconstruction of Bitcoin's first days from Finney's debug.log, emails, and ExtraNonce patterns. https://1stbitcoinminer.com/
11. Alex Waltz, *Copy-Paste & LLM-friendly transcriptions* (2026). Plain-text transcriptions of primary sources used in the 1st Bitcoin Miner investigation. https://1stbitcoinminer.com/transcriptions/
12. Satoshi Nakamoto, *Bitcoin genesis block (block 0)* (January 3, 2009). Coinbase embeds The Times headline as timestamp proof. https://mempool.space/block/000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
13. Satoshi Nakamoto, *Last known email to Mike Hearn ("I've moved on to other things")* (April 23, 2011). "I've moved on to other things. It's in good hands with Gavin and everyone.". https://cointelegraph.com/news/satoshi-nakamoto-final-email-revisited-after-13-years
14. Satoshi Nakamoto, *BitcoinTalk — Re: Flood attack 0.00000001 BC* (August 5, 2010). "Free transactions are nice and we can keep it that way if people don't abuse them.". https://bitcointalk.org/index.php?topic=287.msg7687#msg7687
15. Satoshi Nakamoto, *Emails to Mike Hearn* (2009–2011). Includes coin supply rationale and the abandoned eBay-style marketplace. https://plan99.net/~mike/satoshi-emails/thread4.html
16. Pieter Wuille, *BIP 42 — A finite monetary supply for Bitcoin* (2014). Right-shifting nSubsidy past bit width would reinflate; soft-fork capped halvings. https://github.com/bitcoin/bips/blob/master/bip-0042.mediawiki
17. Jeremy Rubin, *satoshis-version — Annotated Bitcoin v0.01 codebase* (2019). Line-by-line //# comments on Satoshi's original release; best read commit-by-commit. https://github.com/JeremyRubin/satoshis-version/

18. Satoshi Nakamoto, *BitcoinTalk — block interval and difficulty retarget rationale* (2010). Thread "Proof-of-work difficulty increasing" (Feb–Jul 2010): 10-minute rationale (msg376), retarget formula (msg540), hashmeter (msg1648), ExtraNonce (msg1323), 120-block maturity (msg3537). https://bitcointalk.org/index.php?topic=43.0

19. Bitcoin community, *Value overflow incident (block 74638)* (August 15, 2010). Transaction created ~184 billion BTC; fixed in subsequent soft fork. https://en.bitcoin.it/wiki/Value_overflow_incident

20. Sergio Demian Lerner, *The Return of the Deniers and the Revenge of Patoshi* (April 16, 2019). Three v0.1.0 ExtraNonce privacy flaws; nonce-LSB range R; zero Patoshi→Patoshi timestamp inversions in first 50k blocks — argues one miner (~22k / ~1.1M BTC). https://bitslog.com/2019/04/16/the-return-of-the-deniers-and-the-revenge-of-patoshi/

21. Hal Finney, *[bitcoin-list] Crash in bitcoin 0.1.0 (with debug.log)* (January 10, 2009). SourceForge post attaching Finney's debug.log; forensic evidence he joined near block 49. https://web.archive.org/web/20141130200234/https://sourceforge.net/p/bitcoin/mailman/message/21295694/

main.h

1318 lines CPP

ANATOMY OF A COIN

This header defines the fundamental data structures of Bitcoin: transactions, blocks, and the indexes that make the blockchain queryable. If `main.cpp` is the brain, `main.h` is the anatomy—the skeleton upon which everything hangs. These structures aren't arbitrary; each reflects Satoshi's solutions to specific distributed systems problems that stumped cryptographers for decades.

Satoshi described the core structure in Section 2 of the whitepaper: "We define an electronic coin as a chain of digital signatures. Each owner transfers the coin to the next by digitally signing a hash of the previous transaction and the public key of the next owner and adding these to the end of the coin."[1] The classes here—`CTransaction`, `CTxIn`, `CTxOut`—implement that vision in C++, turning abstract cryptographic concepts into working code.

The architecture here introduced what's now called the "UTXO model" (Unspent Transaction Output). Unlike bank accounts that track balances, Bitcoin tracks individual coins—each output is like a physical bill that must be spent entirely. This model enables parallel validation, simple proofs, and clear ownership chains. Ethereum later chose a different path (account model), but most cryptocurrencies follow Satoshi's UTXO design.

Economic policy shows up as constants here — `COIN`, coinbase maturity, proof-of-work limit — while the ~21 million supply cap emerges from the halving formula in `GetBlockValue()` (declared below, implemented in `main.cpp`). Unlike fiat parameters set by central banks, these rules are enforced by code every participant runs. Change requires global consensus — a feature, not a bug.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

class COutPoint;
class CInPoint;
class CDiskTxPos;
class CCoinBase;
class CTxIn;
class CTxOut;
class CTransaction;
class CBlock;
class CBlockIndex;
class CWalletTx;
class CKeyItem;

```

`MAX_SIZE` = 32 MB (0×02000000 bytes). This general-purpose limit applies to blocks, transactions, and scripts. Notably, this is NOT the famous '1 MB block size limit'—that was introduced by Satoshi in July 2010 (commit a30b56e, at first only as a mining limit) and made a validation rule that September (commits 8c9479c and 172f006), over a year after v0.1. The 1 MB limit was likely a temporary anti-spam measure that became permanent, eventually causing the 'block size wars' and the Bitcoin Cash fork in 2017. In v0.1.0, blocks could theoretically reach 32 MB, though mining code limited them to 16 MB (MAX_SIZE/2).

```
static const unsigned int MAX_SIZE = 0x02000000;
```

`COIN` = 100,000,000 satoshis. Satoshi chose 8 decimal places for maximum divisibility—more than any fiat currency. Why 100 million? It allows a single bitcoin to be divided into pieces small enough for micropayments even if BTC reaches millions of dollars per coin. The unit was later named 'satoshi' by the community; Satoshi himself never used the term. Internally, all calculations use satoshis (integers) to avoid floating-point precision errors.

```
static const int64 COIN = 100000000;
```

`CENT` = 1,000,000 satoshis (0.01 BTC). This naming reveals Satoshi's framing: Bitcoin as currency, not commodity. He consistently used monetary language—'coins,' 'cents,' 'transactions'—to shape how people thought about the system. The cent unit appeared in fee calculations and the GUI, making Bitcoin feel like familiar money.

```
static const int64 CENT = 1000000;
```

`COINBASE_MATURITY` = 100 blocks (~17 hours). Freshly mined coins cannot be spent for 100 blocks. Why? If a miner's block gets orphaned in a chain reorganization, the mining reward disappears—it never existed on the winning chain. Without this delay, recipients of those coins would lose money through no fault of their own. 100 blocks provides statistical certainty that reorganizations won't affect mined coins. The wallet is more cautious still: `GetBlocksToMaturity()` waits `COINBASE_MATURITY+20` = 120 blocks, which is why Satoshi described mined coins as spendable after '120 conf / 6 blocks per hour = 20 hours.'[2]

```
static const int COINBASE_MATURITY = 100;

```

`bnProofOfWorkLimit`—the easiest possible difficulty, used for the genesis block. Expressed as `~uint256(0) >> 32`, meaning valid hashes must have at least 32 leading zero bits. This was trivially easy by modern standards, though a single 2009 CPU still needed hours, on average, to find a block alone. As more miners joined, difficulty automatically increased. Today's difficulty requires ~80+ leading zeros—approximately 100 trillion times harder than this initial value.

```
22 static const CBigNum bnProofOfWorkLimit(~uint256(0) >> 32);
23
24
25
26
27
28
29 extern CCriticalSection cs_main;
```

`mapBlockIndex` — in-memory block tree: every known hash → `CBlockIndex` with height, disk location, and chain links. Loaded from BDB at startup; walking the best chain is O(1) per hop.[1]

```
30 extern map<uint256, CBlockIndex*> mapBlockIndex;
```

`hashGenesisBlock` — hard-coded hash of block 0. Startup verifies the reconstructed genesis matches this constant; every later block links back through `hashPrevBlock`.[3]

```
31 extern const uint256 hashGenesisBlock;
32 extern CBlockIndex* pindexGenesisBlock;
```

`nBestHeight` - current chain height. This simple integer represents humanity's consensus on Bitcoin's state. Elegant simplicity.

```
33 extern int nBestHeight;
34 extern uint256 hashBestChain;
```

`pindexBest` - pointer to chain tip. New blocks normally extend this tip; a block that extends another known block starts a side branch, and if that branch grows taller, the node reorganizes onto it. The 'longest chain' rule implemented as a single pointer.

```
35 extern CBlockIndex* pindexBest;
36 extern unsigned int nTransactionsUpdated;
37 extern string strSetDataDir;
38 extern int nDropMessagesTest;
39
40 // Settings
```

`fGenerateBitcoins` - mining flag. Satoshi's 'one-CPU-one-vote' vision. In 2009, turning this on meant your laptop joined the mining democracy.

```
41 extern int fGenerateBitcoins;
```

`nTransactionFee` — default fee setting (satoshis). Extern here; the Options dialog and wallet.dat persist it. Miners still collect whatever inputs−outputs leaves behind.

```
42 extern int64 nTransactionFee;
```

```
extern CAddress addrIncoming;
```

```
string GetAppDir();
FILE* OpenBlockFile(unsigned int nFile, unsigned int nBlockPos, const char* pszMode="rb");
FILE* AppendBlockFile(unsigned int& nFileRet);
bool AddKey(const CKey& key);
vector<unsigned char> GenerateNewKey();
bool AddToWallet(const CWalletTx& wtxIn);
void ReacceptWalletTransactions();
void RelayWalletTransactions();
bool LoadBlockIndex(bool fAllowNew=true);
void PrintBlockTree();
bool BitcoinMiner();
bool ProcessMessages(CNode* pfrom);
bool ProcessMessage(CNode* pfrom, string strCommand, CDataStream& vRecv);
bool SendMessages(CNode* pto);
int64 GetBalance();
```

Wallet API surface — `CreateTransaction`, `CommitTransactionSpent`, `SendMoney`, `GetBalance`. The GUI and IP-payment paths call these; consensus validation stays in the `CTransaction` / `CBlock` methods below.

```
bool CreateTransaction(CScript scriptPubKey, int64 nValue, CWalletTx& txNew, int64& nFeeRequiredRet);
bool CommitTransactionSpent(const CWalletTx& wtxNew);
bool SendMoney(CScript scriptPubKey, int64 nValue, CWalletTx& wtxNew);
```

`CDiskTxPos` — where a transaction lives on disk: file number, block offset, tx offset inside the block. The UTXO index stores these so `ConnectInputs()` can seek straight to the previous output without scanning the whole chain.

```
class CDiskTxPos
{
public:
unsigned int nFile;
unsigned int nBlockPos;
unsigned int nTxPos;

CDiskTxPos()
{
SetNull();
}

CDiskTxPos(unsigned int nFileIn, unsigned int nBlockPosIn, unsigned int nTxPosIn)
{
        nFile = nFileIn;
        nBlockPos = nBlockPosIn;
        nTxPos = nTxPosIn;
}

```

```
IMPLEMENT_SERIALIZE( READWRITE(FLATDATA(*this)); )
void SetNull() { nFile = -1; nBlockPos = 0; nTxPos = 0; }
bool IsNull() const { return (nFile == -1); }

friend bool operator==(const CDiskTxPos& a, const CDiskTxPos& b)
{
return (a.nFile     == b.nFile &&
                a.nBlockPos == b.nBlockPos &&
                a.nTxPos    == b.nTxPos);
}

friend bool operator!=(const CDiskTxPos& a, const CDiskTxPos& b)
{
return !(a == b);
}

    string ToString() const
{
if (IsNull())
return strprintf("null");
else
return strprintf("(nFile=%d, nBlockPos=%d, nTxPos=%d)", nFile, nBlockPos, nTxPos);
}

void print() const
{
printf("%s", ToString().c_str());
}
};
```

`CInPoint` — in-memory pointer to a transaction input (`CTransaction*` + index). Counterpart to `COutPoint`, but not serialized — used while building and validating transactions in RAM.

```
class CInPoint
{
public:
    CTransaction* ptx;
unsigned int n;

CInPoint() { SetNull(); }
CInPoint(CTransaction* ptxIn, unsigned int nIn) { ptx = ptxIn; n = nIn; }
void SetNull() { ptx = NULL; n = -1; }
bool IsNull() const { return (ptx == NULL && n == -1); }
};
```

`COutPoint`—a pointer to a specific coin in the UTXO set. Just two fields: a transaction hash and an output index. Together they uniquely identify any output ever created. This simple structure is the foundation of Bitcoin's ownership model: you don't own 'bitcoin in an account,' you own the right to spend specific outputs. Your balance is the sum of unspent outputs recorded in your wallet (`mapWallet`), which collected them as blocks and transactions arrived.

```
class COutPoint
{
public:
    uint256 hash;
unsigned int n;

COutPoint() { SetNull(); }
```

```
154 COutPoint(uint256 hashIn, unsigned int nIn) { hash = hashIn; n = nIn; }
155 IMPLEMENT_SERIALIZE( READWRITE(FLATDATA(*this)); )
156 void SetNull() { hash = 0; n = -1; }
157 bool IsNull() const { return (hash == 0 && n == -1); }
158
159 friend bool operator<(const COutPoint& a, const COutPoint& b)
160 {
161 return (a.hash < b.hash || (a.hash == b.hash && a.n < b.n));
162 }
163
164 friend bool operator==(const COutPoint& a, const COutPoint& b)
165 {
166 return (a.hash == b.hash && a.n == b.n);
167 }
168
169 friend bool operator!=(const COutPoint& a, const COutPoint& b)
170 {
171 return !(a == b);
172 }
173
174     string ToString() const
175 {
176 return strprintf("COutPoint(%s, %d)", hash.ToString().substr(0,6).c_str(), n);
177 }
178
179 void print() const
180 {
181 printf("%s\n", ToString().c_str());
182 }
183 };
184
185
186
187
188 //
189 // An input of a transaction.  It contains the location of the previous
190 // transaction's output that it claims and a signature that matches the
191 // output's public key.
192 //
```

CTxIn—how coins are consumed. Each input contains: (1) a COutPoint referencing which output to spend, (2) a scriptSig providing proof of ownership (typically a signature), and (3) nSequence for transaction replacement (mostly unused in v0.1.0). When a transaction is validated, each CTxIn must reference an existing, unspent output, and the scriptSig must satisfy that output's spending conditions. This is the 'chain of digital signatures' from the whitepaper.[1]

```
193 class CTxIn
194 {
195 public:
196     COutPoint prevout;
197     CScript scriptSig;
```

nSequence — per-input sequence number. UINT_MAX means final; lower values were reserved for future replacement / relative lock-time. In v0.1.0 almost everything uses UINT_MAX; later upgrades (BIP 68/112 at consensus, BIP 125 as mempool policy) gave this field real meaning.

```
198 unsigned int nSequence;
199
200 CTxIn()
201 {
202         nSequence = UINT_MAX;
203 }
204
205 explicit CTxIn(COutPoint prevoutIn, CScript scriptSigIn=CScript(), unsigned int nSequenceIn=UINT_MAX)
206 {
```

```
        prevout = prevoutIn;
        scriptSig = scriptSigIn;
        nSequence = nSequenceIn;
}

CTxIn(uint256 hashPrevTx, unsigned int nOut, CScript scriptSigIn=CScript(), unsigned int
nSequenceIn=UINT_MAX)
{
        prevout = COutPoint(hashPrevTx, nOut);
        scriptSig = scriptSigIn;
        nSequence = nSequenceIn;
}

    IMPLEMENT_SERIALIZE
(
READWRITE(prevout);
READWRITE(scriptSig);
READWRITE(nSequence);
)

bool IsFinal() const
{
return (nSequence == UINT_MAX);
}

friend bool operator==(const CTxIn& a, const CTxIn& b)
{
return (a.prevout   == b.prevout &&
                a.scriptSig == b.scriptSig &&
                a.nSequence == b.nSequence);
}

friend bool operator!=(const CTxIn& a, const CTxIn& b)
{
return !(a == b);
}

    string ToString() const
{
        string str;
        str += strprintf("CTxIn(");
        str += prevout.ToString();
if (prevout.IsNull())
            str += strprintf(", coinbase %s", HexStr(scriptSig.begin(), scriptSig.end(),
false).c_str());
else
            str += strprintf(", scriptSig=%s", scriptSig.ToString().substr(0,24).c_str());
if (nSequence != UINT_MAX)
            str += strprintf(", nSequence=%u", nSequence);
        str += ")";
return str;
}

void print() const
{
printf("%s\n", ToString().c_str());
}

bool IsMine() const;
    int64 GetDebit() const;
};




//
// An output of a transaction.  It contains the public key that the next input
// must be able to sign with to claim it.
//
```

`CTxOut`—how coins are created. Contains just two fields: nValue (amount in satoshis) and scriptPubKey (the spending conditions). The scriptPubKey is a mini program that specifies who can spend this output—typically 'whoever can provide a signature matching this public key.' Once created, a CTxOut exists forever in the blockchain history, but can only be spent once. The set of all unspent CTxOuts is the UTXO set—the complete state of who owns what.

```
274 class CTxOut
275 {
276 public:
277     int64 nValue;
278     CScript scriptPubKey;
279 
280 public:
281 CTxOut()
282 {
283 SetNull();
284 }
285 
286 CTxOut(int64 nValueIn, CScript scriptPubKeyIn)
287 {
288         nValue = nValueIn;
289         scriptPubKey = scriptPubKeyIn;
290 }
291 
292     IMPLEMENT_SERIALIZE
293 (
294 READWRITE(nValue);
295 READWRITE(scriptPubKey);
296 )
297 
298 void SetNull()
299 {
300         nValue = -1;
301         scriptPubKey.clear();
302 }
303 
304 bool IsNull()
305 {
306 return (nValue == -1);
307 }
308 
309     uint256 GetHash() const
310 {
311 return SerializeHash(*this);
312 }
313 
314 bool IsMine() const
315 {
316 return ::IsMine(scriptPubKey);
317 }
318 
319     int64 GetCredit() const
320 {
321 if (IsMine())
322 return nValue;
323 return 0;
324 }
325 
326 friend bool operator==(const CTxOut& a, const CTxOut& b)
327 {
328 return (a.nValue       == b.nValue &&
329                 a.scriptPubKey == b.scriptPubKey);
330 }
331 
332 friend bool operator!=(const CTxOut& a, const CTxOut& b)
333 {
334 return !(a == b);
```

```
}

    string ToString() const
{
if (scriptPubKey.size() < 6)
return "CTxOut(error)";
return strprintf("CTxOut(nValue=%I64d.%08I64d, scriptPubKey=%s)", nValue / COIN, nValue % COIN,
scriptPubKey.ToString().substr(0,24).c_str());
}

void print() const
{
printf("%s\n", ToString().c_str());
}
};



//
// The basic transaction that is broadcasted on the network and contained in
// blocks.  A transaction can contain multiple inputs and outputs.
//
```

`CTransaction`—the fundamental unit of value transfer. Every bitcoin movement is a CTransaction: a set of inputs (coins being spent) and outputs (coins being created). The transaction is atomic—either all inputs are spent and all outputs created, or nothing happens. This atomicity enables trustless exchanges: Alice can pay Bob in one indivisible operation. Satoshi's whitepaper Section 2: 'We define an electronic coin as a chain of digital signatures.'[1]

```
class CTransaction
{
public:
int nVersion;
    vector<CTxIn> vin;
    vector<CTxOut> vout;
```

`nLockTime` — in v0.1.0, the block height the chain must pass before the transaction is final (a timestamp interpretation came later). `0` means immediate. Combined with non-final `nSequence`, this is the primitive behind payment channels and later CLTV — present from day one.[4]

```
int nLockTime;


CTransaction()
{
SetNull();
}

    IMPLEMENT_SERIALIZE
(
READWRITE(this->nVersion);
        nVersion = this->nVersion;
READWRITE(vin);
READWRITE(vout);
READWRITE(nLockTime);
)

void SetNull()
{
        nVersion = 1;
        vin.clear();
        vout.clear();
        nLockTime = 0;
}
```

```
387
388 bool IsNull() const
389 {
390 return (vin.empty() && vout.empty());
391 }
392
```

`GetHash()` — transaction ID: double-SHA256 of the serialized transaction (`SerializeHash`). This is the txid peers advertise in `inv` messages and the hash stored in every `COutPoint`.

```
393     uint256 GetHash() const
394 {
395 return SerializeHash(*this);
396 }
397
```

`IsFinal()` — true when `nLockTime` is zero or below the current height, or every input is final. Honest miners skip non-final transactions, but in v0.1.0 no validation rule (`CheckBlock`, `AcceptBlock`, `ConnectBlock`) rejects a block containing one yet.

```
398 bool IsFinal() const
399 {
400 if (nLockTime == 0 || nLockTime < nBestHeight)
401 return true;
402 foreach(const CTxIn& txin, vin)
403 if (!txin.IsFinal())
404 return false;
405 return true;
406 }
407
408 bool IsNewerThan(const CTransaction& old) const
409 {
410 if (vin.size() != old.vin.size())
411 return false;
412 for (int i = 0; i < vin.size(); i++)
413 if (vin[i].prevout != old.vin[i].prevout)
414 return false;
415
416 bool fNewer = false;
417 unsigned int nLowest = UINT_MAX;
418 for (int i = 0; i < vin.size(); i++)
419 {
420 if (vin[i].nSequence != old.vin[i].nSequence)
421 {
422 if (vin[i].nSequence <= nLowest)
423 {
424                     fNewer = false;
425                     nLowest = vin[i].nSequence;
426 }
427 if (old.vin[i].nSequence < nLowest)
428 {
429                     fNewer = true;
430                     nLowest = old.vin[i].nSequence;
431 }
432 }
433 }
434 return fNewer;
435 }
436
```

`IsCoinBase()` — one input whose `prevout` is null. That single test distinguishes money creation from money transfer; coinbase outputs still wait `COINBASE_MATURITY` blocks before spending.

```
437 bool IsCoinBase() const
```

```
{
return (vin.size() == 1 && vin[0].prevout.IsNull());
}

```

`CheckTransaction()` — context-free sanity: non-empty vin/vout, no negative outputs, coinbase scriptSig size bounds. Full script and double-spend checks happen later in `ConnectInputs()`.

```
bool CheckTransaction() const
{
// Basic checks that don't depend on any context
if (vin.empty() || vout.empty())
return error("CTransaction::CheckTransaction() : vin or vout empty");

// Check for negative values
foreach(const CTxOut& txout, vout)
if (txout.nValue < 0)
return error("CTransaction::CheckTransaction() : txout.nValue negative");

if (IsCoinBase())
{
if (vin[0].scriptSig.size() < 2 || vin[0].scriptSig.size() > 100)
return error("CTransaction::CheckTransaction() : coinbase script size");
}
else
{
foreach(const CTxIn& txin, vin)
if (txin.prevout.IsNull())
return error("CTransaction::CheckTransaction() : prevout is null");
}

return true;
}

bool IsMine() const
{
foreach(const CTxOut& txout, vout)
if (txout.IsMine())
return true;
return false;
}

    int64 GetDebit() const
{
        int64 nDebit = 0;
foreach(const CTxIn& txin, vin)
            nDebit += txin.GetDebit();
return nDebit;
}

    int64 GetCredit() const
{
        int64 nCredit = 0;
foreach(const CTxOut& txout, vout)
            nCredit += txout.GetCredit();
return nCredit;
}

    int64 GetValueOut() const
{
        int64 nValueOut = 0;
foreach(const CTxOut& txout, vout)
{
if (txout.nValue < 0)
throw runtime_error("CTransaction::GetValueOut() : negative value");
            nValueOut += txout.nValue;
}
return nValueOut;
}
```

503

`GetMinFee()`—calculates the minimum required fee. Transaction fees are implicit: total inputs minus total outputs. Since inputs don't store amounts (only references), nodes must look up each referenced output to calculate the fee. This elegant design means fees don't bloat transaction size. In v0.1.0, the minimum was 0.01 BTC plus 0.01 BTC per full 1000 bytes, waived for transactions under 10KB among a block's first ~100. The wallet applies the discounted rule when sending (so small payments go free), and miners enforce the fee beyond the free allowance. As Satoshi later wrote: 'Free transactions are nice and we can keep it that way if people don't abuse them.'[5]

```
504     int64 GetMinFee(bool fDiscount=false) const
505 {
506 unsigned int nBytes = ::GetSerializeSize(*this, SER_NETWORK);
507 if (fDiscount && nBytes < 10000)
508 return 0;
509 return (1 + (int64)nBytes / 1000) * CENT;
510 }
511
512
513
514 bool ReadFromDisk(CDiskTxPos pos, FILE** pfileRet=NULL)
515 {
516         CAutoFile filein = OpenBlockFile(pos.nFile, 0, pfileRet ? "rb+" : "rb");
517 if (!filein)
518 return error("CTransaction::ReadFromDisk() : OpenBlockFile failed");
519
520 // Read transaction
521 if (fseek(filein, pos.nTxPos, SEEK_SET) != 0)
522 return error("CTransaction::ReadFromDisk() : fseek failed");
523         filein >> *this;
524
525 // Return file pointer
526 if (pfileRet)
527 {
528 if (fseek(filein, pos.nTxPos, SEEK_SET) != 0)
529 return error("CTransaction::ReadFromDisk() : second fseek failed");
530 *pfileRet = filein.release();
531 }
532 return true;
533 }
534
535
536 friend bool operator==(const CTransaction& a, const CTransaction& b)
537 {
538 return (a.nVersion  == b.nVersion &&
539                 a.vin       == b.vin &&
540                 a.vout      == b.vout &&
541                 a.nLockTime == b.nLockTime);
542 }
543
544 friend bool operator!=(const CTransaction& a, const CTransaction& b)
545 {
546 return !(a == b);
547 }
548
549
550     string ToString() const
551 {
552         string str;
553         str += strprintf("CTransaction(hash=%s, ver=%d, vin.size=%d, vout.size=%d, nLockTime=%d)\n",
554 GetHash().ToString().substr(0,6).c_str(),
555             nVersion,
556             vin.size(),
557             vout.size(),
558             nLockTime);
559 for (int i = 0; i < vin.size(); i++)
560             str += "    " + vin[i].ToString() + "\n";
```

```
for (int i = 0; i < vout.size(); i++)
            str += "    " + vout[i].ToString() + "\n";
return str;
}

void print() const
{
printf("%s", ToString().c_str());
}



bool DisconnectInputs(CTxDB& txdb);
```

`ConnectInputs()` — declaration of the UTXO validation engine (implemented in `main.cpp`). Looks up each prior output, verifies scripts, rejects double-spends, accumulates fees.[1]

```
bool ConnectInputs(CTxDB& txdb, map<uint256, CTxIndex>& mapTestPool, CDiskTxPos posThisTx, int nHeight, int64& nFees, bool fBlock, bool fMiner, int64 nMinFee=0);
bool ClientConnectInputs();

```

`AcceptTransaction()` — mempool admission entry point. Context-free checks, then optional `ConnectInputs`, then `AddToMemoryPool`. Every P2P `tx` and wallet send funnels through here.

```
bool AcceptTransaction(CTxDB& txdb, bool fCheckInputs=true, bool* pfMissingInputs=NULL);

bool AcceptTransaction(bool fCheckInputs=true, bool* pfMissingInputs=NULL)
{
        CTxDB txdb("r");
return AcceptTransaction(txdb, fCheckInputs, pfMissingInputs);
}

protected:
bool AddToMemoryPool();
public:
bool RemoveFromMemoryPool();
};




//
// A transaction with a merkle branch linking it to the block chain
//
```

`CMerkleTx` - transaction with merkle proof. Whitepaper Section 8: enables SPV clients to verify inclusion without full blockchain.[16]

```
class CMerkleTx : public CTransaction
{
public:
    uint256 hashBlock;
    vector<uint256> vMerkleBranch;
int nIndex;

// memory only
mutable bool fMerkleVerified;


CMerkleTx()
{
Init();
```

```
}

CMerkleTx(const CTransaction& txIn) : CTransaction(txIn)
{
Init();
}

void Init()
{
        hashBlock = 0;
        nIndex = -1;
        fMerkleVerified = false;
}

    int64 GetCredit() const
{
// Must wait until coinbase is safely deep enough in the chain before valuing it
if (IsCoinBase() && GetBlocksToMaturity() > 0)
return 0;
return CTransaction::GetCredit();
}

    IMPLEMENT_SERIALIZE
(
        nSerSize += SerReadWrite(s, *(CTransaction*)this, nType, nVersion, ser_action);
        nVersion = this->nVersion;
READWRITE(hashBlock);
READWRITE(vMerkleBranch);
READWRITE(nIndex);
)


int SetMerkleBranch(const CBlock* pblock=NULL);
int GetDepthInMainChain() const;
bool IsInMainChain() const { return GetDepthInMainChain() > 0; }
int GetBlocksToMaturity() const;
bool AcceptTransaction(CTxDB& txdb, bool fCheckInputs=true);
bool AcceptTransaction() { CTxDB txdb("r"); return AcceptTransaction(txdb); }
};



//
// A transaction with a bunch of additional info that only the owner cares
// about.  It includes any unrecorded transactions needed to link it back
// to the block chain.
//
```

`CWalletTx` - your transaction view. Adds wallet metadata (credit, debit, time) to the base transaction. The user-facing layer.

```
class CWalletTx : public CMerkleTx
{
public:
    vector<CMerkleTx> vtxPrev;
    map<string, string> mapValue;
    vector<pair<string, string> > vOrderForm;
unsigned int fTimeReceivedIsTxTime;
unsigned int nTimeReceived; // time received by this node
char fFromMe;
char fSpent;
//// probably need to sign the order info so know it came from payer

// memory only
mutable unsigned int nTimeDisplayed;


CWalletTx()
```

```
{
Init();
}

CWalletTx(const CMerkleTx& txIn) : CMerkleTx(txIn)
{
Init();
}

CWalletTx(const CTransaction& txIn) : CMerkleTx(txIn)
{
Init();
}

void Init()
{
        fTimeReceivedIsTxTime = false;
        nTimeReceived = 0;
        fFromMe = false;
        fSpent = false;
        nTimeDisplayed = 0;
}

    IMPLEMENT_SERIALIZE
(
        nSerSize += SerReadWrite(s, *(CMerkleTx*)this, nType, nVersion, ser_action);
        nVersion = this->nVersion;
READWRITE(vtxPrev);
READWRITE(mapValue);
READWRITE(vOrderForm);
READWRITE(fTimeReceivedIsTxTime);
READWRITE(nTimeReceived);
READWRITE(fFromMe);
READWRITE(fSpent);
)

bool WriteToDisk()
{
return CWalletDB().WriteTx(GetHash(), *this);
}


    int64 GetTxTime() const;

void AddSupportingTransactions(CTxDB& txdb);

bool AcceptWalletTransaction(CTxDB& txdb, bool fCheckInputs=true);
bool AcceptWalletTransaction() { CTxDB txdb("r"); return AcceptWalletTransaction(txdb); }

void RelayWalletTransaction(CTxDB& txdb);
void RelayWalletTransaction() { CTxDB txdb("r"); RelayWalletTransaction(txdb); }
};



//
// A txdb record that contains the disk location of a transaction and the
// locations of transactions that spend its outputs.  vSpent is really only
// used as a flag, but having the location is very helpful for debugging.
//
```

`CTxIndex` — disk index entry: where the tx sits (`pos`) plus `vSpent`, one slot per output recording which later tx spent it (or null if still unspent). This is Bitcoin's spent-output bookkeeping before a dedicated UTXO set database.

```
class CTxIndex
{
public:
```

```
    CDiskTxPos pos;
    vector<CDiskTxPos> vSpent;

CTxIndex()
{
SetNull();
}

CTxIndex(const CDiskTxPos& posIn, unsigned int nOutputs)
{
        pos = posIn;
        vSpent.resize(nOutputs);
}

    IMPLEMENT_SERIALIZE
(
if (!(nType & SER_GETHASH))
READWRITE(nVersion);
READWRITE(pos);
READWRITE(vSpent);
)

void SetNull()
{
        pos.SetNull();
        vSpent.clear();
}

bool IsNull()
{
return pos.IsNull();
}

friend bool operator==(const CTxIndex& a, const CTxIndex& b)
{
if (a.pos != b.pos || a.vSpent.size() != b.vSpent.size())
return false;
for (int i = 0; i < a.vSpent.size(); i++)
if (a.vSpent[i] != b.vSpent[i])
return false;
return true;
}

friend bool operator!=(const CTxIndex& a, const CTxIndex& b)
{
return !(a == b);
}
};



//
// Nodes collect new transactions into a block, hash them into a hash tree,
// and scan through nonce values to make the block's hash satisfy proof-of-work
// requirements.  When they solve the proof-of-work, they broadcast the block
// to everyone and the block is added to the block chain.  The first transaction
// in the block is a special one that creates a new coin owned by the creator
// of the block.
//
// Blocks are appended to blk0001.dat files on disk.  Their location on disk
// is indexed by CBlockIndex objects in memory.
//
```

`CBlock` - a block of transactions with proof-of-work. The atomic unit of blockchain consensus. Each ~10 minutes, one is added.

```
class CBlock
```

```
{
public:
// header
```

80-byte header fields: `nVersion`, `hashPrevBlock`, `hashMerkleRoot`, `nTime`, `nBits` (compact target), `nNonce`. Only these bytes are hashed for proof-of-work — transactions affect the header solely through the merkle root.

```
    int nVersion;
    uint256 hashPrevBlock;
    uint256 hashMerkleRoot;
unsigned int nTime;
unsigned int nBits;
unsigned int nNonce;

// network and disk
    vector<CTransaction> vtx;

// memory only
mutable vector<uint256> vMerkleTree;


CBlock()
{
SetNull();
}

    IMPLEMENT_SERIALIZE
(
READWRITE(this->nVersion);
        nVersion = this->nVersion;
READWRITE(hashPrevBlock);
READWRITE(hashMerkleRoot);
READWRITE(nTime);
READWRITE(nBits);
READWRITE(nNonce);

// ConnectBlock depends on vtx being last so it can calculate offset
if (!(nType & (SER_GETHASH|SER_BLOCKHEADERONLY)))
READWRITE(vtx);
else if (fRead)
const_cast<CBlock*>(this)->vtx.clear();
)

void SetNull()
{
        nVersion = 1;
        hashPrevBlock = 0;
        hashMerkleRoot = 0;
        nTime = 0;
        nBits = 0;
        nNonce = 0;
        vtx.clear();
        vMerkleTree.clear();
}

bool IsNull() const
{
return (nBits == 0);
}

```

`GetHash()` — block ID: double-SHA256 from `nVersion` through `nNonce` (exactly 80 bytes). Must be ≤ the difficulty target implied by `nBits`.

```
    uint256 GetHash() const
```

```
{
return Hash(BEGIN(nVersion), END(nNonce));
}


```

`BuildMerkleTree()` — pairwise double-SHA256 of txids up to a single root. Odd nodes duplicate the last hash (Bitcoin's classic merkle quirk). The root commits the entire transaction list into the header.[6]

```
    uint256 BuildMerkleTree() const
{
        vMerkleTree.clear();
foreach(const CTransaction& tx, vtx)
            vMerkleTree.push_back(tx.GetHash());
int j = 0;
for (int nSize = vtx.size(); nSize > 1; nSize = (nSize + 1) / 2)
{
for (int i = 0; i < nSize; i += 2)
{
int i2 = min(i+1, nSize-1);
                vMerkleTree.push_back(Hash(BEGIN(vMerkleTree[j+i]), END(vMerkleTree[j+i]),
BEGIN(vMerkleTree[j+i2]), END(vMerkleTree[j+i2])));
}
            j += nSize;
}
return (vMerkleTree.empty() ? 0 : vMerkleTree.back());
}

    vector<uint256> GetMerkleBranch(int nIndex) const
{
if (vMerkleTree.empty())
BuildMerkleTree();
        vector<uint256> vMerkleBranch;
int j = 0;
for (int nSize = vtx.size(); nSize > 1; nSize = (nSize + 1) / 2)
{
int i = min(nIndex^1, nSize-1);
            vMerkleBranch.push_back(vMerkleTree[j+i]);
            nIndex >>= 1;
            j += nSize;
}
return vMerkleBranch;
}


```

`CheckMerkleBranch()` — verify a txid against a merkle branch and root without the full block. Powers `CMerkleTx` SPV-style inclusion proofs (whitepaper §8).[1]

```
static uint256 CheckMerkleBranch(uint256 hash, const vector<uint256>& vMerkleBranch, int nIndex)
{
if (nIndex == -1)
return 0;
foreach(const uint256& otherside, vMerkleBranch)
{
if (nIndex & 1)
                hash = Hash(BEGIN(otherside), END(otherside), BEGIN(hash), END(hash));
else
                hash = Hash(BEGIN(hash), END(hash), BEGIN(otherside), END(otherside));
            nIndex >>= 1;
}
return hash;
}


bool WriteToDisk(bool fWriteTransactions, unsigned int& nFileRet, unsigned int& nBlockPosRet)
```

```
{
// Open history file to append
        CAutoFile fileout = AppendBlockFile(nFileRet);
if (!fileout)
return error("CBlock::WriteToDisk() : AppendBlockFile failed");
if (!fWriteTransactions)
            fileout.nType |= SER_BLOCKHEADERONLY;

// Write index header
unsigned int nSize = fileout.GetSerializeSize(*this);
        fileout << FLATDATA(pchMessageStart) << nSize;

// Write block
        nBlockPosRet = ftell(fileout);
if (nBlockPosRet == -1)
return error("CBlock::WriteToDisk() : ftell failed");
        fileout << *this;

return true;
}

bool ReadFromDisk(unsigned int nFile, unsigned int nBlockPos, bool fReadTransactions)
{
SetNull();

// Open history file to read
        CAutoFile filein = OpenBlockFile(nFile, nBlockPos, "rb");
if (!filein)
return error("CBlock::ReadFromDisk() : OpenBlockFile failed");
if (!fReadTransactions)
            filein.nType |= SER_BLOCKHEADERONLY;

// Read block
        filein >> *this;

// Check the header
if (CBigNum().SetCompact(nBits) > bnProofOfWorkLimit)
return error("CBlock::ReadFromDisk() : nBits errors in block header");
if (GetHash() > CBigNum().SetCompact(nBits).getuint256())
return error("CBlock::ReadFromDisk() : GetHash() errors in block header");

return true;
}




void print() const
{
printf("CBlock(hash=%s, ver=%d, hashPrevBlock=%s, hashMerkleRoot=%s, nTime=%u, nBits=%08x, nNonce=%u,
    vtx=%d)\n",
GetHash().ToString().substr(0,14).c_str(),
            nVersion,
            hashPrevBlock.ToString().substr(0,14).c_str(),
            hashMerkleRoot.ToString().substr(0,6).c_str(),
            nTime, nBits, nNonce,
            vtx.size());
for (int i = 0; i < vtx.size(); i++)
{
printf("  ");
            vtx[i].print();
}
printf("  vMerkleTree: ");
for (int i = 0; i < vMerkleTree.size(); i++)
printf("%s ", vMerkleTree[i].ToString().substr(0,6).c_str());
printf("\n");
}
```

Block lifecycle methods: `GetBlockValue` (50 BTC subsidy + fees, halving every 210k — the ~21M supply falls out of this schedule), `ConnectBlock`/`DisconnectBlock`, `CheckBlock`/`AcceptBlock`. Declarations here; bodies in `main.cpp`.[7]

```
    int64 GetBlockValue(int64 nFees) const;
bool DisconnectBlock(CTxDB& txdb, CBlockIndex* pindex);
bool ConnectBlock(CTxDB& txdb, CBlockIndex* pindex);
bool ReadFromDisk(const CBlockIndex* blockindex, bool fReadTransactions);
bool AddToBlockIndex(unsigned int nFile, unsigned int nBlockPos);
bool CheckBlock() const;
bool AcceptBlock();
};









//
// The block chain is a tree shaped structure starting with the
// genesis block at the root, with each block potentially having multiple
// candidates to be the next block.  pprev and pnext link a path through the
// main/longest chain.  A blockindex may have multiple pprev pointing back
// to it, but pnext will only point forward to the longest branch, or will
// be null if the block is not part of the longest chain.
//
```

`CBlockIndex` - in-memory block metadata. Height, disk position, and header fields (including the timestamp). Enables fast chain traversal without reading full blocks from disk.

```
class CBlockIndex
{
public:
```

Chain links: `pprev`/`pnext` walk the tree; only the best chain has a forward `pnext`. `nHeight`, file position, and a copy of header fields let the node compare heights and locate raw blocks without reloading every `CBlock`.

```
const uint256* phashBlock;
    CBlockIndex* pprev;
    CBlockIndex* pnext;
unsigned int nFile;
unsigned int nBlockPos;
int nHeight;

// block header
int nVersion;
    uint256 hashMerkleRoot;
unsigned int nTime;
unsigned int nBits;
unsigned int nNonce;


CBlockIndex()
{
        phashBlock = NULL;
        pprev = NULL;
        pnext = NULL;
        nFile = 0;
        nBlockPos = 0;
        nHeight = 0;

        nVersion       = 0;
```

```
1037         hashMerkleRoot = 0;
1038         nTime          = 0;
1039         nBits          = 0;
1040         nNonce         = 0;
1041 }
1042
1043 CBlockIndex(unsigned int nFileIn, unsigned int nBlockPosIn, CBlock& block)
1044 {
1045         phashBlock = NULL;
1046         pprev = NULL;
1047         pnext = NULL;
1048         nFile = nFileIn;
1049         nBlockPos = nBlockPosIn;
1050         nHeight = 0;
1051
1052         nVersion       = block.nVersion;
1053         hashMerkleRoot = block.hashMerkleRoot;
1054         nTime          = block.nTime;
1055         nBits          = block.nBits;
1056         nNonce         = block.nNonce;
1057 }
1058
1059     uint256 GetBlockHash() const
1060 {
1061 return *phashBlock;
1062 }
1063
1064 bool IsInMainChain() const
1065 {
1066 return (pnext || this == pindexBest);
1067 }
1068
1069 bool EraseBlockFromDisk()
1070 {
1071 // Open history file
1072         CAutoFile fileout = OpenBlockFile(nFile, nBlockPos, "rb+");
1073 if (!fileout)
1074 return false;
1075
1076 // Overwrite with empty null block
1077         CBlock block;
1078         block.SetNull();
1079         fileout << block;
1080
1081 return true;
1082 }
1083
1084 enum { nMedianTimeSpan=11 };
1085
1086     int64 GetMedianTimePast() const
1087 {
1088 unsigned int pmedian[nMedianTimeSpan];
1089 unsigned int* pbegin = &pmedian[nMedianTimeSpan];
1090 unsigned int* pend = &pmedian[nMedianTimeSpan];
1091
1092 const CBlockIndex* pindex = this;
1093 for (int i = 0; i < nMedianTimeSpan && pindex; i++, pindex = pindex->pprev)
1094 *(--pbegin) = pindex->nTime;
1095
1096 sort(pbegin, pend);
1097 return pbegin[(pend - pbegin)/2];
1098 }
1099
1100     int64 GetMedianTime() const
1101 {
1102 const CBlockIndex* pindex = this;
1103 for (int i = 0; i < nMedianTimeSpan/2; i++)
1104 {
1105 if (!pindex->pnext)
1106 return nTime;
1107             pindex = pindex->pnext;
1108 }
```

```
return pindex->GetMedianTimePast();
}



    string ToString() const
{
return strprintf("CBlockIndex(nprev=%08x, pnext=%08x, nFile=%d, nBlockPos=%-6d nHeight=%d, merkle=%s,
hashBlock=%s)",
            pprev, pnext, nFile, nBlockPos, nHeight,
            hashMerkleRoot.ToString().substr(0,6).c_str(),
GetBlockHash().ToString().substr(0,14).c_str());
}

void print() const
{
printf("%s\n", ToString().c_str());
}
};



//
// Used to marshal pointers into hashes for db storage.
//
```

`CDiskBlockIndex` — BDB-serializable block metadata. In-memory pointers become `hashPrev`/`hashNext` on disk so the tree rebuilds after restart without holding every `CBlock` in RAM.

```
class CDiskBlockIndex : public CBlockIndex
{
public:
    uint256 hashPrev;
    uint256 hashNext;

CDiskBlockIndex()
{
        hashPrev = 0;
        hashNext = 0;
}

explicit CDiskBlockIndex(CBlockIndex* pindex) : CBlockIndex(*pindex)
{
        hashPrev = (pprev ? pprev->GetBlockHash() : 0);
        hashNext = (pnext ? pnext->GetBlockHash() : 0);
}

    IMPLEMENT_SERIALIZE
(
if (!(nType & SER_GETHASH))
READWRITE(nVersion);

READWRITE(hashNext);
READWRITE(nFile);
READWRITE(nBlockPos);
READWRITE(nHeight);

// block header
READWRITE(this->nVersion);
READWRITE(hashPrev);
READWRITE(hashMerkleRoot);
READWRITE(nTime);
READWRITE(nBits);
READWRITE(nNonce);
)

    uint256 GetBlockHash() const
{
        CBlock block;
```

```
        block.nVersion        = nVersion;
        block.hashPrevBlock   = hashPrev;
        block.hashMerkleRoot  = hashMerkleRoot;
        block.nTime           = nTime;
        block.nBits           = nBits;
        block.nNonce          = nNonce;
return block.GetHash();
}


    string ToString() const
{
        string str = "CDiskBlockIndex(";
        str += CBlockIndex::ToString();
        str += strprintf("\n                hashBlock=%s, hashPrev=%s, hashNext=%s)",
GetBlockHash().ToString().c_str(),
            hashPrev.ToString().substr(0,14).c_str(),
            hashNext.ToString().substr(0,14).c_str());
return str;
}

void print() const
{
printf("%s\n", ToString().c_str());
}
};









//
// Describes a place in the block chain to another node such that if the
// other node doesn't have the same branch, it can find a recent common trunk.
// The further back it is, the further before the fork it may be.
//
```

`CBlockLocator` - compact chain representation for syncing. Exponentially spaced block hashes let nodes find a common ancestor efficiently.

```
class CBlockLocator
{
protected:
    vector<uint256> vHave;
public:

CBlockLocator()
{
}

explicit CBlockLocator(const CBlockIndex* pindex)
{
Set(pindex);
}

explicit CBlockLocator(uint256 hashBlock)
{
        map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hashBlock);
if (mi != mapBlockIndex.end())
Set((*mi).second);
}

    IMPLEMENT_SERIALIZE
(
if (!(nType & SER_GETHASH))
READWRITE(nVersion);
```

```
1238 READWRITE(vHave);
1239 )
1240
1241 void Set(const CBlockIndex* pindex)
1242 {
1243         vHave.clear();
1244 int nStep = 1;
1245 while (pindex)
1246 {
1247             vHave.push_back(pindex->GetBlockHash());
1248
1249 // Exponentially larger steps back
1250 for (int i = 0; pindex && i < nStep; i++)
1251                 pindex = pindex->pprev;
1252 if (vHave.size() > 10)
1253                 nStep *= 2;
1254 }
1255         vHave.push_back(hashGenesisBlock);
1256 }
1257
1258     CBlockIndex* GetBlockIndex()
1259 {
1260 // Find the first block the caller has in the main chain
1261 foreach(const uint256& hash, vHave)
1262 {
1263             map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hash);
1264 if (mi != mapBlockIndex.end())
1265 {
1266                 CBlockIndex* pindex = (*mi).second;
1267 if (pindex->IsInMainChain())
1268 return pindex;
1269 }
1270 }
1271 return pindexGenesisBlock;
1272 }
1273
1274     uint256 GetBlockHash()
1275 {
1276 // Find the first block the caller has in the main chain
1277 foreach(const uint256& hash, vHave)
1278 {
1279             map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hash);
1280 if (mi != mapBlockIndex.end())
1281 {
1282                 CBlockIndex* pindex = (*mi).second;
1283 if (pindex->IsInMainChain())
1284 return hash;
1285 }
1286 }
1287 return hashGenesisBlock;
1288 }
1289
1290 int GetHeight()
1291 {
1292         CBlockIndex* pindex = GetBlockIndex();
1293 if (!pindex)
1294 return 0;
1295 return pindex->nHeight;
1296 }
1297 };
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
```

```
1310 extern map<uint256, CTransaction> mapTransactions;
1311 extern map<uint256, CWalletTx> mapWallet;
1312 extern vector<pair<uint256, bool> > vWalletUpdated;
1313 extern CCriticalSection cs_mapWallet;
1314 extern map<vector<unsigned char>, CPrivKey> mapKeys;
1315 extern map<uint160, vector<unsigned char> > mapPubKeys;
1316 extern CCriticalSection cs_mapKeys;
1317 extern CKey keyUser;
1318
```

SUMMARY

These data structures implement what Satoshi described in the whitepaper's opening line as "a purely peer-to-peer version of electronic cash."[1] The design decisions encoded here have proven remarkably durable—over seventeen years later, the core structures remain essentially unchanged despite securing trillions of dollars in cumulative value transfer.

The UTXO model that Satoshi invented (tracking individual coin outputs rather than account balances) enables several key properties: transactions can be validated in parallel since each input is independent; ownership proofs are simple (just show the chain of signatures); and the entire monetary supply can be audited by summing all unspent outputs. When Ethereum chose the account model instead, they gained programmer convenience but lost these properties.

Notice what's NOT in this header: no 'Account' class, no 'Balance' field, no central ledger. Bitcoin doesn't know your balance—your wallet calculates it by scanning for outputs your keys can spend. This stateless design is why Bitcoin scales to millions of users without centralized databases. Your coins exist as scattered outputs across the blockchain, united only by your private keys.

KEY TERMS

UTXO

An unspent transaction output, the atomic coin of this system; a wallet's balance is nothing more than the sum of outputs its keys can spend.

COutPoint

The pair of a transaction hash and an output index that uniquely names one coin, and the pointer every input uses to say what it spends.

scriptPubKey

The small program stored in each output that states the conditions for spending it, typically a signature matching a given public key hash.

scriptSig

The data packed into an input to satisfy the previous output's scriptPubKey, usually a signature plus the public key it was made with.

txid

A transaction's unique identifier, the double-SHA256 of its serialized bytes as returned by CTransaction::GetHash().

block header

The 80 bytes that are hashed for proof-of-work: version, previous block hash, merkle root, timestamp, nBits, and nonce.

CBlockIndex

The in-memory record holding a known block's metadata and links to its predecessor and successor, so the node can walk the chain without loading full blocks from disk.

coinbase maturity

The rule (COINBASE_MATURITY, 100 blocks) that freshly mined coins may not be spent until their block is buried 100 deep.

nLockTime

The transaction field that, in v0.1.0, gives the block height the chain must pass before the transaction is final; zero means it is immediately final. Later versions also let it hold a timestamp.

satoshi

The smallest money unit in this code, one hundred-millionth of a bitcoin (COIN = 100,000,000), used as the integer unit for all amounts.

SOURCES

1. Satoshi Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (October 2008). https://bitcoin.org/bitcoin.pdf
2. Satoshi Nakamoto, *BitcoinTalk — block interval and difficulty retarget rationale* (2010). Thread "Proof-of-work difficulty increasing" (Feb–Jul 2010): 10-minute rationale (msg376), retarget formula (msg540), hashmeter (msg1648), ExtraNonce (msg1323), 120-block maturity (msg3537). https://bitcointalk.org/index.php?topic=43.0
3. Satoshi Nakamoto, *Bitcoin genesis block (block 0)* (January 3, 2009). Coinbase embeds The Times headline as timestamp proof. https://mempool.space/block/000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
4. Satoshi Nakamoto, *BitcoinTalk — Re: Transactions and Scripts* (June 2010). On supporting a tremendous variety of transaction types. https://bitcointalk.org/index.php?topic=195.0
5. Satoshi Nakamoto, *BitcoinTalk — Re: Flood attack 0.00000001 BC* (August 5, 2010). "Free transactions are nice and we can keep it that way if people don't abuse them.". https://bitcointalk.org/index.php?topic=287.msg7687#msg7687
6. Ralph C. Merkle, *Protocols for Public Key Cryptosystems* (1980). Introduced Merkle trees (hash trees). https://www.merkle.com/papers/Protocols.pdf
7. Satoshi Nakamoto, *BitcoinTalk — Re: Bitcoin 0.3.21* (June 17, 2010). On the core design being set in stone for the rest of its lifetime. https://bitcointalk.org/index.php?topic=195.msg1611#msg1611

makefile

84 lines MAKEFILE

ASSEMBLY REQUIRED (MINGW)

This is the makefile for building Bitcoin v0.1.0 with MinGW GCC on Windows. In 83 lines, it defines the entire build process: compiler flags, include paths, library dependencies, compilation rules for each source file, and the final link step that produces `bitcoin.exe`. Reading a makefile is reading the architect's blueprint — it reveals every component of the system and how they connect.

Satoshi chose **GNU Make** with **MinGW GCC 3.4.5** as the primary build system. MinGW (Minimalist GNU for Windows) provides a port of GCC that produces native Windows executables without requiring a Unix compatibility layer like Cygwin. This was a pragmatic choice: GCC was free, well-understood, and produced efficient code, while MSVC 6.0 (the alternative, see `makefile.vc`) was a commercial product with known C++ compliance issues.

The makefile reveals Bitcoin's complete dependency graph. Four external libraries are linked: Berkeley DB for persistent storage,[1] OpenSSL (as `libeay32`) for cryptography,[2] wxWidgets for the GUI,[3] and fourteen Windows system libraries for networking, graphics, and COM. Ten C++ source files are compiled into object files, each with explicitly declared header dependencies. A precompiled header (`headers.h.gch`) accelerates compilation. The entire project compiles into a single Windows GUI executable.

Several details in this makefile betray the era and the developer's priorities. Optimization is set to `-O0` (no optimization) even in release mode — Satoshi prioritized debuggability over performance. Warnings are suppressed with `-w` rather than fixed. The SHA-256 implementation gets special treatment with `-O3`, because proof-of-work hashing is the one place where performance directly matters. And the link step begins with `kill /f bitcoin.exe` — a Windows command to force-terminate any running instance before overwriting the binary, a habit of a developer who tested by running the program directly from the build directory.

The standard copyright and license header. Every file in Bitcoin v0.1.0 carries this three-line attribution to Satoshi Nakamoto under the MIT/X11 license.[5]

```
# Copyright (c) 2009 Satoshi Nakamoto
# Distributed under the MIT/X11 software license, see the accompanying
# file license.txt or http://www.opensource.org/licenses/mit-license.php.


```

Build mode selection using GNU Make conditionals. If `BUILD` is not set to 'debug' or 'release' on the command line, it defaults to 'debug' — which matters more than it looks: only debug builds define `__WXDEBUG__`, and without it util.h's logger never writes `debug.log`. The `D` variable is set to 'd' in debug mode, which is appended to wxWidgets library names (e.g., `wxmsw28d_core` vs `wxmsw28_core`) to link against the debug-instrumented versions. This is a wxWidgets convention — debug libraries include assertion checks and memory leak detection.

```
ifneq "$(BUILD)" "debug"
ifneq "$(BUILD)" "release"
BUILD=debug
endif
endif
ifeq "$(BUILD)" "debug"
D=d
# note: gcc 3.x profile doesn't work
```

The commented-out profiling flags (`-pg`) reveal that Satoshi experimented with GCC's gprof profiling but found it didn't work with GCC 3.x — likely due to MinGW's limited support for profiling instrumentation on Windows. The active debug flags are `-g` (generate debug symbols for GDB) and `-D__WXDEBUG__` (enable wxWidgets debug assertions). No optimization flags — the code compiles unoptimized for easier debugging.

```
#DEBUGFLAGS=-O0 -g -pg -D__WXDEBUG__
DEBUGFLAGS=-g -D__WXDEBUG__
endif


```

Include paths — five directories where the compiler searches for header files. The paths use absolute root-relative notation (`/boost`, `/DB/build_unix`, etc.), meaning each dependency is installed at the root of a drive. Note `/DB/build_unix` — Satoshi built Berkeley DB using the Unix-style build system under MSYS, even though the target is Windows. The wxWidgets path `vc_lib/mswd` points at the MSVC debug build's directory (home of the build-specific `setup.h`), even though the libraries themselves are linked from `gcc_lib` — a cross-reference between the two builds.

```
INCLUDEPATHS=-I"/boost" -I"/DB/build_unix" -I"/OpenSSL/include" -I"/wxWidgets/lib/vc_lib/mswd" -
I"/wxWidgets/include"
LIBPATHS=-L"/DB/build_unix" -L"/OpenSSL/out" -L"/wxWidgets/lib/gcc_lib"
```

Library dependencies — the complete list of everything Bitcoin links against. Three layers: (1) `db_cxx` and `eay32` — Berkeley DB C++ bindings and OpenSSL's crypto library. (2) Ten wxWidgets libraries — richtext, HTML, core, base, plus image format support (TIFF, JPEG, PNG) and utilities (zlib, regex, expat). (3) Fourteen Windows system libraries — kernel32 (OS fundamentals), user32/gdi32/comdlg32 (GUI and common dialogs), winspool/winmm/shell32 (printing, multimedia, shell), comctl32/ole32/oleaut32/uuid (COM and controls), rpcrt4 (RPC runtime), advapi32 (registry and security), ws2_32 (Winsock 2 networking).

```
LIBS= \
 -l db_cxx \
 -l eay32 \
 -l wxmsw28$(D)_richtext -l wxmsw28$(D)_html -l wxmsw28$(D)_core -l wxbase28$(D) -l wxtiff$(D) -l wxjpeg$(D) -l wxpng$(D) -l wxzlib$(D) -l wxregex$(D) -l wxexpat$(D) \
 -l kernel32 -l user32 -l gdi32 -l comdlg32 -l winspool -l winmm -l shell32 -l comctl32 -l ole32 -l oleaut32 -l uuid -l rpcrt4 -l advapi32 -l ws2_32
WXDEFS=-DWIN32 -D__WXMSW__ -D_WINDOWS -DNOPCH
```

Compiler flags. `-mthreads` enables thread-safe exception handling on MinGW. `-O0` disables all optimization — unusual for production code, but Satoshi prioritized debuggability. `-w` suppresses *all* warnings, which modern practice would consider reckless, but was common in projects using template-heavy libraries that generated hundreds of harmless warnings. `-Wno-invalid-offsetof` suppresses a specific C++ warning about using `offsetof` on non-POD types — likely triggered by Bitcoin's serialization macros. `-Wformat` asks for format string checking — but in GCC, `-w` silences every warning regardless of any `-W` flags, so in this combination the check never reports anything.

```
CFLAGS=-mthreads -O0 -w -Wno-invalid-offsetof -Wformat $(DEBUGFLAGS) $(WXDEFS) $(INCLUDEPATHS)
```

`HEADERS` — the dependency list for the precompiled header. When any of these files change, `headers.h.gch` is rebuilt, which forces recompilation of every source file. This list matches the includes at the bottom of `headers.h` (minus `net.h`, `irc.h`, `market.h`, `uibase.h`, and `ui.h`, which are tracked separately in individual rules).

```
HEADERS=headers.h util.h main.h serialize.h uint256.h key.h bignum.h script.h db.h base58.h



all: bitcoin.exe


```

Precompiled header rule. `headers.h.gch` is GCC's precompiled header format — a binary snapshot of all parsed declarations. The `-c` flag compiles without linking. The dependency list includes the core headers plus `net.h`, `irc.h`, `market.h`, `uibase.h`, and `ui.h` — every header in the project except `sha.h`. When GCC finds `headers.h.gch` alongside `headers.h`, it uses the precompiled version automatically, dramatically speeding up compilation.

```
headers.h.gch: headers.h $(HEADERS) net.h irc.h market.h uibase.h ui.h
	g++ -c $(CFLAGS) -o $@ $<

```

Compilation rules — ten source files, each with its own rule and explicit dependency tracking. The pattern is consistent: `g++ -c $(CFLAGS) -o $@ $<` compiles the source (`$<`, the first prerequisite) into the object file (`$@`, the target). Dependencies after the source filename are header files — if any change, the object is rebuilt. Note that most files depend on `$(HEADERS)` (the precompiled header set), while some add specific extras: `db.cpp` needs `market.h`, `main.cpp` needs `net.h`, `market.h`, and `sha.h`.

```
obj/util.o: util.cpp                $(HEADERS)
        g++ -c $(CFLAGS) -o $@ $<

obj/script.o: script.cpp            $(HEADERS)
        g++ -c $(CFLAGS) -o $@ $<

obj/db.o: db.cpp                    $(HEADERS) market.h
        g++ -c $(CFLAGS) -o $@ $<

obj/net.o: net.cpp                  $(HEADERS) net.h
        g++ -c $(CFLAGS) -o $@ $<

obj/main.o: main.cpp                $(HEADERS) net.h market.h sha.h
        g++ -c $(CFLAGS) -o $@ $<

obj/market.o: market.cpp            $(HEADERS) market.h
        g++ -c $(CFLAGS) -o $@ $<

obj/ui.o: ui.cpp                    $(HEADERS) net.h uibase.h ui.h market.h
        g++ -c $(CFLAGS) -o $@ $<

obj/uibase.o: uibase.cpp            uibase.h
        g++ -c $(CFLAGS) -o $@ $<

```

`sha.o` gets special treatment: `-O3` (maximum optimization) instead of the project-wide `-O0`. SHA-256 is the inner loop of proof-of-work mining — the one function called millions of times per second when searching for a valid block hash. Optimizing it makes mining measurably faster. This is the only file in the project with optimization enabled, revealing Satoshi's understanding that mining performance was the one critical hot path.

```
obj/sha.o: sha.cpp                  sha.h
        g++ -c $(CFLAGS) -O3 -o $@ $<

obj/irc.o:  irc.cpp                 $(HEADERS)
        g++ -c $(CFLAGS) -o $@ $<

```

Resource compilation. `windres` compiles `ui.rc` (a Windows resource script) into an object file containing the Bitcoin icon, checkbox icon, and toolbar button bitmaps. These are embedded directly in the executable — no external image files needed at runtime. The resource dependencies list every bitmap and icon file, ensuring the executable is rebuilt when any visual asset changes.

```
obj/ui_res.o: ui.rc  rc/bitcoin.ico rc/check.ico rc/send16.bmp rc/send16mask.bmp
rc/send16masknoshadow.bmp rc/send20.bmp rc/send20mask.bmp rc/addressbook16.bmp rc/addressbook16mask.bmp
rc/addressbook20.bmp rc/addressbook20mask.bmp
        windres $(WXDEFS) $(INCLUDEPATHS) -o $@ -i $<


```

`OBJS` — the complete list of eleven object files that constitute Bitcoin. This is the full bill of materials: `util.o` (utilities), `script.o` (Script interpreter), `db.o` (database), `net.o` (networking), `main.o` (consensus), `market.o` (unused marketplace), `ui.o` (GUI logic), `uibase.o` (GUI layout), `sha.o` (hashing), `irc.o` (peer discovery), and `ui_res.o` (icons/bitmaps).

```
OBJS=obj/util.o obj/script.o obj/db.o obj/net.o obj/main.o obj/market.o  \
        obj/ui.o obj/uibase.o obj/sha.o obj/irc.o obj/ui_res.o

bitcoin.exe: headers.h.gch $(OBJS)
```

`kill /f bitcoin.exe` — a Windows command to force-terminate any running Bitcoin process before the linker overwrites the executable. The leading `-` (Make's error-suppression prefix) ensures the build continues even if no process is running. This tells us Satoshi's workflow: edit code, run `make`, and the build automatically kills the running instance, relinks, and produces a fresh binary. The developer was testing in-place from the build directory.

```
        -kill /f bitcoin.exe
```

The link step. `-mwindows` and `-Wl,--subsystem,windows` produce a Windows GUI application (no console window). The linker combines all eleven object files with the external libraries to produce the final `bitcoin.exe`. The order matters: `$(LIBPATHS)` before `$(OBJS)` before `$(LIBS)` ensures the linker searches library directories, processes object files, and resolves external symbols in the correct order.

```
        g++ $(CFLAGS) -mwindows -Wl,--subsystem,windows -o $@ $(LIBPATHS) $(OBJS) $(LIBS)

```

`clean` target — deletes all compiled object files and the precompiled header, forcing a full rebuild on the next `make`. Uses `del /Q` (Windows quiet delete) rather than Unix `rm -f`, confirming this makefile targets the Windows command shell despite using GNU Make. The leading `-` suppresses errors if the files don't exist.

```
clean:
        -del /Q obj\*
        -del /Q headers.h.gch

```

SUMMARY

This makefile compiles ten C++ source files and one resource file into a single Windows executable — the entirety of Bitcoin v0.1.0. The build has no configure step, no autotools, no CMake. Dependencies are assumed to be installed at hardcoded paths. The developer runs `make` and either gets a binary or an error. This simplicity reflects a solo developer's workflow — Satoshi was the only person building this code, and he knew where everything was on his machine.

The performance choices are revealing. Everything compiles at `-O0` except `sha.cpp`, which gets `-O3`. This tells us Satoshi understood the performance profile of his system: only SHA-256 hashing is computationally intensive, and only during mining. The rest of the code — transaction validation, block processing, network communication, GUI rendering — runs infrequently enough that optimization is irrelevant compared to the debugging convenience of unoptimized builds.

Bitcoin Core's modern build system uses CMake, supports dozens of platforms, and produces optimized binaries with link-time optimization and hardware-accelerated SHA-256.[4] But it still compiles roughly the same logical components — consensus, networking, storage, scripting, and a user interface — that Satoshi laid out in these 83 lines.

KEY TERMS

MinGW

Minimalist GNU for Windows, a port of the GCC compiler and tools that builds native Windows programs; the README names version 3.4.5.

GNU Make

The build tool that reads this makefile, compares file timestamps against each rule's dependencies, and reruns only the commands whose outputs are out of date.

headers.h.gch

GCC's precompiled-header file, built from headers.h by its own makefile rule and picked up automatically whenever a source file includes headers.h.

optimization level

A compiler setting trading build speed and debuggability for faster code; this makefile uses -O0 (none) everywhere except sha.cpp, which gets -O3.

__WXDEBUG__

A preprocessor symbol defined only in debug builds that turns on wxWidgets' debug checks and, through util.h, Bitcoin's debug.log file.

windres

The GNU resource compiler that turns the ui.rc resource script and its icons and bitmaps into an object file linked into bitcoin.exe.

SOURCES

1. Oracle Corporation, *Berkeley DB License* (2009). Sleepycat-era open-source linking requirements referenced in readme.txt.
2. OpenSSL Project, *OpenSSL License / SSLeay License* (1998–2009). https://www.openssl.org/source/license-openssl-ssleay.txt
3. wxWidgets Project, *wxWidgets Licence* (2005). https://www.wxwidgets.org/about/licence/
4. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
5. Massachusetts Institute of Technology, *MIT License* (1988). License chosen for Bitcoin v0.1.0 (license.txt). https://opensource.org/licenses/MIT

makefile.vc

78 lines MAKEFILE

ASSEMBLY REQUIRED (VISUAL C++)

This is the companion to the MinGW makefile — a build configuration for Microsoft Visual C++ 6.0, the other compiler Satoshi officially supported. Where the MinGW makefile uses GCC conventions (`-I`, `-L`, `-l`, `-o`), this file speaks MSVC's dialect (`/I`, `/LIBPATH:`, `.lib`, `/Fo`). The same ten C++ source files are compiled into the same `bitcoin.exe`, but through Microsoft's toolchain instead of GNU's.

MSVC 6.0 was released in 1998 — over a decade before Bitcoin. By 2009, Microsoft had shipped Visual Studio 2008 (MSVC 9.0), yet Satoshi chose to support the ancient 6.0 release. This was likely a compatibility decision: MSVC 6.0 was still widely installed, its runtime library (`msvcrt.dll`) shipped with every copy of Windows, and it produced smaller executables. Many open-source Windows projects of the era maintained MSVC 6.0 compatibility for these reasons. The tradeoff was poor C++ standards compliance — MSVC 6.0 predated the finalization of C++98 and had notorious template bugs.

Comparing this file with the MinGW makefile reveals subtle differences in how the two toolchains handle the same project. MSVC uses a precompiled header via `/YX` and `/Fp` flags rather than GCC's `.gch` file. Library names gain `.lib` suffixes instead of `-l` prefixes. Berkeley DB is linked as `libdb47s.lib` (static library, version 4.7) rather than `-l db_cxx`, though the MinGW build linked it statically too — the released `bitcoin.exe` shipped with only `libeay32.dll` and `mingwm10.dll`. And the final link step uses `link.exe` with `/SUBSYSTEM:WINDOWS` rather than `g++` with `-mwindows`. The same program, two dialects.

The existence of two makefiles — one for GCC, one for MSVC — tells us something about Satoshi's development process. He tested with both compilers, which is a form of verification: if the code compiles and works correctly under two different C++ implementations, it is less likely to depend on undefined behavior or compiler-specific quirks. Cross-compiler testing is a surprisingly disciplined practice for a solo developer in 2009.

The same copyright header as every other file — Satoshi Nakamoto, MIT/X11 license.[2] The `.vc` suffix conventionally indicates a Visual C++ specific file, distinguishing it from the default `makefile` used by MinGW/GCC.

```
# Copyright (c) 2009 Satoshi Nakamoto
# Distributed under the MIT/X11 software license, see the accompanying
# file license.txt or http://www.opensource.org/licenses/mit-license.php.


```

Build mode selection using NMAKE syntax. `!IF`, `!ENDIF` are NMAKE preprocessor directives — Microsoft's make utility uses a different conditional syntax than GNU Make's `ifeq`/`endif`. The logic is identical: default to debug mode if no `BUILD` variable is set. Run with `nmake /f makefile.vc BUILD=release` for a release build.

```
!IF "$(BUILD)" != "debug" && "$(BUILD)" != "release"
BUILD=debug
!ENDIF
!IF "$(BUILD)" == "debug"
D=d
```

MSVC debug flags. `/Zi` generates a full program database (`.pdb` file) for debugging — the MSVC equivalent of GCC's `-g`. `/Od` disables optimization (equivalent to `-O0`). `/D__WXDEBUG__` enables wxWidgets debug assertions. These produce a binary that can be single-stepped in Visual Studio's debugger with full variable inspection.

```
DEBUGFLAGS=/Zi /Od /D__WXDEBUG__
!ENDIF


```

Include paths in MSVC syntax. `/I` is the include-directory flag (GCC uses `-I`). Note `/DB/build_windows` instead of the MinGW makefile's `/DB/build_unix` — Satoshi built Berkeley DB separately for each compiler, using the platform-native build system. The wxWidgets path still references `vc_lib/mswd`, the MSVC debug build.

```
INCLUDEPATHS=/I"/boost" /I"/DB/build_windows" /I"/OpenSSL/include" /I"/wxWidgets/lib/vc_lib/mswd"
/I"/wxWidgets/include"
LIBPATHS=/LIBPATH:"/DB/build_windows/$(BUILD)" /LIBPATH:"/OpenSSL/out" /LIBPATH:"/wxWidgets/lib/vc_lib"
```

Library dependencies. Key differences from the MinGW version: `libdb47s$(D).lib` is a statically linked Berkeley DB 4.7 (the 's' suffix means static); MinGW links `db_cxx`, and since the released binary shipped without any Berkeley DB DLL, that build was evidently static as well. The wxWidgets and Windows system libraries are identical in content but use `.lib` suffixes instead of `-l` prefixes. MSVC's linker resolves libraries by filename, not by naming conventions.

```
LIBS= \
    libdb47s$(D).lib \
    libeay32.lib \
    wxmsw28$(D)_richtext.lib wxmsw28$(D)_html.lib wxmsw28$(D)_core.lib wxbase28$(D).lib wxtiff$(D).lib
wxjpeg$(D).lib wxpng$(D).lib wxzlib$(D).lib wxregex$(D).lib wxexpat$(D).lib \
    kernel32.lib user32.lib gdi32.lib comdlg32.lib winspool.lib winmm.lib shell32.lib comctl32.lib
ole32.lib oleaut32.lib uuid.lib rpcrt4.lib advapi32.lib ws2_32.lib
WXDEFS=/DWIN32 /D__WXMSW__ /D_WINDOWS /DNOPCH
```

MSVC compiler flags — a dense set of options. `/c` compiles without linking. `/nologo` suppresses the Microsoft copyright banner. `/Ob0` disables inline function expansion. `/MD$(D)` links against the multithreaded DLL runtime (`MSVCRT.DLL` for release, `MSVCRTD.DLL` for debug). `/EHsc` enables C++ exception handling with extern 'C' default nothrow. `/GR` enables runtime type information (RTTI). `/Zm300` triples the precompiled header memory limit — necessary for the large number of includes in `headers.h`. `/YX` enables automatic precompiled header usage, and `/Fpobj/headers.pch` specifies the PCH file location.

```
CFLAGS=/c /nologo /Ob0 /MD$(D) /EHsc /GR /Zm300 /YX /Fpobj/headers.pch $(DEBUGFLAGS) $(WXDEFS)
$(INCLUDEPATHS)
```

`HEADERS` — the same dependency list as the MinGW makefile. Both build systems track the same set of core headers to determine when recompilation is needed. This consistency confirms that the two makefiles are intended to produce identical binaries from identical source code.

```
HEADERS=headers.h util.h main.h serialize.h uint256.h key.h bignum.h script.h db.h base58.h


all: bitcoin.exe


```

Compilation rules. Each rule invokes `cl` (the MSVC compiler) with `/Fo$@` to specify the output object file. The `%s` at the end is an NMAKE special macro that expands to the first dependency with the matching extension — effectively the source file. This is more concise than the MinGW makefile's `$<` but functionally equivalent. Object files use backslash paths (`obj\util.obj`) and the `.obj` extension instead of GCC's `.o`.

```
obj\util.obj: util.cpp        $(HEADERS)
    cl $(CFLAGS) /Fo$@ %s

obj\script.obj: script.cpp    $(HEADERS)
    cl $(CFLAGS) /Fo$@ %s

obj\db.obj: db.cpp            $(HEADERS) market.h
    cl $(CFLAGS) /Fo$@ %s

obj\net.obj: net.cpp          $(HEADERS) net.h
    cl $(CFLAGS) /Fo$@ %s

obj\main.obj: main.cpp        $(HEADERS) net.h market.h
    cl $(CFLAGS) /Fo$@ %s

obj\market.obj: market.cpp    $(HEADERS) market.h
    cl $(CFLAGS) /Fo$@ %s

obj\ui.obj: ui.cpp            $(HEADERS) net.h uibase.h ui.h market.h
    cl $(CFLAGS) /Fo$@ %s

obj\uibase.obj: uibase.cpp    uibase.h
    cl $(CFLAGS) /Fo$@ %s

```

`sha.obj` gets `/O2` (maximize speed) — MSVC's equivalent of GCC's `-O3`. The same logic applies: SHA-256 is the mining hot path, and this is the only file compiled with optimization. `/O2` enables inlining, loop unrolling, and instruction scheduling. The performance difference between `/Od` and `/O2` for tight computational loops like SHA-256 can easily be 5-10x.

```
obj\sha.obj: sha.cpp sha.h
    cl $(CFLAGS) /O2 /Fo$@ %s

obj\irc.obj:  irc.cpp         $(HEADERS)
    cl $(CFLAGS) /Fo$@ %s

```

Resource compilation. MSVC uses `rc` (the Resource Compiler) instead of MinGW's `windres`. The output is `obj\ui.res` — a compiled resource file that the linker embeds in the final executable. The dependency list is identical: the same icons and bitmaps are compiled into the same resources regardless of which toolchain builds them.

```
obj\ui.res: ui.rc  rc/bitcoin.ico rc/check.ico rc/send16.bmp rc/send16mask.bmp rc/send16masknoshadow.bmp rc/send20.bmp rc/send20mask.bmp rc/addressbook16.bmp rc/addressbook16mask.bmp rc/addressbook20.bmp rc/addressbook20mask.bmp
    rc $(INCLUDEPATHS) $(WXDEFS) /Fo$@ %s



OBJS=obj\util.obj obj\script.obj obj\db.obj obj\net.obj obj\main.obj obj\market.obj \
  obj\ui.obj obj\uibase.obj obj\sha.obj obj\irc.obj obj\ui.res

```

Link step. `link /nologo /DEBUG /SUBSYSTEM:WINDOWS /OUT:$@` invokes the MSVC linker. `/DEBUG` generates the `.pdb` debug database (enabling post-mortem debugging with WinDbg or Visual Studio). `/SUBSYSTEM:WINDOWS` marks the executable as a GUI application (equivalent to GCC's `-mwindows`). The `$**` NMAKE macro expands to all dependencies — the object files listed in `OBJS`.

```
bitcoin.exe: $(OBJS)
```

`kill /f bitcoin.exe & sleep 1` — same pattern as the MinGW makefile but with an added one-second delay. The `sleep 1` gives Windows time to release the file handle after killing the process — without it, the linker might fail with an 'access denied' error because Windows hasn't finished cleaning up the terminated process. The leading `-` suppresses errors if no Bitcoin process is running.

```
    -kill /f bitcoin.exe & sleep 1
    link /nologo /DEBUG /SUBSYSTEM:WINDOWS /OUT:$@ $(LIBPATHS) $** $(LIBS)

```

`clean` target — more thorough than the MinGW version. Besides deleting object files, it also removes `.ilk` (incremental linker state) and `.pdb` (program database / debug symbols) files. These are MSVC-specific artifacts that have no GCC equivalent. A clean build from this makefile removes all compiler and linker outputs.

```
clean:
    -del /Q obj\*
    -del *.ilk
    -del *.pdb

```

SUMMARY

The MSVC makefile is a near-mirror of the MinGW version, translated into Microsoft's toolchain dialect. The same ten C++ source files compile into the same executable with the same dependencies. The differences are syntactic (flag formats, path conventions, file extensions) rather than semantic. This parallelism was intentional — Satoshi maintained both build systems to ensure the code compiled correctly under two independent C++ implementations.

The choice to support MSVC 6.0 — a compiler from 1998 — is characteristic of Satoshi's conservative engineering. Rather than using the latest compiler features, he wrote code that worked with the lowest common denominator. This made Bitcoin buildable on the widest range of Windows development environments, lowering the barrier for other developers to compile and audit the code. It also forced a disciplined coding style: no C++11 features, limited template metaprogramming, and explicit workarounds for compiler-specific bugs.

Modern Bitcoin Core has long since dropped MSVC 6.0 support and no longer ships NMAKE files.[1] The build system migrated through autotools to CMake, supporting GCC, Clang, and modern MSVC across Linux, macOS, Windows, and various BSDs. But the fundamental build structure — compile source files with their dependencies, optimize the cryptographic hot path, link against system and third-party libraries, produce a single executable — remains unchanged from what Satoshi defined in these 77 lines.

KEY TERMS

MSVC

Microsoft Visual C++, the compiler and linker suite targeted by this makefile; the README names version 6.0 with Service Pack 6.

NMAKE

Microsoft's make utility, which reads makefile.vc and uses its own syntax, such as !IF and !ENDIF directives, instead of GNU Make's ifeq and endif.

/YX

An MSVC option that creates and reuses a precompiled header automatically; paired here with /Fp, which stores it as obj/headers.pch.

program database

The .pdb file MSVC writes when building with /Zi or linking with /DEBUG, holding the symbols a debugger needs to map machine code back to source.

rc

Microsoft's Resource Compiler, which turns ui.rc and its icon and bitmap files into a compiled .res file that the linker embeds in bitcoin.exe.

static library

A library whose code is copied into the executable at link time rather than loaded from a DLL; the trailing s in libdb47s.lib marks Berkeley DB's.

SOURCES

1. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
2. Massachusetts Institute of Technology, *MIT License* (1988). License chosen for Bitcoin v0.1.0 (license.txt). https://opensource.org/licenses/MIT

market.cpp

265 lines CPP

THE ROAD NOT TAKEN

This file is one of the most fascinating archaeological artifacts in Bitcoin's codebase: a partially-implemented eBay-style marketplace that Satoshi planned to build directly into the Bitcoin client. Products, reviews, seller reputation -- all embedded in the same application that managed your wallet and mined blocks. It is the clearest window into Satoshi's original vision of Bitcoin as a *complete economic system*, not merely a currency.

In a 2011 email to Mike Hearn, Satoshi explained: "I was trying to implement an eBay style marketplace built in to the client. Publish/subscribe would be used for broadcasting product offers and ratings/reviews. Your reviews would be weighted by the blocks you've generated."[1] The marketplace code uses a novel "atom" propagation system -- a form of trust diffusion where reputation flows through a social graph of reviewers. A seller gains atoms when a user reviews them; those atoms propagate outward through subsequent links, decaying as they spread. This was Satoshi's attempt to solve the reputation bootstrapping problem without a central authority.

The code was never completed. Commented-out notification callbacks, TODO comments about persistence (`//// later figure out how these are persisted`), and dead code blocks reveal a feature abandoned mid-development. Even the supply of reputation was unfinished: `AcceptBlock` in `main.cpp` still contains commented-out logic that would mint a random atom for whoever mined a block — disabled before launch, so atoms never auto-issued from mining.[2] Satoshi ultimately decided to focus on getting the core payment system right before layering commerce on top. The marketplace was stripped from later releases entirely — `market.cpp` is already gone in v0.3.0 (July 2010).[3] Projects like OpenBazaar (2014) and Bisq (launched as Bitsquare in 2016) would pursue this concept years later -- peer-to-peer commerce with decentralized reputation -- the same idea Satoshi had prototyped in January 2009.[4]

What remains is 264 lines of C++ that reveal a mind thinking far beyond digital cash. Satoshi was designing an entire trustless economy: money, commerce, and reputation, all woven together. The marketplace failed as software, but it

succeeded as prophecy.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

#include "headers.h"









//
// Global state variables
//

```

//// later figure out how these are persisted -- Satoshi's own TODO comment, preserved verbatim. This reveals the marketplace was still in the design phase when v0.1.0 shipped. mapMyProducts holds your own listed products in memory, but there is no code to save them to disk. Restart the client and your listings vanish. Satoshi never got around to adding BDB persistence for marketplace data the way he did for wallet transactions.

```
//// later figure out how these are persisted
map<uint256, CProduct> mapMyProducts;




```

mapProducts -- the global product catalog, mapping product hashes to CProduct objects. This is the marketplace equivalent of mapTransactions (the mempool). Products from all sellers across the network would live here. cs_mapProducts is the mutex protecting it from concurrent access -- the same threading pattern used throughout Bitcoin's core.

```
map<uint256, CProduct> mapProducts;
CCriticalSection cs_mapProducts;

```

AdvertInsert() -- adds or updates a product listing in the global marketplace. The logic is idempotent: if the product hash already exists, it only overwrites when the incoming nSequence number is higher (line 43), preventing stale advertisements from overwriting newer ones. This sequence-number versioning is a simple but effective way to handle out-of-order message delivery in a P2P network. (Address relay in net.cpp has no such counter; it just merges service bits.)

```
CALLED FROM  AdvertStartPublish net.h p. 365
bool AdvertInsert(const CProduct& product)
{
    uint256 hash = product.GetHash();
bool fNew = false;
bool fUpdated = false;

CRITICAL_BLOCK(cs_mapProducts)
{
// Insert or find existing product
        pair<map<uint256, CProduct>::iterator, bool> item = mapProducts.insert(make_pair(hash,
product));
```

```
        CProduct* pproduct = &(*(item.first)).second;
        fNew = item.second;

// Update if newer
if (product.nSequence > pproduct->nSequence)
{
*pproduct = product;
            fUpdated = true;
}
}

```

Three commented-out notification callbacks: `NotifyProductAdded`, `NotifyProductUpdated`, `NotifyProductDeleted`. These would have triggered UI updates when the marketplace changed -- the same observer pattern used by the wallet's `vWalletUpdated` signal. The fact that they are stubbed out but never implemented confirms this feature was abandoned before the UI integration was complete.

```
//if (fNew)
//    NotifyProductAdded(hash);
//else if (fUpdated)
//    NotifyProductUpdated(hash);

return (fNew || fUpdated);
}

```

`AdvertErase()` -- removes a product listing. Only six lines, using the same `CRITICAL_BLOCK` locking pattern found everywhere in Bitcoin. The commented-out `NotifyProductDeleted` call again shows incomplete UI wiring.

```
CALLED FROM  AdvertStopPublish net.h p. 365
void AdvertErase(const CProduct& product)
{
    uint256 hash = product.GetHash();
CRITICAL_BLOCK(cs_mapProducts)
        mapProducts.erase(hash);
//NotifyProductDeleted(hash);
}

```

`Union()` -- a generic sorted-set-union utility. The commented-out block above (lines 91-96) is Satoshi's earlier attempt using `std::set_union` with erase, replaced by the cleaner `back_inserter` version. He left the old code as a note to himself: `///// need to check that this is equivalent, then delete this comment`. He never came back to clean it up -- another sign of code frozen mid-development.

```
84 template<typename T>
   CALLED FROM  CUser::AddAtom p. 306
85 unsigned int Union(T& v1, T& v2)
86 {
87 // v1 = v1 union v2
88 // v1 and v2 must be sorted
89 // returns the number of elements added to v1
90
91 ///// need to check that this is equivalent, then delete this comment
92 //vector<unsigned short> vUnion(v1.size() + v2.size());
93 //vUnion.erase(set_union(v1.begin(), v1.end(),
94 //                       v2.begin(), v2.end(),
95 //                       vUnion.begin()),
96 //             vUnion.end());
97
98     T vUnion;
99     vUnion.reserve(v1.size() + v2.size());
100 set_union(v1.begin(), v1.end(),
101               v2.begin(), v2.end(),
102 back_inserter(vUnion));
103 unsigned int nAdded = vUnion.size() - v1.size();
104 if (nAdded > 0)
105         v1 = vUnion;
106 return nAdded;
107 }
108
```

`CUser::AddAtom()` -- the heart of Satoshi's reputation diffusion algorithm. Each user maintains three vectors: `vAtomsIn` (received atoms, sorted), `vAtomsNew` (pending atoms, unsorted), and `vAtomsOut` (atoms forwarded to downstream users). When a user accumulates enough new atoms (reaching `nFlowthroughRate`, set to 2 in market.h), a random atom is selected to propagate outward. This stochastic forwarding creates a trust signal that decays with social distance -- reviewers close to you carry more weight than distant strangers.

```
   CALLED FROM  AddAtomsAndPropagate p. 307
109 void CUser::AddAtom(unsigned short nAtom, bool fOrigin)
110 {
111 // Ignore duplicates
112 if (binary_search(vAtomsIn.begin(), vAtomsIn.end(), nAtom) ||
113 find(vAtomsNew.begin(), vAtomsNew.end(), nAtom) != vAtomsNew.end())
114 return;
115
116 //// instead of zero atom, should change to free atom that propagates,
117 //// limited to lower than a certain value like 5 so conflicts quickly
118 // The zero atom never propagates,
119 // new atoms always propagate through the user that created them
```

The zero atom (`nAtom == 0`) is special: it never propagates beyond the immediate recipient. Satoshi's comment explains this was a placeholder: `//// instead of zero atom, should change to free atom that propagates, limited to lower than a certain value like 5`. The design was still evolving. The `fOrigin` flag marks atoms created by the user themselves, which always propagate -- you always vouch for your own reputation.

```
120 if (nAtom == 0 || fOrigin)
121 {
122         vector<unsigned short> vTmp(1, nAtom);
123 Union(vAtomsIn, vTmp);
124 if (fOrigin)
125             vAtomsOut.push_back(nAtom);
126 return;
127 }
128
129     vAtomsNew.push_back(nAtom);
130
131 if (vAtomsNew.size() >= nFlowthroughRate || vAtomsOut.empty())
132 {
133 // Select atom to flow through to vAtomsOut
134         vAtomsOut.push_back(vAtomsNew[GetRand(vAtomsNew.size())]);
135
136 // Merge vAtomsNew into vAtomsIn
137 sort(vAtomsNew.begin(), vAtomsNew.end());
138 Union(vAtomsIn, vAtomsNew);
139         vAtomsNew.clear();
140 }
141 }
142
```

`AddAtomsAndPropagate()` -- breadth-first propagation through the social graph. Uses a double-buffer technique: `pmapPropagate[2]` alternates between 'current wave' and 'next wave' using `side = 1 - side` (line 149). For each user in the current wave, atoms are added via `AddAtom()`; any newly-generated outgoing atoms are queued for the next wave across all of that user's `vLinksOut`. The propagation stops naturally when no user produces new outgoing atoms. This is essentially a flood-fill algorithm over a directed trust graph, persisted through `CReviewDB`.

```
    CALLED FROM  CReview::AcceptReview p. 308
143 bool AddAtomsAndPropagate(uint256 hashUserStart, const vector<unsigned short>& vAtoms, bool fOrigin)
144 {
145     CReviewDB reviewdb;
146     map<uint256, vector<unsigned short> > pmapPropagate[2];
147     pmapPropagate[0][hashUserStart] = vAtoms;
148
149 for (int side = 0; !pmapPropagate[side].empty(); side = 1 - side)
150 {
151         map<uint256, vector<unsigned short> >& mapFrom = pmapPropagate[side];
152         map<uint256, vector<unsigned short> >& mapTo = pmapPropagate[1 - side];
153
154 for (map<uint256, vector<unsigned short> >::iterator mi = mapFrom.begin(); mi != mapFrom.end(); ++mi)
155 {
156 const uint256& hashUser = (*mi).first;
157 const vector<unsigned short>& vReceived = (*mi).second;
158
159 ///// this would be a lot easier on the database if it put the new atom at the beginning of the list,
160 ///// so the change would be right next to the vector size.
161
162 // Read user
163             CUser user;
164             reviewdb.ReadUser(hashUser, user);
165 unsigned int nIn = user.vAtomsIn.size();
166 unsigned int nNew = user.vAtomsNew.size();
167 unsigned int nOut = user.vAtomsOut.size();
```

```
168
169 // Add atoms received
170 foreach(unsigned short nAtom, vReceived)
171                 user.AddAtom(nAtom, fOrigin);
172             fOrigin = false;
173
174 // Don't bother writing to disk if no changes
175 if (user.vAtomsIn.size() == nIn && user.vAtomsNew.size() == nNew)
176 continue;
177
178 // Propagate
179 if (user.vAtomsOut.size() > nOut)
180 foreach(const uint256& hash, user.vLinksOut)
181                     mapTo[hash].insert(mapTo[hash].end(), user.vAtomsOut.begin() + nOut,
    user.vAtomsOut.end());
182
183 // Write back
184 if (!reviewdb.WriteUser(hashUser, user))
185 return false;
186 }
187         mapFrom.clear();
188 }
189 return true;
190 }
191
192
193
194
195
196
```

`CReview::AcceptReview()` -- validates and stores a user review. The sequence is: (1) timestamp it with current time, (2) verify the ECDSA signature against the reviewer's public key using the same `CKey::Verify()` used for transaction signatures, (3) append the review to the recipient's review list in BDB, (4) add a social link from reviewer to recipient, and (5) propagate reputation atoms. Reviews are cryptographically signed -- you cannot forge a review from someone else's identity. This is the same authentication model as Bitcoin transactions, applied to social trust.

CALLED FROM ProcessMessage main.cpp p. 235 · CEditReviewDialog::OnButtonSubmit ui.cpp p. 514

```
197 bool CReview::AcceptReview()
198 {
199 // Timestamp
200     nTime = GetTime();
201
202 // Check signature
203 if (!CKey::Verify(vchPubKeyFrom, GetSigHash(), vchSig))
204 return false;
205
206     CReviewDB reviewdb;
207
208 // Add review text to recipient
209     vector<CReview> vReviews;
210     reviewdb.ReadReviews(hashTo, vReviews);
211     vReviews.push_back(*this);
212 if (!reviewdb.WriteReviews(hashTo, vReviews))
213 return false;
214
215 // Add link from sender
216     CUser user;
217     uint256 hashFrom = Hash(vchPubKeyFrom.begin(), vchPubKeyFrom.end());
218     reviewdb.ReadUser(hashFrom, user);
219     user.vLinksOut.push_back(hashTo);
220 if (!reviewdb.WriteUser(hashFrom, user))
221 return false;
222
223     reviewdb.Close();
224
```

```
225 // Propagate atoms to recipient
```

When propagating atoms after a review, if the reviewer has no atoms of their own, a 'zero atom' vector is sent instead (line 227). This ensures that even a brand-new user with no reputation can still create a link in the trust graph. The zero atom, which does not propagate further, acts as a minimal seed -- acknowledging the connection exists without granting transitive reputation.

```
226     vector<unsigned short> vZeroAtom(1, 0);
227 if (!AddAtomsAndPropagate(hashTo, user.vAtomsOut.size() ? user.vAtomsOut : vZeroAtom, false))
228 return false;
229
230 return true;
231 }
232
233
234
235
236
```

`CProduct::CheckSignature()` and `CProduct::CheckProduct()` -- product validation. Like transactions, product listings are cryptographically signed by the seller. `CheckProduct()` enforces that only 'summary' products (no details or order form data) pass validation -- detailed product information would presumably be fetched on demand to save bandwidth. The seller's atom count is looked up from the review database (line 255), giving each product a reputation score derived from the trust graph.

```
    CALLED FROM  CProduct::CheckProduct p. 308 · CViewProductDialog::OnReply1 ui.cpp p. 510
237 bool CProduct::CheckSignature()
238 {
239 return (CKey::Verify(vchPubKeyFrom, GetSigHash(), vchSig));
240 }
241
    CALLED FROM  CEditProductDialog::OnButtonSend ui.cpp p. 508
242 bool CProduct::CheckProduct()
243 {
244 if (!CheckSignature())
245 return false;
246
247 // Make sure it's a summary product
248 if (!mapDetails.empty() || !vOrderForm.empty())
249 return false;
250
251 // Look up seller's atom count
252     CReviewDB reviewdb("r");
253     CUser user;
254     reviewdb.ReadUser(GetUserHash(), user);
255     nAtoms = user.GetAtomCount();
256     reviewdb.Close();
257
258 ////// delme, this is now done by AdvertInsert
259 //// Store to memory
260 //CRITICAL_BLOCK(cs_mapProducts)
261 //    mapProducts[GetHash()] = *this;
262
263 return true;
264 }
265
```

SUMMARY

The marketplace code reveals that Satoshi's ambition extended far beyond digital cash. He was building a decentralized commerce platform where products, reviews, and reputation all operated without trusted intermediaries. The atom propagation system -- a form of trust diffusion through social graphs -- was a genuinely novel approach to decentralized reputation, years ahead of its time.

The decision to abandon this feature was arguably one of the most important editorial choices in Bitcoin's history. By shipping a focused, minimal product -- send money, receive money, mine blocks -- Satoshi ensured the core payment protocol was robust before layering complexity on top. Had the marketplace shipped in its half-finished state, bugs in the reputation system could have undermined confidence in Bitcoin itself.

The marketplace's ghost lives on in Bitcoin's DNA. The `CProduct` and `CReview` classes are still compiled into v0.1.0. The UI has product listing dialogs (see ui.h). The P2P protocol has message types for advertising products. All of it functional enough to hint at what might have been, incomplete enough to remind us that even Satoshi had to choose what to ship and what to cut.

KEY TERMS

atom propagation

The wave-by-wave process in AddAtomsAndPropagate that adds atoms to a user, then passes any newly forwarded atoms to everyone that user reviewed, until no one forwards anything new.

zero atom

The atom numbered 0, sent by a reviewer who has no outgoing atoms of their own; its recipient records it but never forwards it further.

origin atom

An atom added with fOrigin set, meaning the user created it; it is recorded and queued for forwarding at once, a path only the commented-out mining-reward code in main.cpp would have used.

trust link

An entry in a user's vLinksOut, added by AcceptReview when that user reviews someone, forming the directed edges along which reputation atoms flow.

AdvertInsert

The function that adds a product to mapProducts, or replaces an existing entry with the same hash only when the incoming listing carries a higher nSequence.

AcceptReview

The routine run on each incoming review message that verifies its signature, appends it to the recipient's stored reviews, records the reviewer's trust link, and propagates atoms.

summary product

A product listing with no details or order form, the only kind CheckProduct accepts; the check also stamps it with the seller's current atom count from the review database.

CReviewDB

The CDB subclass for reviews.dat, storing each user's atom state under a "user" key and the reviews they have received under a "reviews" key.

SOURCES

1. Satoshi Nakamoto, *Emails to Mike Hearn* (2009–2011). Includes coin supply rationale and the abandoned eBay-style marketplace. https://plan99.net/~mike/satoshi-emails/thread4.html

2. Jeremy Rubin, *satoshis-version — Annotated Bitcoin v0.01 codebase* (2019). Line-by-line //# comments on Satoshi's original release; best read commit-by-commit. https://github.com/JeremyRubin/satoshis-version/
3. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
4. Satoshi Nakamoto Institute, *Bitcoin v0.1.0 Source Code Archive* (2009). https://satoshi.nakamotoinstitute.org/code/

market.h

183 lines CPP

SATOSHI'S UNFINISHED DREAM

This header defines the data structures for Satoshi's abandoned built-in marketplace: users with reputation atoms, cryptographically signed reviews, and product listings with order forms. Together with `market.cpp`, these 182 lines describe a complete peer-to-peer commerce system that was prototyped, partially wired into the UI, and then quietly shelved before Bitcoin's public release.

The three classes here -- `CUser`, `CReview`, and `CProduct` -- mirror the architecture of Bitcoin's transaction system. Products are signed like transactions. Reviews are verified with ECDSA like spending scripts. Users accumulate reputation through a novel "atom" propagation algorithm instead of a centralized rating database. Satoshi applied the same trust model everywhere: mathematics replaces authority.

The header also reveals Satoshi's approach to serialization. Every marketplace class uses the same `IMPLEMENT_SERIALIZE` macro found in `main.h` for transactions and blocks. The conditional serialization flags -- `SER_GETHASH`, `SER_DISK`, `SER_SKIPSIG` -- ensure that signature fields are excluded when computing hashes (preventing circular dependencies) and that disk-only fields like atom counts are not sent over the network. This layered serialization design, used consistently across all Bitcoin data structures, was one of Satoshi's most practical engineering decisions.

Services like OpenBazaar (2014), Bisq (launched as Bitsquare in 2016), and Particl (2017) would later attempt to build what Satoshi sketched here -- decentralized marketplaces with integrated reputation systems. Particl went so far as to present its marketplace as a continuation of this unfinished project, quoting Satoshi's 2011 email to Mike Hearn about an "eBay style marketplace built in to the client."[1]

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

```

Forward declarations for the three marketplace types. C++ requires these because the classes reference each other: `CProduct` uses `CUser` to look up atom counts, and `CReview` creates links between `CUser` objects. The forward declarations break the circular dependency without requiring separate header files.

```
class CUser;
class CReview;
class CProduct;

```

`nFlowthroughRate` = 2 -- the threshold controlling how quickly reputation propagates through the social graph. When a user accumulates 2 new atoms in `vAtomsNew`, one is randomly selected to flow through to `vAtomsOut` and propagate to downstream users. A lower value means reputation spreads faster but is more easily gamed; a higher value means slower propagation but more resistance to Sybil attacks. Satoshi chose 2 as a balance -- every second atom received generates one outgoing atom, a 50% propagation rate.

```
static const unsigned int nFlowthroughRate = 2;



bool AdvertInsert(const CProduct& product);
void AdvertErase(const CProduct& product);
bool AddAtomsAndPropagate(uint256 hashUserStart, const vector<unsigned short>& vAtoms, bool fOrigin);









```

`CUser` -- a node in the reputation graph. Each user maintains three atom vectors: `vAtomsIn` (all atoms received, kept sorted for binary search), `vAtomsNew` (atoms pending propagation), and `vAtomsOut` (atoms forwarded to downstream users). `vLinksOut` stores the hashes of users this person has reviewed, forming the directed edges of the trust graph. The design is essentially a weighted directed graph where edge weights are atom counts.

```
class CUser
{
public:
    vector<unsigned short> vAtomsIn;
    vector<unsigned short> vAtomsNew;
    vector<unsigned short> vAtomsOut;
    vector<uint256> vLinksOut;

CUser()
{
}

```

`IMPLEMENT_SERIALIZE` -- the same serialization macro used for `CTransaction` and `CBlock`. The `SER_GETHASH` flag check on line 39 ensures that the version number is excluded when computing the user's hash -- preventing the hash from changing when the serialization format is upgraded. All four vectors are serialized: the complete atom state and social graph links are persisted to BDB via `CReviewDB`.

```
37     IMPLEMENT_SERIALIZE
38 (
39 if (!(nType & SER_GETHASH))
40 READWRITE(nVersion);
41 READWRITE(vAtomsIn);
42 READWRITE(vAtomsNew);
43 READWRITE(vAtomsOut);
44 READWRITE(vLinksOut);
45 )
46
47 void SetNull()
48 {
49         vAtomsIn.clear();
50         vAtomsNew.clear();
51         vAtomsOut.clear();
52         vLinksOut.clear();
53 }
54
55     uint256 GetHash() const { return SerializeHash(*this); }
56
57
```

`GetAtomCount()` returns the total number of atoms a user has accumulated: both fully processed atoms (`vAtomsIn`) and atoms still pending propagation (`vAtomsNew`). This count becomes a product's reputation score in `CProduct::CheckProduct()` -- a seller's trustworthiness is literally measured by how many reputation atoms have flowed to them through the review graph.

```
58 int GetAtomCount() const
59 {
60 return (vAtomsIn.size() + vAtomsNew.size());
61 }
62
63 void AddAtom(unsigned short nAtom, bool fOrigin);
64 };
65
66
67
68
69
70
71
```

`CReview` -- a cryptographically signed review. `hashTo` identifies the recipient (a product or user being reviewed). `mapValue` stores arbitrary key-value pairs -- the review text, rating, title, etc. `vchPubKeyFrom` is the reviewer's public key, and `vchSig` is the ECDSA signature over the review's hash. The `nTime` and `nAtoms` fields are memory-only (not serialized to disk), computed fresh each time.

```
72 class CReview
73 {
74 public:
75 int nVersion;
76     uint256 hashTo;
77     map<string, string> mapValue;
78     vector<unsigned char> vchPubKeyFrom;
```

```
79     vector<unsigned char> vchSig;
80
81 // memory only
82 unsigned int nTime;
83 int nAtoms;
84
85
86 CReview()
87 {
88         nVersion = 1;
89         hashTo = 0;
90         nTime = 0;
91         nAtoms = 0;
92 }
93
```

Review serialization is carefully layered. `hashTo` is excluded when serializing for disk (`SER_DISK` check, line 98) because on disk, reviews are stored in a list keyed by the recipient hash -- writing `hashTo` inside each review would be redundant. The signature is excluded from the hash computation (`SER_GETHASH` check, line 102) to avoid the bootstrap problem: you cannot sign a hash that includes the signature itself.

```
94     IMPLEMENT_SERIALIZE
95 (
96 READWRITE(this->nVersion);
97         nVersion = this->nVersion;
98 if (!(nType & SER_DISK))
99 READWRITE(hashTo);
100 READWRITE(mapValue);
101 READWRITE(vchPubKeyFrom);
102 if (!(nType & SER_GETHASH))
103 READWRITE(vchSig);
104 )
105
```

Three hash functions on a single class: `GetHash()` for unique identification, `GetSigHash()` for the bytes that get signed (excluding the signature itself via the `SER_GETHASH` check), and `GetUserHash()` which hashes the public key to derive a user identity. This triple-hash pattern reappears in `CProduct` below -- Satoshi reused the same authentication architecture across all marketplace objects.

```
106     uint256 GetHash() const { return SerializeHash(*this); }
107     uint256 GetSigHash() const { return SerializeHash(*this, SER_GETHASH|SER_SKIPSIG); }
108     uint256 GetUserHash() const { return Hash(vchPubKeyFrom.begin(), vchPubKeyFrom.end()); }
109
110
111 bool AcceptReview();
112 };
113
114
115
116
117
118
119
```

`CProduct` -- a marketplace listing. `addr` is the seller's network address for direct IP-to-IP transactions. `mapValue` holds summary fields (title, price, category). `mapDetails` holds extended product information fetched on demand. `vOrderForm` defines the order form fields a buyer must fill out (name-value pairs like 'shipping address', 'quantity'). `nSequence` is a version counter -- higher values replace older listings.

```
120 class CProduct
121 {
122 public:
123 int nVersion;
124     CAddress addr;
125     map<string, string> mapValue;
126     map<string, string> mapDetails;
127     vector<pair<string, string> > vOrderForm;
128 unsigned int nSequence;
129     vector<unsigned char> vchPubKeyFrom;
130     vector<unsigned char> vchSig;
131
```

`nAtoms` is marked 'disk only' -- the seller's reputation score is stored alongside the product on disk but not transmitted over the network. Each node independently computes the atom count by querying its own review database (`CheckProduct()`, line 254 of market.cpp). This prevents sellers from lying about their reputation; every node verifies it independently, the same trust model as transaction validation.

```
132 // disk only
133 int nAtoms;
134
135 // memory only
136     set<unsigned int> setSources;
137
138 CProduct()
139 {
140         nVersion = 1;
141         nAtoms = 0;
142         nSequence = 0;
143 }
144
```

Product serialization separates summary data from detail data. `mapDetails`, `vOrderForm`, and `nSequence` are excluded from the hash (`SER_GETHASH` check, line 151). This means a product's identity hash is determined solely by its address, summary values, and public key -- the details can be updated without changing the product's hash. A practical optimization: search results show summaries, and full details are fetched only when a buyer clicks through.

```
145     IMPLEMENT_SERIALIZE
146 (
147 READWRITE(this->nVersion);
148         nVersion = this->nVersion;
149 READWRITE(addr);
150 READWRITE(mapValue);
151 if (!(nType & SER_GETHASH))
152 {
153 READWRITE(mapDetails);
154 READWRITE(vOrderForm);
155 READWRITE(nSequence);
156 }
157 READWRITE(vchPubKeyFrom);
158 if (!(nType & SER_GETHASH))
159 READWRITE(vchSig);
```

```
160 if (nType & SER_DISK)
161 READWRITE(nAtoms);
162 )
163
164     uint256 GetHash() const { return SerializeHash(*this); }
165     uint256 GetSigHash() const { return SerializeHash(*this, SER_GETHASH|SER_SKIPSIG); }
166     uint256 GetUserHash() const { return Hash(vchPubKeyFrom.begin(), vchPubKeyFrom.end()); }
167
168
169 bool CheckSignature();
170 bool CheckProduct();
171 };
172
173
174
175
176
177
178
179
```

Global state declarations: `mapProducts` is the in-memory product catalog (equivalent to the mempool for transactions), `cs_mapProducts` is its mutex guard, and `mapMyProducts` holds your own listings. The `extern` keyword means these are defined in `market.cpp` and shared across all translation units that include this header -- the same global-state pattern used for `mapTransactions` and `mapBlockIndex` in `main.cpp`.

```
180 extern map<uint256, CProduct> mapProducts;
181 extern CCriticalSection cs_mapProducts;
182 extern map<uint256, CProduct> mapMyProducts;
183
```

SUMMARY

The marketplace header reveals a carefully thought-out architecture. Satoshi did not simply bolt e-commerce onto a payment system -- he designed the marketplace classes to use exactly the same patterns as the rest of Bitcoin: ECDSA signatures for authentication, layered serialization for efficiency, hash-based identity for deduplication, and mutex-protected global maps for concurrent access. The marketplace was a first-class citizen of the protocol.

The atom-based reputation system is particularly notable. Rather than a simple star rating (trivially gamed by Sybil accounts), Satoshi designed a trust propagation network where reputation must flow through real social connections. The stochastic forwarding (randomly selecting which atoms propagate) provides natural resistance to reputation inflation. It is a genuinely original design that anticipated later work on decentralized reputation by several years.

That Satoshi abandoned this feature speaks to his engineering discipline. Rather than shipping a half-working marketplace that could undermine trust in the core payment system, he cut it loose and focused on the consensus layer. The marketplace classes remain in v0.1.0 as compiled-but-unused code -- a fascinating glimpse of the larger system Satoshi imagined but chose not to build.[2]

KEY TERMS

CUser

A participant in the marketplace's reputation graph, stored in reviews.dat under a user hash, holding received, pending, and forwarded atoms plus links to the users it has reviewed.

CReview

A signed marketplace review: the recipient's hash, free-form key-value fields, the reviewer's public key, and an ECDSA signature over everything except the signature itself.

CProduct

A signed marketplace listing carrying the seller's network address, summary fields, optional details and order form, a sequence number, and the seller's public key and signature.

reputation atom

A small unsigned number serving as one unit of reputation; atoms flow from reviewer to reviewed, and a user's score is simply how many distinct atoms have reached them.

nFlowthroughRate

The constant 2 governing atom forwarding: once a user has collected this many new atoms, or has never forwarded one, a randomly chosen new atom flows on to the users they reviewed.

user hash

A marketplace participant's identity, computed by GetUserHash as the double-SHA256 hash of their public key, so reputation attaches to a key rather than to a name.

mapProducts

The global, mutex-guarded in-memory catalog of product listings keyed by product hash; its sibling mapMyProducts holds the user's own listings.

SOURCES

1. Satoshi Nakamoto, *Emails to Mike Hearn* (2009–2011). Includes coin supply rationale and the abandoned eBay-style marketplace. https://plan99.net/~mike/satoshi-emails/thread4.html
2. Satoshi Nakamoto Institute, *Bitcoin v0.1.0 Source Code Archive* (2009). https://satoshi.nakamotoinstitute.org/code/

net.cpp

1021 lines CPP

THE GOSSIP NETWORK

Bitcoin operates as a peer-to-peer network with no central server. This file implements all networking: node discovery, connection management, message routing, and the gossip protocol that propagates transactions and blocks across the globe in seconds. At any moment, thousands of nodes are running descendants of this code, forming the backbone of the Bitcoin network.

Satoshi's design goal from the whitepaper's opening line: "A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution."[1] This file makes that vision technically possible. There's no Bitcoin Inc. server—just a mesh of equal peers, each running the same rules.

The network is resilient by design. Nodes join and leave freely. Messages propagate via gossip—each node tells its neighbors, who tell theirs. If half the network vanished tomorrow, the other half would continue producing blocks (slowly at first, until the difficulty adjusted). This architecture has survived government crackdowns, DDoS attacks, and internet outages across seventeen years without a single hour of downtime.

The protocol uses a simple message format: 4-byte magic number (to identify the Bitcoin network), 12-byte command name, 4-byte payload size, then the payload. Messages include: "version" (handshake), "addr" (peer addresses), "inv" (announce data), "getdata" (request data), "tx" (transaction), "block" (block data). This simplicity enabled implementations in dozens of languages.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

#include "headers.h"
#include <winsock2.h>

void ThreadMessageHandler2(void* parg);
void ThreadSocketHandler2(void* parg);
void ThreadOpenConnections2(void* parg);




//
// Global state variables
//
```

`fClient` — whether to run in lightweight mode without full blockchain. If true, NODE_NETWORK bit is cleared from nLocalServices, signaling to peers that this node cannot serve historical blocks.

```
bool fClient = false;
```

`nLocalServices` — bitmask of capabilities advertised to peers. NODE_NETWORK (bit 0) means the node stores and serves the full blockchain. Nodes use this to decide which peers to ask for block data.

```
uint64 nLocalServices = (fClient ? 0 : NODE_NETWORK);
CAddress addrLocalHost(0, DEFAULT_PORT, nLocalServices);
CNode nodeLocalHost(INVALID_SOCKET, CAddress("127.0.0.1", nLocalServices));
CNode* pnodeLocalHost = &nodeLocalHost;
bool fShutdown = false;
```

`vfThreadRunning` — status flags for the four worker threads (0 socket, 1 open-connections, 2 message-handler, 3 miner). Plain `bool`s, not atomics — fine on the single-writer pattern Satoshi used, fragile by modern standards.[2]

```
array<bool, 10> vfThreadRunning;
```

`vNodes` — the live peer roster. Every TCP connection to another Bitcoin node is represented here as a CNode*. The socket handler, message handler, and connection opener all share this list via cs_vNodes. In v0.1.0 the connection opener stops seeking new peers once vNodes reaches 15.

```
vector<CNode*> vNodes;
CCriticalSection cs_vNodes;
```

`mapAddresses` — the address book of known peers. Keys are serialized IP+port; values are CAddress structs with last-seen time and service flags. Persisted to addr.dat on disk so peers survive restarts.

```
map<vector<unsigned char>, CAddress> mapAddresses;
CCriticalSection cs_mapAddresses;
```

`mapRelay` — short-lived cache for relayed objects. When a node broadcasts a transaction it stores it here for 15 minutes so peers who ask for it via getdata can receive it. vRelayExpiration tracks when entries expire.

```
map<CInv, CDataStream> mapRelay;
deque<pair<int64, CInv> > vRelayExpiration;
CCriticalSection cs_mapRelay;
```

`mapAlreadyAskedFor` — throttles duplicate requests. When we ask a peer for an object (by CInv hash), we record the time here. If another peer announces the same object, we skip asking until the timeout expires — preventing bandwidth waste.

```
map<CInv, int64> mapAlreadyAskedFor;


CAddress addrProxy;

```

`ConnectSocket` — creates a raw TCP connection to another node. Checks whether the target IP is a private (RFC 1918) address; if a SOCKS4 proxy is configured and the target is routable, routes through it instead of connecting directly.

```
CALLED FROM  ThreadIRCSeed irc.cpp p. 165 · GetMyExternalIP p. 326 · ConnectNode p. 330
bool ConnectSocket(const CAddress& addrConnect, SOCKET& hSocketRet)
{
    hSocketRet = INVALID_SOCKET;

    SOCKET hSocket = socket(AF_INET, SOCK_STREAM, 0);
if (hSocket == INVALID_SOCKET)
return false;

```

Private IP detection: 10.x.x.x and 192.168.x.x are RFC 1918 private ranges, not routable on the public internet. Satoshi skips proxy for private IPs since they're presumably on the same LAN.

```
bool fRoutable = !(addrConnect.GetByte(3) == 10 || (addrConnect.GetByte(3) == 192 &&
addrConnect.GetByte(2) == 168));
bool fProxy = (addrProxy.ip && fRoutable);
struct sockaddr_in sockaddr = (fProxy ? addrProxy.GetSockAddr() : addrConnect.GetSockAddr());

if (connect(hSocket, (struct sockaddr*)&sockaddr, sizeof(sockaddr)) == SOCKET_ERROR)
{
closesocket(hSocket);
return false;
}

if (fProxy)
{
printf("Proxy connecting to %s\n", addrConnect.ToString().c_str());
```

SOCKS4 handshake: the 13-byte request encodes VN=4, CD=1 (CONNECT), the destination port (big-endian), and IP (4 bytes). The string "user" is a null-terminated USERID field. A 0×5A response byte means the proxy accepted the connection.

```
char pszSocks4IP[] = "\4\1\0\0\0\0\0\0user";
memcpy(pszSocks4IP + 2, &addrConnect.port, 2);
```

```
63 memcpy(pszSocks4IP + 4, &addrConnect.ip, 4);
64 char* pszSocks4 = pszSocks4IP;
65 int nSize = sizeof(pszSocks4IP);
66
67 int ret = send(hSocket, pszSocks4, nSize, 0);
68 if (ret != nSize)
69 {
70 closesocket(hSocket);
71 return error("Error sending to proxy\n");
72 }
73 char pchRet[8];
74 if (recv(hSocket, pchRet, 8, 0) != 8)
75 {
76 closesocket(hSocket);
77 return error("Error reading proxy response\n");
78 }
79 if (pchRet[1] != 0x5a)
80 {
81 closesocket(hSocket);
82 return error("Proxy returned error %d\n", pchRet[1]);
83 }
84 printf("Proxy connection established %s\n", addrConnect.ToString().c_str());
85 }
86
87     hSocketRet = hSocket;
88 return true;
89 }
90
91
```

`GetMyExternalIP` — discovers the node's public IP by HTTP GET to whatismyip.com. Needed behind NAT so peers can dial inbound. The service has no obligation to reply faithfully — a quiet trust dependency on a third party for identity.[2]

```
   CALLED FROM  StartNode p. 341
92 bool GetMyExternalIP(unsigned int& ipRet)
93 {
94     CAddress addrConnect("72.233.89.199:80"); // whatismyip.com 198-200
95
96     SOCKET hSocket;
97 if (!ConnectSocket(addrConnect, hSocket))
98 return error("GetMyExternalIP() : connection to %s failed\n", addrConnect.ToString().c_str());
99
100 char* pszGet =
101 "GET /automation/n09230945.asp HTTP/1.1\r\n"
102 "Host: www.whatismyip.com\r\n"
103 "User-Agent: Bitcoin/0.1\r\n"
104 "Connection: close\r\n"
105 "\r\n";
106 send(hSocket, pszGet, strlen(pszGet), 0);
107
108     string strLine;
109 while (RecvLine(hSocket, strLine))
110 {
111 if (strLine.empty())
112 {
113 if (!RecvLine(hSocket, strLine))
114 {
115 closesocket(hSocket);
116 return false;
117 }
118 closesocket(hSocket);
119             CAddress addr(strLine.c_str());
120 printf("GetMyExternalIP() received [%s] %s\n", strLine.c_str(), addr.ToString().c_str());
121 if (addr.ip == 0)
122 return false;
123             ipRet = addr.ip;
124 return true;
125 }
```

```
126 }
127 closesocket(hSocket);
128 return error("GetMyExternalIP() : connection closed\n");
129 }
130
131
132
133
134
135
```

`AddAddress` — adds a peer to the address book, deduplicating by key. Only stores routable (public internet) IPs; ignores our own address. If services bits have expanded for a known address, updates the record.

```
    CALLED FROM CAddrDB::LoadAddresses db.cpp p. 127 · ThreadIRCSeed irc.cpp p. 168 ·
    ProcessMessage main.cpp p. 232
136 bool AddAddress(CAddrDB& addrdb, const CAddress& addr)
137 {
138 if (!addr.IsRoutable())
139 return false;
140 if (addr.ip == addrLocalHost.ip)
141 return false;
142 CRITICAL_BLOCK(cs_mapAddresses)
143 {
144         map<vector<unsigned char>, CAddress>::iterator it = mapAddresses.find(addr.GetKey());
145 if (it == mapAddresses.end())
146 {
147 // New address
148             mapAddresses.insert(make_pair(addr.GetKey(), addr));
149             addrdb.WriteAddress(addr);
150 return true;
151 }
152 else
153 {
154             CAddress& addrFound = (*it).second;
155 if ((addrFound.nServices | addr.nServices) != addrFound.nServices)
156 {
157 // Services have been added
158                 addrFound.nServices |= addr.nServices;
159                 addrdb.WriteAddress(addrFound);
160 return true;
161 }
162 }
163 }
164 return false;
165 }
166
167
168
169
170
171 void AbandonRequests(void (*fn)(void*, CDataStream&), void* param1)
172 {
173 // If the dialog might get closed before the reply comes back,
174 // call this in the destructor so it doesn't get called after it's deleted.
175 CRITICAL_BLOCK(cs_vNodes)
176 {
177 foreach(CNode* pnode, vNodes)
178 {
179 CRITICAL_BLOCK(pnode->cs_mapRequests)
180 {
181 for (map<uint256, CRequestTracker>::iterator mi = pnode->mapRequests.begin(); mi != pnode-
    >mapRequests.end();)
182 {
183                     CRequestTracker& tracker = (*mi).second;
184 if (tracker.fn == fn && tracker.param1 == param1)
185                         pnode->mapRequests.erase(mi++);
```

```
188 }
189 }
190 }
191 }
192 }
193
194
195
196
197
198
199
200 //
201 // Subscription methods for the broadcast and subscription system.
202 // Channel numbers are message numbers, i.e. MSG_TABLE and MSG_PRODUCT.
203 //
204 // The subscription system uses a meet-in-the-middle strategy.
205 // With 100,000 nodes, if senders broadcast to 1000 random nodes and receivers
206 // subscribe to 1000 random nodes, 99.995% (1 - 0.99^1000) of messages will get through.
207 //
208
```

Pub/sub system — Bitcoin v0.1.0 included a publish-subscribe layer for relaying marketplace messages (products and orders). Nodes could subscribe to channels (MSG_TABLE, MSG_PRODUCT) and receive broadcasts from any subscriber. This was the embryo of an online marketplace — and only half of it was built: v0.1.0 sends `subscribe` and `publish` messages, but `ProcessMessage` in main.cpp has no handler for them, so a receiving node logs 'Ignored unknown message' and drops them. The marketplace code (market.cpp) was gone by v0.3.0.[3]

```
    CALLED FROM  CNode::Subscribe p. 328 · CNode::CancelSubscribe p. 329
209 bool AnySubscribed(unsigned int nChannel)
210 {
211 if (pnodeLocalHost->IsSubscribed(nChannel))
212 return true;
213 CRITICAL_BLOCK(cs_vNodes)
214 foreach(CNode* pnode, vNodes)
215 if (pnode->IsSubscribed(nChannel))
216 return true;
217 return false;
218 }
219
    CALLED FROM  AnySubscribed p. 328 · AdvertStartPublish net.h p. 365 · AdvertStopPublish net.h p. 365
220 bool CNode::IsSubscribed(unsigned int nChannel)
221 {
222 if (nChannel >= vfSubscribe.size())
223 return false;
224 return vfSubscribe[nChannel];
225 }
226
```

`CNode::Subscribe` — if no other node is currently subscribed to a channel, propagates a "subscribe" message to all peers (meet-in-the-middle relay). The comment calculates that with 100,000 nodes each subscribing to 1,000 random peers, 99.995% message delivery is achievable.

```
227 void CNode::Subscribe(unsigned int nChannel, unsigned int nHops)
228 {
229 if (nChannel >= vfSubscribe.size())
230 return;
231
232 if (!AnySubscribed(nChannel))
233 {
234 // Relay subscribe
235 CRITICAL_BLOCK(cs_vNodes)
236 foreach(CNode* pnode, vNodes)
```

```
if (pnode != this)
                    pnode->PushMessage("subscribe", nChannel, nHops);
}

    vfSubscribe[nChannel] = true;
}

```

CALLED FROM CNode::Disconnect p. 331

```
void CNode::CancelSubscribe(unsigned int nChannel)
{
if (nChannel >= vfSubscribe.size())
return;

// Prevent from relaying cancel if wasn't subscribed
if (!vfSubscribe[nChannel])
return;
    vfSubscribe[nChannel] = false;

if (!AnySubscribed(nChannel))
{
// Relay subscription cancel
CRITICAL_BLOCK(cs_vNodes)
foreach(CNode* pnode, vNodes)
if (pnode != this)
                    pnode->PushMessage("sub-cancel", nChannel);

// Clear memory, no longer subscribed
if (nChannel == MSG_PRODUCT)
CRITICAL_BLOCK(cs_mapProducts)
                mapProducts.clear();
}
}
```

`FindNode` — O(n) search through vNodes. With at most a few dozen peers (the opener stops at 15) this is fast enough. Returns the CNode* if connected, NULL otherwise. Two overloads: one searches by raw IP, the other by full CAddress (IP+port).

```
CNode* FindNode(unsigned int ip)
{
CRITICAL_BLOCK(cs_vNodes)
{
foreach(CNode* pnode, vNodes)
if (pnode->addr.ip == ip)
return (pnode);
}
return NULL;
}

CNode* FindNode(CAddress addr)
{
CRITICAL_BLOCK(cs_vNodes)
{
foreach(CNode* pnode, vNodes)
if (pnode->addr == addr)
return (pnode);
}
return NULL;
}

```

`ConnectNode` — high-level peer connection: returns existing connection if already connected, otherwise calls ConnectSocket and creates a new CNode, adding it to vNodes. On failure, records the current time as nLastFailed to implement retry backoff.

CALLED FROM ThreadOpenConnections2 p. 338 · CSendingDialog::StartTransfer ui.cpp p. 499 · CSendingDialog::OnReply2 ui.cpp p. 500 · ThreadRequestProductDetails ui.cpp p. 510

```
299 CNode* ConnectNode(CAddress addrConnect, int64 nTimeout)
300 {
301 if (addrConnect.ip == addrLocalHost.ip)
302 return NULL;
303
304 // Look for an existing connection
305     CNode* pnode = FindNode(addrConnect.ip);
306 if (pnode)
307 {
308 if (nTimeout != 0)
309             pnode->AddRef(nTimeout);
310 else
311             pnode->AddRef();
312 return pnode;
313 }
314
315 /// debug print
316 printf("trying %s\n", addrConnect.ToString().c_str());
317
318 // Connect
319     SOCKET hSocket;
320 if (ConnectSocket(addrConnect, hSocket))
321 {
322 /// debug print
323 printf("connected %s\n", addrConnect.ToString().c_str());
324
325 // Add node
326         CNode* pnode = new CNode(hSocket, addrConnect, false);
327 if (nTimeout != 0)
328             pnode->AddRef(nTimeout);
329 else
330             pnode->AddRef();
331 CRITICAL_BLOCK(cs_vNodes)
332             vNodes.push_back(pnode);
333
334 CRITICAL_BLOCK(cs_mapAddresses)
335             mapAddresses[addrConnect.GetKey()].nLastFailed = 0;
336 return pnode;
337 }
338 else
339 {
340 CRITICAL_BLOCK(cs_mapAddresses)
341             mapAddresses[addrConnect.GetKey()].nLastFailed = GetTime();
342 return NULL;
343 }
344 }
345
```

CALLED FROM ThreadSocketHandler2 p. 332 · CMainFrameBase::~CMainFrameBase uibase.cpp p. 547 · CTxDetailsDialogBase::~CTxDetailsDialogBase uibase.cpp p. 548 · COptionsDialogBase::~COptionsDialogBase uibase.cpp p. 549 · CAboutDialogBase::~CAboutDialogBase uibase.cpp p. 551 · CSendDialogBase::~CSendDialogBase uibase.cpp p. 552 · *and 11 more*

```
346 void CNode::Disconnect()
347 {
348 printf("disconnecting node %s\n", addr.ToString().c_str());
349
350 closesocket(hSocket);
351
352 // All of a nodes broadcasts and subscriptions are automatically torn down
353 // when it goes down, so a node has to stay up to keep its broadcast going.
354
```

```
for (map<uint256, CProduct>::iterator mi = mapProducts.begin(); mi != mapProducts.end();)
AdvertRemoveSource(this, MSG_PRODUCT, 0, (*(mi++)).second);

// Cancel subscriptions
for (unsigned int nChannel = 0; nChannel < vfSubscribe.size(); nChannel++)
if (vfSubscribe[nChannel])
CancelSubscribe(nChannel);
}
```

CALLED FROM StartNode p. 342

```
void ThreadSocketHandler(void* parg)
{
IMPLEMENT_RANDOMIZE_STACK(ThreadSocketHandler(parg));

    loop
{
        vfThreadRunning[0] = true;
CheckForShutdown(0);
try
{
ThreadSocketHandler2(parg);
}
CATCH_PRINT_EXCEPTION("ThreadSocketHandler()")
        vfThreadRunning[0] = false;
Sleep(5000);
}
}
```

`ThreadSocketHandler2` — the I/O engine. Runs in a dedicated thread, looping forever. Each iteration: disconnect stale nodes, call select() to wait for socket readiness, accept inbound connections, then read/write data for every active socket. The Sleep(10) at the end yields the CPU briefly each cycle.

CALLED FROM ThreadSocketHandler p. 331

```
void ThreadSocketHandler2(void* parg)
{
printf("ThreadSocketHandler started\n");
    SOCKET hListenSocket = *(SOCKET*)parg;
    list<CNode*> vNodesDisconnected;
int nPrevNodeCount = 0;

    loop
{
//
// Disconnect nodes
//
CRITICAL_BLOCK(cs_vNodes)
{
// Disconnect duplicate connections
```

Duplicate connection resolution: if two nodes connect to each other simultaneously, both would keep a duplicate. Only the lower-IP node runs the dedup; it keeps the better-connected `CNode` and marks the extra for disconnect — anti-symmetric, so both sides do not fight.[2]

```
            map<unsigned int, CNode*> mapFirst;
foreach(CNode* pnode, vNodes)
{
if (pnode->fDisconnect)
continue;
unsigned int ip = pnode->addr.ip;
if (mapFirst.count(ip) && addrLocalHost.ip < ip)
{
// In case two nodes connect to each other at once,
// the lower ip disconnects its outbound connection
                    CNode* pnodeExtra = mapFirst[ip];

if (pnodeExtra->GetRefCount() > (pnodeExtra->fNetworkNode ? 1 : 0))
swap(pnodeExtra, pnode);

if (pnodeExtra->GetRefCount() <= (pnodeExtra->fNetworkNode ? 1 : 0))
{
printf("(%d nodes) disconnecting duplicate: %s\n", vNodes.size(), pnodeExtra->addr.ToString().c_str());
if (pnodeExtra->fNetworkNode && !pnode->fNetworkNode)
{
                            pnode->AddRef();
swap(pnodeExtra->fNetworkNode, pnode->fNetworkNode);
                            pnodeExtra->Release();
}
                        pnodeExtra->fDisconnect = true;
}
}
                mapFirst[ip] = pnode;
}

// Disconnect unused nodes
            vector<CNode*> vNodesCopy = vNodes;
foreach(CNode* pnode, vNodesCopy)
{
```

`ReadyToDisconnect()` is true when the node is flagged `fDisconnect` or its reference count has dropped to zero; this line additionally requires both send/receive buffers to be empty before closing. Disconnected nodes are held in a pool for at least 5 minutes (longer if threads still hold references) before deletion.

```
if (pnode->ReadyToDisconnect() && pnode->vRecv.empty() && pnode->vSend.empty())
{
// remove from vNodes
                    vNodes.erase(remove(vNodes.begin(), vNodes.end(), pnode), vNodes.end());
                    pnode->Disconnect();

// hold in disconnected pool until all refs are released
                    pnode->nReleaseTime = max(pnode->nReleaseTime, GetTime() + 5 * 60);
if (pnode->fNetworkNode)
                        pnode->Release();
                    vNodesDisconnected.push_back(pnode);
}
}

// Delete disconnected nodes
            list<CNode*> vNodesDisconnectedCopy = vNodesDisconnected;
foreach(CNode* pnode, vNodesDisconnectedCopy)
{
// wait until threads are done using it
if (pnode->GetRefCount() <= 0)
{
```

```
bool fDelete = false;
TRY_CRITICAL_BLOCK(pnode->cs_vSend)
TRY_CRITICAL_BLOCK(pnode->cs_vRecv)
TRY_CRITICAL_BLOCK(pnode->cs_mapRequests)
TRY_CRITICAL_BLOCK(pnode->cs_inventory)
                        fDelete = true;
if (fDelete)
{
                        vNodesDisconnected.remove(pnode);
delete pnode;
}
}
}
}
if (vNodes.size() != nPrevNodeCount)
{
            nPrevNodeCount = vNodes.size();
MainFrameRepaint();
}


//
// Find which sockets have data to receive
```

`select()` multiplexing — classic Unix network I/O. Builds fd_sets for all active sockets plus the listening socket, then blocks for up to 50ms waiting for any socket to become readable or writable. Avoids spinning and keeps CPU usage low while watching many connections simultaneously.

```
//
struct timeval timeout;
        timeout.tv_sec  = 0;
        timeout.tv_usec = 50000; // frequency to poll pnode->vSend

struct fd_set fdsetRecv;
struct fd_set fdsetSend;
FD_ZERO(&fdsetRecv);
FD_ZERO(&fdsetSend);
        SOCKET hSocketMax = 0;
FD_SET(hListenSocket, &fdsetRecv);
        hSocketMax = max(hSocketMax, hListenSocket);
CRITICAL_BLOCK(cs_vNodes)
{
foreach(CNode* pnode, vNodes)
{
FD_SET(pnode->hSocket, &fdsetRecv);
                hSocketMax = max(hSocketMax, pnode->hSocket);
TRY_CRITICAL_BLOCK(pnode->cs_vSend)
if (!pnode->vSend.empty())
FD_SET(pnode->hSocket, &fdsetSend);
}
}

        vfThreadRunning[0] = false;
int nSelect = select(hSocketMax + 1, &fdsetRecv, &fdsetSend, NULL, &timeout);
        vfThreadRunning[0] = true;
CheckForShutdown(0);
if (nSelect == SOCKET_ERROR)
{
int nErr = WSAGetLastError();
printf("select failed: %d\n", nErr);
for (int i = 0; i <= hSocketMax; i++)
{
FD_SET(i, &fdsetRecv);
FD_SET(i, &fdsetSend);
}
Sleep(timeout.tv_usec/1000);
}
RandAddSeed();

```

```
//// debug print
//foreach(CNode* pnode, vNodes)
//{
//    printf("vRecv = %-5d ", pnode->vRecv.size());
//    printf("vSend = %-5d    ", pnode->vSend.size());
//}
//printf("\n");


//
// Accept new connections
//
```

Accept inbound connections: when the listening socket fires in the read set, accept() pulls the pending connection off the kernel queue, creates a CNode marked fInbound=true (inbound), and adds it to vNodes. Inbound nodes are treated identically to outbound ones.

```
if (FD_ISSET(hListenSocket, &fdsetRecv))
{
struct sockaddr_in sockaddr;
int len = sizeof(sockaddr);
            SOCKET hSocket = accept(hListenSocket, (struct sockaddr*)&sockaddr, &len);
            CAddress addr(sockaddr);
if (hSocket == INVALID_SOCKET)
{
if (WSAGetLastError() != WSAEWOULDBLOCK)
printf("ERROR ThreadSocketHandler accept failed: %d\n", WSAGetLastError());
}
else
{
printf("accepted connection from %s\n", addr.ToString().c_str());
                CNode* pnode = new CNode(hSocket, addr, true);
                pnode->AddRef();
CRITICAL_BLOCK(cs_vNodes)
                    vNodes.push_back(pnode);
}
}


//
// Service each socket
//
        vector<CNode*> vNodesCopy;
CRITICAL_BLOCK(cs_vNodes)
            vNodesCopy = vNodes;
foreach(CNode* pnode, vNodesCopy)
{
CheckForShutdown(0);
            SOCKET hSocket = pnode->hSocket;

//
// Receive
//
```

Receive path: reads up to 64 KB from the socket into pnode->vRecv in one syscall. A zero return means the remote end closed the connection gracefully; negative means a socket error. Data accumulates in vRecv until ThreadMessageHandler processes complete messages.

```
if (FD_ISSET(hSocket, &fdsetRecv))
{
TRY_CRITICAL_BLOCK(pnode->cs_vRecv)
{
                    CDataStream& vRecv = pnode->vRecv;
unsigned int nPos = vRecv.size();

// typical socket buffer is 8K-64K
```

```
const unsigned int nBufSize = 0x10000;
                    vRecv.resize(nPos + nBufSize);
int nBytes = recv(hSocket, &vRecv[nPos], nBufSize, 0);
                    vRecv.resize(nPos + max(nBytes, 0));
if (nBytes == 0)
{
// socket closed gracefully
if (!pnode->fDisconnect)
printf("recv: socket closed\n");
                        pnode->fDisconnect = true;
}
else if (nBytes < 0)
{
// socket error
int nErr = WSAGetLastError();
if (nErr != WSAEWOULDBLOCK && nErr != WSAEMSGSIZE && nErr != WSAEINTR && nErr != WSAEINPROGRESS)
{
if (!pnode->fDisconnect)
printf("recv failed: %d\n", nErr);
                            pnode->fDisconnect = true;
}
}
}
}

//
// Send
//
```

Send path: flushes as much of vSend as the kernel will accept in one send() call, then erases the bytes sent. Unsent bytes remain in vSend, so partial sends are handled correctly. Only marked writable (fdsetSend) if vSend is non-empty.

```
if (FD_ISSET(hSocket, &fdsetSend))
{
TRY_CRITICAL_BLOCK(pnode->cs_vSend)
{
                    CDataStream& vSend = pnode->vSend;
if (!vSend.empty())
{
int nBytes = send(hSocket, &vSend[0], vSend.size(), 0);
if (nBytes > 0)
{
                            vSend.erase(vSend.begin(), vSend.begin() + nBytes);
}
else if (nBytes == 0)
{
if (pnode->ReadyToDisconnect())
                                pnode->vSend.clear();
}
else
{
printf("send error %d\n", nBytes);
if (pnode->ReadyToDisconnect())
                                pnode->vSend.clear();
}
}
}
}
}


Sleep(10);
}
}
```

```
649
650
651
652
653
654
```

CALLED FROM StartNode p. 342

```
655 void ThreadOpenConnections(void* parg)
656 {
657 IMPLEMENT_RANDOMIZE_STACK(ThreadOpenConnections(parg));
658
659     loop
660 {
661         vfThreadRunning[1] = true;
662 CheckForShutdown(1);
663 try
664 {
665 ThreadOpenConnections2(parg);
666 }
667 CATCH_PRINT_EXCEPTION("ThreadOpenConnections()")
668         vfThreadRunning[1] = false;
669 Sleep(5000);
670 }
671 }
672
```

ThreadOpenConnections2 — proactively seeks new peers when below nMaxConnections (15). Selects from mapAddresses using the class-C diversity algorithm below. Once connected, sends our address and requests more addresses with "getaddr".

CALLED FROM ThreadOpenConnections p. 336

```
673 void ThreadOpenConnections2(void* parg)
674 {
675 printf("ThreadOpenConnections started\n");
676
677 // Initiate network connections
678 const int nMaxConnections = 15;
679     loop
680 {
681 // Wait
682         vfThreadRunning[1] = false;
683 Sleep(500);
684 while (vNodes.size() >= nMaxConnections || vNodes.size() >= mapAddresses.size())
685 {
686 CheckForShutdown(1);
687 Sleep(2000);
688 }
689         vfThreadRunning[1] = true;
690 CheckForShutdown(1);
691
692
```

Class-C diversity algorithm: groups known IPs by /24 subnet. When choosing a peer, first picks a random /24, then a random IP within it. This means an attacker flooding with 255 IPs in one subnet gets the same share of attention as a single legitimate node in another subnet — Sybil resistance through IP diversity.

```
693 // Make a list of unique class C's
694 unsigned char pchIPCMask[4] = { 0xff, 0xff, 0xff, 0x00 };
695 unsigned int nIPCMask = *(unsigned int*)pchIPCMask;
696         vector<unsigned int> vIPC;
697 CRITICAL_BLOCK(cs_mapAddresses)
698 {
699             vIPC.reserve(mapAddresses.size());
700 unsigned int nPrev = 0;
```

```
foreach(const PAIRTYPE(vector<unsigned char>, CAddress)& item, mapAddresses)
{
const CAddress& addr = item.second;
if (!addr.IsIPv4())
continue;

// Taking advantage of mapAddresses being in sorted order,
// with IPs of the same class C grouped together.
unsigned int ipC = addr.ip & nIPCMask;
if (ipC != nPrev)
                    vIPC.push_back(nPrev = ipC);
}
}

//
// The IP selection process is designed to limit vulnerability to address flooding.
// Any class C (a.b.c.?) has an equal chance of being chosen, then an IP is
// chosen within the class C.  An attacker may be able to allocate many IPs, but
// they would normally be concentrated in blocks of class C's.  They can hog the
// attention within their class C, but not the whole IP address space overall.
// A lone node in a class C will get as much attention as someone holding all 255
// IPs in another class C.
//
bool fSuccess = false;
int nLimit = vIPC.size();
while (!fSuccess && nLimit-- > 0)
{
// Choose a random class C
unsigned int ipC = vIPC[GetRand(vIPC.size())];

// Organize all addresses in the class C by IP
            map<unsigned int, vector<CAddress> > mapIP;
CRITICAL_BLOCK(cs_mapAddresses)
{
```

Retry delay for peers that failed before — scaled by how many peers you already have, not by how many times the address failed. nDelay starts at 30 minutes with no connections and doubles with each connection in vNodes — with 8 peers it would be 30 min × 2^8 = 128 hours, capped at 8 hours. So a lonely node retries aggressively; a well-connected one is picky. nRandomizer (derived from the address and its failure time) scales each address's wait between 0 and 2× nDelay so retries are staggered. An address that has never failed (nLastFailed = 0) gets a randomizer of 0 and is always eligible.

```
unsigned int nDelay = ((30 * 60) << vNodes.size());
if (nDelay > 8 * 60 * 60)
                    nDelay = 8 * 60 * 60;
for (map<vector<unsigned char>, CAddress>::iterator mi = mapAddresses.lower_bound(CAddress(ipC,
0).GetKey());
                     mi != mapAddresses.upper_bound(CAddress(ipC | ~nIPCMask, 0xffff).GetKey());
++mi)
{
const CAddress& addr = (*mi).second;
unsigned int nRandomizer = (addr.nLastFailed * addr.ip * 7777U) % 20000;
if (GetTime() - addr.nLastFailed > nDelay * nRandomizer / 10000)
                        mapIP[addr.ip].push_back(addr);
}
}
if (mapIP.empty())
break;

// Choose a random IP in the class C
            map<unsigned int, vector<CAddress> >::iterator mi = mapIP.begin();
advance(mi, GetRand(mapIP.size()));

// Once we've chosen an IP, we'll try every given port before moving on
foreach(const CAddress& addrConnect, (*mi).second)
{
```

```
758 if (addrConnect.ip == addrLocalHost.ip || !addrConnect.IsIPv4() || FindNode(addrConnect.ip))
759 continue;
760
761                 CNode* pnode = ConnectNode(addrConnect);
762 if (!pnode)
763 continue;
764                 pnode->fNetworkNode = true;
765
766 if (addrLocalHost.IsRoutable())
767 {
768 // Advertise our address
769                     vector<CAddress> vAddrToSend;
770                     vAddrToSend.push_back(addrLocalHost);
771                     pnode->PushMessage("addr", vAddrToSend);
772 }
773
774 // Get as many addresses as we can
775                 pnode->PushMessage("getaddr");
776
777 ////// should the one on the receiving end do this too?
778 // Subscribe our local subscription list
779 const unsigned int nHops = 0;
780 for (unsigned int nChannel = 0; nChannel < pnodeLocalHost->vfSubscribe.size(); nChannel++)
781 if (pnodeLocalHost->vfSubscribe[nChannel])
782                         pnode->PushMessage("subscribe", nChannel, nHops);
783
784                 fSuccess = true;
785 break;
786 }
787 }
788 }
789 }
790
791
792
793
794
795
796
797
```

CALLED FROM StartNode p. 342

```
798 void ThreadMessageHandler(void* parg)
799 {
800 IMPLEMENT_RANDOMIZE_STACK(ThreadMessageHandler(parg));
801
802     loop
803 {
804         vfThreadRunning[2] = true;
805 CheckForShutdown(2);
806 try
807 {
808 ThreadMessageHandler2(parg);
809 }
810 CATCH_PRINT_EXCEPTION("ThreadMessageHandler()")
811         vfThreadRunning[2] = false;
812 Sleep(5000);
813 }
814 }
815
```

ThreadMessageHandler2 — protocol dispatch loop. Runs at THREAD_PRIORITY_BELOW_NORMAL to avoid starving the I/O thread. Every 100ms, iterates all peers: calls ProcessMessages to handle received data, then SendMessages to queue outbound protocol messages.

CALLED FROM ThreadMessageHandler p. 338

```
816 void ThreadMessageHandler2(void* parg)
817 {
818 printf("ThreadMessageHandler started\n");
```

`THREAD_PRIORITY_BELOW_NORMAL` — the message handler runs at reduced priority deliberately. Network I/O (socket handler) must be responsive; protocol processing can lag slightly. This deliberate priority split ensures data is received before it is processed.

```
819 SetThreadPriority(GetCurrentThread(), THREAD_PRIORITY_BELOW_NORMAL);
820     loop
821 {
822 // Poll the connected nodes for messages
823         vector<CNode*> vNodesCopy;
824 CRITICAL_BLOCK(cs_vNodes)
825             vNodesCopy = vNodes;
826 foreach(CNode* pnode, vNodesCopy)
827 {
828             pnode->AddRef();
829
830 // Receive messages
831 TRY_CRITICAL_BLOCK(pnode->cs_vRecv)
832 ProcessMessages(pnode);
833
834 // Send messages
835 TRY_CRITICAL_BLOCK(pnode->cs_vSend)
836 SendMessages(pnode);
837
838             pnode->Release();
839 }
840
841 // Wait and allow messages to bunch up
842         vfThreadRunning[2] = false;
843 Sleep(100);
844         vfThreadRunning[2] = true;
845 CheckForShutdown(2);
846 }
847 }
848
849
850
851
852
853
854
855
856
857 //// todo: start one thread per processor, use getenv("NUMBER_OF_PROCESSORS")
```

`BitcoinMiner` thread: proof-of-work mining runs in its own thread (index 3). The comment "todo: start one thread per processor" shows Satoshi was aware of multi-core mining from the start. This single-threaded miner was quickly replaced by multi-threaded and then GPU miners.

```
    CALLED FROM  CMainFrame::OnMenuOptionsGenerate ui.cpp p. 489 · CMyApp::OnInit2 ui.cpp p. 517
858 void ThreadBitcoinMiner(void* parg)
859 {
860     vfThreadRunning[3] = true;
861 CheckForShutdown(3);
862 try
863 {
864 bool fRet = BitcoinMiner();
865 printf("BitcoinMiner returned %s\n\n\n\n", fRet ? "true" : "false");
866 }
867 CATCH_PRINT_EXCEPTION("BitcoinMiner()")
868     vfThreadRunning[3] = false;
869 }
870
871
872
873
```

```
874
875
876
877
878
879
880
```

`StartNode` — the network bootstrap sequence. Initializes Winsock, resolves local hostname, creates and binds the TCP listen socket, sets it to non-blocking, discovers external IP, then spawns four threads: IRC seed, socket handler, connection opener, and message handler.

```
    CALLED FROM  CMyApp::OnInit2 ui.cpp p. 517
881 bool StartNode(string& strError)
882 {
883     strError = "";
884
885 // Sockets startup
886     WSADATA wsadata;
```

`WSAStartup` — Windows Sockets API initialization. MAKEWORD(2,2) requests Winsock version 2.2. This must be called before any socket operations on Windows. The equivalent on Linux/macOS would be nothing — POSIX sockets need no initialization.

```
887 int ret = WSAStartup(MAKEWORD(2,2), &wsadata);
888 if (ret != NO_ERROR)
889 {
890         strError = strprintf("Error: TCP/IP socket library failed to start (WSAStartup returned error
    %d)", ret);
891 printf("%s\n", strError.c_str());
892 return false;
893 }
894
895 // Get local host ip
896 char pszHostName[255];
897 if (gethostname(pszHostName, 255) == SOCKET_ERROR)
898 {
899         strError = strprintf("Error: Unable to get IP address of this computer (gethostname returned
    error %d)", WSAGetLastError());
900 printf("%s\n", strError.c_str());
901 return false;
902 }
903 struct hostent* pHostEnt = gethostbyname(pszHostName);
904 if (!pHostEnt)
905 {
906         strError = strprintf("Error: Unable to get IP address of this computer (gethostbyname returned
    error %d)", WSAGetLastError());
907 printf("%s\n", strError.c_str());
908 return false;
909 }
910     addrLocalHost = CAddress(*(long*)(pHostEnt->h_addr_list[0]),
911                              DEFAULT_PORT,
912                              nLocalServices);
913 printf("addrLocalHost = %s\n", addrLocalHost.ToString().c_str());
914
915 // Create socket for listening for incoming connections
916     SOCKET hListenSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
917 if (hListenSocket == INVALID_SOCKET)
918 {
919         strError = strprintf("Error: Couldn't open socket for incoming connections (socket returned
    error %d)", WSAGetLastError());
920 printf("%s\n", strError.c_str());
921 return false;
922 }
923
924 // Set to nonblocking, incoming connections will also inherit this
```

```
925     u_long nOne = 1;
```

Non-blocking sockets: FIONBIO with value 1 sets the listen socket to non-blocking mode via ioctlsocket(), and accepted inbound sockets inherit it — their recv/send calls return immediately with WSAEWOULDBLOCK rather than blocking. Outbound sockets created in ConnectSocket() are never switched over and stay blocking; the select() loop, which only touches sockets reported ready, is what keeps them from stalling.

```
926 if (ioctlsocket(hListenSocket, FIONBIO, &nOne) == SOCKET_ERROR)
927 {
928         strError = strprintf("Error: Couldn't set properties on socket for incoming connections
    (ioctlsocket returned error %d)", WSAGetLastError());
929 printf("%s\n", strError.c_str());
930 return false;
931 }
932
933 // The sockaddr_in structure specifies the address family,
934 // IP address, and port for the socket that is being bound
935 int nRetryLimit = 15;
936 struct sockaddr_in sockaddr = addrLocalHost.GetSockAddr();
937 if (bind(hListenSocket, (struct sockaddr*)&sockaddr, sizeof(sockaddr)) == SOCKET_ERROR)
938 {
939 int nErr = WSAGetLastError();
940 if (nErr == WSAEADDRINUSE)
941             strError = strprintf("Error: Unable to bind to port %s on this computer. The program is
    probably already running.", addrLocalHost.ToString().c_str());
942 else
943             strError = strprintf("Error: Unable to bind to port %s on this computer (bind returned error
    %d)", addrLocalHost.ToString().c_str(), nErr);
944 printf("%s\n", strError.c_str());
945 return false;
946 }
947 printf("bound to addrLocalHost = %s\n\n", addrLocalHost.ToString().c_str());
948
949 // Listen for incoming connections
950 if (listen(hListenSocket, SOMAXCONN) == SOCKET_ERROR)
951 {
952         strError = strprintf("Error: Listening for incoming connections failed (listen returned error
    %d)", WSAGetLastError());
953 printf("%s\n", strError.c_str());
954 return false;
955 }
956
957 // Get our external IP address for incoming connections
958 if (addrIncoming.ip)
959         addrLocalHost.ip = addrIncoming.ip;
960
```

`addrIncoming` — if a previous session discovered our external IP, it was saved to wallet.dat. We restore it here as a fallback in case whatismyip.com is unreachable; GetMyExternalIP still runs and updates it if successful.

```
961 if (GetMyExternalIP(addrLocalHost.ip))
962 {
963         addrIncoming = addrLocalHost;
964 CWalletDB().WriteSetting("addrIncoming", addrIncoming);
965 }
966
967 // Get addresses from IRC and advertise ours
```

`ThreadIRCSeed` — connects to IRC to bootstrap peer discovery. Early nodes joined `#bitcoin` (v0.1.0 uses `chat.freenode.net`) and encoded their IP in the nickname. Satoshi's elegant hack: reuse a global chat directory before the network could self-sustain. Hal Finney's attached `debug.log` from January 10, 2009 is a snapshot of that path working — and of how tiny the network still was.[45]

```
968 if (_beginthread(ThreadIRCSeed, 0, NULL) == -1)
969 printf("Error: _beginthread(ThreadIRCSeed) failed\n");
970
971 //
972 // Start threads
973 //
```

Thread startup sequence: four threads created with _beginthread() (Windows CRT). All but ThreadIRCSeed run inside wrappers (like ThreadSocketHandler) that appear to catch exceptions and restart the inner function after a 5-second sleep. In practice the restart never happens: CATCH_PRINT_EXCEPTION calls PrintException(), which logs the error, shows a message box, and then re-throws (`throw;` in util.cpp), so the exception escapes the loop before the Sleep(5000). The three threads started here use indices 0–2 of vfThreadRunning; the miner, started from ui.cpp, uses index 3.

```
974 if (_beginthread(ThreadSocketHandler, 0, new SOCKET(hListenSocket)) == -1)
975 {
976         strError = "Error: _beginthread(ThreadSocketHandler) failed";
977 printf("%s\n", strError.c_str());
978 return false;
979 }
980
981 if (_beginthread(ThreadOpenConnections, 0, NULL) == -1)
982 {
983         strError = "Error: _beginthread(ThreadOpenConnections) failed";
984 printf("%s\n", strError.c_str());
985 return false;
986 }
987
988 if (_beginthread(ThreadMessageHandler, 0, NULL) == -1)
989 {
990         strError = "Error: _beginthread(ThreadMessageHandler) failed";
991 printf("%s\n", strError.c_str());
992 return false;
993 }
994
995 return true;
996 }
997
```

`StopNode` — orderly shutdown. Sets fShutdown=true (all threads check CheckForShutdown() periodically), increments nTransactionsUpdated to wake waiting threads, then polls until all threads have exited. WSACleanup releases Winsock resources.

```
    CALLED FROM  Shutdown ui.cpp p. 481
998 bool StopNode()
999 {
1000 printf("StopNode()\n");
1001     fShutdown = true;
1002     nTransactionsUpdated++;
1003 while (count(vfThreadRunning.begin(), vfThreadRunning.end(), true))
1004 Sleep(10);
1005 Sleep(50);
1006
1007 // Sockets shutdown
```

```
1008 WSACleanup();
1009 return true;
1010 }
1011
```

`CheckForShutdown` — the cooperative shutdown mechanism. Every thread's inner loop calls this periodically. If fShutdown is set, the thread marks itself as not running and calls _endthread() to exit. No forced killing — threads exit at their next safe yield point.

CALLED FROM ProcessMessages main.cpp p. 230 · SendMessages main.cpp p. 238 · BitcoinMiner main.cpp p. 240 · ThreadSocketHandler p. 331 · ThreadSocketHandler2 p. 333 · ThreadOpenConnections p. 336 · *and 4 more*

```
1012 void CheckForShutdown(int n)
1013 {
1014 if (fShutdown)
1015 {
1016 if (n != -1)
1017             vfThreadRunning[n] = false;
1018 _endthread();
1019 }
1020 }
1021
```

SUMMARY

The networking layer embodies Satoshi's trustless philosophy. Nodes do not trust their peers—they validate everything independently. A malicious peer can waste your bandwidth but cannot steal your coins or corrupt your view of the blockchain. The protocol assumes the worst about every connection and is correct to do so.

Message propagation uses "inv" (inventory) announcements: nodes advertise what they have, peers request what they need. This pull-based model prevents spam, lets nodes prioritize important data (blocks over transactions), and ensures that no node can force another to download unwanted data.

The design's elegance is in what it omits: no authentication, no identity, no trust hierarchy, no permissions. Every node is equal. The consensus rules encoded in `main.cpp` ensure that even a network of anonymous strangers—some of whom are actively trying to cheat—converges on the same truth. This is the property that makes Bitcoin censorship-resistant: there is no privileged node to coerce.

KEY TERMS

vNodes

The global vector of live peer connections, guarded by cs_vNodes and shared by the socket-handling, connection-opening, and message-handling threads.

mapAddresses

The in-memory table of known peer addresses, keyed by serialized IP and port, loaded from addr.dat at startup and consulted whenever the node chooses whom to dial next.

class C

A block of IPv4 addresses sharing their first three bytes (a /24); ThreadOpenConnections picks a random class C before picking an IP inside it, blunting address flooding from a single block.

SOCKS4 proxy

An optional relay configured in addrProxy; when set, ConnectSocket dials the proxy and sends a SOCKS4 CONNECT request instead of reaching routable peers directly.

ThreadSocketHandler

The network I/O thread that uses select() to accept inbound connections, read incoming bytes into each node's vRecv buffer, flush its vSend buffer, and delete disconnected nodes.

ThreadOpenConnections

The thread that, while fewer than 15 peers are connected, dials a known address, advertises the node's own address if routable, and sends getaddr to learn more.

ThreadMessageHandler

The below-normal-priority thread that, roughly every 100 milliseconds, runs ProcessMessages and SendMessages for each connected node.

reference count

The per-node count raised by AddRef and lowered by Release, plus an optional timed hold in nReleaseTime, that keeps a disconnected node in memory until no thread still uses it.

SOURCES

1. Satoshi Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (October 2008). https://bitcoin.org/bitcoin.pdf
2. Jeremy Rubin, *satoshis-version — Annotated Bitcoin v0.01 codebase* (2019). Line-by-line //# comments on Satoshi's original release; best read commit-by-commit. https://github.com/JeremyRubin/satoshis-version/

3. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
4. Hal Finney, *[bitcoin-list] Crash in bitcoin 0.1.0 (with debug.log)* (January 10, 2009). SourceForge post attaching Finney's debug.log; forensic evidence he joined near block 49. https://web.archive.org/web/20141130200234/https://sourceforge.net/p/bitcoin/mailman/message/21295694/
5. Alex Waltz, *Who was the 1st Bitcoin Miner?* (2026). Forensic reconstruction of Bitcoin's first days from Finney's debug.log, emails, and ExtraNonce patterns. https://1stbitcoinminer.com/

net.h

857 lines CPP

PEERS WITHOUT TRUST

This header defines the network message types and node connection structures that form Bitcoin's peer-to-peer protocol. Every message that Bitcoin nodes exchange—blocks, transactions, peer addresses, version handshakes—is defined here. These definitions are the "language" that Bitcoin nodes speak.

The key abstraction is the `CNode` class: a single connection to another Bitcoin node. Each CNode maintains send/receive buffers, tracks what data the peer has (to avoid redundant transfers), and handles the message serialization. In v0.1.0 the connection opener stops seeking peers once 15 are connected; modern Bitcoin Core allows up to 125.

Satoshi designed the protocol with minimal trust: nodes don't believe what peers tell them—they verify independently. A peer claiming to have a valid block must prove it. This skepticism is why Bitcoin survives in adversarial environments where some participants actively try to cheat.

The message types defined here form a request-response pattern: nodes announce what they have ("inv"), peers request what they want ("getdata"), and data flows in response ("tx", "block"). This pull-based model prevents spam—you only download data you explicitly request.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

class CMessageHeader;
class CAddress;
class CInv;
class CRequestTracker;
class CNode;
```

`DEFAULT_PORT = 8333`—Bitcoin's well-known port number. Satoshi chose 8333 with no documented rationale—it wasn't assigned by IANA for any other purpose. The `htons()` call converts to network byte order (big-endian). Every Bitcoin node listens on this port; firewalls must allow it for full participation. Some speculate 8333 was chosen because 8+3+3+3 = 17, but this is probably coincidental.

```
static const unsigned short DEFAULT_PORT = htons(8333);
static const unsigned int PUBLISH_HOPS = 5;
enum
{
```

`NODE_NETWORK`—service flag indicating a full node. In v0.1.0, this was the only flag. Later versions added NODE_BLOOM (supports bloom filters), NODE_WITNESS (supports SegWit), etc. When connecting, nodes exchange service flags to understand each other's capabilities. A node advertising NODE_NETWORK commits to serving historical blocks on request.

```
    NODE_NETWORK = (1 << 0),
};



bool ConnectSocket(const CAddress& addrConnect, SOCKET& hSocketRet);
bool GetMyExternalIP(unsigned int& ipRet);
bool AddAddress(CAddrDB& addrdb, const CAddress& addr);
CNode* FindNode(unsigned int ip);
CNode* ConnectNode(CAddress addrConnect, int64 nTimeout=0);
void AbandonRequests(void (*fn)(void*, CDataStream&), void* param1);
bool AnySubscribed(unsigned int nChannel);
void ThreadBitcoinMiner(void* parg);
bool StartNode(string& strError=REF(string()));
bool StopNode();
void CheckForShutdown(int n);



//
// Message header
//  (4) message start
//  (12) command
//  (4) size

// The message start string is designed to be unlikely to occur in normal data.
// The characters are rarely used upper ascii, not valid as UTF-8, and produce
```

```
// a large 4-byte int at any alignment.
```

`pchMessageStart`—the 4-byte 'magic number' that begins every Bitcoin message: `0xf9beb4d9`. Satoshi's comment explains: 'designed to be unlikely to occur in normal data.' These bytes are invalid UTF-8, rare in binary files, and form a large 32-bit integer at any alignment. When scanning a data stream, finding this sequence likely indicates a Bitcoin message boundary. Testnet uses different magic bytes to prevent cross-network contamination.

```
static const char pchMessageStart[4] = { 0xf9, 0xbe, 0xb4, 0xd9 };

```

`CMessageHeader`—the 20-byte header prepended to every network message. Structure: 4-byte magic, 12-byte command (null-padded), 4-byte payload size. Later versions added a 4-byte checksum after the size. The fixed-size command field allows efficient parsing—you can read exactly 20 bytes, validate the header, then read exactly nMessageSize more bytes.

```
class CMessageHeader
{
public:
```

`COMMAND_SIZE=12`—maximum command name length. Commands like 'version', 'inv', 'getdata', 'block', 'tx', 'ping' easily fit. The padding with zeros allows fixed-size parsing. Longer command names would require protocol changes—but 12 characters has proven sufficient for all Bitcoin message types.

```
enum { COMMAND_SIZE=12 };
char pchMessageStart[sizeof(::pchMessageStart)];
char pchCommand[COMMAND_SIZE];
unsigned int nMessageSize;

CMessageHeader()
{
memcpy(pchMessageStart, ::pchMessageStart, sizeof(pchMessageStart));
memset(pchCommand, 0, sizeof(pchCommand));
        pchCommand[1] = 1;
        nMessageSize = -1;
}

CMessageHeader(const char* pszCommand, unsigned int nMessageSizeIn)
{
memcpy(pchMessageStart, ::pchMessageStart, sizeof(pchMessageStart));
strncpy(pchCommand, pszCommand, COMMAND_SIZE);
        nMessageSize = nMessageSizeIn;
}

    IMPLEMENT_SERIALIZE
(
READWRITE(FLATDATA(pchMessageStart));
READWRITE(FLATDATA(pchCommand));
READWRITE(nMessageSize);
)

    string GetCommand()
{
if (pchCommand[COMMAND_SIZE-1] == 0)
return string(pchCommand, pchCommand + strlen(pchCommand));
else
return string(pchCommand, pchCommand + COMMAND_SIZE);
}

```

`IsValid()`—validates message header before processing payload. Checks: (1) magic bytes match (rejects non-Bitcoin data), (2) command contains only printable ASCII then zeros (rejects malformed commands), (3) message size is reasonable (rejects memory exhaustion attacks). The `0x10000000` (256MB) limit was generous for 2009—modern Bitcoin has stricter limits.

```
bool IsValid()
{
// Check start string
if (memcmp(pchMessageStart, ::pchMessageStart, sizeof(pchMessageStart)) != 0)
return false;

// Check the command string for errors
for (char* p1 = pchCommand; p1 < pchCommand + COMMAND_SIZE; p1++)
{
if (*p1 == 0)
{
// Must be all zeros after the first zero
for (; p1 < pchCommand + COMMAND_SIZE; p1++)
if (*p1 != 0)
return false;
}
else if (*p1 < ' ' || *p1 > 0x7E)
return false;
}

// Message size
if (nMessageSize > 0x10000000)
{
printf("CMessageHeader::IsValid() : nMessageSize too large %u\n", nMessageSize);
return false;
}

return true;
}
};
```

`pchIPv4`—IPv4-mapped IPv6 prefix. Bitcoin's address format reserves 16 bytes for IP addresses to support IPv6. IPv4 addresses are stored with this 12-byte prefix followed by the 4-byte IPv4 address. This forward-thinking design meant Bitcoin didn't need protocol changes when IPv6 adoption grew.

```
static const unsigned char pchIPv4[12] = { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0xff, 0xff };

```

`CAddress`—represents a network peer's address. Contains: services (capabilities bitmap), IP address (16 bytes for IPv6 compatibility), port, timestamp (when last seen), and nLastFailed (connection attempt tracking). This structure is shared between nodes during 'addr' messages to propagate peer discovery information. The timestamp helps nodes prefer recently-active peers.

```
class CAddress
{
public:
    uint64 nServices;
unsigned char pchReserved[12];
unsigned int ip;
```

```
138 unsigned short port;
139
140 // disk only
```

`nTime`—timestamp when this address was last known to be active. Only serialized to disk (SER_DISK flag), not sent over the network in v0.1. Later versions include timestamps in 'addr' messages to help nodes prioritize fresh addresses. Addresses with old timestamps are less likely to be reachable.

```
141 unsigned int nTime;
142
143 // memory only
144 unsigned int nLastFailed;
145
146 CAddress()
147 {
148         nServices = 0;
149 memcpy(pchReserved, pchIPv4, sizeof(pchReserved));
150         ip = 0;
151         port = DEFAULT_PORT;
152         nTime = GetAdjustedTime();
153         nLastFailed = 0;
154 }
155
156 CAddress(unsigned int ipIn, unsigned short portIn, uint64 nServicesIn=0)
157 {
158         nServices = nServicesIn;
159 memcpy(pchReserved, pchIPv4, sizeof(pchReserved));
160         ip = ipIn;
161         port = portIn;
162         nTime = GetAdjustedTime();
163         nLastFailed = 0;
164 }
165
166 explicit CAddress(const struct sockaddr_in& sockaddr, uint64 nServicesIn=0)
167 {
168         nServices = nServicesIn;
169 memcpy(pchReserved, pchIPv4, sizeof(pchReserved));
170         ip = sockaddr.sin_addr.s_addr;
171         port = sockaddr.sin_port;
172         nTime = GetAdjustedTime();
173         nLastFailed = 0;
174 }
175
176 explicit CAddress(const char* pszIn, uint64 nServicesIn=0)
177 {
178         nServices = nServicesIn;
179 memcpy(pchReserved, pchIPv4, sizeof(pchReserved));
180         ip = 0;
181         port = DEFAULT_PORT;
182         nTime = GetAdjustedTime();
183         nLastFailed = 0;
184
185 char psz[100];
186 if (strlen(pszIn) > ARRAYLEN(psz)-1)
187 return;
188 strcpy(psz, pszIn);
189 unsigned int a, b, c, d, e;
190 if (sscanf(psz, "%u.%u.%u.%u:%u", &a, &b, &c, &d, &e) < 4)
191 return;
192 char* pszPort = strchr(psz, ':');
193 if (pszPort)
194 {
195 *pszPort++ = '\0';
196             port = htons(atoi(pszPort));
197 }
198         ip = inet_addr(psz);
199 }
```

200

IMPLEMENT_SERIALIZE—the serialization specification. This macro generates code for reading/writing CAddress to disk or network. The SER_DISK check means timestamps are saved locally but not transmitted—a privacy consideration (timestamps could reveal when you learned about addresses).

```
201     IMPLEMENT_SERIALIZE
202 (
203 if (nType & SER_DISK)
204 {
205 READWRITE(nVersion);
206 READWRITE(nTime);
207 }
208 READWRITE(nServices);
209 READWRITE(FLATDATA(pchReserved));
210 READWRITE(ip);
211 READWRITE(port);
212 )
213
214 friend inline bool operator==(const CAddress& a, const CAddress& b)
215 {
216 return (memcmp(a.pchReserved, b.pchReserved, sizeof(a.pchReserved)) == 0 &&
217                 a.ip   == b.ip &&
218                 a.port == b.port);
219 }
220
221 friend inline bool operator<(const CAddress& a, const CAddress& b)
222 {
223 int ret = memcmp(a.pchReserved, b.pchReserved, sizeof(a.pchReserved));
224 if (ret < 0)
225 return true;
226 else if (ret == 0)
227 {
228 if (ntohl(a.ip) < ntohl(b.ip))
229 return true;
230 else if (a.ip == b.ip)
231 return ntohs(a.port) < ntohs(b.port);
232 }
233 return false;
234 }
235
236     vector<unsigned char> GetKey() const
237 {
238         CDataStream ss;
239         ss.reserve(18);
240         ss << FLATDATA(pchReserved) << ip << port;
241
242 #if defined(_MSC_VER) && _MSC_VER < 1300
243 return vector<unsigned char>((unsigned char*)&ss.begin()[0], (unsigned char*)&ss.end()[0]);
244 #else
245 return vector<unsigned char>(ss.begin(), ss.end());
246 #endif
247 }
248
249 struct sockaddr_in GetSockAddr() const
250 {
251 struct sockaddr_in sockaddr;
252         sockaddr.sin_family = AF_INET;
253         sockaddr.sin_addr.s_addr = ip;
254         sockaddr.sin_port = port;
255 return sockaddr;
256 }
257
258 bool IsIPv4() const
259 {
260 return (memcmp(pchReserved, pchIPv4, sizeof(pchIPv4)) == 0);
261 }
```

```
262
```

`IsRoutable()`—checks if an IP address is publicly routable. Returns false for private networks (`10.x.x.x`, `192.168.x.x`). Bitcoin nodes shouldn't relay private addresses since they're unreachable from the public internet. This prevents polluting the address database with useless entries.

```
263 bool IsRoutable() const
264 {
265 return !(GetByte(3) == 10 || (GetByte(3) == 192 && GetByte(2) == 168));
266 }
267
268 unsigned char GetByte(int n) const
269 {
270 return ((unsigned char*)&ip)[3-n];
271 }
272
273     string ToStringIPPort() const
274 {
275 return strprintf("%u.%u.%u.%u:%u", GetByte(3), GetByte(2), GetByte(1), GetByte(0), ntohs(port));
276 }
277
278     string ToStringIP() const
279 {
280 return strprintf("%u.%u.%u.%u", GetByte(3), GetByte(2), GetByte(1), GetByte(0));
281 }
282
283     string ToString() const
284 {
285 return strprintf("%u.%u.%u.%u:%u", GetByte(3), GetByte(2), GetByte(1), GetByte(0), ntohs(port));
286 //return strprintf("%u.%u.%u.%u", GetByte(3), GetByte(2), GetByte(1), GetByte(0));
287 }
288
289 void print() const
290 {
291 printf("CAddress(%s)\n", ToString().c_str());
292 }
293 };
294
295
296
297
298
299
300
```

Message type constants: `MSG_TX`, `MSG_BLOCK`, etc. These identify what kind of inventory item is being advertised or requested. The `inv` message contains a list of (type, hash) pairs; `getdata` requests specific items by (type, hash). `MSG_REVIEW` and `MSG_PRODUCT` hint at Satoshi's marketplace vision that was later removed.

```
301 enum
302 {
303     MSG_TX = 1,
304     MSG_BLOCK,
305     MSG_REVIEW,
306     MSG_PRODUCT,
307     MSG_TABLE,
308 };
309
310 static const char* ppszTypeName[] =
311 {
312 "ERROR",
313 "tx",
314 "block",
315 "review",
316 "product",
```

```
"table",
};
```

`CInv`—an 'inventory item' identifying data by type and hash. When a node receives a new transaction, it announces via 'inv' messages containing CInv entries. Peers check if they already have each item; if not, they request it via 'getdata'. This pattern minimizes redundant data transfer—you only download what you don't have.

```
class CInv
{
public:
int type;
    uint256 hash;

CInv()
{
        type = 0;
        hash = 0;
}

CInv(int typeIn, const uint256& hashIn)
{
        type = typeIn;
        hash = hashIn;
}

CInv(const string& strType, const uint256& hashIn)
{
int i;
for (i = 1; i < ARRAYLEN(ppszTypeName); i++)
{
if (strType == ppszTypeName[i])
{
                type = i;
break;
}
}
if (i == ARRAYLEN(ppszTypeName))
throw std::out_of_range(strprintf("CInv::CInv(string, uint256) : unknown type '%s'", strType.c_str()));
        hash = hashIn;
}

    IMPLEMENT_SERIALIZE
(
READWRITE(type);
READWRITE(hash);
)

friend inline bool operator<(const CInv& a, const CInv& b)
{
return (a.type < b.type || (a.type == b.type && a.hash < b.hash));
}

bool IsKnownType() const
{
return (type >= 1 && type < ARRAYLEN(ppszTypeName));
}

const char* GetCommand() const
{
if (!IsKnownType())
throw std::out_of_range(strprintf("CInv::GetCommand() : type=% unknown type", type));
return ppszTypeName[type];
}

    string ToString() const
{
```

```
return strprintf("%s %s", GetCommand(), hash.ToString().substr(0,14).c_str());
}

void print() const
{
printf("CInv(%s)\n", ToString().c_str());
}
};
```

`CRequestTracker`—callback mechanism for async requests. Some operations (like requesting market data) need to process the response when it arrives. `CRequestTracker` stores a function pointer and parameter, invoked when the response comes. This predates C++11 lambdas—today you'd use `std::function`.

```
class CRequestTracker
{
public:
void (*fn)(void*, CDataStream&);
void* param1;

explicit CRequestTracker(void (*fnIn)(void*, CDataStream&)=NULL, void* param1In=NULL)
{
        fn = fnIn;
        param1 = param1In;
}

bool IsNull()
{
return fn == NULL;
}
};
```

Global network state variables. `vNodes` is the list of all connected peers. `mapAddresses` stores known peer addresses for reconnection. `mapRelay` caches recently-relayed data to serve to multiple peers without re-fetching. `cs_*` variables are mutexes (`CCriticalSection`) protecting these shared structures from race conditions.

```
extern bool fClient;
extern uint64 nLocalServices;
extern CAddress addrLocalHost;
extern CNode* pnodeLocalHost;
extern bool fShutdown;
extern array<bool, 10> vfThreadRunning;
extern vector<CNode*> vNodes;
extern CCriticalSection cs_vNodes;
extern map<vector<unsigned char>, CAddress> mapAddresses;
extern CCriticalSection cs_mapAddresses;
extern map<CInv, CDataStream> mapRelay;
extern deque<pair<int64, CInv> > vRelayExpiration;
extern CCriticalSection cs_mapRelay;
extern map<CInv, int64> mapAlreadyAskedFor;
extern CAddress addrProxy;
```

```
432
433
```

`CNode`—the heart of Bitcoin's networking: a single peer connection. Each CNode wraps a socket and maintains: send/receive buffers, version information, connection state, inventory tracking (what the peer has/wants), and address gossip state. A v0.1.0 node actively dials out until it has 15 CNode instances (modern Core allows up to 125), each representing one peer relationship.

```
434 class CNode
435 {
436 public:
437 // socket
438     uint64 nServices;
439     SOCKET hSocket;
```

Network buffers: `vSend` and `vRecv` are `CDataStream` objects holding outgoing and incoming data. The `CDataStream` class (defined in serialize.h) handles Bitcoin's custom serialization format. Critical sections `cs_vSend` and `cs_vRecv` protect these buffers—multiple threads may try to send messages or process received data simultaneously.

```
440     CDataStream vSend;
441     CDataStream vRecv;
442     CCriticalSection cs_vSend;
443     CCriticalSection cs_vRecv;
444 unsigned int nPushPos;
445     CAddress addr;
446 int nVersion;
447 bool fClient;
448 bool fInbound;
449 bool fNetworkNode;
450 bool fDisconnect;
451 protected:
452 int nRefCount;
453 public:
454     int64 nReleaseTime;
455     map<uint256, CRequestTracker> mapRequests;
456     CCriticalSection cs_mapRequests;
457
458 // flood
```

`vAddrToSend`—addresses to gossip to this peer. When your node learns a new peer address, it queues it here for eventual transmission. `setAddrKnown` tracks addresses this peer already knows, avoiding redundant announcements. This gossip protocol is how the network discovers new peers without centralized coordination.

```
459     vector<CAddress> vAddrToSend;
460     set<CAddress> setAddrKnown;
461
462 // inventory based relay
```

Inventory tracking: `setInventoryKnown` tracks what this peer has (or has been told about). When relaying a transaction, we check this set to avoid re-announcing. `vInventoryToSend` queues items to announce. This tracking prevents bandwidth waste—a transaction propagating through 10,000 nodes shouldn't be sent twice to any peer.

```
463     set<CInv> setInventoryKnown;
464     set<CInv> setInventoryKnown2;
465     vector<CInv> vInventoryToSend;
466     CCriticalSection cs_inventory;
```

`mapAskFor`—items we want to request from this peer, keyed by when to ask. When we receive an 'inv' for something we don't have, the first request is scheduled for right now. If a second peer announces the same item, its request is scheduled 2 minutes after the first (in case the first peer never delivers), and so on. Before sending, the node checks whether the item has already arrived; if another peer delivered it first, the request is skipped.

```
467     multimap<int64, CInv> mapAskFor;
468
469 // publish and subscription
470     vector<char> vfSubscribe;
471
472
473 CNode(SOCKET hSocketIn, CAddress addrIn, bool fInboundIn=false)
474 {
475         nServices = 0;
476         hSocket = hSocketIn;
477         vSend.SetType(SER_NETWORK);
478         vRecv.SetType(SER_NETWORK);
479         nPushPos = -1;
480         addr = addrIn;
481         nVersion = 0;
482         fClient = false; // set by version message
483         fInbound = fInboundIn;
484         fNetworkNode = false;
485         fDisconnect = false;
486         nRefCount = 0;
487         nReleaseTime = 0;
488         vfSubscribe.assign(256, false);
489
490 // Push a version message
491 /// when NTP implemented, change to just nTime = GetAdjustedTime()
492         int64 nTime = (fInbound ? GetAdjustedTime() : GetTime());
```

Version message on connect: `PushMessage("version", VERSION, nLocalServices, nTime, addr)`. The first message any node sends is `version`, announcing its protocol version, capabilities, and current time. Because it is pushed from the `CNode` constructor, both sides send it unprompted the moment the connection exists — inbound and outbound alike — rather than one replying to the other. There is no `verack` acknowledgment yet in v0.1.0; that came later. This exchange establishes the connection and negotiates compatible features.

```
493 PushMessage("version", VERSION, nLocalServices, nTime, addr);
494 }
495
496 ~CNode()
497 {
498 if (hSocket != INVALID_SOCKET)
499 closesocket(hSocket);
500 }
501
502 private:
503 CNode(const CNode&);
504 void operator=(const CNode&);
505 public:
506
507
508 bool ReadyToDisconnect()
509 {
510 return fDisconnect || GetRefCount() <= 0;
511 }
512
513 int GetRefCount()
514 {
515 return max(nRefCount, 0) + (GetTime() < nReleaseTime ? 1 : 0);
```

```
516 }
517
518 void AddRef(int64 nTimeout=0)
519 {
520 if (nTimeout != 0)
521             nReleaseTime = max(nReleaseTime, GetTime() + nTimeout);
522 else
523             nRefCount++;
524 }
525
526 void Release()
527 {
528         nRefCount--;
529 }
530
531
532
```

AddInventoryKnown() and PushInventory()—the relay optimization. When we have something to announce, we first check if the peer already knows it. If not, we queue it for announcement. This simple check prevents the quadratic blowup that would occur if every node blindly relayed everything to every peer.

```
533 void AddInventoryKnown(const CInv& inv)
534 {
535 CRITICAL_BLOCK(cs_inventory)
536             setInventoryKnown.insert(inv);
537 }
538
539 void PushInventory(const CInv& inv)
540 {
541 CRITICAL_BLOCK(cs_inventory)
542 if (!setInventoryKnown.count(inv))
543                 vInventoryToSend.push_back(inv);
544 }
545
```

AskFor()—schedules a data request with rate limiting. The comment explains: 'We're using mapAskFor as a priority queue.' Requests are spaced 2 minutes apart for the same item (retry interval). This prevents hammering peers for data they may not have. The microsecond timestamps ensure FIFO ordering even for requests scheduled at the same second.

```
546 void AskFor(const CInv& inv)
547 {
548 // We're using mapAskFor as a priority queue,
549 // the key is the earliest time the request can be sent
550         int64& nRequestTime = mapAlreadyAskedFor[inv];
551 printf("askfor %s  %I64d\n", inv.ToString().c_str(), nRequestTime);
552
553 // Make sure not to reuse time indexes to keep things in the same order
554         int64 nNow = (GetTime() - 1) * 1000000;
555 static int64 nLastTime;
556         nLastTime = nNow = max(nNow, ++nLastTime);
557
558 // Each retry is 2 minutes after the last
559         nRequestTime = max(nRequestTime + 2 * 60 * 1000000, nNow);
560         mapAskFor.insert(make_pair(nRequestTime, inv));
561 }
562
563
564
```

`BeginMessage()`—starts constructing an outbound message. Takes the mutex, records current buffer position, writes the header. `EndMessage()` patches in the actual size and releases the mutex. If anything goes wrong between Begin and End, `AbortMessage()` rolls back the buffer. This transactional pattern prevents malformed messages from being sent.

```
565 void BeginMessage(const char* pszCommand)
566 {
567 EnterCriticalSection(&cs_vSend);
568 if (nPushPos != -1)
569 AbortMessage();
570         nPushPos = vSend.size();
571         vSend << CMessageHeader(pszCommand, 0);
572 printf("sending: %-12s ", pszCommand);
573 }
574
575 void AbortMessage()
576 {
577 if (nPushPos == -1)
578 return;
579         vSend.resize(nPushPos);
580         nPushPos = -1;
581 LeaveCriticalSection(&cs_vSend);
582 printf("(aborted)\n");
583 }
584
585 void EndMessage()
586 {
587 extern int nDropMessagesTest;
588 if (nDropMessagesTest > 0 && GetRand(nDropMessagesTest) == 0)
589 {
590 printf("dropmessages DROPPING SEND MESSAGE\n");
591 AbortMessage();
592 return;
593 }
594
595 if (nPushPos == -1)
596 return;
597
598 // Patch in the size
599 unsigned int nSize = vSend.size() - nPushPos - sizeof(CMessageHeader);
600 memcpy((char*)&vSend[nPushPos] + offsetof(CMessageHeader, nMessageSize), &nSize, sizeof(nSize));
601
602 printf("(%d bytes)  ", nSize);
603 //for (int i = nPushPos+sizeof(CMessageHeader); i < min(vSend.size(),
    nPushPos+sizeof(CMessageHeader)+20U); i++)
604 //    printf("%02x ", vSend[i] & 0xff);
605 printf("\n");
606
607         nPushPos = -1;
608 LeaveCriticalSection(&cs_vSend);
609 }
610
611 void EndMessageAbortIfEmpty()
612 {
613 if (nPushPos == -1)
614 return;
615 int nSize = vSend.size() - nPushPos - sizeof(CMessageHeader);
616 if (nSize > 0)
617 EndMessage();
618 else
619 AbortMessage();
620 }
621
622 const char* GetMessageCommand() const
623 {
624 if (nPushPos == -1)
625 return "";
```

```
return &vSend[nPushPos] + offsetof(CMessageHeader, pchCommand);
}
```

`PushMessage()`—five overloaded templates for sending messages with zero to four arguments (C++ had no variadic templates until C++11). The syntax `vSend << a1 << a2` uses operator overloading to serialize each argument. This elegant design lets you write `PushMessage("block", block)` and have the block automatically serialized according to its type's `IMPLEMENT_SERIALIZE` definition.

```
void PushMessage(const char* pszCommand)
{
try
{
BeginMessage(pszCommand);
EndMessage();
}
catch (...)
{
AbortMessage();
throw;
}
}

template<typename T1>
void PushMessage(const char* pszCommand, const T1& a1)
{
try
{
BeginMessage(pszCommand);
            vSend << a1;
EndMessage();
}
catch (...)
{
AbortMessage();
throw;
}
}

template<typename T1, typename T2>
void PushMessage(const char* pszCommand, const T1& a1, const T2& a2)
{
try
{
BeginMessage(pszCommand);
            vSend << a1 << a2;
EndMessage();
}
catch (...)
{
AbortMessage();
throw;
}
}

template<typename T1, typename T2, typename T3>
void PushMessage(const char* pszCommand, const T1& a1, const T2& a2, const T3& a3)
{
try
{
BeginMessage(pszCommand);
            vSend << a1 << a2 << a3;
EndMessage();
}
```

```
catch (...)
{
AbortMessage();
throw;
}
}

template<typename T1, typename T2, typename T3, typename T4>
void PushMessage(const char* pszCommand, const T1& a1, const T2& a2, const T3& a3, const T4& a4)
{
try
{
BeginMessage(pszCommand);
            vSend << a1 << a2 << a3 << a4;
EndMessage();
}
catch (...)
{
AbortMessage();
throw;
}
}


void PushRequest(const char* pszCommand,
void (*fn)(void*, CDataStream&), void* param1)
{
        uint256 hashReply;
RAND_bytes((unsigned char*)&hashReply, sizeof(hashReply));

CRITICAL_BLOCK(cs_mapRequests)
            mapRequests[hashReply] = CRequestTracker(fn, param1);

PushMessage(pszCommand, hashReply);
}

template<typename T1>
void PushRequest(const char* pszCommand, const T1& a1,
void (*fn)(void*, CDataStream&), void* param1)
{
        uint256 hashReply;
RAND_bytes((unsigned char*)&hashReply, sizeof(hashReply));

CRITICAL_BLOCK(cs_mapRequests)
            mapRequests[hashReply] = CRequestTracker(fn, param1);

PushMessage(pszCommand, hashReply, a1);
}

template<typename T1, typename T2>
void PushRequest(const char* pszCommand, const T1& a1, const T2& a2,
void (*fn)(void*, CDataStream&), void* param1)
{
        uint256 hashReply;
RAND_bytes((unsigned char*)&hashReply, sizeof(hashReply));

CRITICAL_BLOCK(cs_mapRequests)
            mapRequests[hashReply] = CRequestTracker(fn, param1);

PushMessage(pszCommand, hashReply, a1, a2);
}



bool IsSubscribed(unsigned int nChannel);
void Subscribe(unsigned int nChannel, unsigned int nHops=0);
void CancelSubscribe(unsigned int nChannel);
void Disconnect();
};


```

```
759
760
761
762
763
764
765
```

`RelayInventory()`—broadcasts an announcement to all peers. Iterates through `vNodes` calling `PushInventory()` on each. The per-peer filtering (via `setInventoryKnown`) ensures no redundant announcements. This is how new transactions propagate: the sender broadcasts to their peers, those peers relay to their peers, exponentially spreading across the network.

```
    CALLED FROM  CBlock::AcceptBlock main.cpp p. 223
766 inline void RelayInventory(const CInv& inv)
767 {
768 // Put on lists to offer to the other nodes
769 CRITICAL_BLOCK(cs_vNodes)
770 foreach(CNode* pnode, vNodes)
771             pnode->PushInventory(inv);
772 }
773
774 template<typename T>
    CALLED FROM  CWalletTx::RelayWalletTransaction main.cpp p. 212 · ProcessMessage main.cpp p. 235 ·
    CEditReviewDialog::OnButtonSubmit ui.cpp p. 514
775 void RelayMessage(const CInv& inv, const T& a)
776 {
777     CDataStream ss(SER_NETWORK);
778     ss.reserve(10000);
779     ss << a;
780 RelayMessage(inv, ss);
781 }
782
```

`RelayMessage()`—caches and relays data. When relaying transactions or blocks, we might need to serve the same data to multiple peers. Rather than re-serializing each time, `mapRelay` caches the serialized form for 15 minutes. The template specialization handles `CDataStream` efficiently while the generic version accepts any serializable type.

```
783 template<>
784 inline void RelayMessage<>(const CInv& inv, const CDataStream& ss)
785 {
786 CRITICAL_BLOCK(cs_mapRelay)
787 {
788 // Expire old relay messages
789 while (!vRelayExpiration.empty() && vRelayExpiration.front().first < GetTime())
790 {
791             mapRelay.erase(vRelayExpiration.front().second);
792             vRelayExpiration.pop_front();
793 }
794
795 // Save original serialized message so newer versions are preserved
796         mapRelay[inv] = ss;
797         vRelayExpiration.push_back(make_pair(GetTime() + 15 * 60, inv));
798 }
799
800 RelayInventory(inv);
801 }
802
803
804
805
806
807
808
```

```
809
810 //
811 // Templates for the publish and subscription system.
812 // The object being published as T& obj needs to have:
813 //   a set<unsigned int> setSources member
814 //   specializations of AdvertInsert and AdvertErase
815 // Currently implemented for CTable and CProduct.
816 //
817
```

Publish/subscribe system—Satoshi's marketplace feature. `AdvertStartPublish()` and related functions implement a pub/sub mechanism for `CTable` and `CProduct` objects. Nodes were meant to subscribe to channels and receive product listings — but v0.1.0's message handler in main.cpp has no case for `publish` or `subscribe`, so a receiving node simply ignores them as unknown commands. This code was largely unused and removed in later versions,[2] but hints at Satoshi's vision of Bitcoin as a platform for decentralized commerce.

```
818 template<typename T>
    CALLED FROM CEditProductDialog::OnButtonSend ui.cpp p. 508
819 void AdvertStartPublish(CNode* pfrom, unsigned int nChannel, unsigned int nHops, T& obj)
820 {
821 // Add to sources
822     obj.setSources.insert(pfrom->addr.ip);
823
824 if (!AdvertInsert(obj))
825 return;
826
827 // Relay
828 CRITICAL_BLOCK(cs_vNodes)
829 foreach(CNode* pnode, vNodes)
830 if (pnode != pfrom && (nHops < PUBLISH_HOPS || pnode->IsSubscribed(nChannel)))
831                 pnode->PushMessage("publish", nChannel, nHops, obj);
832 }
833
834 template<typename T>
    CALLED FROM AdvertRemoveSource p. 365
835 void AdvertStopPublish(CNode* pfrom, unsigned int nChannel, unsigned int nHops, T& obj)
836 {
837     uint256 hash = obj.GetHash();
838
839 CRITICAL_BLOCK(cs_vNodes)
840 foreach(CNode* pnode, vNodes)
841 if (pnode != pfrom && (nHops < PUBLISH_HOPS || pnode->IsSubscribed(nChannel)))
842                 pnode->PushMessage("pub-cancel", nChannel, nHops, hash);
843
844 AdvertErase(obj);
845 }
846
847 template<typename T>
    CALLED FROM CNode::Disconnect net.cpp p. 331
848 void AdvertRemoveSource(CNode* pfrom, unsigned int nChannel, unsigned int nHops, T& obj)
849 {
850 // Remove a source
851     obj.setSources.erase(pfrom->addr.ip);
852
853 // If no longer supported by any sources, cancel it
854 if (obj.setSources.empty())
855 AdvertStopPublish(pfrom, nChannel, nHops, obj);
856 }
857
```

SUMMARY

The CNode class is Bitcoin's window to the world. Each instance represents a relationship with another participant in the network—a relationship built on mutual verification rather than trust. The careful tracking of what each peer knows, the rate-limited requests, the gossip protocols for addresses and inventory—all work together to create a resilient network where no single point of failure can bring down the system.

Satoshi's protocol design reveals deep understanding of distributed systems. The pull-based model (inv/getdata) prevents bandwidth abuse. Peer address gossip enables organic network growth without any central directory. The version handshake allows protocol evolution without flag days. These patterns have proven extraordinarily robust: Bitcoin's peer-to-peer layer has operated continuously since January 2009,[1] surviving attacks, bugs, and growth from a handful of nodes to tens of thousands worldwide.

The unused marketplace code (MSG_PRODUCT, MSG_REVIEW, pub/sub) reveals a broader vision that Satoshi ultimately didn't pursue. Bitcoin became focused as a currency and settlement layer, with commerce applications built on top rather than integrated into the base protocol. But these remnants show Satoshi thinking beyond simple value transfer.

KEY TERMS

CNode

The class representing one live connection to a peer, holding its buffers, negotiated version, and records of what that peer has and knows.

inv

The inventory message advertising a list of (type, hash) pairs, so peers can request only the transactions or blocks they lack.

getdata

The message a node sends to pull the actual transaction or block after learning of it from a peer's inv announcement.

message header

The 20-byte prefix on every network message: four magic bytes, a 12-byte command name, and the payload size.

magic bytes

The fixed 4-byte marker 0xf9beb4d9 that starts every message, chosen to be unlikely in ordinary data so message boundaries are easy to find.

version handshake

The exchange of version messages when two nodes connect, negotiating protocol version, capabilities, and time before any other traffic flows.

addr gossip

The relaying of peer addresses in addr messages, letting the network discover new participants without any central directory.

NODE_NETWORK

The service flag a node sets to declare itself a full node willing to serve historical blocks to its peers.

relay

The forwarding of transactions and blocks to every connected peer except those already known to have them, flooding new data across the mesh.

SOURCES

1. Satoshi Nakamoto, *Bitcoin genesis block (block 0)* (January 3, 2009). Coinbase embeds The Times headline as timestamp proof. https://mempool.space/block/000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f

2. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/

readme.txt

77 lines TEXT

SATOSHI'S INSTRUCTIONS

This is the original README file distributed with Bitcoin v0.1.0 — a primary historical document from the birth of cryptocurrency. Written by Satoshi Nakamoto himself, it is part build instructions, part architectural overview, and part window into the mind of Bitcoin's creator.[1] Every sentence reveals something about the design decisions, constraints, and priorities of the project in January 2009.

The document is striking for what it emphasizes and what it omits. There is no explanation of what Bitcoin *is* — no mention of digital cash, peer-to-peer payments, or decentralized consensus. Satoshi had already published the whitepaper on the cryptography mailing list in October 2008;[2][3] by the time this README was written, the audience was assumed to already understand the concept. Instead, the focus is entirely practical: how to obtain the dependencies, how to configure OpenSSL, and how to compile the code. This is a builder's document, not a manifesto.

Perhaps the most remarkable line is on line 36: "Bitcoin does not use any encryption."[1] This single sentence distinguishes Bitcoin from every cypherpunk project that came before it. Tools like PGP and Tor relied on encryption to protect privacy, and DigiCash hid its users behind blind signatures issued by a central mint.[4] Bitcoin uses cryptography — specifically digital signatures and hash functions — but never encrypts data. All transactions are broadcast in the clear. All blocks are publicly readable. Privacy comes not from secrecy but from pseudonymity: your identity is hidden behind a cryptographic public key, but your transactions are visible to everyone. Satoshi never said whether it was a motive, but the design also kept Bitcoin clear of the encryption export controls that had hindered earlier cypherpunk projects (and that, by 2009, had already been largely relaxed).[5]

The build instructions also reveal that Bitcoin v0.1.0 was a *Windows desktop application* with a graphical interface. The supported compilers — MinGW GCC 3.4.5 and Microsoft Visual C++ 6.0 SP6 — were both Windows toolchains. The dependency paths all use Windows-style backslash notation. There is no mention of Linux, macOS, or any Unix-like system. Satoshi built Bitcoin to run on the most common desktop operating system of 2009.

'BitCoin v0.01 ALPHA' — note the camel-case spelling with a capital C. Satoshi originally wrote the name as two words ('Bit Coin') or camel-case ('BitCoin') before settling on the now-standard 'Bitcoin' with a lowercase 'c'. The 'v0.01' version number (the release is catalogued today as v0.1.0) and 'ALPHA' designation signal that Satoshi considered this a very early release — not even a beta. He was being cautious about the software's maturity while still making it available for others to examine and test.

```
BitCoin v0.01 ALPHA

```

Copyright and license boilerplate. The MIT/X11 license reference and the OpenSSL attribution are legally required. The OpenSSL Toolkit uses a dual BSD/SSLeay license that requires acknowledgment in derived works. Eric Young (eay@cryptsoft.com) wrote the original SSLeay library that OpenSSL was forked from — his name appears here because OpenSSL's license mandates it.

```
Copyright (c) 2009 Satoshi Nakamoto
Distributed under the MIT/X11 software license, see the accompanying
file license.txt or http://www.opensource.org/licenses/mit-license.php.
This product includes software developed by the OpenSSL Project for use in
the OpenSSL Toolkit (http://www.openssl.org/).  This product includes
cryptographic software written by Eric Young (eay@cryptsoft.com).


```

Compilers Supported — only two, both Windows-only. MinGW (Minimalist GNU for Windows) provides GCC 3.4.5 on Windows; MSVC 6.0 SP6 was Microsoft's C++ compiler from 1998, still widely used in 2009 for its stability. The specific version numbers suggest Satoshi tested with exactly these compilers. GCC 3.4.5 was already old in 2009 (GCC 4.x had been out for years), suggesting Satoshi prioritized compatibility over modern language features.

```
Compilers Supported
-------------------
MinGW GCC (v3.4.5)
Microsoft Visual C++ 6.0 SP6


```

Dependencies — four external libraries form Bitcoin's foundation. The default paths (backslash-prefixed root directories like `\wxWidgets`) are Windows conventions, suggesting Satoshi installed each library at the root of a drive. This section is essentially the `package.json` or `Cargo.toml` of its era — manual dependency management before package managers existed for C++.

```
Dependencies
------------
Libraries you need to obtain separately to build:

             default path   download
```

wxWidgets — a cross-platform C++ GUI framework. Satoshi chose it over alternatives like Qt (which had licensing concerns at the time — Qt was GPL, not LGPL, until 2009)[6] and FLTK (too minimal). wxWidgets' LGPL license with liberal exceptions meant Bitcoin could use it without being forced into GPL copyleft.[8] Bitcoin Core eventually switched to Qt in 2011 after Qt adopted the LGPL.[7]

```
22 wxWidgets      \wxWidgets     http://www.wxwidgets.org/downloads/
```

OpenSSL — provides ECDSA (transaction signing), SHA-256 (proof-of-work hashing), and RIPEMD-160 (address generation). In 2009, OpenSSL was the de facto standard cryptographic library for C/C++.[9] Its 'Old BSD license with the problematic advertising requirement' refers to an 'advertising clause' in the style of the original BSD license: OpenSSL's terms require any advertising material mentioning features of the software to credit the OpenSSL Project and Eric Young — a clause widely considered a nuisance. It is the reason lines 6-8 of this README exist.

```
23 OpenSSL        \OpenSSL       http://www.openssl.org/source/
```

Berkeley DB — the embedded database storing blocks, transactions, and wallet data. Its license required that 'linked software must be free open source' — a significant constraint.[10] This effectively prevented anyone from building a proprietary Bitcoin node. Later versions of Bitcoin Core replaced Berkeley DB with LevelDB for the block index and chainstate (v0.8, 2013) — a move made for performance and reliability rather than licensing — though `wallet.dat` continued using BDB for years.[7]

```
24 Berkeley DB    \DB            http://www.oracle.com/technology/software/products/berkeley-db/index.html
```

Boost — the Swiss Army knife of C++ libraries. Satoshi used it for `BOOST_FOREACH` (range iteration), `lexical_cast` (string conversion), `tuple` (multiple return values), and `array` (fixed-size containers). The 'MIT-like license' made it the least legally complicated dependency.[11] Many Boost features were later absorbed into C++11, and Bitcoin Core gradually removed Boost dependencies over the following decade.

```
25 Boost          \Boost         http://www.boost.org/users/download/
26
27 Their licenses:
28 wxWidgets      LGPL 2.1 with very liberal exceptions
29 OpenSSL        Old BSD license with the problematic advertising requirement
30 Berkeley DB    New BSD license with additional requirement that linked software must be free open source
31 Boost          MIT-like license
32
33
```

OpenSSL build instructions — this section is remarkably detailed and reveals a key insight. Satoshi provides step-by-step instructions for building a *minimal* OpenSSL that excludes all encryption algorithms (RC2, RC4, RC5, IDEA, DES, Blowfish, CAST, AES, Camellia, SEED, RSA, DH). He strips OpenSSL down to only what Bitcoin actually needs: ECDSA, SHA-256, and RIPEMD-160.

```
34 OpenSSL
35 -------
```

'Bitcoin does not use any encryption.' A profound statement.[1] Earlier digital cash designs leaned on secrecy for privacy — David Chaum's DigiCash (1990) on blind signatures from a central mint,[4] Wei Dai's b-money proposal (1998) on an untraceable network of pseudonyms.[12] Satoshi took a fundamentally different approach: all data is public, all transactions are broadcast in cleartext, and privacy comes from pseudonymous keys rather than encrypted channels. A side effect — Satoshi never called it a motive — was distance from the U.S. encryption export regulations (ITAR/EAR) that had hamstrung earlier cypherpunk tools like PGP, though by 2009 those rules had been largely relaxed.[5]

```
Bitcoin does not use any encryption.  If you want to do a no-everything
build of OpenSSL to exclude encryption routines, a few patches are required.
(OpenSSL v0.9.8h)

```

Patching OpenSSL to remove RSA dependencies. The `#ifndef OPENSSL_NO_RSA` guard around `#include <openssl/rsa.h>` and the stub function `ERR_load_RSA_strings(void) { }` are surgical modifications to allow OpenSSL to compile without its RSA implementation. This is Satoshi being thorough — not just avoiding encryption in Bitcoin's code, but ensuring the compiled binary literally cannot perform RSA operations.

```
Edit engines\e_gmp.c and put this #ifndef around #include <openssl/rsa.h>
  #ifndef OPENSSL_NO_RSA
  #include <openssl/rsa.h>
  #endif

Add this to crypto\err\err_all.c before the ERR_load_crypto_strings line:
  void ERR_load_RSA_strings(void) { }

```

The `no-everything` configure line. Satoshi explicitly disables every symmetric cipher and public-key algorithm except the ones Bitcoin needs. The comment about 'batch files can't handle more than 9 parameters' is a pragmatic workaround for a Windows CMD limitation — the configure options are too numerous to pass as command-line arguments, so they must be embedded in the batch file itself.

```
Edit ms\mingw32.bat and replace the Configure line's parameters with this
no-everything list.  You have to put this in the batch file because batch
files can't handle more than 9 parameters.
  perl Configure mingw threads no-rc2 no-rc4 no-rc5 no-idea no-des no-bf no-cast no-aes no-camellia no-
seed no-rsa no-dh

```

A classic build system hack: the OpenSSL build fails partway through after successfully producing `libeay32` (all Bitcoin cares about). Rather than fix the build, Satoshi simply comments out the error check (`REM if errorlevel 1 goto end`) so the script continues past the failure and runs dllwrap to generate `libeay32.dll`. Pragmatic, not elegant — but it works.

```
Also REM out the following line in ms\mingw32.bat.  The build fails after it's
already finished building libeay32, which is all we care about, but the
failure aborts the script before it runs dllwrap to generate libeay32.dll.
  REM  if errorlevel 1 goto end

Build
  ms\mingw32.bat

If you want to use it with MSVC, generate the .lib file
```

```
  lib /machine:i386 /def:ms\libeay32.def /out:out\libeay32.lib


```

Berkeley DB build instructions for MinGW with MSYS. The `--enable-mingw` flag cross-compiles for Windows, and `--enable-cxx` generates the C++ bindings that Bitcoin's `db.cpp` uses. These are terse instructions — just three lines — suggesting Berkeley DB was relatively straightforward to build compared to OpenSSL.

```
Berkeley DB
-----------
MinGW with MSYS:
cd \DB\build_unix
sh ../dist/configure --enable-mingw --enable-cxx
make


```

Boost compatibility note — Satoshi warns that Boost 1.37 doesn't compile with MSVC 6.0 and recommends version 1.35 instead. MSVC 6.0 predated the C++98 standard's full adoption, and newer Boost versions relied on template features that MSVC 6.0's compiler couldn't handle. This is a reminder of the era's toolchain constraints — Satoshi was working with decade-old compiler technology.

```
Boost
-----
You may need Boost version 1.35 to build with MSVC 6.0.  I couldn't get
version 1.37 to compile with MSVC 6.0.

```

SUMMARY

This README is a time capsule from January 2009. It documents a project built by one person using Windows-era C++ toolchains, manually managing four external dependencies, and patching OpenSSL by hand to strip out encryption. There is no `./configure && make && make install` — each dependency must be separately downloaded, built, and installed into the correct path. The barrier to building Bitcoin from source in 2009 was substantial, which helps explain why so few people ran it in the early months.

The document also serves as an architectural map. The four dependencies — wxWidgets, OpenSSL, Berkeley DB, and Boost — define the four pillars of Bitcoin v0.1.0: a graphical user interface, cryptographic primitives, persistent storage, and C++ utility support. Every one of these dependencies would eventually be replaced (Qt, libsecp256k1, LevelDB, C++11/17 standard library),[67] but the four-pillar architecture — presentation, cryptography, storage, utilities — endures in Bitcoin Core today.

Most importantly, the README establishes that Bitcoin was designed to be built and verified by anyone. Despite the manual dependency management, Satoshi provided enough detail for a competent C++ developer to reproduce the build. This transparency — the ability to compile the code yourself and verify that the binary matches — is foundational to the trustless ethos of the entire system. You don't have to trust Satoshi. You can read the code and build it yourself.

KEY TERMS

Berkeley DB

An embedded key-value database library, linked directly into the program, that Bitcoin v0.1.0 uses for its block and transaction index, wallet, and peer addresses; raw blocks go in flat files.

libeay32

The Windows build of OpenSSL's general cryptography library, the only part of OpenSSL that Bitcoin needs; the SSL protocol library is never used.

no-everything build

Satoshi's name for an OpenSSL build configured with options like no-rsa and no-aes, leaving out the encryption routines Bitcoin never calls.

advertising clause

A license term inherited from the original BSD license that obliges credit to the OpenSSL Project and Eric Young, which is why the README names them.

LGPL

The GNU Lesser General Public License, a weaker form of copyleft that permits linking a library into programs under other licenses; wxWidgets used version 2.1.

MSYS

A minimal Unix-style shell environment for Windows that accompanies MinGW, used here to run Berkeley DB's configure script and make.

SOURCES

1. Satoshi Nakamoto, *readme.txt (BitCoin v0.01 ALPHA)* (January 2009). Primary source — distributed with Bitcoin v0.1.0.
2. Satoshi Nakamoto, *Bitcoin P2P e-cash paper (Cryptography Mailing List)* (October 31, 2008). Initial announcement of the Bitcoin whitepaper. https://satoshi.nakamotoinstitute.org/emails/cryptography/1/
3. Satoshi Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (October 2008). https://bitcoin.org/bitcoin.pdf
4. David Chaum, *Blind Signatures for Untraceable Payments* (1983). Foundational DigiCash cryptography; Advances in Cryptology. https://sceweb.sce.uhcl.edu/yang/teaching/csci5234WebSecurityFall2011/Chaum-blind-signatures.PDF
5. U.S. Department of Commerce / Department of State, *Encryption export controls (ITAR/EAR)* (1990s–2000s). Historical export regulations that constrained earlier cypherpunk software. https://www.eff.org/deeplinks/2015/06/short-history-crypto-war

6. Qt Company / Nokia, *Qt licensed under LGPL* (January 2009). Enabled Bitcoin Core's later migration from wxWidgets to Qt. https://blog.qt.io/blog/2009/01/14/nokia-qt-lgpl/
7. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
8. wxWidgets Project, *wxWidgets Licence* (2005). https://www.wxwidgets.org/about/licence/
9. OpenSSL Project, *OpenSSL License / SSLeay License* (1998–2009). https://www.openssl.org/source/license-openssl-ssleay.txt
10. Oracle Corporation, *Berkeley DB License* (2009). Sleepycat-era open-source linking requirements referenced in readme.txt.
11. Boost.org, *Boost Software License 1.0* (2003). https://www.boost.org/LICENSE_1_0.txt
12. Wei Dai, *b-money* (1998). http://www.weidai.com/bmoney.txt

script.cpp

1128 lines CPP

THE VIRTUAL MACHINE

This file implements Bitcoin's script interpreter — the virtual machine that determines who can spend coins. Every bitcoin ever transferred — hundreds of billions of dollars in value — passed through this exact code. The interpreter is deceptively simple: a stack of byte arrays, a program counter, and roughly 100 opcodes. Yet this simplicity is its strength.

Satoshi designed Script using "reverse Polish notation" (RPN), a Forth-style syntax where operators come after operands. Instead of "3 + 4", you write "3 4 +". The interpreter maintains a stack; data pushes go onto the stack, operations pop arguments and push results. This approach eliminates parsing complexity — no operator precedence, no parentheses, no ambiguity. The stack state after execution determines validity: if the top element is non-zero (truthy), the script passes.

What makes this remarkable is its generality. Satoshi didn't hardcode "check signature against public key" as the only spending rule. Instead, he created a language that expresses arbitrary conditions. Multi-signature escrow, hash locks, time-locked payments — all possible without protocol changes, just different scripts. This foresight enabled P2SH, SegWit, Taproot, and Lightning without ever replacing the core spending mechanism.[1]

The script system is also why Bitcoin survived its early bugs. When critical vulnerabilities were found (like the OP_LSHIFT crash and the OP_RETURN spend-anything bug of July 2010),[2] Satoshi could disable individual opcodes without breaking the entire system.[3][4] Scripts using disabled opcodes simply fail; everything else continues working.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

#include "headers.h"

bool CheckSig(vector<unsigned char> vchSig, vector<unsigned char> vchPubKey, CScript scriptCode, const
CTransaction& txTo, unsigned int nIn, int nHashType);
```

`valtype` — the fundamental unit of Script. A script stack element is a vector of bytes, which can represent a number, a hash, a public key, or a signature. Everything on the stack is just bytes; opcodes interpret them according to context.

```
typedef vector<unsigned char> valtype;
```

Script boolean constants: vchFalse and vchZero are empty byte vectors (the falsy value), vchTrue is a single byte 0×01. The BigNum versions (bnZero, bnOne) are for arithmetic opcodes. Satoshi defines both representations up front for clarity.

```
static const valtype vchFalse(0);
static const valtype vchZero(0);
static const valtype vchTrue(1, 1);
static const CBigNum bnZero(0);
static const CBigNum bnOne(1);
static const CBigNum bnFalse(0);
static const CBigNum bnTrue(1);
```

`CastToBool` — the truthiness rule: a stack element is true if it is a non-zero BigNum. An empty vector or a vector encoding zero is false. This means 0×00, 0×0000, and an empty byte array are all false; anything else is true.

```
CALLED FROM  EvalScript p. 385
bool CastToBool(const valtype& vch)
{
return (CBigNum(vch) != bnZero);
}

CALLED FROM  EvalScript p. 390
void MakeSameSize(valtype& vch1, valtype& vch2)
{
// Lengthen the shorter one
if (vch1.size() < vch2.size())
        vch1.resize(vch2.size(), 0);
if (vch2.size() < vch1.size())
        vch2.resize(vch1.size(), 0);
}



//
// Script is a stack machine (like Forth) that evaluates a predicate
// returning a bool indicating valid or not.  There are no loops.
//
```

`stacktop(i)` macro: negative indexing into the stack vector — stacktop(-1) is the top element, stacktop(-2) is second from top. stack.at() throws if out of range, providing bounds checking. This Forth-style negative indexing matches the mental model of a stack.

```
#define stacktop(i) (stack.at(stack.size()+(i)))
#define altstacktop(i) (altstack.at(altstack.size()+(i)))

```

`EvalScript` — the interpreter entry point. Takes a complete script (scriptSig prepended to scriptPubKey via OP_CODESEPARATOR), the spending transaction, the input index, and hash type. Returns true only if execution completes without error and the top of the stack is truthy.

```
CALLED FROM SignSignature p. 402 · VerifySignature p. 402
bool EvalScript(const CScript& script, const CTransaction& txTo, unsigned int nIn, int nHashType,
                vector<vector<unsigned char> >* pvStackRet)
{
    CAutoBN_CTX pctx;
    CScript::const_iterator pc = script.begin();
    CScript::const_iterator pend = script.end();
    CScript::const_iterator pbegincodehash = script.begin();
```

`vfExec` — the conditional execution stack. Each OP_IF pushes a boolean; OP_ELSE toggles the top; OP_ENDIF pops it. fExec (line 60) is true only when all levels are true — unexecuted branches are skipped but still parsed. This enables nested conditionals in scripts.

```
    vector<bool> vfExec;
    vector<valtype> stack;
    vector<valtype> altstack;
if (pvStackRet)
        pvStackRet->clear();


```

The main interpreter loop iterates over opcodes one by one, advancing the program counter pc. There are no jumps, no loops, no recursion — Script is intentionally not Turing complete. This makes script execution time provably bounded, which is essential for validating thousands of transactions per block.

```
while (pc < pend)
{
```

`fExec` — whether the current position is in an active (non-skipped) branch. If any level of vfExec contains false, the current code is inside a not-taken branch. Data pushes and most opcodes are skipped; only OP_IF/OP_ELSE/OP_ENDIF are processed while in a skipped branch to maintain correct nesting.

```
bool fExec = !count(vfExec.begin(), vfExec.end(), false);

//
// Read instruction
//
        opcodetype opcode;
        valtype vchPushValue;
if (!script.GetOp(pc, opcode, vchPushValue))
return false;

```

Data push dispatch: opcodes 0×01–0×4b push that many bytes. OP_PUSHDATA1 uses the next byte as a length, OP_PUSHDATA2 uses two bytes, OP_PUSHDATA4 uses four. These cover everything from a 1-byte integer to pushes of arbitrary size — v0.1.0 sets no maximum (a 5,000-byte cap arrived in mid-2010, cut to 520 bytes in August 2010).[4] The vast majority of real scripts are mostly push operations followed by a few crypto opcodes.

```
if (fExec && opcode <= OP_PUSHDATA4)
            stack.push_back(vchPushValue);
else if (fExec || (OP_IF <= opcode && opcode <= OP_ENDIF))
switch (opcode)
{
//
// Push value
//
```

`OP_1` through OP_16 push small integers without a separate data push—the value is encoded in the opcode byte itself. The value is computed from the opcode byte: (opcode − OP_1 + 1). OP_1NEGATE pushes −1. Multi-signature scripts use these: e.g. OP_2 for "require 2 of…" in a 2-of-3 multisig.

```
case OP_1NEGATE:
case OP_1:
case OP_2:
case OP_3:
case OP_4:
case OP_5:
case OP_6:
case OP_7:
case OP_8:
case OP_9:
case OP_10:
case OP_11:
case OP_12:
case OP_13:
case OP_14:
case OP_15:
case OP_16:
{
// ( -- value)
                CBigNum bn((int)opcode - (int)(OP_1 - 1));
                stack.push_back(bn.getvch());
}
break;


//
// Control
//
case OP_NOP:
break;

```

`OP_VER` pushes the client's VERSION constant. This was intended to allow version-sensitive scripts — a script could branch based on protocol version. It was deprecated before widespread use; OP_VERIF and OP_VERNOTIF below were its conditional counterparts.

```
case OP_VER:
{
                CBigNum bn(VERSION);
                stack.push_back(bn.getvch());
}
```

```
114 break;
115
```

OP_IF/OP_NOTIF: pop the top stack element, evaluate as boolean, push result onto vfExec. If false, the branch up to OP_ELSE/OP_ENDIF is skipped. OP_VERIF/OP_VERNOTIF compare the popped value against the client's VERSION instead — a forward-compatibility mechanism Satoshi included but never deployed.

```
116 case OP_IF:
117 case OP_NOTIF:
118 case OP_VERIF:
119 case OP_VERNOTIF:
120 {
121 // <expression> if [statements] [else [statements]] endif
122 bool fValue = false;
123 if (fExec)
124 {
125 if (stack.size() < 1)
126 return false;
127                     valtype& vch = stacktop(-1);
128 if (opcode == OP_VERIF || opcode == OP_VERNOTIF)
129                         fValue = (CBigNum(VERSION) >= CBigNum(vch));
130 else
131                         fValue = CastToBool(vch);
132 if (opcode == OP_NOTIF || opcode == OP_VERNOTIF)
133                         fValue = !fValue;
134                     stack.pop_back();
135 }
136                 vfExec.push_back(fValue);
137 }
138 break;
139
140 case OP_ELSE:
141 {
142 if (vfExec.empty())
143 return false;
144                 vfExec.back() = !vfExec.back();
145 }
146 break;
147
148 case OP_ENDIF:
149 {
150 if (vfExec.empty())
151 return false;
152                 vfExec.pop_back();
153 }
154 break;
155
```

OP_VERIFY: if the top element is false, abort execution (set pc = pend, which will make the script fail). If true, pop the element and continue. Used as a suffix to convert "push bool" opcodes into immediate fail conditions — OP_EQUALVERIFY is OP_EQUAL + OP_VERIFY.

```
156 case OP_VERIFY:
157 {
158 // (true -- ) or
159 // (false -- false) and return
160 if (stack.size() < 1)
161 return false;
162 bool fValue = CastToBool(stacktop(-1));
163 if (fValue)
164                     stack.pop_back();
165 else
166                     pc = pend;
167 }
168 break;
```

```
169
170 case OP_RETURN:
171 {
172                 pc = pend;
173 }
174 break;
175
176
177 //
178 // Stack ops
179 //
```

Alt stack: a secondary stack separate from the main stack. OP_TOALTSTACK moves the top element there; OP_FROMALTSTACK brings it back. Allows scripts to save a value, do other operations, and restore it later — essential for complex multi-step scripts without stack pollution.

```
180 case OP_TOALTSTACK:
181 {
182 if (stack.size() < 1)
183 return false;
184                 altstack.push_back(stacktop(-1));
185                 stack.pop_back();
186 }
187 break;
188
189 case OP_FROMALTSTACK:
190 {
191 if (altstack.size() < 1)
192 return false;
193                 stack.push_back(altstacktop(-1));
194                 altstack.pop_back();
195 }
196 break;
197
198 case OP_2DROP:
199 {
200 // (x1 x2 -- )
201                 stack.pop_back();
202                 stack.pop_back();
203 }
204 break;
205
206 case OP_2DUP:
207 {
208 // (x1 x2 -- x1 x2 x1 x2)
209 if (stack.size() < 2)
210 return false;
211                 valtype vch1 = stacktop(-2);
212                 valtype vch2 = stacktop(-1);
213                 stack.push_back(vch1);
214                 stack.push_back(vch2);
215 }
216 break;
217
218 case OP_3DUP:
219 {
220 // (x1 x2 x3 -- x1 x2 x3 x1 x2 x3)
221 if (stack.size() < 3)
222 return false;
223                 valtype vch1 = stacktop(-3);
224                 valtype vch2 = stacktop(-2);
225                 valtype vch3 = stacktop(-1);
226                 stack.push_back(vch1);
227                 stack.push_back(vch2);
228                 stack.push_back(vch3);
229 }
230 break;
231
232 case OP_2OVER:
```

```
{
// (x1 x2 x3 x4 -- x1 x2 x3 x4 x1 x2)
if (stack.size() < 4)
return false;
                valtype vch1 = stacktop(-4);
                valtype vch2 = stacktop(-3);
                stack.push_back(vch1);
                stack.push_back(vch2);
}
break;

case OP_2ROT:
{
// (x1 x2 x3 x4 x5 x6 -- x3 x4 x5 x6 x1 x2)
if (stack.size() < 6)
return false;
                valtype vch1 = stacktop(-6);
                valtype vch2 = stacktop(-5);
                stack.erase(stack.end()-6, stack.end()-4);
                stack.push_back(vch1);
                stack.push_back(vch2);
}
break;

case OP_2SWAP:
{
// (x1 x2 x3 x4 -- x3 x4 x1 x2)
if (stack.size() < 4)
return false;
swap(stacktop(-4), stacktop(-2));
swap(stacktop(-3), stacktop(-1));
}
break;

case OP_IFDUP:
{
// (x - 0 | x x)
if (stack.size() < 1)
return false;
                valtype vch = stacktop(-1);
if (CastToBool(vch))
                    stack.push_back(vch);
}
break;

case OP_DEPTH:
{
// -- stacksize
                CBigNum bn(stack.size());
                stack.push_back(bn.getvch());
}
break;

case OP_DROP:
{
// (x -- )
if (stack.size() < 1)
return false;
                stack.pop_back();
}
break;

case OP_DUP:
{
// (x -- x x)
if (stack.size() < 1)
return false;
                valtype vch = stacktop(-1);
                stack.push_back(vch);
}
break;

```

```
305 case OP_NIP:
306 {
307 // (x1 x2 -- x2)
308 if (stack.size() < 2)
309 return false;
310                 stack.erase(stack.end() - 2);
311 }
312 break;
313
314 case OP_OVER:
315 {
316 // (x1 x2 -- x1 x2 x1)
317 if (stack.size() < 2)
318 return false;
319                 valtype vch = stacktop(-2);
320                 stack.push_back(vch);
321 }
322 break;
323
324 case OP_PICK:
325 case OP_ROLL:
326 {
327 // (xn ... x2 x1 x0 n - xn ... x2 x1 x0 xn)
328 // (xn ... x2 x1 x0 n - ... x2 x1 x0 xn)
329 if (stack.size() < 2)
330 return false;
331 int n = CBigNum(stacktop(-1)).getint();
332                 stack.pop_back();
333 if (n < 0 || n >= stack.size())
334 return false;
335                 valtype vch = stacktop(-n-1);
336 if (opcode == OP_ROLL)
337                     stack.erase(stack.end()-n-1);
338                 stack.push_back(vch);
339 }
340 break;
341
342 case OP_ROT:
343 {
344 // (x1 x2 x3 -- x2 x3 x1)
345 //  x2 x1 x3  after first swap
346 //  x2 x3 x1  after second swap
347 if (stack.size() < 3)
348 return false;
349 swap(stacktop(-3), stacktop(-2));
350 swap(stacktop(-2), stacktop(-1));
351 }
352 break;
353
354 case OP_SWAP:
355 {
356 // (x1 x2 -- x2 x1)
357 if (stack.size() < 2)
358 return false;
359 swap(stacktop(-2), stacktop(-1));
360 }
361 break;
362
363 case OP_TUCK:
364 {
365 // (x1 x2 -- x2 x1 x2)
366 if (stack.size() < 2)
367 return false;
368                 valtype vch = stacktop(-1);
369                 stack.insert(stack.end()-2, vch);
370 }
371 break;
372
373
374 //
375 // Splice ops
376 //
```

`OP_CAT` — concatenates the top two stack elements. This opcode was present in v0.1.0 but disabled in August 2010 along with the other splice, bitwise, and multiply/shift opcodes.[43] The danger is easy to see: repeated `OP_DUP OP_CAT` doubles an element each time, so a few dozen opcodes could request gigabytes of memory.

```
377 case OP_CAT:
378 {
379 // (x1 x2 -- out)
380 if (stack.size() < 2)
381 return false;
382                 valtype& vch1 = stacktop(-2);
383                 valtype& vch2 = stacktop(-1);
384                 vch1.insert(vch1.end(), vch2.begin(), vch2.end());
385                 stack.pop_back();
386 }
387 break;
388
389 case OP_SUBSTR:
390 {
391 // (in begin size -- out)
392 if (stack.size() < 3)
393 return false;
394                 valtype& vch = stacktop(-3);
395 int nBegin = CBigNum(stacktop(-2)).getint();
396 int nEnd = nBegin + CBigNum(stacktop(-1)).getint();
397 if (nBegin < 0 || nEnd < nBegin)
398 return false;
399 if (nBegin > vch.size())
400                     nBegin = vch.size();
401 if (nEnd > vch.size())
402                     nEnd = vch.size();
403                 vch.erase(vch.begin() + nEnd, vch.end());
404                 vch.erase(vch.begin(), vch.begin() + nBegin);
405                 stack.pop_back();
406                 stack.pop_back();
407 }
408 break;
409
410 case OP_LEFT:
411 case OP_RIGHT:
412 {
413 // (in size -- out)
414 if (stack.size() < 2)
415 return false;
416                 valtype& vch = stacktop(-2);
417 int nSize = CBigNum(stacktop(-1)).getint();
418 if (nSize < 0)
419 return false;
420 if (nSize > vch.size())
421                     nSize = vch.size();
422 if (opcode == OP_LEFT)
423                     vch.erase(vch.begin() + nSize, vch.end());
424 else
425                     vch.erase(vch.begin(), vch.end() - nSize);
426                 stack.pop_back();
427 }
428 break;
429
430 case OP_SIZE:
431 {
432 // (in -- in size)
433 if (stack.size() < 1)
434 return false;
435                 CBigNum bn(stacktop(-1).size());
436                 stack.push_back(bn.getvch());
437 }
438 break;
```

```
439
440
441 //
442 // Bitwise logic
443 //
```

Bitwise opcodes: OP_INVERT flips all bits; OP_AND/OP_OR/OP_XOR operate bitwise on equal-length byte vectors (MakeSameSize pads the shorter one with zeros). These are powerful for bitmask operations but were disabled along with OP_CAT.[3] Current Bitcoin scripts cannot use bitwise logic.

```
444 case OP_INVERT:
445 {
446 // (in - out)
447 if (stack.size() < 1)
448 return false;
449                 valtype& vch = stacktop(-1);
450 for (int i = 0; i < vch.size(); i++)
451                     vch[i] = ~vch[i];
452 }
453 break;
454
455 case OP_AND:
456 case OP_OR:
457 case OP_XOR:
458 {
459 // (x1 x2 - out)
460 if (stack.size() < 2)
461 return false;
462                 valtype& vch1 = stacktop(-2);
463                 valtype& vch2 = stacktop(-1);
464 MakeSameSize(vch1, vch2);
465 if (opcode == OP_AND)
466 {
467 for (int i = 0; i < vch1.size(); i++)
468                         vch1[i] &= vch2[i];
469 }
470 else if (opcode == OP_OR)
471 {
472 for (int i = 0; i < vch1.size(); i++)
473                         vch1[i] |= vch2[i];
474 }
475 else if (opcode == OP_XOR)
476 {
477 for (int i = 0; i < vch1.size(); i++)
478                         vch1[i] ^= vch2[i];
479 }
480                 stack.pop_back();
481 }
482 break;
483
```

OP_EQUAL compares two stack elements as byte arrays (exact byte-for-byte equality, not numeric). The comment about OP_NOTEQUAL explains why Satoshi left it out: numeric encoding is not unique (0×01 = 0×0001), so byte inequality doesn't imply numeric inequality. Numeric comparison uses OP_NUMNOTEQUAL instead.

```
484 case OP_EQUAL:
485 case OP_EQUALVERIFY:
486 //case OP_NOTEQUAL: // use OP_NUMNOTEQUAL
487 {
488 // (x1 x2 - bool)
489 if (stack.size() < 2)
490 return false;
491                 valtype& vch1 = stacktop(-2);
```

```
492                 valtype& vch2 = stacktop(-1);
493 bool fEqual = (vch1 == vch2);
494 // OP_NOTEQUAL is disabled because it would be too easy to say
495 // something like n != 1 and have some wiseguy pass in 1 with extra
496 // zero bytes after it (numerically, 0x01 == 0x0001 == 0x000001)
497 //if (opcode == OP_NOTEQUAL)
498 //    fEqual = !fEqual;
499                 stack.pop_back();
500                 stack.pop_back();
501                 stack.push_back(fEqual ? vchTrue : vchFalse);
502 if (opcode == OP_EQUALVERIFY)
503 {
504 if (fEqual)
505                         stack.pop_back();
506 else
507                         pc = pend;
508 }
509 }
510 break;
511
512
513 //
514 // Numeric
515 //
```

Unary numeric opcodes operate on a BigNum decoded from the top stack element. OP_2MUL and OP_2DIV are simple shifts (<<1, >>1). OP_NOT is logical — returns 1 if input is zero. All operate through OpenSSL's CBigNum which handles arbitrary precision; of these, only OP_2MUL and OP_2DIV were later disabled.

```
516 case OP_1ADD:
517 case OP_1SUB:
518 case OP_2MUL:
519 case OP_2DIV:
520 case OP_NEGATE:
521 case OP_ABS:
522 case OP_NOT:
523 case OP_0NOTEQUAL:
524 {
525 // (in -- out)
526 if (stack.size() < 1)
527 return false;
528                 CBigNum bn(stacktop(-1));
529 switch (opcode)
530 {
531 case OP_1ADD:       bn += bnOne; break;
532 case OP_1SUB:       bn -= bnOne; break;
533 case OP_2MUL:       bn <<= 1; break;
534 case OP_2DIV:       bn >>= 1; break;
535 case OP_NEGATE:     bn = -bn; break;
536 case OP_ABS: if (bn < bnZero) bn = -bn; break;
537 case OP_NOT:        bn = (bn == bnZero); break;
538 case OP_0NOTEQUAL:  bn = (bn != bnZero); break;
539 }
540                 stack.pop_back();
541                 stack.push_back(bn.getvch());
542 }
543 break;
544
```

Binary arithmetic opcodes: OP_MUL and OP_DIV use OpenSSL BN_mul/BN_div for arbitrary precision. OP_MOD uses BN_mod. These, along with OP_LSHIFT and OP_RSHIFT, were disabled in August 2010 because arbitrary-precision arithmetic on untrusted inputs is a DoS vector.[4][3] OP_LSHIFT was the worst offender: a crafted shift could crash nodes outright (CVE-2010-5137, fixed in July 2010).[2]

```
545 case OP_ADD:
546 case OP_SUB:
547 case OP_MUL:
548 case OP_DIV:
549 case OP_MOD:
550 case OP_LSHIFT:
551 case OP_RSHIFT:
552 case OP_BOOLAND:
553 case OP_BOOLOR:
554 case OP_NUMEQUAL:
555 case OP_NUMEQUALVERIFY:
556 case OP_NUMNOTEQUAL:
557 case OP_LESSTHAN:
558 case OP_GREATERTHAN:
559 case OP_LESSTHANOREQUAL:
560 case OP_GREATERTHANOREQUAL:
561 case OP_MIN:
562 case OP_MAX:
563 {
564 // (x1 x2 -- out)
565 if (stack.size() < 2)
566 return false;
567                 CBigNum bn1(stacktop(-2));
568                 CBigNum bn2(stacktop(-1));
569                 CBigNum bn;
570 switch (opcode)
571 {
572 case OP_ADD:
573                     bn = bn1 + bn2;
574 break;
575
576 case OP_SUB:
577                     bn = bn1 - bn2;
578 break;
579
580 case OP_MUL:
581 if (!BN_mul(&bn, &bn1, &bn2, pctx))
582 return false;
583 break;
584
585 case OP_DIV:
586 if (!BN_div(&bn, NULL, &bn1, &bn2, pctx))
587 return false;
588 break;
589
590 case OP_MOD:
591 if (!BN_mod(&bn, &bn1, &bn2, pctx))
592 return false;
593 break;
594
595 case OP_LSHIFT:
596 if (bn2 < bnZero)
597 return false;
598                     bn = bn1 << bn2.getulong();
599 break;
600
601 case OP_RSHIFT:
602 if (bn2 < bnZero)
603 return false;
604                     bn = bn1 >> bn2.getulong();
605 break;
```

```
606
607 case OP_BOOLAND:             bn = (bn1 != bnZero && bn2 != bnZero); break;
608 case OP_BOOLOR:              bn = (bn1 != bnZero || bn2 != bnZero); break;
609 case OP_NUMEQUAL:            bn = (bn1 == bn2); break;
610 case OP_NUMEQUALVERIFY:      bn = (bn1 == bn2); break;
611 case OP_NUMNOTEQUAL:         bn = (bn1 != bn2); break;
612 case OP_LESSTHAN:            bn = (bn1 < bn2); break;
613 case OP_GREATERTHAN:         bn = (bn1 > bn2); break;
614 case OP_LESSTHANOREQUAL:     bn = (bn1 <= bn2); break;
615 case OP_GREATERTHANOREQUAL:  bn = (bn1 >= bn2); break;
616 case OP_MIN:                 bn = (bn1 < bn2 ? bn1 : bn2); break;
617 case OP_MAX:                 bn = (bn1 > bn2 ? bn1 : bn2); break;
618 }
619                 stack.pop_back();
620                 stack.pop_back();
621                 stack.push_back(bn.getvch());
622
623 if (opcode == OP_NUMEQUALVERIFY)
624 {
625 if (CastToBool(stacktop(-1)))
626                         stack.pop_back();
627 else
628                         pc = pend;
629 }
630 }
631 break;
632
633 case OP_WITHIN:
634 {
635 // (x min max -- out)
636 if (stack.size() < 3)
637 return false;
638                 CBigNum bn1(stacktop(-3));
639                 CBigNum bn2(stacktop(-2));
640                 CBigNum bn3(stacktop(-1));
641 bool fValue = (bn2 <= bn1 && bn1 < bn3);
642                 stack.pop_back();
643                 stack.pop_back();
644                 stack.pop_back();
645                 stack.push_back(fValue ? vchTrue : vchFalse);
646 }
647 break;
648
649
650 //
651 // Crypto
652 //
```

Hash opcodes: five hash functions available in Script. OP_RIPEMD160 (20 bytes), OP_SHA1 (20 bytes), OP_SHA256 (32 bytes) are single-pass. OP_HASH160 = RIPEMD160(SHA256(x)) — this is the Bitcoin address hash. OP_HASH256 = SHA256(SHA256(x)) — the double-SHA256 used for block and transaction IDs. All implemented via OpenSSL.

```
653 case OP_RIPEMD160:
654 case OP_SHA1:
655 case OP_SHA256:
656 case OP_HASH160:
657 case OP_HASH256:
658 {
659 // (in -- hash)
660 if (stack.size() < 1)
661 return false;
662                 valtype& vch = stacktop(-1);
663                 valtype vchHash(opcode == OP_RIPEMD160 || opcode == OP_SHA1 || opcode == OP_HASH160 ? 20
    : 32);
664 if (opcode == OP_RIPEMD160)
665 RIPEMD160(&vch[0], vch.size(), &vchHash[0]);
666 else if (opcode == OP_SHA1)
```

```
667 SHA1(&vch[0], vch.size(), &vchHash[0]);
668 else if (opcode == OP_SHA256)
669 SHA256(&vch[0], vch.size(), &vchHash[0]);
670 else if (opcode == OP_HASH160)
671 {
672                     uint160 hash160 = Hash160(vch);
673 memcpy(&vchHash[0], &hash160, sizeof(hash160));
674 }
675 else if (opcode == OP_HASH256)
676 {
677                     uint256 hash = Hash(vch.begin(), vch.end());
678 memcpy(&vchHash[0], &hash, sizeof(hash));
679 }
680                 stack.pop_back();
681                 stack.push_back(vchHash);
682 }
683 break;
684
```

`OP_CODESEPARATOR` — marks the start of the "code" that is signed. SignatureHash (below) hashes only the script from the most recent OP_CODESEPARATOR to the end. This allows a script to split its signing scope, though in practice standard P2PKH scripts never use it. It also appears implicitly between scriptSig and scriptPubKey during verification.

```
685 case OP_CODESEPARATOR:
686 {
687 // Hash starts after the code separator
688                 pbegincodehash = pc;
689 }
690 break;
691
```

`OP_CHECKSIG` — the most important opcode in Bitcoin. Pops a public key and signature from the stack, calls CheckSig() which calls SignatureHash() and ECDSA verify. Pushes 1 if valid, 0 if not. Every standard P2PKH and P2PK transaction ends with this opcode. It is the lock that ensures only the key holder can spend; preventing the same coin from being spent twice is a separate job, done by the transaction index in main.cpp.

```
692 case OP_CHECKSIG:
693 case OP_CHECKSIGVERIFY:
694 {
695 // (sig pubkey -- bool)
696 if (stack.size() < 2)
697 return false;
698
699                 valtype& vchSig    = stacktop(-2);
700                 valtype& vchPubKey = stacktop(-1);
701
702 ////// debug print
703 //PrintHex(vchSig.begin(), vchSig.end(), "sig: %s\n");
704 //PrintHex(vchPubKey.begin(), vchPubKey.end(), "pubkey: %s\n");
705
706 // Subset of script starting at the most recent codeseparator
707                 CScript scriptCode(pbegincodehash, pend);
708
```

`scriptCode.FindAndDelete(CScript(vchSig))` removes the signature itself from the script before hashing. Without this, a signature would need to sign itself — which is impossible. This is why the "code" being signed can never contain the signature that will be checked against it.

```
709 // Drop the signature, since there's no way for a signature to sign itself
710                 scriptCode.FindAndDelete(CScript(vchSig));
```

```
711
712 bool fSuccess = CheckSig(vchSig, vchPubKey, scriptCode, txTo, nIn, nHashType);
713
714                 stack.pop_back();
715                 stack.pop_back();
716                 stack.push_back(fSuccess ? vchTrue : vchFalse);
717 if (opcode == OP_CHECKSIGVERIFY)
718 {
719 if (fSuccess)
720                         stack.pop_back();
721 else
722                         pc = pend;
723 }
724 }
725 break;
726
```

`OP_CHECKMULTISIG` — M-of-N multi-signature. Reads nKeysCount public keys, then nSigsCount signatures. Verifies each signature against keys in order; succeeds if at least nSigsCount signatures verify. One bug: consumes an extra stack element (a historical accident that required a dummy OP_0 push — this off-by-one was never fixed for compatibility reasons).

```
727 case OP_CHECKMULTISIG:
728 case OP_CHECKMULTISIGVERIFY:
729 {
730 // ([sig ...] num_of_signatures [pubkey ...] num_of_pubkeys -- bool)
731
732 int i = 1;
733 if (stack.size() < i)
734 return false;
735
736 int nKeysCount = CBigNum(stacktop(-i)).getint();
737 if (nKeysCount < 0)
738 return false;
739 int ikey = ++i;
740                 i += nKeysCount;
741 if (stack.size() < i)
742 return false;
743
744 int nSigsCount = CBigNum(stacktop(-i)).getint();
745 if (nSigsCount < 0 || nSigsCount > nKeysCount)
746 return false;
747 int isig = ++i;
748                 i += nSigsCount;
749 if (stack.size() < i)
750 return false;
751
752 // Subset of script starting at the most recent codeseparator
753                 CScript scriptCode(pbegincodehash, pend);
754
755 // Drop the signatures, since there's no way for a signature to sign itself
756 for (int i = 0; i < nSigsCount; i++)
757 {
758                     valtype& vchSig = stacktop(-isig-i);
759                     scriptCode.FindAndDelete(CScript(vchSig));
760 }
761
762 bool fSuccess = true;
763 while (fSuccess && nSigsCount > 0)
764 {
765                     valtype& vchSig    = stacktop(-isig);
766                     valtype& vchPubKey = stacktop(-ikey);
767
768 // Check signature
769 if (CheckSig(vchSig, vchPubKey, scriptCode, txTo, nIn, nHashType))
770 {
771                         isig++;
772                         nSigsCount--;
```

```
}
                    ikey++;
                    nKeysCount--;

// If there are more signatures left than keys left,
// then too many signatures have failed
if (nSigsCount > nKeysCount)
                        fSuccess = false;
}

while (i-- > 0)
                    stack.pop_back();
                stack.push_back(fSuccess ? vchTrue : vchFalse);

if (opcode == OP_CHECKMULTISIGVERIFY)
{
if (fSuccess)
                        stack.pop_back();
else
                        pc = pend;
}
}
break;

default:
return false;
}
}
```

Final result: execution succeeds if the stack is non-empty and the top element is truthy (CastToBool). An empty stack is failure. If pvStackRet is provided, the full stack is returned — used by the wallet to inspect script results without re-running the interpreter.

```
if (pvStackRet)
*pvStackRet = stack;
return (stack.empty() ? false : CastToBool(stack.back()));
}

#undef top
```

`SignatureHash` — constructs the exact bytes that get signed. Creates a modified copy of the transaction, clears other inputs' signatures, then hashes txTmp plus the hash type using double-SHA256. The result is what the private key signs and OP_CHECKSIG verifies. Getting this wrong in any implementation breaks cross-client compatibility.

```
CALLED FROM  CheckSig p. 398 · SignSignature p. 402
uint256 SignatureHash(CScript scriptCode, const CTransaction& txTo, unsigned int nIn, int nHashType)
{
if (nIn >= txTo.vin.size())
{
printf("ERROR: SignatureHash() : nIn=%d out of range\n", nIn);
return 1;
}
    CTransaction txTmp(txTo);
```

```
826
827 // In case concatenating two scripts ends up with two codeseparators,
828 // or an extra one at the end, this prevents all those possible incompatibilities.
829     scriptCode.FindAndDelete(CScript(OP_CODESEPARATOR));
830
831 // Blank out other inputs' signatures
```

SIGHASH_ALL (default): signs all inputs and all outputs — the complete transaction is committed to. No one can alter the recipients, amounts, or other inputs without invalidating the signature.

```
832 for (int i = 0; i < txTmp.vin.size(); i++)
833         txTmp.vin[i].scriptSig = CScript();
834     txTmp.vin[nIn].scriptSig = scriptCode;
835
836 // Blank out some of the outputs
```

SIGHASH_NONE: signs inputs but clears all outputs — the signer authorizes spending without specifying where coins go. Anyone can modify outputs after signing. Intended for "blank check" scenarios; rarely used in practice.

```
837 if ((nHashType & 0x1f) == SIGHASH_NONE)
838 {
839 // Wildcard payee
840         txTmp.vout.clear();
841
842 // Let the others update at will
843 for (int i = 0; i < txTmp.vin.size(); i++)
844 if (i != nIn)
845                 txTmp.vin[i].nSequence = 0;
846 }
```

SIGHASH_SINGLE: locks only the output at the same index as the signed input. Other outputs can change. Useful for partially-signed transactions where parties contribute independently. The edge case where nOut >= vout.size() is a known bug that returns hash=1 rather than failing — discovered years later as the "SIGHASH_SINGLE bug".

```
847 else if ((nHashType & 0x1f) == SIGHASH_SINGLE)
848 {
849 // Only lockin the txout payee at same index as txin
850 unsigned int nOut = nIn;
851 if (nOut >= txTmp.vout.size())
852 {
853 printf("ERROR: SignatureHash() : nOut=%d out of range\n", nOut);
854 return 1;
855 }
856         txTmp.vout.resize(nOut+1);
857 for (int i = 0; i < nOut; i++)
858             txTmp.vout[i].SetNull();
859
860 // Let the others update at will
861 for (int i = 0; i < txTmp.vin.size(); i++)
862 if (i != nIn)
863                 txTmp.vin[i].nSequence = 0;
864 }
865
866 // Blank out other inputs completely, not recommended for open transactions
```

SIGHASH_ANYONECANPAY: combined with any of the above, strips all inputs except the one being signed. Other signers can add their own inputs later. Enables crowdfunding-style transactions where contributions can be added after initial signatures.

```
867 if (nHashType & SIGHASH_ANYONECANPAY)
868 {
869         txTmp.vin[0] = txTmp.vin[nIn];
870         txTmp.vin.resize(1);
871 }
872
873 // Serialize and hash
874     CDataStream ss(SER_GETHASH);
875     ss.reserve(10000);
876     ss << txTmp << nHashType;
877 return Hash(ss.begin(), ss.end());
878 }
879
880
```

CheckSig — the security gateway. Loads the public key, extracts the hash type from the last byte of the signature (or uses the provided nHashType if specified), strips the hash type byte, then calls key.Verify() which performs the ECDSA signature verification via OpenSSL. Returns false on any error.

```
    CALLED FROM  EvalScript p. 395
881 bool CheckSig(vector<unsigned char> vchSig, vector<unsigned char> vchPubKey, CScript scriptCode,
882 const CTransaction& txTo, unsigned int nIn, int nHashType)
883 {
884     CKey key;
885 if (!key.SetPubKey(vchPubKey))
886 return false;
887
888 // Hash type is one byte tacked on to the end of the signature
889 if (vchSig.empty())
890 return false;
891 if (nHashType == 0)
892         nHashType = vchSig.back();
893 else if (nHashType != vchSig.back())
894 return false;
895     vchSig.pop_back();
896
897 if (key.Verify(SignatureHash(scriptCode, txTo, nIn, nHashType), vchSig))
898 return true;
899
900 return false;
901 }
902
903
904
905
906
907
908
909
910
911
912
```

`Solver` (pattern matcher) — identifies the type of a scriptPubKey by comparing against known templates. Two templates recognized in v0.1.0: P2PK (direct pubkey + CHECKSIG) and P2PKH (DUP HASH160 pubkeyhash EQUALVERIFY CHECKSIG). Returns the extracted placeholder values (public key or hash).

```
913 bool Solver(const CScript& scriptPubKey, vector<pair<opcodetype, valtype> >& vSolutionRet)
914 {
915 // Templates
916 static vector<CScript> vTemplates;
917 if (vTemplates.empty())
918 {
919 // Standard tx, sender provides pubkey, receiver adds signature
920         vTemplates.push_back(CScript() << OP_PUBKEY << OP_CHECKSIG);
921
922 // Short account number tx, sender provides hash of pubkey, receiver provides signature and pubkey
923         vTemplates.push_back(CScript() << OP_DUP << OP_HASH160 << OP_PUBKEYHASH << OP_EQUALVERIFY <<
    OP_CHECKSIG);
924 }
925
926 // Scan templates
927 const CScript& script1 = scriptPubKey;
928 foreach(const CScript& script2, vTemplates)
929 {
930         vSolutionRet.clear();
931         opcodetype opcode1, opcode2;
932         vector<unsigned char> vch1, vch2;
933
934 // Compare
935         CScript::const_iterator pc1 = script1.begin();
936         CScript::const_iterator pc2 = script2.begin();
937         loop
938 {
939 bool f1 = script1.GetOp(pc1, opcode1, vch1);
940 bool f2 = script2.GetOp(pc2, opcode2, vch2);
941 if (!f1 && !f2)
942 {
943 // Success
944 reverse(vSolutionRet.begin(), vSolutionRet.end());
945 return true;
946 }
947 else if (f1 != f2)
948 {
949 break;
950 }
951 else if (opcode2 == OP_PUBKEY)
952 {
953 if (vch1.size() <= sizeof(uint256))
954 break;
955                 vSolutionRet.push_back(make_pair(opcode2, vch1));
956 }
957 else if (opcode2 == OP_PUBKEYHASH)
958 {
959 if (vch1.size() != sizeof(uint160))
960 break;
961                 vSolutionRet.push_back(make_pair(opcode2, vch1));
962 }
963 else if (opcode1 != opcode2)
964 {
965 break;
966 }
967 }
968 }
969
970     vSolutionRet.clear();
971 return false;
972 }
973
```

974

`Solver` (signing overload) — given a scriptPubKey and a hash to sign, uses pattern matching to build the scriptSig. For P2PK: signs with the private key for that pubkey. For P2PKH: looks up the pubkey from mapPubKeys by hash, then signs. Appends the nHashType byte to each signature.

```
975 bool Solver(const CScript& scriptPubKey, uint256 hash, int nHashType, CScript& scriptSigRet)
976 {
977     scriptSigRet.clear();
978
979     vector<pair<opcodetype, valtype> > vSolution;
980 if (!Solver(scriptPubKey, vSolution))
981 return false;
982
983 // Compile solution
984 CRITICAL_BLOCK(cs_mapKeys)
985 {
986 foreach(PAIRTYPE(opcodetype, valtype)& item, vSolution)
987 {
988 if (item.first == OP_PUBKEY)
989 {
990 // Sign
991 const valtype& vchPubKey = item.second;
992 if (!mapKeys.count(vchPubKey))
993 return false;
994 if (hash != 0)
995 {
996                     vector<unsigned char> vchSig;
997 if (!CKey::Sign(mapKeys[vchPubKey], hash, vchSig))
998 return false;
999                     vchSig.push_back((unsigned char)nHashType);
1000                    scriptSigRet << vchSig;
1001 }
1002 }
1003 else if (item.first == OP_PUBKEYHASH)
1004 {
1005 // Sign and give pubkey
1006                map<uint160, valtype>::iterator mi = mapPubKeys.find(uint160(item.second));
1007 if (mi == mapPubKeys.end())
1008 return false;
1009 const vector<unsigned char>& vchPubKey = (*mi).second;
1010 if (!mapKeys.count(vchPubKey))
1011 return false;
1012 if (hash != 0)
1013 {
1014                     vector<unsigned char> vchSig;
1015 if (!CKey::Sign(mapKeys[vchPubKey], hash, vchSig))
1016 return false;
1017                     vchSig.push_back((unsigned char)nHashType);
1018                     scriptSigRet << vchSig << vchPubKey;
1019 }
1020 }
1021 }
1022 }
1023
1024 return true;
1025 }
1026
1027
```

`IsMine` — the wallet ownership check. Calls Solver with hash=0 ("just match the pattern, don't actually sign"). Returns true if this node's wallet holds the private key needed to satisfy the scriptPubKey. Used pervasively to filter incoming transactions for wallet balance.

```
1028 bool IsMine(const CScript& scriptPubKey)
1029 {
```

```
    CScript scriptSig;
return Solver(scriptPubKey, 0, 0, scriptSig);
}


```

CALLED FROM CreateTransaction main.cpp p. 247 · CMainFrame::InsertTransaction ui.cpp p. 484 · CTxDetailsDialog::CTxDetailsDialog ui.cpp p. 492

```
bool ExtractPubKey(const CScript& scriptPubKey, bool fMineOnly, vector<unsigned char>& vchPubKeyRet)
{
    vchPubKeyRet.clear();

    vector<pair<opcodetype, valtype> > vSolution;
if (!Solver(scriptPubKey, vSolution))
return false;

CRITICAL_BLOCK(cs_mapKeys)
{
foreach(PAIRTYPE(opcodetype, valtype)& item, vSolution)
{
            valtype vchPubKey;
if (item.first == OP_PUBKEY)
{
                vchPubKey = item.second;
}
else if (item.first == OP_PUBKEYHASH)
{
                map<uint160, valtype>::iterator mi = mapPubKeys.find(uint160(item.second));
if (mi == mapPubKeys.end())
continue;
                vchPubKey = (*mi).second;
}
if (!fMineOnly || mapKeys.count(vchPubKey))
{
                vchPubKeyRet = vchPubKey;
return true;
}
}
}
return false;
}


```

CALLED FROM CMainFrame::InsertTransaction ui.cpp p. 485 · CTxDetailsDialog::CTxDetailsDialog ui.cpp p. 493

```
bool ExtractHash160(const CScript& scriptPubKey, uint160& hash160Ret)
{
    hash160Ret = 0;

    vector<pair<opcodetype, valtype> > vSolution;
if (!Solver(scriptPubKey, vSolution))
return false;

foreach(PAIRTYPE(opcodetype, valtype)& item, vSolution)
{
if (item.first == OP_PUBKEYHASH)
{
            hash160Ret = uint160(item.second);
return true;
}
}
return false;
}


```

`SignSignature` — produces a complete signed input for a transaction. Calls SignatureHash on the output being spent, calls Solver to build the scriptSig, then verifies the result by re-running EvalScript. The self-test at line 1108 catches signing errors before they propagate.

```
    CALLED FROM  CreateTransaction main.cpp p. 248
1090 bool SignSignature(const CTransaction& txFrom, CTransaction& txTo, unsigned int nIn, int nHashType,
    CScript scriptPrereq)
1091 {
1092 assert(nIn < txTo.vin.size());
1093     CTxIn& txin = txTo.vin[nIn];
1094 assert(txin.prevout.n < txFrom.vout.size());
1095 const CTxOut& txout = txFrom.vout[txin.prevout.n];
1096
1097 // Leave out the signature from the hash, since a signature can't sign itself.
1098 // The checksig op will also drop the signatures from its hash.
1099     uint256 hash = SignatureHash(scriptPrereq + txout.scriptPubKey, txTo, nIn, nHashType);
1100
1101 if (!Solver(txout.scriptPubKey, hash, nHashType, txin.scriptSig))
1102 return false;
1103
1104     txin.scriptSig = scriptPrereq + txin.scriptSig;
1105
1106 // Test solution
1107 if (scriptPrereq.empty())
1108 if (!EvalScript(txin.scriptSig + CScript(OP_CODESEPARATOR) + txout.scriptPubKey, txTo, nIn))
1109 return false;
1110
1111 return true;
1112 }
1113
1114
```

`VerifySignature` — the full validation path for a transaction input. Concatenates scriptSig + OP_CODESEPARATOR + scriptPubKey into a single script, then calls EvalScript. This is how every transaction in a block is validated during ConnectInputs in main.cpp.

```
    CALLED FROM  CTransaction::ConnectInputs main.cpp p. 216 ·
    CTransaction::ClientConnectInputs main.cpp p. 217
1115 bool VerifySignature(const CTransaction& txFrom, const CTransaction& txTo, unsigned int nIn, int
    nHashType)
1116 {
1117 assert(nIn < txTo.vin.size());
1118 const CTxIn& txin = txTo.vin[nIn];
1119 if (txin.prevout.n >= txFrom.vout.size())
1120 return false;
1121 const CTxOut& txout = txFrom.vout[txin.prevout.n];
1122
1123 if (txin.prevout.hash != txFrom.GetHash())
1124 return false;
1125
```

This concatenation is v0.1.0's deepest bug. Because scriptSig and scriptPubKey run as one program, anything the spender's half does can short-circuit the owner's lock: a scriptSig of `OP_1 OP_RETURN` jumps to the end (line 172), never reaching the scriptPubKey, and leaves true on the stack. Anyone could have spent anyone's coins. Satoshi made OP_RETURN fail in v0.3.5 (July 2010),[2] and later versions evaluate the two scripts separately.

```
1126 return EvalScript(txin.scriptSig + CScript(OP_CODESEPARATOR) + txout.scriptPubKey, txTo, nIn,
    nHashType);
1127 }
1128
```

SUMMARY

The standard Pay-to-Public-Key-Hash (P2PKH) script executes as: `OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG`

1. OP_DUP: Duplicate the provided public key
2. OP_HASH160: Hash it to get the address
3. OP_EQUALVERIFY: Compare to the expected address, fail if mismatch
4. OP_CHECKSIG: Verify the signature proves ownership of that public key

This elegance—proving ownership in four operations—is why Satoshi's scripting design has survived seventeen years of attacks, over a trillion dollars in value secured, and multiple generations of protocol upgrades without replacement. Every complex Bitcoin feature built since—multisig wallets, payment channels, Lightning Network, Taproot—is ultimately an extension of these same opcodes operating on this same stack machine.[1]

KEY TERMS

opcode

A single-byte instruction such as OP_DUP or OP_CHECKSIG that the interpreter executes one after another.

stack

The last-in-first-out list of byte arrays scripts operate on: pushes add data, and each opcode pops its arguments and pushes its result.

reverse Polish notation

The Forth-style syntax Script uses, where operators follow their operands (3 4 + instead of 3 + 4), so no parsing or precedence rules are needed.

EvalScript

The interpreter function that runs a whole script and returns true only if execution finishes with a truthy value on top of the stack.

OP_CHECKSIG

The opcode that pops a public key and a signature and verifies them with ECDSA, pushing 1 on success; the lock on virtually every standard payment.

sighash

The flag byte appended to a signature (SIGHASH_ALL, NONE, or SINGLE, optionally with ANYONECANPAY) that selects which inputs and outputs the signature commits to.

P2PKH

Pay-to-public-key-hash, the standard script pattern OP_DUP OP_HASH160 hash OP_EQUALVERIFY OP_CHECKSIG that pays whoever can sign with the key behind an address.

OP_CHECKMULTISIG

The opcode implementing M-of-N multi-signature, succeeding when at least M of the supplied signatures verify against the N listed public keys.

alt stack

A secondary stack on which a script can park a value with OP_TOALTSTACK and retrieve it later with OP_FROMALTSTACK.

SOURCES

1. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/

2. Bitcoin Wiki, *Common Vulnerabilities and Exposures — CVE-2010-5141 (OP_RETURN) and CVE-2010-5137* (July 28, 2010). Announced 2010-07-28; fixed in Bitcoin 0.3.5. https://en.bitcoin.it/wiki/Common_Vulnerabilities_and_Exposures
3. Satoshi Nakamoto / Bitcoin developers, *OP_RETURN and dangerous opcodes disabled* (July 2010). Response to early Script vulnerabilities. https://en.bitcoin.it/wiki/Script#Disabled_opcodes
4. Satoshi Nakamoto, *bitcoin/bitcoin commit 4bd188c — "misc changes" (disables 15 opcodes, 520-byte push limit)* (15 August 2010). https://github.com/bitcoin/bitcoin/commit/4bd188c4383d6e614e18f79dc337fbabe8464c82

script.h

598 lines CPP

PROGRAMMABLE MONEY

Bitcoin is not merely a ledger—it includes a complete programming language. This header defines that language: Script, a Forth-like stack-based system for expressing arbitrary spending conditions. When Satoshi designed this, he invented what we now call "programmable money"—years before Ethereum popularized the term "smart contracts."

Satoshi explained on BitcoinTalk: "The design supports a tremendous variety of possible transaction types that I designed years ago. Escrow transactions, bonded contracts, third party arbitration, multi-party signature, etc. If Bitcoin catches on in a big way, these are things we'll want to explore in the future, but they all had to be designed at the beginning to make sure they would be possible later."[1] This prescient design meant Bitcoin could evolve without hard forks—just creative use of existing opcodes.

Critically, Script is **intentionally NOT Turing-complete**. There are no loops, no function calls, no unbounded recursion. Why? Because anyone can create a script that every full node must execute. If Script allowed infinite loops, a malicious script could hang every node on the network—a trivial denial-of-service attack. Additionally, Turing-complete contracts are difficult to analyze for bugs; Ethereum's DAO hack demonstrated this danger when $50 million was stolen through an unintended behavior. Satoshi's conservative design traded expressiveness for security.

The opcodes defined here represent Satoshi's careful balance between power and safety. Each one was included for a reason—and many were later disabled when vulnerabilities emerged.[2] The ghost opcodes (OP_CAT, OP_MUL, etc.) remain in the enum as reminders of what not to enable carelessly.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

class CTransaction;

enum
{
```

SIGHASH_* flags — what part of the transaction a signature commits to. ALL (default) signs every input and output; NONE/SINGLE leave room for others to amend outputs; ANYONECANPAY lets others add inputs. These are how multi-party protocols compose without a single signer controlling the whole tx.[1]

```
    SIGHASH_ALL = 1,
    SIGHASH_NONE = 2,
    SIGHASH_SINGLE = 3,
    SIGHASH_ANYONECANPAY = 0x80,
};
```

opcodetype enum — the complete instruction set (one byte each, with a two-byte escape noted below). Many codes are reserved or later disabled; in August 2010 Satoshi turned off OP_CAT, OP_MUL, OP_DIV and others after DoS bugs.[2][3] The names remain as 'ghost opcodes.'

```
enum opcodetype
{
// push value
    OP_0=0,
    OP_FALSE=OP_0,
    OP_PUSHDATA1=76,
    OP_PUSHDATA2,
    OP_PUSHDATA4,
    OP_1NEGATE,
    OP_RESERVED,
    OP_1,
    OP_TRUE=OP_1,
    OP_2,
    OP_3,
    OP_4,
    OP_5,
    OP_6,
    OP_7,
    OP_8,
    OP_9,
    OP_10,
    OP_11,
    OP_12,
    OP_13,
    OP_14,
    OP_15,
    OP_16,

// control
    OP_NOP,
    OP_VER,
```

OP_IF / OP_ELSE / OP_ENDIF — conditional execution. Enables scripts like "pay Alice OR (pay Bob after timeout)" — the building block for escrow and later Lightning-style contracts.[1]

```
    OP_IF,
```

```
49     OP_NOTIF,
50     OP_VERIF,
51     OP_VERNOTIF,
52     OP_ELSE,
53     OP_ENDIF,
54     OP_VERIFY,
```

`OP_RETURN` in v0.1.0 simply jumps to the end of the script—it does NOT fail. Because scriptSig and scriptPubKey were run as one concatenated script (script.cpp:1126), a scriptSig of `OP_1 OP_RETURN` skipped the owner's lock entirely and left true on the stack: anyone could spend anyone's coins. Satoshi fixed this in v0.3.5 (July 2010) by making OP_RETURN fail outright.[4] That fix later made OP_RETURN outputs a clean, provably unspendable place to embed data without bloating the UTXO set: standard at 40 bytes in 2014 (later 80), as policy, not consensus.[5] Bitcoin Core 30 (2025) lifted the default cap,[6] reigniting an unresolved tension in Bitcoin's identity: is it purely a monetary system, or also a data layer? The question isn't technical but philosophical—what should Bitcoin be?

```
55     OP_RETURN,
56
57 // stack ops
58     OP_TOALTSTACK,
59     OP_FROMALTSTACK,
60     OP_2DROP,
61     OP_2DUP,
62     OP_3DUP,
63     OP_2OVER,
64     OP_2ROT,
65     OP_2SWAP,
66     OP_IFDUP,
67     OP_DEPTH,
68     OP_DROP,
```

`OP_DUP` — duplicate top stack item. Combined with OP_HASH160 and OP_EQUALVERIFY, this forms P2PKH (Pay to Public Key Hash), Bitcoin's standard script.

```
69     OP_DUP,
70     OP_NIP,
71     OP_OVER,
72     OP_PICK,
73     OP_ROLL,
74     OP_ROT,
75     OP_SWAP,
76     OP_TUCK,
77
78 // splice ops
79     OP_CAT,
80     OP_SUBSTR,
81     OP_LEFT,
82     OP_RIGHT,
83     OP_SIZE,
84
85 // bit logic
86     OP_INVERT,
87     OP_AND,
88     OP_OR,
89     OP_XOR,
90     OP_EQUAL,
91     OP_EQUALVERIFY,
92     OP_RESERVED1,
93     OP_RESERVED2,
94
95 // numeric
96     OP_1ADD,
97     OP_1SUB,
```

```
 98     OP_2MUL,
 99     OP_2DIV,
100     OP_NEGATE,
101     OP_ABS,
102     OP_NOT,
103     OP_0NOTEQUAL,
104
105     OP_ADD,
106     OP_SUB,
107     OP_MUL,
108     OP_DIV,
109     OP_MOD,
110     OP_LSHIFT,
111     OP_RSHIFT,
112
113     OP_BOOLAND,
114     OP_BOOLOR,
115     OP_NUMEQUAL,
116     OP_NUMEQUALVERIFY,
117     OP_NUMNOTEQUAL,
118     OP_LESSTHAN,
119     OP_GREATERTHAN,
120     OP_LESSTHANOREQUAL,
121     OP_GREATERTHANOREQUAL,
122     OP_MIN,
123     OP_MAX,
124
125     OP_WITHIN,
126
127 // crypto
128     OP_RIPEMD160,
129     OP_SHA1,
130     OP_SHA256,
```

`OP_HASH160` — RIPEMD160(SHA256(x)). Creates the 20-byte hash used in Bitcoin addresses. Double hashing adds security margin.

```
131     OP_HASH160,
```

`OP_HASH256` — SHA256(SHA256(x)). Double SHA256 used for block hashes and txids. Defense against length extension attacks.

```
132     OP_HASH256,
133     OP_CODESEPARATOR,
```

`OP_CHECKSIG` — the signature verification opcode. This single instruction is why Bitcoin works: proves private key ownership.

```
134     OP_CHECKSIG,
```

`OP_CHECKSIGVERIFY` — CHECKSIG + VERIFY. Fails script immediately if signature invalid. Used in multi-sig scripts.

```
135     OP_CHECKSIGVERIFY,
```

`OP_CHECKMULTISIG` — M-of-N multi-signature verification, present from day one. Requires M valid signatures from a set of N public keys. Enables shared custody: a company treasury might require 2-of-3 executives to sign, or an escrow might need 2-of-3 (buyer, seller, arbitrator). Satoshi mentioned 'multi-party signature' in his BitcoinTalk posts as a planned feature.[1] The implementation (script.cpp:727) iterates through signatures and keys, matching each signature to a key in order. A subtle bug existed: an extra unused value was consumed from the stack, constrained years later in BIP 147.

```
136     OP_CHECKMULTISIG,
137     OP_CHECKMULTISIGVERIFY,
138
139
140 // multi-byte opcodes
```

`OP_SINGLEBYTE_END = 0xF0` — Satoshi reserved an escape hatch: in v0.1.0 any byte ≥ 0xF0 begins a two-byte opcode (see `GetOp`, line 491), leaving room for thousands of future instructions. The wallet's template placeholders `OP_PUBKEY` and `OP_PUBKEYHASH` live up there, beyond anything a real script could contain. The double-byte scheme was later removed.

```
141     OP_SINGLEBYTE_END = 0xF0,
142     OP_DOUBLEBYTE_BEGIN = 0xF000,
143
144 // template matching params
145     OP_PUBKEY,
146     OP_PUBKEYHASH,
147
148
149
150     OP_INVALIDOPCODE = 0xFFFF,
151 };
152
153
154
155
156
157
158
159
    CALLED FROM  CScript (inline) p. 417
160 inline const char* GetOpName(opcodetype opcode)
161 {
162 switch (opcode)
163 {
164 // push value
165 case OP_0                       : return "0";
166 case OP_PUSHDATA1               : return "OP_PUSHDATA1";
167 case OP_PUSHDATA2               : return "OP_PUSHDATA2";
168 case OP_PUSHDATA4               : return "OP_PUSHDATA4";
169 case OP_1NEGATE                 : return "-1";
170 case OP_RESERVED                : return "OP_RESERVED";
171 case OP_1                       : return "1";
172 case OP_2                       : return "2";
173 case OP_3                       : return "3";
174 case OP_4                       : return "4";
175 case OP_5                       : return "5";
176 case OP_6                       : return "6";
177 case OP_7                       : return "7";
178 case OP_8                       : return "8";
179 case OP_9                       : return "9";
180 case OP_10                      : return "10";
181 case OP_11                      : return "11";
182 case OP_12                      : return "12";
183 case OP_13                      : return "13";
```

```
184 case OP_14                     : return "14";
185 case OP_15                     : return "15";
186 case OP_16                     : return "16";
187
188 // control
189 case OP_NOP                    : return "OP_NOP";
190 case OP_VER                    : return "OP_VER";
191 case OP_IF                     : return "OP_IF";
192 case OP_NOTIF                  : return "OP_NOTIF";
193 case OP_VERIF                  : return "OP_VERIF";
194 case OP_VERNOTIF               : return "OP_VERNOTIF";
195 case OP_ELSE                   : return "OP_ELSE";
196 case OP_ENDIF                  : return "OP_ENDIF";
197 case OP_VERIFY                 : return "OP_VERIFY";
198 case OP_RETURN                 : return "OP_RETURN";
199
200 // stack ops
201 case OP_TOALTSTACK             : return "OP_TOALTSTACK";
202 case OP_FROMALTSTACK           : return "OP_FROMALTSTACK";
203 case OP_2DROP                  : return "OP_2DROP";
204 case OP_2DUP                   : return "OP_2DUP";
205 case OP_3DUP                   : return "OP_3DUP";
206 case OP_2OVER                  : return "OP_2OVER";
207 case OP_2ROT                   : return "OP_2ROT";
208 case OP_2SWAP                  : return "OP_2SWAP";
209 case OP_IFDUP                  : return "OP_IFDUP";
210 case OP_DEPTH                  : return "OP_DEPTH";
211 case OP_DROP                   : return "OP_DROP";
212 case OP_DUP                    : return "OP_DUP";
213 case OP_NIP                    : return "OP_NIP";
214 case OP_OVER                   : return "OP_OVER";
215 case OP_PICK                   : return "OP_PICK";
216 case OP_ROLL                   : return "OP_ROLL";
217 case OP_ROT                    : return "OP_ROT";
218 case OP_SWAP                   : return "OP_SWAP";
219 case OP_TUCK                   : return "OP_TUCK";
220
221 // splice ops
222 case OP_CAT                    : return "OP_CAT";
223 case OP_SUBSTR                 : return "OP_SUBSTR";
224 case OP_LEFT                   : return "OP_LEFT";
225 case OP_RIGHT                  : return "OP_RIGHT";
226 case OP_SIZE                   : return "OP_SIZE";
227
228 // bit logic
229 case OP_INVERT                 : return "OP_INVERT";
230 case OP_AND                    : return "OP_AND";
231 case OP_OR                     : return "OP_OR";
232 case OP_XOR                    : return "OP_XOR";
233 case OP_EQUAL                  : return "OP_EQUAL";
234 case OP_EQUALVERIFY            : return "OP_EQUALVERIFY";
235 case OP_RESERVED1              : return "OP_RESERVED1";
236 case OP_RESERVED2              : return "OP_RESERVED2";
237
238 // numeric
239 case OP_1ADD                   : return "OP_1ADD";
240 case OP_1SUB                   : return "OP_1SUB";
241 case OP_2MUL                   : return "OP_2MUL";
242 case OP_2DIV                   : return "OP_2DIV";
243 case OP_NEGATE                 : return "OP_NEGATE";
244 case OP_ABS                    : return "OP_ABS";
245 case OP_NOT                    : return "OP_NOT";
246 case OP_0NOTEQUAL              : return "OP_0NOTEQUAL";
247 case OP_ADD                    : return "OP_ADD";
248 case OP_SUB                    : return "OP_SUB";
249 case OP_MUL                    : return "OP_MUL";
250 case OP_DIV                    : return "OP_DIV";
251 case OP_MOD                    : return "OP_MOD";
252 case OP_LSHIFT                 : return "OP_LSHIFT";
253 case OP_RSHIFT                 : return "OP_RSHIFT";
254 case OP_BOOLAND                : return "OP_BOOLAND";
255 case OP_BOOLOR                 : return "OP_BOOLOR";
```

```
case OP_NUMEQUAL               : return "OP_NUMEQUAL";
case OP_NUMEQUALVERIFY         : return "OP_NUMEQUALVERIFY";
case OP_NUMNOTEQUAL            : return "OP_NUMNOTEQUAL";
case OP_LESSTHAN               : return "OP_LESSTHAN";
case OP_GREATERTHAN            : return "OP_GREATERTHAN";
case OP_LESSTHANOREQUAL        : return "OP_LESSTHANOREQUAL";
case OP_GREATERTHANOREQUAL     : return "OP_GREATERTHANOREQUAL";
case OP_MIN                    : return "OP_MIN";
case OP_MAX                    : return "OP_MAX";
case OP_WITHIN                 : return "OP_WITHIN";

// crypto
case OP_RIPEMD160              : return "OP_RIPEMD160";
case OP_SHA1                   : return "OP_SHA1";
case OP_SHA256                 : return "OP_SHA256";
case OP_HASH160                : return "OP_HASH160";
case OP_HASH256                : return "OP_HASH256";
case OP_CODESEPARATOR          : return "OP_CODESEPARATOR";
case OP_CHECKSIG               : return "OP_CHECKSIG";
case OP_CHECKSIGVERIFY         : return "OP_CHECKSIGVERIFY";
case OP_CHECKMULTISIG          : return "OP_CHECKMULTISIG";
case OP_CHECKMULTISIGVERIFY    : return "OP_CHECKMULTISIGVERIFY";



// multi-byte opcodes
case OP_SINGLEBYTE_END         : return "OP_SINGLEBYTE_END";
case OP_DOUBLEBYTE_BEGIN       : return "OP_DOUBLEBYTE_BEGIN";
case OP_PUBKEY                 : return "OP_PUBKEY";
case OP_PUBKEYHASH             : return "OP_PUBKEYHASH";



case OP_INVALIDOPCODE          : return "OP_INVALIDOPCODE";
default:
return "UNKNOWN_OPCODE";
}
};




```

CALLED FROM StackString p. 413 · CScript (inline) p. 417

```
inline string ValueString(const vector<unsigned char>& vch)
{
if (vch.size() <= 4)
return strprintf("%d", CBigNum(vch).getint());
else
return HexNumStr(vch.begin(), vch.end());
//return string("(") + HexStr(vch.begin(), vch.end()) + string(")");
}

inline string StackString(const vector<vector<unsigned char> >& vStack)
{
    string str;
foreach(const vector<unsigned char>& vch, vStack)
{
if (!str.empty())
            str += " ";
        str += ValueString(vch);
}
return str;
}









```

326

`CScript` class — a Bitcoin script as bytes. Inherits from vector<unsigned char>. Scripts are data until evaluated.

```
327 class CScript : public vector<unsigned char>
328 {
329 protected:
330     CScript& push_int64(int64 n)
331 {
332 if (n == -1 || (n >= 1 && n <= 16))
333 {
334 push_back(n + (OP_1 - 1));
335 }
336 else
337 {
338             CBigNum bn(n);
339 *this << bn.getvch();
340 }
341 return (*this);
342 }
343
344     CScript& push_uint64(uint64 n)
345 {
346 if (n == -1 || (n >= 1 && n <= 16))
347 {
348 push_back(n + (OP_1 - 1));
349 }
350 else
351 {
352             CBigNum bn(n);
353 *this << bn.getvch();
354 }
355 return (*this);
356 }
357
358 public:
359 CScript() { }
360 CScript(const CScript& b) : vector<unsigned char>(b.begin(), b.end()) { }
361 CScript(const_iterator pbegin, const_iterator pend) : vector<unsigned char>(pbegin, pend) { }
362 #ifndef _MSC_VER
363 CScript(const unsigned char* pbegin, const unsigned char* pend) : vector<unsigned char>(pbegin, pend) {
    }
364 #endif
365
366     CScript& operator+=(const CScript& b)
367 {
368 insert(end(), b.begin(), b.end());
369 return *this;
370 }
371
372 friend CScript operator+(const CScript& a, const CScript& b)
373 {
374         CScript ret = a;
375         ret += b;
376 return (ret);
377 }
378
379
380 explicit CScript(char b) { operator<<(b); }
381 explicit CScript(short b) { operator<<(b); }
382 explicit CScript(int b) { operator<<(b); }
383 explicit CScript(long b) { operator<<(b); }
384 explicit CScript(int64 b) { operator<<(b); }
385 explicit CScript(unsigned char b) { operator<<(b); }
386 explicit CScript(unsigned int b) { operator<<(b); }
387 explicit CScript(unsigned short b) { operator<<(b); }
388 explicit CScript(unsigned long b) { operator<<(b); }
389 explicit CScript(uint64 b) { operator<<(b); }
```

```
390
391 explicit CScript(opcodetype b) { operator<<(b); }
392 explicit CScript(const uint256& b) { operator<<(b); }
393 explicit CScript(const CBigNum& b) { operator<<(b); }
394 explicit CScript(const vector<unsigned char>& b) { operator<<(b); }
395
396
397     CScript& operator<<(char b) { return (push_int64(b)); }
398     CScript& operator<<(short b) { return (push_int64(b)); }
399     CScript& operator<<(int b) { return (push_int64(b)); }
400     CScript& operator<<(long b) { return (push_int64(b)); }
401     CScript& operator<<(int64 b) { return (push_int64(b)); }
402     CScript& operator<<(unsigned char b) { return (push_uint64(b)); }
403     CScript& operator<<(unsigned int b) { return (push_uint64(b)); }
404     CScript& operator<<(unsigned short b) { return (push_uint64(b)); }
405     CScript& operator<<(unsigned long b) { return (push_uint64(b)); }
406     CScript& operator<<(uint64 b) { return (push_uint64(b)); }
407
408     CScript& operator<<(opcodetype opcode)
409 {
410 if (opcode <= OP_SINGLEBYTE_END)
411 {
412 insert(end(), (unsigned char)opcode);
413 }
414 else
415 {
416 assert(opcode >= OP_DOUBLEBYTE_BEGIN);
417 insert(end(), (unsigned char)(opcode >> 8));
418 insert(end(), (unsigned char)(opcode & 0xFF));
419 }
420 return (*this);
421 }
422
423     CScript& operator<<(const uint160& b)
424 {
425 insert(end(), sizeof(b));
426 insert(end(), (unsigned char*)&b, (unsigned char*)&b + sizeof(b));
427 return (*this);
428 }
429
430     CScript& operator<<(const uint256& b)
431 {
432 insert(end(), sizeof(b));
433 insert(end(), (unsigned char*)&b, (unsigned char*)&b + sizeof(b));
434 return (*this);
435 }
436
437     CScript& operator<<(const CBigNum& b)
438 {
439 *this << b.getvch();
440 return (*this);
441 }
442
443     CScript& operator<<(const vector<unsigned char>& b)
444 {
445 if (b.size() < OP_PUSHDATA1)
446 {
447 insert(end(), (unsigned char)b.size());
448 }
449 else if (b.size() <= 0xff)
450 {
451 insert(end(), OP_PUSHDATA1);
452 insert(end(), (unsigned char)b.size());
453 }
454 else
455 {
456 insert(end(), OP_PUSHDATA2);
457 unsigned short nSize = b.size();
458 insert(end(), (unsigned char*)&nSize, (unsigned char*)&nSize + sizeof(nSize));
459 }
460 insert(end(), b.begin(), b.end());
461 return (*this);
```

```
}

    CScript& operator<<(const CScript& b)
{
// I'm not sure if this should push the script or concatenate scripts.
// If there's ever a use for pushing a script onto a script, delete this member fn
assert(("warning: pushing a CScript onto a CScript with << is probably not intended, use + to concatenate", false));
return (*this);
}


bool GetOp(iterator& pc, opcodetype& opcodeRet, vector<unsigned char>& vchRet)
{
// This is why people hate C++
        const_iterator pc2 = pc;
bool fRet = GetOp(pc2, opcodeRet, vchRet);
        pc = begin() + (pc2 - begin());
return fRet;
}

bool GetOp(const_iterator& pc, opcodetype& opcodeRet, vector<unsigned char>& vchRet) const
{
        opcodeRet = OP_INVALIDOPCODE;
        vchRet.clear();
if (pc >= end())
return false;

// Read instruction
unsigned int opcode = *pc++;
if (opcode >= OP_SINGLEBYTE_END)
{
if (pc + 1 > end())
return false;
            opcode <<= 8;
            opcode |= *pc++;
}

// Immediate operand
if (opcode <= OP_PUSHDATA4)
{
unsigned int nSize = opcode;
if (opcode == OP_PUSHDATA1)
{
if (pc + 1 > end())
return false;
                nSize = *pc++;
}
else if (opcode == OP_PUSHDATA2)
{
if (pc + 2 > end())
return false;
                nSize = 0;
memcpy(&nSize, &pc[0], 2);
                pc += 2;
}
else if (opcode == OP_PUSHDATA4)
{
if (pc + 4 > end())
return false;
memcpy(&nSize, &pc[0], 4);
                pc += 4;
}
if (pc + nSize > end())
return false;
            vchRet.assign(pc, pc + nSize);
            pc += nSize;
}

        opcodeRet = (opcodetype)opcode;
return true;
}
```

```
533
534
535 void FindAndDelete(const CScript& b)
536 {
537         iterator pc = begin();
538         opcodetype opcode;
539         vector<unsigned char> vchPushValue;
540 int count = 0;
541 do
542 {
543 while (end() - pc >= b.size() && memcmp(&pc[0], &b[0], b.size()) == 0)
544 {
545 erase(pc, pc + b.size());
546                 count++;
547 }
548 }
549 while (GetOp(pc, opcode, vchPushValue));
550 //printf("FindAndDeleted deleted %d items\n", count); /// debug
551 }
552
553
554 void PrintHex() const
555 {
556 printf("CScript(%s)\n", HexStr(begin(), end()).c_str());
557 }
558
559     string ToString() const
560 {
561         string str;
562         opcodetype opcode;
563         vector<unsigned char> vch;
564         const_iterator it = begin();
565 while (GetOp(it, opcode, vch))
566 {
567 if (!str.empty())
568                 str += " ";
569 if (opcode <= OP_PUSHDATA4)
570                 str += ValueString(vch);
571 else
572                 str += GetOpName(opcode);
573 }
574 return str;
575 }
576
577 void print() const
578 {
579 printf("%s\n", ToString().c_str());
580 }
581 };
582
583
584
585
586
587
588
589
```

`EvalScript()` — the interpreter. Runs a script against a stack in the context of the spending transaction and input index. Returns true only if execution finishes with a true top-of-stack — the gate every payment must pass.

```
590 bool EvalScript(const CScript& script, const CTransaction& txTo, unsigned int nIn, int nHashType=0,
591                 vector<vector<unsigned char> >* pvStackRet=NULL);
```

`SignatureHash()` — builds the hash that ECDSA actually signs: a modified serialization of the tx shaped by the `SIGHASH_*` flags. Sign and verify must agree on this exact digest or signatures fail.

```
uint256 SignatureHash(CScript scriptCode, const CTransaction& txTo, unsigned int nIn, int nHashType);
bool IsMine(const CScript& scriptPubKey);
bool ExtractPubKey(const CScript& scriptPubKey, bool fMineOnly, vector<unsigned char>& vchPubKeyRet);
bool ExtractHash160(const CScript& scriptPubKey, uint160& hash160Ret);
```

`SignSignature()` / `VerifySignature()` — high-level helpers that produce or check a scriptSig for one input. Wallet send paths call Sign; `ConnectInputs` calls Verify for every non-coinbase input.

```
bool SignSignature(const CTransaction& txFrom, CTransaction& txTo, unsigned int nIn, int nHashType=SIGHASH_ALL, CScript scriptPrereq=CScript());
bool VerifySignature(const CTransaction& txFrom, const CTransaction& txTo, unsigned int nIn, int nHashType=0);

```

SUMMARY

Script's intentional limitations are features, not bugs. By avoiding Turing-completeness, Satoshi ensured scripts always terminate and have bounded resource usage. This makes validation predictable and prevents the halting problem that plagues other smart contract platforms.

The August 2010 opcode disabling (commit 4bd188c, titled only "misc changes") is forensically significant: it quietly turned off fifteen opcodes and capped stack elements at 520 bytes, closing a whole family of denial-of-service risks.[3][2] It followed weeks after the July 2010 fixes for the OP_LSHIFT crash and the OP_RETURN spend-anything bug.[4]

KEY TERMS

script

Bitcoin's stack-based language, defined by this header, that encodes the conditions under which coins can be spent.

opcode

One entry of the opcodetype enum naming a single-byte Script instruction; many entries are reserved or were disabled in 2010.

CScript

The class that holds a script as a vector of bytes; a script is inert data until EvalScript runs it.

Turing-complete

Describes a language that can loop forever; Script deliberately has no loops or recursion, so every script's running time is provably bounded.

OP_RETURN

The opcode that in v0.1.0 merely ended script execution, letting anyone spend any coin; v0.3.5 (July 2010) made it fail immediately, which later gave a clean way to embed data in provably unspendable outputs.

OP_HASH160

The opcode computing RIPEMD160(SHA256(x)), the 20-byte hash of a public key that a Bitcoin address encodes.

multisig

A spending condition requiring M of N possible signatures, verified by OP_CHECKMULTISIG and used for shared custody such as escrows and treasuries.

disabled opcode

An instruction such as OP_CAT or OP_MUL that stays in the enum but was turned off in August 2010 after denial-of-service bugs; any script using it fails.

SignatureHash

The function that builds the exact byte string a signature signs: a modified copy of the transaction shaped by the sighash flags.

SOURCES

1. Satoshi Nakamoto, *BitcoinTalk — Re: Transactions and Scripts* (June 2010). On supporting a tremendous variety of transaction types. https://bitcointalk.org/index.php?topic=195.0

2. Satoshi Nakamoto / Bitcoin developers, *OP_RETURN and dangerous opcodes disabled* (July 2010). Response to early Script vulnerabilities. https://en.bitcoin.it/wiki/Script#Disabled_opcodes
3. Satoshi Nakamoto, *bitcoin/bitcoin commit 4bd188c — "misc changes" (disables 15 opcodes, 520-byte push limit)* (15 August 2010). https://github.com/bitcoin/bitcoin/commit/4bd188c4383d6e614e18f79dc337fbabe8464c82
4. Bitcoin Wiki, *Common Vulnerabilities and Exposures — CVE-2010-5141 (OP_RETURN) and CVE-2010-5137* (July 28, 2010). Announced 2010-07-28; fixed in Bitcoin 0.3.5. https://en.bitcoin.it/wiki/Common_Vulnerabilities_and_Exposures
5. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
6. Bitcoin Core Contributors, *Bitcoin Core 30.0 release notes* (October 10, 2025). -datacarriersize default 100,000; legacy BDB wallets removed; Qt 6; bitcoin command. https://bitcoincore.org/en/releases/30.0/

serialize.h

1152 lines CPP

SPEAKING IN BYTES

For Bitcoin nodes to communicate, they need a common language for encoding data. This file defines Bitcoin's serialization system—the binary format for transactions, blocks, and network messages. At 39KB, it's one of the largest files in v0.1.0, reflecting the complexity of making C++ objects into portable byte streams.

Satoshi designed a custom format optimized for compactness. Variable-length integers ("CompactSize") save bytes for common small values. The format is deterministic: same data always produces same bytes, critical for hashing. Without determinism, two nodes serializing the same transaction could compute different hashes—catastrophic for consensus.

The ingenious `IMPLEMENT_SERIALIZE` macro is a masterclass in C++ metaprogramming. You write the serialization logic once, and the macro generates three functions: one to calculate size, one to write bytes, one to read bytes. The same code handles both directions, eliminating the bugs that plague separate encode/decode implementations.

This serialization format is now a de facto standard. Every Bitcoin implementation—whether written in C++, Rust, Go, Python, or JavaScript—must reproduce it byte for byte. A single-byte divergence causes a consensus fork. The legacy encoding still defines every txid today; SegWit (2017) added a witness extension alongside it rather than replacing it.[1] That makes it one of the most stable binary protocols in computing history—a testament to Satoshi's foresight in designing for permanence over convenience.[2]

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

#include <vector>
#include <map>
#include <boost/type_traits/is_fundamental.hpp>
#if defined(_MSC_VER) || defined(__BORLANDC__)
typedef __int64  int64;
typedef unsigned __int64  uint64;
#else
typedef long long  int64;
typedef unsigned long long  uint64;
#endif
#if defined(_MSC_VER) && _MSC_VER < 1300
#define for if (false) ; else for
#endif
class CScript;
class CDataStream;
class CAutoFile;

```

`VERSION = 101`—the protocol version embedded in serialized data. This allows format evolution: when reading old data, code can handle legacy formats. Later releases made the scheme explicit (300 for 0.3.0, 31300 for 0.3.13). This version is included in network messages and stored data for compatibility checking.

```
static const int VERSION = 101;




////////////////////////////////////////////////////////////////
//
// Templates for serializing to anything that looks like a stream,
// i.e. anything that supports .read(char*, int) and .write(char*, int)
//

```

Serialization context flags. `SER_NETWORK` means data going over the wire—includes all fields. `SER_DISK` means local storage—may include extra metadata. `SER_GETHASH` means computing a hash—a block's transaction list is excluded, so the block hash covers only the header. The modifier `SER_SKIPSIG` was meant to omit signatures—in v0.1.0 only the market code sets it, and nothing reads it; `SER_BLOCKHEADERONLY` serializes just the 80-byte header without transactions.

```
enum
{
// primary actions
    SER_NETWORK         = (1 << 0),
    SER_DISK            = (1 << 1),
    SER_GETHASH         = (1 << 2),

// modifiers
    SER_SKIPSIG         = (1 << 16),
    SER_BLOCKHEADERONLY = (1 << 17),
};

```

`IMPLEMENT_SERIALIZE`—the heart of Bitcoin's serialization. This macro generates three methods from one block of statements: (1) `GetSerializeSize()` counts bytes without writing, (2) `Serialize()` writes to a stream, (3) `Unserialize()` reads from a stream. The magic is in the boolean flags (`fGetSize`, `fWrite`, `fRead`) and the `READWRITE` macro that checks them.

```
#define IMPLEMENT_SERIALIZE(statements)\
unsigned int GetSerializeSize(int nType=0, int nVersion=VERSION) const  \
{                                                                        \
        CSerActionGetSerializeSize ser_action;  \
const bool fGetSize = true;             \
const bool fWrite = false;              \
const bool fRead = false;               \
unsigned int nSerSize = 0;              \
```

`ser_streamplaceholder`—a dummy stream for size calculation. When counting bytes, we don't need an actual output buffer. This placeholder satisfies the template but does nothing. The `nType` and `nVersion` are passed through for context-aware sizing (e.g., different versions might serialize differently).

```
        ser_streamplaceholder s;                \
        s.nType = nType;                        \
        s.nVersion = nVersion;                  \
{statements}                            \
return nSerSize;                        \
}                                           \
template<typename Stream>                   \
void Serialize(Stream& s, int nType=0, int nVersion=VERSION) const  \
{                                           \
        CSerActionSerialize ser_action;         \
const bool fGetSize = false;            \
const bool fWrite = true;               \
const bool fRead = false;               \
unsigned int nSerSize = 0;              \
{statements}                            \
}                                           \
template<typename Stream>                   \
void Unserialize(Stream& s, int nType=0, int nVersion=VERSION)  \
{                                           \
        CSerActionUnserialize ser_action;       \
const bool fGetSize = false;            \
const bool fWrite = false;              \
const bool fRead = true;                \
unsigned int nSerSize = 0;              \
{statements}                            \
}

```

`READWRITE(obj)`—the polymorphic serialization operator. Despite its name, it does three things depending on context: calculates size, writes, or reads. The `::SerReadWrite()` function dispatches based on `ser_action`'s type. This single macro handles integers, strings, vectors, maps, and any custom type with `IMPLEMENT_SERIALIZE`. The elegance: you write `READWRITE(nVersion);` `READWRITE(vTxIn); READWRITE(vTxOut);` and it just works for all three operations.

```
#define READWRITE(obj) (nSerSize += ::SerReadWrite(s, (obj), nType, nVersion, ser_action))
```

```
```

```
```

```
```

```
```

```
//
```

```
// Basic types
//
```

Raw data macros: `WRITEDATA` and `READDATA`. These directly read/write bytes without interpretation. Used for primitive types where the in-memory representation matches the serialized format. The `(char*)&(obj)` cast gets a byte pointer; `sizeof(obj)` determines how many bytes. No endianness conversion. Because the bytes are copied raw, the x86 little-endian layout effectively became the protocol; a big-endian build of v0.1.0 would have spoken a different, incompatible dialect.

```
#define WRITEDATA(s, obj)   s.write((char*)&(obj), sizeof(obj))
#define READDATA(s, obj)    s.read((char*)&(obj), sizeof(obj))

```

Primitive type serialization—overloaded for every basic type. Each `GetSerializeSize()` returns `sizeof(type)`; each `Serialize()` calls `WRITEDATA`; each `Unserialize()` calls `READDATA`. The repetitive overloads ensure type safety: the compiler selects the right version based on argument type. This is verbose but explicit—no surprises about how data is encoded.

```
inline unsigned int GetSerializeSize(char a, int, int=0) { return sizeof(a); }
inline unsigned int GetSerializeSize(signed char a, int, int=0) { return sizeof(a); }
inline unsigned int GetSerializeSize(unsigned char a, int, int=0) { return sizeof(a); }
inline unsigned int GetSerializeSize(signed short a, int, int=0) { return sizeof(a); }
inline unsigned int GetSerializeSize(unsigned short a, int, int=0) { return sizeof(a); }
inline unsigned int GetSerializeSize(signed int a, int, int=0) { return sizeof(a); }
inline unsigned int GetSerializeSize(unsigned int a, int, int=0) { return sizeof(a); }
inline unsigned int GetSerializeSize(signed long a, int, int=0) { return sizeof(a); }
inline unsigned int GetSerializeSize(unsigned long a, int, int=0) { return sizeof(a); }
inline unsigned int GetSerializeSize(int64 a, int, int=0) { return sizeof(a); }
inline unsigned int GetSerializeSize(uint64 a, int, int=0) { return sizeof(a); }
inline unsigned int GetSerializeSize(float a, int, int=0) { return sizeof(a); }
inline unsigned int GetSerializeSize(double a, int, int=0) { return sizeof(a); }

template<typename Stream> inline void Serialize(Stream& s, char a, int, int=0) { WRITEDATA(s, a); }
template<typename Stream> inline void Serialize(Stream& s, signed char a, int, int=0) { WRITEDATA(s, a);
}
template<typename Stream> inline void Serialize(Stream& s, unsigned char a, int, int=0) { WRITEDATA(s,
a); }
template<typename Stream> inline void Serialize(Stream& s, signed short a, int, int=0) { WRITEDATA(s,
a); }
template<typename Stream> inline void Serialize(Stream& s, unsigned short a, int, int=0) { WRITEDATA(s,
a); }
template<typename Stream> inline void Serialize(Stream& s, signed int a, int, int=0) { WRITEDATA(s, a);
}
template<typename Stream> inline void Serialize(Stream& s, unsigned int a, int, int=0) { WRITEDATA(s,
a); }
template<typename Stream> inline void Serialize(Stream& s, signed long a, int, int=0) { WRITEDATA(s, a);
}
template<typename Stream> inline void Serialize(Stream& s, unsigned long a, int, int=0) { WRITEDATA(s,
a); }
template<typename Stream> inline void Serialize(Stream& s, int64 a, int, int=0) { WRITEDATA(s, a); }
template<typename Stream> inline void Serialize(Stream& s, uint64 a, int, int=0) { WRITEDATA(s, a); }
template<typename Stream> inline void Serialize(Stream& s, float a, int, int=0) { WRITEDATA(s, a); }
template<typename Stream> inline void Serialize(Stream& s, double a, int, int=0) { WRITEDATA(s, a); }

template<typename Stream> inline void Unserialize(Stream& s, char& a, int, int=0) { READDATA(s, a); }
template<typename Stream> inline void Unserialize(Stream& s, signed char& a, int, int=0) { READDATA(s,
a); }
template<typename Stream> inline void Unserialize(Stream& s, unsigned char& a, int, int=0) { READDATA(s,
a); }
template<typename Stream> inline void Unserialize(Stream& s, signed short& a, int, int=0) { READDATA(s,
a); }
template<typename Stream> inline void Unserialize(Stream& s, unsigned short& a, int, int=0) {
READDATA(s, a); }
template<typename Stream> inline void Unserialize(Stream& s, signed int& a, int, int=0) { READDATA(s,
a); }
```

```
template<typename Stream> inline void Unserialize(Stream& s, unsigned int& a, int, int=0) { READDATA(s, a); }
template<typename Stream> inline void Unserialize(Stream& s, signed long& a, int, int=0) { READDATA(s, a); }
template<typename Stream> inline void Unserialize(Stream& s, unsigned long& a, int, int=0) { READDATA(s, a); }
template<typename Stream> inline void Unserialize(Stream& s, int64& a, int, int=0) { READDATA(s, a); }
template<typename Stream> inline void Unserialize(Stream& s, uint64& a, int, int=0) { READDATA(s, a); }
template<typename Stream> inline void Unserialize(Stream& s, float& a, int, int=0) { READDATA(s, a); }
template<typename Stream> inline void Unserialize(Stream& s, double& a, int, int=0) { READDATA(s, a); }

```

Boolean serialization—stored as a single byte. Note: `true` becomes `1`, `false` becomes `0`. When reading, any non-zero byte becomes `true`. This matches C's truthiness convention but means serialized bools are always `0x00` or `0x01`—never garbage values. Important for deterministic serialization.

```
inline unsigned int GetSerializeSize(bool a, int, int=0) { return sizeof(char); }
template<typename Stream> inline void Serialize(Stream& s, bool a, int, int=0) { char f=a; WRITEDATA(s, f); }
template<typename Stream> inline void Unserialize(Stream& s, bool& a, int, int=0) { char f; READDATA(s, f); a=f; }
```

`CompactSize` format—Satoshi's comment explains the encoding. Most values in Bitcoin are small: transaction input counts, output counts, script lengths. Using 4 bytes for a count that's usually 1-10 wastes space. CompactSize uses 1 byte for values 0-252, growing only when needed. This saves millions of bytes across the blockchain.

```
//
// Compact size
//  size <  253        -- 1 byte
//  size <= USHRT_MAX  -- 3 bytes  (253 + 2 bytes)
//  size <= UINT_MAX   -- 5 bytes  (254 + 4 bytes)
//  size >  UINT_MAX   -- 9 bytes  (255 + 8 bytes)
//
```

`GetSizeOfCompactSize()`—returns how many bytes a CompactSize will occupy. Essential for pre-calculating buffer sizes. The thresholds: <253 uses 1 byte, <65536 uses 3 bytes (marker + 2-byte value), <4 billion uses 5 bytes, larger uses 9 bytes. Most transaction data stays in the 1-byte range.

```
CALLED FROM  CBlock::ConnectBlock main.cpp p. 218 · GetSerializeSize p. 431 · GetSerializeSize_impl p. 432
inline unsigned int GetSizeOfCompactSize(uint64 nSize)
{
if (nSize < UCHAR_MAX-2) return sizeof(unsigned char);
else if (nSize <= USHRT_MAX) return sizeof(unsigned char) + sizeof(unsigned short);
else if (nSize <= UINT_MAX) return sizeof(unsigned char) + sizeof(unsigned int);
else return sizeof(unsigned char) + sizeof(uint64);
}

template<typename Stream>
```

`WriteCompactSize()`—encodes an integer in minimum bytes. The marker bytes `253`, `254`, `255` signal 'next 2/4/8 bytes contain the actual value.' This self-describing format means you can parse without external length information. Example: the number 500 serializes as `0xFD 0xF4 0x01` (`253` marker, then 500 in little-endian).

```
    CALLED FROM  Serialize p. 431 · Serialize_impl p. 432
161 void WriteCompactSize(Stream& os, uint64 nSize)
162 {
163 if (nSize < UCHAR_MAX-2)
164 {
165 unsigned char chSize = nSize;
166 WRITEDATA(os, chSize);
167 }
168 else if (nSize <= USHRT_MAX)
169 {
170 unsigned char chSize = UCHAR_MAX-2;
171 unsigned short xSize = nSize;
172 WRITEDATA(os, chSize);
173 WRITEDATA(os, xSize);
174 }
175 else if (nSize <= UINT_MAX)
176 {
177 unsigned char chSize = UCHAR_MAX-1;
178 unsigned int xSize = nSize;
179 WRITEDATA(os, chSize);
180 WRITEDATA(os, xSize);
181 }
182 else
183 {
184 unsigned char chSize = UCHAR_MAX;
185 WRITEDATA(os, chSize);
186 WRITEDATA(os, nSize);
187 }
188 return;
189 }
190
```

`ReadCompactSize()`—decodes a CompactSize integer. Reads one byte; if `<253`, that's the value. If `253`/`254`/`255`, reads 2/4/8 more bytes for the actual value. The return type is `uint64`—large enough for any CompactSize. Callers typically cast to appropriate smaller types after validation.

```
191 template<typename Stream>
    CALLED FROM  Unserialize p. 431 · Unserialize_impl p. 433
192 uint64 ReadCompactSize(Stream& is)
193 {
194 unsigned char chSize;
195 READDATA(is, chSize);
196 if (chSize < UCHAR_MAX-2)
197 {
198 return chSize;
199 }
200 else if (chSize == UCHAR_MAX-2)
201 {
202 unsigned short nSize;
203 READDATA(is, nSize);
204 return nSize;
205 }
206 else if (chSize == UCHAR_MAX-1)
207 {
208 unsigned int nSize;
209 READDATA(is, nSize);
210 return nSize;
211 }
212 else
```

```
{
        uint64 nSize;
READDATA(is, nSize);
return nSize;
}
}



//
// Wrapper for serializing arrays and POD
// There's a clever template way to make arrays serialize normally, but MSVC6 doesn't support it
//
```

`FLATDATA(obj)` — serialize a POD / array as raw bytes with no CompactSize length prefix. Used for fixed-layout structs like `COutPoint` and message-header fields where the wire format is exactly the in-memory layout.

```
#define FLATDATA(obj) REF(CFlatData((char*)&(obj), (char*)&(obj) + sizeof(obj)))
class CFlatData
{
protected:
char* pbegin;
char* pend;
public:
CFlatData(void* pbeginIn, void* pendIn) : pbegin((char*)pbeginIn), pend((char*)pendIn) { }
char* begin() { return pbegin; }
const char* begin() const { return pbegin; }
char* end() { return pend; }
const char* end() const { return pend; }

unsigned int GetSerializeSize(int, int=0) const
{
return pend - pbegin;
}

template<typename Stream>
void Serialize(Stream& s, int, int=0) const
{
        s.write(pbegin, pend - pbegin);
}

template<typename Stream>
void Unserialize(Stream& s, int, int=0)
{
        s.read(pbegin, pend - pbegin);
}
};



//
// string stored as a fixed length field
//
template<std::size_t LEN>
class CFixedFieldString
{
protected:
const string* pcstr;
    string* pstr;
public:
explicit CFixedFieldString(const string& str) : pcstr(&str), pstr(NULL) { }
explicit CFixedFieldString(string& str) : pcstr(&str), pstr(&str) { }

unsigned int GetSerializeSize(int, int=0) const
{
return LEN;
}

```

```
277 template<typename Stream>
278 void Serialize(Stream& s, int, int=0) const
279 {
280 char pszBuf[LEN];
281 strncpy(pszBuf, pcstr→c_str(), LEN);
282         s.write(pszBuf, LEN);
283 }
284
285 template<typename Stream>
286 void Unserialize(Stream& s, int, int=0)
287 {
288 if (pstr == NULL)
289 throw std::ios_base::failure("CFixedFieldString::Unserialize : trying to unserialize to const string");
290 char pszBuf[LEN+1];
291         s.read(pszBuf, LEN);
292         pszBuf[LEN] = '\0';
293 *pstr = pszBuf;
294 }
295 };
296
297
298
299
300
301 //
302 // Forward declarations
303 //
304
305 // string
306 template<typename C> unsigned int GetSerializeSize(const basic_string<C>& str, int, int=0);
307 template<typename Stream, typename C> void Serialize(Stream& os, const basic_string<C>& str, int,
    int=0);
308 template<typename Stream, typename C> void Unserialize(Stream& is, basic_string<C>& str, int, int=0);
309
310 // vector
```

Vector serialization — CompactSize length, then each element. Fundamentally typed vs untyped (byte) vectors use different impls so `vector<char>` dumps as a blob while `vector<CTxIn>` recurses into each input's `IMPLEMENT_SERIALIZE`.

```
311 template<typename T, typename A> unsigned int GetSerializeSize_impl(const std::vector<T, A>& v, int
    nType, int nVersion, const boost::true_type&);
312 template<typename T, typename A> unsigned int GetSerializeSize_impl(const std::vector<T, A>& v, int
    nType, int nVersion, const boost::false_type&);
313 template<typename T, typename A> inline unsigned int GetSerializeSize(const std::vector<T, A>& v, int
    nType, int nVersion=VERSION);
314 template<typename Stream, typename T, typename A> void Serialize_impl(Stream& os, const std::vector<T,
    A>& v, int nType, int nVersion, const boost::true_type&);
315 template<typename Stream, typename T, typename A> void Serialize_impl(Stream& os, const std::vector<T,
    A>& v, int nType, int nVersion, const boost::false_type&);
316 template<typename Stream, typename T, typename A> inline void Serialize(Stream& os, const std::vector<T,
    A>& v, int nType, int nVersion=VERSION);
317 template<typename Stream, typename T, typename A> void Unserialize_impl(Stream& is, std::vector<T, A>&
    v, int nType, int nVersion, const boost::true_type&);
318 template<typename Stream, typename T, typename A> void Unserialize_impl(Stream& is, std::vector<T, A>&
    v, int nType, int nVersion, const boost::false_type&);
319 template<typename Stream, typename T, typename A> inline void Unserialize(Stream& is, std::vector<T, A>&
    v, int nType, int nVersion=VERSION);
320
321 // others derived from vector
322 extern inline unsigned int GetSerializeSize(const CScript& v, int nType, int nVersion=VERSION);
323 template<typename Stream> void Serialize(Stream& os, const CScript& v, int nType, int nVersion=VERSION);
324 template<typename Stream> void Unserialize(Stream& is, CScript& v, int nType, int nVersion=VERSION);
325
326 // pair
327 template<typename K, typename T> unsigned int GetSerializeSize(const std::pair<K, T>& item, int nType,
    int nVersion=VERSION);
328 template<typename Stream, typename K, typename T> void Serialize(Stream& os, const std::pair<K, T>&
    item, int nType, int nVersion=VERSION);
```

```
329 template<typename Stream, typename K, typename T> void Unserialize(Stream& is, std::pair<K, T>& item,
    int nType, int nVersion=VERSION);
330
331 // map
332 template<typename K, typename T, typename Pred, typename A> unsigned int GetSerializeSize(const
    std::map<K, T, Pred, A>& m, int nType, int nVersion=VERSION);
333 template<typename Stream, typename K, typename T, typename Pred, typename A> void Serialize(Stream& os,
    const std::map<K, T, Pred, A>& m, int nType, int nVersion=VERSION);
334 template<typename Stream, typename K, typename T, typename Pred, typename A> void Unserialize(Stream&
    is, std::map<K, T, Pred, A>& m, int nType, int nVersion=VERSION);
335
336 // set
337 template<typename K, typename Pred, typename A> unsigned int GetSerializeSize(const std::set<K, Pred,
    A>& m, int nType, int nVersion=VERSION);
338 template<typename Stream, typename K, typename Pred, typename A> void Serialize(Stream& os, const
    std::set<K, Pred, A>& m, int nType, int nVersion=VERSION);
339 template<typename Stream, typename K, typename Pred, typename A> void Unserialize(Stream& is,
    std::set<K, Pred, A>& m, int nType, int nVersion=VERSION);
340
341
342
343
344
345 //
346 // If none of the specialized versions above matched, default to calling member function.
347 // "int nType" is changed to "long nType" to keep from getting an ambiguous overload error.
348 // The compiler will only cast int to long if none of the other templates matched.
349 // Thanks to Boost serialization for this idea.
350 //
351 template<typename T>
352 inline unsigned int GetSerializeSize(const T& a, long nType, int nVersion=VERSION)
353 {
354 return a.GetSerializeSize((int)nType, nVersion);
355 }
356
357 template<typename Stream, typename T>
358 inline void Serialize(Stream& os, const T& a, long nType, int nVersion=VERSION)
359 {
360     a.Serialize(os, (int)nType, nVersion);
361 }
362
363 template<typename Stream, typename T>
364 inline void Unserialize(Stream& is, T& a, long nType, int nVersion=VERSION)
365 {
366     a.Unserialize(is, (int)nType, nVersion);
367 }
368
369
370
371
372
373 //
374 // string
375 //
376 template<typename C>
377 unsigned int GetSerializeSize(const basic_string<C>& str, int, int)
378 {
379 return GetSizeOfCompactSize(str.size()) + str.size() * sizeof(str[0]);
380 }
381
382 template<typename Stream, typename C>
383 void Serialize(Stream& os, const basic_string<C>& str, int, int)
384 {
385 WriteCompactSize(os, str.size());
386 if (!str.empty())
387         os.write((char*)&str[0], str.size() * sizeof(str[0]));
388 }
389
390 template<typename Stream, typename C>
391 void Unserialize(Stream& is, basic_string<C>& str, int, int)
392 {
393 unsigned int nSize = ReadCompactSize(is);
```

```
394     str.resize(nSize);
395 if (nSize != 0)
396         is.read((char*)&str[0], nSize * sizeof(str[0]));
397 }
398
399
400
401 //
402 // vector
403 //
404 template<typename T, typename A>
405 unsigned int GetSerializeSize_impl(const std::vector<T, A>& v, int nType, int nVersion, const
    boost::true_type&)
406 {
407 return (GetSizeOfCompactSize(v.size()) + v.size() * sizeof(T));
408 }
409
410 template<typename T, typename A>
411 unsigned int GetSerializeSize_impl(const std::vector<T, A>& v, int nType, int nVersion, const
    boost::false_type&)
412 {
413 unsigned int nSize = GetSizeOfCompactSize(v.size());
414 for (typename std::vector<T, A>::const_iterator vi = v.begin(); vi != v.end(); ++vi)
415         nSize += GetSerializeSize((*vi), nType, nVersion);
416 return nSize;
417 }
418
419 template<typename T, typename A>
420 inline unsigned int GetSerializeSize(const std::vector<T, A>& v, int nType, int nVersion)
421 {
422 return GetSerializeSize_impl(v, nType, nVersion, boost::is_fundamental<T>());
423 }
424
425
426 template<typename Stream, typename T, typename A>
427 void Serialize_impl(Stream& os, const std::vector<T, A>& v, int nType, int nVersion, const
    boost::true_type&)
428 {
429 WriteCompactSize(os, v.size());
430 if (!v.empty())
431         os.write((char*)&v[0], v.size() * sizeof(T));
432 }
433
434 template<typename Stream, typename T, typename A>
435 void Serialize_impl(Stream& os, const std::vector<T, A>& v, int nType, int nVersion, const
    boost::false_type&)
436 {
437 WriteCompactSize(os, v.size());
438 for (typename std::vector<T, A>::const_iterator vi = v.begin(); vi != v.end(); ++vi)
439 ::Serialize(os, (*vi), nType, nVersion);
440 }
441
442 template<typename Stream, typename T, typename A>
443 inline void Serialize(Stream& os, const std::vector<T, A>& v, int nType, int nVersion)
444 {
445 Serialize_impl(os, v, nType, nVersion, boost::is_fundamental<T>());
446 }
447
448
449 template<typename Stream, typename T, typename A>
450 void Unserialize_impl(Stream& is, std::vector<T, A>& v, int nType, int nVersion, const
    boost::true_type&)
451 {
452 //unsigned int nSize = ReadCompactSize(is);
453 //v.resize(nSize);
454 //is.read((char*)&v[0], nSize * sizeof(T));
455
```

Chunked vector unserialize — grow in ~5 MB slices instead of `resize(nSize)` in one shot. A forged CompactSize claiming billions of elements cannot allocate the whole vector at once — an early memory DoS defense.

```
456 // Limit size per read so bogus size value won't cause out of memory
457     v.clear();
458 unsigned int nSize = ReadCompactSize(is);
459 unsigned int i = 0;
460 while (i < nSize)
461 {
462 unsigned int blk = min(nSize - i, 1 + 4999999 / sizeof(T));
463         v.resize(i + blk);
464         is.read((char*)&v[i], blk * sizeof(T));
465         i += blk;
466 }
467 }
468
469 template<typename Stream, typename T, typename A>
470 void Unserialize_impl(Stream& is, std::vector<T, A>& v, int nType, int nVersion, const
    boost::false_type&)
471 {
472 //unsigned int nSize = ReadCompactSize(is);
473 //v.resize(nSize);
474 //for (std::vector<T, A>::iterator vi = v.begin(); vi != v.end(); ++vi)
475 //    Unserialize(is, (*vi), nType, nVersion);
476
477     v.clear();
478 unsigned int nSize = ReadCompactSize(is);
479 unsigned int i = 0;
480 unsigned int nMid = 0;
481 while (nMid < nSize)
482 {
483         nMid += 5000000 / sizeof(T);
484 if (nMid > nSize)
485             nMid = nSize;
486         v.resize(nMid);
487 for (; i < nMid; i++)
488 Unserialize(is, v[i], nType, nVersion);
489 }
490 }
491
492 template<typename Stream, typename T, typename A>
493 inline void Unserialize(Stream& is, std::vector<T, A>& v, int nType, int nVersion)
494 {
495 Unserialize_impl(is, v, nType, nVersion, boost::is_fundamental<T>());
496 }
497
498
499
500 //
501 // others derived from vector
502 //
503 inline unsigned int GetSerializeSize(const CScript& v, int nType, int nVersion)
504 {
505 return GetSerializeSize((const vector<unsigned char>&)v, nType, nVersion);
506 }
507
508 template<typename Stream>
509 void Serialize(Stream& os, const CScript& v, int nType, int nVersion)
510 {
511 Serialize(os, (const vector<unsigned char>&)v, nType, nVersion);
512 }
513
514 template<typename Stream>
515 void Unserialize(Stream& is, CScript& v, int nType, int nVersion)
516 {
517 Unserialize(is, (vector<unsigned char>&)v, nType, nVersion);
518 }
```

```
519
520
521
522 //
523 // pair
524 //
525 template<typename K, typename T>
526 unsigned int GetSerializeSize(const std::pair<K, T>& item, int nType, int nVersion)
527 {
528 return GetSerializeSize(item.first, nType, nVersion) + GetSerializeSize(item.second, nType, nVersion);
529 }
530
531 template<typename Stream, typename K, typename T>
532 void Serialize(Stream& os, const std::pair<K, T>& item, int nType, int nVersion)
533 {
534 Serialize(os, item.first, nType, nVersion);
535 Serialize(os, item.second, nType, nVersion);
536 }
537
538 template<typename Stream, typename K, typename T>
539 void Unserialize(Stream& is, std::pair<K, T>& item, int nType, int nVersion)
540 {
541 Unserialize(is, item.first, nType, nVersion);
542 Unserialize(is, item.second, nType, nVersion);
543 }
544
545
546
547 //
548 // map
549 //
550 template<typename K, typename T, typename Pred, typename A>
551 unsigned int GetSerializeSize(const std::map<K, T, Pred, A>& m, int nType, int nVersion)
552 {
553 unsigned int nSize = GetSizeOfCompactSize(m.size());
554 for (typename std::map<K, T, Pred, A>::const_iterator mi = m.begin(); mi != m.end(); ++mi)
555         nSize += GetSerializeSize((*mi), nType, nVersion);
556 return nSize;
557 }
558
559 template<typename Stream, typename K, typename T, typename Pred, typename A>
560 void Serialize(Stream& os, const std::map<K, T, Pred, A>& m, int nType, int nVersion)
561 {
562 WriteCompactSize(os, m.size());
563 for (typename std::map<K, T, Pred, A>::const_iterator mi = m.begin(); mi != m.end(); ++mi)
564 Serialize(os, (*mi), nType, nVersion);
565 }
566
567 template<typename Stream, typename K, typename T, typename Pred, typename A>
568 void Unserialize(Stream& is, std::map<K, T, Pred, A>& m, int nType, int nVersion)
569 {
570     m.clear();
571 unsigned int nSize = ReadCompactSize(is);
572 typename std::map<K, T, Pred, A>::iterator mi = m.begin();
573 for (unsigned int i = 0; i < nSize; i++)
574 {
575         pair<K, T> item;
576 Unserialize(is, item, nType, nVersion);
577         mi = m.insert(mi, item);
578 }
579 }
580
581
582
583 //
584 // set
585 //
586 template<typename K, typename Pred, typename A>
587 unsigned int GetSerializeSize(const std::set<K, Pred, A>& m, int nType, int nVersion)
588 {
589 unsigned int nSize = GetSizeOfCompactSize(m.size());
590 for (typename std::set<K, Pred, A>::const_iterator it = m.begin(); it != m.end(); ++it)
```

```
591         nSize += GetSerializeSize((*it), nType, nVersion);
592 return nSize;
593 }
594
595 template<typename Stream, typename K, typename Pred, typename A>
596 void Serialize(Stream& os, const std::set<K, Pred, A>& m, int nType, int nVersion)
597 {
598 WriteCompactSize(os, m.size());
599 for (typename std::set<K, Pred, A>::const_iterator it = m.begin(); it != m.end(); ++it)
600 Serialize(os, (*it), nType, nVersion);
601 }
602
603 template<typename Stream, typename K, typename Pred, typename A>
604 void Unserialize(Stream& is, std::set<K, Pred, A>& m, int nType, int nVersion)
605 {
606     m.clear();
607 unsigned int nSize = ReadCompactSize(is);
608 typename std::set<K, Pred, A>::iterator it = m.begin();
609 for (unsigned int i = 0; i < nSize; i++)
610 {
611         K key;
612 Unserialize(is, key, nType, nVersion);
613         it = m.insert(it, key);
614 }
615 }
616
617
618
619 //
620 // Support for IMPLEMENT_SERIALIZE and READWRITE macro
621 //
622 class CSerActionGetSerializeSize { };
623 class CSerActionSerialize { };
624 class CSerActionUnserialize { };
625
626 template<typename Stream, typename T>
627 inline unsigned int SerReadWrite(Stream& s, const T& obj, int nType, int nVersion,
    CSerActionGetSerializeSize ser_action)
628 {
629 return ::GetSerializeSize(obj, nType, nVersion);
630 }
631
632 template<typename Stream, typename T>
633 inline unsigned int SerReadWrite(Stream& s, const T& obj, int nType, int nVersion, CSerActionSerialize
    ser_action)
634 {
635 ::Serialize(s, obj, nType, nVersion);
636 return 0;
637 }
638
639 template<typename Stream, typename T>
640 inline unsigned int SerReadWrite(Stream& s, T& obj, int nType, int nVersion, CSerActionUnserialize
    ser_action)
641 {
642 ::Unserialize(s, obj, nType, nVersion);
643 return 0;
644 }
645
646 struct ser_streamplaceholder
647 {
648 int nType;
649 int nVersion;
650 };
651
652
653
654
655
656
657
658
659
```

```
660 //
661 // Allocator that clears its contents before deletion
662 //
663 template<typename T>
```

secure_allocator — zeros memory on deallocate before returning it to the heap. CDataStream uses this so transient copies of keys and scripts are less likely to linger in RAM after free.

```
664 struct secure_allocator : public std::allocator<T>
665 {
666 // MSVC8 default copy constructor is broken
667 typedef std::allocator<T> base;
668 typedef typename base::size_type size_type;
669 typedef typename base::difference_type  difference_type;
670 typedef typename base::pointer pointer;
671 typedef typename base::const_pointer const_pointer;
672 typedef typename base::reference reference;
673 typedef typename base::const_reference const_reference;
674 typedef typename base::value_type value_type;
675 secure_allocator() throw() {}
676 secure_allocator(const secure_allocator& a) throw() : base(a) {}
677 ~secure_allocator() throw() {}
678 template<typename _Other> struct rebind
679 { typedef secure_allocator<_Other> other; };
680
681 void deallocate(T* p, std::size_t n)
682 {
683 if (p != NULL)
684 memset(p, 0, sizeof(T) * n);
685 allocator<T>::deallocate(p, n);
686 }
687 };
688
689
690
691 //
692 // Double ended buffer combining vector and stream-like interfaces.
693 // >> and << read and write unformatted data using the above serialization templates.
694 // Fills with data in linear time; some stringstream implementations take N^2 time.
695 //
```

CDataStream — Bitcoin's byte buffer: a vector with stream <</>> that invoke the serialization templates. Network messages, disk records, and hash preimages all pass through this type. Comment notes it fills in linear time unlike some stringstream implementations.

```
696 class CDataStream
697 {
698 protected:
699 typedef vector<char, secure_allocator<char> > vector_type;
700     vector_type vch;
701 unsigned int nReadPos;
702 short state;
703 short exceptmask;
704 public:
```

nType / nVersion — every stream carries serialization context (network vs disk vs gethash) and protocol version so the same object can omit fields when hashing or serving headers-only peers.

```
705 int nType;
706 int nVersion;
707
708 typedef vector_type::allocator_type   allocator_type;
709 typedef vector_type::size_type        size_type;
710 typedef vector_type::difference_type  difference_type;
```

```
711 typedef vector_type::reference        reference;
712 typedef vector_type::const_reference  const_reference;
713 typedef vector_type::value_type       value_type;
714 typedef vector_type::iterator         iterator;
715 typedef vector_type::const_iterator   const_iterator;
716 typedef vector_type::reverse_iterator reverse_iterator;
717
718 explicit CDataStream(int nTypeIn=0, int nVersionIn=VERSION)
719 {
720 Init(nTypeIn, nVersionIn);
721 }
722
723 CDataStream(const_iterator pbegin, const_iterator pend, int nTypeIn=0, int nVersionIn=VERSION) :
    vch(pbegin, pend)
724 {
725 Init(nTypeIn, nVersionIn);
726 }
727
728 #if !defined(_MSC_VER) || _MSC_VER >= 1300
729 CDataStream(const char* pbegin, const char* pend, int nTypeIn=0, int nVersionIn=VERSION) : vch(pbegin,
    pend)
730 {
731 Init(nTypeIn, nVersionIn);
732 }
733 #endif
734
735 CDataStream(const vector_type& vchIn, int nTypeIn=0, int nVersionIn=VERSION) : vch(vchIn.begin(),
    vchIn.end())
736 {
737 Init(nTypeIn, nVersionIn);
738 }
739
740 CDataStream(const vector<char>& vchIn, int nTypeIn=0, int nVersionIn=VERSION) : vch(vchIn.begin(),
    vchIn.end())
741 {
742 Init(nTypeIn, nVersionIn);
743 }
744
745 CDataStream(const vector<unsigned char>& vchIn, int nTypeIn=0, int nVersionIn=VERSION) :
    vch((char*)&vchIn.begin()[0], (char*)&vchIn.end()[0])
746 {
747 Init(nTypeIn, nVersionIn);
748 }
749
750 void Init(int nTypeIn=0, int nVersionIn=VERSION)
751 {
752         nReadPos = 0;
753         nType = nTypeIn;
754         nVersion = nVersionIn;
755         state = 0;
756         exceptmask = ios::badbit | ios::failbit;
757 }
758
759     CDataStream& operator+=(const CDataStream& b)
760 {
761         vch.insert(vch.end(), b.begin(), b.end());
762 return *this;
763 }
764
765 friend CDataStream operator+(const CDataStream& a, const CDataStream& b)
766 {
767         CDataStream ret = a;
768         ret += b;
769 return (ret);
770 }
771
772     string str() const
773 {
774 return (string(begin(), end()));
775 }
776
777
```

```
//
// Vector subset
//
    const_iterator begin() const { return vch.begin() + nReadPos; }
    iterator begin() { return vch.begin() + nReadPos; }
    const_iterator end() const { return vch.end(); }
    iterator end() { return vch.end(); }
    size_type size() const { return vch.size() - nReadPos; }
bool empty() const { return vch.size() == nReadPos; }
void resize(size_type n, value_type c=0) { vch.resize(n + nReadPos, c); }
void reserve(size_type n) { vch.reserve(n + nReadPos); }
    const_reference operator[](size_type pos) const { return vch[pos + nReadPos]; }
    reference operator[](size_type pos) { return vch[pos + nReadPos]; }
void clear() { vch.clear(); nReadPos = 0; }
    iterator insert(iterator it, const char& x=char()) { return vch.insert(it, x); }
void insert(iterator it, size_type n, const char& x) { vch.insert(it, n, x); }

void insert(iterator it, const_iterator first, const_iterator last)
{
if (it == vch.begin() + nReadPos && last - first <= nReadPos)
{
// special case for inserting at the front when there's room
            nReadPos -= (last - first);
memcpy(&vch[nReadPos], &first[0], last - first);
}
else
            vch.insert(it, first, last);
}

#if !defined(_MSC_VER) || _MSC_VER >= 1300
void insert(iterator it, const char* first, const char* last)
{
insert(it, (const_iterator)first, (const_iterator)last);
}
#endif

    iterator erase(iterator it)
{
if (it == vch.begin() + nReadPos)
{
// special case for erasing from the front
if (++nReadPos >= vch.size())
{
// whenever we reach the end, we take the opportunity to clear the buffer
                nReadPos = 0;
return vch.erase(vch.begin(), vch.end());
}
return vch.begin() + nReadPos;
}
else
return vch.erase(it);
}

    iterator erase(iterator first, iterator last)
{
if (first == vch.begin() + nReadPos)
{
// special case for erasing from the front
if (last == vch.end())
{
                nReadPos = 0;
return vch.erase(vch.begin(), vch.end());
}
else
{
                nReadPos = (last - vch.begin());
return last;
}
}
else
return vch.erase(first, last);
}
```

```
850
851 inline void Compact()
852 {
853         vch.erase(vch.begin(), vch.begin() + nReadPos);
854         nReadPos = 0;
855 }
856
857 bool Rewind(size_type n)
858 {
859 // Rewind by n characters if the buffer hasn't been compacted yet
860 if (n > nReadPos)
861 return false;
862         nReadPos -= n;
863 return true;
864 }
865
866
867 //
868 // Stream subset
869 //
870 void setstate(short bits, const char* psz)
871 {
872         state |= bits;
873 if (state & exceptmask)
874 throw std::ios_base::failure(psz);
875 }
876
877 bool eof() const { return size() == 0; }
878 bool fail() const { return state & (ios::badbit | ios::failbit); }
879 bool good() const { return !eof() && (state == 0); }
880 void clear(short n) { state = n; } // name conflict with vector clear()
881 short exceptions() { return exceptmask; }
882 short exceptions(short mask) { short prev = exceptmask; exceptmask = mask; setstate(0, "CDataStream");
    return prev; }
883     CDataStream* rdbuf() { return this; }
884 int in_avail() { return size(); }
885
886 void SetType(int n) { nType = n; }
887 int GetType() { return nType; }
888 void SetVersion(int n) { nVersion = n; }
889 int GetVersion() { return nVersion; }
890 void ReadVersion() { *this >> nVersion; }
891 void WriteVersion() { *this << nVersion; }
892
893     CDataStream& read(char* pch, int nSize)
894 {
895 // Read from the beginning of the buffer
896 assert(nSize >= 0);
897 unsigned int nReadPosNext = nReadPos + nSize;
898 if (nReadPosNext >= vch.size())
899 {
900 if (nReadPosNext > vch.size())
901 {
902 setstate(ios::failbit, "CDataStream::read() : end of data");
903 memset(pch, 0, nSize);
904                 nSize = vch.size() - nReadPos;
905 }
906 memcpy(pch, &vch[nReadPos], nSize);
907             nReadPos = 0;
908             vch.clear();
909 return (*this);
910 }
911 memcpy(pch, &vch[nReadPos], nSize);
912         nReadPos = nReadPosNext;
913 return (*this);
914 }
915
916     CDataStream& ignore(int nSize)
917 {
918 // Ignore from the beginning of the buffer
919 assert(nSize >= 0);
920 unsigned int nReadPosNext = nReadPos + nSize;
```

```
if (nReadPosNext >= vch.size())
{
if (nReadPosNext > vch.size())
{
setstate(ios::failbit, "CDataStream::ignore() : end of data");
                nSize = vch.size() - nReadPos;
}
            nReadPos = 0;
            vch.clear();
return (*this);
}
        nReadPos = nReadPosNext;
return (*this);
}

    CDataStream& write(const char* pch, int nSize)
{
// Write to the end of the buffer
assert(nSize >= 0);
        vch.insert(vch.end(), pch, pch + nSize);
return (*this);
}

template<typename Stream>
void Serialize(Stream& s, int nType=0, int nVersion=VERSION) const
{
// Special case: stream << stream concatenates like stream += stream
if (!vch.empty())
            s.write((char*)&vch[0], vch.size() * sizeof(vch[0]));
}

template<typename T>
unsigned int GetSerializeSize(const T& obj)
{
// Tells the size of the object if serialized to this stream
return ::GetSerializeSize(obj, nType, nVersion);
}

template<typename T>
    CDataStream& operator<<(const T& obj)
{
// Serialize to this stream
::Serialize(*this, obj, nType, nVersion);
return (*this);
}

template<typename T>
    CDataStream& operator>>(T& obj)
{
// Unserialize from this stream
::Unserialize(*this, obj, nType, nVersion);
return (*this);
}
};

#ifdef TESTCDATASTREAM
// VC6sp6
// CDataStream:
// n=1000       0 seconds
// n=2000       0 seconds
// n=4000       0 seconds
// n=8000       0 seconds
// n=16000      0 seconds
// n=32000      0 seconds
// n=64000      1 seconds
// n=128000     1 seconds
// n=256000     2 seconds
// n=512000     4 seconds
// n=1024000    8 seconds
// n=2048000    16 seconds
// n=4096000    32 seconds
// stringstream:
```

```
// n=1000       1 seconds
// n=2000       1 seconds
// n=4000       13 seconds
// n=8000       87 seconds
// n=16000      400 seconds
// n=32000      1660 seconds
// n=64000      6749 seconds
// n=128000     27241 seconds
// n=256000     109804 seconds
#include <iostream>
int main(int argc, char *argv[])
{
    vector<unsigned char> vch(0xcc, 250);
printf("CDataStream:\n");
for (int n = 1000; n <= 4500000; n *= 2)
{
        CDataStream ss;
        time_t nStart = time(NULL);
for (int i = 0; i < n; i++)
            ss.write((char*)&vch[0], vch.size());
printf("n=%-10d %d seconds\n", n, time(NULL) - nStart);
}
printf("stringstream:\n");
for (int n = 1000; n <= 4500000; n *= 2)
{
        stringstream ss;
        time_t nStart = time(NULL);
for (int i = 0; i < n; i++)
            ss.write((char*)&vch[0], vch.size());
printf("n=%-10d %d seconds\n", n, time(NULL) - nStart);
}
}
#endif









//
// Automatic closing wrapper for FILE*
//  - Will automatically close the file when it goes out of scope if not null.
//  - If you're returning the file pointer, return file.release().
//  - If you need to close the file early, use file.fclose() instead of fclose(file).
//
```

`CAutoFile` — RAII FILE* wrapper with the same <</>> serialization interface as `CDataStream`. Block files and wallet dumps use this so fclose happens even when exceptions abort a read.

```
class CAutoFile
{
protected:
    FILE* file;
short state;
short exceptmask;
public:
int nType;
int nVersion;

typedef FILE element_type;

CAutoFile(FILE* filenew=NULL, int nTypeIn=SER_DISK, int nVersionIn=VERSION)
{
        file = filenew;
        nType = nTypeIn;
```

```
1058         nVersion = nVersionIn;
1059         state = 0;
1060         exceptmask = ios::badbit | ios::failbit;
1061 }
1062
1063 ~CAutoFile()
1064 {
1065 fclose();
1066 }
1067
1068 void fclose()
1069 {
1070 if (file != NULL && file != stdin && file != stdout && file != stderr)
1071 ::fclose(file);
1072         file = NULL;
1073 }
1074
1075     FILE* release() { FILE* ret = file; file = NULL; return ret; }
1076 operator FILE*() { return file; }
1077     FILE* operator->() { return file; }
1078     FILE& operator*() { return *file; }
1079     FILE** operator&() { return &file; }
1080     FILE* operator=(FILE* pnew) { return file = pnew; }
1081 bool operator!() { return (file == NULL); }
1082
1083
1084 //
1085 // Stream subset
1086 //
1087 void setstate(short bits, const char* psz)
1088 {
1089         state |= bits;
1090 if (state & exceptmask)
1091 throw std::ios_base::failure(psz);
1092 }
1093
1094 bool fail() const { return state & (ios::badbit | ios::failbit); }
1095 bool good() const { return state == 0; }
1096 void clear(short n = 0) { state = n; }
1097 short exceptions() { return exceptmask; }
1098 short exceptions(short mask) { short prev = exceptmask; exceptmask = mask; setstate(0, "CAutoFile");
    return prev; }
1099
1100 void SetType(int n) { nType = n; }
1101 int GetType() { return nType; }
1102 void SetVersion(int n) { nVersion = n; }
1103 int GetVersion() { return nVersion; }
1104 void ReadVersion() { *this >> nVersion; }
1105 void WriteVersion() { *this << nVersion; }
1106
1107     CAutoFile& read(char* pch, int nSize)
1108 {
1109 if (!file)
1110 throw std::ios_base::failure("CAutoFile::read : file handle is NULL");
1111 if (fread(pch, 1, nSize, file) != nSize)
1112 setstate(ios::failbit, feof(file) ? "CAutoFile::read : end of file" : "CAutoFile::read : fread failed");
1113 return (*this);
1114 }
1115
1116     CAutoFile& write(const char* pch, int nSize)
1117 {
1118 if (!file)
1119 throw std::ios_base::failure("CAutoFile::write : file handle is NULL");
1120 if (fwrite(pch, 1, nSize, file) != nSize)
1121 setstate(ios::failbit, "CAutoFile::write : write failed");
1122 return (*this);
1123 }
1124
1125 template<typename T>
1126 unsigned int GetSerializeSize(const T& obj)
1127 {
1128 // Tells the size of the object if serialized to this stream
```

```
return ::GetSerializeSize(obj, nType, nVersion);
}

template<typename T>
    CAutoFile& operator<<(const T& obj)
{
// Serialize to this stream
if (!file)
throw std::ios_base::failure("CAutoFile::operator<< : file handle is NULL");
::Serialize(*this, obj, nType, nVersion);
return (*this);
}

template<typename T>
    CAutoFile& operator>>(T& obj)
{
// Unserialize from this stream
if (!file)
throw std::ios_base::failure("CAutoFile::operator>> : file handle is NULL");
::Unserialize(*this, obj, nType, nVersion);
return (*this);
}
};

```

SUMMARY

The serialization format is consensus-critical. If two nodes serialize differently, they'll compute different transaction/block hashes and fork. Satoshi's IMPLEMENT_SERIALIZE macro ensures consistency: one definition, no chance for read/write mismatch. This design pattern has prevented countless potential bugs.

The format's success lies in its simplicity and stability. No schema versioning complexity (version number is inline), no compression (bytes are bytes), no encryption at this layer. Just raw, deterministic encoding. Every alternative cryptocurrency that uses Bitcoin's code inherits this format—it's become the Rosetta Stone of blockchain data.

The CompactSize encoding deserves recognition as elegant engineering. Most blockchain data involves small counts (typically 1-3 inputs/outputs), so optimizing for small values saves significant space. Across millions of transactions, these byte savings compound into gigabytes of reduced blockchain size. Satoshi optimized for the common case while gracefully handling the rare large values.

KEY TERMS

serialization

The conversion of in-memory objects to bytes for disk, network, or hashing, and back again.

CompactSize

The variable-length integer encoding that stores a count in one byte when it is under 253, growing to 3, 5, or 9 bytes only for larger values.

CDataStream

Bitcoin's byte buffer with stream operators that serialize objects as they pass through; every network message flows through one.

FLATDATA

The macro serializing a fixed-layout struct as raw bytes with no length prefix, for fields whose wire format is exactly their memory layout.

IMPLEMENT_SERIALIZE

The macro that expands one block of READWRITE statements into three functions, size, write, and read, so encoding and decoding can never drift apart.

READWRITE

The macro applied to each field that counts, writes, or reads it depending on which of the three operations is in progress.

deterministic

The property that the same object always serializes to the same bytes; without it two nodes would hash the same transaction differently and fork.

protocol version

The VERSION constant (101 for v0.1.0) carried in every stream so newer software can still parse data written by older releases.

secure_allocator

The allocator that zeroes memory before freeing it, used for stream buffers that may hold private keys.

SOURCES

1. Eric Lombrozo, Pieter Wuille, *BIP 144 — Segregated Witness (Peer Services)* (2016). https://github.com/bitcoin/bips/blob/master/bip-0144.mediawiki
2. Satoshi Nakamoto, *BitcoinTalk — Re: Bitcoin 0.3.21* (June 17, 2010). On the core design being set in stone for the rest of its lifetime. https://bitcointalk.org/index.php?topic=195.msg1611#msg1611

sha.cpp

555 lines CPP

THE HASH THAT SECURES TRILLIONS

This file is not Satoshi's code. It is Crypto++ 5.5.2, written by Wei Dai, who adapted it from Steve Reid's public domain SHA-1 implementation.[1] Three authors, two algorithms, one file. In v0.1.0 it has exactly one customer: the CPU miner, which calls `SHA256::Transform` directly for speed. Everywhere else — transaction IDs, block validation, Merkle roots, addresses — Bitcoin hashes with OpenSSL's SHA-256 (`Hash()` in util.h). Two independent implementations of the same algorithm, and they must agree.

A hash function takes arbitrary input and produces a fixed-size output — 256 bits for SHA-256, 160 bits for SHA-1. The output is a *digest*: a fingerprint that changes completely if even one bit of input changes. The key properties are preimage resistance (you cannot find the input from the output), second-preimage resistance (you cannot find a different input with the same output), and collision resistance (you cannot find any two inputs with the same output). SHA-256 provides roughly 256 bits of security against the preimage attacks and 128 bits against collisions — strong enough that no practical weakness has ever been found.

Bitcoin uses SHA-256 in a double-application pattern. Every transaction ID, block hash, and Merkle tree node is computed as `SHA256(SHA256(data))` — abbreviated SHA256d. The double application was a common defensive practice in 2009, designed to eliminate a class of length-extension attacks that affect single SHA-256. In retrospect it was unnecessary for Bitcoin's specific usage, but the cost is negligible.

SHA-1 appears here because Crypto++ included it, not because Bitcoin uses this copy. Block and transaction hashing is SHA-256 throughout.[2] (Script does expose SHA-1 as the `OP_SHA1` opcode, but that calls OpenSSL, not this file.) SHA-1 was already known to be weakening in 2009 — Wang Xiaoyun had demonstrated theoretical collision attacks in 2005.[3] Its presence in the file is a historical artifact, not design.

```
// This file is public domain
// SHA routines extracted as a standalone file from:
// Crypto++: a C++ Class Library of Cryptographic Schemes
// Version 5.5.2 (9/24/2007)
// http://www.cryptopp.com

```

Three authorship layers. Steve Reid wrote the original SHA-1 in C (sha1.c), public domain. Wei Dai ported it to C++ for Crypto++ and added SHA-256.[1] Satoshi included Crypto++ 5.5.2 verbatim rather than writing hash functions from scratch — the same delegation strategy he used with OpenSSL for elliptic curve arithmetic. The comment attributing Reid and Wei Dai is doing important work: it documents the provenance of code that secures billions of dollars.

```
// sha.cpp - modified by Wei Dai from Steve Reid's public domain sha1.c

// Steve Reid implemented SHA-1. Wei Dai implemented SHA-2.
// Both are in the public domain.

#include <assert.h>
#include <memory.h>
#include "sha.h"

namespace CryptoPP
{

// start of Steve Reid's code

#define blk0(i) (W[i] = data[i])
#define blk1(i) (W[i&15] = rotlFixed(W[(i+13)&15]^W[(i+8)&15]^W[(i+2)&15]^W[i&15],1))

```

SHA-1 initial state: five 32-bit magic constants. The first four — 0×67452301, 0xEFCDAB89, 0×98BADCFE, 0×10325476 — are inherited unchanged from MD4 and MD5: a simple pattern of ascending and descending hex digits, not a mathematical derivation. 0xC3D2E1F0 was added for the fifth word. The 'nothing up my sleeve' principle belongs to the round constants below and to SHA-256's initial state: constants derived from well-known mathematical expressions cannot encode a backdoor, because anyone can verify the derivation.

```
void SHA1::InitState(HashWordType *state)
{
    state[0] = 0x67452301L;
    state[1] = 0xEFCDAB89L;
    state[2] = 0x98BADCFEL;
    state[3] = 0x10325476L;
    state[4] = 0xC3D2E1F0L;
}

```

SHA-1's four Boolean mixing functions. f1 is Choose: if x then y else z. f2 is Parity: x XOR y XOR z. f3 is Majority: the value held by at least two of the three inputs. f4 is Parity again. These four functions are used in four groups of 20 rounds each. The mathematical diversity of the mixing operations — bitwise AND, OR, XOR in different combinations — is what makes SHA-1 resist differential cryptanalysis. The same principle governs SHA-256's Ch and Maj macros.

```
#define f1(x,y,z) (z^(x&(y^z)))
#define f2(x,y,z) (x^y^z)
#define f3(x,y,z) ((x&y)|(z&(x|y)))
#define f4(x,y,z) (x^y^z)

/* (R0+R1), R2, R3, R4 are the different operations used in SHA1 */
```

SHA-1 round constants: one per 20-round group. 0×5A827999 = floor($2^{30} \times \sqrt{2}$), 0×6ED9EBA1 = floor($2^{30} \times \sqrt{3}$), 0×8F1BBCDC = floor($2^{30} \times \sqrt{5}$), 0xCA62C1D6 = floor($2^{30} \times \sqrt{10}$). Again, square roots of small integers — verifiably non-random. SHA-256 uses 64 distinct constants (cube roots of 64 primes) instead of 4 repeated ones. The more varied schedule was part of the SHA-2 hardening.

```
#define R0(v,w,x,y,z,i) z+=f1(w,x,y)+blk0(i)+0x5A827999+rotlFixed(v,5);w=rotlFixed(w,30);
#define R1(v,w,x,y,z,i) z+=f1(w,x,y)+blk1(i)+0x5A827999+rotlFixed(v,5);w=rotlFixed(w,30);
#define R2(v,w,x,y,z,i) z+=f2(w,x,y)+blk1(i)+0x6ED9EBA1+rotlFixed(v,5);w=rotlFixed(w,30);
#define R3(v,w,x,y,z,i) z+=f3(w,x,y)+blk1(i)+0x8F1BBCDC+rotlFixed(v,5);w=rotlFixed(w,30);
#define R4(v,w,x,y,z,i) z+=f4(w,x,y)+blk1(i)+0xCA62C1D6+rotlFixed(v,5);w=rotlFixed(w,30);

```

SHA-1 compression function: 80 rounds, fully unrolled. The loop is manually unrolled into 80 individual macro calls — no loop overhead, no branch prediction misses. At each round, the five working variables (a,b,c,d,e) rotate: e gets the new computed value, and all others shift by one position. The message block W[16] is expanded in-place via blk1's XOR-rotate schedule. By round 80, every input bit has influenced every output bit — the avalanche effect.

```
void SHA1::Transform(word32 *state, const word32 *data)
{
    word32 W[16];
/* Copy context->state[] to working vars */
    word32 a = state[0];
    word32 b = state[1];
    word32 c = state[2];
    word32 d = state[3];
    word32 e = state[4];
/* 4 rounds of 20 operations each. Loop unrolled. */
R0(a,b,c,d,e, 0); R0(e,a,b,c,d, 1); R0(d,e,a,b,c, 2); R0(c,d,e,a,b, 3);
R0(b,c,d,e,a, 4); R0(a,b,c,d,e, 5); R0(e,a,b,c,d, 6); R0(d,e,a,b,c, 7);
R0(c,d,e,a,b, 8); R0(b,c,d,e,a, 9); R0(a,b,c,d,e,10); R0(e,a,b,c,d,11);
R0(d,e,a,b,c,12); R0(c,d,e,a,b,13); R0(b,c,d,e,a,14); R0(a,b,c,d,e,15);
R1(e,a,b,c,d,16); R1(d,e,a,b,c,17); R1(c,d,e,a,b,18); R1(b,c,d,e,a,19);
R2(a,b,c,d,e,20); R2(e,a,b,c,d,21); R2(d,e,a,b,c,22); R2(c,d,e,a,b,23);
R2(b,c,d,e,a,24); R2(a,b,c,d,e,25); R2(e,a,b,c,d,26); R2(d,e,a,b,c,27);
R2(c,d,e,a,b,28); R2(b,c,d,e,a,29); R2(a,b,c,d,e,30); R2(e,a,b,c,d,31);
R2(d,e,a,b,c,32); R2(c,d,e,a,b,33); R2(b,c,d,e,a,34); R2(a,b,c,d,e,35);
R2(e,a,b,c,d,36); R2(d,e,a,b,c,37); R2(c,d,e,a,b,38); R2(b,c,d,e,a,39);
R3(a,b,c,d,e,40); R3(e,a,b,c,d,41); R3(d,e,a,b,c,42); R3(c,d,e,a,b,43);
R3(b,c,d,e,a,44); R3(a,b,c,d,e,45); R3(e,a,b,c,d,46); R3(d,e,a,b,c,47);
R3(c,d,e,a,b,48); R3(b,c,d,e,a,49); R3(a,b,c,d,e,50); R3(e,a,b,c,d,51);
R3(d,e,a,b,c,52); R3(c,d,e,a,b,53); R3(b,c,d,e,a,54); R3(a,b,c,d,e,55);
R3(e,a,b,c,d,56); R3(d,e,a,b,c,57); R3(c,d,e,a,b,58); R3(b,c,d,e,a,59);
R4(a,b,c,d,e,60); R4(e,a,b,c,d,61); R4(d,e,a,b,c,62); R4(c,d,e,a,b,63);
R4(b,c,d,e,a,64); R4(a,b,c,d,e,65); R4(e,a,b,c,d,66); R4(d,e,a,b,c,67);
R4(c,d,e,a,b,68); R4(b,c,d,e,a,69); R4(a,b,c,d,e,70); R4(e,a,b,c,d,71);
R4(d,e,a,b,c,72); R4(c,d,e,a,b,73); R4(b,c,d,e,a,74); R4(a,b,c,d,e,75);
R4(e,a,b,c,d,76); R4(d,e,a,b,c,77); R4(c,d,e,a,b,78); R4(b,c,d,e,a,79);
/* Add the working vars back into context.state[] */
```

Merkle-Damgård finalization. After 80 rounds, the working variables are added back to the incoming state. This is the Merkle-Damgård construction: the output of each 512-bit block feeds into the next as initial state. A single-bit change anywhere in the input propagates through all subsequent blocks. It is also why SHA-1 is vulnerable to length-extension — the chaining value after the last block is directly the hash output, allowing an attacker to continue hashing without knowing the original input.

```
    state[0] += a;
    state[1] += b;
```

```
    state[2] += c;
    state[3] += d;
    state[4] += e;
}

// end of Steve Reid's code

// ************************************************************

void SHA224::InitState(HashWordType *state)
{
static const word32 s[8] = {0xc1059ed8, 0x367cd507, 0x3070dd17, 0xf70e5939, 0xffc00b31, 0x68581511,
0x64f98fa7, 0xbefa4fa4};
memcpy(state, s, sizeof(s));
}

```

SHA-256 initial hash values: eight 32-bit constants. These are the first 32 bits of the fractional parts of the square roots of the first eight primes (2, 3, 5, 7, 11, 13, 17, 19). 0×6a09e667 ≈ frac(√2) × 2^{32}, 0xbb67ae85 ≈ frac(√3) × 2^{32}, and so on through prime 19. Every SHA-256 computation in Bitcoin — every transaction ID, every block hash — starts here, with these eight values. They are the fixed point from which all digests are computed.

```
void SHA256::InitState(HashWordType *state)
{
static const word32 s[8] = {0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a, 0x510e527f, 0x9b05688c,
0x1f83d9ab, 0x5be0cd19};
memcpy(state, s, sizeof(s));
}

```

SHA-256 round constants: 64 values, one per round. Each is the first 32 bits of the fractional part of the cube root of a prime: 0×428a2f98 ≈ frac(∛2) × 2^{32}, 0×71374491 ≈ frac(∛3) × 2^{32}, through the 64th prime. These constants break the symmetry that would otherwise make the round function self-similar. Without distinct constants per round, an attacker could find structural cancellations across rounds — the constants prevent it. Bitcoin miners compute these 64 additions billions of times per second.

```
static const word32 SHA256_K[64] = {
0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5,
0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3,
0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc,
0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7,
0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13,
0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3,
0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5,
0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208,
0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
};

```

Message schedule expansion. blk0 loads raw input; blk2 computes the expanded schedule. W[i&15] is updated in-place, so only 16 words of storage are needed for a 64-round schedule. The expansion mixes four previous words with two sigma rotations: s1 applied to W[i-2] and s0 applied to W[i-15]. This ensures that each expanded word depends on many input bits — a change to any input word affects 48 subsequent schedule words.

```
#define blk2(i) (W[i&15]+=s1(W[(i-2)&15])+W[(i-7)&15]+s0(W[(i-15)&15]))

```

Ch and Maj: the Boolean heart of SHA-256. Ch(x,y,z) = (x & y) | (~x & z) — 'choose': if bit of x is 1, take the bit from y; if 0, take from z. Maj(x,y,z) = (x & y) | (z & (x | y)) — 'majority': the bit value held by at least two of the three inputs. Wei Dai's implementation uses the identity (z ^ (x & (y ^ z))) for Ch to avoid computing ~x — arithmetically equivalent but one fewer NOT. Both functions are the same as the f1 and f3 from SHA-1, now applied to all 64 rounds rather than alternating.

```
#define Ch(x,y,z) (z^(x&(y^z)))
#define Maj(x,y,z) ((x&y)|(z&(x|y)))

```

Circular state indexing. Instead of eight named variables a–h that must be explicitly rotated each round, Wei Dai stores the state in T[8] and accesses it via modular index macros. a(i) = T[(0-i)&7], b(i) = T[(1-i)&7], etc. At round i, 'a' refers to whatever slot currently holds the top of the rotation. The effect: no actual rotation happens. This is a performance optimization — the compiler sees eight array accesses instead of eight assignments. The mathematical result is identical; the generated machine code is faster.

```
#define a(i) T[(0-i)&7]
#define b(i) T[(1-i)&7]
#define c(i) T[(2-i)&7]
#define d(i) T[(3-i)&7]
#define e(i) T[(4-i)&7]
#define f(i) T[(5-i)&7]
#define g(i) T[(6-i)&7]
#define h(i) T[(7-i)&7]

```

The SHA-256 round macro in full. One round of SHA-256 in two lines: T1 = h + S1(e) + Ch(e,f,g) + K[i+j] + W[i]; d += T1; h = T1 + S0(a) + Maj(a,b,c). The big-sigma functions S0 and S1 are three-way XOR of rotations: S0(x) = ROTR(x,2) ^ ROTR(x,13) ^ ROTR(x,22); S1(x) = ROTR(x,6) ^ ROTR(x,11) ^ ROTR(x,25). These specific rotation amounts were chosen by NIST to maximize diffusion — each output bit depends on many input bits within one round.

```
#define R(i) h(i)+=S1(e(i))+Ch(e(i),f(i),g(i))+SHA256_K[i+j]+(j?blk2(i):blk0(i));\
d(i)+=h(i);h(i)+=S0(a(i))+Maj(a(i),b(i),c(i))

// for SHA256
```

Big-sigma vs. little-sigma. S0 and S1 (capital) operate on state variables; s0 and s1 (lowercase) operate on the message schedule. SHA-256 uses only XOR of rotations for the big-sigma functions and XOR of rotations plus a right-shift for the little-sigma functions. The shift (>> 3 and >> 10) introduces asymmetry that prevents the schedule from being self-similar. The macro names are #undef'd at line 241 before SHA-512 redefines them with different rotation amounts — SHA-512 uses 64-bit words and different constants, but the same algebraic structure.

```
#define S0(x) (rotrFixed(x,2)^rotrFixed(x,13)^rotrFixed(x,22))
#define S1(x) (rotrFixed(x,6)^rotrFixed(x,11)^rotrFixed(x,25))
#define s0(x) (rotrFixed(x,7)^rotrFixed(x,18)^(x>>3))
#define s1(x) (rotrFixed(x,17)^rotrFixed(x,19)^(x>>10))

```

`SHA256::Transform`: the miner's engine. In v0.1.0 only `BitcoinMiner` calls this Crypto++ routine (through `BlockSHA256` in main.cpp), because hashing a pre-formatted header block by block is faster. Every node validates hashes with OpenSSL's SHA-256 instead (`Hash()` in util.h). Satoshi made the two check each other: before broadcasting a block, the miner asserts `hash == pblock->GetHash()`. Had this code contained a wrong rotation or an off-by-one, the miner would have produced blocks the network rejects—wasted work, not a chain split. Consensus rests on the SHA-256 algorithm, not on this particular copy of it.

```
void SHA256::Transform(word32 *state, const word32 *data)
{
    word32 W[16];
    word32 T[8];
/* Copy context->state[] to working vars */
```

`T[8]` holds the working state; state[] holds the persistent chain value. memcpy copies the incoming state (the previous block's output, or the initial values for the first block) into T. After 64 rounds of mixing, the result is added back to state[]. This addition — not replacement — is the Merkle-Damgård construction. It ensures that even if an attacker can engineer a specific T[] output, the final result still depends on the full prior chain.

```
memcpy(T, state, sizeof(T));
/* 64 operations, partially loop unrolled */
```

64 rounds, partially unrolled into groups of 16. Full unrolling would produce 64 × 2 = 128 lines; Wei Dai chose 16 per iteration as a compromise between code size and branch overhead. The variable j steps through 0, 16, 32, 48. Inside each group, j is fixed, so K[i+j] and (j?blk2(i):blk0(i)) correctly select either raw input (j=0, first 16 rounds) or expanded schedule (j>0, rounds 17–64). Bitcoin miners run this loop for every nonce they try — billions of times per second.

```
for (unsigned int j=0; j<64; j+=16)
{
R( 0); R( 1); R( 2); R( 3);
R( 4); R( 5); R( 6); R( 7);
R( 8); R( 9); R(10); R(11);
R(12); R(13); R(14); R(15);
}
/* Add the working vars back into context.state[] */
    state[0] += a(0);
    state[1] += b(0);
    state[2] += c(0);
    state[3] += d(0);
    state[4] += e(0);
    state[5] += f(0);
```

```
    state[6] += g(0);
    state[7] += h(0);
}

```

The commented-out 'smaller but slower' alternative. This is Wei Dai's original reference implementation, preserved in a block comment. It uses explicit variables rather than the circular-index trick, making the SHA-256 round structure much clearer. It was commented out because the indexed version is measurably faster on real hardware — the compiler can better schedule the modular array accesses. The comment is a rare gift: two complete implementations of SHA-256 in the same file, one optimized, one readable.

```
/*
// smaller but slower
void SHA256_Transform(word32 *state, const word32 *data)
{
    word32 T[20];
    word32 W[32];
unsigned int i = 0, j = 0;
    word32 *t = T+8;

memcpy(t, state, 8*4);
    word32 e = t[4], a = t[0];

do
{
        word32 w = data[j];
        W[j] = w;
        w += K[j];
        w += t[7];
        w += S1(e);
        w += Ch(e, t[5], t[6]);
        e = t[3] + w;
        t[3] = t[3+8] = e;
        w += S0(t[0]);
        a = w + Maj(a, t[1], t[2]);
        t[-1] = t[7] = a;
--t;
++j;
if (j%8 == 0)
            t += 8;
} while (j<16);

do
{
        i = j&0xf;
        word32 w = s1(W[i+16-2]) + s0(W[i+16-15]) + W[i] + W[i+16-7];
        W[i+16] = W[i] = w;
        w += K[j];
        w += t[7];
        w += S1(e);
        w += Ch(e, t[5], t[6]);
        e = t[3] + w;
        t[3] = t[3+8] = e;
        w += S0(t[0]);
        a = w + Maj(a, t[1], t[2]);
        t[-1] = t[7] = a;

        w = s1(W[(i+1)+16-2]) + s0(W[(i+1)+16-15]) + W[(i+1)] + W[(i+1)+16-7];
        W[(i+1)+16] = W[(i+1)] = w;
        w += K[j+1];
        w += (t-1)[7];
        w += S1(e);
        w += Ch(e, (t-1)[5], (t-1)[6]);
        e = (t-1)[3] + w;
(t-1)[3] = (t-1)[3+8] = e;
        w += S0((t-1)[0]);
        a = w + Maj(a, (t-1)[1], (t-1)[2]);
```

```
(t-1)[-1] = (t-1)[7] = a;

        t-=2;
        j+=2;
if (j%8 == 0)
            t += 8;
} while (j<64);

    state[0] += a;
    state[1] += t[1];
    state[2] += t[2];
    state[3] += t[3];
    state[4] += e;
    state[5] += t[5];
    state[6] += t[6];
    state[7] += t[7];
}
*/

#undef S0
#undef S1
#undef s0
#undef s1
#undef R

// ************************************************************

#ifdef WORD64_AVAILABLE

void SHA384::InitState(HashWordType *state)
{
static const word64 s[8] = {
W64LIT(0xcbbb9d5dc1059ed8), W64LIT(0x629a292a367cd507),
W64LIT(0x9159015a3070dd17), W64LIT(0x152fecd8f70e5939),
W64LIT(0x67332667ffc00b31), W64LIT(0x8eb44a8768581511),
W64LIT(0xdb0c2e0d64f98fa7), W64LIT(0x47b5481dbefa4fa4)};
memcpy(state, s, sizeof(s));
}

void SHA512::InitState(HashWordType *state)
{
static const word64 s[8] = {
W64LIT(0x6a09e667f3bcc908), W64LIT(0xbb67ae8584caa73b),
W64LIT(0x3c6ef372fe94f82b), W64LIT(0xa54ff53a5f1d36f1),
W64LIT(0x510e527fade682d1), W64LIT(0x9b05688c2b3e6c1f),
W64LIT(0x1f83d9abfb41bd6b), W64LIT(0x5be0cd19137e2179)};
memcpy(state, s, sizeof(s));
}

```

SHA-512 round constants. 80 entries (SHA-512 runs 80 rounds versus SHA-256's 64), each a 64-bit word derived from the cube roots of the first 80 primes. The same nothing-up-my-sleeve derivation, extended to 64-bit precision. Bitcoin does not use SHA-512 — it appears here because Crypto++ 5.5.2 was included wholesale. Ethereum later adopted Keccak-256—the pre-standardization variant of what became SHA-3—a completely different construction from the NIST SHA-2 family.

```
CRYPTOPP_ALIGN_DATA(16) static const word64 SHA512_K[80] CRYPTOPP_SECTION_ALIGN16 = {
W64LIT(0x428a2f98d728ae22), W64LIT(0x7137449123ef65cd),
W64LIT(0xb5c0fbcfec4d3b2f), W64LIT(0xe9b5dba58189dbbc),
W64LIT(0x3956c25bf348b538), W64LIT(0x59f111f1b605d019),
W64LIT(0x923f82a4af194f9b), W64LIT(0xab1c5ed5da6d8118),
W64LIT(0xd807aa98a3030242), W64LIT(0x12835b0145706fbe),
W64LIT(0x243185be4ee4b28c), W64LIT(0x550c7dc3d5ffb4e2),
W64LIT(0x72be5d74f27b896f), W64LIT(0x80deb1fe3b1696b1),
W64LIT(0x9bdc06a725c71235), W64LIT(0xc19bf174cf692694),
W64LIT(0xe49b69c19ef14ad2), W64LIT(0xefbe4786384f25e3),
W64LIT(0x0fc19dc68b8cd5b5), W64LIT(0x240ca1cc77ac9c65),
```

```
282 W64LIT(0x2de92c6f592b0275), W64LIT(0x4a7484aa6ea6e483),
283 W64LIT(0x5cb0a9dcbd41fbd4), W64LIT(0x76f988da831153b5),
284 W64LIT(0x983e5152ee66dfab), W64LIT(0xa831c66d2db43210),
285 W64LIT(0xb00327c898fb213f), W64LIT(0xbf597fc7beef0ee4),
286 W64LIT(0xc6e00bf33da88fc2), W64LIT(0xd5a79147930aa725),
287 W64LIT(0x06ca6351e003826f), W64LIT(0x142929670a0e6e70),
288 W64LIT(0x27b70a8546d22ffc), W64LIT(0x2e1b21385c26c926),
289 W64LIT(0x4d2c6dfc5ac42aed), W64LIT(0x53380d139d95b3df),
290 W64LIT(0x650a73548baf63de), W64LIT(0x766a0abb3c77b2a8),
291 W64LIT(0x81c2c92e47edaee6), W64LIT(0x92722c851482353b),
292 W64LIT(0xa2bfe8a14cf10364), W64LIT(0xa81a664bbc423001),
293 W64LIT(0xc24b8b70d0f89791), W64LIT(0xc76c51a30654be30),
294 W64LIT(0xd192e819d6ef5218), W64LIT(0xd69906245565a910),
295 W64LIT(0xf40e35855771202a), W64LIT(0x106aa07032bbd1b8),
296 W64LIT(0x19a4c116b8d2d0c8), W64LIT(0x1e376c085141ab53),
297 W64LIT(0x2748774cdf8eeb99), W64LIT(0x34b0bcb5e19b48a8),
298 W64LIT(0x391c0cb3c5c95a63), W64LIT(0x4ed8aa4ae3418acb),
299 W64LIT(0x5b9cca4f7763e373), W64LIT(0x682e6ff3d6b2b8a3),
300 W64LIT(0x748f82ee5defb2fc), W64LIT(0x78a5636f43172f60),
301 W64LIT(0x84c87814a1f0ab72), W64LIT(0x8cc702081a6439ec),
302 W64LIT(0x90befffa23631e28), W64LIT(0xa4506cebde82bde9),
303 W64LIT(0xbef9a3f7b2c67915), W64LIT(0xc67178f2e372532b),
304 W64LIT(0xca273eceea26619c), W64LIT(0xd186b8c721c0c207),
305 W64LIT(0xeada7dd6cde0eb1e), W64LIT(0xf57d4f7fee6ed178),
306 W64LIT(0x06f067aa72176fba), W64LIT(0x0a637dc5a2c898a6),
307 W64LIT(0x113f9804bef90dae), W64LIT(0x1b710b35131c471b),
308 W64LIT(0x28db77f523047d84), W64LIT(0x32caab7b40c72493),
309 W64LIT(0x3c9ebe0a15c9bebc), W64LIT(0x431d67c49c100d4c),
310 W64LIT(0x4cc5d4becb3e42b6), W64LIT(0x597f299cfc657e2a),
311 W64LIT(0x5fcb6fab3ad6faec), W64LIT(0x6c44198c4a475817)
312 };
313
314 #if CRYPTOPP_BOOL_SSE2_ASM_AVAILABLE && CRYPTOPP_BOOL_X86
315 // put assembly version in separate function, otherwise MSVC 2005 SP1 doesn't generate correct code for
    the non-assembly version
```

SSE2-accelerated SHA-512: inline assembly for x86. Wei Dai wrote a hand-optimized MMX/SSE2 implementation that operates on two 64-bit words simultaneously using 128-bit XMM registers. The macros SSE2_S0_S1, SSE2_s0, SSE2_s1 implement the sigma functions entirely in SIMD instructions. This is 2007-era performance engineering — before compiler auto-vectorization was reliable, hand-written assembly was the only way to fully exploit Intel's vector units. Bitcoin v0.1.0 ships this code but never calls it.

```
    CALLED FROM  SHA512::Transform p. 458
316 CRYPTOPP_NAKED static void CRYPTOPP_FASTCALL SHA512_SSE2_Transform(word64 *state, const word64 *data)
317 {
318 #ifdef __GNUC__
319     __asm__ __volatile__
320 (
321 ".intel_syntax noprefix;"
322 AS1(    push    ebx)
323 AS2(    mov     ebx, eax)
324 #else
325 AS1(    push    ebx)
326 AS1(    push    esi)
327 AS1(    push    edi)
328 AS2(    lea     ebx, SHA512_K)
329 #endif
330
331 AS2(    mov     eax, esp)
332 AS2( and     esp, 0xffffff0)
333 AS2(    sub     esp, 27*16) // 17*16 for expanded data, 20*8 for state
334 AS1(    push    eax)
335 AS2( xor     eax, eax)
336 AS2(    lea     edi, [esp+4+8*8]) // start at middle of state buffer. will decrement pointer each round
    to avoid copying
337 AS2(    lea     esi, [esp+4+20*8+8]) // 16-byte alignment, then add 8
```

```
338
339 AS2(    movq    mm4, [ecx+0*8])
340 AS2(    movq    [edi+0*8], mm4)
341 AS2(    movq    mm0, [ecx+1*8])
342 AS2(    movq    [edi+1*8], mm0)
343 AS2(    movq    mm0, [ecx+2*8])
344 AS2(    movq    [edi+2*8], mm0)
345 AS2(    movq    mm0, [ecx+3*8])
346 AS2(    movq    [edi+3*8], mm0)
347 AS2(    movq    mm5, [ecx+4*8])
348 AS2(    movq    [edi+4*8], mm5)
349 AS2(    movq    mm0, [ecx+5*8])
350 AS2(    movq    [edi+5*8], mm0)
351 AS2(    movq    mm0, [ecx+6*8])
352 AS2(    movq    [edi+6*8], mm0)
353 AS2(    movq    mm0, [ecx+7*8])
354 AS2(    movq    [edi+7*8], mm0)
355 ASJ(    jmp, 0, f)
356
357 #define SSE2_S0_S1(r, a, b, c)\
358 AS2(    movq    mm6, r)\
359 AS2(    psrlq   r, a)\
360 AS2(    movq    mm7, r)\
361 AS2(    psllq   mm6, 64-c)\
362 AS2(    pxor    mm7, mm6)\
363 AS2(    psrlq   r, b-a)\
364 AS2(    pxor    mm7, r)\
365 AS2(    psllq   mm6, c-b)\
366 AS2(    pxor    mm7, mm6)\
367 AS2(    psrlq   r, c-b)\
368 AS2(    pxor    r, mm7)\
369 AS2(    psllq   mm6, b-a)\
370 AS2(    pxor    r, mm6)
371
372 #define SSE2_s0(r, a, b, c) \
373 AS2(    movdqa  xmm6, r)\
374 AS2(    psrlq   r, a)\
375 AS2(    movdqa  xmm7, r)\
376 AS2(    psllq   xmm6, 64-c)\
377 AS2(    pxor    xmm7, xmm6)\
378 AS2(    psrlq   r, b-a)\
379 AS2(    pxor    xmm7, r)\
380 AS2(    psrlq   r, c-b)\
381 AS2(    pxor    r, xmm7)\
382 AS2(    psllq   xmm6, c-a)\
383 AS2(    pxor    r, xmm6)
384
385 #define SSE2_s1(r, a, b, c) \
386 AS2(    movdqa  xmm6, r)\
387 AS2(    psrlq   r, a)\
388 AS2(    movdqa  xmm7, r)\
389 AS2(    psllq   xmm6, 64-c)\
390 AS2(    pxor    xmm7, xmm6)\
391 AS2(    psrlq   r, b-a)\
392 AS2(    pxor    xmm7, r)\
393 AS2(    psllq   xmm6, c-b)\
394 AS2(    pxor    xmm7, xmm6)\
395 AS2(    psrlq   r, c-b)\
396 AS2(    pxor    r, xmm7)
397
398 ASL(SHA512_Round)
399 // k + w is in mm0, a is in mm4, e is in mm5
400 AS2(    paddq   mm0, [edi+7*8]) // h
401 AS2(    movq    mm2, [edi+5*8]) // f
402 AS2(    movq    mm3, [edi+6*8]) // g
403 AS2(    pxor    mm2, mm3)
404 AS2(    pand    mm2, mm5)
405 SSE2_S0_S1(mm5,14,18,41)
406 AS2(    pxor    mm2, mm3)
407 AS2(    paddq   mm0, mm2) // h += Ch(e,f,g)
408 AS2(    paddq   mm5, mm0) // h += S1(e)
409 AS2(    movq    mm2, [edi+1*8]) // b
```

```
AS2(    movq    mm1, mm2)
AS2(    por     mm2, mm4)
AS2(    pand    mm2, [edi+2*8]) // c
AS2(    pand    mm1, mm4)
AS2(    por     mm1, mm2)
AS2(    paddq   mm1, mm5) // temp = h + Maj(a,b,c)
AS2(    paddq   mm5, [edi+3*8]) // e = d + h
AS2(    movq    [edi+3*8], mm5)
AS2(    movq    [edi+11*8], mm5)
SSE2_S0_S1(mm4,28,34,39) // S0(a)
AS2(    paddq   mm4, mm1) // a = temp + S0(a)
AS2(    movq    [edi-8], mm4)
AS2(    movq    [edi+7*8], mm4)
AS1(    ret)

// first 16 rounds
ASL(0)
AS2(    movq    mm0, [edx+eax*8])
AS2(    movq    [esi+eax*8], mm0)
AS2(    movq    [esi+eax*8+16*8], mm0)
AS2(    paddq   mm0, [ebx+eax*8])
ASC(    call,   SHA512_Round)
AS1(    inc     eax)
AS2(    sub     edi, 8)
AS2(    test    eax, 7)
ASJ(    jnz, 0, b)
AS2(    add     edi, 8*8)
AS2(    cmp     eax, 16)
ASJ(    jne, 0, b)

// rest of the rounds
AS2(    movdqu  xmm0, [esi+(16-2)*8])
ASL(1)
// data expansion, W[i-2] already in xmm0
AS2(    movdqu  xmm3, [esi])
AS2(    paddq   xmm3, [esi+(16-7)*8])
AS2(    movdqa  xmm2, [esi+(16-15)*8])
SSE2_s1(xmm0, 6, 19, 61)
AS2(    paddq   xmm0, xmm3)
SSE2_s0(xmm2, 1, 7, 8)
AS2(    paddq   xmm0, xmm2)
AS2(    movdq2q mm0, xmm0)
AS2(    movhlps xmm1, xmm0)
AS2(    paddq   mm0, [ebx+eax*8])
AS2(    movlps  [esi], xmm0)
AS2(    movlps  [esi+8], xmm1)
AS2(    movlps  [esi+8*16], xmm0)
AS2(    movlps  [esi+8*17], xmm1)
// 2 rounds
ASC(    call,   SHA512_Round)
AS2(    sub     edi, 8)
AS2(    movdq2q mm0, xmm1)
AS2(    paddq   mm0, [ebx+eax*8+8])
ASC(    call,   SHA512_Round)
// update indices and loop
AS2(    add     esi, 16)
AS2(    add     eax, 2)
AS2(    sub     edi, 8)
AS2(    test    eax, 7)
ASJ(    jnz, 1, b)
// do housekeeping every 8 rounds
AS2(    mov     esi, 0xf)
AS2( and     esi, eax)
AS2(    lea     esi, [esp+4+20*8+8+esi*8])
AS2(    add     edi, 8*8)
AS2(    cmp     eax, 80)
ASJ(    jne, 1, b)

#define SSE2_CombineState(i)\
AS2(    movq    mm0, [edi+i*8])\
AS2(    paddq   mm0, [ecx+i*8])\
AS2(    movq    [ecx+i*8], mm0)
```

```
482
483 SSE2_CombineState(0)
484 SSE2_CombineState(1)
485 SSE2_CombineState(2)
486 SSE2_CombineState(3)
487 SSE2_CombineState(4)
488 SSE2_CombineState(5)
489 SSE2_CombineState(6)
490 SSE2_CombineState(7)
491
492 AS1(    pop     esp)
493 AS1(    emms)
494
495 #if defined(__GNUC__)
496 AS1(    pop     ebx)
497 ".att_syntax prefix;"
498 :
499 : "a" (SHA512_K), "c" (state), "d" (data)
500 : "%esi", "%edi", "memory", "cc"
501 );
502 #else
503 AS1(    pop     edi)
504 AS1(    pop     esi)
505 AS1(    pop     ebx)
506 AS1(    ret)
507 #endif
508 }
509 #endif // #if CRYPTOPP_BOOL_SSE2_ASM_AVAILABLE
510
```

SHA-512 pure C++ fallback: same structure as SHA-256, 80 rounds, 64-bit words. S0/S1/s0/s1 are redefined here with SHA-512's rotation amounts (28/34/39 for S0; 14/18/41 for S1; 1/8/7 for s0; 19/61/6 for s1) — different from SHA-256's constants but the same algebraic form. The #undef/#define pattern means the macros from the SHA-256 section are replaced cleanly. SHA-512 processes 1024-bit blocks; SHA-256 processes 512-bit blocks. Otherwise the designs are parallel.

```
511 void SHA512::Transform(word64 *state, const word64 *data)
512 {
513 #if CRYPTOPP_BOOL_SSE2_ASM_AVAILABLE && CRYPTOPP_BOOL_X86
514 if (HasSSE2())
515 {
516 SHA512_SSE2_Transform(state, data);
517 return;
518 }
519 #endif
520
521 #define S0(x) (rotrFixed(x,28)^rotrFixed(x,34)^rotrFixed(x,39))
522 #define S1(x) (rotrFixed(x,14)^rotrFixed(x,18)^rotrFixed(x,41))
523 #define s0(x) (rotrFixed(x,1)^rotrFixed(x,8)^(x>>7))
524 #define s1(x) (rotrFixed(x,19)^rotrFixed(x,61)^(x>>6))
525
526 #define R(i) h(i)+=S1(e(i))+Ch(e(i),f(i),g(i))+SHA512_K[i+j]+(j?blk2(i):blk0(i));\
527 d(i)+=h(i);h(i)+=S0(a(i))+Maj(a(i),b(i),c(i))
528
529     word64 W[16];
530     word64 T[8];
531 /* Copy context->state[] to working vars */
532 memcpy(T, state, sizeof(T));
533 /* 80 operations, partially loop unrolled */
534 for (unsigned int j=0; j<80; j+=16)
535 {
536 R( 0); R( 1); R( 2); R( 3);
537 R( 4); R( 5); R( 6); R( 7);
538 R( 8); R( 9); R(10); R(11);
539 R(12); R(13); R(14); R(15);
540 }
541 /* Add the working vars back into context.state[] */
```

```
    state[0] += a(0);
    state[1] += b(0);
    state[2] += c(0);
    state[3] += d(0);
    state[4] += e(0);
    state[5] += f(0);
    state[6] += g(0);
    state[7] += h(0);
}

#endif

}

```

SUMMARY

`sha.cpp` holds the routine that found Bitcoin's first blocks: `SHA256::Transform`, the engine of v0.1.0's CPU miner. Validation never depended on this copy — nodes checked every hash with OpenSSL — and Bitcoin Core later replaced the Crypto++ code with its own implementation. What has never changed is the SHA-256 *algorithm* itself. Every block ever mined and every transaction ever confirmed rests on it, and any change to its output would instantly fork the network.

The file's structure reveals the engineering pragmatism of the era. Satoshi did not write hash functions; he imported Crypto++ 5.5.2.[1] Wei Dai did not write SHA-1; he adapted Steve Reid's public domain C. The dependency chain extends back to 1995, when Reid published his implementation. Bitcoin's cryptographic foundation rests on this accumulated public-domain work.

The "nothing up my sleeve" constants — initial values from square roots, round constants from cube roots — are the hash function community's answer to the backdoor question. They are the mathematical proof that the designers had no special knowledge of a weakness: anyone can verify the derivation, and any unexplained constant is a red flag. secp256k1 applied the same principle to elliptic curve parameters. Bitcoin's security assumptions are, wherever possible, publicly verifiable.

SHA-256 has held since its publication in 2001.[2] No practical preimage, second-preimage, or collision attack exists. The best known attacks reduce the effective round count from 64 to roughly 31—still 33 rounds of security margin. For Bitcoin, where a SHA-256 collision would enable double-spending at the Merkle tree level, this margin is the mathematical bedrock beneath every unspent output in every wallet on Earth.

KEY TERMS

SHA-256

The NIST hash standard (FIPS 180-2) that maps any input to 256 bits through 64 rounds of mixing, and underlies every block hash and transaction ID.

digest

The fixed-size output a hash function produces, which changes completely if even one input bit flips.

compression function

The core routine SHA256::Transform that mixes one 512-bit block into the running hash state, executed trillions of times per second across the mining network.

round constant

One of the 64 fixed 32-bit values added per round, each taken from the fractional cube root of a prime to keep the rounds from being self-similar.

nothing-up-my-sleeve number

A constant derived from a simple verifiable formula such as square roots of primes, proving the designers had no freedom to hide a weakness.

Merkle-Damgård

The construction that chains input blocks together by adding each block's output into the next block's starting state, so one input bit affects the entire final hash.

avalanche effect

The property that flipping a single input bit changes about half the output bits, which is why mining is a lottery with no partial progress.

message schedule

The expansion that stretches 16 words of input into the 64 words the rounds consume, so every input bit influences many rounds.

SHA-1

The older 160-bit hash implemented in this file's first half; Bitcoin never uses it, and it shipped only because the Crypto++ source came as a bundle.

SOURCES

1. Wei Dai, *Crypto++ Library* (1995–). SHA-256 implementation lineage referenced in sha.cpp comments. https://www.cryptopp.com/

2. NIST, *FIPS PUB 180-2 — Secure Hash Standard (SHA-256)* (2002). SHA-256 published August 2002 (draft 2001). https://csrc.nist.gov/publications/detail/fips/180/2/archive/2002-08-01
3. Xiaoyun Wang, Yiqun Lisa Yin, and Hongbo Yu, *Finding Collisions in the Full SHA-1* (2005). CRYPTO 2005; motivated moving away from SHA-1. https://www.iacr.org/archive/crypto2005/36210017/36210017.pdf

sha.h

178 lines CPP

THE ONE-WAY FUNCTION

This file contains the SHA-256 implementation—the cryptographic hash function that secures Bitcoin. SHA-256 (Secure Hash Algorithm, 256-bit) takes any input and produces a 256-bit fingerprint with remarkable properties: nearly impossible to reverse (one-way), tiny changes produce completely different outputs (avalanche effect), and finding two inputs with the same output is computationally infeasible (collision resistance).

Satoshi extracted this code from Crypto++, a well-respected open-source cryptography library by Wei Dai (who also invented b-money, a Bitcoin precursor).[12] The file is public domain, allowing unrestricted use. Rather than depending on the full Crypto++ library, Satoshi extracted just the SHA routines—keeping Bitcoin's dependencies minimal and auditable.

SHA-256 permeates Bitcoin: block hashes (double SHA-256), transaction IDs (double SHA-256), merkle trees (double SHA-256), address derivation (SHA-256 followed by RIPEMD-160), and proof-of-work (finding hashes below target). Mining is essentially a SHA-256 lottery—billions of attempts until one wins.

The performance of SHA-256 directly impacts mining profitability. The `Transform()` function here is called trillions of times per second across the Bitcoin network. Modern miners use custom ASICs that implement these exact operations in silicon, achieving efficiencies impossible in software.

Public domain dedication—no license restrictions. Satoshi borrowed from Wei Dai's Crypto++ library.[1] Wei Dai is also the creator of b-money (1998), which Satoshi cited in the Bitcoin whitepaper.[2][3] The cryptographic community is small; influences interconnect.

```
// This file is public domain
// SHA routines extracted as a standalone file from:
// Crypto++: a C++ Class Library of Cryptographic Schemes
// Version 5.5.2 (9/24/2007)
// http://www.cryptopp.com
#ifndef CRYPTOPP_SHA_H
#define CRYPTOPP_SHA_H
#include <stdlib.h>

namespace CryptoPP
{

//
// Dependencies
//

```

Type definitions for portability. In 2009, C++ lacked standardized fixed-width types (`uint32_t` came with C++11). These typedefs ensure `word32` is exactly 32 bits across compilers. SHA-256 operates on 32-bit words; incorrect sizes would produce wrong hashes—catastrophic for consensus.

```
typedef unsigned char byte;
typedef unsigned short word16;
typedef unsigned int word32;
#if defined(_MSC_VER) || defined(__BORLANDC__)
typedef unsigned __int64 word64;
#else
typedef unsigned long long word64;
#endif

```

`rotlFixed()` and `rotrFixed()`—bitwise rotation primitives. SHA-256's compression function heavily uses rotations: `rotr(x,2) XOR rotr(x,13) XOR rotr(x,22)` appears in the Σ_0 function. Many CPUs have dedicated rotation instructions; these templates let the compiler use them when available.

```
template <class T> inline T rotlFixed(T x, unsigned int y)
{
assert(y < sizeof(T)*8);
return T((x<<y) | (x>>(sizeof(T)*8-y)));
}

template <class T> inline T rotrFixed(T x, unsigned int y)
{
assert(y < sizeof(T)*8);
return T((x>>y) | (x<<(sizeof(T)*8-y)));
}

// ************** endian reversal ***************

```

`CRYPTOPP_FAST_ROTATE`—compiler-specific optimizations. Detects whether the target CPU has fast rotation instructions. Intel x86/x64, PowerPC, and modern ARM do; older architectures emulate rotations with shifts and ORs (slower). This macro guides code generation for optimal performance per platform.

```
40 #ifdef _MSC_VER
41 #if _MSC_VER >= 1400
42 #define CRYPTOPP_FAST_ROTATE(x) 1
43 #elif _MSC_VER >= 1300
44 #define CRYPTOPP_FAST_ROTATE(x) ((x) == 32 | (x) == 64)
45 #else
46 #define CRYPTOPP_FAST_ROTATE(x) ((x) == 32)
47 #endif
48 #elif (defined(__MWERKS__) && TARGET_CPU_PPC) || \
49 (defined(__GNUC__) && (defined(_ARCH_PWR2) || defined(_ARCH_PWR) || defined(_ARCH_PPC) ||
   defined(_ARCH_PPC64) || defined(_ARCH_COM)))
50 #define CRYPTOPP_FAST_ROTATE(x) ((x) == 32)
51 #elif defined(__GNUC__) && (CRYPTOPP_BOOL_X64 || CRYPTOPP_BOOL_X86) // depend on GCC's peephole
   optimization to generate rotate instructions
52 #define CRYPTOPP_FAST_ROTATE(x) 1
53 #else
54 #define CRYPTOPP_FAST_ROTATE(x) 0
55 #endif
56
57 inline byte ByteReverse(byte value)
58 {
59 return value;
60 }
61
62 inline word16 ByteReverse(word16 value)
63 {
64 #ifdef CRYPTOPP_BYTESWAP_AVAILABLE
65 return bswap_16(value);
66 #elif defined(_MSC_VER) && _MSC_VER >= 1300
67 return _byteswap_ushort(value);
68 #else
69 return rotlFixed(value, 8U);
70 #endif
71 }
72
```

`ByteReverse()`—endianness conversion. SHA-256 is defined using big-endian (network byte order), but most modern CPUs are little-endian. This function swaps byte order. The GCC inline assembly `bswap %0` uses the single-instruction byte swap on x86—crucial when called billions of times during mining.

```
73 inline word32 ByteReverse(word32 value)
74 {
75 #if defined(__GNUC__)
76 __asm__ ("bswap %0" : "=r" (value) : "0" (value));
77 return value;
78 #elif defined(CRYPTOPP_BYTESWAP_AVAILABLE)
79 return bswap_32(value);
80 #elif defined(__MWERKS__) && TARGET_CPU_PPC
81 return (word32)__lwbrx(&value,0);
82 #elif _MSC_VER >= 1400 || (_MSC_VER >= 1300 && !defined(_DLL))
83 return _byteswap_ulong(value);
84 #elif CRYPTOPP_FAST_ROTATE(32)
85 // 5 instructions with rotate instruction, 9 without
86 return (rotrFixed(value, 8U) & 0xff00ff00) | (rotlFixed(value, 8U) & 0x00ff00ff);
87 #else
88 // 6 instructions with rotate instruction, 8 without
89     value = ((value & 0xFF00FF00) >> 8) | ((value & 0x00FF00FF) << 8);
90 return rotlFixed(value, 16U);
```

```
#endif
}

#ifdef WORD64_AVAILABLE
inline word64 ByteReverse(word64 value)
{
#if defined(__GNUC__) && defined(__x86_64__)
__asm__ ("bswap %0" : "=r" (value) : "0" (value));
return value;
#elif defined(CRYPTOPP_BYTESWAP_AVAILABLE)
return bswap_64(value);
#elif defined(_MSC_VER) && _MSC_VER >= 1300
return _byteswap_uint64(value);
#elif defined(CRYPTOPP_SLOW_WORD64)
return (word64(ByteReverse(word32(value))) << 32) | ByteReverse(word32(value>>32));
#else
    value = ((value & W64LIT(0xFF00FF00FF00FF00)) >> 8) | ((value & W64LIT(0x00FF00FF00FF00FF)) << 8);
    value = ((value & W64LIT(0xFFFF0000FFFF0000)) >> 16) | ((value & W64LIT(0x0000FFFF0000FFFF)) << 16);
return rotlFixed(value, 32U);
#endif
}
#endif


//
// SHA
//

// http://www.weidai.com/scan-mirror/md.html#SHA-1
```

SHA1—160-bit hash, predecessor to SHA-256. Bitcoin's block and transaction hashing never touches SHA1—addresses are RIPEMD160(SHA256(pubkey)), and RIPEMD160 is an independent design, not built on SHA1. Script does expose it as the `OP_SHA1` opcode (script.cpp:667, via OpenSSL), still valid today; this Crypto++ copy is unused. SHA1 is considered cryptographically weak (broken theoretically in 2005,[4] with a full collision demonstrated by Google in 2017), but Bitcoin never relied on it—only on SHA256's collision resistance.

```
class SHA1
{
public:
typedef word32 HashWordType;
static void InitState(word32 *state);
static void Transform(word32 *digest, const word32 *data);
static const char * StaticAlgorithmName() {return "SHA-1";}
};

typedef SHA1 SHA; // for backwards compatibility

// implements the SHA-256 standard
```

SHA256—the cryptographic heart of Bitcoin. The `Transform()` method processes 512-bit blocks through 64 rounds of mixing, producing 256-bit output. `InitState()` sets the initial hash values (derived from the fractional parts of square roots of the first 8 primes). This is NIST FIPS 180-2's SHA-256, unchanged since standardization in 2001.[5]

```
class SHA256
{
public:
typedef word32 HashWordType;
static void InitState(word32 *state);
```

`Transform()`—the compression function, called once per 64-byte block of input. Bitcoin mining is essentially: set up a block header (80 bytes, which pads to two 64-byte blocks), call Transform three times—twice for the first SHA-256, once for the second—check if the result is below target, increment nonce, repeat. That is exactly what v0.1.0's `BitcoinMiner` does. ASICs implement Transform directly in hardware—billions of transforms per second per chip.

```
137 static void Transform(word32 *digest, const word32 *data);
138 static const char * StaticAlgorithmName() {return "SHA-256";}
139 };
140
141 // implements the SHA-224 standard
```

`SHA224`—224-bit variant, same Transform as SHA256 but different initial values and truncated output. Not used in Bitcoin. Its presence here is incidental (came with the Crypto++ extraction). Demonstrates how SHA-2 family shares implementations.

```
142 class SHA224
143 {
144 public:
145 typedef word32 HashWordType;
146 static void InitState(word32 *state);
147 static void Transform(word32 *digest, const word32 *data) {SHA256::Transform(digest, data);}
148 static const char * StaticAlgorithmName() {return "SHA-224";}
149 };
150
151 #ifdef WORD64_AVAILABLE
152
153 // implements the SHA-512 standard
```

`SHA512`—512-bit hash using 64-bit words. Potentially useful for future cryptographic needs (key derivation, HMAC). Not used in Bitcoin's consensus rules, but available in this implementation. SHA-512 is actually faster than SHA-256 on 64-bit CPUs due to better parallelism.

```
154 class SHA512
155 {
156 public:
157 typedef word64 HashWordType;
158 static void InitState(word64 *state);
159 static void Transform(word64 *digest, const word64 *data);
160 static const char * StaticAlgorithmName() {return "SHA-512";}
161 };
162
163 // implements the SHA-384 standard
164 class SHA384
165 {
166 public:
167 typedef word64 HashWordType;
168 static void InitState(word64 *state);
169 static void Transform(word64 *digest, const word64 *data) {SHA512::Transform(digest, data);}
170 static const char * StaticAlgorithmName() {return "SHA-384";}
171 };
172
173 #endif
174
175 }
176
177 #endif
178
```

SUMMARY

SHA-256's security properties are what make Bitcoin work. The one-way nature means you can't reverse-engineer a private key from a public key hash. The avalanche effect means changing one bit in a block header completely changes the hash—no gradual approach to a valid proof-of-work. Collision resistance means attackers can't create two transactions with the same txid.

The computation Transform performs is the most-executed algorithm in human history. Today's network performs hundreds of exahashes per second—each hash a double SHA-256 needing several compression-function calls—so the Transform operation runs on the order of 10^{21} times every second, now in ASIC silicon rather than this C++. No other computation comes close. This relentless hashing secures over a trillion dollars in value.

Satoshi's choice of double-SHA256 (hashing twice) adds a safety margin. SHA-256's theoretical weakness to length-extension attacks doesn't apply when the result is hashed again. Whether Satoshi knew this or just chose defense-in-depth, the double-hash has proven robust for over seventeen years.

KEY TERMS

hash function

A function mapping any input to a fixed-size fingerprint that is easy to compute but infeasible to reverse.

preimage resistance

The property that given a hash output you cannot find an input producing it, which makes proof-of-work a lottery rather than a puzzle to solve.

collision resistance

The property that no one can find two different inputs with the same hash, keeping transaction IDs and block hashes unique.

double-SHA256

Bitcoin's standard construction SHA256(SHA256(data)), used for block hashes and transaction IDs as a safety margin against length-extension attacks.

RIPEMD-160

The 160-bit hash applied after SHA-256 when deriving addresses, an independent second design so addresses never depend on one algorithm alone.

endianness

The byte-order convention of a machine; SHA-256 is defined big-endian while most CPUs are little-endian, so ByteReverse swaps bytes before hashing.

rotation

A bit shift in which bits pushed off one end reappear at the other; SHA-256's mixing functions are XORs of rotations such as rotr(x,2), rotr(x,13), and rotr(x,22).

ASIC

An application-specific integrated circuit; modern miners are chips that perform this file's Transform routine in silicon billions of times per second.

SOURCES

1. Wei Dai, *Crypto++ Library* (1995–). SHA-256 implementation lineage referenced in sha.cpp comments. https://www.cryptopp.com/
2. Wei Dai, *b-money* (1998). http://www.weidai.com/bmoney.txt
3. Satoshi Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (October 2008). https://bitcoin.org/bitcoin.pdf

4. Xiaoyun Wang, Yiqun Lisa Yin, and Hongbo Yu, *Finding Collisions in the Full SHA-1* (2005). CRYPTO 2005; motivated moving away from SHA-1. https://www.iacr.org/archive/crypto2005/36210017/36210017.pdf
5. NIST, *FIPS PUB 180-2 — Secure Hash Standard (SHA-256)* (2002). SHA-256 published August 2002 (draft 2001). https://csrc.nist.gov/publications/detail/fips/180/2/archive/2002-08-01

ui.cpp

3229 lines CPP

THE FACE OF BITCOIN

At 3,228 lines, ui.cpp is the largest file in Bitcoin v0.1.0. It implements the entire graphical user interface using wxWidgets — a cross-platform GUI toolkit that lets the same C++ code run on Windows, macOS, and Linux.[1] This is where the abstract machinery of cryptography, networking, and consensus becomes a window a person can interact with.

The original Bitcoin client looks deceptively simple: a transaction list, a balance, a "Send Coins" button, and a status bar showing connections and blocks. But beneath this surface, the UI handles a genuinely difficult problem: presenting a live, multithreaded financial system to a user who expects a coherent, responsive interface.

The fundamental challenge is thread safety. Bitcoin's networking and block processing run in background threads that constantly update shared state. The UI must display this state without stalling the network threads or showing inconsistent data. Satoshi's solution is a combination of TRY_CRITICAL_BLOCK (non-blocking mutex attempts), wxYield() to let the event loop breathe during bulk operations, and a cross-thread event system to safely dispatch UI updates from worker threads.

The file also contains Bitcoin v0.1.0's most ambitious unfinished feature: an IP-to-IP payment protocol where the sender connects directly to the recipient's node, requests a fresh public key, and delivers a signed transaction over the peer-to-peer network. This "online" payment mode coexisted with the now-standard Bitcoin address (offline) mode. It was ultimately abandoned — privacy, NAT traversal, and the need to be online to receive made it impractical — but its traces remain throughout the code.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

#include "headers.h"
#ifdef _MSC_VER
#include <crtdbg.h>
#endif


```

wxWidgets custom event types. CROSSTHREADCALL allows worker threads to post events to the main UI thread safely. REPLY1/2/3 carry network response payloads to dialog handlers. TABLE_ADDED/UPDATED/DELETED were for the marketplace feature.

```
DEFINE_EVENT_TYPE(wxEVT_CROSSTHREADCALL)
DEFINE_EVENT_TYPE(wxEVT_REPLY1)
DEFINE_EVENT_TYPE(wxEVT_REPLY2)
DEFINE_EVENT_TYPE(wxEVT_REPLY3)
DEFINE_EVENT_TYPE(wxEVT_TABLEADDED)
DEFINE_EVENT_TYPE(wxEVT_TABLEUPDATED)
DEFINE_EVENT_TYPE(wxEVT_TABLEDELETED)

```

`pframeMain` — the global singleton for the main window. Worker threads check this pointer before posting UI events; if NULL, the window has been destroyed and no callbacks should fire. The networking and wallet threads indirectly use this to trigger UI repaints.

```
CMainFrame* pframeMain = NULL;
```

`mapAddressBook` — the human-readable address book. Maps Bitcoin addresses (base58 strings) to contact names. Stored in wallet.dat as simple name records, separate from the keys themselves; allows labeling received addresses with names like "Alice" or "Donation pool".

```
map<string, string> mapAddressBook;


void ThreadRequestProductDetails(void* parg);
void ThreadRandSendTest(void* parg);
bool fRandSendTest = false;
void RandSend();




////////////////////////////////////////////////////////////////////////////
//
// Util
//
```

CALLED FROM CMainFrame (inline) ui.h p. 525 · CSendDialog (inline) ui.h p. 527 · CProductsDialog (inline) ui.h p. 529 · CEditProductDialog (inline) ui.h p. 529 · CEditReviewDialog (inline) ui.h p. 530

```
void HandleCtrlA(wxKeyEvent& event)
{
// Ctrl-a select all
    wxTextCtrl* textCtrl = (wxTextCtrl*)event.GetEventObject();
if (event.GetModifiers() == wxMOD_CONTROL && event.GetKeyCode() == 'A')
        textCtrl->SetSelection(-1, -1);
    event.Skip();
}

```

CALLED FROM DateTimeStr p. 477

```
bool Is24HourTime()
{
//char pszHourFormat[256];
//pszHourFormat[0] = '\0';
```

```
return true;
}
```

CALLED FROM CViewProductDialog::UpdateProductDisplay p. 511

```
string DateStr(int64 nTime)
{
return (string)wxDateTime((time_t)nTime).FormatDate();
}
```

CALLED FROM PrintBlockTree main.cpp p. 229 · CMainFrame::CMainFrame p. 480 · CMainFrame::InsertTransaction p. 484 · CTxDetailsDialog::CTxDetailsDialog p. 491 · CViewOrderDialog::CViewOrderDialog p. 513

```
string DateTimeStr(int64 nTime)
{
    wxDateTime datetime((time_t)nTime);
if (Is24HourTime())
return (string)datetime.Format("%x %H:%M");
else
return (string)datetime.Format("%x ") + itostr((datetime.GetHour() + 11) % 12 + 1) + (string)datetime.Format(":%M %p");
}
```

CALLED FROM CMainFrame::InsertLine p. 482 · CMainFrame::RefreshStatus p. 486 · CMainFrame::OnListItemActivatedAllTransactions p. 490 · CYourAddressDialog::GetAddress p. 502 · CYourAddressDialog::OnListEndLabelEdit p. 502 · CYourAddressDialog::OnButtonRename p. 502 · *and 4 more*

```
wxString GetItemText(wxListCtrl* listCtrl, int nIndex, int nColumn)
{
// Helper to simplify access to listctrl
    wxListItem item;
    item.m_itemId = nIndex;
    item.m_col = nColumn;
    item.m_mask = wxLIST_MASK_TEXT;
if (!listCtrl->GetItem(item))
return "";
return item.GetText();
}

int InsertLine(wxListCtrl* listCtrl, const wxString& str0, const wxString& str1)
{
int nIndex = listCtrl->InsertItem(listCtrl->GetItemCount(), str0);
    listCtrl->SetItem(nIndex, 1, str1);
return nIndex;
}

int InsertLine(wxListCtrl* listCtrl, const wxString& str0, const wxString& str1, const wxString& str2, const wxString& str3,
const wxString& str4)
{
int nIndex = listCtrl->InsertItem(listCtrl->GetItemCount(), str0);
    listCtrl->SetItem(nIndex, 1, str1);
    listCtrl->SetItem(nIndex, 2, str2);
    listCtrl->SetItem(nIndex, 3, str3);
    listCtrl->SetItem(nIndex, 4, str4);
return nIndex;
}

int InsertLine(wxListCtrl* listCtrl, void* pdata, const wxString& str0, const wxString& str1, const wxString& str2, const
wxString& str3, const wxString& str4)
{
int nIndex = listCtrl->InsertItem(listCtrl->GetItemCount(), str0);
    listCtrl->SetItemPtrData(nIndex, (wxUIntPtr)pdata);
    listCtrl->SetItem(nIndex, 1, str1);
    listCtrl->SetItem(nIndex, 2, str2);
    listCtrl->SetItem(nIndex, 3, str3);
    listCtrl->SetItem(nIndex, 4, str4);
return nIndex;
}
```

CALLED FROM HandleCtrlA p. 476 · CMainFrame::CMainFrame p. 480 · CMainFrame::OnSetFocusAddress p. 490 · CMainFrame::OnMouseEventsAddress p. 490 · CSendDialog::CSendDialog p. 495 · CYourAddressDialog::OnButtonNew p. 503 · *and 4 more*

```
void SetSelection(wxListCtrl* listCtrl, int nIndex)
{
int nSize = listCtrl->GetItemCount();
long nState = (wxLIST_STATE_SELECTED|wxLIST_STATE_FOCUSED);
for (int i = 0; i < nSize; i++)
        listCtrl->SetItemState(i, (i == nIndex ? nState : 0), nState);
}
```

CALLED FROM CYourAddressDialog::GetAddress p. 502 · CYourAddressDialog::OnButtonRename p. 502 · CAddressBookDialog::GetAddress p. 504 · CAddressBookDialog::OnButtonEdit p. 504 · CEditReviewDialog::OnButtonSubmit p. 514 · CEditReviewDialog::GetReview p. 514

```
int GetSelection(wxListCtrl* listCtrl)
{
int nSize = listCtrl->GetItemCount();
for (int i = 0; i < nSize; i++)
```

```
return i;
return -1;
}


string HtmlEscape(const char* psz, bool fMultiLine=false)
{
int len = 0;
for (const char* p = psz; *p; p++)
{
if (*p == '<') len += 4;
else if (*p == '>') len += 4;
else if (*p == '&') len += 5;
else if (*p == '"') len += 6;
else if (*p == ' ' && p > psz && p[-1] == ' ' && p[1] == ' ') len += 6;
else if (*p == '\n' && fMultiLine) len += 5;
else
            len++;
}
    string str;
    str.reserve(len);
for (const char* p = psz; *p; p++)
{
if (*p == '<') str += "&lt;";
else if (*p == '>') str += "&gt;";
else if (*p == '&') str += "&amp;";
else if (*p == '"') str += "&quot;";
else if (*p == ' ' && p > psz && p[-1] == ' ' && p[1] == ' ') str += " ";
else if (*p == '\n' && fMultiLine) str += "<br>\n";
else
            str += *p;
}
return str;
}

string HtmlEscape(const string& str, bool fMultiLine=false)
{
return HtmlEscape(str.c_str(), fMultiLine);
}

    CALLED FROM  CEditProductDialog::OnButtonSend p. 508
void AddToMyProducts(CProduct product)
{
    CProduct& productInsert = mapMyProducts[product.GetHash()];
    productInsert = product;
InsertLine(pframeMain->m_listCtrlProductsSent, &productInsert,
                product.mapValue["category"],
                product.mapValue["title"].substr(0, 100),
                product.mapValue["description"].substr(0, 100),
                product.mapValue["price"],
"");
}




////////////////////////////////////////////////////////////////////////////////
//
// Custom events
//

```

`setCallbackAvailable` — a thread-safe registry of dialog pointers that are still alive. When a network response arrives, the handler first checks IsCallbackAvailable() before calling the dialog method. If the dialog was closed while the network call was in flight, the pointer is no longer in the set and the callback is safely dropped.

```
set<void*> setCallbackAvailable;
CCriticalSection cs_setCallbackAvailable;

    CALLED FROM  CViewProductDialog::CViewProductDialog p. 509
void AddCallbackAvailable(void* p)
{
CRITICAL_BLOCK(cs_setCallbackAvailable)
        setCallbackAvailable.insert(p);
}

    CALLED FROM  CViewProductDialog::~CViewProductDialog p. 510
void RemoveCallbackAvailable(void* p)
```

```
{
CRITICAL_BLOCK(cs_setCallbackAvailable)
        setCallbackAvailable.erase(p);
}

```

CALLED FROM AddPendingReplyEvent1 p. 479 · AddPendingReplyEvent2 p. 479 · AddPendingReplyEvent3 p. 479

```
bool IsCallbackAvailable(void* p)
{
CRITICAL_BLOCK(cs_setCallbackAvailable)
return setCallbackAvailable.count(p);
return false;
}

```

`AddPendingCustomEvent` — the cross-thread messaging bridge. Worker threads cannot call wxWidgets methods directly (not thread-safe). Instead, they serialize data into a wxCommandEvent string payload and add it to the event queue. The UI thread processes it on its next iteration. This is a producer/consumer queue built on wxWidgets' existing event system.

```
template<typename T>
void AddPendingCustomEvent(wxEvtHandler* pevthandler, int nEventID, const T pbeginIn, const T pendIn)
{
if (!pevthandler)
return;

const char* pbegin = (pendIn != pbeginIn) ? &pbeginIn[0] : NULL;
const char* pend = pbegin + (pendIn - pbeginIn) * sizeof(pbeginIn[0]);
    wxCommandEvent event(nEventID);
    wxString strData(wxChar(0), (pend - pbegin) / sizeof(wxChar) + 1);
memcpy(&strData[0], pbegin, pend - pbegin);
    event.SetString(strData);
    event.SetInt(pend - pbegin);

    pevthandler->AddPendingEvent(event);
}

template<class T>
void AddPendingCustomEvent(wxEvtHandler* pevthandler, int nEventID, const T& obj)
{
    CDataStream ss;
    ss << obj;
AddPendingCustomEvent(pevthandler, nEventID, ss.begin(), ss.end());
}

```

CALLED FROM ThreadRequestProductDetails p. 510

```
void AddPendingReplyEvent1(void* pevthandler, CDataStream& vRecv)
{
if (IsCallbackAvailable(pevthandler))
AddPendingCustomEvent((wxEvtHandler*)pevthandler, wxEVT_REPLY1, vRecv.begin(), vRecv.end());
}

void AddPendingReplyEvent2(void* pevthandler, CDataStream& vRecv)
{
if (IsCallbackAvailable(pevthandler))
AddPendingCustomEvent((wxEvtHandler*)pevthandler, wxEVT_REPLY2, vRecv.begin(), vRecv.end());
}

void AddPendingReplyEvent3(void* pevthandler, CDataStream& vRecv)
{
if (IsCallbackAvailable(pevthandler))
AddPendingCustomEvent((wxEvtHandler*)pevthandler, wxEVT_REPLY3, vRecv.begin(), vRecv.end());
}

```

CALLED FROM CViewProductDialog::OnReply1 p. 510

```
CDataStream GetStreamFromEvent(const wxCommandEvent& event)
{
    wxString strData = event.GetString();
return CDataStream(strData.begin(), strData.begin() + event.GetInt(), SER_NETWORK);
}









//////////////////////////////////////////////////////////////////////////////
//
// CMainFrame
//

```

`CMainFrame` constructor — wires up the main window. Creates the toolbar with masked bitmap icons loaded from Windows resources, sets up 7 columns in the transaction list (hidden sort key, hidden hash, Status, Date, Description, Debit, Credit), and calls RefreshListCtrl() to populate from the wallet database.

```
CMainFrame::CMainFrame(wxWindow* parent) : CMainFrameBase(parent)
{
Connect(wxEVT_CROSSTHREADCALL, wxCommandEventHandler(CMainFrame::OnCrossThreadCall), NULL, this);

// Init
    fRefreshListCtrl = false;
    fRefreshListCtrlRunning = false;
    fOnSetFocusAddress = false;
    pindexBestLast = NULL;
    m_choiceFilter->SetSelection(0);
```

Initial balance display: reads from GetBalance() (sum of all unspent wallet outputs), formatted via FormatMoney() as a decimal BTC value. The trailing spaces prevent the label from being cut off when the number grows longer.

```
    m_staticTextBalance->SetLabel(FormatMoney(GetBalance()) + "  ");
    m_listCtrl->SetFocus();
SetIcon(wxICON(bitcoin));
```

`m_menuOptions->Check` — reflects the saved fGenerateBitcoins setting in the menu checkbox. If the user was mining when they last quit, the menu appears checked immediately on startup before the miner thread is launched.

```
    m_menuOptions->Check(wxID_OPTIONSGENERATEBITCOINS, fGenerateBitcoins);

// Init toolbar with transparency masked bitmaps
    m_toolBar->ClearTools();

//// shouldn't have to do mask separately anymore, bitmap alpha support added in wx 2.8.9,
    wxBitmap bmpSend(wxT("send20"), wxBITMAP_TYPE_RESOURCE);
    bmpSend.SetMask(new wxMask(wxBitmap(wxT("send20mask"), wxBITMAP_TYPE_RESOURCE)));
    m_toolBar->AddTool(wxID_BUTTONSEND, wxT("&Send Coins"), bmpSend, wxNullBitmap, wxITEM_NORMAL, wxEmptyString, wxEmptyString);

    wxBitmap bmpAddressBook(wxT("addressbook20"), wxBITMAP_TYPE_RESOURCE);
    bmpAddressBook.SetMask(new wxMask(wxBitmap(wxT("addressbook20mask"), wxBITMAP_TYPE_RESOURCE)));
    m_toolBar->AddTool(wxID_BUTTONRECEIVE, wxT("&Address Book"), bmpAddressBook, wxNullBitmap, wxITEM_NORMAL, wxEmptyString,
wxEmptyString);

    m_toolBar->Realize();

// Init column headers
```

Date column width is computed dynamically from the formatted width of a specific timestamp (1229413914 = December 16, 2008 — weeks before Bitcoin launched). Multiplying by 6 approximates pixels-per-character for the monospace font. This ensures the column fits dates in any locale's format.

```
int nDateWidth = DateTimeStr(1229413914).size() * 6 + 8;
    m_listCtrl->InsertColumn(0, "",             wxLIST_FORMAT_LEFT, 0);
    m_listCtrl->InsertColumn(1, "",             wxLIST_FORMAT_LEFT, 0);
    m_listCtrl->InsertColumn(2, "Status",       wxLIST_FORMAT_LEFT, 90);
    m_listCtrl->InsertColumn(3, "Date",         wxLIST_FORMAT_LEFT,   nDateWidth);
    m_listCtrl->InsertColumn(4, "Description",  wxLIST_FORMAT_LEFT, 409 - nDateWidth);
    m_listCtrl->InsertColumn(5, "Debit",        wxLIST_FORMAT_RIGHT, 79);
    m_listCtrl->InsertColumn(6, "Credit",       wxLIST_FORMAT_RIGHT, 79);

//m_listCtrlProductsSent->InsertColumn(0, "Category",      wxLIST_FORMAT_LEFT,  100);
//m_listCtrlProductsSent->InsertColumn(1, "Title",         wxLIST_FORMAT_LEFT,  100);
//m_listCtrlProductsSent->InsertColumn(2, "Description",   wxLIST_FORMAT_LEFT,  100);
```

```
//m_listCtrlProductsSent->InsertColumn(3, "Price",         wxLIST_FORMAT_LEFT,  100);
//m_listCtrlProductsSent->InsertColumn(4, "",              wxLIST_FORMAT_LEFT,  100);

//m_listCtrlOrdersSent->InsertColumn(0, "Time",          wxLIST_FORMAT_LEFT,  100);
//m_listCtrlOrdersSent->InsertColumn(1, "Price",         wxLIST_FORMAT_LEFT,  100);
//m_listCtrlOrdersSent->InsertColumn(2, "",              wxLIST_FORMAT_LEFT,  100);
//m_listCtrlOrdersSent->InsertColumn(3, "",              wxLIST_FORMAT_LEFT,  100);
//m_listCtrlOrdersSent->InsertColumn(4, "",              wxLIST_FORMAT_LEFT,  100);

//m_listCtrlOrdersReceived->InsertColumn(0, "Time",            wxLIST_FORMAT_LEFT,  100);
//m_listCtrlOrdersReceived->InsertColumn(1, "Price",           wxLIST_FORMAT_LEFT,  100);
//m_listCtrlOrdersReceived->InsertColumn(2, "Payment Status",  wxLIST_FORMAT_LEFT,  100);
//m_listCtrlOrdersReceived->InsertColumn(3, "",                wxLIST_FORMAT_LEFT,  100);
//m_listCtrlOrdersReceived->InsertColumn(4, "",                wxLIST_FORMAT_LEFT,  100);

// Init status bar
int pnWidths[3] = { -100, 81, 286 };
    m_statusBar->SetFieldsCount(3, pnWidths);

// Fill your address text box
    vector<unsigned char> vchPubKey;
```

`ReadDefaultKey` — loads the user's primary public key from wallet.dat and converts it to a Bitcoin address displayed in the "Your Bitcoin Address" box. This is the address users share to receive payments. Satoshi designed Bitcoin with single-address wallets in mind; address reuse was not yet a privacy concern.

```
if (CWalletDB("r").ReadDefaultKey(vchPubKey))
        m_textCtrlAddress->SetValue(PubKeyToAddress(vchPubKey));

// Fill listctrl with wallet transactions
RefreshListCtrl();
}

CMainFrame::~CMainFrame()
{
    pframeMain = NULL;
}

```

`Shutdown` — the clean exit sequence. Sets fShutdown to stop all network threads, flushes the Berkeley DB to disk (twice: once to checkpoint, once to force sync), then calls exit(0). Run in a separate thread so the window can close immediately while disk I/O completes in the background.

CALLED FROM CMainFrame::OnClose p. 481 · CMyApp::OnExit p. 518

```
void Shutdown(void* parg)
{
static CCriticalSection cs_Shutdown;
CRITICAL_BLOCK(cs_Shutdown)
{
        fShutdown = true;
        nTransactionsUpdated++;
DBFlush(false);
StopNode();
DBFlush(true);

printf("Bitcoin exiting\n");
exit(0);
}
}

void CMainFrame::OnClose(wxCloseEvent& event)
{
Destroy();
_beginthread(Shutdown, 0, NULL);
}

```

`OnMouseEvents` — harvests entropy from mouse movements for OpenSSL's random number generator. Every mouse event adds the X and Y coordinates to the entropy pool (RAND_add). Bitcoin keys require high-quality randomness; this ensures user activity continuously replenishes the pool. RandAddSeed() additionally mixes in the CPU performance counter.

```
365 void CMainFrame::OnMouseEvents(wxMouseEvent& event)
366 {
367 RandAddSeed();
368 RAND_add(&event.m_x, sizeof(event.m_x), 0.25);
369 RAND_add(&event.m_y, sizeof(event.m_y), 0.25);
370 }
371
372 void CMainFrame::OnListColBeginDrag(wxListEvent& event)
373 {
374 // Hidden columns not resizeable
375 if (event.GetColumn() <= 1 && !fDebug)
376         event.Veto();
377 }
378
```

`InsertLine` — the transaction list's insert/update method. Uses a sort key as column 0 (hidden, determines list order) and the transaction hash as column 1 (hidden, used to find rows for updates). When the sort key changes (e.g., a transaction gets confirmed and moves from INT_MAX height to a real block height), the row must be deleted and re-inserted to maintain sort order.

```
379 void CMainFrame::InsertLine(bool fNew, int nIndex, uint256 hashKey, string strSort, const wxString& str2, const wxString& str3,
    const wxString& str4, const wxString& str5, const wxString& str6)
380 {
381     string str0 = strSort;
382 long nData = *(long*)&hashKey;
383
384 if (fNew)
385 {
386         nIndex = m_listCtrl->InsertItem(0, str0);
387 }
388 else
389 {
390 if (nIndex == -1)
391 {
392 // Find item
393 while ((nIndex = m_listCtrl->FindItem(nIndex, nData)) != -1)
394 if (GetItemText(m_listCtrl, nIndex, 1) == hashKey.ToString())
395 break;
396 if (nIndex == -1)
397 {
398 printf("CMainFrame::InsertLine : Couldn't find item to be updated\n");
399 return;
400 }
401 }
402
403 // If sort key changed, must delete and reinsert to make it relocate
404 if (GetItemText(m_listCtrl, nIndex, 0) != str0)
405 {
406             m_listCtrl->DeleteItem(nIndex);
407             nIndex = m_listCtrl->InsertItem(0, str0);
408 }
409 }
410
411     m_listCtrl->SetItem(nIndex, 1, hashKey.ToString());
412     m_listCtrl->SetItem(nIndex, 2, str2);
413     m_listCtrl->SetItem(nIndex, 3, str3);
414     m_listCtrl->SetItem(nIndex, 4, str4);
415     m_listCtrl->SetItem(nIndex, 5, str5);
416     m_listCtrl->SetItem(nIndex, 6, str6);
417     m_listCtrl->SetItemData(nIndex, nData);
418 }
419
```

`FormatTxStatus` — converts confirmation depth to a human-readable status string. Unconfirmed transactions show "0/unconfirmed". Transactions with fewer than 6 confirmations show "N/unconfirmed". At 6+ confirmations the block depth is shown. The whitepaper never names 6; its probability tables show how the odds of a successful double-spend fall off exponentially with each block (for an attacker with 10% of the hash power, under 0.1% after 5 blocks), and 6 became the conventional rule of thumb.[3]

CALLED FROM `CMainFrame::InsertTransaction` p. 483 · `CMainFrame::RefreshStatus` p. 486 · `CTxDetailsDialog::CTxDetailsDialog` p. 491 · `CViewOrderDialog::CViewOrderDialog` p. 513

```
420 string FormatTxStatus(const CWalletTx& wtx)
421 {
422 // Status
423 int nDepth = wtx.GetDepthInMainChain();
424 if (!wtx.IsFinal())
425 return strprintf("Open for %d blocks", nBestHeight - wtx.nLockTime);
426 else if (nDepth < 6)
427 return strprintf("%d/unconfirmed", nDepth);
428 else
429 return strprintf("%d blocks", nDepth);
430 }
431
```

`InsertTransaction` — the core display logic for all transaction types. A transaction can appear in up to three ways: as a credit (bitcoin received), as a debit (bitcoin sent with one row per recipient output), or as "Payment to yourself" (when all inputs and outputs belong to the same wallet). Coinbase transactions get special treatment.

CALLED FROM `CMainFrame::RefreshStatus` p. 486 · `CMainFrame::OnIdle` p. 487 · `CMainFrame::OnPaintListCtrl` p. 488

```
432 void CMainFrame::InsertTransaction(const CWalletTx& wtx, bool fNew, int nIndex)
433 {
434     int64 nTime = wtx.nTimeDisplayed = wtx.GetTxTime();
435     int64 nCredit = wtx.GetCredit();
436     int64 nDebit = wtx.GetDebit();
437     int64 nNet = nCredit - nDebit;
438     uint256 hash = wtx.GetHash();
439     string strStatus = FormatTxStatus(wtx);
440     map<string, string> mapValue = wtx.mapValue;
441
442 // Find the block the tx is in
443     CBlockIndex* pindex = NULL;
444     map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(wtx.hashBlock);
445 if (mi != mapBlockIndex.end())
446         pindex = (*mi).second;
447
448 // Sort order, unrecorded transactions sort to the top
```

Sort key construction: zero-pads block height and time so lexicographic sort matches chronological order. Unconfirmed transactions (not yet in a block) sort to the top using INT_MAX as height. The list control is created with wxLC_SORT_DESCENDING, so higher keys go on top: within the same block, a coinbase (flag 1) sorts above regular transactions (flag 0), putting mined coins above received ones.

```
449     string strSort = strprintf("%010d-%01d-%010u",
450 (pindex ? pindex->nHeight : INT_MAX),
451 (wtx.IsCoinBase() ? 1 : 0),
452         wtx.nTimeReceived);
453
454 // Insert line
455 if (nNet > 0 || wtx.IsCoinBase())
456 {
457 //
458 // Credit
459 //
460         string strDescription;
```

Coinbase display: mined coins appear as "Generated" with a maturity countdown. Until 120 blocks deep, the credit is shown as "(X BTC matures in N blocks)" and not counted in the balance. This countdown is the visual manifestation of the coinbase maturity rule that prevents miners from spending freshly minted coins immediately — with a twist: consensus only requires 100 blocks (COINBASE_MATURITY in main.h), but GetBlocksToMaturity() counts to COINBASE_MATURITY+20. The wallet is deliberately stricter than the network, adding a 20-block safety margin.

```
462 if (wtx.IsCoinBase())
463 {
464 // Coinbase
465             strDescription = "Generated";
466 if (nCredit == 0)
467 {
468                 int64 nUnmatured = 0;
469 foreach(const CTxOut& txout, wtx.vout)
470                     nUnmatured += txout.GetCredit();
471 if (wtx.IsInMainChain())
472                     strDescription += strprintf(" (%s matures in %d blocks)", FormatMoney(nUnmatured).c_str(),
    wtx.GetBlocksToMaturity());
473 else
474                     strDescription += " (not accepted)";
475 }
476 }
477 else if (!mapValue["from"].empty() || !mapValue["message"].empty())
478 {
479 // Online transaction
480 if (!mapValue["from"].empty())
481                 strDescription += "From: " + mapValue["from"];
482 if (!mapValue["message"].empty())
483 {
484 if (!strDescription.empty())
485                     strDescription += " - ";
486                 strDescription += mapValue["message"];
487 }
488 }
489 else
490 {
491 // Offline transaction
492 foreach(const CTxOut& txout, wtx.vout)
493 {
494 if (txout.IsMine())
495 {
496                     vector<unsigned char> vchPubKey;
497 if (ExtractPubKey(txout.scriptPubKey, true, vchPubKey))
498 {
499                         string strAddress = PubKeyToAddress(vchPubKey);
500 if (mapAddressBook.count(strAddress))
501 {
502                             strDescription += "Received with: ";
503 if (!mapAddressBook[strAddress].empty())
504                                 strDescription += mapAddressBook[strAddress] + " ";
505                             strDescription += strAddress;
506 }
507 }
508 break;
509 }
510 }
511 }
512
513         string strDescription2;
514 foreach(int c, strDescription)
515 if (c >= ' ')
516                 strDescription2 += c;
517
518 InsertLine(fNew, nIndex, hash, strSort,
519                    strStatus,
520                    nTime ? DateTimeStr(nTime) : "",
521                    strDescription2,
522 "",
523 FormatMoney(nNet, true));
524 }
525 else
526 {
```

Payment classification logic: fAllFromMe checks whether every input is controlled by this wallet. fAllToMe checks the same for outputs. The matrix determines how the transaction is displayed: (both) = self-payment, (fromMe only) = debit with recipient breakdown, (neither/mixed) = complex transaction shown as net amount only.

```
bool fAllFromMe = true;
foreach(const CTxIn& txin, wtx.vin)
            fAllFromMe = fAllFromMe && txin.IsMine();

bool fAllToMe = true;
foreach(const CTxOut& txout, wtx.vout)
            fAllToMe = fAllToMe && txout.IsMine();

if (fAllFromMe && fAllToMe)
{
// Payment to self
            int64 nValue = wtx.vout[0].nValue;
InsertLine(fNew, nIndex, hash, strSort,
                        strStatus,
                        nTime ? DateTimeStr(nTime) : "",
"Payment to yourself",
FormatMoney(nNet - nValue, true),
FormatMoney(nValue, true));
}
else if (fAllFromMe)
{
//
// Debit
//
```

Debit display: iterates outputs to find the recipient (non-wallet) outputs. Skips our own change outputs (txout.IsMine()). Shows each recipient separately with address lookup. The transaction fee is computed as nDebit − GetValueOut() — the difference between what we sent and what recipients receive.

```
            int64 nTxFee = nDebit - wtx.GetValueOut();
for (int nOut = 0; nOut < wtx.vout.size(); nOut++)
{
const CTxOut& txout = wtx.vout[nOut];
if (txout.IsMine())
continue;

                string strAddress;
if (!mapValue["to"].empty())
{
// Online transaction
                    strAddress = mapValue["to"];
}
else
{
// Offline transaction
                    uint160 hash160;
if (ExtractHash160(txout.scriptPubKey, hash160))
                        strAddress = Hash160ToAddress(hash160);
}

                string strDescription = "To: ";
if (mapAddressBook.count(strAddress) && !mapAddressBook[strAddress].empty())
                    strDescription += mapAddressBook[strAddress] + " ";
                strDescription += strAddress;
if (!mapValue["message"].empty())
{
if (!strDescription.empty())
                        strDescription += " - ";
                    strDescription += mapValue["message"];
}

                string strDescription2;
foreach(int c, strDescription)
if (c >= ' ')
                        strDescription2 += c;

                int64 nValue = txout.nValue;
if (nOut == 0 && nTxFee > 0)
                    nValue += nTxFee;

```

```
InsertLine(fNew, nIndex, hash, strprintf("%s-%d", strSort.c_str(), nOut),
                           strStatus,
                           nTime ? DateTimeStr(nTime) : "",
                           strDescription2,
FormatMoney(-nValue, true),
"");
}
}
else
{
//
// Mixed debit transaction, can't break down payees
//
bool fAllMine = true;
foreach(const CTxOut& txout, wtx.vout)
                fAllMine = fAllMine && txout.IsMine();
foreach(const CTxIn& txin, wtx.vin)
                fAllMine = fAllMine && txin.IsMine();

InsertLine(fNew, nIndex, hash, strSort,
                       strStatus,
                       nTime ? DateTimeStr(nTime) : "",
"",
FormatMoney(nNet, true),
"");
}
}
}

```

`RefreshStatus` — updates only the visible rows' status columns without rebuilding the entire list. Limits to the top 100 visible items for performance. Skips the update if neither the scroll position nor the best block has changed since last call — a simple dirty-flag optimization.

CALLED FROM `CMainFrame::OnPaintListCtrl` p. 488

```
void CMainFrame::RefreshStatus()
{
static int nLastTop;
int nTop = m_listCtrl->GetTopItem();
if (nTop == nLastTop && pindexBestLast == pindexBest)
return;

TRY_CRITICAL_BLOCK(cs_mapWallet)
{
int nStart = nTop;
int nEnd = min(nStart + 100, m_listCtrl->GetItemCount());
if (pindexBestLast == pindexBest)
{
if (nStart >= nLastTop && nStart < nLastTop + 100)
                nStart = nLastTop + 100;
if (nEnd >= nLastTop && nEnd < nLastTop + 100)
                nEnd = nLastTop;
}
        nLastTop = nTop;
        pindexBestLast = pindexBest;

for (int nIndex = nStart; nIndex < nEnd; nIndex++)
{
            uint256 hash((string)GetItemText(m_listCtrl, nIndex, 1));
            map<uint256, CWalletTx>::iterator mi = mapWallet.find(hash);
if (mi == mapWallet.end())
{
printf("CMainFrame::RefreshStatus() : tx not found in mapWallet\n");
continue;
}
const CWalletTx& wtx = (*mi).second;
if (wtx.IsCoinBase() || wtx.GetTxTime() != wtx.nTimeDisplayed)
InsertTransaction(wtx, false, nIndex);
else
                m_listCtrl->SetItem(nIndex, 2, FormatTxStatus(wtx));
}
}
}

```

`RefreshListCtrl` — schedules a full wallet reload by setting fRefreshListCtrl=true and calling wxWakeUpIdle(). The actual work happens in OnIdle() to avoid blocking the UI thread. Called whenever new transactions arrive or blocks are connected.

CALLED FROM CMainFrame::CMainFrame p. 481 · CYourAddressDialog::OnListEndLabelEdit p. 502 · CYourAddressDialog::OnButtonRename p. 503 · CAddressBookDialog::OnListEndLabelEdit p. 504 · CAddressBookDialog::OnButtonEdit p. 504 · CAddressBookDialog::OnButtonDelete p. 505

```
660 void CMainFrame::RefreshListCtrl()
661 {
662     fRefreshListCtrl = true;
663 ::wxWakeUpIdle();
664 }
665
```

`OnIdle` — the lazy refresh handler. Runs during UI idle time, preventing it from starving the event loop. Sorts all wallet transactions by time (newest first using UINT_MAX minus timestamp), clears the list, then re-inserts all transactions. Calls wxYield() every 100 or 500 items to keep the UI responsive during large wallet loads.

```
666 void CMainFrame::OnIdle(wxIdleEvent& event)
667 {
668 if (fRefreshListCtrl)
669 {
670 // Collect list of wallet transactions and sort newest first
671 bool fEntered = false;
672         vector<pair<unsigned int, uint256> > vSorted;
673 TRY_CRITICAL_BLOCK(cs_mapWallet)
674 {
675 printf("RefreshListCtrl starting\n");
676             fEntered = true;
677             fRefreshListCtrl = false;
678             vWalletUpdated.clear();
679
680 // Do the newest transactions first
681             vSorted.reserve(mapWallet.size());
682 for (map<uint256, CWalletTx>::iterator it = mapWallet.begin(); it != mapWallet.end(); ++it)
683 {
684 const CWalletTx& wtx = (*it).second;
685 unsigned int nTime = UINT_MAX - wtx.GetTxTime();
686                 vSorted.push_back(make_pair(nTime, (*it).first));
687 }
688             m_listCtrl->DeleteAllItems();
689 }
690 if (!fEntered)
691 return;
692
693 sort(vSorted.begin(), vSorted.end());
694
695 // Fill list control
696 for (int i = 0; i < vSorted.size();)
697 {
698 if (fShutdown)
699 return;
700 bool fEntered = false;
701 TRY_CRITICAL_BLOCK(cs_mapWallet)
702 {
703                 fEntered = true;
704                 uint256& hash = vSorted[i++].second;
705                 map<uint256, CWalletTx>::iterator mi = mapWallet.find(hash);
706 if (mi != mapWallet.end())
707 InsertTransaction((*mi).second, true);
708 }
709 if (!fEntered || i == 100 || i % 500 == 0)
710 wxYield();
711 }
712
713 printf("RefreshListCtrl done\n");
714 }
715 else
716 {
717 // Check for time updates
718 static int64 nLastTime;
719 if (GetTime() > nLastTime + 30)
720 {
721 TRY_CRITICAL_BLOCK(cs_mapWallet)
```

```
{
                nLastTime = GetTime();
for (map<uint256, CWalletTx>::iterator it = mapWallet.begin(); it != mapWallet.end(); ++it)
{
                    CWalletTx& wtx = (*it).second;
if (wtx.nTimeDisplayed && wtx.nTimeDisplayed != wtx.GetTxTime())
InsertTransaction(wtx, false);
}
}
}
}
}

void CMainFrame::OnPaint(wxPaintEvent& event)
{
    event.Skip();
}

```

`OnPaintListCtrl` — the combined repaint handler for the transaction list. Fires on every paint event: processes pending wallet updates (vWalletUpdated), calls RefreshStatus for visible rows, updates the status bar, and repaints the balance. This single repaint event drives all UI liveness.

```
void CMainFrame::OnPaintListCtrl(wxPaintEvent& event)
{
// Update listctrl contents
if (!vWalletUpdated.empty())
{
TRY_CRITICAL_BLOCK(cs_mapWallet)
{
            pair<uint256, bool> item;
foreach(item, vWalletUpdated)
{
bool fNew = item.second;
                map<uint256, CWalletTx>::iterator mi = mapWallet.find(item.first);
if (mi != mapWallet.end())
{
printf("vWalletUpdated: %s %s\n", (*mi).second.GetHash().ToString().substr(0,6).c_str(), fNew ? "new" : "");
InsertTransaction((*mi).second, fNew);
}
}
            m_listCtrl->ScrollList(0, INT_MAX);
            vWalletUpdated.clear();
}
}

// Update status column of visible items only
RefreshStatus();

// Update status bar
    string strGen = "";
if (fGenerateBitcoins)
        strGen = "    Generating";
if (fGenerateBitcoins && vNodes.empty())
        strGen = "(not connected)";
    m_statusBar->SetStatusText(strGen, 1);

```

Status bar format: three fields. Left: empty. Middle (81px): mining status — "Generating" or "(not connected)". Right: a live snapshot of connections, block count, and the number of transactions in your wallet's list. The format string encodes the user's current view of the network's health in one line.

```
    string strStatus = strprintf("     %d connections     %d blocks     %d transactions", vNodes.size(), nBestHeight + 1, m_listCtrl->GetItemCount());
    m_statusBar->SetStatusText(strStatus, 2);

// Balance total
TRY_CRITICAL_BLOCK(cs_mapWallet)
        m_staticTextBalance->SetLabel(FormatMoney(GetBalance()) + "  ");

    m_listCtrl->OnPaint(event);
}

void CrossThreadCall(wxCommandEvent& event)
{
if (pframeMain)
```

```
787         pframeMain->GetEventHandler()->AddPendingEvent(event);
788 }
789
790 void CrossThreadCall(int nID, void* pdata)
791 {
792     wxCommandEvent event;
793     event.SetInt(nID);
794     event.SetClientData(pdata);
795 if (pframeMain)
796         pframeMain->GetEventHandler()->AddPendingEvent(event);
797 }
798
799 void CMainFrame::OnCrossThreadCall(wxCommandEvent& event)
800 {
801 void* pdata = event.GetClientData();
802 switch (event.GetInt())
803 {
804 case UICALL_ADDORDER:
805 {
806 break;
807 }
808
809 case UICALL_UPDATEORDER:
810 {
811 break;
812 }
813 }
814 }
815
816 void CMainFrame::OnMenuFileExit(wxCommandEvent& event)
817 {
818 Close(true);
819 }
820
```

`OnMenuOptionsGenerate` — the mining toggle. Writes the new value to wallet.dat settings, then spawns a ThreadBitcoinMiner thread if enabled. Unchecking it stops mining cleanly: BitcoinMiner() loops `while (fGenerateBitcoins)` and also checks the flag inside its hashing loop, so it notices the change and exits.

```
821 void CMainFrame::OnMenuOptionsGenerate(wxCommandEvent& event)
822 {
823     fGenerateBitcoins = event.IsChecked();
824     nTransactionsUpdated++;
825 CWalletDB().WriteSetting("fGenerateBitcoins", fGenerateBitcoins);
826
827 if (fGenerateBitcoins)
828 if (_beginthread(ThreadBitcoinMiner, 0, NULL) == -1)
829 printf("Error: _beginthread(ThreadBitcoinMiner) failed\n");
830
831 Refresh();
832     wxPaintEvent eventPaint;
833 AddPendingEvent(eventPaint);
834 }
835
836 void CMainFrame::OnMenuOptionsOptions(wxCommandEvent& event)
837 {
838     COptionsDialog dialog(this);
839     dialog.ShowModal();
840 }
841
842 void CMainFrame::OnMenuHelpAbout(wxCommandEvent& event)
843 {
844     CAboutDialog dialog(this);
845     dialog.ShowModal();
846 }
847
```

`OnButtonSend` — opens the CSendDialog. Contains a hidden "random send test" mode (fRandSendTest) for stress-testing, which Satoshi left in. The Send dialog supports two payment modes: Bitcoin address (P2PKH scriptPubKey) and IP address (live peer connection).

```
848 void CMainFrame::OnButtonSend(wxCommandEvent& event)
849 {
850 /// debug test
851 if (fRandSendTest)
852 {
```

```
RandSend();
return;
}

// Toolbar: Send
    CSendDialog dialog(this);
    dialog.ShowModal();
}

void CMainFrame::OnButtonAddressBook(wxCommandEvent& event)
{
// Toolbar: Address Book
    CAddressBookDialog dialogAddr(this, "", false);
if (dialogAddr.ShowModal() == 2)
{
// Send
        CSendDialog dialogSend(this, dialogAddr.GetAddress());
        dialogSend.ShowModal();
}
}

void CMainFrame::OnSetFocusAddress(wxFocusEvent& event)
{
// Automatically select-all when entering window
    m_textCtrlAddress->SetSelection(-1, -1);
    fOnSetFocusAddress = true;
    event.Skip();
}

void CMainFrame::OnMouseEventsAddress(wxMouseEvent& event)
{
if (fOnSetFocusAddress)
        m_textCtrlAddress->SetSelection(-1, -1);
    fOnSetFocusAddress = false;
    event.Skip();
}

void CMainFrame::OnButtonCopy(wxCommandEvent& event)
{
// Copy address box to clipboard
if (wxTheClipboard->Open())
{
        wxTheClipboard->SetData(new wxTextDataObject(m_textCtrlAddress->GetValue()));
        wxTheClipboard->Close();
}
}

```

`OnButtonChange` — allows selecting a different receiving address. Validates the address by converting it to a hash160 and checking mapPubKeys for the corresponding public key. Only addresses whose private keys are in the wallet can be set as the default — you can't receive to an address you can't spend from.

```
void CMainFrame::OnButtonChange(wxCommandEvent& event)
{
    CYourAddressDialog dialog(this, string(m_textCtrlAddress->GetValue()));
if (!dialog.ShowModal())
return;
    string strAddress = (string)dialog.GetAddress();
if (strAddress != m_textCtrlAddress->GetValue())
{
        uint160 hash160;
if (!AddressToHash160(strAddress, hash160))
return;
if (!mapPubKeys.count(hash160))
return;
CWalletDB().WriteDefaultKey(mapPubKeys[hash160]);
        m_textCtrlAddress->SetValue(strAddress);
}
}

void CMainFrame::OnListItemActivatedAllTransactions(wxListEvent& event)
{
    uint256 hash((string)GetItemText(m_listCtrl, event.GetIndex(), 1));
    CWalletTx wtx;
CRITICAL_BLOCK(cs_mapWallet)
{
        map<uint256, CWalletTx>::iterator mi = mapWallet.find(hash);
if (mi == mapWallet.end())
{
printf("CMainFrame::OnListItemActivatedAllTransactions() : tx not found in mapWallet\n");
return;
}
```

```
        wtx = (*mi).second;
}
    CTxDetailsDialog dialog(this, wtx);
    dialog.ShowModal();
//CTxDetailsDialog* pdialog = new CTxDetailsDialog(this, wtx);
//pdialog->Show();
}

void CMainFrame::OnListItemActivatedProductsSent(wxListEvent& event)
{
    CProduct& product = *(CProduct*)event.GetItem().GetData();
    CEditProductDialog* pdialog = new CEditProductDialog(this);
    pdialog->SetProduct(product);
    pdialog->Show();
}

void CMainFrame::OnListItemActivatedOrdersSent(wxListEvent& event)
{
    CWalletTx& order = *(CWalletTx*)event.GetItem().GetData();
    CViewOrderDialog* pdialog = new CViewOrderDialog(this, order, false);
    pdialog->Show();
}

void CMainFrame::OnListItemActivatedOrdersReceived(wxListEvent& event)
{
    CWalletTx& order = *(CWalletTx*)event.GetItem().GetData();
    CViewOrderDialog* pdialog = new CViewOrderDialog(this, order, true);
    pdialog->Show();
}









//////////////////////////////////////////////////////////////////////////////
//
// CTxDetailsDialog
//

```

`CTxDetailsDialog` — the transaction detail popup. Builds an HTML string describing the transaction in human terms: status, date, from/to addresses (with address book names if available), credit/debit amounts, transaction fee, and optional message field. When fDebug is enabled, shows raw transaction hex and input spending details.

```
CTxDetailsDialog::CTxDetailsDialog(wxWindow* parent, CWalletTx wtx) : CTxDetailsDialogBase(parent)
{
    string strHTML;
    strHTML.reserve(4000);

    int64 nTime = wtx.GetTxTime();
    int64 nCredit = wtx.GetCredit();
    int64 nDebit = wtx.GetDebit();
    int64 nNet = nCredit - nDebit;




    strHTML += "<b>Status:</b> " + FormatTxStatus(wtx) + "<br>";
    strHTML += "<b>Date:</b> " + (nTime ? DateTimeStr(nTime) : "") + "<br>";


//
// From
//
if (wtx.IsCoinBase())
{
        strHTML += "<b>Source:</b> Generated<br>";
}
else if (!wtx.mapValue["from"].empty())
{
// Online transaction
if (!wtx.mapValue["from"].empty())
            strHTML += "<b>From:</b> " + HtmlEscape(wtx.mapValue["from"]) + "<br>";
}
else
{
// Offline transaction
foreach(const CTxOut& txout, wtx.vout)
{
if (txout.IsMine())
```

```
{
                vector<unsigned char> vchPubKey;
if (ExtractPubKey(txout.scriptPubKey, true, vchPubKey))
{
                    string strAddress = PubKeyToAddress(vchPubKey);
if (mapAddressBook.count(strAddress))
{
                        strHTML += "<b>Received with:</b> ";
if (!mapAddressBook[strAddress].empty())
                            strHTML += mapAddressBook[strAddress] + " ";
                        strHTML += HtmlEscape(strAddress);
                        strHTML += "<br>";
}
}
break;
}
}
}


//
// To
//
    string strAddress;
if (!wtx.mapValue["to"].empty())
{
// Online transaction
        strAddress = wtx.mapValue["to"];
        strHTML += "<b>To:</b> ";
if (mapAddressBook.count(strAddress) && !mapAddressBook[strAddress].empty())
            strHTML += mapAddressBook[strAddress] + " ";
        strHTML += HtmlEscape(strAddress) + "<br>";
}


//
// Amount
//
if (wtx.IsCoinBase() && nCredit == 0)
{
//
// Coinbase
//
        int64 nUnmatured = 0;
foreach(const CTxOut& txout, wtx.vout)
            nUnmatured += txout.GetCredit();
if (wtx.IsInMainChain())
            strHTML += strprintf("<b>Credit:</b> (%s matures in %d blocks)<br>", FormatMoney(nUnmatured).c_str(),
    wtx.GetBlocksToMaturity());
else
            strHTML += "<b>Credit:</b> (not accepted)<br>";
}
else if (nNet > 0)
{
//
// Credit
//
        strHTML += "<b>Credit:</b> " + FormatMoney(nNet) + "<br>";
}
else
{
bool fAllFromMe = true;
foreach(const CTxIn& txin, wtx.vin)
            fAllFromMe = fAllFromMe && txin.IsMine();

bool fAllToMe = true;
foreach(const CTxOut& txout, wtx.vout)
            fAllToMe = fAllToMe && txout.IsMine();

if (fAllFromMe)
{
//
// Debit
//
foreach(const CTxOut& txout, wtx.vout)
{
if (txout.IsMine())
continue;

                string strAddress;
if (!wtx.mapValue["to"].empty())
{
// Online transaction
                    strAddress = wtx.mapValue["to"];
}
else
{
// Offline transaction
                    uint160 hash160;
```

```
1095 if (ExtractHash160(txout.scriptPubKey, hash160))
1096                         strAddress = Hash160ToAddress(hash160);
1097 }
1098
1099                 strHTML += "<b>Debit:</b> " + FormatMoney(-txout.nValue) + "    ";
1100                 strHTML += "(to ";
1101 if (mapAddressBook.count(strAddress) && !mapAddressBook[strAddress].empty())
1102                     strHTML += mapAddressBook[strAddress] + " ";
1103                 strHTML += strAddress;
1104                 strHTML += ")<br>";
1105 }
1106
1107 if (fAllToMe)
1108 {
1109 // Payment to self
1110                 int64 nValue = wtx.vout[0].nValue;
1111                 strHTML += "<b>Debit:</b> " + FormatMoney(-nValue) + "<br>";
1112                 strHTML += "<b>Credit:</b> " + FormatMoney(nValue) + "<br>";
1113 }
1114
1115             int64 nTxFee = nDebit - wtx.GetValueOut();
1116 if (nTxFee > 0)
1117                 strHTML += "<b>Transaction fee:</b> " + FormatMoney(-nTxFee) + "<br>";
1118 }
1119 else
1120 {
1121 //
1122 // Mixed debit transaction
1123 //
1124 foreach(const CTxIn& txin, wtx.vin)
1125 if (txin.IsMine())
1126                     strHTML += "<b>Debit:</b> " + FormatMoney(-txin.GetDebit()) + "<br>";
1127 foreach(const CTxOut& txout, wtx.vout)
1128 if (txout.IsMine())
1129                     strHTML += "<b>Credit:</b> " + FormatMoney(txout.GetCredit()) + "<br>";
1130 }
1131 }
1132
1133     strHTML += "<b>Net amount:</b> " + FormatMoney(nNet, true) + "<br>";
1134
1135
1136 //
1137 // Message
1138 //
1139 if (!wtx.mapValue["message"].empty())
1140         strHTML += "<br><b>Message:</b><br>" + HtmlEscape(wtx.mapValue["message"], true) + "<br>";
```

`mapValue["message"]` — Bitcoin v0.1.0 supported attaching free-text messages to transactions. For IP-to-IP payments, sender and recipient contact details were also stored in mapValue["from"] and mapValue["to"]. These fields never go into the block chain or get relayed across the network, but they are not purely local either: for an IP payment, the whole wallet transaction — message included — is sent directly to the recipient's node in the "submitorder" message, and their wallet stores it.

```
1141
1142
1143 //
1144 // Debug view
1145 //
1146 if (fDebug)
1147 {
1148         strHTML += "<hr><br>debug print<br><br>";
1149 foreach(const CTxIn& txin, wtx.vin)
1150 if (txin.IsMine())
1151                 strHTML += "<b>Debit:</b> " + FormatMoney(-txin.GetDebit()) + "<br>";
1152 foreach(const CTxOut& txout, wtx.vout)
1153 if (txout.IsMine())
1154                 strHTML += "<b>Credit:</b> " + FormatMoney(txout.GetCredit()) + "<br>";
1155
1156         strHTML += "<b>Inputs:</b><br>";
1157 CRITICAL_BLOCK(cs_mapWallet)
1158 {
1159 foreach(const CTxIn& txin, wtx.vin)
1160 {
1161                 COutPoint prevout = txin.prevout;
1162                 map<uint256, CWalletTx>::iterator mi = mapWallet.find(prevout.hash);
1163 if (mi != mapWallet.end())
1164 {
1165 const CWalletTx& prev = (*mi).second;
1166 if (prevout.n < prev.vout.size())
1167 {
1168                         strHTML += HtmlEscape(prev.ToString(), true);
1169                         strHTML += "    " + FormatTxStatus(prev) + ", ";
```

```
                        strHTML = strHTML + "IsMine=" + (prev.vout[prevout.n].IsMine() ? "true" : "false") + "<br>";
}
}
}
}

        strHTML += "<br><hr><br><b>Transaction:</b><br>";
        strHTML += HtmlEscape(wtx.ToString(), true);
}


string(strHTML.begin(), strHTML.end()).swap(strHTML);
    m_htmlWin->SetPage(strHTML);
    m_buttonOK->SetFocus();
}

void CTxDetailsDialog::OnButtonOK(wxCommandEvent& event)
{
Close();
//Destroy();
}




//////////////////////////////////////////////////////////////////////////////
//
// COptionsDialog
//

```

`COptionsDialog` — minimal settings dialog. In v0.1.0, the only user-configurable option is the transaction fee. ParseMoney() handles decimal input ("0.01" = 1,000,000 satoshis). The fee is persisted to wallet.dat under the key "nTransactionFee".

```
COptionsDialog::COptionsDialog(wxWindow* parent) : COptionsDialogBase(parent)
{
    m_textCtrlTransactionFee->SetValue(FormatMoney(nTransactionFee));
    m_buttonOK->SetFocus();
}

void COptionsDialog::OnKillFocusTransactionFee(wxFocusEvent& event)
{
    int64 nTmp = nTransactionFee;
ParseMoney(m_textCtrlTransactionFee->GetValue(), nTmp);
    m_textCtrlTransactionFee->SetValue(FormatMoney(nTmp));
}

void COptionsDialog::OnButtonOK(wxCommandEvent& event)
{
// nTransactionFee
    int64 nPrevTransactionFee = nTransactionFee;
if (ParseMoney(m_textCtrlTransactionFee->GetValue(), nTransactionFee) && nTransactionFee != nPrevTransactionFee)
CWalletDB().WriteSetting("nTransactionFee", nTransactionFee);

Close();
}

void COptionsDialog::OnButtonCancel(wxCommandEvent& event)
{
Close();
}




//////////////////////////////////////////////////////////////////////////////
//
// CAboutDialog
//

CAboutDialog::CAboutDialog(wxWindow* parent) : CAboutDialogBase(parent)
{
```

`CAboutDialog` — shows the version string: "version 0.N.N Alpha" computed from the VERSION constant. The UTF-8 workaround strips the stray lead byte of the UTF-8-encoded copyright symbol — the 'Â' artifact that wxWidgets produced without UTF-8 support — a practical limitation of the 2009 era toolkit.

```
1241     m_staticTextVersion->SetLabel(strprintf("version 0.%d.%d Alpha", VERSION/100, VERSION%100));
1242
1243 // Workaround until upgrade to wxWidgets supporting UTF-8
1244     wxString str = m_staticTextMain->GetLabel();
1245 if (str.Find('Â') != wxNOT_FOUND)
1246         str.Remove(str.Find('Â'), 1);
1247     m_staticTextMain->SetLabel(str);
1248 }
1249
1250 void CAboutDialog::OnButtonOK(wxCommandEvent& event)
1251 {
1252 Close();
1253 }
1254
1255
1256
1257
1258
1259
1260 //////////////////////////////////////////////////////////////////////////////
1261 //
1262 // CSendDialog
1263 //
1264
```

`CSendDialog` — the payment form. Detects whether the entered address is a Bitcoin address (base58, starts with 1) or an IP address. Bitcoin address mode disables the From/Message fields — those only apply to IP-to-IP payments where the recipient can see them.

```
1265 CSendDialog::CSendDialog(wxWindow* parent, const wxString& strAddress) : CSendDialogBase(parent)
1266 {
1267 // Init
1268     m_textCtrlAddress->SetValue(strAddress);
1269     m_choiceTransferType->SetSelection(0);
1270     m_bitmapCheckMark->Show(false);
1271 //// todo: should add a display of your balance for convenience
1272
1273 // Set Icon
1274     wxBitmap bmpSend(wxT("send16"), wxBITMAP_TYPE_RESOURCE);
1275     bmpSend.SetMask(new wxMask(wxBitmap(wxT("send16masknoshadow"), wxBITMAP_TYPE_RESOURCE)));
1276     wxIcon iconSend;
1277     iconSend.CopyFromBitmap(bmpSend);
1278 SetIcon(iconSend);
1279
1280     wxCommandEvent event;
1281 OnTextAddress(event);
1282
1283 // Fixup the tab order
1284     m_buttonPaste->MoveAfterInTabOrder(m_buttonCancel);
1285     m_buttonAddress->MoveAfterInTabOrder(m_buttonPaste);
1286 this->Layout();
1287 }
1288
     CALLED FROM  CSendDialog::CSendDialog p. 495
1289 void CSendDialog::OnTextAddress(wxCommandEvent& event)
1290 {
1291 // Check mark
```

Address validation feedback: IsValidBitcoinAddress() checks base58 encoding and checksum. A green checkmark appears live as the user types — immediate feedback without a submit button. This UX detail shows Satoshi's care for usability even in an alpha.

```
1292 bool fBitcoinAddress = IsValidBitcoinAddress(m_textCtrlAddress->GetValue());
1293     m_bitmapCheckMark->Show(fBitcoinAddress);
1294
```

```
1295 // Grey out message if bitcoin address
1296 bool fEnable = !fBitcoinAddress;
1297     m_staticTextFrom->Enable(fEnable);
1298     m_textCtrlFrom->Enable(fEnable);
1299     m_staticTextMessage->Enable(fEnable);
1300     m_textCtrlMessage->Enable(fEnable);
1301     m_textCtrlMessage->SetBackgroundColour(wxSystemSettings::GetColour(fEnable ? wxSYS_COLOUR_WINDOW : wxSYS_COLOUR_BTNFACE));
1302 }
1303
1304 void CSendDialog::OnKillFocusAmount(wxFocusEvent& event)
1305 {
1306 // Reformat the amount
1307 if (m_textCtrlAmount->GetValue().Trim().empty())
1308 return;
1309     int64 nTmp;
1310 if (ParseMoney(m_textCtrlAmount->GetValue(), nTmp))
1311         m_textCtrlAmount->SetValue(FormatMoney(nTmp));
1312 }
1313
1314 void CSendDialog::OnButtonAddressBook(wxCommandEvent& event)
1315 {
1316 // Open address book
1317     CAddressBookDialog dialog(this, m_textCtrlAddress->GetValue(), true);
1318 if (dialog.ShowModal())
1319         m_textCtrlAddress->SetValue(dialog.GetAddress());
1320 }
1321
1322 void CSendDialog::OnButtonPaste(wxCommandEvent& event)
1323 {
1324 // Copy clipboard to address box
1325 if (wxTheClipboard->Open())
1326 {
1327 if (wxTheClipboard->IsSupported(wxDF_TEXT))
1328 {
1329             wxTextDataObject data;
1330             wxTheClipboard->GetData(data);
1331             m_textCtrlAddress->SetValue(data.GetText());
1332 }
1333         wxTheClipboard->Close();
1334 }
1335 }
1336
```

`OnButtonSend` — two payment paths. Bitcoin address: builds a P2PKH scriptPubKey directly (OP_DUP OP_HASH160 hash160 OP_EQUALVERIFY OP_CHECKSIG) and calls SendMoney(). IP address: creates a CSendingDialog which runs the full peer-to-peer payment handshake in a background thread.

```
1337 void CSendDialog::OnButtonSend(wxCommandEvent& event)
1338 {
1339     CWalletTx wtx;
1340     string strAddress = (string)m_textCtrlAddress->GetValue();
1341
1342 // Parse amount
1343     int64 nValue = 0;
1344 if (!ParseMoney(m_textCtrlAmount->GetValue(), nValue) || nValue <= 0)
1345 {
1346 wxMessageBox("Error in amount ");
1347 return;
1348 }
1349 if (nValue > GetBalance())
1350 {
1351 wxMessageBox("Amount exceeds your balance ");
1352 return;
1353 }
1354 if (nValue + nTransactionFee > GetBalance())
1355 {
1356 wxMessageBox(string("Total exceeds your balance when the ") + FormatMoney(nTransactionFee) + " transaction fee is included ");
1357 return;
1358 }
1359
1360 // Parse bitcoin address
1361     uint160 hash160;
1362 bool fBitcoinAddress = AddressToHash160(strAddress, hash160);
1363
1364 if (fBitcoinAddress)
1365 {
1366 // Send to bitcoin address
1367         CScript scriptPubKey;
```

P2PKH script construction inline: << OP_DUP << OP_HASH160 << hash160 << OP_EQUALVERIFY << OP_CHECKSIG. The << operator on CScript appends opcodes and push data. This five-opcode script is the standard Bitcoin address locking script, unchanged to this day.

```
        scriptPubKey << OP_DUP << OP_HASH160 << hash160 << OP_EQUALVERIFY << OP_CHECKSIG;

if (!SendMoney(scriptPubKey, nValue, wtx))
return;

wxMessageBox("Payment sent ", "Sending...");
}
else
{
// Parse IP address
        CAddress addr(strAddress.c_str());
if (addr.ip == 0)
{
wxMessageBox("Invalid address ");
return;
}

// Message
        wtx.mapValue["to"] = strAddress;
        wtx.mapValue["from"] = m_textCtrlFrom->GetValue();
        wtx.mapValue["message"] = m_textCtrlMessage->GetValue();

// Send to IP address
        CSendingDialog* pdialog = new CSendingDialog(this, addr, nValue, wtx);
if (!pdialog->ShowModal())
return;
}

if (!mapAddressBook.count(strAddress))
SetAddressBookName(strAddress, "");

EndModal(true);
}

void CSendDialog::OnButtonCancel(wxCommandEvent& event)
{
// Cancel
EndModal(false);
}




////////////////////////////////////////////////////////////////////////////
//
// CSendingDialog
//

```

`CSendingDialog` — the IP-to-IP payment protocol UI. Runs the payment handshake in a background thread (SendingDialogStartTransfer) while showing a status dialog to the user. The fWorkDone/fUIDone flags implement a two-party shutdown: whichever side finishes last destroys the dialog object.

```
CSendingDialog::CSendingDialog(wxWindow* parent, const CAddress& addrIn, int64 nPriceIn, const CWalletTx& wtxIn) :
    CSendingDialogBase(NULL) // we have to give null so parent can't destroy us
{
    addr = addrIn;
    nPrice = nPriceIn;
    wtx = wtxIn;
    start = wxDateTime::UNow();
    strStatus = "";
    fCanCancel = true;
    fAbort = false;
    fSuccess = false;
    fUIDone = false;
    fWorkDone = false;

SetTitle(strprintf("Sending %s to %s...", FormatMoney(nPrice).c_str(), wtx.mapValue["to"].c_str()));
    m_textCtrlStatus->SetValue("");
```

```
1433
1434 _beginthread(SendingDialogStartTransfer, 0, this);
1435 }
1436
1437 CSendingDialog::~CSendingDialog()
1438 {
1439 printf("~CSendingDialog()\n");
1440 }
1441
1442 void CSendingDialog::Close()
1443 {
1444 // Last one out turn out the lights.
1445 // fWorkDone signals that work side is done and UI thread should call destroy.
1446 // fUIDone signals that UI window has closed and work thread should call destroy.
1447 // This allows the window to disappear and end modality when cancelled
1448 // without making the user wait for ConnectNode to return.  The dialog object
1449 // hangs around in the background until the work thread exits.
1450 if (IsModal())
1451 EndModal(fSuccess);
1452 else
1453 Show(false);
1454 if (fWorkDone)
1455 Destroy();
1456 else
1457         fUIDone = true;
1458 }
1459
1460 void CSendingDialog::OnClose(wxCloseEvent& event)
1461 {
1462 if (!event.CanVeto() || fWorkDone || fAbort || !fCanCancel)
1463 {
1464 Close();
1465 }
1466 else
1467 {
1468         event.Veto();
1469         wxCommandEvent cmdevent;
1470 OnButtonCancel(cmdevent);
1471 }
1472 }
1473
1474 void CSendingDialog::OnButtonOK(wxCommandEvent& event)
1475 {
1476 if (fWorkDone)
1477 Close();
1478 }
1479
1480 void CSendingDialog::OnButtonCancel(wxCommandEvent& event)
1481 {
1482 if (fCanCancel)
1483         fAbort = true;
1484 }
1485
1486 void CSendingDialog::OnPaint(wxPaintEvent& event)
1487 {
1488 if (strStatus.size() > 130)
1489         m_textCtrlStatus->SetValue(string("\n") + strStatus);
1490 else
1491         m_textCtrlStatus->SetValue(string("\n\n") + strStatus);
1492     m_staticTextSending->SetFocus();
1493 if (!fCanCancel)
1494         m_buttonCancel->Enable(false);
1495 if (fWorkDone)
1496 {
1497         m_buttonOK->Enable(true);
1498         m_buttonOK->SetFocus();
1499         m_buttonCancel->Enable(false);
1500 }
1501 if (fAbort && fCanCancel && IsShown())
1502 {
1503         strStatus = "CANCELLED";
1504         m_buttonOK->Enable(true);
1505         m_buttonOK->SetFocus();
1506         m_buttonCancel->Enable(false);
1507         m_buttonCancel->SetLabel("Cancelled");
1508 Close();
1509 wxMessageBox("Transfer cancelled ", "Sending...", wxOK, this);
1510 }
1511     event.Skip();
1512
1513 /// debug test
1514 if (fRandSendTest && fWorkDone && fSuccess)
1515 {
1516 Close();
1517 Sleep(1000);
1518 RandSend();
1519 }
1520 }
1521
```

```
1522
1523 //
1524 // Everything from here on is not in the UI thread and must only communicate
1525 // with the rest of the dialog through variables and calling repaint.
1526 //
1527
```

CALLED FROM CSendingDialog::Status p. 499

```
1528 void CSendingDialog::Repaint()
1529 {
1530 Refresh();
1531     wxPaintEvent event;
1532 AddPendingEvent(event);
1533 }
1534
1535 bool CSendingDialog::Status()
1536 {
1537 if (fUIDone)
1538 {
1539 Destroy();
1540 return false;
1541 }
1542 if (fAbort && fCanCancel)
1543 {
1544         strStatus = "CANCELLED";
1545 Repaint();
1546         fWorkDone = true;
1547 return false;
1548 }
1549 return true;
1550 }
1551
1552 bool CSendingDialog::Status(const string& str)
1553 {
1554 if (!Status())
1555 return false;
1556     strStatus = str;
1557 Repaint();
1558 return true;
1559 }
1560
```

CALLED FROM CSendingDialog::StartTransfer p. 499 · CSendingDialog::OnReply2 p. 500 · CSendingDialog::OnReply3 p. 501

```
1561 bool CSendingDialog::Error(const string& str)
1562 {
1563     fCanCancel = false;
1564     fWorkDone = true;
1565 Status(string("Error: ") + str);
1566 return false;
1567 }
1568
```

CALLED FROM CSendingDialog::CSendingDialog p. 498

```
1569 void SendingDialogStartTransfer(void* parg)
1570 {
1571 ((CSendingDialog*)parg)->StartTransfer();
1572 }
1573
```

`StartTransfer` — the IP-to-IP payment protocol: (1) connect to the recipient's node, (2) send a "checkorder" request carrying the order (at this point just the to/from/message fields — no inputs or outputs yet), (3) receive a response with the recipient's public key (OnReply2), (4) build and sign the transaction, (5) send "submitorder" with the signed payment. The recipient's wallet responds automatically without user action.

CALLED FROM SendingDialogStartTransfer p. 499

```
1574 void CSendingDialog::StartTransfer()
1575 {
1576 // Make sure we have enough money
1577 if (nPrice + nTransactionFee > GetBalance())
1578 {
1579 Error("You don't have enough money");
1580 return;
1581 }
1582
1583 // We may have connected already for product details
1584 if (!Status("Connecting..."))
1585 return;
1586     CNode* pnode = ConnectNode(addr, 5 * 60);
1587 if (!pnode)
1588 {
```

```
}

// Send order to seller, with response going to OnReply2 via event handler
if (!Status("Requesting public key..."))
return;
```

PushRequest("checkorder") — sends the order to the recipient for approval. The protocol leaves room for a refusal, but v0.1.0's handler never refuses: it carries only the comment '/// we have a chance to check the order here' and always replies with a scriptPubKey. That script is pay-to-public-key (<pubkey> OP_CHECKSIG), not a pay-to-address script, and the key is reused per sender IP ('Keep giving the same key to the same ip until they use it'); a new key is generated only after a payment from that IP arrives.

```
    pnode->PushRequest("checkorder", wtx, SendingDialogOnReply2, this);
}

void SendingDialogOnReply2(void* parg, CDataStream& vRecv)
{
((CSendingDialog*)parg)->OnReply2(vRecv);
}
```

CALLED FROM SendingDialogOnReply2 p. 500

```
void CSendingDialog::OnReply2(CDataStream& vRecv)
{
if (!Status("Received public key..."))
return;

    CScript scriptPubKey;
int nRet;
try
{
        vRecv >> nRet;
if (nRet > 0)
{
            string strMessage;
            vRecv >> strMessage;
Error("Transfer was not accepted");
//// todo: enlarge the window and enable a hidden white box to put seller's message
return;
}
        vRecv >> scriptPubKey;
}
catch (...)
{
//// what do we want to do about this?
Error("Invalid response received");
return;
}

// Should already be connected
    CNode* pnode = ConnectNode(addr, 5 * 60);
if (!pnode)
{
Error("Lost connection");
return;
}

// Pause to give the user a chance to cancel
```

2-second cancel window: after receiving the recipient's public key but before committing the transaction, the dialog waits until at least 2 seconds have passed since it opened (start), so a fast reply still leaves the user a moment. During this window the user can still cancel. Once CommitTransactionSpent() is called below, the coins are gone — hence fCanCancel = false immediately before commit.

```
while (wxDateTime::UNow() < start + wxTimeSpan(0, 0, 0, 2 * 1000))
{
Sleep(200);
if (!Status())
return;
}

CRITICAL_BLOCK(cs_main)
{
```

```
// Pay
if (!Status("Creating transaction..."))
return;
if (nPrice + nTransactionFee > GetBalance())
{
Error("You don't have enough money");
return;
}
        int64 nFeeRequired;
if (!CreateTransaction(scriptPubKey, nPrice, wtx, nFeeRequired))
{
if (nPrice + nFeeRequired > GetBalance())
Error(strprintf("This is an oversized transaction that requires a transaction fee of %s", FormatMoney(nFeeRequired).c_str()));
else
Error("Transaction creation failed");
return;
}

// Last chance to cancel
Sleep(50);
if (!Status())
return;
        fCanCancel = false;
if (fAbort)
{
            fCanCancel = true;
if (!Status())
return;
            fCanCancel = false;
}
if (!Status("Sending payment..."))
return;

// Commit
```

`CommitTransactionSpent` — marks the inputs as spent in the wallet database, making them unavailable for other transactions. This is irreversible locally. AcceptTransaction() then validates the transaction against the current mempool, and RelayWalletTransaction() broadcasts it to all connected peers.

```
if (!CommitTransactionSpent(wtx))
{
Error("Error finalizing payment");
return;
}

// Send payment tx to seller, with response going to OnReply3 via event handler
        pnode->PushRequest("submitorder", wtx, SendingDialogOnReply3, this);

// Accept and broadcast transaction
if (!wtx.AcceptTransaction())
printf("ERROR: CSendingDialog : wtxNew.AcceptTransaction() %s failed\n", wtx.GetHash().ToString().c_str());
        wtx.RelayWalletTransaction();

Status("Waiting for confirmation...");
MainFrameRepaint();
}
}

void SendingDialogOnReply3(void* parg, CDataStream& vRecv)
{
((CSendingDialog*)parg)->OnReply3(vRecv);
}
```

CALLED FROM SendingDialogOnReply3 p. 501

```
void CSendingDialog::OnReply3(CDataStream& vRecv)
{
int nRet;
try
{
        vRecv >> nRet;
if (nRet > 0)
{
Error("The payment was sent, but the recipient was unable to verify it.\n"
"The transaction is recorded and will credit to the recipient if it is valid,\n"
"but without comment information.");
return;
}
}
catch (...)
{
//// what do we want to do about this?
```

```
Error("Payment was sent, but an invalid response was received");
return;
}

    fSuccess = true;
    fWorkDone = true;
Status("Payment completed");
}




//////////////////////////////////////////////////////////////////////////////
//
// CYourAddressDialog
//

CYourAddressDialog::CYourAddressDialog(wxWindow* parent, const string& strInitSelected) : CYourAddressDialogBase(parent)
{
// Init column headers
    m_listCtrl->InsertColumn(0, "Name", wxLIST_FORMAT_LEFT, 200);
    m_listCtrl->InsertColumn(1, "Bitcoin Address", wxLIST_FORMAT_LEFT, 350);
    m_listCtrl->SetFocus();

// Fill listctrl with address book data
CRITICAL_BLOCK(cs_mapKeys)
{
foreach(const PAIRTYPE(string, string)& item, mapAddressBook)
{
            string strAddress = item.first;
            string strName = item.second;
            uint160 hash160;
bool fMine = (AddressToHash160(strAddress, hash160) && mapPubKeys.count(hash160));
if (fMine)
{
int nIndex = InsertLine(m_listCtrl, strName, strAddress);
if (strAddress == strInitSelected)
                    m_listCtrl->SetItemState(nIndex, wxLIST_STATE_SELECTED|wxLIST_STATE_FOCUSED,
    wxLIST_STATE_SELECTED|wxLIST_STATE_FOCUSED);
}
}
}
}

wxString CYourAddressDialog::GetAddress()
{
int nIndex = GetSelection(m_listCtrl);
if (nIndex == -1)
return "";
return GetItemText(m_listCtrl, nIndex, 1);
}

void CYourAddressDialog::OnListEndLabelEdit(wxListEvent& event)
{
// Update address book with edited name
if (event.IsEditCancelled())
return;
    string strAddress = (string)GetItemText(m_listCtrl, event.GetIndex(), 1);
SetAddressBookName(strAddress, string(event.GetText()));
    pframeMain->RefreshListCtrl();
}

void CYourAddressDialog::OnListItemSelected(wxListEvent& event)
{
}

void CYourAddressDialog::OnListItemActivated(wxListEvent& event)
{
// Doubleclick returns selection
EndModal(true);
}

void CYourAddressDialog::OnButtonRename(wxCommandEvent& event)
{
// Ask new name
int nIndex = GetSelection(m_listCtrl);
if (nIndex == -1)
return;
    string strName = (string)m_listCtrl->GetItemText(nIndex);
    string strAddress = (string)GetItemText(m_listCtrl, nIndex, 1);
    CGetTextFromUserDialog dialog(this, "Rename Bitcoin Address", "New Name", strName);
if (!dialog.ShowModal())
return;
    strName = dialog.GetValue();

// Change name
SetAddressBookName(strAddress, strName);
```

```
1812     m_listCtrl->SetItemText(nIndex, strName);
1813     pframeMain->RefreshListCtrl();
1814 }
1815
1816 void CYourAddressDialog::OnButtonNew(wxCommandEvent& event)
1817 {
1818 // Ask name
1819     CGetTextFromUserDialog dialog(this, "New Bitcoin Address", "Name", "");
1820 if (!dialog.ShowModal())
1821 return;
1822     string strName = dialog.GetValue();
1823
1824 // Generate new key
1825     string strAddress = PubKeyToAddress(GenerateNewKey());
1826 SetAddressBookName(strAddress, strName);
1827
1828 // Add to list and select it
1829 int nIndex = InsertLine(m_listCtrl, strName, strAddress);
1830 SetSelection(m_listCtrl, nIndex);
1831     m_listCtrl->SetFocus();
1832 }
1833
1834 void CYourAddressDialog::OnButtonCopy(wxCommandEvent& event)
1835 {
1836 // Copy address box to clipboard
1837 if (wxTheClipboard->Open())
1838 {
1839         wxTheClipboard->SetData(new wxTextDataObject(GetAddress()));
1840         wxTheClipboard->Close();
1841 }
1842 }
1843
1844 void CYourAddressDialog::OnButtonOK(wxCommandEvent& event)
1845 {
1846 // OK
1847 EndModal(true);
1848 }
1849
1850 void CYourAddressDialog::OnButtonCancel(wxCommandEvent& event)
1851 {
1852 // Cancel
1853 EndModal(false);
1854 }
1855
1856 void CYourAddressDialog::OnClose(wxCloseEvent& event)
1857 {
1858 // Close
1859 EndModal(false);
1860 }
1861
1862
1863
1864
1865
1866
1867 //////////////////////////////////////////////////////////////////////////////
1868 //
1869 // CAddressBookDialog
1870 //
1871
1872 CAddressBookDialog::CAddressBookDialog(wxWindow* parent, const wxString& strInitSelected, bool fSendingIn) :
    CAddressBookDialogBase(parent)
1873 {
1874     fSending = fSendingIn;
1875 if (!fSending)
1876         m_buttonCancel->Show(false);
1877
1878 // Init column headers
1879     m_listCtrl->InsertColumn(0, "Name", wxLIST_FORMAT_LEFT, 200);
1880     m_listCtrl->InsertColumn(1, "Address", wxLIST_FORMAT_LEFT, 350);
1881     m_listCtrl->SetFocus();
1882
1883 // Set Icon
1884     wxBitmap bmpAddressBook(wxT("addressbook16"), wxBITMAP_TYPE_RESOURCE);
1885     bmpAddressBook.SetMask(new wxMask(wxBitmap(wxT("addressbook16mask"), wxBITMAP_TYPE_RESOURCE)));
1886     wxIcon iconAddressBook;
1887     iconAddressBook.CopyFromBitmap(bmpAddressBook);
1888 SetIcon(iconAddressBook);
1889
1890 // Fill listctrl with address book data
1891 CRITICAL_BLOCK(cs_mapKeys)
1892 {
1893 foreach(const PAIRTYPE(string, string)& item, mapAddressBook)
1894 {
1895             string strAddress = item.first;
1896             string strName = item.second;
1897             uint160 hash160;
1898 bool fMine = (AddressToHash160(strAddress, hash160) && mapPubKeys.count(hash160));
1899 if (!fMine)
```

```
{
int nIndex = InsertLine(m_listCtrl, strName, strAddress);
if (strAddress == strInitSelected)
                    m_listCtrl->SetItemState(nIndex, wxLIST_STATE_SELECTED|wxLIST_STATE_FOCUSED,
    wxLIST_STATE_SELECTED|wxLIST_STATE_FOCUSED);
}
}
}
}

wxString CAddressBookDialog::GetAddress()
{
int nIndex = GetSelection(m_listCtrl);
if (nIndex == -1)
return "";
return GetItemText(m_listCtrl, nIndex, 1);
}

void CAddressBookDialog::OnListEndLabelEdit(wxListEvent& event)
{
// Update address book with edited name
if (event.IsEditCancelled())
return;
    string strAddress = (string)GetItemText(m_listCtrl, event.GetIndex(), 1);
SetAddressBookName(strAddress, string(event.GetText()));
    pframeMain->RefreshListCtrl();
}

void CAddressBookDialog::OnListItemSelected(wxListEvent& event)
{
}

void CAddressBookDialog::OnListItemActivated(wxListEvent& event)
{
if (fSending)
{
// Doubleclick returns selection
EndModal(GetAddress() != "" ? 2 : 0);
}
else
{
// Doubleclick edits item
        wxCommandEvent event;
OnButtonEdit(event);
}
}

```

CALLED FROM CAddressBookDialog::OnListItemActivated p. 504

```
void CAddressBookDialog::OnButtonEdit(wxCommandEvent& event)
{
// Ask new name
int nIndex = GetSelection(m_listCtrl);
if (nIndex == -1)
return;
    string strName = (string)m_listCtrl->GetItemText(nIndex);
    string strAddress = (string)GetItemText(m_listCtrl, nIndex, 1);
    string strAddressOrg = strAddress;
    CGetTextFromUserDialog dialog(this, "Edit Address", "Name", strName, "Address", strAddress);
if (!dialog.ShowModal())
return;
    strName = dialog.GetValue1();
    strAddress = dialog.GetValue2();

// Change name
if (strAddress != strAddressOrg)
CWalletDB().EraseName(strAddressOrg);
SetAddressBookName(strAddress, strName);
    m_listCtrl->SetItem(nIndex, 1, strAddress);
    m_listCtrl->SetItemText(nIndex, strName);
    pframeMain->RefreshListCtrl();
}

void CAddressBookDialog::OnButtonNew(wxCommandEvent& event)
{
// Ask name
    CGetTextFromUserDialog dialog(this, "New Address", "Name", "", "Address", "");
if (!dialog.ShowModal())
return;
    string strName = dialog.GetValue1();
    string strAddress = dialog.GetValue2();

// Add to list and select it
SetAddressBookName(strAddress, strName);
int nIndex = InsertLine(m_listCtrl, strName, strAddress);
SetSelection(m_listCtrl, nIndex);
    m_listCtrl->SetFocus();
}

void CAddressBookDialog::OnButtonDelete(wxCommandEvent& event)
```

```
{
for (int nIndex = m_listCtrl->GetItemCount()-1; nIndex >= 0; nIndex--)
{
if (m_listCtrl->GetItemState(nIndex, wxLIST_STATE_SELECTED))
{
            string strAddress = (string)GetItemText(m_listCtrl, nIndex, 1);
CWalletDB().EraseName(strAddress);
            m_listCtrl->DeleteItem(nIndex);
}
}
    pframeMain->RefreshListCtrl();
}

void CAddressBookDialog::OnButtonCopy(wxCommandEvent& event)
{
// Copy address box to clipboard
if (wxTheClipboard->Open())
{
        wxTheClipboard->SetData(new wxTextDataObject(GetAddress()));
        wxTheClipboard->Close();
}
}

void CAddressBookDialog::OnButtonOK(wxCommandEvent& event)
{
// OK
EndModal(GetAddress() != "" ? 1 : 0);
}

void CAddressBookDialog::OnButtonCancel(wxCommandEvent& event)
{
// Cancel
EndModal(0);
}

void CAddressBookDialog::OnClose(wxCloseEvent& event)
{
// Close
EndModal(0);
}









//////////////////////////////////////////////////////////////////////////////
//
// CProductsDialog
//

bool CompareIntStringPairBestFirst(const pair<int, string>& item1, const pair<int, string>& item2)
{
return (item1.first > item2.first);
}

CProductsDialog::CProductsDialog(wxWindow* parent) : CProductsDialogBase(parent)
{
// Init column headers
    m_listCtrl->InsertColumn(0, "Title",  wxLIST_FORMAT_LEFT, 200);
    m_listCtrl->InsertColumn(1, "Price",  wxLIST_FORMAT_LEFT, 80);
    m_listCtrl->InsertColumn(2, "Seller", wxLIST_FORMAT_LEFT, 80);
    m_listCtrl->InsertColumn(3, "Stars",  wxLIST_FORMAT_LEFT, 50);
    m_listCtrl->InsertColumn(4, "Power",  wxLIST_FORMAT_LEFT, 50);

// Tally top categories
    map<string, int> mapTopCategories;
CRITICAL_BLOCK(cs_mapProducts)
for (map<uint256, CProduct>::iterator mi = mapProducts.begin(); mi != mapProducts.end(); ++mi)
            mapTopCategories[(*mi).second.mapValue["category"]]++;

// Sort top categories
    vector<pair<int, string> > vTopCategories;
for (map<string, int>::iterator mi = mapTopCategories.begin(); mi != mapTopCategories.end(); ++mi)
        vTopCategories.push_back(make_pair((*mi).second, (*mi).first));
sort(vTopCategories.begin(), vTopCategories.end(), CompareIntStringPairBestFirst);

// Fill categories combo box
int nLimit = 250;
for (vector<pair<int, string> >::iterator it = vTopCategories.begin(); it != vTopCategories.end() && nLimit-- > 0; ++it)
        m_comboBoxCategory->Append((*it).second);

// Fill window with initial search
//wxCommandEvent event;
//OnButtonSearch(event);
}

void CProductsDialog::OnCombobox(wxCommandEvent& event)
{
```

```
OnButtonSearch(event);
}

bool CompareProductsBestFirst(const CProduct* p1, const CProduct* p2)
{
return (p1->nAtoms > p2->nAtoms);
}

```

CALLED FROM CProductsDialog::OnCombobox p. 506

```
void CProductsDialog::OnButtonSearch(wxCommandEvent& event)
{
    string strCategory = (string)m_comboBoxCategory->GetValue();
    string strSearch = (string)m_textCtrlSearch->GetValue();

// Search products
    vector<CProduct*> vProductsFound;
CRITICAL_BLOCK(cs_mapProducts)
{
for (map<uint256, CProduct>::iterator mi = mapProducts.begin(); mi != mapProducts.end(); ++mi)
{
            CProduct& product = (*mi).second;
if (product.mapValue["category"].find(strCategory) != -1)
{
if (product.mapValue["title"].find(strSearch) != -1 ||
                    product.mapValue["description"].find(strSearch) != -1 ||
                    product.mapValue["seller"].find(strSearch) != -1)
{
                    vProductsFound.push_back(&product);
}
}
}
}

// Sort
sort(vProductsFound.begin(), vProductsFound.end(), CompareProductsBestFirst);

// Display
foreach(CProduct* pproduct, vProductsFound)
{
InsertLine(m_listCtrl,
                   pproduct->mapValue["title"],
                   pproduct->mapValue["price"],
                   pproduct->mapValue["seller"],
                   pproduct->mapValue["stars"],
itostr(pproduct->nAtoms));
}
}

void CProductsDialog::OnListItemActivated(wxListEvent& event)
{
// Doubleclick opens product
    CViewProductDialog* pdialog = new CViewProductDialog(this, m_vProduct[event.GetIndex()]);
    pdialog->Show();
}









//////////////////////////////////////////////////////////////////////////////
//
// CEditProductDialog
//

CEditProductDialog::CEditProductDialog(wxWindow* parent) : CEditProductDialogBase(parent)
{
    m_textCtrlLabel[0 ] = m_textCtrlLabel0;
    m_textCtrlLabel[1 ] = m_textCtrlLabel1;
    m_textCtrlLabel[2 ] = m_textCtrlLabel2;
    m_textCtrlLabel[3 ] = m_textCtrlLabel3;
    m_textCtrlLabel[4 ] = m_textCtrlLabel4;
    m_textCtrlLabel[5 ] = m_textCtrlLabel5;
    m_textCtrlLabel[6 ] = m_textCtrlLabel6;
    m_textCtrlLabel[7 ] = m_textCtrlLabel7;
    m_textCtrlLabel[8 ] = m_textCtrlLabel8;
    m_textCtrlLabel[9 ] = m_textCtrlLabel9;
    m_textCtrlLabel[10] = m_textCtrlLabel10;
    m_textCtrlLabel[11] = m_textCtrlLabel11;
    m_textCtrlLabel[12] = m_textCtrlLabel12;
    m_textCtrlLabel[13] = m_textCtrlLabel13;
    m_textCtrlLabel[14] = m_textCtrlLabel14;
    m_textCtrlLabel[15] = m_textCtrlLabel15;
    m_textCtrlLabel[16] = m_textCtrlLabel16;
    m_textCtrlLabel[17] = m_textCtrlLabel17;
    m_textCtrlLabel[18] = m_textCtrlLabel18;
    m_textCtrlLabel[19] = m_textCtrlLabel19;

```

```
    m_textCtrlField[0 ] = m_textCtrlField0;
    m_textCtrlField[1 ] = m_textCtrlField1;
    m_textCtrlField[2 ] = m_textCtrlField2;
    m_textCtrlField[3 ] = m_textCtrlField3;
    m_textCtrlField[4 ] = m_textCtrlField4;
    m_textCtrlField[5 ] = m_textCtrlField5;
    m_textCtrlField[6 ] = m_textCtrlField6;
    m_textCtrlField[7 ] = m_textCtrlField7;
    m_textCtrlField[8 ] = m_textCtrlField8;
    m_textCtrlField[9 ] = m_textCtrlField9;
    m_textCtrlField[10] = m_textCtrlField10;
    m_textCtrlField[11] = m_textCtrlField11;
    m_textCtrlField[12] = m_textCtrlField12;
    m_textCtrlField[13] = m_textCtrlField13;
    m_textCtrlField[14] = m_textCtrlField14;
    m_textCtrlField[15] = m_textCtrlField15;
    m_textCtrlField[16] = m_textCtrlField16;
    m_textCtrlField[17] = m_textCtrlField17;
    m_textCtrlField[18] = m_textCtrlField18;
    m_textCtrlField[19] = m_textCtrlField19;

    m_buttonDel[0 ] = m_buttonDel0;
    m_buttonDel[1 ] = m_buttonDel1;
    m_buttonDel[2 ] = m_buttonDel2;
    m_buttonDel[3 ] = m_buttonDel3;
    m_buttonDel[4 ] = m_buttonDel4;
    m_buttonDel[5 ] = m_buttonDel5;
    m_buttonDel[6 ] = m_buttonDel6;
    m_buttonDel[7 ] = m_buttonDel7;
    m_buttonDel[8 ] = m_buttonDel8;
    m_buttonDel[9 ] = m_buttonDel9;
    m_buttonDel[10] = m_buttonDel10;
    m_buttonDel[11] = m_buttonDel11;
    m_buttonDel[12] = m_buttonDel12;
    m_buttonDel[13] = m_buttonDel13;
    m_buttonDel[14] = m_buttonDel14;
    m_buttonDel[15] = m_buttonDel15;
    m_buttonDel[16] = m_buttonDel16;
    m_buttonDel[17] = m_buttonDel17;
    m_buttonDel[18] = m_buttonDel18;
    m_buttonDel[19] = m_buttonDel19;

for (int i = 1; i < FIELDS_MAX; i++)
ShowLine(i, false);

LayoutAll();
}
```

CALLED FROM CEditProductDialog::CEditProductDialog p. 507 · CEditProductDialog::OnButtonDel p. 508 · CEditProductDialog::OnButtonAddField p. 508

```
void CEditProductDialog::LayoutAll()
{
    m_scrolledWindow->Layout();
    m_scrolledWindow->GetSizer()->Fit(m_scrolledWindow);
this->Layout();
}
```

CALLED FROM CEditProductDialog::CEditProductDialog p. 507 · CEditProductDialog::OnButtonDel p. 508 · CEditProductDialog::OnButtonAddField p. 508

```
void CEditProductDialog::ShowLine(int i, bool fShow)
{
    m_textCtrlLabel[i]->Show(fShow);
    m_textCtrlField[i]->Show(fShow);
    m_buttonDel[i]->Show(fShow);
}

void CEditProductDialog::OnButtonDel0(wxCommandEvent& event) { OnButtonDel(event, 0); }
void CEditProductDialog::OnButtonDel1(wxCommandEvent& event) { OnButtonDel(event, 1); }
void CEditProductDialog::OnButtonDel2(wxCommandEvent& event) { OnButtonDel(event, 2); }
void CEditProductDialog::OnButtonDel3(wxCommandEvent& event) { OnButtonDel(event, 3); }
void CEditProductDialog::OnButtonDel4(wxCommandEvent& event) { OnButtonDel(event, 4); }
void CEditProductDialog::OnButtonDel5(wxCommandEvent& event) { OnButtonDel(event, 5); }
void CEditProductDialog::OnButtonDel6(wxCommandEvent& event) { OnButtonDel(event, 6); }
void CEditProductDialog::OnButtonDel7(wxCommandEvent& event) { OnButtonDel(event, 7); }
void CEditProductDialog::OnButtonDel8(wxCommandEvent& event) { OnButtonDel(event, 8); }
void CEditProductDialog::OnButtonDel9(wxCommandEvent& event) { OnButtonDel(event, 9); }
void CEditProductDialog::OnButtonDel10(wxCommandEvent& event) { OnButtonDel(event, 10); }
void CEditProductDialog::OnButtonDel11(wxCommandEvent& event) { OnButtonDel(event, 11); }
void CEditProductDialog::OnButtonDel12(wxCommandEvent& event) { OnButtonDel(event, 12); }
void CEditProductDialog::OnButtonDel13(wxCommandEvent& event) { OnButtonDel(event, 13); }
void CEditProductDialog::OnButtonDel14(wxCommandEvent& event) { OnButtonDel(event, 14); }
void CEditProductDialog::OnButtonDel15(wxCommandEvent& event) { OnButtonDel(event, 15); }
void CEditProductDialog::OnButtonDel16(wxCommandEvent& event) { OnButtonDel(event, 16); }
void CEditProductDialog::OnButtonDel17(wxCommandEvent& event) { OnButtonDel(event, 17); }
void CEditProductDialog::OnButtonDel18(wxCommandEvent& event) { OnButtonDel(event, 18); }
void CEditProductDialog::OnButtonDel19(wxCommandEvent& event) { OnButtonDel(event, 19); }
```

CALLED FROM CEditProductDialog::OnButtonDel0 p. 507 · CEditProductDialog::OnButtonDel1 p. 507 · CEditProductDialog::OnButtonDel2 p. 507 · CEditProductDialog::OnButtonDel3 p. 507 · CEditProductDialog::OnButtonDel4 p. 507 · CEditProductDialog::OnButtonDel5 p. 507 · *and 14 more*

```
2247 void CEditProductDialog::OnButtonDel(wxCommandEvent& event, int n)
2248 {
2249 Freeze();
2250 int x, y;
2251     m_scrolledWindow->GetViewStart(&x, &y);
2252 int i;
2253 for (i = n; i < FIELDS_MAX-1; i++)
2254 {
2255         m_textCtrlLabel[i]->SetValue(m_textCtrlLabel[i+1]->GetValue());
2256         m_textCtrlField[i]->SetValue(m_textCtrlField[i+1]->GetValue());
2257 if (!m_buttonDel[i+1]->IsShown())
2258 break;
2259 }
2260     m_textCtrlLabel[i]->SetValue("");
2261     m_textCtrlField[i]->SetValue("");
2262 ShowLine(i, false);
2263     m_buttonAddField->Enable(true);
2264 LayoutAll();
2265     m_scrolledWindow->Scroll(0, y);
2266 Thaw();
2267 }
2268
2269 void CEditProductDialog::OnButtonAddField(wxCommandEvent& event)
2270 {
2271 for (int i = 0; i < FIELDS_MAX; i++)
2272 {
2273 if (!m_buttonDel[i]->IsShown())
2274 {
2275 Freeze();
2276 ShowLine(i, true);
2277 if (i == FIELDS_MAX-1)
2278                 m_buttonAddField->Enable(false);
2279 LayoutAll();
2280             m_scrolledWindow->Scroll(0, 99999);
2281 Thaw();
2282 break;
2283 }
2284 }
2285 }
2286
2287 void CEditProductDialog::OnButtonSend(wxCommandEvent& event)
2288 {
2289     CProduct product;
2290 GetProduct(product);
2291
2292 // Sign the detailed product
2293     product.vchPubKeyFrom = keyUser.GetPubKey();
2294 if (!keyUser.Sign(product.GetSigHash(), product.vchSig))
2295 {
2296 wxMessageBox("Error digitally signing the product ");
2297 return;
2298 }
2299
2300 // Save detailed product
2301 AddToMyProducts(product);
2302
2303 // Strip down to summary product
2304     product.mapDetails.clear();
2305     product.vOrderForm.clear();
2306
2307 // Sign the summary product
2308 if (!keyUser.Sign(product.GetSigHash(), product.vchSig))
2309 {
2310 wxMessageBox("Error digitally signing the product ");
2311 return;
2312 }
2313
2314 // Verify
2315 if (!product.CheckProduct())
2316 {
2317 wxMessageBox("Errors found in product ");
2318 return;
2319 }
2320
2321 // Broadcast
2322 AdvertStartPublish(pnodeLocalHost, MSG_PRODUCT, 0, product);
2323
2324 Destroy();
2325 }
2326
2327 void CEditProductDialog::OnButtonPreview(wxCommandEvent& event)
2328 {
2329     CProduct product;
2330 GetProduct(product);
2331     CViewProductDialog* pdialog = new CViewProductDialog(this, product);
```

```
    pdialog->Show();
}

void CEditProductDialog::OnButtonCancel(wxCommandEvent& event)
{
Destroy();
}
```

CALLED FROM CMainFrame::OnListItemActivatedProductsSent p. 491

```
void CEditProductDialog::SetProduct(const CProduct& productIn)
{
    CProduct product = productIn;

    m_comboBoxCategory->SetValue(product.mapValue["category"]);
    m_textCtrlTitle->SetValue(product.mapValue["title"]);
    m_textCtrlPrice->SetValue(product.mapValue["price"]);
    m_textCtrlDescription->SetValue(product.mapValue["description"]);
    m_textCtrlInstructions->SetValue(product.mapValue["instructions"]);

for (int i = 0; i < FIELDS_MAX; i++)
{
bool fUsed = i < product.vOrderForm.size();
        m_buttonDel[i]->Show(fUsed);
        m_textCtrlLabel[i]->Show(fUsed);
        m_textCtrlField[i]->Show(fUsed);
if (!fUsed)
continue;

        m_textCtrlLabel[i]->SetValue(product.vOrderForm[i].first);
        string strControl = product.vOrderForm[i].second;
if (strControl.substr(0, 5) == "text=")
            m_textCtrlField[i]->SetValue("");
else if (strControl.substr(0, 7) == "choice=")
            m_textCtrlField[i]->SetValue(strControl.substr(7));
else
            m_textCtrlField[i]->SetValue(strControl);
}
}
```

CALLED FROM CEditProductDialog::OnButtonSend p. 508 · CEditProductDialog::OnButtonPreview p. 508

```
void CEditProductDialog::GetProduct(CProduct& product)
{
// map<string, string> mapValue;
// vector<pair<string, string> > vOrderForm;

    product.mapValue["category"] = m_comboBoxCategory->GetValue().Trim();
    product.mapValue["title"] = m_textCtrlTitle->GetValue().Trim();
    product.mapValue["price"] = m_textCtrlPrice->GetValue().Trim();
    product.mapValue["description"] = m_textCtrlDescription->GetValue().Trim();
    product.mapValue["instructions"] = m_textCtrlInstructions->GetValue().Trim();

for (int i = 0; i < FIELDS_MAX; i++)
{
if (m_buttonDel[i]->IsShown())
{
            string strLabel = (string)m_textCtrlLabel[i]->GetValue().Trim();
            string strControl = (string)m_textCtrlField[i]->GetValue();
if (strControl.empty())
                strControl = "text=";
else
                strControl = "choice=" + strControl;
            product.vOrderForm.push_back(make_pair(strLabel, strControl));
}
}
}
```

```
//////////////////////////////////////////////////////////////////////////////
//
// CViewProductDialog
//

CViewProductDialog::CViewProductDialog(wxWindow* parent, const CProduct& productIn) : CViewProductDialogBase(parent)
{
Connect(wxEVT_REPLY1, wxCommandEventHandler(CViewProductDialog::OnReply1), NULL, this);
AddCallbackAvailable(GetEventHandler());

// Fill display with product summary while waiting for details
    product = productIn;
UpdateProductDisplay(false);

    m_buttonBack->Enable(false);
    m_buttonNext->Enable(!product.vOrderForm.empty());
```

```
    m_htmlWinReviews->Show(true);
    m_scrolledWindow->Show(false);
this->Layout();

// Request details from seller
_beginthread(ThreadRequestProductDetails, 0, new pair<CProduct, wxEvtHandler*>(product, GetEventHandler()));
}

CViewProductDialog::~CViewProductDialog()
{
RemoveCallbackAvailable(GetEventHandler());
}

```

CALLED FROM CViewProductDialog::CViewProductDialog p. 510

```
void ThreadRequestProductDetails(void* parg)
{
// Extract parameters
    pair<CProduct, wxEvtHandler*>* pitem = (pair<CProduct, wxEvtHandler*>*)parg;
    CProduct product = pitem->first;
    wxEvtHandler* pevthandler = pitem->second;
delete pitem;

// Connect to seller
    CNode* pnode = ConnectNode(product.addr, 5 * 60);
if (!pnode)
{
        CDataStream ssEmpty;
AddPendingReplyEvent1(pevthandler, ssEmpty);
return;
}

// Request detailed product, with response going to OnReply1 via dialog's event handler
    pnode->PushRequest("getdetails", product.GetHash(), AddPendingReplyEvent1, (void*)pevthandler);
}

void CViewProductDialog::OnReply1(wxCommandEvent& event)
{
    CDataStream ss = GetStreamFromEvent(event);
if (ss.empty())
{
        product.mapValue["description"] = "-- CAN'T CONNECT TO SELLER --\n";
UpdateProductDisplay(true);
return;
}

int nRet;
    CProduct product2;
try
{
        ss >> nRet;
if (nRet > 0)
throw false;
        ss >> product2;
if (product2.GetHash() != product.GetHash())
throw false;
if (!product2.CheckSignature())
throw false;
}
catch (...)
{
        product.mapValue["description"] = "-- INVALID RESPONSE --\n";
UpdateProductDisplay(true);
return;
}

    product = product2;
UpdateProductDisplay(true);
}

bool CompareReviewsBestFirst(const CReview* p1, const CReview* p2)
{
return (p1->nAtoms > p2->nAtoms);
}

```

CALLED FROM CViewProductDialog::CViewProductDialog p. 509 · CViewProductDialog::OnReply1 p. 510

```
void CViewProductDialog::UpdateProductDisplay(bool fDetails)
{
// Product and reviews
    string strHTML;
    strHTML.reserve(4000);
    strHTML += "<html>\n"
"<head>\n"
"<meta http-equiv=\"content-type\" content=\"text/html; charset=UTF-8\">\n"
"</head>\n"
"<body>\n";
    strHTML += "<b>Category:</b> " + HtmlEscape(product.mapValue["category"]) + "<br>\n";
    strHTML += "<b>Title:</b> " + HtmlEscape(product.mapValue["title"]) + "<br>\n";
    strHTML += "<b>Price:</b> " + HtmlEscape(product.mapValue["price"]) + "<br>\n";
```

```
2504
2505 if (!fDetails)
2506         strHTML += "<b>Loading details...</b><br>\n<br>\n";
2507 else
2508         strHTML += HtmlEscape(product.mapValue["description"], true) + "<br>\n<br>\n";
2509
2510     strHTML += "<b>Reviews:</b><br>\n<br>\n";
2511
2512 if (!product.vchPubKeyFrom.empty())
2513 {
2514         CReviewDB reviewdb("r");
2515
2516 // Get reviews
2517         vector<CReview> vReviews;
2518         reviewdb.ReadReviews(product.GetUserHash(), vReviews);
2519
2520 // Get reviewer's number of atoms
2521         vector<CReview*> vSortedReviews;
2522         vSortedReviews.reserve(vReviews.size());
2523 for (vector<CReview>::reverse_iterator it = vReviews.rbegin(); it != vReviews.rend(); ++it)
2524 {
2525             CReview& review = *it;
2526             CUser user;
2527             reviewdb.ReadUser(review.GetUserHash(), user);
2528             review.nAtoms = user.GetAtomCount();
2529             vSortedReviews.push_back(&review);
2530 }
2531
2532         reviewdb.Close();
2533
2534 // Sort
2535 stable_sort(vSortedReviews.begin(), vSortedReviews.end(), CompareReviewsBestFirst);
2536
2537 // Format reviews
2538 foreach(CReview* preview, vSortedReviews)
2539 {
2540             CReview& review = *preview;
2541 int nStars = atoi(review.mapValue["stars"].c_str());
2542 if (nStars < 1 || nStars > 5)
2543 continue;
2544
2545             strHTML += "<b>" + itostr(nStars) + (nStars == 1 ? " star" : " stars") + "</b>";
2546             strHTML += "     ";
2547             strHTML += DateStr(atoi64(review.mapValue["date"])) + "<br>\n";
2548             strHTML += HtmlEscape(review.mapValue["review"], true);
2549             strHTML += "<br>\n<br>\n";
2550 }
2551 }
2552
2553     strHTML += "</body>\n</html>\n";
2554
2555 // Shrink capacity to fit
2556 string(strHTML.begin(), strHTML.end()).swap(strHTML);
2557
2558     m_htmlWinReviews->SetPage(strHTML);
2559
2560 ///// need to find some other indicator to use so can allow empty order form
2561 if (product.vOrderForm.empty())
2562 return;
2563
2564 // Order form
2565     m_staticTextInstructions->SetLabel(product.mapValue["instructions"]);
2566 for (int i = 0; i < FIELDS_MAX; i++)
2567 {
2568         m_staticTextLabel[i] = NULL;
2569         m_textCtrlField[i] = NULL;
2570         m_choiceField[i] = NULL;
2571 }
2572
2573 // Construct flexgridsizer
2574     wxBoxSizer* bSizer21 = (wxBoxSizer*)m_scrolledWindow->GetSizer();
2575     wxFlexGridSizer* fgSizer;
2576     fgSizer = new wxFlexGridSizer(0, 2, 0, 0);
2577     fgSizer->AddGrowableCol(1);
2578     fgSizer->SetFlexibleDirection(wxBOTH);
2579     fgSizer->SetNonFlexibleGrowMode(wxFLEX_GROWMODE_SPECIFIED);
2580
2581 // Construct order form fields
2582     wxWindow* windowLast = NULL;
2583 for (int i = 0; i < product.vOrderForm.size(); i++)
2584 {
2585         string strLabel = product.vOrderForm[i].first;
2586         string strControl = product.vOrderForm[i].second;
2587
2588 if (strLabel.size() < 20)
2589             strLabel.insert(strLabel.begin(), 20 - strLabel.size(), ' ');
2590
2591         m_staticTextLabel[i] = new wxStaticText(m_scrolledWindow, wxID_ANY, strLabel, wxDefaultPosition, wxDefaultSize,
    wxALIGN_RIGHT);
```

```
2592        m_staticTextLabel[i]->Wrap(-1);
2593        fgSizer->Add(m_staticTextLabel[i], 0, wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxALL, 5);
2594
2595 if (strControl.substr(0, 5) == "text=")
2596 {
2597            m_textCtrlField[i] = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize,
   0);
2598            fgSizer->Add(m_textCtrlField[i], 1, wxALL|wxALIGN_CENTER_VERTICAL|wxEXPAND, 5);
2599            windowLast = m_textCtrlField[i];
2600 }
2601 else if (strControl.substr(0, 7) == "choice=")
2602 {
2603            vector<string> vChoices;
2604 ParseString(strControl.substr(7), ',', vChoices);
2605
2606            wxArrayString arraystring;
2607 foreach(const string& str, vChoices)
2608                arraystring.Add(str);
2609
2610            m_choiceField[i] = new wxChoice(m_scrolledWindow, wxID_ANY, wxDefaultPosition, wxDefaultSize, arraystring, 0);
2611            fgSizer->Add(m_choiceField[i], 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
2612            windowLast = m_choiceField[i];
2613 }
2614 else
2615 {
2616            m_textCtrlField[i] = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize,
   0);
2617            fgSizer->Add(m_textCtrlField[i], 1, wxALL|wxALIGN_CENTER_VERTICAL|wxEXPAND, 5);
2618            m_staticTextLabel[i]->Show(false);
2619            m_textCtrlField[i]->Show(false);
2620 }
2621 }
2622
2623 // Insert after instructions and before submit/cancel buttons
2624     bSizer21->Insert(2, fgSizer, 0, wxEXPAND|wxRIGHT|wxLEFT, 5);
2625     m_scrolledWindow->Layout();
2626     bSizer21->Fit(m_scrolledWindow);
2627 this->Layout();
2628
2629 // Fixup the tab order
2630     m_buttonSubmitForm->MoveAfterInTabOrder(windowLast);
2631     m_buttonCancelForm->MoveAfterInTabOrder(m_buttonSubmitForm);
2632 //m_buttonBack->MoveAfterInTabOrder(m_buttonCancelForm);
2633 //m_buttonNext->MoveAfterInTabOrder(m_buttonBack);
2634 //m_buttonCancel->MoveAfterInTabOrder(m_buttonNext);
2635 this->Layout();
2636 }
2637
```

CALLED FROM CViewProductDialog::OnButtonSubmitForm p. 512

```
2638 void CViewProductDialog::GetOrder(CWalletTx& wtx)
2639 {
2640     wtx.SetNull();
2641 for (int i = 0; i < product.vOrderForm.size(); i++)
2642 {
2643        string strValue;
2644 if (m_textCtrlField[i])
2645            strValue = m_textCtrlField[i]->GetValue().Trim();
2646 else
2647            strValue = m_choiceField[i]->GetStringSelection();
2648        wtx.vOrderForm.push_back(make_pair(m_staticTextLabel[i]->GetLabel(), strValue));
2649 }
2650 }
2651
2652 void CViewProductDialog::OnButtonSubmitForm(wxCommandEvent& event)
2653 {
2654     m_buttonSubmitForm->Enable(false);
2655     m_buttonCancelForm->Enable(false);
2656
2657     CWalletTx wtx;
2658 GetOrder(wtx);
2659
2660     CSendingDialog* pdialog = new CSendingDialog(this, product.addr, atoi64(product.mapValue["price"]), wtx);
2661 if (!pdialog->ShowModal())
2662 {
2663        m_buttonSubmitForm->Enable(true);
2664        m_buttonCancelForm->Enable(true);
2665 return;
2666 }
2667 }
2668
2669 void CViewProductDialog::OnButtonCancelForm(wxCommandEvent& event)
2670 {
2671 Destroy();
2672 }
2673
2674 void CViewProductDialog::OnButtonBack(wxCommandEvent& event)
2675 {
2676 Freeze();
2677     m_htmlWinReviews->Show(true);
```

```
    m_scrolledWindow->Show(false);
    m_buttonBack->Enable(false);
    m_buttonNext->Enable(!product.vOrderForm.empty());
this->Layout();
Thaw();
}

void CViewProductDialog::OnButtonNext(wxCommandEvent& event)
{
if (!product.vOrderForm.empty())
{
Freeze();
        m_htmlWinReviews->Show(false);
        m_scrolledWindow->Show(true);
        m_buttonBack->Enable(true);
        m_buttonNext->Enable(false);
this->Layout();
Thaw();
}
}

void CViewProductDialog::OnButtonCancel(wxCommandEvent& event)
{
Destroy();
}









//////////////////////////////////////////////////////////////////////////////
//
// CViewOrderDialog
//

CViewOrderDialog::CViewOrderDialog(wxWindow* parent, CWalletTx order, bool fReceived) : CViewOrderDialogBase(parent)
{
    int64 nPrice = (fReceived ? order.GetCredit() : order.GetDebit());

    string strHTML;
    strHTML.reserve(4000);
    strHTML += "<html>\n"
"<head>\n"
"<meta http-equiv=\"content-type\" content=\"text/html; charset=UTF-8\">\n"
"</head>\n"
"<body>\n";
    strHTML += "<b>Time:</b> " + HtmlEscape(DateTimeStr(order.nTimeReceived)) + "<br>\n";
    strHTML += "<b>Price:</b> " + HtmlEscape(FormatMoney(nPrice)) + "<br>\n";
    strHTML += "<b>Status:</b> " + HtmlEscape(FormatTxStatus(order)) + "<br>\n";

    strHTML += "<table>\n";
for (int i = 0; i < order.vOrderForm.size(); i++)
{
        strHTML += " <tr><td><b>" + HtmlEscape(order.vOrderForm[i].first) + ":</b></td>";
        strHTML += "<td>" + HtmlEscape(order.vOrderForm[i].second) + "</td></tr>\n";
}
    strHTML += "</table>\n";

    strHTML += "</body>\n</html>\n";

// Shrink capacity to fit
// (strings are ref counted, so it may live on in SetPage)
string(strHTML.begin(), strHTML.end()).swap(strHTML);

    m_htmlWin->SetPage(strHTML);
}

void CViewOrderDialog::OnButtonOK(wxCommandEvent& event)
{
Destroy();
}









//////////////////////////////////////////////////////////////////////////////
//
// CEditReviewDialog
//

CEditReviewDialog::CEditReviewDialog(wxWindow* parent) : CEditReviewDialogBase(parent)
{
}

```

```
2767 void CEditReviewDialog::OnButtonSubmit(wxCommandEvent& event)
2768 {
2769 if (m_choiceStars->GetSelection() == -1)
2770 {
2771 wxMessageBox("Please select a rating ");
2772 return;
2773 }
2774
2775     CReview review;
2776 GetReview(review);
2777
2778 // Sign the review
2779     review.vchPubKeyFrom = keyUser.GetPubKey();
2780 if (!keyUser.Sign(review.GetSigHash(), review.vchSig))
2781 {
2782 wxMessageBox("Unable to digitally sign the review ");
2783 return;
2784 }
2785
2786 // Broadcast
2787 if (!review.AcceptReview())
2788 {
2789 wxMessageBox("Save failed ");
2790 return;
2791 }
2792 RelayMessage(CInv(MSG_REVIEW, review.GetHash()), review);
2793
2794 Destroy();
2795 }
2796
2797 void CEditReviewDialog::OnButtonCancel(wxCommandEvent& event)
2798 {
2799 Destroy();
2800 }
2801
```

CALLED FROM CEditReviewDialog::OnButtonSubmit p. 514

```
2802 void CEditReviewDialog::GetReview(CReview& review)
2803 {
2804     review.mapValue["time"] = i64tostr(GetAdjustedTime());
2805     review.mapValue["stars"] = itostr(m_choiceStars->GetSelection()+1);
2806     review.mapValue["review"] = m_textCtrlReview->GetValue();
2807 }
2808
2809
2810
2811
2812
2813
2814
2815
2816
2817
2818
2819
2820 //////////////////////////////////////////////////////////////////////////////
2821 //
2822 // CMyApp
2823 //
2824
2825 // Define a new application
2826 class CMyApp: public wxApp
2827 {
2828 public:
2829 CMyApp(){};
2830 ~CMyApp(){};
2831 bool OnInit();
2832 bool OnInit2();
2833 int OnExit();
2834
2835 // 2nd-level exception handling: we get all the exceptions occurring in any
2836 // event handler here
2837 virtual bool OnExceptionInMainLoop();
2838
2839 // 3rd, and final, level exception handling: whenever an unhandled
2840 // exception is caught, this function is called
2841 virtual void OnUnhandledException();
2842
2843 // and now for something different: this function is called in case of a
2844 // crash (e.g. dereferencing null pointer, division by 0, ...)
2845 virtual void OnFatalException();
2846 };
2847
2848 IMPLEMENT_APP(CMyApp)
2849
```

`CMyApp::OnInit()` — wxWidgets entry point (this is `main`). Wraps `OnInit2` in a try/catch that logs a startup failure and shows a message box — but `PrintException()` then re-throws (`throw;` in util.cpp), so the `return false` after the catch is never reached and the exception still ends the program.

```
2850 bool CMyApp::OnInit()
2851 {
2852 try
2853 {
2854 return OnInit2();
2855 }
2856 catch (std::exception& e) {
2857 PrintException(&e, "OnInit()");
2858 } catch (...) {
2859 PrintException(NULL, "OnInit()");
2860 }
2861 return false;
2862 }
2863
```

CALLED FROM CMyApp::OnInit2 p. 516

```
2864 map<string, string> ParseParameters(int argc, char* argv[])
2865 {
2866     map<string, string> mapArgs;
2867 for (int i = 0; i < argc; i++)
2868 {
2869 char psz[10000];
2870 strcpy(psz, argv[i]);
2871 char* pszValue = "";
2872 if (strchr(psz, '='))
2873 {
2874             pszValue = strchr(psz, '=');
2875 *pszValue++ = '\0';
2876 }
2877 strlwr(psz);
2878 if (psz[0] == '-')
2879             psz[0] = '/';
2880         mapArgs[psz] = pszValue;
2881 }
2882 return mapArgs;
2883 }
2884
```

`OnInit2()` — real startup: single-instance check, parse argv, load wallet/block index, then show the main frame, call `StartNode`, and optionally spawn the miner. Bitcoin boots as a desktop app, not a daemon.

CALLED FROM CMyApp::OnInit p. 515

```
2885 bool CMyApp::OnInit2()
2886 {
2887 #ifdef _MSC_VER
2888 // Turn off microsoft heap dump noise for now
2889 _CrtSetReportMode(_CRT_WARN, _CRTDBG_MODE_FILE);
2890 _CrtSetReportFile(_CRT_WARN, CreateFile("NUL", GENERIC_WRITE, 0, NULL, OPEN_EXISTING, 0, 0));
2891 #endif
2892
2893 //// debug print
2894 printf("\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n");
2895 printf("Bitcoin CMyApp::OnInit()\n");
2896
2897 //
2898 // Limit to single instance per user
2899 // Required to protect the database files if we're going to keep deleting log.*
2900 //
2901     wxString strMutexName = wxString("Bitcoin.") + getenv("HOMEPATH");
2902 for (int i = 0; i < strMutexName.size(); i++)
2903 if (!isalnum(strMutexName[i]))
2904             strMutexName[i] = '.';
2905     wxSingleInstanceChecker* psingleinstancechecker = new wxSingleInstanceChecker(strMutexName);
2906 if (psingleinstancechecker->IsAnotherRunning())
2907 {
2908 printf("Existing instance found\n");
2909 unsigned int nStart = GetTime();
2910         loop
2911 {
2912 // Show the previous instance and exit
```

```
            HWND hwndPrev = FindWindow("wxWindowClassNR", "Bitcoin");
if (hwndPrev)
{
if (IsIconic(hwndPrev))
ShowWindow(hwndPrev, SW_RESTORE);
SetForegroundWindow(hwndPrev);
return false;
}

if (GetTime() > nStart + 60)
return false;

// Resume this instance if the other exits
delete psingleinstancechecker;
Sleep(1000);
            psingleinstancechecker = new wxSingleInstanceChecker(strMutexName);
if (!psingleinstancechecker->IsAnotherRunning())
break;
}
}

//
// Parameters
//
    wxImage::AddHandler(new wxPNGHandler);
    map<string, string> mapArgs = ParseParameters(argc, argv);

if (mapArgs.count("/datadir"))
        strSetDataDir = mapArgs["/datadir"];

if (mapArgs.count("/proxy"))
        addrProxy = CAddress(mapArgs["/proxy"].c_str());

if (mapArgs.count("/debug"))
        fDebug = true;

if (mapArgs.count("/dropmessages"))
{
        nDropMessagesTest = atoi(mapArgs["/dropmessages"]);
if (nDropMessagesTest == 0)
            nDropMessagesTest = 20;
}

if (mapArgs.count("/loadblockindextest"))
{
        CTxDB txdb("r");
        txdb.LoadBlockIndex();
PrintBlockTree();
ExitProcess(0);
}

//
// Load data files
//
    string strErrors;
    int64 nStart, nEnd;

printf("Loading addresses...\n");
QueryPerformanceCounter((LARGE_INTEGER*)&nStart);
if (!LoadAddresses())
        strErrors += "Error loading addr.dat      \n";
QueryPerformanceCounter((LARGE_INTEGER*)&nEnd);
printf(" addresses   %20I64d\n", nEnd - nStart);

printf("Loading block index...\n");
QueryPerformanceCounter((LARGE_INTEGER*)&nStart);
if (!LoadBlockIndex())
        strErrors += "Error loading blkindex.dat      \n";
QueryPerformanceCounter((LARGE_INTEGER*)&nEnd);
printf(" block index %20I64d\n", nEnd - nStart);

printf("Loading wallet...\n");
QueryPerformanceCounter((LARGE_INTEGER*)&nStart);
if (!LoadWallet())
        strErrors += "Error loading wallet.dat      \n";
QueryPerformanceCounter((LARGE_INTEGER*)&nEnd);
printf(" wallet      %20I64d\n", nEnd - nStart);

printf("Done loading\n");

//// debug print
printf("mapBlockIndex.size() = %d\n",   mapBlockIndex.size());
printf("nBestHeight = %d\n",            nBestHeight);
printf("mapKeys.size() = %d\n",         mapKeys.size());
printf("mapPubKeys.size() = %d\n",      mapPubKeys.size());
printf("mapWallet.size() = %d\n",       mapWallet.size());
printf("mapAddressBook.size() = %d\n",  mapAddressBook.size());

if (!strErrors.empty())
```

```
{
wxMessageBox(strErrors);
OnExit();
return false;
}

```

`ReacceptWalletTransactions()` — after loading the wallet, re-submit txs that are not yet in a block into the local mempool. Without this, a restart would drop unconfirmed sends from the node's view.

```
// Add wallet transactions that aren't already in a block to mapTransactions
ReacceptWalletTransactions();

//
// Parameters
//
if (mapArgs.count("/printblockindex") || mapArgs.count("/printblocktree"))
{
PrintBlockTree();
OnExit();
return false;
}

if (mapArgs.count("/gen"))
{
if (mapArgs["/gen"].empty())
            fGenerateBitcoins = true;
else
            fGenerateBitcoins = atoi(mapArgs["/gen"].c_str());
}

//
// Create the main frame window
//
{
        pframeMain = new CMainFrame(NULL);
        pframeMain->Show();

```

`StartNode()` — bring up sockets, IRC seed, and message threads. If this fails, the UI shows the error string and keeps running; without peers or listen capability the node is useless.

```
if (!StartNode(strErrors))
wxMessageBox(strErrors);

if (fGenerateBitcoins)
if (_beginthread(ThreadBitcoinMiner, 0, NULL) == -1)
printf("Error: _beginthread(ThreadBitcoinMiner) failed\n");

//
// Tests
//
if (argc >= 2 && stricmp(argv[1], "/send") == 0)
{
            int64 nValue = 1;
if (argc >= 3)
ParseMoney(argv[2], nValue);

            string strAddress;
if (argc >= 4)
                strAddress = argv[3];
            CAddress addr(strAddress.c_str());

            CWalletTx wtx;
            wtx.mapValue["to"] = strAddress;
            wtx.mapValue["from"] = addrLocalHost.ToString();
            wtx.mapValue["message"] = "command line send";

// Send to IP address
            CSendingDialog* pdialog = new CSendingDialog(pframeMain, addr, nValue, wtx);
if (!pdialog->ShowModal())
return false;
}

if (mapArgs.count("/randsendtest"))
{
if (!mapArgs["/randsendtest"].empty())
_beginthread(ThreadRandSendTest, 0, new string(mapArgs["/randsendtest"]));
else
```

```
                fRandSendTest = true;
            fDebug = true;
}
}

return true;
}

int CMyApp::OnExit()
{
Shutdown(NULL);
return wxApp::OnExit();
}

bool CMyApp::OnExceptionInMainLoop()
{
try
{
throw;
}
catch (std::exception& e)
{
PrintException(&e, "CMyApp::OnExceptionInMainLoop()");
wxLogWarning(_T("Exception %s %s"), typeid(e).name(), e.what());
Sleep(1000);
throw;
}
catch (...)
{
PrintException(NULL, "CMyApp::OnExceptionInMainLoop()");
wxLogWarning(_T("Unknown exception"));
Sleep(1000);
throw;
}

return true;
}

void CMyApp::OnUnhandledException()
{
// this shows how we may let some exception propagate uncaught
try
{
throw;
}
catch (std::exception& e)
{
PrintException(&e, "CMyApp::OnUnhandledException()");
wxLogWarning(_T("Exception %s %s"), typeid(e).name(), e.what());
Sleep(1000);
throw;
}
catch (...)
{
PrintException(NULL, "CMyApp::OnUnhandledException()");
wxLogWarning(_T("Unknown exception"));
Sleep(1000);
throw;
}
}

void CMyApp::OnFatalException()
{
wxMessageBox("Program has crashed and will terminate. ", "", wxOK | wxICON_ERROR);
}



```

CALLED FROM AddToWallet main.cpp p. 203 · CBlock::AddToBlockIndex main.cpp p. 221 · CommitTransactionSpent main.cpp p. 248 · SendMoney main.cpp p. 249 · ThreadSocketHandler2 net.cpp p. 333 · CSendingDialog::OnReply2 p. 501

```
void MainFrameRepaint()
{
if (pframeMain)
{
printf("MainFrameRepaint()\n");
        wxPaintEvent event;
        pframeMain->Refresh();
        pframeMain->AddPendingEvent(event);
}
}



```

CALLED FROM CMyApp::OnInit2 p. 517

```
3159 void ThreadRandSendTest(void* parg)
3160 {
3161     string strAddress = *(string*)parg;
3162     uint160 hash160;
3163 if (!AddressToHash160(strAddress, hash160))
3164 {
3165 wxMessageBox(strprintf("ThreadRandSendTest: Bitcoin address '%s' not valid ", strAddress.c_str()));
3166 return;
3167 }
3168
3169     loop
3170 {
3171 Sleep(GetRand(30) * 1000 + 100);
3172
3173 // Message
3174         CWalletTx wtx;
3175         wtx.mapValue["to"] = strAddress;
3176         wtx.mapValue["from"] = addrLocalHost.ToString();
3177 static int nRep;
3178         wtx.mapValue["message"] = strprintf("randsendtest %d\n", ++nRep);
3179
3180 // Value
3181         int64 nValue = (GetRand(9) + 1) * 100 * CENT;
3182 if (GetBalance() < nValue)
3183 {
3184 wxMessageBox("Out of money ");
3185 return;
3186 }
3187         nValue += (nRep % 100) * CENT;
3188
3189 // Send to bitcoin address
3190         CScript scriptPubKey;
3191         scriptPubKey << OP_DUP << OP_HASH160 << hash160 << OP_EQUALVERIFY << OP_CHECKSIG;
3192
3193 if (!SendMoney(scriptPubKey, nValue, wtx))
3194 return;
3195 }
3196 }
3197
3198
3199 // randsendtest to any connected node
```

CALLED FROM CMainFrame::OnButtonSend p. 490 · CSendingDialog::OnPaint p. 498

```
3200 void RandSend()
3201 {
3202     CWalletTx wtx;
3203
3204 while (vNodes.empty())
3205 Sleep(1000);
3206     CAddress addr;
3207 CRITICAL_BLOCK(cs_vNodes)
3208         addr = vNodes[GetRand(vNodes.size())]->addr;
3209
3210 // Message
3211     wtx.mapValue["to"] = addr.ToString();
3212     wtx.mapValue["from"] = addrLocalHost.ToString();
3213 static int nRep;
3214     wtx.mapValue["message"] = strprintf("randsendtest %d\n", ++nRep);
3215
3216 // Value
3217     int64 nValue = (GetRand(999) + 1) * CENT;
3218 if (GetBalance() < nValue)
3219 {
3220 wxMessageBox("Out of money ");
3221 return;
3222 }
3223
3224 // Send to IP address
3225     CSendingDialog* pdialog = new CSendingDialog(pframeMain, addr, nValue, wtx);
3226 if (!pdialog->Show())
3227 wxMessageBox("ShowModal Failed ");
3228 }
3229
```

SUMMARY

The UI reveals what Bitcoin v0.1.0 was designed to be: not just a protocol, but a complete payment application. The address book, the From/Message fields, the IP-to-IP handshake — these were features for a world where Bitcoin would be used for everyday commerce between online merchants and their customers.

Much of this vision was shed as Bitcoin evolved. IP-to-IP payments lost their user interface when the wxWidgets GUI was replaced in v0.5.0 (2011), and the protocol messages behind them were removed by v0.8.0 (2013) (privacy and NAT problems proved insurmountable).[2] The marketplace features—products, orders, reviews—were never completed. What survived is the simple, elegant core: a transaction list, a balance, an address to receive, a button to send. That minimal interface—connecting a cryptographic engine to a human being—proved to be exactly the right abstraction for a global monetary network. Every Bitcoin wallet built since, from Electrum to hardware wallets, follows the template Satoshi established in this file.

KEY TERMS

CMyApp

The wxApp subclass that serves as the program's entry point; IMPLEMENT_APP generates the real main function, and OnInit2 loads the wallet, opens the window, and starts the node.

cross-thread call

Posting a wxCommandEvent to the main window's queue with AddPendingEvent, so that the UI thread rather than a worker thread performs the actual screen update.

setCallbackAvailable

The set of dialog pointers still open, checked before a network reply is posted, so a response that arrives after its dialog has closed is quietly dropped.

wxYield

A wxWidgets call that briefly processes pending events from inside a long loop, keeping the window responsive while ui.cpp reloads a large wallet's transactions.

idle-time refresh

The pattern in which RefreshListCtrl only sets a flag and wakes the idle loop, leaving OnIdle to rebuild the transaction list when the interface has nothing else to do.

pay-to-IP-address

v0.1.0's online payment mode, in which the sender connects to the recipient's node, obtains a public key with checkorder, and delivers the signed transaction with submitorder.

mapAddressBook

The map from Bitcoin address strings to contact names that labels rows in the transaction list; it is saved in wallet.dat as "name" records.

single-instance check

The startup guard built on wxSingleInstanceChecker that keeps a second copy of Bitcoin from running and brings the existing window forward instead, protecting the database files.

SOURCES

1. wxWidgets Project, *wxWidgets Licence* (2005). https://www.wxwidgets.org/about/licence/
2. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
3. Satoshi Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (October 2008). https://bitcoin.org/bitcoin.pdf

ui.h

418 lines CPP

WINDOWS TO THE BLOCKCHAIN

Bitcoin was not designed to be used from a terminal. From the very beginning, Satoshi built a graphical application -- a Windows desktop program with buttons, menus, and dialog boxes. This header declares every window class in that application: the main transaction list, the send-coins dialog, the address book, the options panel, and the entire abandoned marketplace UI. It is 417 lines of wxWidgets C++ that reveal what Satoshi thought Bitcoin should *look like*.

Satoshi chose **wxWidgets** as the GUI toolkit -- a cross-platform C++ library that generates native-looking interfaces on Windows, macOS, and Linux.[1] In practice, v0.1.0 only shipped for Windows,[2] but the choice of wxWidgets over the Windows-native MFC or Win32 API shows Satoshi was already thinking about portability. The base classes (`CMainFrameBase`, `CSendDialogBase`, etc.) are auto-generated by wxFormBuilder from the visual layout file `uiproject.fbp`, and the classes declared here add the actual behavior -- event handlers, state management, and integration with the Bitcoin backend.

The architecture reveals a pragmatic separation of concerns. The UI thread communicates with the network and mining threads through custom wxWidgets events (`wxEVT_CROSSTHREADCALL`, `wxEVT_REPLY1` through `wxEVT_REPLY3`). This event-driven threading model avoids locking the UI while waiting for network operations -- a common source of "frozen window" bugs in early desktop applications. Satoshi understood that if the UI felt unresponsive, users would assume Bitcoin was broken.

Perhaps most striking is how much of this header is dedicated to the marketplace feature that was never completed. `CProductsDialog`, `CEditProductDialog`, `CViewProductDialog`, `CEditReviewDialog` -- nearly half the dialog classes serve the abandoned e-commerce system. The UI was built before the backend was ready, suggesting Satoshi designed the user experience first and worked backward to the protocol.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.
```

DECLARE_EVENT_TYPE -- wxWidgets macro that registers custom event types for inter-thread communication. wxEVT_CROSSTHREADCALL is the general-purpose mechanism: background threads post this event with an ID and payload, and the UI thread's OnCrossThreadCall() dispatches it. wxEVT_REPLY1 through wxEVT_REPLY3 carry network replies back to waiting dialogs -- wxEVT_REPLY1 returns marketplace product details to the order form, while wxEVT_REPLY2 and wxEVT_REPLY3 handle the two stages of the IP-to-IP payment handshake (the recipient's key, then final confirmation). wxEVT_TABLEADDED, wxEVT_TABLEUPDATED, and wxEVT_TABLEDELETED are marketplace notifications -- never fully wired up.

```
DECLARE_EVENT_TYPE(wxEVT_CROSSTHREADCALL, -1)
DECLARE_EVENT_TYPE(wxEVT_REPLY1, -1)
DECLARE_EVENT_TYPE(wxEVT_REPLY2, -1)
DECLARE_EVENT_TYPE(wxEVT_REPLY3, -1)
DECLARE_EVENT_TYPE(wxEVT_TABLEADDED, -1)
DECLARE_EVENT_TYPE(wxEVT_TABLEUPDATED, -1)
DECLARE_EVENT_TYPE(wxEVT_TABLEDELETED, -1)
```

UICALL_ADDORDER and UICALL_UPDATEORDER -- integer IDs for cross-thread calls related to marketplace orders. The CrossThreadCall() function (declared on line 27) posts these IDs to the UI event queue, where the main thread handles them safely. This enumeration would have grown as more features were added; only two values exist because the marketplace was abandoned.

```
enum
{
    UICALL_ADDORDER = 1,
    UICALL_UPDATEORDER,
};
```

Free-standing UI helper functions. HandleCtrlA() implements select-all (Ctrl+A) across all text fields -- a small usability detail that shows Satoshi's attention to user experience. DateTimeStr() formats Unix timestamps for display. FormatTxStatus() converts a wallet transaction's confirmation state into human-readable text like '3/unconfirmed' or '42 blocks'. MainFrameRepaint() triggers a UI refresh when blockchain state changes.

```
extern void HandleCtrlA(wxKeyEvent& event);
extern string DateTimeStr(int64 nTime);
extern string FormatTxStatus(const CWalletTx& wtx);
extern void CrossThreadCall(int nID, void* pdata);
extern void MainFrameRepaint();
```

`CMainFrame` -- the primary application window, inheriting from the auto-generated `CMainFrameBase` layout. This is what users see when they open Bitcoin: a transaction list, their address, a balance display, and send/address-book buttons. The class overrides 21 event handlers for everything from mouse clicks to window paint events. The destructor (`~CMainFrame()`) clears the global window pointer so no further UI events fire during shutdown.

```
class CMainFrame : public CMainFrameBase
{
protected:
// Event handlers
void OnClose(wxCloseEvent& event);
void OnMouseEvents(wxMouseEvent& event);
void OnKeyDown(wxKeyEvent& event) { HandleCtrlA(event); }
void OnIdle(wxIdleEvent& event);
void OnPaint(wxPaintEvent& event);
void OnPaintListCtrl(wxPaintEvent& event);
void OnMenuFileExit(wxCommandEvent& event);
void OnMenuOptionsGenerate(wxCommandEvent& event);
void OnMenuOptionsOptions(wxCommandEvent& event);
void OnMenuHelpAbout(wxCommandEvent& event);
void OnButtonSend(wxCommandEvent& event);
void OnButtonAddressBook(wxCommandEvent& event);
void OnSetFocusAddress(wxFocusEvent& event);
void OnMouseEventsAddress(wxMouseEvent& event);
void OnButtonCopy(wxCommandEvent& event);
void OnButtonChange(wxCommandEvent& event);
void OnListColBeginDrag(wxListEvent& event);
void OnListItemActivatedAllTransactions(wxListEvent& event);
void OnListItemActivatedProductsSent(wxListEvent& event);
void OnListItemActivatedOrdersSent(wxListEvent& event);
void OnListItemActivatedOrdersReceived(wxListEvent& event);

public:
/** Constructor */
CMainFrame(wxWindow* parent);
~CMainFrame();

// Custom
```

Main window state variables. `fRefreshListCtrl` and `fRefreshListCtrlRunning` form a simple debouncing mechanism: when the blockchain changes, the refresh flag is set, and the idle handler repaints the transaction list when the UI is not busy. `pindexBestLast` tracks the last-displayed chain tip to detect when new blocks arrive. `setUnmaturedDisplayed` was evidently meant to track coinbase transactions still waiting to mature, but nothing in v0.1.0 ever uses it; RefreshStatus() simply redraws every visible coinbase row instead.

```
bool fRefreshListCtrl;
bool fRefreshListCtrlRunning;
bool fOnSetFocusAddress;
    CBlockIndex* pindexBestLast;
    set<uint256> setUnmaturedDisplayed;

void OnCrossThreadCall(wxCommandEvent& event);
```

`InsertLine()` and `InsertTransaction()` -- the transaction list rendering pipeline. `InsertTransaction()` converts a `CWalletTx` into five display columns (status, date, description, debit, credit) and calls `InsertLine()` to add or update the row. The `strSort` parameter controls ordering -- transactions are sorted by block height first (unconfirmed ones on top), then by time received, with the most recent at the top.

```
void InsertLine(bool fNew, int nIndex, uint256 hashKey, string strSort, const wxString& str1, const wxString& str2, const wxString& str3, const wxString& str4, const wxString& str5);
void InsertTransaction(const CWalletTx& wtx, bool fNew, int nIndex=-1);
void RefreshListCtrl();
void RefreshStatus();
};

```

```
80
81
82
```

`CTxDetailsDialog` -- displays the full details of a single transaction. Double-clicking any transaction in the main list opens this dialog. It shows status, date, addresses, amounts, and fee paid, plus the raw transaction data when debug mode is on. The `CWalletTx` is stored by value (line 94), not by reference -- a defensive copy, so the dialog can never point at a wallet entry that has been changed or moved. The tradeoff is that it shows a snapshot: if the transaction confirms while the dialog is open, the dialog won't update.

```
83 class CTxDetailsDialog : public CTxDetailsDialogBase
84 {
85 protected:
86 // Event handlers
87 void OnButtonOK(wxCommandEvent& event);
88
89 public:
90 /** Constructor */
91 CTxDetailsDialog(wxWindow* parent, CWalletTx wtx);
92
93 // State
94     CWalletTx wtx;
95 };
96
97
98
```

`COptionsDialog` -- the settings panel. In v0.1.0, the only configurable option was the transaction fee (`OnKillFocusTransactionFee` validates the input when the user tabs away). The mining toggle was in the menu bar, not in options. The simplicity reflects Satoshi's design philosophy: fewer settings means fewer things users can misconfigure.

```
99 class COptionsDialog : public COptionsDialogBase
100 {
101 protected:
102 // Event handlers
103 void OnKillFocusTransactionFee(wxFocusEvent& event);
104 void OnButtonOK(wxCommandEvent& event);
105 void OnButtonCancel(wxCommandEvent& event);
106
107 public:
108 /** Constructor */
109 COptionsDialog(wxWindow* parent);
110 };
111
112
113
114 class CAboutDialog : public CAboutDialogBase
115 {
116 protected:
117 // Event handlers
118 void OnButtonOK(wxCommandEvent& event);
119
120 public:
121 /** Constructor */
122 CAboutDialog(wxWindow* parent);
123 };
124
125
126
```

`CSendDialog` -- the 'Send Coins' dialog, arguably the most important UI component. Takes a destination address (optionally pre-filled), validates it, accepts an amount, and initiates the transaction. `OnTextAddress()` provides real-time validation as the user types -- the address field updates with visual feedback before the user clicks Send.

```
127 class CSendDialog : public CSendDialogBase
128 {
```

```
129 protected:
130 // Event handlers
131 void OnKeyDown(wxKeyEvent& event) { HandleCtrlA(event); }
132 void OnTextAddress(wxCommandEvent& event);
133 void OnKillFocusAmount(wxFocusEvent& event);
134 void OnButtonAddressBook(wxCommandEvent& event);
135 void OnButtonPaste(wxCommandEvent& event);
136 void OnButtonSend(wxCommandEvent& event);
137 void OnButtonCancel(wxCommandEvent& event);
138
139 public:
140 /** Constructor */
141 CSendDialog(wxWindow* parent, const wxString& strAddress="");
142 };
143
144
145
```

`CSendingDialog` -- the real-time transaction progress dialog for IP-to-IP payments. This was Satoshi's original payment model: connect directly to the recipient's IP address, negotiate the transaction, and confirm delivery. The state machine has five boolean flags (`fCanCancel`, `fAbort`, `fSuccess`, `fUIDone`, `fWorkDone`) tracking the concurrent UI and network threads. `OnReply2()` and `OnReply3()` handle the server's responses. Both payment modes ended up as ordinary transactions in the block chain; the IP mode only added a live handshake for fetching the recipient's key and delivering a message. This entire dialog became obsolete when sending to Bitcoin addresses replaced IP-to-IP transactions.

```
146 class CSendingDialog : public CSendingDialogBase
147 {
148 public:
149 // Event handlers
150 void OnClose(wxCloseEvent& event);
151 void OnButtonOK(wxCommandEvent& event);
152 void OnButtonCancel(wxCommandEvent& event);
153 void OnPaint(wxPaintEvent& event);
154
155 public:
156 /** Constructor */
157 CSendingDialog(wxWindow* parent, const CAddress& addrIn, int64 nPriceIn, const CWalletTx& wtxIn);
158 ~CSendingDialog();
159
160 // State
161     CAddress addr;
162     int64 nPrice;
163     CWalletTx wtx;
164     wxDateTime start;
165     string strStatus;
166 bool fCanCancel;
167 bool fAbort;
168 bool fSuccess;
169 bool fUIDone;
170 bool fWorkDone;
171
172 void Close();
173 void Repaint();
174 bool Status();
175 bool Status(const string& str);
176 bool Error(const string& str);
177 void StartTransfer();
178 void OnReply2(CDataStream& vRecv);
179 void OnReply3(CDataStream& vRecv);
180 };
181
182 void SendingDialogStartTransfer(void* parg);
183 void SendingDialogOnReply2(void* parg, CDataStream& vRecv);
184 void SendingDialogOnReply3(void* parg, CDataStream& vRecv);
185
186
187
```

`CYourAddressDialog` -- manages your own Bitcoin addresses. Allows renaming (labeling) addresses, generating new ones, and copying them to the clipboard. The two constructors (lines 204-205) handle opening the dialog fresh versus opening it with a pre-selected address. This is the ancestor of the 'Receive' tab in modern Bitcoin wallets.

```
188 class CYourAddressDialog : public CYourAddressDialogBase
189 {
190 protected:
191 // Event handlers
192 void OnListEndLabelEdit(wxListEvent& event);
193 void OnListItemSelected(wxListEvent& event);
194 void OnListItemActivated(wxListEvent& event);
195 void OnButtonRename(wxCommandEvent& event);
196 void OnButtonNew(wxCommandEvent& event);
197 void OnButtonCopy(wxCommandEvent& event);
198 void OnButtonOK(wxCommandEvent& event);
199 void OnButtonCancel(wxCommandEvent& event);
200 void OnClose(wxCloseEvent& event);
201
202 public:
203 /** Constructor */
204 CYourAddressDialog(wxWindow* parent);
205 CYourAddressDialog(wxWindow* parent, const string& strInitSelected);
206
207 // Custom
208     wxString GetAddress();
209 };
210
211
212
```

`CAddressBookDialog` -- the contact list for other people's addresses. Supports adding, editing, deleting, and copying addresses. The `fSending` flag (line 233) determines the dialog's mode: when opened from the Send dialog, it returns the selected address; when opened standalone, it just manages the address book. This dual-mode pattern avoids duplicating the address list UI.

```
213 class CAddressBookDialog : public CAddressBookDialogBase
214 {
215 protected:
216 // Event handlers
217 void OnListEndLabelEdit(wxListEvent& event);
218 void OnListItemSelected(wxListEvent& event);
219 void OnListItemActivated(wxListEvent& event);
220 void OnButtonEdit(wxCommandEvent& event);
221 void OnButtonDelete(wxCommandEvent& event);
222 void OnButtonNew(wxCommandEvent& event);
223 void OnButtonCopy(wxCommandEvent& event);
224 void OnButtonOK(wxCommandEvent& event);
225 void OnButtonCancel(wxCommandEvent& event);
226 void OnClose(wxCloseEvent& event);
227
228 public:
229 /** Constructor */
230 CAddressBookDialog(wxWindow* parent, const wxString& strInitSelected, bool fSendingIn);
231
232 // Custom
233 bool fSending;
234     wxString GetAddress();
235 };
236
237
238
```

`CProductsDialog` -- the marketplace product browser. Would have displayed a searchable, filterable list of products advertised on the network. `m_vProduct` (line 253) holds the current search results. The search and combobox handlers suggest Satoshi planned category-based browsing. This dialog was never even wired into the UI menus -- it had no functioning backend.

```
239 class CProductsDialog : public CProductsDialogBase
```

```
{
protected:
// Event handlers
void OnKeyDown(wxKeyEvent& event) { HandleCtrlA(event); }
void OnCombobox(wxCommandEvent& event);
void OnButtonSearch(wxCommandEvent& event);
void OnListItemActivated(wxListEvent& event);

public:
/** Constructor */
CProductsDialog(wxWindow* parent);

// Custom
    vector<CProduct> m_vProduct;
};
```

`CEditProductDialog` -- product listing editor with up to 20 custom fields (`FIELDS_MAX = 20`). The 20 individual `OnButtonDel` handlers (lines 263-282) are a notable example of pre-C++11 limitations: without lambdas or std::bind, Satoshi had to write a separate handler for each delete button. Each one delegates to `OnButtonDel(event, n)` with the field index. Modern C++ would use a single lambda with a captured index.

```
class CEditProductDialog : public CEditProductDialogBase
{
protected:
// Event handlers
void OnKeyDown(wxKeyEvent& event) { HandleCtrlA(event); }
void OnButtonDel0(wxCommandEvent& event);
void OnButtonDel1(wxCommandEvent& event);
void OnButtonDel2(wxCommandEvent& event);
void OnButtonDel3(wxCommandEvent& event);
void OnButtonDel4(wxCommandEvent& event);
void OnButtonDel5(wxCommandEvent& event);
void OnButtonDel6(wxCommandEvent& event);
void OnButtonDel7(wxCommandEvent& event);
void OnButtonDel8(wxCommandEvent& event);
void OnButtonDel9(wxCommandEvent& event);
void OnButtonDel10(wxCommandEvent& event);
void OnButtonDel11(wxCommandEvent& event);
void OnButtonDel12(wxCommandEvent& event);
void OnButtonDel13(wxCommandEvent& event);
void OnButtonDel14(wxCommandEvent& event);
void OnButtonDel15(wxCommandEvent& event);
void OnButtonDel16(wxCommandEvent& event);
void OnButtonDel17(wxCommandEvent& event);
void OnButtonDel18(wxCommandEvent& event);
void OnButtonDel19(wxCommandEvent& event);
void OnButtonAddField(wxCommandEvent& event);
void OnButtonSend(wxCommandEvent& event);
void OnButtonPreview(wxCommandEvent& event);
void OnButtonCancel(wxCommandEvent& event);

public:
/** Constructor */
CEditProductDialog(wxWindow* parent);

// Custom
enum { FIELDS_MAX = 20 };
    wxTextCtrl* m_textCtrlLabel[FIELDS_MAX];
    wxTextCtrl* m_textCtrlField[FIELDS_MAX];
    wxButton*   m_buttonDel[FIELDS_MAX];

void LayoutAll();
void ShowLine(int i, bool fShow=true);
void OnButtonDel(wxCommandEvent& event, int n);
void SetProduct(const CProduct& productIn);
void GetProduct(CProduct& product);

};
```

`CViewProductDialog` -- the buyer's view of a product listing. Displays product details with up to 20 fields using a mix of static text labels, text inputs, and dropdown choices (`m_choiceField`). `GetOrder()` converts the filled-out form into a `CWalletTx` for payment. `OnReply1()` handles the seller's response to the order -- part of the IP-to-IP transaction protocol.

```
308 class CViewProductDialog : public CViewProductDialogBase
309 {
310 protected:
311 // Event handlers
312 void OnButtonSubmitForm(wxCommandEvent& event);
313 void OnButtonCancelForm(wxCommandEvent& event);
314 void OnButtonBack(wxCommandEvent& event);
315 void OnButtonNext(wxCommandEvent& event);
316 void OnButtonCancel(wxCommandEvent& event);
317
318 public:
319 /** Constructor */
320 CViewProductDialog(wxWindow* parent, const CProduct& productIn);
321 ~CViewProductDialog();
322
323 // Custom
324     CProduct product;
325 enum { FIELDS_MAX = 20 };
326     wxStaticText* m_staticTextLabel[FIELDS_MAX];
327     wxTextCtrl*   m_textCtrlField[FIELDS_MAX];
328     wxChoice*     m_choiceField[FIELDS_MAX];
329
330 void GetOrder(CWalletTx& order);
331 void UpdateProductDisplay(bool fDetails);
332 void OnReply1(wxCommandEvent& event);
333 };
334
335
336
337 class CViewOrderDialog : public CViewOrderDialogBase
338 {
339 protected:
340 // Event handlers
341 void OnButtonOK(wxCommandEvent& event);
342
343 public:
344 /** Constructor */
345 CViewOrderDialog(wxWindow* parent, CWalletTx order, bool fReceived);
346
347 // Custom
348 bool fReceived;
349 };
350
351
352
353 class CEditReviewDialog : public CEditReviewDialogBase
354 {
355 protected:
356 // Event handlers
357 void OnKeyDown(wxKeyEvent& event) { HandleCtrlA(event); }
358 void OnButtonSubmit(wxCommandEvent& event);
359 void OnButtonCancel(wxCommandEvent& event);
360
361 public:
362 /** Constructor */
363 CEditReviewDialog(wxWindow* parent);
364
365 // Custom
366 void GetReview(CReview& review);
367 };
368
369
370
```

`CGetTextFromUserDialog` -- a generic text-input modal dialog, reused throughout the application whenever a simple string is needed (entering a label for an address, for example). The constructor accepts up to two message/value pairs, making it a flexible one-or-two-field input box. The Enter key handling on line 381 (`GetKeyCode() == '\r'`) ensures both Return and NumPad Enter close the dialog -- another small usability detail.

```
371 class CGetTextFromUserDialog : public CGetTextFromUserDialogBase
372 {
373 protected:
374 // Event handlers
375 void OnButtonOK(wxCommandEvent& event) { EndModal(true); }
376 void OnButtonCancel(wxCommandEvent& event) { EndModal(false); }
377 void OnClose(wxCloseEvent& event) { EndModal(false); }
378
379 void OnKeyDown(wxKeyEvent& event)
380 {
381 if (event.GetKeyCode() == '\r' || event.GetKeyCode() == WXK_NUMPAD_ENTER)
382 EndModal(true);
383 else
384 HandleCtrlA(event);
385 }
386
387 public:
388 /** Constructor */
389 CGetTextFromUserDialog(wxWindow* parent,
390 const string& strCaption,
391 const string& strMessage1,
392 const string& strValue1="",
393 const string& strMessage2="",
394 const string& strValue2="") : CGetTextFromUserDialogBase(parent, wxID_ANY, strCaption)
395 {
396         m_staticTextMessage1->SetLabel(strMessage1);
397         m_textCtrl1->SetValue(strValue1);
398 if (!strMessage2.empty())
399 {
400             m_staticTextMessage2->Show(true);
401             m_staticTextMessage2->SetLabel(strMessage2);
402             m_textCtrl2->Show(true);
403             m_textCtrl2->SetValue(strValue2);
404 SetSize(wxDefaultCoord, 180);
405 }
406 }
407
408 // Custom
409     string GetValue() { return (string)m_textCtrl1->GetValue(); }
410     string GetValue1() { return (string)m_textCtrl1->GetValue(); }
411     string GetValue2() { return (string)m_textCtrl2->GetValue(); }
412 };
413
414
415
416
417
418
```

SUMMARY

The UI header reveals that Satoshi designed Bitcoin as a consumer application, not a developer tool. Every interaction a user might need -- sending coins, managing addresses, viewing transaction details, adjusting settings -- has a dedicated dialog class with carefully considered event handling. The threading model ensures the interface stays responsive even during slow network operations.

The marketplace dialogs are particularly revealing. Nearly half the UI classes serve a feature that never shipped -- product browsing, listing creation, order forms, reviews. Satoshi built the storefront before finishing the plumbing. This suggests he was designing from the user's perspective inward, imagining what the experience should feel like before worrying about protocol details.

Modern Bitcoin Core has entirely replaced this wxWidgets interface with a Qt-based GUI (Bitcoin-Qt, introduced in 2011).[34] The marketplace dialogs were removed. The IP-to-IP payment protocol and its `CSendingDialog` were deprecated. But the fundamental interaction model -- a transaction list, a send button, an address book -- survives unchanged in every Bitcoin wallet built since. Satoshi's UI design set the template that the entire ecosystem followed.

KEY TERMS

wxWidgets

The free, cross-platform C++ GUI toolkit Bitcoin v0.1.0 is built on, which wraps each operating system's native controls behind a single programming interface.

derived window class

A hand-written class such as CSendDialog that inherits a generated base class from uibase.h and overrides its event handlers to add Bitcoin's behavior.

CMainFrame

The main application window, derived from CMainFrameBase, which shows the balance, receiving address, and transaction list and overrides 21 of the generated event handlers.

custom event type

An application-defined wxWidgets event declared with DECLARE_EVENT_TYPE, such as wxEVT_CROSSTHREADCALL or wxEVT_REPLY1, used to carry messages from worker threads to the UI thread.

CSendDialog

The Send Coins window, which accepts a Bitcoin address or an IP address plus an amount, checks the address as it is typed, and starts the payment.

CSendingDialog

The progress window for a payment to an IP address, tracking the live exchange with the recipient's node through flags such as fCanCancel and fWorkDone.

CAddressBookDialog

The window listing other people's addresses with names; opened standalone it manages contacts, and opened from Send Coins it returns the chosen address.

CYourAddressDialog

The window listing the user's own receiving addresses, where each can be labeled, copied to the clipboard, or joined by a freshly generated one.

SOURCES

1. wxWidgets Project, *wxWidgets Licence* (2005). https://www.wxwidgets.org/about/licence/
2. Satoshi Nakamoto, *Bitcoin v0.1 released (Cryptography Mailing List / BitcoinTalk)* (January 9, 2009). Announced Windows-only for the first release. https://satoshi.nakamotoinstitute.org/emails/cryptography/15/
3. Qt Company / Nokia, *Qt licensed under LGPL* (January 2009). Enabled Bitcoin Core's later migration from wxWidgets to Qt. https://blog.qt.io/blog/2009/01/14/nokia-qt-lgpl/
4. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/

ui.rc

15 lines TEXT

THE BITCOIN ICON

Fourteen lines that gave Bitcoin its first visual identity. This Windows resource script (`.rc` file) tells the Microsoft resource compiler which icons and bitmaps to embed into the `bitcoin.exe` executable. When a user double-clicked the Bitcoin application on their Windows desktop in January 2009,[1] these were the graphics they saw: the application icon in the taskbar, the toolbar buttons for sending coins and managing the address book, and the small checkmark that appeared beside a valid recipient address in the send dialog.

The resource file format dates back to Windows 3.0 (1990) and remains essentially unchanged. Each line declares a symbolic name, a resource type (`ICON` or `BITMAP`), and a path to the image file in the `rc/` subdirectory. The resource compiler (`rc.exe` or `windres` on MinGW) converts these into a binary blob linked into the final executable. This is why the original Bitcoin client had no loose image files alongside the `.exe` -- everything was embedded inside it.

The original Bitcoin icon referenced here was a gold coin with the letters "BC" on it, designed by Satoshi himself. Satoshi refreshed it in February 2010, replacing the "BC" initials with a stroked "B" on the same gold coin. That November, the community adopted the now-iconic design: a white, slanted "B" with two vertical strokes on a flat orange circle, created by a user known as "bitboy" on the BitcoinTalk forums.[2] But for the first year of Bitcoin's existence, the icon compiled into this resource script was the only symbol the cryptocurrency had.

The inclusion of `wx/msw/wx.rc` (line 3) pulls in standard wxWidgets resources -- cursors, default dialog icons, and other Windows UI elements that the framework expects. Without this include, wxWidgets dialog boxes would render with missing icons and broken visual elements. This single line ties Bitcoin's custom resources into the broader wxWidgets resource chain.

`bitcoin ICON "rc/bitcoin.ico"` -- the application icon. This is the image Windows displays in the taskbar, the Alt-Tab switcher, the desktop shortcut, and the title bar. The symbolic name `bitcoin` is referenced in the wxWidgets application initialization code to set the window icon. The `.ico` format supports multiple resolutions (16×16, 32×32, 48×48) in a single file, so Windows can select the appropriate size for each context. Satoshi's original icon was a gold coin with 'BC' embossed on it.

```
bitcoin              ICON    "rc/bitcoin.ico"

```

`#include "wx/msw/wx.rc"` -- pulls in the standard wxWidgets Windows resource definitions. This includes default cursors, system dialog icons, and other visual elements the framework needs. The `msw` path component stands for 'Microsoft Windows' -- wxWidgets uses platform-specific resource files since each OS has different resource formats. On Linux and macOS, resources are handled differently (XPM images, application bundles), so this include is Windows-only.

```
#include "wx/msw/wx.rc"

```

`check ICON "rc/check.ico"` -- the green validation checkmark. It appears next to the 'Pay To' field in the Send Coins dialog the moment the entered text is a valid Bitcoin address -- live feedback, with no submit button required. The visual feedback was important psychologically: users needed to see that the address they were about to pay was well-formed. This tiny icon provided that reassurance.

```
check                ICON    "rc/check.ico"
```

`send16` and `send16mask` -- the 16×16 pixel send icon and its transparency mask. The 16px icon served as the Send Coins dialog's window icon rather than on the toolbar — paired, as it happens, with the no-shadow mask below, leaving plain `send16mask` unused in v0.1.0. In early Windows bitmap rendering, transparency was achieved with a separate one-bit mask bitmap. In these files, which wxWidgets loads as a `wxMask`, the black pixels (the background around the icon) mark the transparent area and the white pixels the opaque shape. This two-bitmap transparency technique predates the PNG alpha channel and was standard practice for Windows toolbar icons in the late 1990s and early 2000s.

```
send16               BITMAP  "rc/send16.bmp"
send16mask           BITMAP  "rc/send16mask.bmp"
```

`send16masknoshadow` -- a variant of the send button mask without a drop shadow effect, and the one v0.1.0 actually uses: the Send Coins dialog builds its window icon from `send16` plus this mask (ui.cpp line 1275). A title-bar icon has no room for a shadow. Having multiple mask variants for a single 16-pixel icon reveals the level of attention paid to visual details even in this earliest release.

```
send16masknoshadow   BITMAP  "rc/send16masknoshadow.bmp"
```

`send20` and `send20mask` -- the 20×20 pixel version of the send button, the size actually used on the main window toolbar (the toolbar's bitmap size is set to 20×20). The 16px variant served elsewhere as a dialog icon; the resource compiler embeds both sizes into the executable.

```
send20               BITMAP  "rc/send20.bmp"
```

```
10 send20mask            BITMAP  "rc/send20mask.bmp"
```

`addressbook16` and `addressbook16mask` -- the smaller counterpart of the address book toolbar icon (the toolbar itself loads the 20px variant; the 16px pair becomes the Address Book dialog's window icon). The address book was a core feature of Bitcoin's UI -- Satoshi understood that users would need to manage contacts' addresses rather than typing 34-character Base58 strings by hand every time. The dedicated toolbar icon reflects this: address management was given equal visual prominence to sending coins.

```
11 addressbook16         BITMAP  "rc/addressbook16.bmp"
12 addressbook16mask     BITMAP  "rc/addressbook16mask.bmp"
```

`addressbook20` and `addressbook20mask` -- the larger address book icon pair. With the send and address-book icons at two sizes each, plus their masks, plus the application and check icons, this small resource file references 11 image files total. Every one is compiled into the executable, giving Bitcoin v0.1.0 a self-contained, professional-looking Windows application with no external dependencies besides the OpenSSL and MinGW runtime DLLs.

```
13 addressbook20         BITMAP  "rc/addressbook20.bmp"
14 addressbook20mask     BITMAP  "rc/addressbook20mask.bmp"
15
```

SUMMARY

A resource script is not code in any meaningful sense -- it cannot fail, it has no logic, it makes no decisions. But it is the first thing users see. When Satoshi chose to create a custom application icon, toolbar bitmaps with transparency masks, and a confirmation checkmark, he was making a statement: Bitcoin was not a command-line experiment for cryptographers. It was a product for ordinary people, and it needed to look like one.

The visual identity defined here lasted just over a year. The "BC" gold coin was replaced by the community-designed orange ₿ logo—created by forum user bitboy—in November 2010,[2] and the wxWidgets GUI was replaced by Qt in 2011.[3][4] But the design principle -- that Bitcoin needs a polished, approachable visual identity -- originated in these fourteen lines. Every wallet app, exchange interface, and cryptocurrency logo that followed is a descendant of Satoshi's decision to ship an icon.

KEY TERMS

`resource script`

A plain-text .rc file listing the icons, bitmaps, and other assets a Windows program needs, each under a symbolic name the code can ask for at run time.

`resource compiler`

The build tool that turns a resource script into binary data linked into the executable: windres in the MinGW makefile, Microsoft's rc in makefile.vc.

`ICON and BITMAP`

The two resource types declared in ui.rc: ICON for Windows icons such as the application coin and the check mark, BITMAP for the toolbar pictures and their masks.

`symbolic resource name`

The identifier opening each line, such as send20 or check, which the code passes to wxBitmap with wxBITMAP_TYPE_RESOURCE, or to the wxICON macro, to fetch the embedded image.

`wx.rc`

The standard wxWidgets resource script, pulled in on line 3, that supplies the cursors and stock images the toolkit itself expects to find inside every Windows program.

SOURCES

1. Satoshi Nakamoto, *Bitcoin v0.1 released (Cryptography Mailing List / BitcoinTalk)* (January 9, 2009). Announced Windows-only for the first release. https://satoshi.nakamotoinstitute.org/emails/cryptography/15/
2. bitboy (BitcoinTalk user), *Orange ₿ logo first shared on BitcoinTalk* (November 1, 2010). Community design that became Bitcoin's standard mark; built on Satoshi's February 2010 gold-coin refresh. https://www.coindesk.com/markets/2019/05/18/about-that-orange-b-the-history-of-bitcoins-logos
3. Qt Company / Nokia, *Qt licensed under LGPL* (January 2009). Enabled Bitcoin Core's later migration from wxWidgets to Qt. https://blog.qt.io/blog/2009/01/14/nokia-qt-lgpl/
4. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/

uibase.cpp

1807 lines CPP

THE SKELETON OF THE INTERFACE

This file is the physical form of Bitcoin — the windows, buttons, and menus that Satoshi designed for ordinary people to send and receive digital cash. At over 1,600 lines, `uibase.cpp` is entirely auto-generated by **wxFormBuilder** (version April 2008), a visual GUI design tool that outputs C++ code for the **wxWidgets** cross-platform toolkit. The comment at the top — `PLEASE DO "NOT" EDIT THIS FILE!` — tells the whole story: this code was never meant to be read or modified by hand.

Satoshi's choice of wxWidgets was pragmatic. In 2008, the main cross-platform GUI options were Qt, GTK+, and wxWidgets. Qt required a commercial license for proprietary use at the time (Nokia would not release it under LGPL until January 2009),[1] and GTK+ had poor Windows support. wxWidgets was free, open source, and used native controls on each platform — meaning the Bitcoin window on Windows looked like a proper Windows application, not a foreign transplant.[2] When Satoshi posted the first release on the cryptography mailing list on January 9, 2009, he noted: "Windows only for now. Open source C++ code is included."[3] The wxWidgets foundation made the later Linux and Mac ports possible — Linux builds arrived with v0.2.0 (December 2009) and Mac with v0.3.0 (July 2010), both while Satoshi was still active.[4]

What makes this file historically significant is not the code itself — it is mechanical widget construction — but what the widgets *reveal* about Satoshi's vision. The main window has hidden tabs for escrows, orders sent, products sent, and orders received. The menu system includes a "Generate Coins" toggle. There is a `CProductsDialogBase` for a built-in marketplace, a `CEditProductDialogBase` for creating product listings, and even a `CPokerDialogBase` for an integrated poker game. Satoshi was not building a minimal payment system — he was building a platform.

The architecture follows the classic wxWidgets two-tier pattern: this file defines *base* classes with pure layout and event wiring, while the derived classes in `ui.cpp` override the virtual event handlers with actual logic. This separation meant Satoshi

could redesign the interface in wxFormBuilder's visual editor, regenerate this file, and preserve all business logic untouched in the derived classes. It is a textbook application of the Template Method design pattern.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

///////////////////////////////////////////////////////////////////////////
```

wxFormBuilder version April 16, 2008 — this is the build date of the wxFormBuilder tool Satoshi used, not the date he generated this file. It tells us only that the GUI was generated with a release from April 2008 or later. It is consistent with, but does not by itself prove, GUI work under way before the whitepaper appeared on October 31, 2008.[5][6]

```
// C++ code generated with wxFormBuilder (version Apr 16 2008)
// http://www.wxformbuilder.org/
//
// PLEASE DO "NOT" EDIT THIS FILE!
///////////////////////////////////////////////////////////////////////////

#include "uibase.h"

///////////////////////////////////////////////////////////////////////////

```

`CMainFrameBase` constructor — builds the entire main window in a single function call. The window title is simply `'Bitcoin'`, the default size is 705×484 pixels. Every widget — menu bar, toolbar, status bar, address display, transaction list — is constructed and laid out here. The `wxDEFAULT_FRAME_STYLE` gives standard minimize/maximize/close buttons.

```
CMainFrameBase::CMainFrameBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos, const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
{
this->SetSizeHints(wxDefaultSize, wxDefaultSize);
this->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_BTNFACE));

```

Menu bar construction. Three menus: **File** (just Exit), **Options** (Generate Coins and Options), and **Help** (About). The `wxID_OPTIONSGENERATEBITCOINS` menu item is a checkbox (`wxITEM_CHECK`) — toggling it starts or stops the CPU miner. In 2009, this single checkbox was the entire mining interface. Satoshi envisioned every user as a miner.

```
    m_menubar = new wxMenuBar(0);
    m_menubar->SetBackgroundColour(wxColour(240, 240, 240));

    m_menuFile = new wxMenu();
    wxMenuItem* m_menuFileExit;
    m_menuFileExit = new wxMenuItem(m_menuFile, wxID_ANY, wxString(wxT("E&xit")) , wxEmptyString, wxITEM_NORMAL);
    m_menuFile->Append(m_menuFileExit);

    m_menubar->Append(m_menuFile, wxT("&File"));

    m_menuOptions = new wxMenu();
    wxMenuItem* m_menuOptionsGenerateBitcoins;
```

`'&Generate Coins'` — the ampersand creates a keyboard accelerator (Alt+G). This is the mining toggle that Satoshi expected ordinary users to enable. On the cryptography mailing list he wrote: 'At first, most users would run network nodes, but as the network grows beyond a certain point, it would be left more and more to specialists with server farms of specialized hardware.'[7] The prediction proved right — GPU mining arrived in late 2010, and by 2011 CPU mining was obsolete.

```
    m_menuOptionsGenerateBitcoins = new wxMenuItem(m_menuOptions, wxID_OPTIONSGENERATEBITCOINS, wxString(wxT("&Generate Coins")) , wxEmptyString, wxITEM_CHECK);
    m_menuOptions->Append(m_menuOptionsGenerateBitcoins);

    wxMenuItem* m_menuOptionsOptions;
```

```
    m_menuOptionsOptions = new wxMenuItem(m_menuOptions, wxID_ANY, wxString(wxT("&Options...")) , wxEmptyString, wxITEM_NORMAL);
    m_menuOptions->Append(m_menuOptionsOptions);

    m_menubar->Append(m_menuOptions, wxT("&Options"));

    m_menuHelp = new wxMenu();
    wxMenuItem* m_menuHelpAbout;
    m_menuHelpAbout = new wxMenuItem(m_menuHelp, wxID_ANY, wxString(wxT("&About...")) , wxEmptyString, wxITEM_NORMAL);
    m_menuHelp->Append(m_menuHelpAbout);

    m_menubar->Append(m_menuHelp, wxT("&Help"));

this->SetMenuBar(m_menubar);

```

Toolbar with two buttons: `'&Send Coins'` and `'&Address Book'`. The bitmap resources (`send20`, `addressbook20`) are 20×20 pixel icons compiled into the Windows executable via `ui.rc`. These two actions — sending money and managing contacts — were the entire user workflow Satoshi designed for. The simplicity is striking compared to modern cryptocurrency wallets.

```
    m_toolBar = this->CreateToolBar(wxTB_FLAT|wxTB_HORZ_TEXT, wxID_ANY);
    m_toolBar->SetToolBitmapSize(wxSize(20,20));
    m_toolBar->SetToolSeparation(1);
    m_toolBar->SetFont(wxFont(wxNORMAL_FONT->GetPointSize(), 70, 90, 90, false, wxEmptyString));

    m_toolBar->AddTool(wxID_BUTTONSEND, wxT("&Send Coins"), wxBitmap(wxT("send20"), wxBITMAP_TYPE_RESOURCE), wxNullBitmap, wxITEM_NORMAL, wxEmptyString, wxEmptyString);
    m_toolBar->AddTool(wxID_BUTTONRECEIVE, wxT("&Address Book"), wxBitmap(wxT("addressbook20"), wxBITMAP_TYPE_RESOURCE), wxNullBitmap, wxITEM_NORMAL, wxEmptyString, wxEmptyString);
    m_toolBar->Realize();

    m_statusBar = this->CreateStatusBar(1, wxST_SIZEGRIP, wxID_ANY);
    m_statusBar->SetBackgroundColour(wxColour(240, 240, 240));

    wxBoxSizer* bSizer2;
    bSizer2 = new wxBoxSizer(wxVERTICAL);


    bSizer2->Add(0, 2, 0, wxEXPAND, 5);

    wxBoxSizer* bSizer85;
    bSizer85 = new wxBoxSizer(wxHORIZONTAL);

```

`'Your Bitcoin Address:'` — a read-only text field displaying the user's default receiving address, with a `'Copy to Clipboard'` button and a `'Change...'` button for generating new addresses. This is the first thing a new user would see. The `wxTE_READONLY` flag prevents editing; the background is set to the system menu color to visually indicate it is not editable.

```
    m_staticText32 = new wxStaticText(this, wxID_ANY, wxT("Your Bitcoin Address:"), wxDefaultPosition, wxDefaultSize, 0);
    m_staticText32->Wrap(-1);
    bSizer85->Add(m_staticText32, 0, wxALIGN_CENTER_VERTICAL|wxLEFT, 5);

    m_textCtrlAddress = new wxTextCtrl(this, wxID_TEXTCTRLADDRESS, wxEmptyString, wxDefaultPosition, wxSize(250,-1), wxTE_READONLY);
    m_textCtrlAddress->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_MENU));

    bSizer85->Add(m_textCtrlAddress, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_buttonCopy = new wxButton(this, wxID_BUTTONCOPY, wxT("&Copy to Clipboard"), wxDefaultPosition, wxDefaultSize, wxBU_EXACTFIT);
    bSizer85->Add(m_buttonCopy, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT, 5);

    m_button91 = new wxButton(this, wxID_BUTTONCHANGE, wxT("C&hange..."), wxDefaultPosition, wxDefaultSize, 0);
    bSizer85->Add(m_button91, 0, wxRIGHT, 5);


    bSizer85->Add(0, 0, 0, wxEXPAND, 5);

    bSizer2->Add(bSizer85, 0, wxEXPAND|wxRIGHT|wxLEFT, 5);

    wxBoxSizer* bSizer3;
    bSizer3 = new wxBoxSizer(wxHORIZONTAL);

    m_panel14 = new wxPanel(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTAB_TRAVERSAL);
    wxBoxSizer* bSizer66;
```

```
97      bSizer66 = new wxBoxSizer(wxHORIZONTAL);
98
```

`'Balance:'` — a right-aligned static text field (120 pixels wide, 8-point font) that displays the user's current balance. The `wxALIGN_RIGHT` and `wxST_NO_AUTORESIZE` flags keep the number right-justified and prevent the layout from jumping as the balance changes. This was the user's entire view of their wealth — a single number on a gray background.

```
99      m_staticText41 = new wxStaticText(m_panel14, wxID_ANY, wxT("Balance:"), wxDefaultPosition, wxSize(-1,15), 0);
100     m_staticText41->Wrap(-1);
101     bSizer66->Add(m_staticText41, 0, wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT, 5);
102
103     m_staticTextBalance = new wxStaticText(m_panel14, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(120,15),
    wxALIGN_RIGHT|wxST_NO_AUTORESIZE);
104     m_staticTextBalance->Wrap(-1);
105     m_staticTextBalance->SetFont(wxFont(8, 70, 90, 90, false, wxEmptyString));
106     m_staticTextBalance->SetBackgroundColour(wxColour(240, 240, 240));
107
108     bSizer66->Add(m_staticTextBalance, 0, wxALIGN_CENTER_VERTICAL|wxALL, 5);
109
110     m_panel14->SetSizer(bSizer66);
111     m_panel14->Layout();
112     bSizer66->Fit(m_panel14);
113     bSizer3->Add(m_panel14, 1, wxEXPAND|wxALIGN_BOTTOM|wxALL, 5);
114
115
116     bSizer3->Add(0, 0, 0, wxEXPAND, 5);
117
```

Transaction filter dropdown with four choices: `'All'`, `'Sent'`, `'Received'`, and `'In Progress'`. Note the call to `Hide()` on line 122 — this filter was created but hidden in v0.1.0, suggesting Satoshi built it for future use but did not consider it ready. The filter categories reveal the transaction states he was tracking internally.

```
118     wxString m_choiceFilterChoices[] = { wxT(" All"), wxT(" Sent"), wxT(" Received"), wxT(" In Progress") };
119 int m_choiceFilterNChoices = sizeof(m_choiceFilterChoices) / sizeof(wxString);
120     m_choiceFilter = new wxChoice(this, wxID_ANY, wxDefaultPosition, wxSize(110,-1), m_choiceFilterNChoices,
    m_choiceFilterChoices, 0);
121     m_choiceFilter->SetSelection(0);
122     m_choiceFilter->Hide();
123
124     bSizer3->Add(m_choiceFilter, 0, wxALIGN_BOTTOM|wxTOP|wxRIGHT|wxLEFT, 5);
125
126     bSizer2->Add(bSizer3, 0, wxEXPAND, 5);
127
128     m_notebook = new wxNotebook(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, 0);
129     m_panel7 = new wxPanel(m_notebook, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTAB_TRAVERSAL);
130     wxBoxSizer* bSizer157;
131     bSizer157 = new wxBoxSizer(wxVERTICAL);
132
```

`m_listCtrl` — the main transaction list, using `wxLC_REPORT` mode (rows and columns like a spreadsheet) with `wxLC_SORT_DESCENDING` to show newest transactions first. The `wxLC_NO_SORT_HEADER` flag disables user sorting — Satoshi wanted a fixed reverse-chronological order. This single list control was the user's complete transaction history.

```
133     m_listCtrl = new wxListCtrl(m_panel7, wxID_ANY, wxDefaultPosition, wxDefaultSize,
    wxLC_NO_SORT_HEADER|wxLC_REPORT|wxLC_SORT_DESCENDING|wxALWAYS_SHOW_SB);
134     bSizer157->Add(m_listCtrl, 1, wxEXPAND|wxALL, 5);
135
136     m_panel7->SetSizer(bSizer157);
137     m_panel7->Layout();
138     bSizer157->Fit(m_panel7);
139     m_notebook->AddPage(m_panel7, wxT("All Transactions"), false);
140
141     bSizer2->Add(m_notebook, 1, wxEXPAND, 5);
142
```

`bSizer_TabsForFutureUse` — the name says it all. Four hidden panels follow: `m_listCtrlEscrows`, `m_listCtrlOrdersSent`, `m_listCtrlProductsSent`, and `m_listCtrlOrdersReceived`. All are created but immediately hidden with `Hide()`. These are the skeleton of Satoshi's planned marketplace — a built-in system for buying and selling goods directly through the Bitcoin client. The marketplace was never fully activated in any public release.

```
143     wxBoxSizer* bSizer_TabsForFutureUse;
144     bSizer_TabsForFutureUse = new wxBoxSizer(wxVERTICAL);
145
146     m_panel9 = new wxPanel(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTAB_TRAVERSAL);
147     m_panel9->Hide();
148
149     wxBoxSizer* bSizer159;
150     bSizer159 = new wxBoxSizer(wxVERTICAL);
151
152     m_listCtrlEscrows = new wxListCtrl(m_panel9, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxLC_NO_SORT_HEADER|wxLC_REPORT);
153     bSizer159->Add(m_listCtrlEscrows, 1, wxALL|wxEXPAND, 5);
154
155     m_panel9->SetSizer(bSizer159);
156     m_panel9->Layout();
157     bSizer159->Fit(m_panel9);
158     bSizer_TabsForFutureUse->Add(m_panel9, 1, wxEXPAND | wxALL, 5);
159
160     m_panel8 = new wxPanel(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTAB_TRAVERSAL);
161     m_panel8->Hide();
162
163     wxBoxSizer* bSizer158;
164     bSizer158 = new wxBoxSizer(wxVERTICAL);
165
166     m_listCtrlOrdersSent = new wxListCtrl(m_panel8, wxID_ANY, wxDefaultPosition, wxDefaultSize,
    wxLC_NO_SORT_HEADER|wxLC_REPORT);
167     bSizer158->Add(m_listCtrlOrdersSent, 1, wxALL|wxEXPAND, 5);
168
169     m_panel8->SetSizer(bSizer158);
170     m_panel8->Layout();
171     bSizer158->Fit(m_panel8);
172     bSizer_TabsForFutureUse->Add(m_panel8, 1, wxEXPAND | wxALL, 5);
173
174     m_panel10 = new wxPanel(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTAB_TRAVERSAL);
175     m_panel10->Hide();
176
177     wxBoxSizer* bSizer160;
178     bSizer160 = new wxBoxSizer(wxVERTICAL);
179
180     m_listCtrlProductsSent = new wxListCtrl(m_panel10, wxID_ANY, wxDefaultPosition, wxDefaultSize,
    wxLC_NO_SORT_HEADER|wxLC_REPORT);
181     bSizer160->Add(m_listCtrlProductsSent, 1, wxALL|wxEXPAND, 5);
182
183     m_panel10->SetSizer(bSizer160);
184     m_panel10->Layout();
185     bSizer160->Fit(m_panel10);
186     bSizer_TabsForFutureUse->Add(m_panel10, 1, wxEXPAND | wxALL, 5);
187
188     m_panel11 = new wxPanel(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTAB_TRAVERSAL);
189     m_panel11->Hide();
190
191     wxBoxSizer* bSizer161;
192     bSizer161 = new wxBoxSizer(wxVERTICAL);
193
194     m_listCtrlOrdersReceived = new wxListCtrl(m_panel11, wxID_ANY, wxDefaultPosition, wxDefaultSize,
    wxLC_NO_SORT_HEADER|wxLC_REPORT);
195     bSizer161->Add(m_listCtrlOrdersReceived, 1, wxALL|wxEXPAND, 5);
196
197     m_panel11->SetSizer(bSizer161);
198     m_panel11->Layout();
199     bSizer161->Fit(m_panel11);
200     bSizer_TabsForFutureUse->Add(m_panel11, 1, wxEXPAND | wxALL, 5);
201
202     bSizer2->Add(bSizer_TabsForFutureUse, 1, wxEXPAND, 5);
203
204 this->SetSizer(bSizer2);
205 this->Layout();
206
```

Event wiring — the `Connect()` calls bind wxWidgets events to virtual handler methods. Every mouse event type (left/right/middle click, double-click, motion, enter, leave, mousewheel) on both the main frame and the address field is captured. The reason is not interactivity but randomness: the main frame's handler, `CMainFrame::OnMouseEvents()` in ui.cpp, feeds every mouse event's X and Y coordinates into OpenSSL's random number pool with `RAND_add()`. Simply moving the mouse over the Bitcoin window strengthened the entropy behind your keys. The address field's handlers do something humbler: select the whole address when you click into it, ready to copy.

```
// Connect Events
this->Connect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CMainFrameBase::OnClose));
this->Connect(wxEVT_IDLE, wxIdleEventHandler(CMainFrameBase::OnIdle));
this->Connect(wxEVT_LEFT_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_LEFT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_MIDDLE_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_MIDDLE_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_RIGHT_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_RIGHT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_MOTION, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_LEFT_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_MIDDLE_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_RIGHT_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_LEAVE_WINDOW, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_ENTER_WINDOW, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_MOUSEWHEEL, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Connect(wxEVT_PAINT, wxPaintEventHandler(CMainFrameBase::OnPaint));
this->Connect(m_menuFileExit->GetId(), wxEVT_COMMAND_MENU_SELECTED, wxCommandEventHandler(CMainFrameBase::OnMenuFileExit));
this->Connect(m_menuOptionsGenerateBitcoins->GetId(), wxEVT_COMMAND_MENU_SELECTED,
wxCommandEventHandler(CMainFrameBase::OnMenuOptionsGenerate));
this->Connect(m_menuOptionsOptions->GetId(), wxEVT_COMMAND_MENU_SELECTED,
wxCommandEventHandler(CMainFrameBase::OnMenuOptionsOptions));
this->Connect(m_menuHelpAbout->GetId(), wxEVT_COMMAND_MENU_SELECTED, wxCommandEventHandler(CMainFrameBase::OnMenuHelpAbout));
this->Connect(wxID_BUTTONSEND, wxEVT_COMMAND_TOOL_CLICKED, wxCommandEventHandler(CMainFrameBase::OnButtonSend));
this->Connect(wxID_BUTTONRECEIVE, wxEVT_COMMAND_TOOL_CLICKED, wxCommandEventHandler(CMainFrameBase::OnButtonAddressBook));
    m_textCtrlAddress->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CMainFrameBase::OnKeyDown), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_LEFT_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_LEFT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_MIDDLE_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_MIDDLE_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_RIGHT_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_RIGHT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_MOTION, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_LEFT_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_MIDDLE_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_RIGHT_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_LEAVE_WINDOW, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_ENTER_WINDOW, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_MOUSEWHEEL, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_SET_FOCUS, wxFocusEventHandler(CMainFrameBase::OnSetFocusAddress), NULL, this);
    m_buttonCopy->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CMainFrameBase::OnButtonCopy), NULL, this);
    m_button91->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CMainFrameBase::OnButtonChange), NULL, this);
    m_listCtrl->Connect(wxEVT_COMMAND_LIST_COL_BEGIN_DRAG, wxListEventHandler(CMainFrameBase::OnListColBeginDrag), NULL, this);
    m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CMainFrameBase::OnListItemActivatedAllTransactions), NULL, this);
    m_listCtrl->Connect(wxEVT_PAINT, wxPaintEventHandler(CMainFrameBase::OnPaintListCtrl), NULL, this);
    m_listCtrlOrdersSent->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CMainFrameBase::OnListItemActivatedOrdersSent), NULL, this);
    m_listCtrlProductsSent->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CMainFrameBase::OnListItemActivatedProductsSent), NULL, this);
    m_listCtrlOrdersReceived->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CMainFrameBase::OnListItemActivatedOrdersReceived), NULL, this);
}

CMainFrameBase::~CMainFrameBase()
{
// Disconnect Events
this->Disconnect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CMainFrameBase::OnClose));
this->Disconnect(wxEVT_IDLE, wxIdleEventHandler(CMainFrameBase::OnIdle));
this->Disconnect(wxEVT_LEFT_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Disconnect(wxEVT_LEFT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Disconnect(wxEVT_MIDDLE_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Disconnect(wxEVT_MIDDLE_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Disconnect(wxEVT_RIGHT_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Disconnect(wxEVT_RIGHT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Disconnect(wxEVT_MOTION, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Disconnect(wxEVT_LEFT_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Disconnect(wxEVT_MIDDLE_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Disconnect(wxEVT_RIGHT_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Disconnect(wxEVT_LEAVE_WINDOW, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Disconnect(wxEVT_ENTER_WINDOW, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
this->Disconnect(wxEVT_MOUSEWHEEL, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
```

```
this->Disconnect(wxEVT_PAINT, wxPaintEventHandler(CMainFrameBase::OnPaint));
this->Disconnect(wxID_ANY, wxEVT_COMMAND_MENU_SELECTED, wxCommandEventHandler(CMainFrameBase::OnMenuFileExit));
this->Disconnect(wxID_ANY, wxEVT_COMMAND_MENU_SELECTED, wxCommandEventHandler(CMainFrameBase::OnMenuOptionsGenerate));
this->Disconnect(wxID_ANY, wxEVT_COMMAND_MENU_SELECTED, wxCommandEventHandler(CMainFrameBase::OnMenuOptionsOptions));
this->Disconnect(wxID_ANY, wxEVT_COMMAND_MENU_SELECTED, wxCommandEventHandler(CMainFrameBase::OnMenuHelpAbout));
this->Disconnect(wxID_BUTTONSEND, wxEVT_COMMAND_TOOL_CLICKED, wxCommandEventHandler(CMainFrameBase::OnButtonSend));
this->Disconnect(wxID_BUTTONRECEIVE, wxEVT_COMMAND_TOOL_CLICKED, wxCommandEventHandler(CMainFrameBase::OnButtonAddressBook));
    m_textCtrlAddress->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CMainFrameBase::OnKeyDown), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_LEFT_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_LEFT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_MIDDLE_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_MIDDLE_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_RIGHT_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_RIGHT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_MOTION, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_LEFT_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_MIDDLE_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_RIGHT_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_LEAVE_WINDOW, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_ENTER_WINDOW, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_MOUSEWHEEL, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_SET_FOCUS, wxFocusEventHandler(CMainFrameBase::OnSetFocusAddress), NULL, this);
    m_buttonCopy->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CMainFrameBase::OnButtonCopy), NULL, this);
    m_button91->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CMainFrameBase::OnButtonChange), NULL, this);
    m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_COL_BEGIN_DRAG, wxListEventHandler(CMainFrameBase::OnListColBeginDrag), NULL,
this);
    m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CMainFrameBase::OnListItemActivatedAllTransactions), NULL, this);
    m_listCtrl->Disconnect(wxEVT_PAINT, wxPaintEventHandler(CMainFrameBase::OnPaintListCtrl), NULL, this);
    m_listCtrlOrdersSent->Disconnect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CMainFrameBase::OnListItemActivatedOrdersSent), NULL, this);
    m_listCtrlProductsSent->Disconnect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CMainFrameBase::OnListItemActivatedProductsSent), NULL, this);
    m_listCtrlOrdersReceived->Disconnect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CMainFrameBase::OnListItemActivatedOrdersReceived), NULL, this);
}

```

`CTxDetailsDialogBase` — a simple dialog with an `wxHtmlWindow` for displaying transaction details in formatted HTML. The 620×450 size is generous for what is essentially a receipt viewer. Using HTML rather than plain text allowed Satoshi to show structured information with bold labels, colored status indicators, and links — a surprisingly sophisticated approach for transaction display.

```
CTxDetailsDialogBase::CTxDetailsDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos, const
wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
{
this->SetSizeHints(wxDefaultSize, wxDefaultSize);

    wxBoxSizer* bSizer64;
    bSizer64 = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* bSizer66;
    bSizer66 = new wxBoxSizer(wxVERTICAL);

    m_htmlWin = new wxHtmlWindow(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxHW_SCROLLBAR_AUTO);
    bSizer66->Add(m_htmlWin, 1, wxALL|wxEXPAND, 5);

    bSizer64->Add(bSizer66, 1, wxEXPAND, 5);

    wxBoxSizer* bSizer65;
    bSizer65 = new wxBoxSizer(wxVERTICAL);

    m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxSize(85,25), 0);
    bSizer65->Add(m_buttonOK, 0, wxALL, 5);

    bSizer64->Add(bSizer65, 0, wxALIGN_RIGHT, 5);

this->SetSizer(bSizer64);
this->Layout();

// Connect Events
    m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CTxDetailsDialogBase::OnButtonOK), NULL, this);
}

CTxDetailsDialogBase::~CTxDetailsDialogBase()
{
// Disconnect Events
    m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CTxDetailsDialogBase::OnButtonOK), NULL, this);
}

```

`COptionsDialogBase` — the entire options dialog contains a single configurable setting: `'Optional transaction fee you give to the nodes that process your transactions.'` The text field `m_textCtrlTransactionFee` is just 70 pixels wide. In v0.1.0, this fee was purely voluntary. Satoshi kept the interface minimal — one setting, clearly explained in plain language.

```
341 COptionsDialogBase::COptionsDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos, const
    wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
342 {
343 this->SetSizeHints(wxDefaultSize, wxDefaultSize);
344
345     wxBoxSizer* bSizer55;
346     bSizer55 = new wxBoxSizer(wxVERTICAL);
347
348     wxBoxSizer* bSizer57;
349     bSizer57 = new wxBoxSizer(wxVERTICAL);
350
351
352     bSizer57->Add(0, 20, 0, wxEXPAND, 5);
353
354     m_staticText32 = new wxStaticText(this, wxID_ANY, wxT("Optional transaction fee you give to the nodes that process your
    transactions."), wxDefaultPosition, wxDefaultSize, 0);
355     m_staticText32->Wrap(-1);
356     bSizer57->Add(m_staticText32, 0, wxALIGN_CENTER_VERTICAL|wxALL, 5);
357
358     wxBoxSizer* bSizer56;
359     bSizer56 = new wxBoxSizer(wxHORIZONTAL);
360
361     m_staticText31 = new wxStaticText(this, wxID_ANY, wxT("Transaction fee:"), wxDefaultPosition, wxDefaultSize, 0);
362     m_staticText31->Wrap(-1);
363     bSizer56->Add(m_staticText31, 0, wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT, 5);
364
365     m_textCtrlTransactionFee = new wxTextCtrl(this, wxID_TRANSACTIONFEE, wxEmptyString, wxDefaultPosition, wxSize(70,-1), 0);
366     bSizer56->Add(m_textCtrlTransactionFee, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
367
368     bSizer57->Add(bSizer56, 0, wxEXPAND, 5);
369
370     bSizer55->Add(bSizer57, 1, wxEXPAND|wxLEFT, 5);
371
372     wxBoxSizer* bSizer58;
373     bSizer58 = new wxBoxSizer(wxHORIZONTAL);
374
375     m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxSize(85,25), 0);
376     bSizer58->Add(m_buttonOK, 0, wxALL, 5);
377
378     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxSize(-1,-1), 0);
379     m_buttonCancel->SetMinSize(wxSize(85,25));
380
381     bSizer58->Add(m_buttonCancel, 0, wxALL, 5);
382
383     bSizer55->Add(bSizer58, 0, wxALIGN_RIGHT, 5);
384
385 this->SetSizer(bSizer55);
386 this->Layout();
387
388 // Connect Events
389     m_textCtrlTransactionFee->Connect(wxEVT_KILL_FOCUS, wxFocusEventHandler(COptionsDialogBase::OnKillFocusTransactionFee),
    NULL, this);
390     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(COptionsDialogBase::OnButtonOK), NULL, this);
391     m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(COptionsDialogBase::OnButtonCancel), NULL,
    this);
392 }
393
394 COptionsDialogBase::~COptionsDialogBase()
395 {
396 // Disconnect Events
397     m_textCtrlTransactionFee->Disconnect(wxEVT_KILL_FOCUS, wxFocusEventHandler(COptionsDialogBase::OnKillFocusTransactionFee),
    NULL, this);
398     m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(COptionsDialogBase::OnButtonOK), NULL, this);
399     m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(COptionsDialogBase::OnButtonCancel), NULL,
    this);
400 }
401
```

`CAboutDialogBase` — Satoshi's self-attribution. The dialog displays `'Bitcoin'` in bold Tahoma font followed by a version string, then the full copyright and license text: `'Copyright © 2009 Satoshi Nakamoto. This is experimental software. Do not rely on it for actual financial transactions.'` The warning about experimental software shows remarkable honesty — Satoshi was launching a monetary system while explicitly telling users not to trust it with real money.

```
402 CAboutDialogBase::CAboutDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos, const wxSize&
    size, long style) : wxDialog(parent, id, title, pos, size, style)
403 {
404 this->SetSizeHints(wxDefaultSize, wxDefaultSize);
405
406     wxBoxSizer* bSizer60;
407     bSizer60 = new wxBoxSizer(wxVERTICAL);
408
409     wxBoxSizer* bSizer62;
410     bSizer62 = new wxBoxSizer(wxHORIZONTAL);
411
412
413     bSizer62->Add(60, 0, 0, wxEXPAND, 5);
414
415     wxBoxSizer* bSizer63;
416     bSizer63 = new wxBoxSizer(wxVERTICAL);
417
418
419     bSizer63->Add(0, 50, 0, wxEXPAND, 5);
420
421     wxBoxSizer* bSizer64;
422     bSizer64 = new wxBoxSizer(wxHORIZONTAL);
423
424     m_staticText40 = new wxStaticText(this, wxID_ANY, wxT("Bitcoin "), wxDefaultPosition, wxDefaultSize, 0);
425     m_staticText40->Wrap(-1);
426     m_staticText40->SetFont(wxFont(10, 74, 90, 92, false, wxT("Tahoma")));
427
428     bSizer64->Add(m_staticText40, 0, wxALIGN_BOTTOM|wxTOP|wxBOTTOM|wxLEFT, 5);
429
430     m_staticTextVersion = new wxStaticText(this, wxID_ANY, wxT("version"), wxDefaultPosition, wxDefaultSize, 0);
431     m_staticTextVersion->Wrap(-1);
432     m_staticTextVersion->SetFont(wxFont(10, 74, 90, 90, false, wxT("Tahoma")));
433
434     bSizer64->Add(m_staticTextVersion, 0, wxALIGN_BOTTOM|wxTOP|wxBOTTOM|wxRIGHT, 5);
435
436     bSizer63->Add(bSizer64, 0, wxEXPAND, 5);
437
438
439     bSizer63->Add(0, 4, 0, wxEXPAND, 5);
440
441     m_staticTextMain = new wxStaticText(this, wxID_ANY, wxT("Copyright © 2009 Satoshi Nakamoto.\n\nThis is experimental
    software.  Do not rely on it for actual financial transactions.\n\nDistributed under the MIT/X11 software license, see the
    accompanying file license.txt or http://www.opensource.org/licenses/mit-license.php.\n\nThis product includes software developed
    by the OpenSSL Project for use in the OpenSSL Toolkit (http://www.openssl.org/) and cryptographic software written by Eric
    Young (eay@cryptsoft.com)."), wxDefaultPosition, wxDefaultSize, 0);
442     m_staticTextMain->Wrap(400);
443     bSizer63->Add(m_staticTextMain, 0, wxALL, 5);
444
445
446     bSizer63->Add(0, 0, 1, wxEXPAND, 5);
447
448     bSizer62->Add(bSizer63, 1, wxEXPAND, 5);
449
450     bSizer60->Add(bSizer62, 1, wxEXPAND, 5);
451
452     wxBoxSizer* bSizer61;
453     bSizer61 = new wxBoxSizer(wxHORIZONTAL);
454
455
456     bSizer61->Add(0, 0, 1, wxEXPAND, 5);
457
458     m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxSize(85,25), 0);
459     bSizer61->Add(m_buttonOK, 0, wxALL, 5);
460
461     bSizer60->Add(bSizer61, 0, wxALIGN_RIGHT|wxEXPAND, 5);
462
463 this->SetSizer(bSizer60);
464 this->Layout();
465
466 // Connect Events
467     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CAboutDialogBase::OnButtonOK), NULL, this);
468 }
469
470 CAboutDialogBase::~CAboutDialogBase()
471 {
```

```
472 // Disconnect Events
473     m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CAboutDialogBase::OnButtonOK), NULL, this);
474 }
475
```

CSendDialogBase — the payment dialog. The instructional text at line 495 reveals a key v0.1.0 feature: `'Enter the recipient's IP address (e.g. 123.45.6.7) for online transfer with comments and confirmation, or bitcoin address...'` In the original design, the *primary* way to send Bitcoin was by IP address — a direct TCP connection to the recipient for interactive payment with comments. Sending to a Bitcoin address was the fallback. This IP-based payment was removed in later versions due to security concerns (no authentication, man-in-the-middle attacks).

```
476 CSendDialogBase::CSendDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos, const wxSize&
    size, long style) : wxDialog(parent, id, title, pos, size, style)
477 {
478 this->SetSizeHints(wxDefaultSize, wxDefaultSize);
479
480     wxBoxSizer* bSizer21;
481     bSizer21 = new wxBoxSizer(wxVERTICAL);
482
483
484     bSizer21->Add(0, 5, 0, wxEXPAND, 5);
485
486     wxFlexGridSizer* fgSizer1;
487     fgSizer1 = new wxFlexGridSizer(3, 2, 0, 0);
488     fgSizer1->AddGrowableCol(1);
489     fgSizer1->SetFlexibleDirection(wxBOTH);
490     fgSizer1->SetNonFlexibleGrowMode(wxFLEX_GROWMODE_SPECIFIED);
491
492
493     fgSizer1->Add(0, 0, 0, wxEXPAND, 5);
494
495     m_staticText14 = new wxStaticText(this, wxID_ANY, wxT("Enter the recipient's IP address (e.g. 123.45.6.7) for online
    transfer with comments and confirmation, \nor bitcoin address (e.g. 1NS17iag9jJgTHD1VXjvLCEnZuQ3rJED9L) if recipient is not
    online."), wxDefaultPosition, wxDefaultSize, 0);
496     m_staticText14->Wrap(-1);
497     fgSizer1->Add(m_staticText14, 0, wxTOP|wxRIGHT|wxLEFT, 5);
498
499     wxBoxSizer* bSizer47;
500     bSizer47 = new wxBoxSizer(wxHORIZONTAL);
501
502     bSizer47->SetMinSize(wxSize(70,-1));
503
504     bSizer47->Add(0, 0, 1, wxEXPAND, 5);
505
506     m_bitmapCheckMark = new wxStaticBitmap(this, wxID_ANY, wxICON(check), wxDefaultPosition, wxSize(16,16), 0);
507     bSizer47->Add(m_bitmapCheckMark, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
508
509     m_staticText36 = new wxStaticText(this, wxID_ANY, wxT("Pay &To:"), wxDefaultPosition, wxSize(-1,-1), wxALIGN_RIGHT);
510     m_staticText36->Wrap(-1);
511     bSizer47->Add(m_staticText36, 0, wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT, 5);
512
513     fgSizer1->Add(bSizer47, 1, wxEXPAND|wxLEFT, 5);
514
515     wxBoxSizer* bSizer19;
516     bSizer19 = new wxBoxSizer(wxHORIZONTAL);
517
518     m_textCtrlAddress = new wxTextCtrl(this, wxID_TEXTCTRLPAYTO, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
519     bSizer19->Add(m_textCtrlAddress, 1, wxALIGN_CENTER_VERTICAL|wxALL, 5);
520
521     m_buttonPaste = new wxButton(this, wxID_BUTTONPASTE, wxT("&Paste"), wxDefaultPosition, wxSize(-1,-1), wxBU_EXACTFIT);
522     bSizer19->Add(m_buttonPaste, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT, 5);
523
524     m_buttonAddress = new wxButton(this, wxID_BUTTONADDRESSBOOK, wxT(" Address &Book..."), wxDefaultPosition, wxDefaultSize,
    0);
525     bSizer19->Add(m_buttonAddress, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT, 5);
526
527     fgSizer1->Add(bSizer19, 1, wxEXPAND|wxRIGHT, 5);
528
529     m_staticText19 = new wxStaticText(this, wxID_ANY, wxT("&Amount:"), wxDefaultPosition, wxSize(-1,-1), wxALIGN_RIGHT);
530     m_staticText19->Wrap(-1);
531     fgSizer1->Add(m_staticText19, 0, wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT|wxALIGN_RIGHT, 5);
532
533     m_textCtrlAmount = new wxTextCtrl(this, wxID_TEXTCTRLAMOUNT, wxEmptyString, wxDefaultPosition, wxSize(145,-1), 0);
534     m_textCtrlAmount->SetMaxLength(20);
535     m_textCtrlAmount->SetFont(wxFont(wxNORMAL_FONT->GetPointSize(), 70, 90, 90, false, wxEmptyString));
536
537     fgSizer1->Add(m_textCtrlAmount, 0, wxALIGN_CENTER_VERTICAL|wxALL, 5);
538
539     m_staticText20 = new wxStaticText(this, wxID_ANY, wxT("T&ransfer:"), wxDefaultPosition, wxSize(-1,-1), wxALIGN_RIGHT);
540     m_staticText20->Wrap(-1);
```

```
    fgSizer1->Add(m_staticText20, 0, wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxLEFT, 5);

    wxString m_choiceTransferTypeChoices[] = { wxT(" Standard") };
int m_choiceTransferTypeNChoices = sizeof(m_choiceTransferTypeChoices) / sizeof(wxString);
    m_choiceTransferType = new wxChoice(this, wxID_CHOICETRANSFERTYPE, wxDefaultPosition, wxDefaultSize,
m_choiceTransferTypeNChoices, m_choiceTransferTypeChoices, 0);
    m_choiceTransferType->SetSelection(0);
    fgSizer1->Add(m_choiceTransferType, 0, wxALIGN_CENTER_VERTICAL|wxALL, 5);

    bSizer21->Add(fgSizer1, 0, wxEXPAND|wxLEFT, 5);

    wxBoxSizer* bSizer672;
    bSizer672 = new wxBoxSizer(wxHORIZONTAL);

    wxBoxSizer* bSizer681;
    bSizer681 = new wxBoxSizer(wxVERTICAL);

    m_staticTextFrom = new wxStaticText(this, wxID_ANY, wxT("&From:"), wxDefaultPosition, wxDefaultSize, 0);
    m_staticTextFrom->Wrap(-1);
    bSizer681->Add(m_staticTextFrom, 0, wxBOTTOM|wxLEFT, 5);

    m_textCtrlFrom = new wxTextCtrl(this, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    bSizer681->Add(m_textCtrlFrom, 0, wxLEFT|wxEXPAND, 5);

    bSizer672->Add(bSizer681, 1, wxEXPAND|wxBOTTOM|wxRIGHT|wxLEFT, 5);

    bSizer21->Add(bSizer672, 0, wxEXPAND, 5);

    wxBoxSizer* bSizer67;
    bSizer67 = new wxBoxSizer(wxHORIZONTAL);

    wxBoxSizer* bSizer68;
    bSizer68 = new wxBoxSizer(wxVERTICAL);

    m_staticTextMessage = new wxStaticText(this, wxID_ANY, wxT("&Message:"), wxDefaultPosition, wxDefaultSize, 0);
    m_staticTextMessage->Wrap(-1);
    bSizer68->Add(m_staticTextMessage, 0, wxTOP|wxBOTTOM|wxLEFT, 5);

    m_textCtrlMessage = new wxTextCtrl(this, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, wxTE_MULTILINE);
    bSizer68->Add(m_textCtrlMessage, 1, wxEXPAND|wxLEFT, 5);

    bSizer67->Add(bSizer68, 1, wxEXPAND|wxBOTTOM|wxRIGHT|wxLEFT, 5);

    bSizer21->Add(bSizer67, 1, wxEXPAND, 5);

    wxBoxSizer* bSizer23;
    bSizer23 = new wxBoxSizer(wxHORIZONTAL);


    bSizer23->Add(0, 0, 1, wxEXPAND, 5);

    m_buttonSend = new wxButton(this, wxID_BUTTONSEND, wxT("&Send"), wxDefaultPosition, wxSize(-1,-1), 0);
    m_buttonSend->SetFont(wxFont(wxNORMAL_FONT->GetPointSize(), 70, 90, 90, false, wxEmptyString));
    m_buttonSend->SetMinSize(wxSize(85,25));

    bSizer23->Add(m_buttonSend, 0, wxALL, 5);

    m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxSize(-1,-1), 0);
    m_buttonCancel->SetMinSize(wxSize(85,25));

    bSizer23->Add(m_buttonCancel, 0, wxALL, 5);

    bSizer21->Add(bSizer23, 0, wxEXPAND, 5);

this->SetSizer(bSizer21);
this->Layout();

// Connect Events
    m_textCtrlAddress->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL, this);
    m_textCtrlAddress->Connect(wxEVT_COMMAND_TEXT_UPDATED, wxCommandEventHandler(CSendDialogBase::OnTextAddress), NULL, this);
    m_buttonPaste->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CSendDialogBase::OnButtonPaste), NULL, this);
    m_buttonAddress->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CSendDialogBase::OnButtonAddressBook), NULL,
this);
    m_textCtrlAmount->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL, this);
    m_textCtrlAmount->Connect(wxEVT_KILL_FOCUS, wxFocusEventHandler(CSendDialogBase::OnKillFocusAmount), NULL, this);
    m_textCtrlFrom->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL, this);
    m_textCtrlMessage->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL, this);
    m_buttonSend->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CSendDialogBase::OnButtonSend), NULL, this);
    m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CSendDialogBase::OnButtonCancel), NULL, this);
}

CSendDialogBase::~CSendDialogBase()
{
// Disconnect Events
    m_textCtrlAddress->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL, this);
    m_textCtrlAddress->Disconnect(wxEVT_COMMAND_TEXT_UPDATED, wxCommandEventHandler(CSendDialogBase::OnTextAddress), NULL,
this);
    m_buttonPaste->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CSendDialogBase::OnButtonPaste), NULL, this);
```

```
626     m_buttonAddress->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CSendDialogBase::OnButtonAddressBook), NULL,
    this);
627     m_textCtrlAmount->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL, this);
628     m_textCtrlAmount->Disconnect(wxEVT_KILL_FOCUS, wxFocusEventHandler(CSendDialogBase::OnKillFocusAmount), NULL, this);
629     m_textCtrlFrom->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL, this);
630     m_textCtrlMessage->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL, this);
631     m_buttonSend->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CSendDialogBase::OnButtonSend), NULL, this);
632     m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CSendDialogBase::OnButtonCancel), NULL,
    this);
633 }
634
```

`CSendingDialogBase` — a progress dialog shown during payment. The text field starts with `'Connecting...'`, the OK button starts disabled (`Enable(false)`), and Cancel is available. This dialog is specifically for the IP-based payment flow: the client connects to the recipient's node, negotiates the transaction interactively, and updates the status text. For address-based payments, this dialog was unnecessary.

```
635 CSendingDialogBase::CSendingDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos, const
    wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
636 {
637 this->SetSizeHints(wxDefaultSize, wxDefaultSize);
638
639     wxBoxSizer* bSizer68;
640     bSizer68 = new wxBoxSizer(wxVERTICAL);
641
642     m_staticTextSending = new wxStaticText(this, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,14), 0);
643     m_staticTextSending->Wrap(-1);
644     bSizer68->Add(m_staticTextSending, 0, wxALIGN_CENTER_VERTICAL|wxEXPAND|wxTOP|wxRIGHT|wxLEFT, 8);
645
646     m_textCtrlStatus = new wxTextCtrl(this, wxID_ANY, wxT("\n\nConnecting..."), wxDefaultPosition, wxDefaultSize,
    wxTE_CENTRE|wxTE_MULTILINE|wxTE_NO_VSCROLL|wxTE_READONLY|wxNO_BORDER);
647     m_textCtrlStatus->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_BTNFACE));
648
649     bSizer68->Add(m_textCtrlStatus, 1, wxEXPAND|wxRIGHT|wxLEFT, 10);
650
651     wxBoxSizer* bSizer69;
652     bSizer69 = new wxBoxSizer(wxHORIZONTAL);
653
654
655     bSizer69->Add(0, 0, 1, wxEXPAND, 5);
656
657     m_buttonOK = new wxButton(this, wxID_ANY, wxT("OK"), wxDefaultPosition, wxDefaultSize, 0);
658     m_buttonOK->Enable(false);
659     m_buttonOK->SetMinSize(wxSize(85,25));
660
661     bSizer69->Add(m_buttonOK, 0, wxALL, 5);
662
663     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxSize(-1,-1), 0);
664     m_buttonCancel->SetMinSize(wxSize(85,25));
665
666     bSizer69->Add(m_buttonCancel, 0, wxALL, 5);
667
668     bSizer68->Add(bSizer69, 0, wxEXPAND, 5);
669
670 this->SetSizer(bSizer68);
671 this->Layout();
672
673 // Connect Events
674 this->Connect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CSendingDialogBase::OnClose));
675 this->Connect(wxEVT_PAINT, wxPaintEventHandler(CSendingDialogBase::OnPaint));
676     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CSendingDialogBase::OnButtonOK), NULL, this);
677     m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CSendingDialogBase::OnButtonCancel), NULL,
    this);
678 }
679
680 CSendingDialogBase::~CSendingDialogBase()
681 {
682 // Disconnect Events
683 this->Disconnect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CSendingDialogBase::OnClose));
684 this->Disconnect(wxEVT_PAINT, wxPaintEventHandler(CSendingDialogBase::OnPaint));
685     m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CSendingDialogBase::OnButtonOK), NULL, this);
686     m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CSendingDialogBase::OnButtonCancel), NULL,
    this);
687 }
688
```

`CYourAddressDialogBase` — manages the user's own receiving addresses. The instructional text reads: `'These are your Bitcoin addresses for receiving payments. You may want to give a different one to each sender so you can keep track of who is paying you.'` The stated reason is bookkeeping — knowing who paid you — but the habit doubles as privacy. The whitepaper had already made that case in October 2008: 'a new key pair should be used for each transaction to keep them from being linked to a common owner.'[6]

```
689 CYourAddressDialogBase::CYourAddressDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos,
    const wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
690 {
691 this->SetSizeHints(wxDefaultSize, wxDefaultSize);
692
693     wxBoxSizer* bSizer68;
694     bSizer68 = new wxBoxSizer(wxVERTICAL);
695
696
697     bSizer68->Add(0, 5, 0, wxEXPAND, 5);
698
699     m_staticText45 = new wxStaticText(this, wxID_ANY, wxT("These are your Bitcoin addresses for receiving payments.\nYou may
    want to give a different one to each sender so you can keep track of who is paying you."), wxDefaultPosition, wxDefaultSize, 0);
700     m_staticText45->Wrap(600);
701     bSizer68->Add(m_staticText45, 0, wxALL, 5);
702
703     m_listCtrl = new wxListCtrl(this, wxID_LISTCTRL, wxDefaultPosition, wxDefaultSize,
    wxLC_NO_SORT_HEADER|wxLC_REPORT|wxLC_SORT_ASCENDING);
704     bSizer68->Add(m_listCtrl, 1, wxALL|wxEXPAND, 5);
705
706     wxBoxSizer* bSizer69;
707     bSizer69 = new wxBoxSizer(wxHORIZONTAL);
708
709
710     bSizer69->Add(0, 0, 1, wxEXPAND, 5);
711
712     m_buttonRename = new wxButton(this, wxID_BUTTONRENAME, wxT("&Rename..."), wxDefaultPosition, wxDefaultSize, 0);
713     m_buttonRename->SetMinSize(wxSize(85,25));
714
715     bSizer69->Add(m_buttonRename, 0, wxALL, 5);
716
717     m_buttonNew = new wxButton(this, wxID_BUTTONNEW, wxT("&New Address..."), wxDefaultPosition, wxSize(-1,-1), 0);
718     m_buttonNew->SetMinSize(wxSize(110,25));
719
720     bSizer69->Add(m_buttonNew, 0, wxALL, 5);
721
722     m_buttonCopy = new wxButton(this, wxID_BUTTONCOPY, wxT("&Copy to Clipboard"), wxDefaultPosition, wxSize(-1,-1), 0);
723     m_buttonCopy->SetMinSize(wxSize(120,25));
724
725     bSizer69->Add(m_buttonCopy, 0, wxALL, 5);
726
727     m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxDefaultSize, 0);
728     m_buttonOK->SetMinSize(wxSize(85,25));
729
730     bSizer69->Add(m_buttonOK, 0, wxALL, 5);
731
732     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxSize(-1,-1), 0);
733     m_buttonCancel->SetMinSize(wxSize(85,25));
734
735     bSizer69->Add(m_buttonCancel, 0, wxALL, 5);
736
737     bSizer68->Add(bSizer69, 0, wxEXPAND, 5);
738
739 this->SetSizer(bSizer68);
740 this->Layout();
741
742 // Connect Events
743 this->Connect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CYourAddressDialogBase::OnClose));
744     m_listCtrl->Connect(wxEVT_COMMAND_LIST_END_LABEL_EDIT, wxListEventHandler(CYourAddressDialogBase::OnListEndLabelEdit), NULL,
    this);
745     m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED, wxListEventHandler(CYourAddressDialogBase::OnListItemActivated),
    NULL, this);
746     m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_SELECTED, wxListEventHandler(CYourAddressDialogBase::OnListItemSelected), NULL,
    this);
747     m_buttonRename->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CYourAddressDialogBase::OnButtonRename), NULL,
    this);
748     m_buttonNew->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CYourAddressDialogBase::OnButtonNew), NULL, this);
749     m_buttonCopy->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CYourAddressDialogBase::OnButtonCopy), NULL,
    this);
750     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CYourAddressDialogBase::OnButtonOK), NULL, this);
751     m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CYourAddressDialogBase::OnButtonCancel), NULL,
    this);
752 }
```

```
753
754 CYourAddressDialogBase::~CYourAddressDialogBase()
755 {
756 // Disconnect Events
757 this->Disconnect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CYourAddressDialogBase::OnClose));
758     m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_END_LABEL_EDIT, wxListEventHandler(CYourAddressDialogBase::OnListEndLabelEdit),
    NULL, this);
759     m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED, wxListEventHandler(CYourAddressDialogBase::OnListItemActivated),
    NULL, this);
760     m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_SELECTED, wxListEventHandler(CYourAddressDialogBase::OnListItemSelected),
    NULL, this);
761     m_buttonRename->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CYourAddressDialogBase::OnButtonRename),
    NULL, this);
762     m_buttonNew->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CYourAddressDialogBase::OnButtonNew), NULL,
    this);
763     m_buttonCopy->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CYourAddressDialogBase::OnButtonCopy), NULL,
    this);
764     m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CYourAddressDialogBase::OnButtonOK), NULL, this);
765     m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CYourAddressDialogBase::OnButtonCancel),
    NULL, this);
766 }
767
768 CAddressBookDialogBase::CAddressBookDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos,
    const wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
769 {
770 this->SetSizeHints(wxDefaultSize, wxDefaultSize);
771
772     wxBoxSizer* bSizer68;
773     bSizer68 = new wxBoxSizer(wxVERTICAL);
774
775
776     bSizer68->Add(0, 5, 0, wxEXPAND, 5);
777
778     m_staticText55 = new wxStaticText(this, wxID_ANY, wxT("Bitcoin Address"), wxDefaultPosition, wxDefaultSize, 0);
779     m_staticText55->Wrap(-1);
780     m_staticText55->Hide();
781
782     bSizer68->Add(m_staticText55, 0, wxTOP|wxRIGHT|wxLEFT, 5);
783
784     m_listCtrl = new wxListCtrl(this, wxID_LISTCTRL, wxDefaultPosition, wxDefaultSize,
    wxLC_NO_SORT_HEADER|wxLC_REPORT|wxLC_SORT_ASCENDING);
785     bSizer68->Add(m_listCtrl, 1, wxALL|wxEXPAND, 5);
786
787     wxBoxSizer* bSizer69;
788     bSizer69 = new wxBoxSizer(wxHORIZONTAL);
789
790
791     bSizer69->Add(0, 0, 1, wxEXPAND, 5);
792
793     m_buttonEdit = new wxButton(this, wxID_BUTTONEDIT, wxT("&Edit..."), wxDefaultPosition, wxDefaultSize, 0);
794     m_buttonEdit->SetMinSize(wxSize(85,25));
795
796     bSizer69->Add(m_buttonEdit, 0, wxALL, 5);
797
798     m_buttonNew = new wxButton(this, wxID_BUTTONNEW, wxT("&New Address..."), wxDefaultPosition, wxDefaultSize, 0);
799     m_buttonNew->SetMinSize(wxSize(110,25));
800
801     bSizer69->Add(m_buttonNew, 0, wxALL, 5);
802
803     m_buttonDelete = new wxButton(this, wxID_BUTTONDELETE, wxT("&Delete"), wxDefaultPosition, wxDefaultSize, 0);
804     m_buttonDelete->SetMinSize(wxSize(85,25));
805
806     bSizer69->Add(m_buttonDelete, 0, wxALL, 5);
807
808     m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxSize(-1,-1), 0);
809     m_buttonOK->SetMinSize(wxSize(85,25));
810
811     bSizer69->Add(m_buttonOK, 0, wxALL, 5);
812
813     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxSize(-1,-1), 0);
814     m_buttonCancel->SetMinSize(wxSize(85,25));
815
816     bSizer69->Add(m_buttonCancel, 0, wxALL, 5);
817
818     bSizer68->Add(bSizer69, 0, wxEXPAND, 5);
819
820 this->SetSizer(bSizer68);
821 this->Layout();
822
823 // Connect Events
824 this->Connect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CAddressBookDialogBase::OnClose));
825     m_listCtrl->Connect(wxEVT_COMMAND_LIST_END_LABEL_EDIT, wxListEventHandler(CAddressBookDialogBase::OnListEndLabelEdit), NULL,
    this);
826     m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED, wxListEventHandler(CAddressBookDialogBase::OnListItemActivated),
    NULL, this);
827     m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_SELECTED, wxListEventHandler(CAddressBookDialogBase::OnListItemSelected), NULL,
    this);
828     m_buttonEdit->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CAddressBookDialogBase::OnButtonEdit), NULL,
    this);
```

```
    m_buttonNew->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CAddressBookDialogBase::OnButtonNew), NULL, this);
    m_buttonDelete->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CAddressBookDialogBase::OnButtonDelete), NULL, this);
    m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CAddressBookDialogBase::OnButtonOK), NULL, this);
    m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CAddressBookDialogBase::OnButtonCancel), NULL, this);
}

CAddressBookDialogBase::~CAddressBookDialogBase()
{
// Disconnect Events
this->Disconnect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CAddressBookDialogBase::OnClose));
    m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_END_LABEL_EDIT, wxListEventHandler(CAddressBookDialogBase::OnListEndLabelEdit), NULL, this);
    m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED, wxListEventHandler(CAddressBookDialogBase::OnListItemActivated), NULL, this);
    m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_SELECTED, wxListEventHandler(CAddressBookDialogBase::OnListItemSelected), NULL, this);
    m_buttonEdit->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CAddressBookDialogBase::OnButtonEdit), NULL, this);
    m_buttonNew->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CAddressBookDialogBase::OnButtonNew), NULL, this);
    m_buttonDelete->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CAddressBookDialogBase::OnButtonDelete), NULL, this);
    m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CAddressBookDialogBase::OnButtonOK), NULL, this);
    m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CAddressBookDialogBase::OnButtonCancel), NULL, this);
}

```

`CProductsDialogBase` — the marketplace browser with a category dropdown, search field, and product list. Title: `'Marketplace'`. This was Satoshi's vision of Bitcoin as more than money — it was to be a complete commerce platform where users could list and discover products for sale, all within the Bitcoin client. The marketplace code exists in `market.cpp` but was never fully connected to the network protocol.

```
CProductsDialogBase::CProductsDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos, const wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
{
this->SetSizeHints(wxDefaultSize, wxDefaultSize);

    wxBoxSizer* bSizer22;
    bSizer22 = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* bSizer23;
    bSizer23 = new wxBoxSizer(wxHORIZONTAL);

    m_comboBoxCategory = new wxComboBox(this, wxID_ANY, wxT("(Any Category)"), wxDefaultPosition, wxSize(150,-1), 0, NULL, 0);
    m_comboBoxCategory->Append(wxT("(Any Category)"));
    bSizer23->Add(m_comboBoxCategory, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlSearch = new wxTextCtrl(this, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    bSizer23->Add(m_textCtrlSearch, 1, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonSearch = new wxButton(this, wxID_ANY, wxT("&Search"), wxDefaultPosition, wxDefaultSize, 0);
    bSizer23->Add(m_buttonSearch, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    bSizer22->Add(bSizer23, 0, wxEXPAND|wxTOP|wxBOTTOM|wxRIGHT, 5);

    m_listCtrl = new wxListCtrl(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxLC_NO_SORT_HEADER|wxLC_REPORT);
    bSizer22->Add(m_listCtrl, 1, wxALL|wxEXPAND, 5);

this->SetSizer(bSizer22);
this->Layout();

// Connect Events
    m_comboBoxCategory->Connect(wxEVT_COMMAND_COMBOBOX_SELECTED, wxCommandEventHandler(CProductsDialogBase::OnCombobox), NULL, this);
    m_textCtrlSearch->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CProductsDialogBase::OnKeyDown), NULL, this);
    m_buttonSearch->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CProductsDialogBase::OnButtonSearch), NULL, this);
    m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED, wxListEventHandler(CProductsDialogBase::OnListItemActivated), NULL, this);
}

CProductsDialogBase::~CProductsDialogBase()
{
// Disconnect Events
    m_comboBoxCategory->Disconnect(wxEVT_COMMAND_COMBOBOX_SELECTED, wxCommandEventHandler(CProductsDialogBase::OnCombobox), NULL, this);
    m_textCtrlSearch->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CProductsDialogBase::OnKeyDown), NULL, this);
```

```
889     m_buttonSearch->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CProductsDialogBase::OnButtonSearch), NULL,
    this);
890     m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED, wxListEventHandler(CProductsDialogBase::OnListItemActivated),
    NULL, this);
891 }
892
```

CEditProductDialogBase — a full product listing editor with fields for category, title, price, description, order form instructions, and up to 20 customizable form fields (label + value pairs with individual delete buttons). The 660×640 size and scrolled window suggest Satoshi expected detailed product listings. This is the most complex dialog in the entire application — more elaborate than the payment dialog — revealing how central the marketplace concept was to Satoshi's original vision.

```
893 CEditProductDialogBase::CEditProductDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos,
    const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
894 {
895 this->SetSizeHints(wxDefaultSize, wxDefaultSize);
896 this->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_MENU));
897
898     wxBoxSizer* bSizer20;
899     bSizer20 = new wxBoxSizer(wxVERTICAL);
900
901     m_scrolledWindow = new wxScrolledWindow(this, wxID_ANY, wxDefaultPosition, wxDefaultSize,
    wxHSCROLL|wxTAB_TRAVERSAL|wxVSCROLL);
902     m_scrolledWindow->SetScrollRate(5, 5);
903     m_scrolledWindow->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_WINDOW));
904
905     wxBoxSizer* bSizer21;
906     bSizer21 = new wxBoxSizer(wxVERTICAL);
907
908     wxFlexGridSizer* fgSizer8;
909     fgSizer8 = new wxFlexGridSizer(0, 2, 0, 0);
910     fgSizer8->AddGrowableCol(1);
911     fgSizer8->SetFlexibleDirection(wxBOTH);
912     fgSizer8->SetNonFlexibleGrowMode(wxFLEX_GROWMODE_SPECIFIED);
913
914     m_staticText106 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Category"), wxDefaultPosition, wxDefaultSize,
    wxALIGN_RIGHT);
915     m_staticText106->Wrap(-1);
916     fgSizer8->Add(m_staticText106, 0, wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxLEFT, 5);
917
918     m_comboBoxCategory = new wxComboBox(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0, NULL,
    0);
919     m_comboBoxCategory->SetMinSize(wxSize(180,-1));
920
921     fgSizer8->Add(m_comboBoxCategory, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
922
923     m_staticText108 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Title"), wxDefaultPosition, wxDefaultSize,
    wxALIGN_RIGHT);
924     m_staticText108->Wrap(-1);
925     fgSizer8->Add(m_staticText108, 0, wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxLEFT, 5);
926
927     m_textCtrlTitle = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
928     fgSizer8->Add(m_textCtrlTitle, 1, wxALL|wxALIGN_CENTER_VERTICAL|wxEXPAND, 5);
929
930     m_staticText107 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Price"), wxDefaultPosition, wxDefaultSize,
    wxALIGN_RIGHT);
931     m_staticText107->Wrap(-1);
932     fgSizer8->Add(m_staticText107, 0, wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxLEFT, 5);
933
934     m_textCtrlPrice = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
935     m_textCtrlPrice->SetMinSize(wxSize(105,-1));
936
937     fgSizer8->Add(m_textCtrlPrice, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
938
939     bSizer21->Add(fgSizer8, 0, wxEXPAND|wxTOP|wxRIGHT|wxLEFT, 5);
940
941     m_staticText22 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Page 1: Description"), wxDefaultPosition, wxDefaultSize,
    0);
942     m_staticText22->Wrap(-1);
943     bSizer21->Add(m_staticText22, 0, wxTOP|wxRIGHT|wxLEFT, 5);
944
945     m_textCtrlDescription = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize,
    wxTE_MULTILINE);
946     m_textCtrlDescription->SetMinSize(wxSize(-1,170));
947
948     bSizer21->Add(m_textCtrlDescription, 0, wxALL|wxEXPAND, 5);
949
950     m_staticText23 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Page 2: Order Form"), wxDefaultPosition, wxDefaultSize,
    0);
```

```
951     m_staticText23->Wrap(-1);
952     bSizer21->Add(m_staticText23, 0, wxTOP|wxRIGHT|wxLEFT, 5);
953
954     m_textCtrlInstructions = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize,
    wxTE_MULTILINE);
955     m_textCtrlInstructions->SetMinSize(wxSize(-1,120));
956
957     bSizer21->Add(m_textCtrlInstructions, 0, wxEXPAND|wxALL, 5);
958
959     fgSizer5 = new wxFlexGridSizer(0, 3, 0, 0);
960     fgSizer5->AddGrowableCol(1);
961     fgSizer5->SetFlexibleDirection(wxBOTH);
962     fgSizer5->SetNonFlexibleGrowMode(wxFLEX_GROWMODE_SPECIFIED);
963
964     m_staticText24 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Label"), wxDefaultPosition, wxDefaultSize, 0);
965     m_staticText24->Wrap(-1);
966     fgSizer5->Add(m_staticText24, 0, wxALIGN_CENTER_VERTICAL|wxTOP|wxRIGHT|wxLEFT, 5);
967
968     m_staticText25 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Comma separated list of choices, or leave blank for text
    field"), wxDefaultPosition, wxDefaultSize, 0);
969     m_staticText25->Wrap(-1);
970     fgSizer5->Add(m_staticText25, 0, wxALIGN_CENTER_VERTICAL|wxTOP|wxRIGHT|wxLEFT, 5);
971
972
973     fgSizer5->Add(0, 0, 1, wxEXPAND, 5);
974
975     m_textCtrlLabel0 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
976     m_textCtrlLabel0->SetMinSize(wxSize(150,-1));
977
978     fgSizer5->Add(m_textCtrlLabel0, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
979
980     m_textCtrlField0 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
981     fgSizer5->Add(m_textCtrlField0, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
982
983     m_buttonDel0 = new wxButton(m_scrolledWindow, wxID_DEL0, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
984     fgSizer5->Add(m_buttonDel0, 0, wxRIGHT|wxLEFT|wxALIGN_CENTER_VERTICAL, 5);
985
986     m_textCtrlLabel1 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
987     m_textCtrlLabel1->SetMinSize(wxSize(150,-1));
988
989     fgSizer5->Add(m_textCtrlLabel1, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
990
991     m_textCtrlField1 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
992     fgSizer5->Add(m_textCtrlField1, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
993
994     m_buttonDel1 = new wxButton(m_scrolledWindow, wxID_DEL1, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
995     fgSizer5->Add(m_buttonDel1, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
996
997     m_textCtrlLabel2 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
998     m_textCtrlLabel2->SetMinSize(wxSize(150,-1));
999
1000    fgSizer5->Add(m_textCtrlLabel2, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1001
1002    m_textCtrlField2 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
1003    fgSizer5->Add(m_textCtrlField2, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1004
1005    m_buttonDel2 = new wxButton(m_scrolledWindow, wxID_DEL2, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
1006    fgSizer5->Add(m_buttonDel2, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1007
1008    m_textCtrlLabel3 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
1009    m_textCtrlLabel3->SetMinSize(wxSize(150,-1));
1010
1011    fgSizer5->Add(m_textCtrlLabel3, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1012
1013    m_textCtrlField3 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
1014    fgSizer5->Add(m_textCtrlField3, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1015
1016    m_buttonDel3 = new wxButton(m_scrolledWindow, wxID_DEL3, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
1017    fgSizer5->Add(m_buttonDel3, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1018
1019    m_textCtrlLabel4 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
1020    m_textCtrlLabel4->SetMinSize(wxSize(150,-1));
1021
1022    fgSizer5->Add(m_textCtrlLabel4, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1023
1024    m_textCtrlField4 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
1025    fgSizer5->Add(m_textCtrlField4, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1026
1027    m_buttonDel4 = new wxButton(m_scrolledWindow, wxID_DEL4, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
1028    fgSizer5->Add(m_buttonDel4, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1029
1030    m_textCtrlLabel5 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
1031    m_textCtrlLabel5->SetMinSize(wxSize(150,-1));
1032
1033    fgSizer5->Add(m_textCtrlLabel5, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1034
1035    m_textCtrlField5 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
1036    fgSizer5->Add(m_textCtrlField5, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1037
```

```
    m_buttonDel5 = new wxButton(m_scrolledWindow, wxID_DEL5, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel5, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel6 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel6->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel6, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField6 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField6, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel6 = new wxButton(m_scrolledWindow, wxID_DEL6, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel6, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel7 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel7->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel7, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField7 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField7, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel7 = new wxButton(m_scrolledWindow, wxID_DEL7, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel7, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel8 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel8->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel8, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField8 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField8, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel8 = new wxButton(m_scrolledWindow, wxID_DEL8, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel8, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel9 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel9->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel9, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField9 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField9, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel9 = new wxButton(m_scrolledWindow, wxID_DEL9, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel9, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel10 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel10->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel10, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField10 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField10, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel10 = new wxButton(m_scrolledWindow, wxID_DEL10, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel10, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel11 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel11->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel11, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField11 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField11, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel11 = new wxButton(m_scrolledWindow, wxID_DEL11, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel11, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel12 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel12->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel12, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField12 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField12, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel12 = new wxButton(m_scrolledWindow, wxID_DEL12, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel12, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel13 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel13->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel13, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField13 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField13, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel13 = new wxButton(m_scrolledWindow, wxID_DEL13, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
```

```
    fgSizer5->Add(m_buttonDel13, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel14 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel14->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel14, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField14 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField14, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel14 = new wxButton(m_scrolledWindow, wxID_DEL14, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel14, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel15 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel15->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel15, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField15 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField15, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel15 = new wxButton(m_scrolledWindow, wxID_DEL15, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel15, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel16 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel16->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel16, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField16 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField16, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel16 = new wxButton(m_scrolledWindow, wxID_DEL16, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel16, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel17 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel17->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel17, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField17 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField17, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel17 = new wxButton(m_scrolledWindow, wxID_DEL17, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel17, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel18 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel18->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel18, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField18 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField18, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel18 = new wxButton(m_scrolledWindow, wxID_DEL18, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel18, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    m_textCtrlLabel19 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_textCtrlLabel19->SetMinSize(wxSize(150,-1));

    fgSizer5->Add(m_textCtrlLabel19, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);

    m_textCtrlField19 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
    fgSizer5->Add(m_textCtrlField19, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);

    m_buttonDel19 = new wxButton(m_scrolledWindow, wxID_DEL19, wxT("Delete"), wxDefaultPosition, wxSize(60,20), 0);
    fgSizer5->Add(m_buttonDel19, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);

    bSizer21->Add(fgSizer5, 0, wxEXPAND, 5);

    wxBoxSizer* bSizer25;
    bSizer25 = new wxBoxSizer(wxHORIZONTAL);

    m_buttonAddField = new wxButton(m_scrolledWindow, wxID_ANY, wxT("&Add Field"), wxDefaultPosition, wxDefaultSize, 0);
    bSizer25->Add(m_buttonAddField, 0, wxALL, 5);

    bSizer21->Add(bSizer25, 0, wxALIGN_CENTER_HORIZONTAL, 5);

    m_scrolledWindow->SetSizer(bSizer21);
    m_scrolledWindow->Layout();
    bSizer21->Fit(m_scrolledWindow);
    bSizer20->Add(m_scrolledWindow, 1, wxEXPAND|wxALL, 5);

    wxBoxSizer* bSizer26;
    bSizer26 = new wxBoxSizer(wxHORIZONTAL);

    m_buttonOK = new wxButton(this, wxID_BUTTONSEND, wxT("&Send"), wxDefaultPosition, wxDefaultSize, 0);
    m_buttonOK->SetMinSize(wxSize(85,25));

```

```
    bSizer26->Add(m_buttonOK, 0, wxALL, 5);

    m_buttonPreview = new wxButton(this, wxID_BUTTONPREVIEW, wxT("&Preview"), wxDefaultPosition, wxDefaultSize, 0);
    m_buttonPreview->SetMinSize(wxSize(85,25));

    bSizer26->Add(m_buttonPreview, 0, wxALL, 5);

    m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxDefaultSize, 0);
    m_buttonCancel->SetMinSize(wxSize(85,25));

    bSizer26->Add(m_buttonCancel, 0, wxALL, 5);

    bSizer20->Add(bSizer26, 0, wxALIGN_RIGHT, 5);

this->SetSizer(bSizer20);
this->Layout();

// Connect Events
    m_textCtrlTitle->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlPrice->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlDescription->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlInstructions->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlLabel0->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField0->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel0->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel0), NULL,
this);
    m_textCtrlLabel1->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField1->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel1->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel1), NULL,
this);
    m_textCtrlLabel2->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField2->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel2->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel2), NULL,
this);
    m_textCtrlLabel3->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField3->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel3->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel3), NULL,
this);
    m_textCtrlLabel4->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField4->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel4->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel4), NULL,
this);
    m_textCtrlLabel5->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField5->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel5->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel5), NULL,
this);
    m_textCtrlLabel6->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField6->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel6->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel6), NULL,
this);
    m_textCtrlLabel7->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField7->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel7->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel7), NULL,
this);
    m_textCtrlLabel8->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField8->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel8->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel8), NULL,
this);
    m_textCtrlLabel9->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField9->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel9->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel9), NULL,
this);
    m_textCtrlLabel10->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField10->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel10->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel10), NULL,
this);
    m_textCtrlLabel11->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField11->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel11->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel11), NULL,
this);
    m_textCtrlLabel12->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField12->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel12->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel12), NULL,
this);
    m_textCtrlLabel13->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField13->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel13->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel13), NULL,
this);
    m_textCtrlLabel14->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField14->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel14->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel14), NULL,
this);
    m_textCtrlLabel15->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField15->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_buttonDel15->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel15), NULL,
this);
    m_textCtrlLabel16->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
    m_textCtrlField16->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
```

```
1288     m_buttonDel16->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel16), NULL,
    this);
1289     m_textCtrlLabel17->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1290     m_textCtrlField17->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1291     m_buttonDel17->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel17), NULL,
    this);
1292     m_textCtrlLabel18->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1293     m_textCtrlField18->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1294     m_buttonDel18->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel18), NULL,
    this);
1295     m_textCtrlLabel19->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1296     m_textCtrlField19->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1297     m_buttonDel19->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel19), NULL,
    this);
1298     m_buttonAddField->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonAddField),
    NULL, this);
1299     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonSend), NULL, this);
1300     m_buttonPreview->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonPreview), NULL,
    this);
1301     m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonCancel), NULL,
    this);
1302 }
1303
1304 CEditProductDialogBase::~CEditProductDialogBase()
1305 {
1306 // Disconnect Events
1307     m_textCtrlTitle->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1308     m_textCtrlPrice->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1309     m_textCtrlDescription->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1310     m_textCtrlInstructions->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1311     m_textCtrlLabel0->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1312     m_textCtrlField0->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1313     m_buttonDel0->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel0), NULL,
    this);
1314     m_textCtrlLabel1->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1315     m_textCtrlField1->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1316     m_buttonDel1->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel1), NULL,
    this);
1317     m_textCtrlLabel2->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1318     m_textCtrlField2->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1319     m_buttonDel2->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel2), NULL,
    this);
1320     m_textCtrlLabel3->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1321     m_textCtrlField3->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1322     m_buttonDel3->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel3), NULL,
    this);
1323     m_textCtrlLabel4->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1324     m_textCtrlField4->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1325     m_buttonDel4->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel4), NULL,
    this);
1326     m_textCtrlLabel5->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1327     m_textCtrlField5->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1328     m_buttonDel5->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel5), NULL,
    this);
1329     m_textCtrlLabel6->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1330     m_textCtrlField6->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1331     m_buttonDel6->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel6), NULL,
    this);
1332     m_textCtrlLabel7->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1333     m_textCtrlField7->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1334     m_buttonDel7->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel7), NULL,
    this);
1335     m_textCtrlLabel8->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1336     m_textCtrlField8->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1337     m_buttonDel8->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel8), NULL,
    this);
1338     m_textCtrlLabel9->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1339     m_textCtrlField9->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1340     m_buttonDel9->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel9), NULL,
    this);
1341     m_textCtrlLabel10->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1342     m_textCtrlField10->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1343     m_buttonDel10->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel10), NULL,
    this);
1344     m_textCtrlLabel11->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1345     m_textCtrlField11->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1346     m_buttonDel11->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel11), NULL,
    this);
1347     m_textCtrlLabel12->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1348     m_textCtrlField12->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1349     m_buttonDel12->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel12), NULL,
    this);
1350     m_textCtrlLabel13->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1351     m_textCtrlField13->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1352     m_buttonDel13->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel13), NULL,
    this);
1353     m_textCtrlLabel14->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1354     m_textCtrlField14->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
```

```
1355     m_buttonDel14->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel14), NULL,
     this);
1356     m_textCtrlLabel15->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1357     m_textCtrlField15->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1358     m_buttonDel15->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel15), NULL,
     this);
1359     m_textCtrlLabel16->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1360     m_textCtrlField16->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1361     m_buttonDel16->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel16), NULL,
     this);
1362     m_textCtrlLabel17->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1363     m_textCtrlField17->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1364     m_buttonDel17->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel17), NULL,
     this);
1365     m_textCtrlLabel18->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1366     m_textCtrlField18->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1367     m_buttonDel18->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel18), NULL,
     this);
1368     m_textCtrlLabel19->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1369     m_textCtrlField19->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1370     m_buttonDel19->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonDel19), NULL,
     this);
1371     m_buttonAddField->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonAddField),
     NULL, this);
1372     m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonSend), NULL,
     this);
1373     m_buttonPreview->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonPreview),
     NULL, this);
1374     m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditProductDialogBase::OnButtonCancel),
     NULL, this);
1375 }
1376
1377 CViewProductDialogBase::CViewProductDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos,
     const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
1378 {
1379 this->SetSizeHints(wxDefaultSize, wxDefaultSize);
1380 this->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_MENU));
1381
1382     wxBoxSizer* bSizer20;
1383     bSizer20 = new wxBoxSizer(wxVERTICAL);
1384
1385     wxBoxSizer* bSizer116;
1386     bSizer116 = new wxBoxSizer(wxHORIZONTAL);
1387
1388     m_htmlWinReviews = new wxHtmlWindow(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxHW_SCROLLBAR_AUTO);
1389     m_htmlWinReviews->Hide();
1390
1391     bSizer116->Add(m_htmlWinReviews, 1, wxALL|wxEXPAND, 5);
1392
1393     m_scrolledWindow = new wxScrolledWindow(this, wxID_ANY, wxDefaultPosition, wxDefaultSize,
     wxHSCROLL|wxTAB_TRAVERSAL|wxVSCROLL);
1394     m_scrolledWindow->SetScrollRate(5, 5);
1395     m_scrolledWindow->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_WINDOW));
1396
1397     wxBoxSizer* bSizer21;
1398     bSizer21 = new wxBoxSizer(wxVERTICAL);
1399
1400     m_richTextHeading = new wxRichTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,50),
     wxTE_READONLY|wxNO_BORDER);
1401     bSizer21->Add(m_richTextHeading, 0, wxEXPAND, 5);
1402
1403     m_staticTextInstructions = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Order Form instructions here\nmultiple
     lines\n1\n2\n3\n4\n5\n6"), wxDefaultPosition, wxDefaultSize, 0);
1404     m_staticTextInstructions->Wrap(-1);
1405     bSizer21->Add(m_staticTextInstructions, 0, wxALL|wxEXPAND, 5);
1406
1407     wxBoxSizer* bSizer25;
1408     bSizer25 = new wxBoxSizer(wxHORIZONTAL);
1409
1410     m_buttonSubmitForm = new wxButton(m_scrolledWindow, wxID_BUTTONSAMPLE, wxT("&Submit"), wxDefaultPosition, wxDefaultSize,
     0);
1411     bSizer25->Add(m_buttonSubmitForm, 0, wxALL, 5);
1412
1413     m_buttonCancelForm = new wxButton(m_scrolledWindow, wxID_CANCEL2, wxT("Cancel"), wxDefaultPosition, wxDefaultSize, 0);
1414     bSizer25->Add(m_buttonCancelForm, 0, wxALL, 5);
1415
1416     bSizer21->Add(bSizer25, 0, wxALIGN_CENTER_HORIZONTAL, 5);
1417
1418     m_scrolledWindow->SetSizer(bSizer21);
1419     m_scrolledWindow->Layout();
1420     bSizer21->Fit(m_scrolledWindow);
1421     bSizer116->Add(m_scrolledWindow, 1, wxEXPAND|wxALL, 5);
1422
1423     bSizer20->Add(bSizer116, 1, wxEXPAND, 5);
1424
1425     wxBoxSizer* bSizer26;
1426     bSizer26 = new wxBoxSizer(wxHORIZONTAL);
1427
1428     m_buttonBack = new wxButton(this, wxID_BUTTONBACK, wxT("< &Back  "), wxDefaultPosition, wxDefaultSize, 0);
```

```
1429     m_buttonBack->Enable(false);
1430     m_buttonBack->SetMinSize(wxSize(85,25));
1431
1432     bSizer26->Add(m_buttonBack, 0, wxALL, 5);
1433
1434     m_buttonNext = new wxButton(this, wxID_BUTTONNEXT, wxT("  &Next >"), wxDefaultPosition, wxDefaultSize, 0);
1435     m_buttonNext->SetMinSize(wxSize(85,25));
1436
1437     bSizer26->Add(m_buttonNext, 0, wxALL, 5);
1438
1439     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxDefaultSize, 0);
1440     m_buttonCancel->SetMinSize(wxSize(85,25));
1441
1442     bSizer26->Add(m_buttonCancel, 0, wxALL, 5);
1443
1444     bSizer20->Add(bSizer26, 0, wxALIGN_RIGHT, 5);
1445
1446 this->SetSizer(bSizer20);
1447 this->Layout();
1448
1449 // Connect Events
1450     m_buttonSubmitForm->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CViewProductDialogBase::OnButtonSubmitForm),
    NULL, this);
1451     m_buttonCancelForm->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CViewProductDialogBase::OnButtonCancelForm),
    NULL, this);
1452     m_buttonBack->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CViewProductDialogBase::OnButtonBack), NULL,
    this);
1453     m_buttonNext->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CViewProductDialogBase::OnButtonNext), NULL,
    this);
1454     m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CViewProductDialogBase::OnButtonCancel), NULL,
    this);
1455 }
1456
1457 CViewProductDialogBase::~CViewProductDialogBase()
1458 {
1459 // Disconnect Events
1460     m_buttonSubmitForm->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
    wxCommandEventHandler(CViewProductDialogBase::OnButtonSubmitForm), NULL, this);
1461     m_buttonCancelForm->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
    wxCommandEventHandler(CViewProductDialogBase::OnButtonCancelForm), NULL, this);
1462     m_buttonBack->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CViewProductDialogBase::OnButtonBack), NULL,
    this);
1463     m_buttonNext->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CViewProductDialogBase::OnButtonNext), NULL,
    this);
1464     m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CViewProductDialogBase::OnButtonCancel),
    NULL, this);
1465 }
1466
1467 CViewOrderDialogBase::CViewOrderDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos, const
    wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
1468 {
1469 this->SetSizeHints(wxDefaultSize, wxDefaultSize);
1470 this->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_MENU));
1471
1472     wxBoxSizer* bSizer20;
1473     bSizer20 = new wxBoxSizer(wxVERTICAL);
1474
1475     wxBoxSizer* bSizer116;
1476     bSizer116 = new wxBoxSizer(wxHORIZONTAL);
1477
1478     m_htmlWin = new wxHtmlWindow(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxHW_SCROLLBAR_AUTO);
1479     bSizer116->Add(m_htmlWin, 1, wxALL|wxEXPAND, 5);
1480
1481     bSizer20->Add(bSizer116, 1, wxEXPAND, 5);
1482
1483     wxBoxSizer* bSizer26;
1484     bSizer26 = new wxBoxSizer(wxHORIZONTAL);
1485
1486     m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxDefaultSize, 0);
1487     m_buttonOK->SetMinSize(wxSize(85,25));
1488
1489     bSizer26->Add(m_buttonOK, 0, wxALL, 5);
1490
1491     bSizer20->Add(bSizer26, 0, wxALIGN_RIGHT, 5);
1492
1493 this->SetSizer(bSizer20);
1494 this->Layout();
1495
1496 // Connect Events
1497     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CViewOrderDialogBase::OnButtonOK), NULL, this);
1498 }
1499
1500 CViewOrderDialogBase::~CViewOrderDialogBase()
1501 {
1502 // Disconnect Events
1503     m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CViewOrderDialogBase::OnButtonOK), NULL, this);
1504 }
1505
```

```
CEditReviewDialogBase::CEditReviewDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos, const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
{
this->SetSizeHints(wxDefaultSize, wxDefaultSize);
this->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_MENU));

    wxBoxSizer* bSizer112;
    bSizer112 = new wxBoxSizer(wxVERTICAL);


    bSizer112->Add(0, 3, 0, 0, 5);

    m_staticTextSeller = new wxStaticText(this, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
    m_staticTextSeller->Wrap(-1);
    bSizer112->Add(m_staticTextSeller, 0, wxALL|wxEXPAND, 5);


    bSizer112->Add(0, 3, 0, 0, 5);

    m_staticText110 = new wxStaticText(this, wxID_ANY, wxT("Rating"), wxDefaultPosition, wxDefaultSize, 0);
    m_staticText110->Wrap(-1);
    bSizer112->Add(m_staticText110, 0, wxTOP|wxRIGHT|wxLEFT, 5);

    wxString m_choiceStarsChoices[] = { wxT(" 1 star"), wxT(" 2 stars"), wxT(" 3 stars"), wxT(" 4 stars"), wxT(" 5 stars") };
int m_choiceStarsNChoices = sizeof(m_choiceStarsChoices) / sizeof(wxString);
    m_choiceStars = new wxChoice(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, m_choiceStarsNChoices, m_choiceStarsChoices, 0);
    m_choiceStars->SetSelection(0);
    bSizer112->Add(m_choiceStars, 0, wxALL, 5);

    m_staticText43 = new wxStaticText(this, wxID_ANY, wxT("Review"), wxDefaultPosition, wxDefaultSize, 0);
    m_staticText43->Wrap(-1);
    bSizer112->Add(m_staticText43, 0, wxTOP|wxRIGHT|wxLEFT, 5);

    m_textCtrlReview = new wxTextCtrl(this, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, wxTE_MULTILINE);
    bSizer112->Add(m_textCtrlReview, 1, wxALL|wxEXPAND, 5);

    wxBoxSizer* bSizer113;
    bSizer113 = new wxBoxSizer(wxHORIZONTAL);

    m_buttonSubmit = new wxButton(this, wxID_SUBMIT, wxT("&Submit"), wxDefaultPosition, wxDefaultSize, 0);
    m_buttonSubmit->SetMinSize(wxSize(85,25));

    bSizer113->Add(m_buttonSubmit, 0, wxALL, 5);

    m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxDefaultSize, 0);
    m_buttonCancel->SetMinSize(wxSize(85,25));

    bSizer113->Add(m_buttonCancel, 0, wxALL, 5);

    bSizer112->Add(bSizer113, 0, wxALIGN_RIGHT, 5);

this->SetSizer(bSizer112);
this->Layout();

// Connect Events
    m_textCtrlReview->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditReviewDialogBase::OnKeyDown), NULL, this);
    m_buttonSubmit->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditReviewDialogBase::OnButtonSubmit), NULL, this);
    m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditReviewDialogBase::OnButtonCancel), NULL, this);
}

CEditReviewDialogBase::~CEditReviewDialogBase()
{
// Disconnect Events
    m_textCtrlReview->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditReviewDialogBase::OnKeyDown), NULL, this);
    m_buttonSubmit->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditReviewDialogBase::OnButtonSubmit), NULL, this);
    m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CEditReviewDialogBase::OnButtonCancel), NULL, this);
}

CPokerLobbyDialogBase::CPokerLobbyDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos, const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
{
this->SetSizeHints(wxDefaultSize, wxDefaultSize);
this->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_BTNFACE));

    wxBoxSizer* bSizer156;
    bSizer156 = new wxBoxSizer(wxHORIZONTAL);

    m_treeCtrl = new wxTreeCtrl(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTR_HAS_BUTTONS|wxTR_HIDE_ROOT|wxTR_LINES_AT_ROOT);
    m_treeCtrl->SetMinSize(wxSize(130,-1));

    bSizer156->Add(m_treeCtrl, 0, wxEXPAND|wxTOP|wxBOTTOM|wxLEFT, 5);

    wxBoxSizer* bSizer172;
```

```
    bSizer172 = new wxBoxSizer(wxVERTICAL);

    m_listCtrl = new wxListCtrl(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxLC_NO_SORT_HEADER|wxLC_REPORT);
    bSizer172->Add(m_listCtrl, 1, wxEXPAND|wxALL, 5);

    m_buttonNewTable = new wxButton(this, wxID_OPENNEWTABLE, wxT("&Open New Table"), wxDefaultPosition, wxDefaultSize, 0);
    bSizer172->Add(m_buttonNewTable, 0, wxALL, 5);

    bSizer156->Add(bSizer172, 1, wxEXPAND, 5);

this->SetSizer(bSizer156);
this->Layout();

// Connect Events
    m_treeCtrl->Connect(wxEVT_COMMAND_TREE_SEL_CHANGED, wxTreeEventHandler(CPokerLobbyDialogBase::OnTreeSelChanged), NULL, this);
    m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED, wxListEventHandler(CPokerLobbyDialogBase::OnListItemActivated), NULL, this);
    m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_SELECTED, wxListEventHandler(CPokerLobbyDialogBase::OnListItemSelected), NULL, this);
    m_buttonNewTable->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CPokerLobbyDialogBase::OnButtonNewTable), NULL, this);
}

CPokerLobbyDialogBase::~CPokerLobbyDialogBase()
{
// Disconnect Events
    m_treeCtrl->Disconnect(wxEVT_COMMAND_TREE_SEL_CHANGED, wxTreeEventHandler(CPokerLobbyDialogBase::OnTreeSelChanged), NULL, this);
    m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED, wxListEventHandler(CPokerLobbyDialogBase::OnListItemActivated), NULL, this);
    m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_SELECTED, wxListEventHandler(CPokerLobbyDialogBase::OnListItemSelected), NULL, this);
    m_buttonNewTable->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CPokerLobbyDialogBase::OnButtonNewTable), NULL, this);
}

CPokerDialogBase::CPokerDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint& pos, const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
{
this->SetSizeHints(wxDefaultSize, wxDefaultSize);

    wxBoxSizer* bSizer174;
    bSizer174 = new wxBoxSizer(wxVERTICAL);

    m_checkSitOut = new wxCheckBox(this, wxID_ANY, wxT("Deal Me Out"), wxDefaultPosition, wxDefaultSize, 0);

    bSizer174->Add(m_checkSitOut, 0, wxALL, 5);

    m_buttonDealHand = new wxButton(this, wxID_DEALHAND, wxT("&Deal Hand"), wxDefaultPosition, wxSize(150,25), 0);
    bSizer174->Add(m_buttonDealHand, 0, wxALL, 5);

    m_buttonFold = new wxButton(this, wxID_FOLD, wxT("&Fold"), wxDefaultPosition, wxSize(80,25), 0);
    bSizer174->Add(m_buttonFold, 0, wxALL, 5);

    m_buttonCall = new wxButton(this, wxID_CALL, wxT("&Call"), wxDefaultPosition, wxSize(80,25), 0);
    bSizer174->Add(m_buttonCall, 0, wxALL, 5);

    m_buttonRaise = new wxButton(this, wxID_RAISE, wxT("&Raise"), wxDefaultPosition, wxSize(80,25), 0);
    bSizer174->Add(m_buttonRaise, 0, wxALL, 5);

    m_buttonLeaveTable = new wxButton(this, wxID_LEAVETABLE, wxT("&Leave Table"), wxDefaultPosition, wxSize(90,25), 0);
    bSizer174->Add(m_buttonLeaveTable, 0, wxALL, 5);

    m_textDitchPlayer = new wxTextCtrl(this, wxID_DITCHPLAYER, wxEmptyString, wxDefaultPosition, wxSize(45,-1), wxTE_PROCESS_ENTER);
    bSizer174->Add(m_textDitchPlayer, 0, wxALL, 5);

    m_checkPreFold = new wxCheckBox(this, wxID_ANY, wxT("FOLD"), wxDefaultPosition, wxSize(100,-1), 0);

    bSizer174->Add(m_checkPreFold, 0, wxALL, 5);

    m_checkPreCall = new wxCheckBox(this, wxID_ANY, wxT("CALL"), wxDefaultPosition, wxSize(100,-1), 0);

    bSizer174->Add(m_checkPreCall, 0, wxALL, 5);

    m_checkPreCallAny = new wxCheckBox(this, wxID_ANY, wxT("CALL ANY"), wxDefaultPosition, wxSize(100,-1), 0);

    bSizer174->Add(m_checkPreCallAny, 0, wxALL, 5);

    m_checkPreRaise = new wxCheckBox(this, wxID_ANY, wxT("RAISE"), wxDefaultPosition, wxSize(100,-1), 0);

    bSizer174->Add(m_checkPreRaise, 0, wxALL, 5);

    m_checkPreRaiseAny = new wxCheckBox(this, wxID_ANY, wxT("RAISE ANY"), wxDefaultPosition, wxSize(100,-1), 0);

    bSizer174->Add(m_checkPreRaiseAny, 0, wxALL, 5);

this->SetSizer(bSizer174);
```

```
this->Layout();
    m_statusBar = this->CreateStatusBar(1, wxST_SIZEGRIP, wxID_ANY);

// Connect Events
this->Connect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CPokerDialogBase::OnClose));
this->Connect(wxEVT_LEFT_DOWN, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_LEFT_UP, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_MIDDLE_DOWN, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_MIDDLE_UP, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_RIGHT_DOWN, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_RIGHT_UP, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_MOTION, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_LEFT_DCLICK, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_MIDDLE_DCLICK, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_RIGHT_DCLICK, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_LEAVE_WINDOW, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_ENTER_WINDOW, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_MOUSEWHEEL, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Connect(wxEVT_PAINT, wxPaintEventHandler(CPokerDialogBase::OnPaint));
this->Connect(wxEVT_SIZE, wxSizeEventHandler(CPokerDialogBase::OnSize));
    m_checkSitOut->Connect(wxEVT_COMMAND_CHECKBOX_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnCheckSitOut), NULL, this);
    m_buttonDealHand->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnButtonDealHand), NULL,
this);
    m_buttonFold->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnButtonFold), NULL, this);
    m_buttonCall->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnButtonCall), NULL, this);
    m_buttonRaise->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnButtonRaise), NULL, this);
    m_buttonLeaveTable->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnButtonLeaveTable), NULL,
this);
    m_textDitchPlayer->Connect(wxEVT_COMMAND_TEXT_ENTER, wxCommandEventHandler(CPokerDialogBase::OnDitchPlayer), NULL, this);
    m_checkPreFold->Connect(wxEVT_COMMAND_CHECKBOX_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnCheckPreFold), NULL,
this);
    m_checkPreCall->Connect(wxEVT_COMMAND_CHECKBOX_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnCheckPreCall), NULL,
this);
    m_checkPreCallAny->Connect(wxEVT_COMMAND_CHECKBOX_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnCheckPreCallAny), NULL,
this);
    m_checkPreRaise->Connect(wxEVT_COMMAND_CHECKBOX_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnCheckPreRaise), NULL,
this);
    m_checkPreRaiseAny->Connect(wxEVT_COMMAND_CHECKBOX_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnCheckPreRaiseAny),
NULL, this);
}

CPokerDialogBase::~CPokerDialogBase()
{
// Disconnect Events
this->Disconnect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CPokerDialogBase::OnClose));
this->Disconnect(wxEVT_LEFT_DOWN, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_LEFT_UP, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_MIDDLE_DOWN, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_MIDDLE_UP, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_RIGHT_DOWN, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_RIGHT_UP, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_MOTION, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_LEFT_DCLICK, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_MIDDLE_DCLICK, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_RIGHT_DCLICK, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_LEAVE_WINDOW, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_ENTER_WINDOW, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_MOUSEWHEEL, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
this->Disconnect(wxEVT_PAINT, wxPaintEventHandler(CPokerDialogBase::OnPaint));
this->Disconnect(wxEVT_SIZE, wxSizeEventHandler(CPokerDialogBase::OnSize));
    m_checkSitOut->Disconnect(wxEVT_COMMAND_CHECKBOX_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnCheckSitOut), NULL,
this);
    m_buttonDealHand->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnButtonDealHand), NULL,
this);
    m_buttonFold->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnButtonFold), NULL, this);
    m_buttonCall->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnButtonCall), NULL, this);
    m_buttonRaise->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnButtonRaise), NULL, this);
    m_buttonLeaveTable->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnButtonLeaveTable),
NULL, this);
    m_textDitchPlayer->Disconnect(wxEVT_COMMAND_TEXT_ENTER, wxCommandEventHandler(CPokerDialogBase::OnDitchPlayer), NULL, this);
    m_checkPreFold->Disconnect(wxEVT_COMMAND_CHECKBOX_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnCheckPreFold), NULL,
this);
    m_checkPreCall->Disconnect(wxEVT_COMMAND_CHECKBOX_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnCheckPreCall), NULL,
this);
    m_checkPreCallAny->Disconnect(wxEVT_COMMAND_CHECKBOX_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnCheckPreCallAny),
NULL, this);
    m_checkPreRaise->Disconnect(wxEVT_COMMAND_CHECKBOX_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnCheckPreRaise), NULL,
this);
    m_checkPreRaiseAny->Disconnect(wxEVT_COMMAND_CHECKBOX_CLICKED, wxCommandEventHandler(CPokerDialogBase::OnCheckPreRaiseAny),
NULL, this);
}

CGetTextFromUserDialogBase::CGetTextFromUserDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint&
pos, const wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
{
this->SetSizeHints(wxDefaultSize, wxDefaultSize);

    wxBoxSizer* bSizer79;
    bSizer79 = new wxBoxSizer(wxVERTICAL);
```

```
1739
1740    wxBoxSizer* bSizer81;
1741    bSizer81 = new wxBoxSizer(wxVERTICAL);
1742
1743
1744    bSizer81->Add(0, 0, 1, wxEXPAND, 5);
1745
1746    m_staticTextMessage1 = new wxStaticText(this, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
1747    m_staticTextMessage1->Wrap(-1);
1748    bSizer81->Add(m_staticTextMessage1, 0, wxTOP|wxRIGHT|wxLEFT, 5);
1749
1750    m_textCtrl1 = new wxTextCtrl(this, wxID_TEXTCTRL, wxEmptyString, wxDefaultPosition, wxDefaultSize, wxTE_PROCESS_ENTER);
1751    bSizer81->Add(m_textCtrl1, 0, wxALL|wxEXPAND|wxALIGN_CENTER_HORIZONTAL, 5);
1752
1753    m_staticTextMessage2 = new wxStaticText(this, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize, 0);
1754    m_staticTextMessage2->Wrap(-1);
1755    m_staticTextMessage2->Hide();
1756
1757    bSizer81->Add(m_staticTextMessage2, 0, wxTOP|wxRIGHT|wxLEFT, 5);
1758
1759    m_textCtrl2 = new wxTextCtrl(this, wxID_TEXTCTRL, wxEmptyString, wxDefaultPosition, wxDefaultSize, wxTE_PROCESS_ENTER);
1760    m_textCtrl2->Hide();
1761
1762    bSizer81->Add(m_textCtrl2, 0, wxALL|wxEXPAND|wxALIGN_CENTER_HORIZONTAL, 5);
1763
1764
1765    bSizer81->Add(0, 0, 1, wxEXPAND, 5);
1766
1767    bSizer79->Add(bSizer81, 1, wxEXPAND|wxALL, 10);
1768
1769    wxBoxSizer* bSizer80;
1770    bSizer80 = new wxBoxSizer(wxHORIZONTAL);
1771
1772
1773    bSizer80->Add(0, 0, 1, wxEXPAND, 5);
1774
1775    m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxSize(-1,-1), 0);
1776    m_buttonOK->SetMinSize(wxSize(85,25));
1777
1778    bSizer80->Add(m_buttonOK, 0, wxALL, 5);
1779
1780    m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxDefaultSize, 0);
1781    m_buttonCancel->SetMinSize(wxSize(85,25));
1782
1783    bSizer80->Add(m_buttonCancel, 0, wxALL, 5);
1784
1785    bSizer79->Add(bSizer80, 0, wxEXPAND, 5);
1786
1787 this->SetSizer(bSizer79);
1788 this->Layout();
1789
1790 // Connect Events
1791 this->Connect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CGetTextFromUserDialogBase::OnClose));
1792    m_textCtrl1->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CGetTextFromUserDialogBase::OnKeyDown), NULL, this);
1793    m_textCtrl2->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CGetTextFromUserDialogBase::OnKeyDown), NULL, this);
1794    m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CGetTextFromUserDialogBase::OnButtonOK), NULL,
    this);
1795    m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CGetTextFromUserDialogBase::OnButtonCancel),
    NULL, this);
1796 }
1797
1798 CGetTextFromUserDialogBase::~CGetTextFromUserDialogBase()
1799 {
1800 // Disconnect Events
1801 this->Disconnect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CGetTextFromUserDialogBase::OnClose));
1802    m_textCtrl1->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CGetTextFromUserDialogBase::OnKeyDown), NULL, this);
1803    m_textCtrl2->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CGetTextFromUserDialogBase::OnKeyDown), NULL, this);
1804    m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CGetTextFromUserDialogBase::OnButtonOK), NULL,
    this);
1805    m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED, wxCommandEventHandler(CGetTextFromUserDialogBase::OnButtonCancel),
    NULL, this);
1806 }
1807
```

SUMMARY

The auto-generated nature of this file makes it easy to dismiss, but the widgets it constructs are a blueprint of Satoshi's ambitions. The hidden marketplace tabs, the product editor with 20 form fields, the poker dialog, the escrow list — these are not stubs or placeholders. They are fully laid-out interfaces for features Satoshi designed in wxFormBuilder's visual editor, generated into code, and shipped in the v0.1.0 binary. The gap between what was visible and what was built reveals a roadmap that was never publicly announced.

The wxWidgets dependency would prove to be one of Bitcoin's longest-standing technical debts. The framework was adequate for a Windows prototype but made cross-platform builds difficult and the codebase harder to maintain. Bitcoin Core finally migrated to Qt in version 0.5 (November 2011), nearly three years after the initial release.[41] The Qt interface, designed by Wladimir van der Laan, was cleaner and more portable — but it dropped the marketplace, poker, and escrow interfaces entirely. Those features existed only here, in Satoshi's original wxFormBuilder project file (`uiproject.fbp`), preserved as archaeological evidence of what Bitcoin was meant to become.

The IP-based payment dialog (`CSendDialogBase`) deserves special attention. Satoshi's original design treated direct IP-to-IP payments as the primary flow and address-based payments as the fallback. This is the opposite of how Bitcoin evolved. The IP payment protocol lacked authentication and was vulnerable to man-in-the-middle attacks, so it was deprecated and eventually removed. BIP-0070 (Payment Protocol, 2013) later attempted to bring back interactive payments with proper authentication, but it too was eventually deprecated. Satoshi's instinct — that payments should be interactive — may yet prove correct, as Lightning Network invoices now serve a similar purpose.

KEY TERMS

generated code

Source written by a tool rather than a person; uibase.cpp opens with PLEASE DO "NOT" EDIT THIS FILE! because wxFormBuilder overwrites it whenever the design is regenerated.

Connect()

The wxWidgets call that binds an event on a widget to a handler method at run time; uibase.cpp makes 200 such calls, each undone by a matching Disconnect() in a destructor.

wxNotebook

A tabbed container widget; in v0.1.0 the main window's notebook holds a single All Transactions page containing the transaction list.

report mode

The wxLC_REPORT style that makes a wxListCtrl show rows under column headings, like a spreadsheet; the main transaction list combines it with descending sort order.

wxHtmlWindow

A wxWidgets widget that renders simple HTML; the Transaction Details dialog uses one so ui.cpp can lay out a transaction's status, amounts, and addresses as markup.

wxStaticBitmap

A non-interactive widget that simply displays an image; the Send Coins dialog uses a 16-pixel one to show the check icon beside a valid address.

hidden widget

A control the generated code creates and then immediately hides with Hide(), as with the main window's transaction filter and the four marketplace list panels.

SOURCES

1. Qt Company / Nokia, *Qt licensed under LGPL* (January 2009). Enabled Bitcoin Core's later migration from wxWidgets to Qt. https://blog.qt.io/blog/2009/01/14/nokia-qt-lgpl/
2. wxWidgets Project, *wxWidgets Licence* (2005). https://www.wxwidgets.org/about/licence/
3. Satoshi Nakamoto, *Bitcoin v0.1 released (Cryptography Mailing List / BitcoinTalk)* (January 9, 2009). Announced Windows-only for the first release. https://satoshi.nakamotoinstitute.org/emails/cryptography/15/
4. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
5. Satoshi Nakamoto, *Bitcoin P2P e-cash paper (Cryptography Mailing List)* (October 31, 2008). Initial announcement of the Bitcoin whitepaper. https://satoshi.nakamotoinstitute.org/emails/cryptography/1/
6. Satoshi Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (October 2008). https://bitcoin.org/bitcoin.pdf

7. Satoshi Nakamoto, *Bitcoin P2P e-cash paper (Cryptography Mailing List)* (November 2008). On specialists with server farms dominating proof-of-work. https://satoshi.nakamotoinstitute.org/emails/cryptography/2/

uibase.h

721 lines CPP

GUI BLUEPRINT

This header file is the architectural drawing of Bitcoin's user interface — the class declarations, widget pointers, and virtual event handlers that define every window the user could see. While `uibase.cpp` constructs the widgets, this file declares what exists and how it can be extended. Like its companion, it was auto-generated by wxFormBuilder and carries the same `PLEASE DO "NOT" EDIT THIS FILE!` warning.

The file declares sixteen GUI classes, and their names read like a feature list for a product far more ambitious than a simple payment tool: `CMainFrameBase`, `CTxDetailsDialogBase`, `COptionsDialogBase`, `CAboutDialogBase`, `CSendDialogBase`, `CSendingDialogBase`, `CYourAddressDialogBase`, `CAddressBookDialogBase`, `CProductsDialogBase`, `CEditProductDialogBase`, `CViewProductDialogBase`, `CViewOrderDialogBase`, `CEditReviewDialogBase`, `CPokerLobbyDialogBase`, `CPokerDialogBase`, and the utility `CGetTextFromUserDialogBase`. The seven marketplace and poker classes, from `CProductsDialogBase` through `CPokerDialogBase`, were never exposed to users in any public release.

The wxWidgets includes at the top of this file — 28 headers from `<wx/string.h>` through `<wx/checkbox.h>` — reveal the full palette of GUI components Satoshi used. Rich text controls, HTML windows, tree views, scrolled windows, notebooks, and combo boxes are all present. This is not a minimal toolkit selection; it is the full wxWidgets arsenal deployed for an application that Satoshi clearly intended to grow into a comprehensive platform.

Architecturally, each class follows the same pattern: `protected` member variables for widgets, `protected virtual` event handlers that default to `event.Skip()` (pass-through), and `public` constructors with default window titles and sizes. The derived classes in `ui.h` and `ui.cpp` override only the event handlers they need. This is the **Template Method pattern** — the base class defines the skeleton of the UI, subclasses fill in the behavior.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

///////////////////////////////////////////////////////////////////////////
// C++ code generated with wxFormBuilder (version Apr 16 2008)
// http://www.wxformbuilder.org/
//
// PLEASE DO "NOT" EDIT THIS FILE!
///////////////////////////////////////////////////////////////////////////

#ifndef __uibase__
#define __uibase__

```

Twenty-eight wxWidgets includes — a comprehensive selection covering strings, bitmaps, menus, toolbars, status bars, text controls, buttons, sizers (layout managers), panels, notebooks, frames, HTML windows, dialogs, combo boxes, scrolled windows, rich text, tree controls, and checkboxes. Each include pulls in a specific widget family. The breadth suggests Satoshi explored wxWidgets thoroughly before designing the interface.

```
#include <wx/string.h>
#include <wx/bitmap.h>
#include <wx/image.h>
#include <wx/icon.h>
#include <wx/menu.h>
#include <wx/gdicmn.h>
#include <wx/font.h>
#include <wx/colour.h>
#include <wx/settings.h>
#include <wx/toolbar.h>
#include <wx/statusbr.h>
#include <wx/stattext.h>
#include <wx/textctrl.h>
#include <wx/button.h>
#include <wx/sizer.h>
#include <wx/panel.h>
#include <wx/choice.h>
#include <wx/listctrl.h>
#include <wx/notebook.h>
#include <wx/frame.h>
#include <wx/html/htmlwin.h>
#include <wx/dialog.h>
#include <wx/statbmp.h>
#include <wx/combobox.h>
#include <wx/scrolwin.h>
#include <wx/richtext/richtextctrl.h>
#include <wx/treectrl.h>
#include <wx/checkbox.h>

///////////////////////////////////////////////////////////////////////////

```

Widget ID constants, starting at 1000. `wxID_MAINFRAME` through `wxID_TEXTCTRL` — 52 unique identifiers. Notable entries: `wxID_OPTIONSGENERATEBITCOINS` (1001) for the mining toggle, `wxID_TRANSACTIONFEE` (1007) for the fee setting, IDs for 20 delete buttons (`wxID_DEL0` through `wxID_DEL19`) for the product editor form fields, and `wxID_OPENNEWTABLE` through `wxID_DITCHPLAYER` for the poker game.

```
#define wxID_MAINFRAME 1000
#define wxID_OPTIONSGENERATEBITCOINS 1001
#define wxID_BUTTONSEND 1002
#define wxID_BUTTONRECEIVE 1003
#define wxID_TEXTCTRLADDRESS 1004
#define wxID_BUTTONCOPY 1005
#define wxID_BUTTONCHANGE 1006
#define wxID_TRANSACTIONFEE 1007
#define wxID_TEXTCTRLPAYTO 1008
#define wxID_BUTTONPASTE 1009
#define wxID_BUTTONADDRESSBOOK 1010
#define wxID_TEXTCTRLAMOUNT 1011
#define wxID_CHOICETRANSFERTYPE 1012
#define wxID_LISTCTRL 1013
#define wxID_BUTTONRENAME 1014
#define wxID_BUTTONNEW 1015
```

```
#define wxID_BUTTONEDIT 1016
#define wxID_BUTTONDELETE 1017
#define wxID_DEL0 1018
#define wxID_DEL1 1019
#define wxID_DEL2 1020
#define wxID_DEL3 1021
#define wxID_DEL4 1022
#define wxID_DEL5 1023
#define wxID_DEL6 1024
#define wxID_DEL7 1025
#define wxID_DEL8 1026
#define wxID_DEL9 1027
#define wxID_DEL10 1028
#define wxID_DEL11 1029
#define wxID_DEL12 1030
#define wxID_DEL13 1031
#define wxID_DEL14 1032
#define wxID_DEL15 1033
#define wxID_DEL16 1034
#define wxID_DEL17 1035
#define wxID_DEL18 1036
#define wxID_DEL19 1037
#define wxID_BUTTONPREVIEW 1038
#define wxID_BUTTONSAMPLE 1039
#define wxID_CANCEL2 1040
#define wxID_BUTTONBACK 1041
#define wxID_BUTTONNEXT 1042
#define wxID_SUBMIT 1043
```

Poker game IDs: `wxID_OPENNEWTABLE` (1044), `wxID_DEALHAND` (1045), `wxID_FOLD` (1046), `wxID_CALL` (1047), `wxID_RAISE` (1048), `wxID_LEAVETABLE` (1049), `wxID_DITCHPLAYER` (1050). A complete poker vocabulary. Satoshi apparently planned a peer-to-peer poker game built directly into the Bitcoin client — a natural application for a trustless digital cash system. Online poker was a massive industry in 2008 (until Black Friday in 2011), and Bitcoin would have solved the payment processing problem that plagued poker sites.

```
#define wxID_OPENNEWTABLE 1044
#define wxID_DEALHAND 1045
#define wxID_FOLD 1046
#define wxID_CALL 1047
#define wxID_RAISE 1048
#define wxID_LEAVETABLE 1049
#define wxID_DITCHPLAYER 1050
#define wxID_TEXTCTRL 1051

///////////////////////////////////////////////////////////////////////////
/// Class CMainFrameBase
///////////////////////////////////////////////////////////////////////////
```

`CMainFrameBase` — the primary window class, inheriting from `wxFrame`. Default title: `'Bitcoin'`, default size: 705×484 pixels. The `protected` section declares pointers to every widget in the main window: menu bar, three menus, toolbar, status bar, address text field, balance display, transaction filter, notebook, and five list controls (one visible, four hidden for the marketplace).

```
class CMainFrameBase : public wxFrame
{
private:

protected:
    wxMenuBar* m_menubar;
    wxMenu* m_menuFile;
    wxMenu* m_menuHelp;
    wxToolBar* m_toolBar;
    wxStatusBar* m_statusBar;

    wxStaticText* m_staticText32;
    wxTextCtrl* m_textCtrlAddress;
    wxButton* m_buttonCopy;
    wxButton* m_button91;

    wxPanel* m_panel14;
    wxStaticText* m_staticText41;
    wxStaticText* m_staticTextBalance;

    wxChoice* m_choiceFilter;
```

```
    wxNotebook* m_notebook;
    wxPanel* m_panel7;
    wxPanel* m_panel9;
    wxPanel* m_panel8;
    wxPanel* m_panel10;
    wxPanel* m_panel11;

```

Virtual event handlers — twenty-one methods that all default to `event.Skip()`, meaning 'pass this event to the next handler in the chain.' The derived `CMainFrame` class in `ui.h` overrides these with actual logic: `OnClose` saves wallet state, `OnMenuOptionsGenerate` starts/stops mining, `OnButtonSend` opens the payment dialog. The `virtual` keyword is what makes the Template Method pattern work — the base class wires events to these methods, the derived class provides the implementation.

```
// Virtual event handlers, overide them in your derived class
virtual void OnClose(wxCloseEvent& event){ event.Skip(); }
virtual void OnIdle(wxIdleEvent& event){ event.Skip(); }
virtual void OnMouseEvents(wxMouseEvent& event){ event.Skip(); }
virtual void OnPaint(wxPaintEvent& event){ event.Skip(); }
virtual void OnMenuFileExit(wxCommandEvent& event){ event.Skip(); }
virtual void OnMenuOptionsGenerate(wxCommandEvent& event){ event.Skip(); }
virtual void OnMenuOptionsOptions(wxCommandEvent& event){ event.Skip(); }
virtual void OnMenuHelpAbout(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonSend(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonAddressBook(wxCommandEvent& event){ event.Skip(); }
virtual void OnKeyDown(wxKeyEvent& event){ event.Skip(); }
virtual void OnMouseEventsAddress(wxMouseEvent& event){ event.Skip(); }
virtual void OnSetFocusAddress(wxFocusEvent& event){ event.Skip(); }
virtual void OnButtonCopy(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonChange(wxCommandEvent& event){ event.Skip(); }
virtual void OnListColBeginDrag(wxListEvent& event){ event.Skip(); }
virtual void OnListItemActivatedAllTransactions(wxListEvent& event){ event.Skip(); }
virtual void OnPaintListCtrl(wxPaintEvent& event){ event.Skip(); }
virtual void OnListItemActivatedOrdersSent(wxListEvent& event){ event.Skip(); }
virtual void OnListItemActivatedProductsSent(wxListEvent& event){ event.Skip(); }
virtual void OnListItemActivatedOrdersReceived(wxListEvent& event){ event.Skip(); }


```

The `public` section exposes `m_menuOptions` and the five list controls. Making `m_menuOptions` public allows the derived class to dynamically add or modify menu items — specifically, the mining checkbox needs to be checked/unchecked from outside the base class. The list controls are public so the derived class can populate them with transaction data, which requires direct access to insert and update rows.

```
public:
    wxMenu* m_menuOptions;
    wxListCtrl* m_listCtrl;
    wxListCtrl* m_listCtrlEscrows;
    wxListCtrl* m_listCtrlOrdersSent;
    wxListCtrl* m_listCtrlProductsSent;
    wxListCtrl* m_listCtrlOrdersReceived;
CMainFrameBase(wxWindow* parent, wxWindowID id = wxID_MAINFRAME, const wxString& title = wxT("Bitcoin"), const wxPoint& pos =
wxDefaultPosition, const wxSize& size = wxSize(705,484), long style = wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER|wxTAB_TRAVERSAL);
~CMainFrameBase();

};

///////////////////////////////////////////////////////////////////////////
/// Class CTxDetailsDialogBase
///////////////////////////////////////////////////////////////////////////
```

`CTxDetailsDialogBase` — transaction detail viewer using `wxHtmlWindow`. Size: 620×450, title: `'Transaction Details'`. The HTML window allowed Satoshi to render transaction details with formatted tables, bold labels, and color-coded status — far richer than plain text. The single `OnButtonOK` handler closes the dialog.

```
class CTxDetailsDialogBase : public wxDialog
{
```

```
171 private:
172
173 protected:
174     wxHtmlWindow* m_htmlWin;
175     wxButton* m_buttonOK;
176
177 // Virtual event handlers, overide them in your derived class
178 virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
179
180
181 public:
182 CTxDetailsDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Transaction Details"), const
    wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(620,450), long style = wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER);
183 ~CTxDetailsDialogBase();
184
185 };
186
187 ///////////////////////////////////////////////////////////////////////////
188 /// Class COptionsDialogBase
189 ///////////////////////////////////////////////////////////////////////////
```

`COptionsDialogBase` — the settings dialog. Size: 500×261, remarkably small because v0.1.0 had exactly one user-configurable option: the transaction fee. The `OnKillFocusTransactionFee` handler fires when the user clicks away from the fee field — used by the derived class to validate and format the entered amount.

```
190 class COptionsDialogBase : public wxDialog
191 {
192 private:
193
194 protected:
195
196     wxStaticText* m_staticText32;
197     wxStaticText* m_staticText31;
198     wxTextCtrl* m_textCtrlTransactionFee;
199     wxButton* m_buttonOK;
200     wxButton* m_buttonCancel;
201
202 // Virtual event handlers, overide them in your derived class
203 virtual void OnKillFocusTransactionFee(wxFocusEvent& event){ event.Skip(); }
204 virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
205 virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }
206
207
208 public:
209 COptionsDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Options"), const wxPoint& pos =
    wxDefaultPosition, const wxSize& size = wxSize(500,261), long style = wxDEFAULT_DIALOG_STYLE);
210 ~COptionsDialogBase();
211
212 };
213
214 ///////////////////////////////////////////////////////////////////////////
215 /// Class CAboutDialogBase
216 ///////////////////////////////////////////////////////////////////////////
217 class CAboutDialogBase : public wxDialog
218 {
219 private:
220
221 protected:
222
223
224     wxStaticText* m_staticText40;
225
226     wxStaticText* m_staticTextMain;
227
228
229     wxButton* m_buttonOK;
230
231 // Virtual event handlers, overide them in your derived class
232 virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
233
234
235 public:
236     wxStaticText* m_staticTextVersion;
237 CAboutDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("About Bitcoin"), const wxPoint& pos =
    wxDefaultPosition, const wxSize& size = wxSize(507,298), long style = wxDEFAULT_DIALOG_STYLE);
238 ~CAboutDialogBase();
239
240 };
241
242 ///////////////////////////////////////////////////////////////////////////
243 /// Class CSendDialogBase
244 ///////////////////////////////////////////////////////////////////////////
```

`CSendDialogBase` — the payment dialog. Size: 675×338. The widget declarations tell the story of v0.1.0's dual payment modes: `m_textCtrlAddress` accepts either an IP address or Bitcoin address, `m_textCtrlAmount` has a 20-character max length, `m_choiceTransferType` offers transfer type selection (only 'Standard' in v0.1.0), and `m_textCtrlMessage` allows attaching a message (only works with IP-based payments). The `m_bitmapCheckMark` provides visual feedback when the address is valid.

```
245 class CSendDialogBase : public wxDialog
246 {
247 private:
248
249 protected:
250
251
252     wxStaticText* m_staticText14;
253
254     wxStaticBitmap* m_bitmapCheckMark;
255     wxStaticText* m_staticText36;
256     wxTextCtrl* m_textCtrlAddress;
257     wxButton* m_buttonPaste;
258     wxButton* m_buttonAddress;
259     wxStaticText* m_staticText19;
260     wxTextCtrl* m_textCtrlAmount;
261     wxStaticText* m_staticText20;
262     wxChoice* m_choiceTransferType;
263     wxStaticText* m_staticTextFrom;
264     wxTextCtrl* m_textCtrlFrom;
265     wxStaticText* m_staticTextMessage;
266     wxTextCtrl* m_textCtrlMessage;
267
268     wxButton* m_buttonSend;
269     wxButton* m_buttonCancel;
270
271 // Virtual event handlers, overide them in your derived class
272 virtual void OnKeyDown(wxKeyEvent& event){ event.Skip(); }
273 virtual void OnTextAddress(wxCommandEvent& event){ event.Skip(); }
274 virtual void OnButtonPaste(wxCommandEvent& event){ event.Skip(); }
275 virtual void OnButtonAddressBook(wxCommandEvent& event){ event.Skip(); }
276 virtual void OnKillFocusAmount(wxFocusEvent& event){ event.Skip(); }
277 virtual void OnButtonSend(wxCommandEvent& event){ event.Skip(); }
278 virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }
279
280
281 public:
282 CSendDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Send Coins"), const wxPoint& pos =
    wxDefaultPosition, const wxSize& size = wxSize(675,338), long style = wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER);
283 ~CSendDialogBase();
284
285 };
286
287 ///////////////////////////////////////////////////////////////////////////
288 /// Class CSendingDialogBase
289 ///////////////////////////////////////////////////////////////////////////
```

`CSendingDialogBase` — the payment progress dialog. Size: 442×151, title: `'Sending...'`. The `OnClose` and `OnPaint` handlers indicate this dialog has custom close behavior (you cannot simply close it while a payment is in progress) and custom painting (likely a progress animation). The OK button starts disabled — it only becomes clickable when the transaction completes or fails.

```
290 class CSendingDialogBase : public wxDialog
291 {
292 private:
293
294 protected:
295     wxStaticText* m_staticTextSending;
296     wxTextCtrl* m_textCtrlStatus;
297
298     wxButton* m_buttonOK;
299     wxButton* m_buttonCancel;
300
301 // Virtual event handlers, overide them in your derived class
302 virtual void OnClose(wxCloseEvent& event){ event.Skip(); }
303 virtual void OnPaint(wxPaintEvent& event){ event.Skip(); }
304 virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
```

```
virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }


public:
CSendingDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Sending..."), const wxPoint& pos =
    wxDefaultPosition, const wxSize& size = wxSize(442,151), long style = wxDEFAULT_DIALOG_STYLE);
~CSendingDialogBase();

};

///////////////////////////////////////////////////////////////////////////
/// Class CYourAddressDialogBase
///////////////////////////////////////////////////////////////////////////
```

`CYourAddressDialogBase` — manages receiving addresses. Buttons include `'Rename'`, `'New Address'`, `'Copy to Clipboard'`. The `OnListEndLabelEdit` handler indicates the list supports inline editing — users could click an address label and rename it directly in the list. This level of usability polish suggests Satoshi spent considerable time on the user experience.

```
class CYourAddressDialogBase : public wxDialog
{
private:

protected:

    wxStaticText* m_staticText45;
    wxListCtrl* m_listCtrl;

    wxButton* m_buttonRename;
    wxButton* m_buttonNew;
    wxButton* m_buttonCopy;
    wxButton* m_buttonOK;
    wxButton* m_buttonCancel;

// Virtual event handlers, overide them in your derived class
virtual void OnClose(wxCloseEvent& event){ event.Skip(); }
virtual void OnListEndLabelEdit(wxListEvent& event){ event.Skip(); }
virtual void OnListItemActivated(wxListEvent& event){ event.Skip(); }
virtual void OnListItemSelected(wxListEvent& event){ event.Skip(); }
virtual void OnButtonRename(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonNew(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonCopy(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }


public:
CYourAddressDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Your Bitcoin Address"), const
    wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(610,390), long style = wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER);
~CYourAddressDialogBase();

};

///////////////////////////////////////////////////////////////////////////
/// Class CAddressBookDialogBase
///////////////////////////////////////////////////////////////////////////
class CAddressBookDialogBase : public wxDialog
{
private:

protected:

    wxStaticText* m_staticText55;
    wxListCtrl* m_listCtrl;

    wxButton* m_buttonEdit;
    wxButton* m_buttonNew;
    wxButton* m_buttonDelete;
    wxButton* m_buttonOK;

// Virtual event handlers, overide them in your derived class
virtual void OnClose(wxCloseEvent& event){ event.Skip(); }
virtual void OnListEndLabelEdit(wxListEvent& event){ event.Skip(); }
virtual void OnListItemActivated(wxListEvent& event){ event.Skip(); }
virtual void OnListItemSelected(wxListEvent& event){ event.Skip(); }
virtual void OnButtonEdit(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonNew(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDelete(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }


public:
```

```
    wxButton* m_buttonCancel;
CAddressBookDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Address Book"), const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(610,390), long style = wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER);
~CAddressBookDialogBase();

};

///////////////////////////////////////////////////////////////////////////
/// Class CProductsDialogBase
///////////////////////////////////////////////////////////////////////////
```

`CProductsDialogBase` — the marketplace browser. Title: `'Marketplace'`, size: 708×535. A `wxComboBox` for category filtering, a `wxTextCtrl` for search, and a list control for results. This is a complete product search interface — Satoshi envisioned users browsing and purchasing goods directly through the Bitcoin client, like a decentralized Amazon built into the wallet.

```
class CProductsDialogBase : public wxDialog
{
private:

protected:
    wxComboBox* m_comboBoxCategory;
    wxTextCtrl* m_textCtrlSearch;
    wxButton* m_buttonSearch;
    wxListCtrl* m_listCtrl;

// Virtual event handlers, overide them in your derived class
virtual void OnCombobox(wxCommandEvent& event){ event.Skip(); }
virtual void OnKeyDown(wxKeyEvent& event){ event.Skip(); }
virtual void OnButtonSearch(wxCommandEvent& event){ event.Skip(); }
virtual void OnListItemActivated(wxListEvent& event){ event.Skip(); }


public:
CProductsDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Marketplace"), const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(708,535), long style = wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER);
~CProductsDialogBase();

};

///////////////////////////////////////////////////////////////////////////
/// Class CEditProductDialogBase
///////////////////////////////////////////////////////////////////////////
```

`CEditProductDialogBase` — product listing editor. Note this inherits from `wxFrame`, not `wxDialog` — making it a full independent window rather than a modal popup. It has a scrolled window containing fields for category, title, price, multi-line description, multi-line order form instructions, and a `wxFlexGridSizer` with 20 customizable form field rows (label + choices + delete button each). The `OnButtonSend` handler (not 'Save') reveals that product listings were broadcast to the network like transactions.

```
class CEditProductDialogBase : public wxFrame
{
private:

protected:
    wxScrolledWindow* m_scrolledWindow;
    wxStaticText* m_staticText106;
    wxComboBox* m_comboBoxCategory;
    wxStaticText* m_staticText108;
    wxTextCtrl* m_textCtrlTitle;
    wxStaticText* m_staticText107;
    wxTextCtrl* m_textCtrlPrice;
    wxStaticText* m_staticText22;
    wxTextCtrl* m_textCtrlDescription;
    wxStaticText* m_staticText23;
    wxTextCtrl* m_textCtrlInstructions;
    wxStaticText* m_staticText24;
    wxStaticText* m_staticText25;

    wxTextCtrl* m_textCtrlLabel0;
    wxTextCtrl* m_textCtrlField0;
    wxButton* m_buttonDel0;
    wxTextCtrl* m_textCtrlLabel1;
    wxTextCtrl* m_textCtrlField1;
```

```
    wxButton* m_buttonDel1;
    wxTextCtrl* m_textCtrlLabel2;
    wxTextCtrl* m_textCtrlField2;
    wxButton* m_buttonDel2;
    wxTextCtrl* m_textCtrlLabel3;
    wxTextCtrl* m_textCtrlField3;
    wxButton* m_buttonDel3;
    wxTextCtrl* m_textCtrlLabel4;
    wxTextCtrl* m_textCtrlField4;
    wxButton* m_buttonDel4;
    wxTextCtrl* m_textCtrlLabel5;
    wxTextCtrl* m_textCtrlField5;
    wxButton* m_buttonDel5;
    wxTextCtrl* m_textCtrlLabel6;
    wxTextCtrl* m_textCtrlField6;
    wxButton* m_buttonDel6;
    wxTextCtrl* m_textCtrlLabel7;
    wxTextCtrl* m_textCtrlField7;
    wxButton* m_buttonDel7;
    wxTextCtrl* m_textCtrlLabel8;
    wxTextCtrl* m_textCtrlField8;
    wxButton* m_buttonDel8;
    wxTextCtrl* m_textCtrlLabel9;
    wxTextCtrl* m_textCtrlField9;
    wxButton* m_buttonDel9;
    wxTextCtrl* m_textCtrlLabel10;
    wxTextCtrl* m_textCtrlField10;
    wxButton* m_buttonDel10;
    wxTextCtrl* m_textCtrlLabel11;
    wxTextCtrl* m_textCtrlField11;
    wxButton* m_buttonDel11;
    wxTextCtrl* m_textCtrlLabel12;
    wxTextCtrl* m_textCtrlField12;
    wxButton* m_buttonDel12;
    wxTextCtrl* m_textCtrlLabel13;
    wxTextCtrl* m_textCtrlField13;
    wxButton* m_buttonDel13;
    wxTextCtrl* m_textCtrlLabel14;
    wxTextCtrl* m_textCtrlField14;
    wxButton* m_buttonDel14;
    wxTextCtrl* m_textCtrlLabel15;
    wxTextCtrl* m_textCtrlField15;
    wxButton* m_buttonDel15;
    wxTextCtrl* m_textCtrlLabel16;
    wxTextCtrl* m_textCtrlField16;
    wxButton* m_buttonDel16;
    wxTextCtrl* m_textCtrlLabel17;
    wxTextCtrl* m_textCtrlField17;
    wxButton* m_buttonDel17;
    wxTextCtrl* m_textCtrlLabel18;
    wxTextCtrl* m_textCtrlField18;
    wxButton* m_buttonDel18;
    wxTextCtrl* m_textCtrlLabel19;
    wxTextCtrl* m_textCtrlField19;
    wxButton* m_buttonDel19;
    wxButton* m_buttonAddField;
    wxButton* m_buttonOK;
    wxButton* m_buttonPreview;
    wxButton* m_buttonCancel;

// Virtual event handlers, overide them in your derived class
virtual void OnKeyDown(wxKeyEvent& event){ event.Skip(); }
virtual void OnButtonDel0(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel1(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel2(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel3(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel4(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel5(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel6(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel7(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel8(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel9(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel10(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel11(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel12(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel13(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel14(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel15(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel16(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel17(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel18(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDel19(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonAddField(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonSend(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonPreview(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }


public:
```

```
    wxFlexGridSizer* fgSizer5;
CEditProductDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Edit Product"), const wxPoint&
    pos = wxDefaultPosition, const wxSize& size = wxSize(660,640), long style =
    wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER|wxTAB_TRAVERSAL);
~CEditProductDialogBase();

};

///////////////////////////////////////////////////////////////////////////////
/// Class CViewProductDialogBase
///////////////////////////////////////////////////////////////////////////////
class CViewProductDialogBase : public wxFrame
{
private:

protected:
    wxHtmlWindow* m_htmlWinReviews;
    wxScrolledWindow* m_scrolledWindow;
    wxRichTextCtrl* m_richTextHeading;
    wxStaticText* m_staticTextInstructions;
    wxButton* m_buttonSubmitForm;
    wxButton* m_buttonCancelForm;
    wxButton* m_buttonBack;
    wxButton* m_buttonNext;
    wxButton* m_buttonCancel;

// Virtual event handlers, overide them in your derived class
virtual void OnButtonSubmitForm(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonCancelForm(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonBack(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonNext(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }


public:
CViewProductDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Order Form"), const wxPoint&
    pos = wxDefaultPosition, const wxSize& size = wxSize(630,520), long style =
    wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER|wxTAB_TRAVERSAL);
~CViewProductDialogBase();

};

///////////////////////////////////////////////////////////////////////////////
/// Class CViewOrderDialogBase
///////////////////////////////////////////////////////////////////////////////
class CViewOrderDialogBase : public wxFrame
{
private:

protected:
    wxHtmlWindow* m_htmlWin;
    wxButton* m_buttonOK;

// Virtual event handlers, overide them in your derived class
virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }


public:
CViewOrderDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("View Order"), const wxPoint& pos
    = wxDefaultPosition, const wxSize& size = wxSize(630,520), long style = wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER|wxTAB_TRAVERSAL);
~CViewOrderDialogBase();

};

///////////////////////////////////////////////////////////////////////////////
/// Class CEditReviewDialogBase
///////////////////////////////////////////////////////////////////////////////
class CEditReviewDialogBase : public wxFrame
{
private:

protected:

    wxStaticText* m_staticTextSeller;

    wxStaticText* m_staticText110;
    wxChoice* m_choiceStars;
    wxStaticText* m_staticText43;
    wxTextCtrl* m_textCtrlReview;
    wxButton* m_buttonSubmit;
    wxButton* m_buttonCancel;

// Virtual event handlers, overide them in your derived class
virtual void OnKeyDown(wxKeyEvent& event){ event.Skip(); }
virtual void OnButtonSubmit(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }


public:
```

```
CEditReviewDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Enter Review"), const wxPoint&
    pos = wxDefaultPosition, const wxSize& size = wxSize(630,440), long style =
    wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER|wxTAB_TRAVERSAL);
~CEditReviewDialogBase();

};

///////////////////////////////////////////////////////////////////////////
/// Class CPokerLobbyDialogBase
///////////////////////////////////////////////////////////////////////////
```

CPokerLobbyDialogBase — a poker game lobby with a wxTreeCtrl for game categories and a wxListCtrl for available tables. Title: 'Poker Lobby', size: 586×457. The tree control suggests a hierarchy of game types (Texas Hold'em, Omaha, etc.) or stake levels. The 'New Table' button would create a new poker game. This is fully laid-out infrastructure for peer-to-peer gambling with real bitcoin — a use case that would later become one of Bitcoin's earliest actual applications.

```
class CPokerLobbyDialogBase : public wxFrame
{
private:

protected:
    wxTreeCtrl* m_treeCtrl;
    wxListCtrl* m_listCtrl;
    wxButton* m_buttonNewTable;

// Virtual event handlers, overide them in your derived class
virtual void OnTreeSelChanged(wxTreeEvent& event){ event.Skip(); }
virtual void OnListItemActivated(wxListEvent& event){ event.Skip(); }
virtual void OnListItemSelected(wxListEvent& event){ event.Skip(); }
virtual void OnButtonNewTable(wxCommandEvent& event){ event.Skip(); }


public:
CPokerLobbyDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Poker Lobby"), const wxPoint&
    pos = wxDefaultPosition, const wxSize& size = wxSize(586,457), long style = wxDEFAULT_FRAME_STYLE|wxTAB_TRAVERSAL);
~CPokerLobbyDialogBase();

};

///////////////////////////////////////////////////////////////////////////
/// Class CPokerDialogBase
///////////////////////////////////////////////////////////////////////////
```

CPokerDialogBase — the poker game table. Size: 806×550 with wxFULL_REPAINT_ON_RESIZE (for custom card rendering) and wxFRAME_NO_TASKBAR. Buttons: Deal Hand, Fold, Call, Raise, Leave Table, plus a 'Ditch Player' text field (for kicking inactive players?). Six checkboxes handle pre-action selections: Sit Out, Pre-Fold, Pre-Call, Pre-Call Any, Pre-Raise, Pre-Raise Any. The OnPaint handler would have rendered the card table. This is a fully designed poker client — not a stub.

```
class CPokerDialogBase : public wxFrame
{
private:

protected:
    wxButton* m_buttonDealHand;
    wxButton* m_buttonFold;
    wxButton* m_buttonCall;
    wxButton* m_buttonRaise;
    wxButton* m_buttonLeaveTable;
    wxTextCtrl* m_textDitchPlayer;

// Virtual event handlers, overide them in your derived class
virtual void OnClose(wxCloseEvent& event){ event.Skip(); }
virtual void OnMouseEvents(wxMouseEvent& event){ event.Skip(); }
virtual void OnPaint(wxPaintEvent& event){ event.Skip(); }
virtual void OnSize(wxSizeEvent& event){ event.Skip(); }
virtual void OnCheckSitOut(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonDealHand(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonFold(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonCall(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonRaise(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonLeaveTable(wxCommandEvent& event){ event.Skip(); }
virtual void OnDitchPlayer(wxCommandEvent& event){ event.Skip(); }
virtual void OnCheckPreFold(wxCommandEvent& event){ event.Skip(); }
```

```
virtual void OnCheckPreCall(wxCommandEvent& event){ event.Skip(); }
virtual void OnCheckPreCallAny(wxCommandEvent& event){ event.Skip(); }
virtual void OnCheckPreRaise(wxCommandEvent& event){ event.Skip(); }
virtual void OnCheckPreRaiseAny(wxCommandEvent& event){ event.Skip(); }


public:
    wxCheckBox* m_checkSitOut;
    wxCheckBox* m_checkPreFold;
    wxCheckBox* m_checkPreCall;
    wxCheckBox* m_checkPreCallAny;
    wxCheckBox* m_checkPreRaise;
    wxCheckBox* m_checkPreRaiseAny;
    wxStatusBar* m_statusBar;
CPokerDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Poker"), const wxPoint& pos =
   wxDefaultPosition, const wxSize& size = wxSize(806,550), long style =
   wxDEFAULT_FRAME_STYLE|wxFRAME_NO_TASKBAR|wxFULL_REPAINT_ON_RESIZE|wxTAB_TRAVERSAL);
~CPokerDialogBase();

};

///////////////////////////////////////////////////////////////////////////
/// Class CGetTextFromUserDialogBase
///////////////////////////////////////////////////////////////////////////
```

CGetTextFromUserDialogBase — a generic input dialog with two text fields and two labels. Size: 403×138. Used throughout the application for prompts like 'Enter a label for this address' or 'Enter the amount.' A reusable utility dialog — evidence of Satoshi's pragmatic approach to GUI design, creating one flexible dialog rather than many specialized ones.

```
class CGetTextFromUserDialogBase : public wxDialog
{
private:

protected:

    wxStaticText* m_staticTextMessage1;
    wxTextCtrl* m_textCtrl1;
    wxStaticText* m_staticTextMessage2;
    wxTextCtrl* m_textCtrl2;


    wxButton* m_buttonOK;
    wxButton* m_buttonCancel;

// Virtual event handlers, overide them in your derived class
virtual void OnClose(wxCloseEvent& event){ event.Skip(); }
virtual void OnKeyDown(wxKeyEvent& event){ event.Skip(); }
virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }


public:
CGetTextFromUserDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxEmptyString, const wxPoint&
   pos = wxDefaultPosition, const wxSize& size = wxSize(403,138), long style = wxDEFAULT_DIALOG_STYLE);
~CGetTextFromUserDialogBase();

};

#endif //__uibase__
```

SUMMARY

This header file is a table of contents for Satoshi's complete vision. The sixteen class declarations describe not just a wallet but an ecosystem: payments, address management, a marketplace with product listings and order forms, a review system, and a poker game. Five of these sixteen classes — the marketplace browser, product editor, product viewer, order viewer, review editor — were never activated in any public release. The poker lobby and table bring it to seven unreleased interfaces out of sixteen total.

The virtual event handler pattern deserves appreciation. By making every handler `virtual` with a default pass-through, Satoshi created a clean separation between layout and logic. The wxFormBuilder tool could regenerate this file at any time without affecting the behavioral code in the derived classes. This is not sophisticated software engineering by modern standards, but it shows that Satoshi understood object-oriented design patterns and applied them consistently. The same disciplined architecture that structures the consensus code also structures the button handlers.

Perhaps most revealing are the poker IDs. The constants `wxID_FOLD`, `wxID_CALL`, and `wxID_RAISE` are not aspirational — they correspond to fully laid-out interfaces with checkboxes for pre-actions, custom paint handlers for card rendering, and a lobby system for finding tables. Satoshi was building an application where you could mine bitcoin, spend it in a marketplace, and gamble with it at a poker table — all within one window. That none of these features shipped publicly does not diminish their significance as evidence of how Satoshi conceived of Bitcoin's role in the world.

KEY TERMS

generated base class

A class such as CMainFrameBase that wxFormBuilder writes out, holding the widgets and empty event handlers; ui.h derives the working window classes from it.

virtual event handler

A protected virtual method such as OnButtonSend whose generated body only calls event.Skip(), waiting for a derived class to override it with real logic.

widget ID

An integer constant such as wxID_BUTTONSEND (1002) that identifies a control so events can be routed to it; uibase.h defines 52 of them, numbered from 1000.

wxFrame

The wxWidgets class for an independent top-level window with its own title bar; the main window and the marketplace and poker windows all derive from it.

wxDialog

The wxWidgets class for a secondary window that can run modally, blocking its parent until closed; Send Coins, Options, About, and the address windows derive from it.

Template Method pattern

The design pattern in which a base class fixes the overall structure and calls overridable hooks, leaving subclasses to supply the specific steps, here the event handlers.

poker dialogs

CPokerLobbyDialogBase and CPokerDialogBase, fully laid-out windows for a card game with Deal Hand, Fold, Call, and Raise controls; no class in ui.h ever derives from them.

uint256.h

751 lines CPP

NUMBERS BEYOND IMAGINATION

Standard computers work with 32-bit or 64-bit integers, but Bitcoin needs 256-bit numbers. This file implements `uint256`—a custom 256-bit unsigned integer type that can represent numbers up to 2^256 - 1. That's approximately 10^77, a number so vast it rivals the estimated count of atoms in the observable universe.

Every SHA-256 hash is a uint256. Every transaction ID is a uint256. Every block hash is a uint256. Every private key is conceptually a uint256 (though stored differently). This type is the atomic unit of Bitcoin's cryptographic identity—the "DNA" that makes every transaction unique and every block identifiable.

The implementation stores 256 bits as an array of eight 32-bit words. Arithmetic operations—addition, subtraction, comparison, bit shifting—are implemented by operating across these words with proper carry/borrow handling. It's essentially reimplementing grade-school arithmetic for numbers too big for the processor to handle natively.

Also defined here is `uint160`, a 160-bit integer used for Bitcoin addresses. The address derivation process (RIPEMD160(SHA256(pubkey))) produces 160 bits. The shorter length was a deliberate trade-off: addresses are 27-34 characters instead of about 50 for a full 256-bit hash, making them more human-manageable while still providing 2^80 security against collision attacks (via the birthday paradox).

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

#include <limits.h>
#include <string>
#if defined(_MSC_VER) || defined(__BORLANDC__)
```

Portable 64-bit integer definitions. In 2009, C++ didn't have standardized 64-bit types (`int64_t` came with C++11). This code detects the compiler (MSVC vs GCC) and uses the appropriate type. Modern Bitcoin uses `<cstdint>`, but this worked across Windows and Linux compilers of the era.

```
typedef __int64  int64;
typedef unsigned __int64  uint64;
#else
typedef long long  int64;
typedef unsigned long long  uint64;
#endif
#if defined(_MSC_VER) && _MSC_VER < 1300
```

MSVC 6.0 compatibility hack: `#define for if(false);else for`. Ancient Visual C++ had a bug where variables declared in `for`-loop initializers leaked into the enclosing scope. This macro forces standard behavior. Satoshi supported very old compilers—Bitcoin needed to run on whatever machines people had.

```
#define for if (false) ; else for
#endif


inline int Testuint256AdHoc(vector<string> vArg);


```

Satoshi's comment explains a C++ subtlety: 'We have to keep a separate base class without constructors so the compiler will let us use it in a union.' C++ unions can't contain members with non-trivial constructors (pre-C++11). By keeping base_uint constructor-free, it can be used in unions elsewhere in the codebase. The derived uint160/uint256 classes add constructors.

```
// We have to keep a separate base class without constructors
// so the compiler will let us use it in a union
template<unsigned int BITS>
```

`base_uint`—template foundation for large integers. The template parameter `BITS` specifies the size (160 or 256). `WIDTH` calculates how many 32-bit words are needed: 256/32 = 8 words. The `pn[]` array stores these words in little-endian order (least significant word first). This simplifies carry propagation in arithmetic.

```
class base_uint
{
protected:
enum { WIDTH=BITS/32 };
unsigned int pn[WIDTH];
public:

```

`operator!()`—tests if the value is zero. Iterates through all words; if any is non-zero, returns `false`. Used in boolean contexts: `if (!hash)` checks for the all-zeros hash, which indicates 'no parent' in the genesis block's `hashPrevBlock` field.

```
bool operator!() const
{
for (int i = 0; i < WIDTH; i++)
if (pn[i] != 0)
return false;
return true;
}

```

`operator~()`—bitwise NOT (complement). Flips every bit. Its key use is `~uint256(0)`, the all-ones maximum value from which `bnProofOfWorkLimit` is built (main.h:22). Each word is independently complemented.

```
const base_uint operator~() const
{
        base_uint ret;
for (int i = 0; i < WIDTH; i++)
            ret.pn[i] = ~pn[i];
return ret;
}

```

`operator-()`—unary negation (two's complement). Computes the additive inverse: `-x` such that `x + (-x) = 0`. Implemented as bitwise NOT plus one (`~x + 1`), the standard two's complement formula. The increment handles the carry across all 8 words correctly.

```
const base_uint operator-() const
{
        base_uint ret;
for (int i = 0; i < WIDTH; i++)
            ret.pn[i] = ~pn[i];
        ret++;
return ret;
}


```

`operator=(uint64)`—assigns a 64-bit value. The low two words get the value (split into low and high 32 bits); remaining words are zeroed. This is the most common initialization: `uint256 hash = 0` creates an all-zeros hash.

```
    base_uint& operator=(uint64 b)
{
        pn[0] = (unsigned int)b;
        pn[1] = (unsigned int)(b >> 32);
for (int i = 2; i < WIDTH; i++)
            pn[i] = 0;
return *this;
}

    base_uint& operator^=(const base_uint& b)
{
for (int i = 0; i < WIDTH; i++)
            pn[i] ^= b.pn[i];
return *this;
```

```
}

    base_uint& operator&=(const base_uint& b)
{
for (int i = 0; i < WIDTH; i++)
            pn[i] &= b.pn[i];
return *this;
}

    base_uint& operator|=(const base_uint& b)
{
for (int i = 0; i < WIDTH; i++)
            pn[i] |= b.pn[i];
return *this;
}

    base_uint& operator^=(uint64 b)
{
        pn[0] ^= (unsigned int)b;
        pn[1] ^= (unsigned int)(b >> 32);
return *this;
}

    base_uint& operator&=(uint64 b)
{
        pn[0] &= (unsigned int)b;
        pn[1] &= (unsigned int)(b >> 32);
return *this;
}

    base_uint& operator|=(uint64 b)
{
        pn[0] |= (unsigned int)b;
        pn[1] |= (unsigned int)(b >> 32);
return *this;
}

```

`operator<<=()`—left bit shift. Shifts the entire 256-bit value left by `shift` positions. The implementation handles cross-word shifts: if shifting by 40 bits, some bits move from word 0 to word 1, others from word 1 to word 2. The calculation `k = shift / 32` gives whole-word shift; `shift % 32` gives within-word shift. Retargeting does not use these shifts; it multiplies and divides the target as a `CBigNum`.

```
    base_uint& operator<<=(unsigned int shift)
{
        base_uint a(*this);
for (int i = 0; i < WIDTH; i++)
            pn[i] = 0;
int k = shift / 32;
        shift = shift % 32;
for (int i = 0; i < WIDTH; i++)
{
if (i+k+1 < WIDTH && shift != 0)
                pn[i+k+1] |= (a.pn[i] >> (32-shift));
if (i+k < WIDTH)
                pn[i+k] |= (a.pn[i] << shift);
}
return *this;
}

```

`operator>>=()`—right bit shift. The mirror of left shift. Its best-known use is `bnProofOfWorkLimit(~uint256(0) >> 32)` (main.h:22): all 256 bits set, shifted right 32 places, giving the easiest target allowed—a hash must start with 32 zero bits. Each right shift by 1 is equivalent to integer division by 2. (The halving's `nSubsidy >>= …` is the same idea on a plain 64-bit integer.)

```
127     base_uint& operator>>=(unsigned int shift)
128 {
129         base_uint a(*this);
130 for (int i = 0; i < WIDTH; i++)
131             pn[i] = 0;
132 int k = shift / 32;
133         shift = shift % 32;
134 for (int i = 0; i < WIDTH; i++)
135 {
136 if (i-k-1 >= 0 && shift != 0)
137                 pn[i-k-1] |= (a.pn[i] << (32-shift));
138 if (i-k >= 0)
139                 pn[i-k] |= (a.pn[i] >> shift);
140 }
141 return *this;
142 }
143
```

`operator+=()`—addition with carry propagation. This is grade-school addition scaled to 256 bits. For each word pair, add plus incoming carry, store low 32 bits as result, carry the high bits to the next word. The `uint64` temporary naturally captures the 33-bit intermediate result.

```
144     base_uint& operator+=(const base_uint& b)
145 {
146         uint64 carry = 0;
147 for (int i = 0; i < WIDTH; i++)
148 {
149             uint64 n = carry + pn[i] + b.pn[i];
150             pn[i] = n & 0xffffffff;
151             carry = n >> 32;
152 }
153 return *this;
154 }
155
156     base_uint& operator-=(const base_uint& b)
157 {
158 *this += -b;
159 return *this;
160 }
161
162     base_uint& operator+=(uint64 b64)
163 {
164         base_uint b;
165         b = b64;
166 *this += b;
167 return *this;
168 }
169
170     base_uint& operator-=(uint64 b64)
171 {
172         base_uint b;
173         b = b64;
174 *this += -b;
175 return *this;
176 }
177
178
```

`operator++()`—prefix increment. The elegant implementation: increment word 0; if it wrapped to 0 (overflow), increment word 1; repeat. The `while (++pn[i] == 0)` chain handles the carry propagation. (The miner's nonce, by contrast, is a plain 32-bit `unsigned int`.)

```
179     base_uint& operator++()
180 {
181 // prefix operator
182 int i = 0;
183 while (++pn[i] == 0 && i < WIDTH-1)
184             i++;
185 return *this;
186 }
187
188 const base_uint operator++(int)
189 {
190 // postfix operator
191 const base_uint ret = *this;
192 ++(*this);
193 return ret;
194 }
195
196     base_uint& operator--()
197 {
198 // prefix operator
199 int i = 0;
200 while (--pn[i] == -1 && i < WIDTH-1)
201             i++;
202 return *this;
203 }
204
205 const base_uint operator--(int)
206 {
207 // postfix operator
208 const base_uint ret = *this;
209 --(*this);
210 return ret;
211 }
212
213
```

`operator<()`—less-than comparison. Compares from most significant word down. This is the core of proof-of-work validation: `hash <= target` checks if a block hash is 'small enough' to meet difficulty. The first non-equal word determines the result. If all words are equal, returns `false` (not strictly less than).

```
214 friend inline bool operator<(const base_uint& a, const base_uint& b)
215 {
216 for (int i = base_uint::WIDTH-1; i >= 0; i--)
217 {
218 if (a.pn[i] < b.pn[i])
219 return true;
220 else if (a.pn[i] > b.pn[i])
221 return false;
222 }
223 return false;
224 }
225
226 friend inline bool operator<=(const base_uint& a, const base_uint& b)
227 {
228 for (int i = base_uint::WIDTH-1; i >= 0; i--)
229 {
230 if (a.pn[i] < b.pn[i])
231 return true;
232 else if (a.pn[i] > b.pn[i])
233 return false;
```

```
}
return true;
}

friend inline bool operator>(const base_uint& a, const base_uint& b)
{
for (int i = base_uint::WIDTH-1; i >= 0; i--)
{
if (a.pn[i] > b.pn[i])
return true;
else if (a.pn[i] < b.pn[i])
return false;
}
return false;
}

friend inline bool operator>=(const base_uint& a, const base_uint& b)
{
for (int i = base_uint::WIDTH-1; i >= 0; i--)
{
if (a.pn[i] > b.pn[i])
return true;
else if (a.pn[i] < b.pn[i])
return false;
}
return true;
}

```

`operator==()`—equality comparison. Compares all words; if any differ, returns `false`. Used to check for specific hashes: `if (hash == hashGenesisBlock)` identifies the genesis block.[2] Note this is a bitwise comparison—there's no concept of 'approximately equal' for cryptographic hashes.

```
friend inline bool operator==(const base_uint& a, const base_uint& b)
{
for (int i = 0; i < base_uint::WIDTH; i++)
if (a.pn[i] != b.pn[i])
return false;
return true;
}

friend inline bool operator==(const base_uint& a, uint64 b)
{
if (a.pn[0] != (unsigned int)b)
return false;
if (a.pn[1] != (unsigned int)(b >> 32))
return false;
for (int i = 2; i < base_uint::WIDTH; i++)
if (a.pn[i] != 0)
return false;
return true;
}

friend inline bool operator!=(const base_uint& a, const base_uint& b)
{
return (!(a == b));
}

friend inline bool operator!=(const base_uint& a, uint64 b)
{
return (!(a == b));
}


```

`GetHex()`—converts to hexadecimal string. Transaction IDs and block hashes are typically displayed as hex. The loop iterates backward through bytes, printing the most significant byte first, so the hex you read is the reverse of the bytes in memory and on the wire. This is why block hashes *display* with a run of leading zeros: proof-of-work makes the number small, and smallness lives at the high end. It is also why txids look byte-reversed when you find them in raw transaction data. The 64-character result is how you see txids in block explorers.

```
294     std::string GetHex() const
295 {
296 char psz[sizeof(pn)*2 + 1];
297 for (int i = 0; i < sizeof(pn); i++)
298 sprintf(psz + i*2, "%02x", ((unsigned char*)pn)[sizeof(pn) - i - 1]);
299 return string(psz, psz + sizeof(pn)*2);
300 }
301
```

`SetHex()`—parses hexadecimal string. The inverse of `GetHex()`: takes a string like `'000000000019d6...'` and populates the `uint256`. Handles optional `0x` prefix and leading whitespace. The `phexdigit[]` lookup table provides fast hex-to-value conversion. Used when loading hardcoded hashes (like genesis block) from source code.[2]

```
302 void SetHex(const std::string& str)
303 {
304 for (int i = 0; i < WIDTH; i++)
305             pn[i] = 0;
306
307 // skip 0x
308 const char* psz = str.c_str();
309 while (isspace(*psz))
310             psz++;
311 if (psz[0] == '0' && tolower(psz[1]) == 'x')
312             psz += 2;
313 while (isspace(*psz))
314             psz++;
315
316 // hex string to uint
317 static char phexdigit[256] = { 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0, 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
    0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0, 0,1,2,3,4,5,6,7,8,9,0,0,0,0,0,0,
    0,0xa,0xb,0xc,0xd,0xe,0xf,0,0,0,0,0,0,0,0,0, 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
    0,0xa,0xb,0xc,0xd,0xe,0xf,0,0,0,0,0,0,0,0,0 };
318 const char* pbegin = psz;
319 while (phexdigit[*psz] || *psz == '0')
320             psz++;
321         psz--;
322 unsigned char* p1 = (unsigned char*)pn;
323 unsigned char* pend = p1 + WIDTH * 4;
324 while (psz >= pbegin && p1 < pend)
325 {
326 *p1 = phexdigit[(unsigned char)*psz--];
327 if (psz >= pbegin)
328 {
329 *p1 |= (phexdigit[(unsigned char)*psz--] << 4);
330                 p1++;
331 }
332 }
333 }
334
335     std::string ToString() const
336 {
337 return (GetHex());
338 }
339
340 unsigned char* begin()
341 {
```

```
return (unsigned char*)&pn[0];
}

unsigned char* end()
{
return (unsigned char*)&pn[WIDTH];
}

unsigned int size()
{
return sizeof(pn);
}


```

Serialization methods: `GetSerializeSize()`, `Serialize()`, `Unserialize()`. These integrate with Bitcoin's serialization framework (serialize.h). A `uint256` serializes as exactly 32 bytes (256 bits), raw binary, little-endian. This is how hashes are stored on disk and transmitted over the network—no formatting overhead.

```
unsigned int GetSerializeSize(int nType=0, int nVersion=VERSION) const
{
return sizeof(pn);
}

template<typename Stream>
void Serialize(Stream& s, int nType=0, int nVersion=VERSION) const
{
        s.write((char*)pn, sizeof(pn));
}

template<typename Stream>
void Unserialize(Stream& s, int nType=0, int nVersion=VERSION)
{
        s.read((char*)pn, sizeof(pn));
}


friend class uint160;
friend class uint256;
friend inline int Testuint256AdHoc(vector<string> vArg);
};

```

Typedefs: `base_uint160` and `base_uint256`. These instantiate the template at 160 and 256 bits. The actual `uint160`/`uint256` classes inherit from these, adding constructors. This two-level hierarchy satisfies C++ union requirements (base class has no constructors) while providing convenient construction syntax.

```
typedef base_uint<160> base_uint160;
typedef base_uint<256> base_uint256;


```

Satoshi's comment on the duplicate code: 'uint160 and uint256 could be implemented as templates, but to keep compile errors and debugging cleaner, they're copy and pasted.' Pragmatic engineering over theoretical purity. Debugging template instantiation errors was even harder in 2009 compilers.

```
//
// uint160 and uint256 could be implemented as templates, but to keep
```

```
// compile errors and debugging cleaner, they're copy and pasted.
// It's safe to search and replace 160 with 256 and vice versa.
//


////////////////////////////////////////////////////////////////////////////
//
// uint160
//

```

uint160—160-bit integer for address hashes. Bitcoin addresses are RIPEMD160(SHA256(pubkey)), producing 160 bits. Why not 256? Shorter addresses are more usable (27-34 characters vs about 50). The security is still 2^80 against birthday attacks—far beyond brute-forceable. Every 'hash160' in Bitcoin's script system is this type.

```
class uint160 : public base_uint160
{
public:
typedef base_uint160 basetype;

uint160()
{
}

uint160(const basetype& b)
{
for (int i = 0; i < WIDTH; i++)
            pn[i] = b.pn[i];
}

    uint160& operator=(const basetype& b)
{
for (int i = 0; i < WIDTH; i++)
            pn[i] = b.pn[i];
return *this;
}

uint160(uint64 b)
{
        pn[0] = (unsigned int)b;
        pn[1] = (unsigned int)(b >> 32);
for (int i = 2; i < WIDTH; i++)
            pn[i] = 0;
}

    uint160& operator=(uint64 b)
{
        pn[0] = (unsigned int)b;
        pn[1] = (unsigned int)(b >> 32);
for (int i = 2; i < WIDTH; i++)
            pn[i] = 0;
return *this;
}

explicit uint160(const std::string& str)
{
SetHex(str);
}

explicit uint160(const std::vector<unsigned char>& vch)
{
if (vch.size() == sizeof(pn))
memcpy(pn, &vch[0], sizeof(pn));
else
*this = 0;
}
```

```
448 };
449
450 inline bool operator==(const uint160& a, uint64 b) { return (base_uint160)a == b; }
451 inline bool operator!=(const uint160& a, uint64 b) { return (base_uint160)a != b; }
452 inline const uint160 operator<<(const base_uint160& a, unsigned int shift) { return uint160(a) <<=
    shift; }
453 inline const uint160 operator>>(const base_uint160& a, unsigned int shift) { return uint160(a) >>=
    shift; }
454 inline const uint160 operator<<(const uint160& a, unsigned int shift) { return uint160(a) <<= shift; }
455 inline const uint160 operator>>(const uint160& a, unsigned int shift) { return uint160(a) >>= shift; }
456
457 inline const uint160 operator^(const base_uint160& a, const base_uint160& b) { return uint160(a) ^= b; }
458 inline const uint160 operator&(const base_uint160& a, const base_uint160& b) { return uint160(a) &= b; }
459 inline const uint160 operator|(const base_uint160& a, const base_uint160& b) { return uint160(a) |= b; }
460 inline const uint160 operator+(const base_uint160& a, const base_uint160& b) { return uint160(a) += b; }
461 inline const uint160 operator-(const base_uint160& a, const base_uint160& b) { return uint160(a) -= b; }
462
463 inline bool operator<(const base_uint160& a, const uint160& b) { return (base_uint160)a <
    (base_uint160)b; }
464 inline bool operator<=(const base_uint160& a, const uint160& b) { return (base_uint160)a <=
    (base_uint160)b; }
465 inline bool operator>(const base_uint160& a, const uint160& b) { return (base_uint160)a >
    (base_uint160)b; }
466 inline bool operator>=(const base_uint160& a, const uint160& b) { return (base_uint160)a >=
    (base_uint160)b; }
467 inline bool operator==(const base_uint160& a, const uint160& b) { return (base_uint160)a ==
    (base_uint160)b; }
468 inline bool operator!=(const base_uint160& a, const uint160& b) { return (base_uint160)a !=
    (base_uint160)b; }
469 inline const uint160 operator^(const base_uint160& a, const uint160& b) { return (base_uint160)a ^
    (base_uint160)b; }
470 inline const uint160 operator&(const base_uint160& a, const uint160& b) { return (base_uint160)a &
    (base_uint160)b; }
471 inline const uint160 operator|(const base_uint160& a, const uint160& b) { return (base_uint160)a |
    (base_uint160)b; }
472 inline const uint160 operator+(const base_uint160& a, const uint160& b) { return (base_uint160)a +
    (base_uint160)b; }
473 inline const uint160 operator-(const base_uint160& a, const uint160& b) { return (base_uint160)a -
    (base_uint160)b; }
474
475 inline bool operator<(const uint160& a, const base_uint160& b) { return (base_uint160)a <
    (base_uint160)b; }
476 inline bool operator<=(const uint160& a, const base_uint160& b) { return (base_uint160)a <=
    (base_uint160)b; }
477 inline bool operator>(const uint160& a, const base_uint160& b) { return (base_uint160)a >
    (base_uint160)b; }
478 inline bool operator>=(const uint160& a, const base_uint160& b) { return (base_uint160)a >=
    (base_uint160)b; }
479 inline bool operator==(const uint160& a, const base_uint160& b) { return (base_uint160)a ==
    (base_uint160)b; }
480 inline bool operator!=(const uint160& a, const base_uint160& b) { return (base_uint160)a !=
    (base_uint160)b; }
481 inline const uint160 operator^(const uint160& a, const base_uint160& b) { return (base_uint160)a ^
    (base_uint160)b; }
482 inline const uint160 operator&(const uint160& a, const base_uint160& b) { return (base_uint160)a &
    (base_uint160)b; }
483 inline const uint160 operator|(const uint160& a, const base_uint160& b) { return (base_uint160)a |
    (base_uint160)b; }
484 inline const uint160 operator+(const uint160& a, const base_uint160& b) { return (base_uint160)a +
    (base_uint160)b; }
485 inline const uint160 operator-(const uint160& a, const base_uint160& b) { return (base_uint160)a -
    (base_uint160)b; }
486
487 inline bool operator<(const uint160& a, const uint160& b) { return (base_uint160)a < (base_uint160)b; }
488 inline bool operator<=(const uint160& a, const uint160& b) { return (base_uint160)a <= (base_uint160)b;
    }
489 inline bool operator>(const uint160& a, const uint160& b) { return (base_uint160)a > (base_uint160)b; }
490 inline bool operator>=(const uint160& a, const uint160& b) { return (base_uint160)a >= (base_uint160)b;
    }
491 inline bool operator==(const uint160& a, const uint160& b) { return (base_uint160)a == (base_uint160)b;
    }
```

```
inline bool operator!=(const uint160& a, const uint160& b) { return (base_uint160)a != (base_uint160)b; }
inline const uint160 operator^(const uint160& a, const uint160& b) { return (base_uint160)a ^ (base_uint160)b; }
inline const uint160 operator&(const uint160& a, const uint160& b) { return (base_uint160)a & (base_uint160)b; }
inline const uint160 operator|(const uint160& a, const uint160& b) { return (base_uint160)a | (base_uint160)b; }
inline const uint160 operator+(const uint160& a, const uint160& b) { return (base_uint160)a + (base_uint160)b; }
inline const uint160 operator-(const uint160& a, const uint160& b) { return (base_uint160)a - (base_uint160)b; }
```

`uint256`—256-bit integer for hashes. The workhorse of Bitcoin: block hashes, transaction hashes, merkle roots, difficulty targets, private keys (conceptually). Every SHA-256 result is a uint256. The choice of 256 bits matches modern cryptographic standards and provides a massive security margin.

```
class uint256 : public base_uint256
{
public:
typedef base_uint256 basetype;

uint256()
{
}

uint256(const basetype& b)
{
for (int i = 0; i < WIDTH; i++)
            pn[i] = b.pn[i];
}

    uint256& operator=(const basetype& b)
{
for (int i = 0; i < WIDTH; i++)
            pn[i] = b.pn[i];
return *this;
}

uint256(uint64 b)
{
        pn[0] = (unsigned int)b;
        pn[1] = (unsigned int)(b >> 32);
for (int i = 2; i < WIDTH; i++)
            pn[i] = 0;
}

    uint256& operator=(uint64 b)
{
        pn[0] = (unsigned int)b;
        pn[1] = (unsigned int)(b >> 32);
for (int i = 2; i < WIDTH; i++)
            pn[i] = 0;
return *this;
}
```

```
548 explicit uint256(const std::string& str)
549 {
550 SetHex(str);
551 }
552
553 explicit uint256(const std::vector<unsigned char>& vch)
554 {
555 if (vch.size() == sizeof(pn))
556 memcpy(pn, &vch[0], sizeof(pn));
557 else
558 *this = 0;
559 }
560 };
561
```

Free-standing operators for `uint256`. C++ requires these for mixed expressions like `hash + 1` where the `uint256` is on the left and a native integer on the right. Each is implemented in terms of the compound assignment operators, keeping code DRY while providing intuitive arithmetic syntax.

```
562 inline bool operator==(const uint256& a, uint64 b) { return (base_uint256)a == b; }
563 inline bool operator!=(const uint256& a, uint64 b) { return (base_uint256)a != b; }
564 inline const uint256 operator<<(const base_uint256& a, unsigned int shift) { return uint256(a) <<=
    shift; }
565 inline const uint256 operator>>(const base_uint256& a, unsigned int shift) { return uint256(a) >>=
    shift; }
566 inline const uint256 operator<<(const uint256& a, unsigned int shift) { return uint256(a) <<= shift; }
567 inline const uint256 operator>>(const uint256& a, unsigned int shift) { return uint256(a) >>= shift; }
568
569 inline const uint256 operator^(const base_uint256& a, const base_uint256& b) { return uint256(a) ^= b; }
570 inline const uint256 operator&(const base_uint256& a, const base_uint256& b) { return uint256(a) &= b; }
571 inline const uint256 operator|(const base_uint256& a, const base_uint256& b) { return uint256(a) |= b; }
572 inline const uint256 operator+(const base_uint256& a, const base_uint256& b) { return uint256(a) += b; }
573 inline const uint256 operator-(const base_uint256& a, const base_uint256& b) { return uint256(a) -= b; }
574
575 inline bool operator<(const base_uint256& a, const uint256& b) { return (base_uint256)a <
    (base_uint256)b; }
576 inline bool operator<=(const base_uint256& a, const uint256& b) { return (base_uint256)a <=
    (base_uint256)b; }
577 inline bool operator>(const base_uint256& a, const uint256& b) { return (base_uint256)a >
    (base_uint256)b; }
578 inline bool operator>=(const base_uint256& a, const uint256& b) { return (base_uint256)a >=
    (base_uint256)b; }
579 inline bool operator==(const base_uint256& a, const uint256& b) { return (base_uint256)a ==
    (base_uint256)b; }
580 inline bool operator!=(const base_uint256& a, const uint256& b) { return (base_uint256)a !=
    (base_uint256)b; }
581 inline const uint256 operator^(const base_uint256& a, const uint256& b) { return (base_uint256)a ^
    (base_uint256)b; }
582 inline const uint256 operator&(const base_uint256& a, const uint256& b) { return (base_uint256)a &
    (base_uint256)b; }
583 inline const uint256 operator|(const base_uint256& a, const uint256& b) { return (base_uint256)a |
    (base_uint256)b; }
584 inline const uint256 operator+(const base_uint256& a, const uint256& b) { return (base_uint256)a +
    (base_uint256)b; }
585 inline const uint256 operator-(const base_uint256& a, const uint256& b) { return (base_uint256)a -
    (base_uint256)b; }
586
587 inline bool operator<(const uint256& a, const base_uint256& b) { return (base_uint256)a <
    (base_uint256)b; }
588 inline bool operator<=(const uint256& a, const base_uint256& b) { return (base_uint256)a <=
    (base_uint256)b; }
589 inline bool operator>(const uint256& a, const base_uint256& b) { return (base_uint256)a >
    (base_uint256)b; }
590 inline bool operator>=(const uint256& a, const base_uint256& b) { return (base_uint256)a >=
    (base_uint256)b; }
591 inline bool operator==(const uint256& a, const base_uint256& b) { return (base_uint256)a ==
    (base_uint256)b; }
592 inline bool operator!=(const uint256& a, const base_uint256& b) { return (base_uint256)a !=
    (base_uint256)b; }
```

```
593 inline const uint256 operator^(const uint256& a, const base_uint256& b) { return (base_uint256)a ^
    (base_uint256)b; }
594 inline const uint256 operator&(const uint256& a, const base_uint256& b) { return (base_uint256)a &
    (base_uint256)b; }
595 inline const uint256 operator|(const uint256& a, const base_uint256& b) { return (base_uint256)a |
    (base_uint256)b; }
596 inline const uint256 operator+(const uint256& a, const base_uint256& b) { return (base_uint256)a +
    (base_uint256)b; }
597 inline const uint256 operator-(const uint256& a, const base_uint256& b) { return (base_uint256)a -
    (base_uint256)b; }
598
599 inline bool operator<(const uint256& a, const uint256& b) { return (base_uint256)a < (base_uint256)b; }
600 inline bool operator<=(const uint256& a, const uint256& b) { return (base_uint256)a <= (base_uint256)b;
    }
601 inline bool operator>(const uint256& a, const uint256& b) { return (base_uint256)a > (base_uint256)b; }
602 inline bool operator>=(const uint256& a, const uint256& b) { return (base_uint256)a >= (base_uint256)b;
    }
603 inline bool operator==(const uint256& a, const uint256& b) { return (base_uint256)a == (base_uint256)b;
    }
604 inline bool operator!=(const uint256& a, const uint256& b) { return (base_uint256)a != (base_uint256)b;
    }
605 inline const uint256 operator^(const uint256& a, const uint256& b) { return (base_uint256)a ^
    (base_uint256)b; }
606 inline const uint256 operator&(const uint256& a, const uint256& b) { return (base_uint256)a &
    (base_uint256)b; }
607 inline const uint256 operator|(const uint256& a, const uint256& b) { return (base_uint256)a |
    (base_uint256)b; }
608 inline const uint256 operator+(const uint256& a, const uint256& b) { return (base_uint256)a +
    (base_uint256)b; }
609 inline const uint256 operator-(const uint256& a, const uint256& b) { return (base_uint256)a -
    (base_uint256)b; }
610
611
612
613
614
615
616
617
618
619
620
621
```

`Testuint256AdHoc()`—Satoshi's test/debugging code. This function exercises the `uint256` implementation with various operations: increment, decrement, shifts, hex parsing. It prints results for manual verification. The code was left in the release—evidence that Satoshi tested thoroughly before shipping. Modern Bitcoin has proper unit tests, but this served its purpose in 2009.

```
622 inline int Testuint256AdHoc(vector<string> vArg)
623 {
624     uint256 g(0);
625
626
627 printf("%s\n", g.ToString().c_str());
628     g--; printf("g--\n");
629 printf("%s\n", g.ToString().c_str());
630     g--; printf("g--\n");
631 printf("%s\n", g.ToString().c_str());
632     g++; printf("g++\n");
633 printf("%s\n", g.ToString().c_str());
634     g++; printf("g++\n");
635 printf("%s\n", g.ToString().c_str());
636     g++; printf("g++\n");
637 printf("%s\n", g.ToString().c_str());
638     g++; printf("g++\n");
639 printf("%s\n", g.ToString().c_str());
640
```

```
641
642
643     uint256 a(7);
644 printf("a=7\n");
645 printf("%s\n", a.ToString().c_str());
646
647     uint256 b;
648 printf("b undefined\n");
649 printf("%s\n", b.ToString().c_str());
650 int c = 3;
651
652     a = c;
653     a.pn[3] = 15;
654 printf("%s\n", a.ToString().c_str());
655     uint256 k(c);
656
657     a = 5;
658     a.pn[3] = 15;
659 printf("%s\n", a.ToString().c_str());
660     b = 1;
661     b <<= 52;
662
663     a |= b;
664
665     a ^= 0x500;
666
667 printf("a %s\n", a.ToString().c_str());
668
669     a = a | b | (uint256)0x1000;
670
671
672 printf("a %s\n", a.ToString().c_str());
673 printf("b %s\n", b.ToString().c_str());
674
675     a = 0xfffffe;
676     a.pn[4] = 9;
677
678 printf("%s\n", a.ToString().c_str());
679     a++;
680 printf("%s\n", a.ToString().c_str());
681     a++;
682 printf("%s\n", a.ToString().c_str());
683     a++;
684 printf("%s\n", a.ToString().c_str());
685     a++;
686 printf("%s\n", a.ToString().c_str());
687
688     a--;
689 printf("%s\n", a.ToString().c_str());
690     a--;
691 printf("%s\n", a.ToString().c_str());
692     a--;
693 printf("%s\n", a.ToString().c_str());
694     uint256 d = a--;
695 printf("%s\n", d.ToString().c_str());
696 printf("%s\n", a.ToString().c_str());
697     a--;
698 printf("%s\n", a.ToString().c_str());
699     a--;
700 printf("%s\n", a.ToString().c_str());
701
702     d = a;
703
704 printf("%s\n", d.ToString().c_str());
705 for (int i = uint256::WIDTH-1; i >= 0; i--) printf("%08x", d.pn[i]); printf("\n");
706
707     uint256 neg = d;
708     neg = ~neg;
709 printf("%s\n", neg.ToString().c_str());
710
711
712     uint256 e = uint256("0xABCDEF123abcdef12345678909832180000011111111");
```

```
printf("\n");
printf("%s\n", e.ToString().c_str());


printf("\n");
    uint256 x1 = uint256("0xABCDEF123abcdef12345678909832180000011111111");
    uint256 x2;
printf("%s\n", x1.ToString().c_str());
for (int i = 0; i < 270; i += 4)
{
        x2 = x1 << i;
printf("%s\n", x2.ToString().c_str());
}

printf("\n");
printf("%s\n", x1.ToString().c_str());
for (int i = 0; i < 270; i += 4)
{
        x2 = x1;
        x2 >>= i;
printf("%s\n", x2.ToString().c_str());
}


for (int i = 0; i < 100; i++)
{
        uint256 k = (~uint256(0) >> i);
printf("%s\n", k.ToString().c_str());
}

for (int i = 0; i < 100; i++)
{
        uint256 k = (~uint256(0) << i);
printf("%s\n", k.ToString().c_str());
}

return (0);
}

```

SUMMARY

The uint256 type is Bitcoin's cryptographic primitive. It represents numbers in a space so vast that random values never collide in practice—the probability of two random uint256 values being identical is approximately 1 in 10^{77}, a number so small it defies physical analogy. This is why Bitcoin can use hashes as globally unique identifiers without any central registry. The caveat is that hashes are deterministic, not random: identical data hashes identically. Two early coinbase transactions were byte-for-byte duplicates of earlier ones and so shared their txids, which is why BIP 30 was added to forbid duplicate transactions.[1]

The implementation is straightforward but careful. Grade-school arithmetic scaled to 256 bits, with proper carry handling and efficient word-at-a-time comparisons. The code prioritizes correctness over micro-optimization—these operations aren't in the critical path (mining uses specialized SHA256 code). What matters is that the arithmetic is provably correct, because wrong results could cause consensus failures or security vulnerabilities.

The uint160 for addresses represents an interesting design decision. Satoshi could have used full 256-bit hashes for addresses (as some later cryptocurrencies did), but 160 bits provides a better user experience with shorter addresses while maintaining enormous security margins. The birthday attack bound of 2^80 is still astronomically secure—no one can find colliding addresses.

KEY TERMS

uint256

A 256-bit unsigned integer stored as eight 32-bit words; the type of every block hash, transaction ID, and merkle root.

uint160

The 160-bit sibling type used for address hashes, short enough to keep addresses at a manageable length.

little-endian

The byte order storing the least significant word first in the pn[] array, which makes carries ripple naturally through the arithmetic.

carry

The overflow from one 32-bit word into the next during addition, propagated word by word like grade-school column addition.

two's complement

The standard way to negate a binary number, flip every bit and add one, used by operator-() to compute additive inverses.

hash160

The 160-bit result of RIPEMD160(SHA256(pubkey)) that sits at the heart of every Bitcoin address, stored as a uint160.

bit shift

An operation sliding every bit left or right; right-shifting the subsidy by the era number divides the block reward by two at each halving.

hex

Base-16 notation using digits 0-9 and a-f, the 64-character form in which hashes appear in debug output and block explorers.

SOURCES

1. Pieter Wuille, *BIP 30 — Duplicate transactions* (2012). https://github.com/bitcoin/bips/blob/master/bip-0030.mediawiki
2. Satoshi Nakamoto, *Bitcoin genesis block (block 0)* (January 3, 2009). Coinbase embeds The Times headline as timestamp proof. https://mempool.space/block/000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f

uiproject.fbp

11826 lines XML

THE INTERFACE SATOSHI BUILT

This is the blueprint for Bitcoin's original graphical interface—not code, but a visual design file created with **wxFormBuilder**, a drag-and-drop GUI designer for wxWidgets applications. Satoshi didn't just write a protocol; he designed an application that regular people could use.

The XML you see here defines every button, menu, text field, and dialog that made up the Bitcoin v0.1.0 experience on Windows. wxFormBuilder would read this file and generate the corresponding C++ code (`uibase.cpp` and `uibase.h`), which Satoshi then extended with Bitcoin-specific logic in `ui.cpp`.

What's remarkable is how *complete* the interface was from day one: a main window with balance display, transaction history, address book, send/receive dialogs, and even a "Generate Coins" menu option for CPU mining. This wasn't a command-line tool for cryptographers—it was software designed for a future where ordinary people would send digital cash to each other.

The design decisions embedded here reveal Satoshi's thinking about usability: the prominent display of "Your Bitcoin Address," the simple Send Coins flow, the transaction fee as an optional setting. Some features hint at plans that never fully materialized—like sending coins directly to IP addresses for "online transfer with comments and confirmation."

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<wxFormBuilder_Project>
<FileVersion major="1" minor="9" />
<object class="Project" expanded="1">
<property name="class_decoration"></property>
<property name="code_generation">C++</property>
<property name="disconnect_events">1</property>
<property name="encoding">UTF-8</property>
<property name="event_generation">connect</property>
<property name="file">uibase</property>
<property name="first_id">1000</property>
<property name="help_provider">none</property>
<property name="internationalize">0</property>
<property name="name"></property>
<property name="namespace"></property>
<property name="path">.</property>
<property name="precompiled_header"></property>
<property name="relative_path">1</property>
<property name="use_enum">0</property>
<property name="use_microsoft_bom">0</property>
<object class="Frame" expanded="0">
<property name="bg">wxSYS_COLOUR_BTNFACE</property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_MAINFRAME</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CMainFrameBase</property>
<property name="pos"></property>
```

705 × 484 pixels—the exact dimensions Satoshi chose for the main window. This modest size reflects the era: most users had 1024×768 monitors, and applications weren't expected to fill the screen. The Bitcoin window was meant to sit alongside other programs, quietly processing transactions.

```
<property name="size">705,484</property>
<property
  name="style">wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER</property>
<property name="subclass"></property>
```

The window title: simply "Bitcoin." No version number, no company branding, no "Wallet" or "Client" suffix. Just the name of something that didn't exist before January 2009.[1] This title would appear in the Windows taskbar as users ran the first cryptocurrency software.

```
<property name="title">Bitcoin</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxTAB_TRAVERSAL</property>
<property name="xrc_skip_sizer">1</property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose">OnClose</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle">OnIdle</event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents">OnMouseEvents</event>
<event name="OnMouseWheel"></event>
<event name="OnPaint">OnPaint</event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxMenuBar" expanded="1">
<property name="bg">240,240,240</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">MyMenuBar</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_menubar</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxMenu" expanded="1">
<property name="label">&amp;File</property>
<property name="name">m_menuFile</property>
<property name="permission">protected</property>
<object class="wxMenuItem" expanded="1">
<property name="bitmap"></property>
<property name="checked">0</property>
<property name="enabled">1</property>
<property name="help"></property>
<property name="id">wxID_ANY</property>
<property name="kind">wxITEM_NORMAL</property>
<property name="label">E&amp;xit</property>
<property name="name">m_menuFileExit</property>
<property name="permission">none</property>
<property name="shortcut"></property>
<property name="unchecked_bitmap"></property>
<event name="OnMenuSelection">OnMenuFileExit</event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="wxMenu" expanded="1">
<property name="label">&amp;Options</property>
<property name="name">m_menuOptions</property>
<property name="permission">public</property>
<object class="wxMenuItem" expanded="1">
<property name="bitmap"></property>
<property name="checked">0</property>
<property name="enabled">1</property>
<property name="help"></property>
```

The mining toggle, hidden in plain sight. Under Options → "Generate Coins" was a simple checkbox that turned your computer into a Bitcoin miner. In 2009, a regular CPU could find blocks—there were no GPUs, no ASICs, no mining pools. Check this box, and your PC would start hashing, competing for the 50 BTC block reward. The menu ID `wxID_OPTIONSGENERATEBITCOINS` is one of Bitcoin's most historically significant UI elements.

```
<property name="id">wxID_OPTIONSGENERATEBITCOINS</property>
<property name="kind">wxITEM_CHECK</property>
<property name="label">&amp;Generate Coins</property>
<property name="name">m_menuOptionsGenerateBitcoins</property>
<property name="permission">none</property>
<property name="shortcut"></property>
<property name="unchecked_bitmap"></property>
<event name="OnMenuSelection">OnMenuOptionsGenerate</event>
<event name="OnUpdateUI"></event>
</object>
<object class="wxMenuItem" expanded="1">
<property name="bitmap"></property>
<property name="checked">0</property>
<property name="enabled">1</property>
<property name="help"></property>
<property name="id">wxID_ANY</property>
<property name="kind">wxITEM_NORMAL</property>
<property name="label">&amp;Options...</property>
<property name="name">m_menuOptionsOptions</property>
<property name="permission">none</property>
<property name="shortcut"></property>
<property name="unchecked_bitmap"></property>
<event name="OnMenuSelection">OnMenuOptionsOptions</event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="wxMenu" expanded="1">
<property name="label">&amp;Help</property>
<property name="name">m_menuHelp</property>
<property name="permission">protected</property>
<object class="wxMenuItem" expanded="1">
<property name="bitmap"></property>
<property name="checked">0</property>
<property name="enabled">1</property>
<property name="help"></property>
<property name="id">wxID_ANY</property>
<property name="kind">wxITEM_NORMAL</property>
<property name="label">&amp;About...</property>
<property name="name">m_menuHelpAbout</property>
<property name="permission">none</property>
<property name="shortcut"></property>
<property name="unchecked_bitmap"></property>
<event name="OnMenuSelection">OnMenuHelpAbout</event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
<object class="wxToolBar" expanded="0">
<property name="bg"></property>
<property name="bitmapsize">20,20</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font">,90,90,-1,70,0</property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="margins"></property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_toolBar</property>
<property name="packing">1</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="separation">1</property>
<property name="size">-1,-1</property>
<property name="style">wxTB_FLAT|wxTB_HORZ_TEXT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="tool" expanded="1">
```

The Send Coins toolbar button. The `send20` bitmap (a 20×20 pixel icon from the `rc/` resources) made sending money as simple as clicking an arrow. This toolbar—Send Coins, Address Book—established the basic wallet interface pattern that every Bitcoin wallet would follow for the next seventeen years.

```
<property name="bitmap">send20; Load From Resource</property>
<property name="id">wxID_BUTTONSEND</property>
<property name="kind">wxITEM_NORMAL</property>
<property name="label">&amp;Send Coins</property>
<property name="name">m_tool1</property>
<property name="statusbar"></property>
<property name="tooltip"></property>
<event name="OnMenuSelection"></event>
<event name="OnToolClicked">OnButtonSend</event>
<event name="OnToolEnter"></event>
<event name="OnToolRClicked"></event>
<event name="OnUpdateUI"></event>
</object>
<object class="tool" expanded="1">
<property name="bitmap">addressbook20; Load From Resource</property>
<property name="id">wxID_BUTTONRECEIVE</property>
<property name="kind">wxITEM_NORMAL</property>
<property name="label">&amp;Address Book</property>
<property name="name">m_tool2</property>
<property name="statusbar"></property>
<property name="tooltip"></property>
<event name="OnMenuSelection"></event>
<event name="OnToolClicked">OnButtonAddressBook</event>
<event name="OnToolEnter"></event>
<event name="OnToolRClicked"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="wxStatusBar" expanded="1">
<property name="bg">240,240,240</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="fields">1</property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_statusBar</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxST_SIZEGRIP</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
```

```
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
<object class="wxBoxSizer" expanded="0">
<property name="minimum_size"></property>
<property name="name">bSizer2</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="spacer" expanded="1">
<property name="height">2</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer85</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
```

"Your Bitcoin Address:"—displayed prominently at the top of the main window. Satoshi understood that the address was the key concept users needed to grasp: this is where people send you money. The label's placement shows a focus on receiving payments, suggesting Satoshi imagined Bitcoin primarily as a payment system.

```
<property name="label">Your Bitcoin Address:</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText32</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg">wxSYS_COLOUR_MENU</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_TEXTCTRLADDRESS</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlAddress</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">250,-1</property>
<property name="style">wxTE_READONLY</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents">OnMouseEventsAddress</event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus">OnSetFocusAddress</event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONCOPY</property>
<property name="label">&amp;amp;Copy to Clipboard</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonCopy</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxBU_EXACTFIT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonCopy</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
```

```
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxRIGHT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONCHANGE</property>
<property name="label">C&amp;hange...</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_button91</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonChange</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxALIGN_BOTTOM|wxALL</property>
<property name="proportion">1</property>
<object class="wxPanel" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_panel14</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxTAB_TRAVERSAL</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer66</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
   name="flag">wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
```

The Balance display. Just a simple label showing your total bitcoin holdings. No USD conversion, no portfolio tracking, no price charts—those would come later as Bitcoin gained market value. In 2009, the balance was just a number of coins with no established exchange rate.

```
<property name="label">Balance:</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText41</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,15</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
```

```
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxALL</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg">240,240,240</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font">,90,90,8,70,0</property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label"></property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticTextBalance</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">120,15</property>
<property name="style">wxALIGN_RIGHT|wxST_NO_AUTORESIZE</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_BOTTOM|wxTOP|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxChoice" expanded="1">
<property name="bg"></property>
<property name="choices">&amp;quot; All&amp;quot; &amp;quot; Sent&amp;quot; &amp;quot;
  Received&amp;quot; &amp;quot; In Progress&amp;quot;</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">1</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_choiceFilter</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="selection">0</property>
<property name="size">110,-1</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnChoice"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxNotebook" expanded="1">
<property name="bg"></property>
<property name="bitmapsize"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_notebook</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnNotebookPageChanged"></event>
<event name="OnNotebookPageChanging"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="notebookpage" expanded="1">
<property name="bitmap"></property>
```

"All Transactions" tab. Every send, receive, and generated block appeared here in a scrolling list. This was your complete financial history on the Bitcoin network, stored locally in your wallet.dat file. The transaction list paradigm—showing debits and credits chronologically—borrowed from familiar banking interfaces.

```
<property name="label">All Transactions</property>
<property name="select">0</property>
<object class="wxPanel" expanded="1">
<property name="bg"></property>
```

```
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_panel7</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxTAB_TRAVERSAL</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer157</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxALL</property>
<property name="proportion">1</property>
<object class="wxListCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_listCtrl</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size"></property>
<property
   name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT|wxLC_SORT_DESCENDING</p
   roperty>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxALWAYS_SHOW_SB</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnListBeginDrag"></event>
<event name="OnListBeginLabelEdit"></event>
<event name="OnListBeginRDrag"></event>
<event name="OnListCacheHint"></event>
<event name="OnListColBeginDrag">OnListColBeginDrag</event>
<event name="OnListColClick"></event>
<event name="OnListColDragging"></event>
<event name="OnListColEndDrag"></event>
<event name="OnListColRightClick"></event>
<event name="OnListDeleteAllItems"></event>
<event name="OnListDeleteItem"></event>
<event name="OnListEndLabelEdit"></event>
<event name="OnListInsertItem"></event>
<event
   name="OnListItemActivated">OnListItemActivatedAllTransactions</event
   >
<event name="OnListItemDeselected"></event>
<event name="OnListItemFocused"></event>
<event name="OnListItemMiddleClick"></event>
<event name="OnListItemRightClick"></event>
<event name="OnListItemSelected"></event>
<event name="OnListKeyDown"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint">OnPaintListCtrl</event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer_TabsForFutureUse</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND | wxALL</property>
<property name="proportion">1</property>
<object class="wxPanel" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">1</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_panel9</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxTAB_TRAVERSAL</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer159</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxListCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_listCtrlEscrows</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
```

```
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnListBeginDrag"></event>
<event name="OnListBeginLabelEdit"></event>
<event name="OnListBeginRDrag"></event>
<event name="OnListCacheHint"></event>
<event name="OnListColBeginDrag"></event>
<event name="OnListColClick"></event>
<event name="OnListColDragging"></event>
<event name="OnListColEndDrag"></event>
<event name="OnListColRightClick"></event>
<event name="OnListDeleteAllItems"></event>
<event name="OnListDeleteItem"></event>
<event name="OnListEndLabelEdit"></event>
<event name="OnListInsertItem"></event>
<event name="OnListItemActivated"></event>
<event name="OnListItemDeselected"></event>
<event name="OnListItemFocused"></event>
<event name="OnListItemMiddleClick"></event>
<event name="OnListItemRightClick"></event>
<event name="OnListItemSelected"></event>
<event name="OnListKeyDown"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND | wxALL</property>
<property name="proportion">1</property>
<object class="wxPanel" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">1</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_panel8</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxTAB_TRAVERSAL</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer158</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxListCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_listCtrlOrdersSent</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnListBeginDrag"></event>
<event name="OnListBeginLabelEdit"></event>
<event name="OnListBeginRDrag"></event>
<event name="OnListCacheHint"></event>
<event name="OnListColBeginDrag"></event>
<event name="OnListColClick"></event>
<event name="OnListColDragging"></event>
<event name="OnListColEndDrag"></event>
<event name="OnListColRightClick"></event>
<event name="OnListDeleteAllItems"></event>
<event name="OnListDeleteItem"></event>
<event name="OnListEndLabelEdit"></event>
<event name="OnListInsertItem"></event>
<event
    name="OnListItemActivated">OnListItemActivatedOrdersSent</event>
<event name="OnListItemDeselected"></event>
<event name="OnListItemFocused"></event>
<event name="OnListItemMiddleClick"></event>
<event name="OnListItemRightClick"></event>
<event name="OnListItemSelected"></event>
<event name="OnListKeyDown"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND | wxALL</property>
<property name="proportion">1</property>
<object class="wxPanel" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">1</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_panel10</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxTAB_TRAVERSAL</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
```

```
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer160</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxListCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_listCtrlProductsSent</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnListBeginDrag"></event>
<event name="OnListBeginLabelEdit"></event>
<event name="OnListBeginRDrag"></event>
<event name="OnListCacheHint"></event>
<event name="OnListColBeginDrag"></event>
<event name="OnListColClick"></event>
<event name="OnListColDragging"></event>
<event name="OnListColEndDrag"></event>
<event name="OnListColRightClick"></event>
<event name="OnListDeleteAllItems"></event>
<event name="OnListDeleteItem"></event>
<event name="OnListEndLabelEdit"></event>
<event name="OnListInsertItem"></event>
<event
    name="OnListItemActivated">OnListItemActivatedProductsSent</event>
<event name="OnListItemDeselected"></event>
<event name="OnListItemFocused"></event>
<event name="OnListItemMiddleClick"></event>
<event name="OnListItemRightClick"></event>
<event name="OnListItemSelected"></event>
<event name="OnListKeyDown"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND | wxALL</property>
<property name="proportion">1</property>
<object class="wxPanel" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">1</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_panel11</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxTAB_TRAVERSAL</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer161</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxListCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_listCtrlOrdersReceived</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnListBeginDrag"></event>
<event name="OnListBeginLabelEdit"></event>
<event name="OnListBeginRDrag"></event>
<event name="OnListCacheHint"></event>
<event name="OnListColBeginDrag"></event>
<event name="OnListColClick"></event>
<event name="OnListColDragging"></event>
<event name="OnListColEndDrag"></event>
<event name="OnListColRightClick"></event>
<event name="OnListDeleteAllItems"></event>
<event name="OnListDeleteItem"></event>
<event name="OnListEndLabelEdit"></event>
<event name="OnListInsertItem"></event>
<event
    name="OnListItemActivated">OnListItemActivatedOrdersReceived</event>
<event name="OnListItemDeselected"></event>
<event name="OnListItemFocused"></event>
<event name="OnListItemMiddleClick"></event>
<event name="OnListItemRightClick"></event>
<event name="OnListItemSelected"></event>
<event name="OnListKeyDown"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
```

```
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="Dialog" expanded="0">
<property name="bg"></property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CTxDetailsDialogBase</property>
<property name="pos"></property>
<property name="size">620,450</property>
<property
    name="style">wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER</property>
<property name="subclass"></property>
<property name="title">Transaction Details</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnInitDialog"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer64</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer66</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxHtmlWindow" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_htmlWin</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxHW_SCROLLBAR_AUTO</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHtmlCellClicked"></event>
<event name="OnHtmlCellHover"></event>
<event name="OnHtmlLinkClicked"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_RIGHT</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer65</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_OK</property>
<property name="label">OK</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonOK</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">85,25</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonOK</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="Dialog" expanded="0">
<property name="bg"></property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
```

```
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">COptionsDialogBase</property>
<property name="pos"></property>
<property name="size">500,261</property>
<property name="style">wxDEFAULT_DIALOG_STYLE</property>
<property name="subclass"></property>
<property name="title">Options</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnInitDialog"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer55</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxLEFT</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer57</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="spacer" expanded="1">
<property name="height">20</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxALL</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Optional transaction fee you give to the
    nodes that process your transactions.</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText32</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer56</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
```

Transaction fee configuration. Even in v0.1.0, Satoshi included optional transaction fees—described as money "you give to the nodes that process your transactions." Fees were optional because blocks weren't full yet. The setting reveals foresight: Satoshi knew that eventually block space would become scarce and fees would matter.

```
<property name="label">Transaction fee:</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText31</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
```

```
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_TRANSACTIONFEE</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size"></property>
<property name="name">m_textCtrlTransactionFee</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">70,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus">OnKillFocusTransactionFee</event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_RIGHT</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer58</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_OK</property>
<property name="label">OK</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonOK</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">85,25</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonOK</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_CANCEL</property>
<property name="label">Cancel</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonCancel</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonCancel</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="Dialog" expanded="0">
<property name="bg"></property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CAboutDialogBase</property>
<property name="pos"></property>
<property name="size">507,298</property>
<property name="style">wxDEFAULT_DIALOG_STYLE</property>
<property name="subclass"></property>
<property name="title">About Bitcoin</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
```

```
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnInitDialog"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="0">
<property name="minimum_size"></property>
<property name="name">bSizer60</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer62</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">60</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer63</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="spacer" expanded="1">
<property name="height">50</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer64</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_BOTTOM|wxTOP|wxBOTTOM|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font">Tahoma,90,92,10,74,0</property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Bitcoin </property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText40</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_BOTTOM|wxTOP|wxBOTTOM|wxRIGHT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font">Tahoma,90,90,10,74,0</property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">version</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticTextVersion</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="spacer" expanded="1">
<property name="height">4</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
```

```
<property name="label">Copyright © 2009 Satoshi Nakamoto.&#x0A;&#x0A;This is experimental software.  Do not rely on it for actual financial transactions.&#x0A;&#x0A;Distributed under the MIT/X11 software license, see the accompanying file license.txt or http://www.opensource.org/licenses/mit-license.php.&#x0A;&#x0A;This product includes software developed by the OpenSSL Project for use in the OpenSSL Toolkit (http://www.openssl.org/) and cryptographic software written by Eric Young (eay@cryptsoft.com).</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticTextMain</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">400</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_RIGHT|wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer61</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_OK</property>
<property name="label">OK</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonOK</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">85,25</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonOK</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="Dialog" expanded="0">
<property name="bg"></property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CSendDialogBase</property>
<property name="pos"></property>
<property name="size">675,338</property>
<property name="style">wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER</property>
<property name="subclass"></property>
<property name="title">Send Coins</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnInitDialog"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer21</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="spacer" expanded="1">
<property name="height">5</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
```

```
<property name="flag">wxEXPAND|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxFlexGridSizer" expanded="0">
<property name="cols">2</property>
<property name="flexible_direction">wxBOTH</property>
<property name="growablecols">1</property>
<property name="growablerows"></property>
<property name="hgap">0</property>
<property name="minimum_size"></property>
<property name="name">fgSizer1</property>
<property
    name="non_flexible_grow_mode">wxFLEX_GROWMODE_SPECIFIED</property>
<property name="permission">none</property>
<property name="rows">3</property>
<property name="vgap">0</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
```

Two ways to send: IP address or Bitcoin address. This instruction text reveals a forgotten feature: you could send coins directly to someone's IP address for "online transfer with comments and confirmation." This peer-to-peer mode allowed sender and receiver to exchange messages and required both parties to be online. It was removed in later versions due to security concerns,[2] but shows Satoshi's original vision of interactive payments.

```
<property name="label">Enter the recipient&amp;apos;s IP address (e.g.
    123.45.6.7) for online transfer with comments and confirmation.
    &amp;#x0A;or bitcoin address (e.g. 1NS17iag9jJgTHD1VXjvLCEnZuQ3rJED9L)
    if recipient is not online.</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText14</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxLEFT</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size">70,-1</property>
<property name="name">bSizer47</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxStaticBitmap" expanded="1">
<property name="bg"></property>
<property name="bitmap">check; Load From Icon Resource [-1; -1]
    </property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_bitmapCheckMark</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">16,16</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Pay &amp;amp;To:</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText36</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style">wxALIGN_RIGHT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
```

```
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxRIGHT</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer19</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxALL</property>
<property name="proportion">1</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_TEXTCTRLPAYTO</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size"></property>
<property name="name">m_textCtrlAddress</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText">OnTextAddress</event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONPASTE</property>
<property name="label">&amp;Paste</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonPaste</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style">wxBU_EXACTFIT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonPaste</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONADDRESSBOOK</property>
<property name="label"> Address &amp;Book...</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonAddress</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonAddressBook</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT|wxALIGN_RIG
    HT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
```

```
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">&amp;Amount:</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText19</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style">wxALIGN_RIGHT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxALL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font">,90,90,-1,70,0</property>
<property name="hidden">0</property>
<property name="id">wxID_TEXTCTRLAMOUNT</property>
<property name="maximum_size"></property>
<property name="maxlength">20</property>
<property name="minimum_size"></property>
<property name="name">m_textCtrlAmount</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">145,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus">OnKillFocusAmount</event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxLE
    FT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">T&amp;ransfer:</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText20</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style">wxALIGN_RIGHT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxALL</property>
<property name="proportion">0</property>
<object class="wxChoice" expanded="1">
<property name="bg"></property>
<property name="choices">&amp;quot; Standard&amp;quot;</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_CHOICETRANSFERTYPE</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_choiceTransferType</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="selection">0</property>
<property name="size"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnChoice"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
```

```
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer672</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxBOTTOM|wxRIGHT|wxLEFT</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer681</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxBOTTOM|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">&amp;From:</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticTextFrom</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxLEFT|wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size"></property>
<property name="name">m_textCtrlFrom</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer67</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxBOTTOM|wxRIGHT|wxLEFT</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer68</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxTOP|wxBOTTOM|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">&amp;Message:</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticTextMessage</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxLEFT</property>
<property name="proportion">1</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
```

```
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size"></property>
<property name="name">m_textCtrlMessage</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxTE_MULTILINE</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer23</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font">,90,90,-1,70,0</property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONSEND</property>
<property name="label">&amp;Send</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonSend</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonSend</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font">,90,90,-1,70,0</property>
<property name="hidden">0</property>
<property name="id">wxID_CANCEL</property>
<property name="label">Cancel</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonCancel</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonCancel</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="Dialog" expanded="1">
<property name="bg"></property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CSendingDialogBase</property>
<property name="pos"></property>
<property name="size">442,151</property>
<property name="style">wxDEFAULT_DIALOG_STYLE</property>
<property name="subclass"></property>
<property name="title">Sending...</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose">OnClose</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnInitDialog"></event>
```

```
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint">OnPaint</event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer68</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">8</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxEXPAND|wxTOP|wxRIGHT|wxLEFT</pr
    operty>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label"></property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticTextSending</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,14</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">10</property>
<property name="flag">wxEXPAND|wxRIGHT|wxLEFT</property>
<property name="proportion">1</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg">wxSYS_COLOUR_BTNFACE</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size"></property>
<property name="name">m_textCtrlStatus</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property
    name="style">wxTE_CENTRE|wxTE_MULTILINE|wxTE_NO_VSCROLL|wxTE_READONL
    Y</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value">&#x0A;&#x0A;Connecting...</property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxNO_BORDER</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer69</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">0</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">OK</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonOK</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonOK</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
```

```
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_CANCEL</property>
<property name="label">Cancel</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonCancel</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonCancel</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="Dialog" expanded="0">
<property name="bg"></property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CYourAddressDialogBase</property>
<property name="pos"></property>
<property name="size">610,390</property>
<property
    name="style">wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER</property>
<property name="subclass"></property>
<property name="title">Your Bitcoin Address</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose">OnClose</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnInitDialog"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer68</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="spacer" expanded="1">
<property name="height">5</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
```

Privacy through multiple addresses. "You may want to give a different one to each sender so you can keep track of who is paying you." The stated reason is bookkeeping, but the advice doubles as a key privacy practice the whitepaper had already spelled out: address reuse makes transactions linkable.[3] Satoshi built address generation into the UI from day one, encouraging users to create fresh addresses— a privacy feature that remains best practice today.

```
<property name="label">These are your Bitcoin addresses for
    receiving payments.&#x0A;You may want to give a different one to each
    sender so you can keep track of who is paying you.</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText45</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">600</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
```

```
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxListCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_LISTCTRL</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_listCtrl</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property
    name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT|wxLC_SORT_ASCENDING</pr
    operty>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnListBeginDrag"></event>
<event name="OnListBeginLabelEdit"></event>
<event name="OnListBeginRDrag"></event>
<event name="OnListCacheHint"></event>
<event name="OnListColBeginDrag"></event>
<event name="OnListColClick"></event>
<event name="OnListColDragging"></event>
<event name="OnListColEndDrag"></event>
<event name="OnListColRightClick"></event>
<event name="OnListDeleteAllItems"></event>
<event name="OnListDeleteItem"></event>
<event name="OnListEndLabelEdit">OnListEndLabelEdit</event>
<event name="OnListInsertItem"></event>
<event name="OnListItemActivated">OnListItemActivated</event>
<event name="OnListItemDeselected"></event>
<event name="OnListItemFocused"></event>
<event name="OnListItemMiddleClick"></event>
<event name="OnListItemRightClick"></event>
<event name="OnListItemSelected">OnListItemSelected</event>
<event name="OnListKeyDown"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer69</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONRENAME</property>
<property name="label">&amp;amp;Rename...</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonRename</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonRename</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONNEW</property>
<property name="label">&amp;amp;New Address...</property>
<property name="maximum_size"></property>
<property name="minimum_size">110,25</property>
<property name="name">m_buttonNew</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonNew</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONCOPY</property>
```

```
<property name="label">&amp;Copy to Clipboard</property>
<property name="maximum_size"></property>
<property name="minimum_size">120,25</property>
<property name="name">m_buttonCopy</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonCopy</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_OK</property>
<property name="label">OK</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonOK</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonOK</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_CANCEL</property>
<property name="label">Cancel</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonCancel</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonCancel</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
<object class="Dialog" expanded="0">
<property name="bg"></property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CAddressBookDialogBase</property>
<property name="pos"></property>
<property name="size">610,390</property>
<property
    name="style">wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER</property>
<property name="subclass"></property>
<property name="title">Address Book</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose">OnClose</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnInitDialog"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer68</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
```

```
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="spacer" expanded="1">
<property name="height">5</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">1</property>
<property name="id">wxID_ANY</property>
<property name="label">Bitcoin Address</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText55</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxListCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_LISTCTRL</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_listCtrl</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property
    name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT|wxLC_SORT_ASCENDING</pr
    operty>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnListBeginDrag"></event>
<event name="OnListBeginLabelEdit"></event>
<event name="OnListBeginRDrag"></event>
<event name="OnListCacheHint"></event>
<event name="OnListColBeginDrag"></event>
<event name="OnListColClick"></event>
<event name="OnListColDragging"></event>
<event name="OnListColEndDrag"></event>
<event name="OnListColRightClick"></event>
<event name="OnListDeleteAllItems"></event>
<event name="OnListDeleteItem"></event>
<event name="OnListEndLabelEdit">OnListEndLabelEdit</event>
<event name="OnListInsertItem"></event>
<event name="OnListItemActivated">OnListItemActivated</event>
<event name="OnListItemDeselected"></event>
<event name="OnListItemFocused"></event>
<event name="OnListItemMiddleClick"></event>
<event name="OnListItemRightClick"></event>
<event name="OnListItemSelected">OnListItemSelected</event>
<event name="OnListKeyDown"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer69</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONEDIT</property>
<property name="label">&amp;amp;Edit...</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonEdit</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonEdit</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
```

```
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONNEW</property>
<property name="label">&amp;amp;New Address...</property>
<property name="maximum_size"></property>
<property name="minimum_size">110,25</property>
<property name="name">m_buttonNew</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonNew</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONDELETE</property>
<property name="label">&amp;amp;Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonDelete</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDelete</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_OK</property>
<property name="label">OK</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonOK</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonOK</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_CANCEL</property>
<property name="label">Cancel</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonCancel</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonCancel</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
```

```
</object>
</object>
<object class="Dialog" expanded="0">
<property name="bg"></property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CProductsDialogBase</property>
<property name="pos"></property>
<property name="size">708,535</property>
<property
    name="style">wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER</property>
<property name="subclass"></property>
<property name="title">Marketplace</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnInitDialog"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer22</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxTOP|wxBOTTOM|wxRIGHT</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer23</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxComboBox" expanded="1">
<property name="bg"></property>
<property name="choices">&amp;quot;(Any Category)&amp;quot;</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_comboBoxCategory</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">150,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value">(Any Category)</property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnCombobox">OnCombobox</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">&amp;amp;Search</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonSearch</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonSearch</event>
<event name="OnChar"></event>
```

```
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxListCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_listCtrl</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnListBeginDrag"></event>
<event name="OnListBeginLabelEdit"></event>
<event name="OnListBeginRDrag"></event>
<event name="OnListCacheHint"></event>
<event name="OnListColBeginDrag"></event>
<event name="OnListColClick"></event>
<event name="OnListColDragging"></event>
<event name="OnListColEndDrag"></event>
<event name="OnListColRightClick"></event>
<event name="OnListDeleteAllItems"></event>
<event name="OnListDeleteItem"></event>
<event name="OnListEndLabelEdit"></event>
<event name="OnListInsertItem"></event>
<event name="OnListItemActivated">OnListItemActivated</event>
<event name="OnListItemDeselected"></event>
<event name="OnListItemFocused"></event>
<event name="OnListItemMiddleClick"></event>
<event name="OnListItemRightClick"></event>
<event name="OnListItemSelected"></event>
<event name="OnListKeyDown"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
<object class="Frame" expanded="0">
<property name="bg">wxSYS_COLOUR_MENU</property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CEditProductDialogBase</property>
<property name="pos"></property>
<property name="size">660,640</property>
<property
    name="style">wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER</property>
<property name="subclass"></property>
<property name="title">Edit Product</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxTAB_TRAVERSAL</property>
<property name="xrc_skip_sizer">1</property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer20</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxALL</property>
<property name="proportion">1</property>
<object class="wxScrolledWindow" expanded="1">
<property name="bg">wxSYS_COLOUR_WINDOW</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_scrolledWindow</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="scroll_rate_x">5</property>
<property name="scroll_rate_y">5</property>
<property name="size"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property
    name="window_style">wxHSCROLL|wxTAB_TRAVERSAL|wxVSCROLL</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
```

```
<property name="name">bSizer21</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxTOP|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxFlexGridSizer" expanded="1">
<property name="cols">2</property>
<property name="flexible_direction">wxBOTH</property>
<property name="growablecols">1</property>
<property name="growablerows"></property>
<property name="hgap">0</property>
<property name="minimum_size"></property>
<property name="name">fgSizer8</property>
<property
    name="non_flexible_grow_mode">wxFLEX_GROWMODE_SPECIFIED</property>
<property name="permission">none</property>
<property name="rows">0</property>
<property name="vgap">0</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxLE
    FT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Category</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText106</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxALIGN_RIGHT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxComboBox" expanded="1">
<property name="bg"></property>
<property name="choices"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size">180,-1</property>
<property name="name">m_comboBoxCategory</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnCombobox"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnUpdateUI"></event>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxLE
    FT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Title</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText108</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxALIGN_RIGHT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxALIGN_CENTER_VERTICAL|wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size"></property>
<property name="name">m_textCtrlTitle</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
```

```
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxLE
    FT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Price</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText107</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxALIGN_RIGHT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">105,-1</property>
<property name="name">m_textCtrlPrice</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Page 1: Description</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText22</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,170</property>
```

```
<property name="name">m_textCtrlDescription</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxTE_MULTILINE</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Page 2: Order Form</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText23</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxALL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,120</property>
<property name="name">m_textCtrlInstructions</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxTE_MULTILINE</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxFlexGridSizer" expanded="0">
<property name="cols">3</property>
<property name="flexible_direction">wxBOTH</property>
<property name="growablecols">1</property>
<property name="growablerows"></property>
<property name="hgap">0</property>
<property name="minimum_size"></property>
<property name="name">fgSizer5</property>
<property
    name="non_flexible_grow_mode">wxFLEX_GROWMODE_SPECIFIED</property>
<property name="permission">public</property>
<property name="rows">0</property>
<property name="vgap">0</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxTOP|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Label</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText24</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
```

```
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxTOP|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Comma separated list of choices, or leave
    blank for text field</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText25</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel0</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxRIGHT|wxLEFT|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL0</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel0</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
```

```
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel0</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel1</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField1</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL1</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel1</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel1</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
```

```
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel2</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField2</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL2</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel2</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel2</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel3</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
```

```
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField3</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL3</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel3</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel3</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel4</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField4</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
```

```
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL4</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel4</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel4</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel5</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField5</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL5</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel5</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
```

```
<event name="OnButtonClick">OnButtonDel5</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel6</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField6</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL6</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel6</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel6</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel7</property>
```

```
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField7</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL7</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel7</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel7</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel8</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
```

```
<property name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField8</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL8</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel8</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel8</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Field 8</property>
<property name="maximum_size"></property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel8</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">150,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField9</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
```

```
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL9</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel9</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel9</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel10</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField10</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL10</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel10</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel10</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
```

```
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel11</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField11</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel12</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL11</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel11</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel11</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel12</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
```

```
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField12</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL12</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel12</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel12</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel113</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
```

```
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField13</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL13</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel13</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel13</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel14</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField14</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
```

```
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL14</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel14</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel14</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel15</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField15</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL15</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel15</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel15</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
```

```
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel16</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField16</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL16</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel16</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel16</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel17</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
```

```
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField17</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL17</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel17</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel17</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel118</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
```

```
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField18</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL18</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel18</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel18</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size">150,-1</property>
<property name="name">m_textCtrlLabel19</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size">-1,-1</property>
<property name="maxlength">0</property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_textCtrlField19</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
```

```
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEL19</property>
<property name="label">Delete</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonDel19</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">60,20</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDel19</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_HORIZONTAL</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer25</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">&amp;Add Field</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonAddField</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonAddField</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_RIGHT</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer26</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONSEND</property>
<property name="label">&amp;Send</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonOK</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonSend</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONPREVIEW</property>
<property name="label">&amp;Preview</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonPreview</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
```

```
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonPreview</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_CANCEL</property>
<property name="label">Cancel</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonCancel</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonCancel</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="Frame" expanded="0">
<property name="bg">wxSYS_COLOUR_MENU</property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CViewProductDialogBase</property>
<property name="pos"></property>
<property name="size">630,520</property>
<property
   name="style">wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER</property>
<property name="subclass"></property>
<property name="title">Order Form</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxTAB_TRAVERSAL</property>
<property name="xrc_skip_sizer">1</property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer20</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer116</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxHtmlWindow" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">1</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_htmlWinReviews</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxHW_SCROLLBAR_AUTO</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHtmlCellClicked"></event>
<event name="OnHtmlCellHover"></event>
<event name="OnHtmlLinkClicked"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
```

```
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxALL</property>
<property name="proportion">1</property>
<object class="wxScrolledWindow" expanded="1">
<property name="bg">wxSYS_COLOUR_WINDOW</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_scrolledWindow</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="scroll_rate_x">5</property>
<property name="scroll_rate_y">5</property>
<property name="size"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property
    name="window_style">wxHSCROLL|wxTAB_TRAVERSAL|wxVSCROLL</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer21</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxRichTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_richTextHeading</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,50</property>
<property name="style">wxTE_READONLY</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxNO_BORDER</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRichTextCharacter"></event>
<event name="OnRichTextContentDeleted"></event>
<event name="OnRichTextContentInserted"></event>
<event name="OnRichTextDelete"></event>
<event name="OnRichTextReturn"></event>
<event name="OnRichTextStyleChanged"></event>
<event name="OnRichTextStyleSheetChanged"></event>
<event name="OnRichTextStyleSheetReplaced"></event>
<event name="OnRichTextStyleSheetReplacing"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Order Form instructions here&#x0A;multiple
    lines&#x0A;1&#x0A;2&#x0A;3&#x0A;4&#x0A;5&#x0A;6</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticTextInstructions</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_CENTER_HORIZONTAL</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer25</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONSAMPLE</property>
<property name="label">&amp;Submit</property>
<property name="maximum_size"></property>
<property name="minimum_size">-1,-1</property>
<property name="name">m_buttonSubmitForm</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonSubmitForm</event>
<event name="OnChar"></event>
```

```
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_CANCEL2</property>
<property name="label">Cancel</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonCancelForm</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonCancelForm</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_RIGHT</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer26</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">0</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONBACK</property>
<property name="label">&amp;lt; &amp;amp;Back  </property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonBack</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonBack</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_BUTTONNEXT</property>
<property name="label">&amp;amp;Next &amp;gt;</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonNext</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonNext</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
```

```
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_CANCEL</property>
<property name="label">Cancel</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonCancel</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonCancel</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="Frame" expanded="0">
<property name="bg">wxSYS_COLOUR_MENU</property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CViewOrderDialogBase</property>
<property name="pos"></property>
<property name="size">630,520</property>
<property
    name="style">wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER</property>
<property name="subclass"></property>
<property name="title">View Order</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxTAB_TRAVERSAL</property>
<property name="xrc_skip_sizer">1</property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer20</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer116</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">1</property>
<object class="wxHtmlWindow" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_htmlWin</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxHW_SCROLLBAR_AUTO</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHtmlCellClicked"></event>
<event name="OnHtmlCellHover"></event>
<event name="OnHtmlLinkClicked"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALIGN_RIGHT</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer26</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_OK</property>
<property name="label">OK</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonOK</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonOK</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
```

```
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="Frame" expanded="0">
<property name="bg">wxSYS_COLOUR_MENU</property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CEditReviewDialogBase</property>
<property name="pos"></property>
<property name="size">630,440</property>
<property
    name="style">wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER</property>
<property name="subclass"></property>
<property name="title">Enter Review</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style">wxTAB_TRAVERSAL</property>
<property name="xrc_skip_sizer">1</property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer112</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag"></property>
<property name="proportion">0</property>
<object class="spacer" expanded="1">
<property name="height">3</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label"></property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticTextSeller</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL|wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Rating</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticText110</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
```

```
1013<object class="sizeritem" expanded="1">
1013<property name="border">5</property>
1013<property name="flag">wxALL</property>
1013<property name="proportion">0</property>
1013<object class="wxChoice" expanded="1">
1013<property name="bg"></property>
1013<property name="choices">&amp;quot; 1 star&amp;quot; &amp;quot; 2 stars&amp;quot;
    &amp;quot; 3 stars&amp;quot; &amp;quot; 4 stars&amp;quot; &amp;quot; 5 stars&amp;quot;
    </property>
1013<property name="context_help"></property>
1013<property name="enabled">1</property>
1014<property name="fg"></property>
1014<property name="font"></property>
1014<property name="hidden">0</property>
1014<property name="id">wxID_ANY</property>
1014<property name="maximum_size"></property>
1014<property name="minimum_size"></property>
1014<property name="name">m_choiceStars</property>
1014<property name="permission">protected</property>
1014<property name="pos"></property>
1014<property name="selection">0</property>
1015<property name="size"></property>
1015<property name="subclass"></property>
1015<property name="tooltip"></property>
1015<property name="window_extra_style"></property>
1015<property name="window_name"></property>
1015<property name="window_style"></property>
1015<event name="OnChar"></event>
1015<event name="OnChoice"></event>
1015<event name="OnEnterWindow"></event>
1015<event name="OnEraseBackground"></event>
1016<event name="OnKeyDown"></event>
1016<event name="OnKeyUp"></event>
1016<event name="OnKillFocus"></event>
1016<event name="OnLeaveWindow"></event>
1016<event name="OnLeftDClick"></event>
1016<event name="OnLeftDown"></event>
1016<event name="OnLeftUp"></event>
1016<event name="OnMiddleDClick"></event>
1016<event name="OnMiddleDown"></event>
1016<event name="OnMiddleUp"></event>
1017<event name="OnMotion"></event>
1017<event name="OnMouseEvents"></event>
1017<event name="OnMouseWheel"></event>
1017<event name="OnPaint"></event>
1017<event name="OnRightDClick"></event>
1017<event name="OnRightDown"></event>
1017<event name="OnRightUp"></event>
1017<event name="OnSetFocus"></event>
1017<event name="OnSize"></event>
1017<event name="OnUpdateUI"></event>
1018</object>
1018</object>
1018<object class="sizeritem" expanded="1">
1018<property name="border">5</property>
1018<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
1018<property name="proportion">0</property>
1018<object class="wxStaticText" expanded="1">
1018<property name="bg"></property>
1018<property name="context_help"></property>
1018<property name="enabled">1</property>
1019<property name="fg"></property>
1019<property name="font"></property>
1019<property name="hidden">0</property>
1019<property name="id">wxID_ANY</property>
1019<property name="label">Review</property>
1019<property name="maximum_size"></property>
1019<property name="minimum_size"></property>
1019<property name="name">m_staticText43</property>
1019<property name="permission">protected</property>
1019<property name="pos"></property>
1020<property name="size"></property>
1020<property name="style"></property>
1020<property name="subclass"></property>
1020<property name="tooltip"></property>
1020<property name="window_extra_style"></property>
1020<property name="window_name"></property>
1020<property name="window_style"></property>
1020<property name="wrap">-1</property>
1020<event name="OnChar"></event>
1020<event name="OnEnterWindow"></event>
1021<event name="OnEraseBackground"></event>
1021<event name="OnKeyDown"></event>
1021<event name="OnKeyUp"></event>
1021<event name="OnKillFocus"></event>
1021<event name="OnLeaveWindow"></event>
1021<event name="OnLeftDClick"></event>
1021<event name="OnLeftDown"></event>
1021<event name="OnLeftUp"></event>
1021<event name="OnMiddleDClick"></event>
1021<event name="OnMiddleDown"></event>
1022<event name="OnMiddleUp"></event>
1022<event name="OnMotion"></event>
1022<event name="OnMouseEvents"></event>
1022<event name="OnMouseWheel"></event>
1022<event name="OnPaint"></event>
1022<event name="OnRightDClick"></event>
1022<event name="OnRightDown"></event>
1022<event name="OnRightUp"></event>
1022<event name="OnSetFocus"></event>
1022<event name="OnSize"></event>
1023<event name="OnUpdateUI"></event>
1023</object>
1023</object>
1023<object class="sizeritem" expanded="1">
1023<property name="border">5</property>
1023<property name="flag">wxALL|wxEXPAND</property>
1023<property name="proportion">1</property>
1023<object class="wxTextCtrl" expanded="1">
1023<property name="bg"></property>
1023<property name="context_help"></property>
1024<property name="enabled">1</property>
1024<property name="fg"></property>
1024<property name="font"></property>
1024<property name="hidden">0</property>
1024<property name="id">wxID_ANY</property>
1024<property name="maximum_size"></property>
1024<property name="maxlength">0</property>
1024<property name="minimum_size"></property>
1024<property name="name">m_textCtrlReview</property>
1024<property name="permission">protected</property>
1025<property name="pos"></property>
1025<property name="size"></property>
1025<property name="style">wxTE_MULTILINE</property>
1025<property name="subclass"></property>
1025<property name="tooltip"></property>
1025<property name="value"></property>
1025<property name="window_extra_style"></property>
1025<property name="window_name"></property>
1025<property name="window_style"></property>
1025<event name="OnChar"></event>
1026<event name="OnEnterWindow"></event>
1026<event name="OnEraseBackground"></event>
1026<event name="OnKeyDown">OnKeyDown</event>
1026<event name="OnKeyUp"></event>
1026<event name="OnKillFocus"></event>
1026<event name="OnLeaveWindow"></event>
1026<event name="OnLeftDClick"></event>
1026<event name="OnLeftDown"></event>
1026<event name="OnLeftUp"></event>
1026<event name="OnMiddleDClick"></event>
1027<event name="OnMiddleDown"></event>
1027<event name="OnMiddleUp"></event>
1027<event name="OnMotion"></event>
1027<event name="OnMouseEvents"></event>
1027<event name="OnMouseWheel"></event>
1027<event name="OnPaint"></event>
1027<event name="OnRightDClick"></event>
1027<event name="OnRightDown"></event>
1027<event name="OnRightUp"></event>
1027<event name="OnSetFocus"></event>
1028<event name="OnSize"></event>
1028<event name="OnText"></event>
1028<event name="OnTextEnter"></event>
1028<event name="OnTextMaxLen"></event>
1028<event name="OnTextURL"></event>
1028<event name="OnUpdateUI"></event>
1028</object>
1028</object>
1028<object class="sizeritem" expanded="1">
1028<property name="border">5</property>
1029<property name="flag">wxALIGN_RIGHT</property>
1029<property name="proportion">0</property>
1029<object class="wxBoxSizer" expanded="1">
1029<property name="minimum_size"></property>
1029<property name="name">bSizer113</property>
1029<property name="orient">wxHORIZONTAL</property>
1029<property name="permission">none</property>
1029<object class="sizeritem" expanded="1">
1029<property name="border">5</property>
1029<property name="flag">wxALL</property>
1030<property name="proportion">0</property>
1030<object class="wxButton" expanded="1">
1030<property name="bg"></property>
1030<property name="context_help"></property>
1030<property name="default">0</property>
1030<property name="enabled">1</property>
1030<property name="fg"></property>
1030<property name="font"></property>
1030<property name="hidden">0</property>
1030<property name="id">wxID_SUBMIT</property>
1031<property name="label">&amp;amp;Submit</property>
1031<property name="maximum_size"></property>
1031<property name="minimum_size">85,25</property>
1031<property name="name">m_buttonSubmit</property>
1031<property name="permission">protected</property>
1031<property name="pos"></property>
1031<property name="size"></property>
1031<property name="style"></property>
1031<property name="subclass"></property>
1031<property name="tooltip"></property>
1032<property name="window_extra_style"></property>
1032<property name="window_name"></property>
1032<property name="window_style"></property>
1032<event name="OnButtonClick">OnButtonSubmit</event>
1032<event name="OnChar"></event>
1032<event name="OnEnterWindow"></event>
1032<event name="OnEraseBackground"></event>
1032<event name="OnKeyDown"></event>
1032<event name="OnKeyUp"></event>
1032<event name="OnKillFocus"></event>
1033<event name="OnLeaveWindow"></event>
1033<event name="OnLeftDClick"></event>
1033<event name="OnLeftDown"></event>
1033<event name="OnLeftUp"></event>
1033<event name="OnMiddleDClick"></event>
```

```
1033<event name="OnMiddleDown"></event>
1033<event name="OnMiddleUp"></event>
1033<event name="OnMotion"></event>
1033<event name="OnMouseEvents"></event>
1033<event name="OnMouseWheel"></event>
1034<event name="OnPaint"></event>
1034<event name="OnRightDClick"></event>
1034<event name="OnRightDown"></event>
1034<event name="OnRightUp"></event>
1034<event name="OnSetFocus"></event>
1034<event name="OnSize"></event>
1034<event name="OnUpdateUI"></event>
1034</object>
1034</object>
1034<object class="sizeritem" expanded="1">
1035<property name="border">5</property>
1035<property name="flag">wxALL</property>
1035<property name="proportion">0</property>
1035<object class="wxButton" expanded="1">
1035<property name="bg"></property>
1035<property name="context_help"></property>
1035<property name="default">0</property>
1035<property name="enabled">1</property>
1035<property name="fg"></property>
1035<property name="font"></property>
1036<property name="hidden">0</property>
1036<property name="id">wxID_CANCEL</property>
1036<property name="label">Cancel</property>
1036<property name="maximum_size"></property>
1036<property name="minimum_size">85,25</property>
1036<property name="name">m_buttonCancel</property>
1036<property name="permission">protected</property>
1036<property name="pos"></property>
1036<property name="size"></property>
1036<property name="style"></property>
1037<property name="subclass"></property>
1037<property name="tooltip"></property>
1037<property name="window_extra_style"></property>
1037<property name="window_name"></property>
1037<property name="window_style"></property>
1037<event name="OnButtonClick">OnButtonCancel</event>
1037<event name="OnChar"></event>
1037<event name="OnEnterWindow"></event>
1037<event name="OnEraseBackground"></event>
1037<event name="OnKeyDown"></event>
1038<event name="OnKeyUp"></event>
1038<event name="OnKillFocus"></event>
1038<event name="OnLeaveWindow"></event>
1038<event name="OnLeftDClick"></event>
1038<event name="OnLeftDown"></event>
1038<event name="OnLeftUp"></event>
1038<event name="OnMiddleDClick"></event>
1038<event name="OnMiddleDown"></event>
1038<event name="OnMiddleUp"></event>
1038<event name="OnMotion"></event>
1039<event name="OnMouseEvents"></event>
1039<event name="OnMouseWheel"></event>
1039<event name="OnPaint"></event>
1039<event name="OnRightDClick"></event>
1039<event name="OnRightDown"></event>
1039<event name="OnRightUp"></event>
1039<event name="OnSetFocus"></event>
1039<event name="OnSize"></event>
1039<event name="OnUpdateUI"></event>
1039</object>
1040</object>
1040</object>
1040</object>
1040</object>
1040</object>
1040<object class="Frame" expanded="0">
1040<property name="bg">wxSYS_COLOUR_BTNFACE</property>
1040<property name="center"></property>
1040<property name="context_help"></property>
1040<property name="enabled">1</property>
1041<property name="extra_style"></property>
1041<property name="fg"></property>
1041<property name="font"></property>
1041<property name="hidden">0</property>
1041<property name="id">wxID_ANY</property>
1041<property name="maximum_size"></property>
1041<property name="minimum_size"></property>
1041<property name="name">CPokerLobbyDialogBase</property>
1041<property name="pos"></property>
1041<property name="size">586,457</property>
1042<property name="style">wxDEFAULT_FRAME_STYLE</property>
1042<property name="subclass"></property>
1042<property name="title">Poker Lobby</property>
1042<property name="tooltip"></property>
1042<property name="window_extra_style"></property>
1042<property name="window_name"></property>
1042<property name="window_style">wxTAB_TRAVERSAL</property>
1042<property name="xrc_skip_sizer">1</property>
1042<event name="OnActivate"></event>
1042<event name="OnActivateApp"></event>
1043<event name="OnChar"></event>
1043<event name="OnClose"></event>
1043<event name="OnEnterWindow"></event>
1043<event name="OnEraseBackground"></event>
1043<event name="OnHibernate"></event>
1043<event name="OnIconize"></event>
1043<event name="OnIdle"></event>
1043<event name="OnKeyDown"></event>
1043<event name="OnKeyUp"></event>
1043<event name="OnKillFocus"></event>
1044<event name="OnLeaveWindow"></event>
1044<event name="OnLeftDClick"></event>
1044<event name="OnLeftDown"></event>
1044<event name="OnLeftUp"></event>
1044<event name="OnMiddleDClick"></event>
1044<event name="OnMiddleDown"></event>
1044<event name="OnMiddleUp"></event>
1044<event name="OnMotion"></event>
1044<event name="OnMouseEvents"></event>
1044<event name="OnMouseWheel"></event>
1045<event name="OnPaint"></event>
1045<event name="OnRightDClick"></event>
1045<event name="OnRightDown"></event>
1045<event name="OnRightUp"></event>
1045<event name="OnSetFocus"></event>
1045<event name="OnSize"></event>
1045<event name="OnUpdateUI"></event>
1045<object class="wxBoxSizer" expanded="1">
1045<property name="minimum_size"></property>
1045<property name="name">bSizer156</property>
1046<property name="orient">wxHORIZONTAL</property>
1046<property name="permission">none</property>
1046<object class="sizeritem" expanded="1">
1046<property name="border">5</property>
1046<property name="flag">wxEXPAND|wxTOP|wxBOTTOM|wxLEFT</property>
1046<property name="proportion">0</property>
1046<object class="wxTreeCtrl" expanded="1">
1046<property name="bg"></property>
1046<property name="context_help"></property>
1046<property name="enabled">1</property>
1047<property name="fg"></property>
1047<property name="font"></property>
1047<property name="hidden">0</property>
1047<property name="id">wxID_ANY</property>
1047<property name="maximum_size"></property>
1047<property name="minimum_size">130,-1</property>
1047<property name="name">m_treeCtrl</property>
1047<property name="permission">protected</property>
1047<property name="pos"></property>
1047<property name="size"></property>
1048<property
    name="style">wxTR_HAS_BUTTONS|wxTR_HIDE_ROOT|wxTR_LINES_AT_ROOT</pro
    perty>
1048<property name="subclass"></property>
1048<property name="tooltip"></property>
1048<property name="window_extra_style"></property>
1048<property name="window_name"></property>
1048<property name="window_style"></property>
1048<event name="OnChar"></event>
1048<event name="OnEnterWindow"></event>
1048<event name="OnEraseBackground"></event>
1048<event name="OnKeyDown"></event>
1049<event name="OnKeyUp"></event>
1049<event name="OnKillFocus"></event>
1049<event name="OnLeaveWindow"></event>
1049<event name="OnLeftDClick"></event>
1049<event name="OnLeftDown"></event>
1049<event name="OnLeftUp"></event>
1049<event name="OnMiddleDClick"></event>
1049<event name="OnMiddleDown"></event>
1049<event name="OnMiddleUp"></event>
1049<event name="OnMotion"></event>
1050<event name="OnMouseEvents"></event>
1050<event name="OnMouseWheel"></event>
1050<event name="OnPaint"></event>
1050<event name="OnRightDClick"></event>
1050<event name="OnRightDown"></event>
1050<event name="OnRightUp"></event>
1050<event name="OnSetFocus"></event>
1050<event name="OnSize"></event>
1050<event name="OnTreeBeginDrag"></event>
1050<event name="OnTreeBeginLabelEdit"></event>
1051<event name="OnTreeBeginRDrag"></event>
1051<event name="OnTreeDeleteItem"></event>
1051<event name="OnTreeEndDrag"></event>
1051<event name="OnTreeEndLabelEdit"></event>
1051<event name="OnTreeGetInfo"></event>
1051<event name="OnTreeItemActivated"></event>
1051<event name="OnTreeItemCollapsed"></event>
1051<event name="OnTreeItemCollapsing"></event>
1051<event name="OnTreeItemExpanded"></event>
1051<event name="OnTreeItemExpanding"></event>
1052<event name="OnTreeItemGetTooltip"></event>
1052<event name="OnTreeItemMenu"></event>
1052<event name="OnTreeItemMiddleClick"></event>
1052<event name="OnTreeItemRightClick"></event>
1052<event name="OnTreeKeyDown"></event>
1052<event name="OnTreeSelChanged">OnTreeSelChanged</event>
1052<event name="OnTreeSelChanging"></event>
1052<event name="OnTreeSetInfo"></event>
1052<event name="OnTreeStateImageClick"></event>
1052<event name="OnUpdateUI"></event>
1053</object>
1053</object>
1053<object class="sizeritem" expanded="1">
1053<property name="border">5</property>
1053<property name="flag">wxEXPAND</property>
1053<property name="proportion">1</property>
1053<object class="wxBoxSizer" expanded="1">
1053<property name="minimum_size"></property>
1053<property name="name">bSizer172</property>
```

```
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND|wxALL</property>
<property name="proportion">1</property>
<object class="wxListCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_listCtrl</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnListBeginDrag"></event>
<event name="OnListBeginLabelEdit"></event>
<event name="OnListBeginRDrag"></event>
<event name="OnListCacheHint"></event>
<event name="OnListColBeginDrag"></event>
<event name="OnListColClick"></event>
<event name="OnListColDragging"></event>
<event name="OnListColEndDrag"></event>
<event name="OnListColRightClick"></event>
<event name="OnListDeleteAllItems"></event>
<event name="OnListDeleteItem"></event>
<event name="OnListEndLabelEdit"></event>
<event name="OnListInsertItem"></event>
<event name="OnListItemActivated">OnListItemActivated</event>
<event name="OnListItemDeselected"></event>
<event name="OnListItemFocused"></event>
<event name="OnListItemMiddleClick"></event>
<event name="OnListItemRightClick"></event>
<event name="OnListItemSelected">OnListItemSelected</event>
<event name="OnListKeyDown"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_OPENNEWTABLE</property>
<property name="label">&amp;amp;Open New Table</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonNewTable</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonNewTable</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
</object>
</object>
</object>
<object class="Frame" expanded="0">
<property name="bg"></property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CPokerDialogBase</property>
<property name="pos"></property>
<property name="size">806,550</property>
<property
    name="style">wxDEFAULT_FRAME_STYLE|wxFRAME_NO_TASKBAR</property>
<property name="subclass"></property>
<property name="title">Poker</property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property
    name="window_style">wxFULL_REPAINT_ON_RESIZE|wxTAB_TRAVERSAL</proper
    ty>
<property name="xrc_skip_sizer">1</property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose">OnClose</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents">OnMouseEvents</event>
<event name="OnMouseWheel"></event>
<event name="OnPaint">OnPaint</event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize">OnSize</event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer174</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxCheckBox" expanded="1">
<property name="bg"></property>
<property name="checked">0</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">Deal Me Out</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_checkSitOut</property>
<property name="permission">public</property>
<property name="pos"></property>
```

```
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnCheckBox">OnCheckSitOut</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DEALHAND</property>
<property name="label">&amp;Deal Hand</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonDealHand</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">150,25</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonDealHand</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_FOLD</property>
<property name="label">&amp;Fold</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonFold</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">80,25</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonFold</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_CALL</property>
<property name="label">&amp;Call</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonCall</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">80,25</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonCall</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_RAISE</property>
<property name="label">&amp;Raise</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonRaise</property>
```

```
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">80,25</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonRaise</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_LEAVETABLE</property>
<property name="label">&amp;amp;Leave Table</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_buttonLeaveTable</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">90,25</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonLeaveTable</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_DITCHPLAYER</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size"></property>
<property name="name">m_textDitchPlayer</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">45,-1</property>
<property name="style">wxTE_PROCESS_ENTER</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter">OnDitchPlayer</event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxCheckBox" expanded="1">
<property name="bg"></property>
<property name="checked">0</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">FOLD</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_checkPreFold</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size">100,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnCheckBox">OnCheckPreFold</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxCheckBox" expanded="1">
<property name="bg"></property>
<property name="checked">0</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
```

```
<property name="id">wxID_ANY</property>
<property name="label">CALL</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_checkPreCall</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size">100,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnCheckBox">OnCheckPreCall</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxCheckBox" expanded="1">
<property name="bg"></property>
<property name="checked">0</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">CALL ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_checkPreCallAny</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size">100,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnCheckBox">OnCheckPreCallAny</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxCheckBox" expanded="1">
<property name="bg"></property>
<property name="checked">0</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">RAISE</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_checkPreRaise</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size">100,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnCheckBox">OnCheckPreRaise</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxCheckBox" expanded="1">
<property name="bg"></property>
<property name="checked">0</property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label">RAISE ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_checkPreRaiseAny</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size">100,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnCheckBox">OnCheckPreRaiseAny</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
</object>
<object class="wxStatusBar" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="fields">1</property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
```

```
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_statusBar</property>
<property name="permission">public</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxST_SIZEGRIP</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="Dialog" expanded="0">
<property name="bg"></property>
<property name="center"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="extra_style"></property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">CGetTextFromUserDialogBase</property>
<property name="pos"></property>
<property name="size">403,138</property>
<property name="style">wxDEFAULT_DIALOG_STYLE</property>
<property name="subclass"></property>
<property name="title"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnActivate"></event>
<event name="OnActivateApp"></event>
<event name="OnChar"></event>
<event name="OnClose">OnClose</event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnHibernate"></event>
<event name="OnIconize"></event>
<event name="OnIdle"></event>
<event name="OnInitDialog"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer79</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">10</property>
<property name="flag">wxEXPAND|wxALL</property>
<property name="proportion">1</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer81</property>
<property name="orient">wxVERTICAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_ANY</property>
<property name="label"></property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticTextMessage1</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_HORIZONTAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_TEXTCTRL</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size"></property>
<property name="name">m_textCtrl1</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxTE_PROCESS_ENTER</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
```

```
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
<property name="proportion">0</property>
<object class="wxStaticText" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">1</property>
<property name="id">wxID_ANY</property>
<property name="label"></property>
<property name="maximum_size"></property>
<property name="minimum_size"></property>
<property name="name">m_staticTextMessage2</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<property name="wrap">-1</property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_HORIZONTAL</property>
<property name="proportion">0</property>
<object class="wxTextCtrl" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">1</property>
<property name="id">wxID_TEXTCTRL</property>
<property name="maximum_size"></property>
<property name="maxlength">0</property>
<property name="minimum_size"></property>
<property name="name">m_textCtrl2</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size"></property>
<property name="style">wxTE_PROCESS_ENTER</property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="value"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown">OnKeyDown</event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnText"></event>
<event name="OnTextEnter"></event>
<event name="OnTextMaxLen"></event>
<event name="OnTextURL"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">0</property>
<object class="wxBoxSizer" expanded="1">
<property name="minimum_size"></property>
<property name="name">bSizer80</property>
<property name="orient">wxHORIZONTAL</property>
<property name="permission">none</property>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxEXPAND</property>
<property name="proportion">1</property>
<object class="spacer" expanded="1">
<property name="height">0</property>
<property name="permission">protected</property>
<property name="width">0</property>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
<property name="flag">wxALL</property>
<property name="proportion">0</property>
<object class="wxButton" expanded="1">
<property name="bg"></property>
<property name="context_help"></property>
<property name="default">0</property>
<property name="enabled">1</property>
<property name="fg"></property>
<property name="font"></property>
<property name="hidden">0</property>
<property name="id">wxID_OK</property>
<property name="label">OK</property>
<property name="maximum_size"></property>
<property name="minimum_size">85,25</property>
<property name="name">m_buttonOK</property>
<property name="permission">protected</property>
<property name="pos"></property>
<property name="size">-1,-1</property>
<property name="style"></property>
<property name="subclass"></property>
<property name="tooltip"></property>
<property name="window_extra_style"></property>
<property name="window_name"></property>
<property name="window_style"></property>
<event name="OnButtonClick">OnButtonOK</event>
<event name="OnChar"></event>
<event name="OnEnterWindow"></event>
<event name="OnEraseBackground"></event>
<event name="OnKeyDown"></event>
<event name="OnKeyUp"></event>
<event name="OnKillFocus"></event>
<event name="OnLeaveWindow"></event>
<event name="OnLeftDClick"></event>
<event name="OnLeftDown"></event>
<event name="OnLeftUp"></event>
<event name="OnMiddleDClick"></event>
<event name="OnMiddleDown"></event>
<event name="OnMiddleUp"></event>
<event name="OnMotion"></event>
<event name="OnMouseEvents"></event>
<event name="OnMouseWheel"></event>
<event name="OnPaint"></event>
<event name="OnRightDClick"></event>
<event name="OnRightDown"></event>
<event name="OnRightUp"></event>
<event name="OnSetFocus"></event>
<event name="OnSize"></event>
<event name="OnUpdateUI"></event>
</object>
</object>
<object class="sizeritem" expanded="1">
<property name="border">5</property>
```

```
1177<property name="flag">wxALL</property>
1177<property name="proportion">0</property>
1177<object class="wxButton" expanded="1">
1177<property name="bg"></property>
1177<property name="context_help"></property>
1177<property name="default">0</property>
1177<property name="enabled">1</property>
1177<property name="fg"></property>
1177<property name="font"></property>
1177<property name="hidden">0</property>
1178<property name="id">wxID_CANCEL</property>
1178<property name="label">Cancel</property>
1178<property name="maximum_size"></property>
1178<property name="minimum_size">85,25</property>
1178<property name="name">m_buttonCancel</property>
1178<property name="permission">protected</property>
1178<property name="pos"></property>
1178<property name="size"></property>
1178<property name="style"></property>
1178<property name="subclass"></property>
1179<property name="tooltip"></property>
1179<property name="window_extra_style"></property>
1179<property name="window_name"></property>
1179<property name="window_style"></property>
1179<event name="OnButtonClick">OnButtonCancel</event>
1179<event name="OnChar"></event>
1179<event name="OnEnterWindow"></event>
1179<event name="OnEraseBackground"></event>
1179<event name="OnKeyDown"></event>
1179<event name="OnKeyUp"></event>
1180<event name="OnKillFocus"></event>
1180<event name="OnLeaveWindow"></event>
1180<event name="OnLeftDClick"></event>
1180<event name="OnLeftDown"></event>
1180<event name="OnLeftUp"></event>
1180<event name="OnMiddleDClick"></event>
1180<event name="OnMiddleDown"></event>
1180<event name="OnMiddleUp"></event>
1180<event name="OnMotion"></event>
1180<event name="OnMouseEvents"></event>
1181<event name="OnMouseWheel"></event>
1181<event name="OnPaint"></event>
1181<event name="OnRightDClick"></event>
1181<event name="OnRightDown"></event>
1181<event name="OnRightUp"></event>
1181<event name="OnSetFocus"></event>
1181<event name="OnSize"></event>
1181<event name="OnUpdateUI"></event>
1181</object>
1181</object>
1182</object>
1182</object>
1182</object>
1182</object>
1182</object>
1182</object>
1182</wxFormBuilder_Project>
11826
```

SUMMARY

Opening `uiproject.fbp` in wxFormBuilder today is like archaeological excavation—you can see exactly how Satoshi laid out every pixel of the original Bitcoin experience. The choices made here influenced every wallet that followed: the address display, the send dialog, the transaction list. This XML file is where cryptocurrency user experience began.

KEY TERMS

wxFormBuilder

A free visual designer for wxWidgets interfaces; Satoshi laid out every window in it, and its April 16, 2008 build generated uibase.cpp and uibase.h from this project file.

.fbp project file

wxFormBuilder's XML save format, recording each window, widget, and setting as nested object and property elements; this one describes sixteen top-level windows in 11,825 lines.

sizer

A wxWidgets layout manager that arranges child widgets in rows, columns, or grids and resizes them with the window; this project nests 58 box sizers and 3 flex-grid sizers.

Load From Resource

The bitmap source chosen for the toolbar buttons, which makes the generated code fetch images such as send20 by name from resources compiled into the executable rather than from files on disk.

SOURCES

1. Satoshi Nakamoto, *Bitcoin v0.1 released (Cryptography Mailing List / BitcoinTalk)* (January 9, 2009). Announced Windows-only for the first release. https://satoshi.nakamotoinstitute.org/emails/cryptography/15/
2. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
3. Satoshi Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (October 2008). https://bitcoin.org/bitcoin.pdf

util.cpp

374 lines CPP

THE TOOLBOX

Behind every elegant cryptographic system lies a collection of mundane but essential utilities: random number generation, time handling, file operations, string formatting, and debugging. This file is Bitcoin's toolbox—the unsexy infrastructure that makes everything else work.

The most critical function here is random number generation. Your private key's security depends entirely on unpredictable randomness. If an attacker can predict or bias the random number generator, they can steal your coins. Satoshi used OpenSSL's `RAND_bytes()`, which draws entropy from operating system sources (keyboard timing, disk activity, network packets). This code adds additional entropy sources to strengthen the randomness pool.

Also here: `FormatMoney()` and `ParseMoney()`—the functions that convert between human-readable "1.50" BTC and machine-friendly 150000000 satoshis. These handle edge cases like leading zeros and decimal precision that would otherwise cause subtle bugs. Getting money formatting wrong could mean losing funds to rounding errors.

The time synchronization code deserves special attention. Bitcoin blocks include timestamps, and nodes must agree roughly on what time it is. But trusting a single clock source would be a centralization risk. Satoshi's solution: take the median of peer clocks (the bounds meant to limit manipulation were left as a TODO in this version). This elegant approach—"never go to sea with two chronometers; take one or three"—appears in the time adjustment code below.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.

#include "headers.h"



bool fDebug = false;




// Init openssl library multithreading support
static HANDLE* lock_cs;

```

`win32_locking_callback` — maps OpenSSL's internal lock array onto Windows mutexes. Network, mining, and UI threads all call into OpenSSL concurrently; without this hook, OpenSSL 0.9.x is not thread-safe.[3]

```
void win32_locking_callback(int mode, int type, const char* file, int line)
{
if (mode & CRYPTO_LOCK)
WaitForSingleObject(lock_cs[type], INFINITE);
else
ReleaseMutex(lock_cs[type]);
}

// Init
```

`CInit`—the static initialization class. C++ guarantees that global objects are constructed before main() runs. Satoshi uses this pattern to initialize OpenSSL's thread safety and seed the random number generator at startup. The `instance_of_cinit` object at line 52 triggers this constructor automatically when Bitcoin launches. This is a common C++ idiom for initialization that must happen exactly once, before any other code runs.

```
class CInit
{
public:
CInit()
{
// Init openssl library multithreading support
        lock_cs = (HANDLE*)OPENSSL_malloc(CRYPTO_num_locks() * sizeof(HANDLE));
for (int i = 0; i < CRYPTO_num_locks(); i++)
            lock_cs[i] = CreateMutex(NULL,FALSE,NULL);
CRYPTO_set_locking_callback(win32_locking_callback);

// Seed random number generator with screen scrape and other hardware sources
```

`RAND_screen()`—Windows-specific entropy harvesting. This OpenSSL function first runs `RAND_poll`, which gathers the Windows CryptoAPI generator, process and heap lists, the cursor position, and other system state, then tries to hash the screen's pixels. The screen half never ran here: in the OpenSSL 0.9.8h build v0.1.0 shipped with, a reversed test made it return early for ordinary desktop programs, a "logical bug" OpenSSL fixed in 2010 (see the OpenSSL appendix).[4] Satoshi's paranoia about entropy was justified—weak RNG caused the 2013 Android Bitcoin wallet vulnerability that drained users' funds.[1]

```
RAND_screen();

// Seed random number generator with perfmon data
```

```
41 RandAddSeed(true);
42 }
43 ~CInit()
44 {
45 // Shutdown openssl library multithreading support
46 CRYPTO_set_locking_callback(NULL);
47 for (int i =0 ; i < CRYPTO_num_locks(); i++)
48 CloseHandle(lock_cs[i]);
49 OPENSSL_free(lock_cs);
50 }
51 }
52 instance_of_cinit;
53
54
55
56
```

RandAddSeed()—Satoshi's defense-in-depth for randomness. Even though OpenSSL seeds from system entropy, this function adds more: CPU performance counters (nanosecond-precision timing) and Windows performance monitor data (250KB of system metrics). The `fPerfmon` parameter controls whether to harvest the expensive perfmon data—only done every 5 minutes or at startup. Why so paranoid? Private key generation calls this. If an attacker could predict the RNG output, they could reconstruct private keys and steal all coins. The 2010 PlayStation 3 ECDSA key leak happened because Sony reused a 'random' value—Satoshi clearly understood this risk.[2]

```
CALLED FROM CDB::CDB db.cpp p. 120 · CDB::Close db.cpp p. 120 · ThreadSocketHandler2 net.cpp p. 333 ·
CMainFrame::OnMouseEvents ui.cpp p. 482
57 void RandAddSeed(bool fPerfmon)
58 {
59 // Seed with CPU performance counter
60     LARGE_INTEGER PerformanceCount;
61 QueryPerformanceCounter(&PerformanceCount);
```

RAND_add()—feeds entropy with a quality estimate. The third parameter (1.5) tells OpenSSL 'this data contributes about 1.5 bytes of entropy.' Conservative estimates are safer—overestimating entropy quality weakens the RNG. The performance counter has high precision but limited unpredictability, hence the modest estimate.

```
62 RAND_add(&PerformanceCount, sizeof(PerformanceCount), 1.5);
63 memset(&PerformanceCount, 0, sizeof(PerformanceCount));
64
65 static int64 nLastPerfmon;
66 if (fPerfmon || GetTime() > nLastPerfmon + 5 * 60)
67 {
68         nLastPerfmon = GetTime();
69
70 // Seed with the entire set of perfmon data
71 unsigned char pdata[250000];
72 memset(pdata, 0, sizeof(pdata));
73 unsigned long nSize = sizeof(pdata);
```

Windows Performance Data—an unusual but clever entropy source. `HKEY_PERFORMANCE_DATA` is a special registry key that returns live system metrics: CPU usage, memory stats, disk activity, network counters. This data is then SHA256-hashed to extract randomness. The 250KB buffer accommodates the substantial data returned. Satoshi adds entropy proportional to data size: `min(nSize/500.0, sizeof(hash))` conservatively estimates 1 byte of entropy per 500 bytes of performance data.

```
74 long ret = RegQueryValueEx(HKEY_PERFORMANCE_DATA, "Global", NULL, NULL, pdata, &nSize);
75 RegCloseKey(HKEY_PERFORMANCE_DATA);
```

```
if (ret == ERROR_SUCCESS)
{
            uint256 hash;
SHA256(pdata, nSize, (unsigned char*)&hash);
RAND_add(&hash, sizeof(hash), min(nSize/500.0, (double)sizeof(hash)));
            hash = 0;
memset(pdata, 0, nSize);
printf("RandAddSeed() got %d bytes of performance data\n", nSize);
}
}
}
```

Satoshi's original comment explains this function: 'Safer snprintf'. The standard C `snprintf()` has platform-dependent behavior regarding null termination and return values. This wrapper guarantees: (1) output is always null-terminated even when truncated, (2) return value is actual characters written. Buffer overflows have caused countless security vulnerabilities; this defensive coding style appears throughout Bitcoin.

```
// Safer snprintf
//  - prints up to limit-1 characters
//  - output string is always null terminated even if limit reached
//  - return value is the number of characters actually printed
int my_snprintf(char* buffer, size_t limit, const char* format, ...)
{
if (limit == 0)
return 0;
    va_list arg_ptr;
va_start(arg_ptr, format);
int ret = _vsnprintf(buffer, limit, format, arg_ptr);
va_end(arg_ptr);
if (ret < 0 || ret >= limit)
{
        ret = limit - 1;
        buffer[limit-1] = 0;
}
return ret;
}
```

`strprintf()`—dynamic string formatting that returns a `std::string` of whatever length the output needs. It first tries a 50KB buffer on the stack, which handles typical cases without any heap allocation; if the output doesn't fit, the `loop` macro (a Satoshi-ism for `for(;;)`) keeps doubling a heap buffer until it does. This function is called throughout Bitcoin for debug logging and error messages.

CALLED FROM CDB::CDB db.cpp p. 119 · ThreadIRCSeed irc.cpp p. 166 · GetAppDir main.cpp p. 225 · OpenBlockFile main.cpp p. 225 · SendMoney main.cpp p. 249 · CTxIn (inline) main.h p. 261 · *and 19 more*

```
string strprintf(const char* format, ...)
{
char buffer[50000];
char* p = buffer;
int limit = sizeof(buffer);
int ret;
```

```
    loop
{
        va_list arg_ptr;
va_start(arg_ptr, format);
        ret = _vsnprintf(p, limit, format, arg_ptr);
va_end(arg_ptr);
if (ret >= 0 && ret < limit)
break;
if (p != buffer)
delete p;
        limit *= 2;
        p = new char[limit];
if (p == NULL)
throw std::bad_alloc();
}
#ifdef _MSC_VER
// msvc optimisation
if (p == buffer)
return string(p, p+ret);
#endif
    string str(p, p+ret);
if (p != buffer)
delete p;
return str;
}


```

`error()`—centralized error reporting. Always returns false, allowing patterns like `return error("something failed")`. The error is logged to console. In production code, you'd see these messages in Bitcoin's debug.log file—invaluable for diagnosing issues in the field.

```
CALLED FROM CTxDB::LoadBlockIndex db.cpp p. 126 · CTransaction::AcceptTransaction main.cpp p. 209 ·
CTransaction::DisconnectInputs main.cpp p. 215 · CTransaction::ConnectInputs main.cpp p. 216 ·
CTransaction::ClientConnectInputs main.cpp p. 217 · Reorganize main.cpp p. 219 · and 9 more
bool error(const char* format, ...)
{
char buffer[50000];
int limit = sizeof(buffer);
    va_list arg_ptr;
va_start(arg_ptr, format);
int ret = _vsnprintf(buffer, limit, format, arg_ptr);
va_end(arg_ptr);
if (ret < 0 || ret >= limit)
{
        ret = limit - 1;
        buffer[limit-1] = 0;
}
printf("ERROR: %s\n", buffer);
return false;
}


CALLED FROM CMyApp::OnInit ui.cpp p. 515 · CMyApp::OnExceptionInMainLoop ui.cpp p. 518 ·
CMyApp::OnUnhandledException ui.cpp p. 518
void PrintException(std::exception* pex, const char* pszThread)
{
char pszModule[260];
    pszModule[0] = '\0';
GetModuleFileName(NULL, pszModule, sizeof(pszModule));
_strlwr(pszModule);
char pszMessage[1000];
if (pex)
snprintf(pszMessage, sizeof(pszMessage),
"EXCEPTION: %s       \n%s       \n%s in %s       \n", typeid(*pex).name(), pex->what(), pszModule,
pszThread);
else
snprintf(pszMessage, sizeof(pszMessage),
```

```
181 "UNKNOWN EXCEPTION       \n%s in %s       \n", pszModule, pszThread);
182 printf("\n\n**********\n%s", pszMessage);
183 if (wxTheApp)
184 wxMessageBox(pszMessage, "Error", wxOK | wxICON_ERROR);
185 throw;
186 //DebugBreak();
187 }
188
189
```

CALLED FROM RecvLineIRC irc.cpp p. 164 · ThreadIRCSeed irc.cpp p. 167 · CViewProductDialog::UpdateProductDisplay ui.cpp p. 512

```
190 void ParseString(const string& str, char c, vector<string>& v)
191 {
192 unsigned int i1 = 0;
193 unsigned int i2;
194 do
195 {
196         i2 = str.find(c, i1);
197         v.push_back(str.substr(i1, i2-i1));
198         i1 = i2+1;
199 }
200 while (i2 != str.npos);
201 }
202
203
```

FormatMoney()—converts internal satoshi representation to human-readable format. Bitcoin stores all amounts as integers (satoshis) to avoid floating-point rounding errors. This function divides by CENT (1,000,000 satoshis), formats with two decimal places, and inserts thousand separators. The fPlus parameter adds '+' prefix for positive values, useful in transaction displays showing debits vs credits. Getting this wrong could cause users to send wrong amounts—the code carefully handles negative values and edge cases.

CALLED FROM SelectCoins main.cpp p. 247 · SendMoney main.cpp p. 249 · CMainFrame::CMainFrame ui.cpp p. 480 · CMainFrame::InsertTransaction ui.cpp p. 484 · CMainFrame::OnPaintListCtrl ui.cpp p. 488 · CTxDetailsDialog::CTxDetailsDialog ui.cpp p. 492 · *and 7 more*

```
204 string FormatMoney(int64 n, bool fPlus)
205 {
```

Division by CENT, not COIN. Bitcoin has two constants: COIN (100,000,000 satoshis = 1 BTC) and CENT (1,000,000 satoshis = 0.01 BTC). The division here means FormatMoney() displays with 2 decimal places (cents), not 8 (satoshis). This was a v0.1.0 design choice—early Bitcoin showed values like '50.00 BTC' rather than '50.00000000 BTC'. Modern Bitcoin software often shows more precision.

```
206     n /= CENT;
207     string str = strprintf("%I64d.%02I64d", (n > 0 ? n : -n)/100, (n > 0 ? n : -n)%100);
208 for (int i = 6; i < str.size(); i += 4)
209 if (isdigit(str[str.size() - i - 1]))
210             str.insert(str.size() - i, 1, ',');
211 if (n < 0)
212         str.insert((unsigned int)0, 1, '-');
213 else if (fPlus && n > 0)
214         str.insert((unsigned int)0, 1, '+');
215 return str;
216 }
217
```

`ParseMoney()`—the inverse of `FormatMoney()`. Converts user-entered strings like '1,234.56' to satoshis. Handles thousand separators, leading/trailing whitespace, and validates format — strictly: if there is a decimal point, exactly two digits must follow, so '1.5' and '1.505' are both rejected. The check `strWhole.size() > 17` caps the whole-number part at 17 digits, a crude guard against overflow that has nothing to do with the 21 million BTC cap (only 8 digits). Returns false for malformed input rather than throwing exceptions, following C convention.

CALLED FROM COptionsDialog::OnKillFocusTransactionFee ui.cpp p. 494 · COptionsDialog::OnButtonOK ui.cpp p. 494 · CSendDialog::OnKillFocusAmount ui.cpp p. 496 · CSendDialog::OnButtonSend ui.cpp p. 496 · CMyApp::OnInit2 ui.cpp p. 517

```
218 bool ParseMoney(const char* pszIn, int64& nRet)
219 {
220     string strWhole;
221     int64 nCents = 0;
222 const char* p = pszIn;
223 while (isspace(*p))
224         p++;
225 for (; *p; p++)
226 {
227 if (*p == ',' && p > pszIn && isdigit(p[-1]) && isdigit(p[1]) && isdigit(p[2]) && isdigit(p[3]) &&
    !isdigit(p[4]))
228 continue;
229 if (*p == '.')
230 {
231             p++;
232 if (!isdigit(p[0]) || !isdigit(p[1]))
233 return false;
234             nCents = atoi64(p);
235 if (nCents < 0 || nCents > 99)
236 return false;
237             p += 2;
238 break;
239 }
240 if (isspace(*p))
241 break;
242 if (!isdigit(*p))
243 return false;
244         strWhole.insert(strWhole.end(), *p);
245 }
246 for (; *p; p++)
247 if (!isspace(*p))
248 return false;
249 if (strWhole.size() > 17)
250 return false;
251     int64 nWhole = atoi64(strWhole);
252     int64 nValue = nWhole * 100 + nCents;
```

Overflow check: `nValue / 100 != nWhole`. After multiplying, division should recover the original. If not, multiplication overflowed. But the check covers only the ×100 step: the next line, `nValue *= CENT`, multiplies by another 1,000,000 with no check at all, so any whole amount above about 92 billion BTC silently wraps around. Harmless in practice (no one has that balance, and sending is checked against it), but the guard is incomplete.

```
253 if (nValue / 100 != nWhole)
254 return false;
255     nValue *= CENT;
256     nRet = nValue;
257 return true;
258 }
259
260
261
262
```

```
264
265
266
267
268
269 bool FileExists(const char* psz)
270 {
271 #ifdef WIN32
272 return GetFileAttributes(psz) != -1;
273 #else
274 return access(psz, 0) != -1;
275 #endif
276 }
277
278 int GetFilesize(FILE* file)
279 {
280 int nSavePos = ftell(file);
281 int nFilesize = -1;
282 if (fseek(file, 0, SEEK_END) == 0)
283         nFilesize = ftell(file);
284 fseek(file, nSavePos, SEEK_SET);
285 return nFilesize;
286 }
287
288
289
290
291
292
293
294
```

`GetRand()`—generates cryptographically secure random numbers. This is the function called when selecting peers, choosing random IRC nicknames, and anywhere else uniform randomness is needed — though notably not for private keys, which OpenSSL generates internally. The `do-while` loop implements rejection sampling: since `RAND_bytes()` produces uniform random bits, but we want uniform distribution over `[0, nMax)`, we must reject values that would bias the result. The calculation `nRange = (_UI64_MAX / nMax) * nMax` finds the largest multiple of `nMax` that fits in 64 bits. Values >= `nRange` would favor lower results if taken mod `nMax`, so they're rejected and we try again.

CALLED FROM ThreadIRCSeed irc.cpp p. 166 · ProcessMessage main.cpp p. 231 · CUser::AddAtom market.cpp p. 306 · ThreadOpenConnections2 net.cpp p. 337 · CNode (inline) net.h p. 361 · ThreadRandSendTest ui.cpp p. 519 · *and 2 more*

```
295 uint64 GetRand(uint64 nMax)
296 {
297 if (nMax == 0)
298 return 0;
299
300 // The range of the random source must be a multiple of the modulus
301 // to give every possible output value an equal possibility
302     uint64 nRange = (_UI64_MAX / nMax) * nMax;
303     uint64 nRand = 0;
304 do
305 RAND_bytes((unsigned char*)&nRand, sizeof(nRand));
306 while (nRand >= nRange);
307 return (nRand % nMax);
308 }
309
310
311
312
313
314
315
316
317
```

The chronometers quote—Satoshi's comment reveals his thinking on distributed time. The saying 'never go to sea with two chronometers; take one or three' means: with one clock, you trust it; with two disagreeing clocks, you don't know which is right; with three, you take the majority. Bitcoin implements this philosophy: rather than trusting any single time source, it takes the median of connected peers' clocks. This prevents a single malicious peer from manipulating timestamps, which could affect block validity checking.

```
319 //
320 // "Never go to sea with two chronometers; take one or three."
321 // Our three chronometers are:
322 //  - System clock
323 //  - Median of other server's clocks
324 //  - NTP servers
325 //
326 // note: NTP isn't implemented yet, so until then we just use the median
327 //  of other nodes clocks to correct ours.
328 //
329
```

CALLED FROM RelayWalletTransactions main.cpp p. 212 · SendMessages main.cpp p. 238 · BitcoinMiner main.cpp p. 243 · CReview::AcceptReview market.cpp p. 307 · ConnectNode net.cpp p. 330 · ThreadSocketHandler2 net.cpp p. 332 · *and 8 more*

```
330 int64 GetTime()
331 {
332 return time(NULL);
333 }
334
335 static int64 nTimeOffset = 0;
336
```

`GetAdjustedTime()`—returns current time adjusted by network consensus. `nTimeOffset` is the difference between local clock and peer median. If your clock is 5 minutes fast, `nTimeOffset` is -300 seconds. Blocks are rejected if their timestamp is too far in the future—this adjustment prevents your node from rejecting valid blocks due to local clock drift.

CALLED FROM AddToWallet main.cpp p. 203 · CBlock::CheckBlock main.cpp p. 222 · ProcessMessage main.cpp p. 236 · BitcoinMiner main.cpp p. 242 · CAddress (inline) net.h p. 353 · CNode (inline) net.h p. 359 · *and 1 more*

```
337 int64 GetAdjustedTime()
338 {
339 return GetTime() + nTimeOffset;
340 }
341
```

`AddTimeData()`—collects time samples from peers. When connecting to a peer, nodes exchange timestamps. This function tracks offsets (peer_time - local_time), deduplicates by IP, and computes the median. The logic waits for 5+ samples — your own clock counts as the first one (the 0 pushed on line 354), so four peers are enough — and only updates on odd counts (ensuring a true median exists). If median offset exceeds 5 minutes, the code notes 'go to NTP servers'—though NTP wasn't actually implemented in v0.1. In this version the branch does nothing beyond the TODO: the median is applied regardless of its size, so the safeguard was aspirational.

CALLED FROM ProcessMessage main.cpp p. 232

```
342 void AddTimeData(unsigned int ip, int64 nTime)
343 {
344     int64 nOffsetSample = nTime - GetTime();
345
346 // Ignore duplicates
347 static set<unsigned int> setKnown;
348 if (!setKnown.insert(ip).second)
```

```
// Add data
static vector<int64> vTimeOffsets;
if (vTimeOffsets.empty())
        vTimeOffsets.push_back(0);
    vTimeOffsets.push_back(nOffsetSample);
printf("Added time data, samples %d, ip %08x, offset %+I64d (%+I64d minutes)\n", vTimeOffsets.size(), ip, vTimeOffsets.back(), vTimeOffsets.back()/60);
if (vTimeOffsets.size() >= 5 && vTimeOffsets.size() % 2 == 1)
{
sort(vTimeOffsets.begin(), vTimeOffsets.end());
        int64 nMedian = vTimeOffsets[vTimeOffsets.size()/2];
        nTimeOffset = nMedian;
```

Safety limit: offsets beyond ±5 minutes trigger a TODO comment about NTP. Satoshi recognized the vulnerability: if an attacker controlled many peer connections, they could manipulate your time offset. The 5-minute cap exists here only as a comment — enforcement was left for later versions, which added stricter checks and warnings when clock offset is large.

```
if ((nMedian > 0 ? nMedian : -nMedian) > 5 * 60)
{
// Only let other nodes change our clock so far before we
// go to the NTP servers
/// todo: Get time from NTP servers, then set a flag
///    to make sure it doesn't get changed again
}
foreach(int64 n, vTimeOffsets)
printf("%+I64d  ", n);
printf("|  nTimeOffset = %+I64d  (%+I64d minutes)\n", nTimeOffset, nTimeOffset/60);
}
}

```

SUMMARY

These mundane-seeming functions are critical infrastructure. The random number generator secures every private key ever generated. The money formatting functions prevent users from accidentally sending wrong amounts. The time synchronization enables consensus without a central time authority.

Satoshi's paranoid approach to randomness proved prescient. In 2013, a weak RNG in Android's Java SecureRandom class allowed attackers to derive private keys from signed transactions, draining wallets.[1] In 2010, Sony's PlayStation 3 signing key was compromised because they used a static 'random' value in ECDSA signatures.[2] Bitcoin's defense-in-depth—multiple entropy sources, conservative estimates, rejection sampling—has prevented similar disasters.

The time synchronization code elegantly solves one of distributed systems' oldest problems: agreeing on the time without trusting any single authority. By taking medians of peer clocks, Bitcoin achieves rough time consensus sufficient for block timestamp validation without introducing centralization through NTP servers or trusted time sources. The approach is both robust and conservative—exactly what you want for infrastructure that must run without human intervention for decades.

KEY TERMS

entropy

Unpredictable input, such as timing jitter or system statistics, fed into a random number generator so its output cannot be guessed by an attacker.

RandAddSeed

The function that mixes the CPU performance counter, and at most every five minutes a hash of Windows performance data, into OpenSSL's random pool.

rejection sampling

Discarding random draws that fall beyond the largest multiple of the desired range, so that taking the remainder gives every result an equal chance.

strprintf

A printf-style formatter that returns a std::string, trying a 50,000-byte stack buffer first and doubling a heap buffer until the output fits.

error()

A logging helper that prints its formatted message prefixed with "ERROR:" and always returns false, so callers can write return error(...) in one line.

FormatMoney

The function that turns an integer amount of satoshis into display text with two decimal places, comma thousands separators, and an optional plus sign.

GetAdjustedTime

The node's notion of network time: the local system clock plus the median offset reported by peers, used for timestamps and the two-hour future-block limit.

AddTimeData

The function that records each new peer's clock offset once per IP address and, after at least five samples, adopts their median as the node's time offset.

SOURCES

1. Bitcoin developers / Google, *Android SecureRandom ECDSA nonce reuse* (August 2013). Non-deterministic RNG led to private-key recovery from repeated r values. https://bitcoin.org/en/alert/2013-08-11-android
2. fail0verflow, *Console Hacking 2010 — PS3 ECDSA nonce reuse* (December 2010). Classic demonstration that reusing ECDSA nonces leaks private keys. https://events.ccc.de/congress/2010/Fahrplan/events/4087.en.html

3. OpenSSL Project, *OpenSSL License / SSLeay License* (1998–2009). https://www.openssl.org/source/license-openssl-ssleay.txt
4. Andy Polyakov (OpenSSL Project), *rand_win.c: fix logical bug in readscreen* (March 22, 2010). Reverses the service test so the screen is skipped only for Windows services; first released in OpenSSL 0.9.8n. https://github.com/openssl/openssl/commit/c3484e0268cfa7b7c6db81e01fc0f837bf375e52

util.h

400 lines CPP

THE SWISS ARMY KNIFE

Every large C++ project has a utility header -- the grab-bag of macros, type definitions, helper functions, and infrastructure that everything else depends on. In Bitcoin, that file is `util.h`: 400 lines that define the platform-abstraction layer, thread synchronization primitives, string formatting, cryptographic hashing, and the debug logging system. Nearly every other file in Bitcoin's source includes this header (via `headers.h`), making it the most widely depended-upon code in the entire project.

Satoshi developed Bitcoin on Windows using Microsoft Visual C++, but from the start he wrote code that could be ported. The first 50 lines of this file are a compatibility layer: `int64` type definitions that work across MSVC, Borland, and GCC; a workaround for a decade-old MSVC scoping bug; platform-specific format specifiers for 64-bit integers. These are the tedious details that make cross-platform C++ possible. Satoshi handled them once, here, so the rest of the codebase could pretend they didn't exist.

The file's most consequential contribution is the `CRITICAL_BLOCK` macro -- Satoshi's hand-rolled RAII locking primitive. Bitcoin is heavily multi-threaded: the three network threads, the mining thread, and the UI thread all share global state. Every access to shared data (`mapTransactions`, `mapBlockIndex`, `mapWallet`) must be wrapped in a `CRITICAL_BLOCK`. The macro's clever double-for-loop construction ensures the lock is always released, even if an exception is thrown, and catches accidental `break` statements with an assertion. This pattern anticipates C++11's `std::lock_guard` by several years.

The hashing functions at the bottom of the file -- `Hash()`, `SerializeHash()`, and `Hash160()` -- implement Bitcoin's two signature hash constructions: double-SHA256 (used for transaction and block hashes) and SHA256+RIPEMD160 (used for Bitcoin addresses). These few dozen lines of code produce every hash in the entire system.

```
// Copyright (c) 2009 Satoshi Nakamoto
// Distributed under the MIT/X11 software license, see the accompanying
// file license.txt or http://www.opensource.org/licenses/mit-license.php.


```

`typedef __int64 int64` -- platform-specific 64-bit integer types. MSVC and Borland use `__int64`; GCC uses `long long`. Bitcoin needs 64-bit integers everywhere: satoshi amounts (up to 21 million BTC = 2.1 quadrillion satoshis), block timestamps, difficulty targets. Satoshi unified these behind `int64` and `uint64` so the rest of the code never has to think about compiler differences. C++11 would later standardize `int64_t`, making this abstraction unnecessary.

```
#if defined(_MSC_VER) || defined(__BORLANDC__)
typedef __int64  int64;
typedef unsigned __int64  uint64;
#else
typedef long long  int64;
typedef unsigned long long  uint64;
#endif
```

`#define for if (false) ; else for` -- a workaround for a notorious MSVC 6.0 bug where variables declared in `for` loop initializers leaked into the enclosing scope, violating the C++ standard. This macro wraps every `for` loop in an `if/else` block that creates a new scope, confining the loop variable. The condition `if (false) ;` is always optimized away. This hack was well-known in the early 2000s Windows C++ community; Satoshi included it defensively even though he likely used a newer compiler.

```
#if defined(_MSC_VER) && _MSC_VER < 1300
#define for if (false) ; else for
#endif

#ifndef _MSC_VER
#define __forceinline inline
#endif

```

`#define foreach BOOST_FOREACH` -- Satoshi's shorthand for Boost's iteration macro. Before C++11 range-based `for` loops, iterating over STL containers required verbose `iterator` boilerplate. `foreach` is used hundreds of times throughout Bitcoin: `foreach(const CTransaction& tx, block.vtx)` reads almost like English. Satoshi also defined `loop` as `for (;;)` -- an infinite loop alias used in the mining and network threads.

```
#define foreach BOOST_FOREACH
#define loop for (;;)
```

`BEGIN(a)` and `END(a)` -- macros that return pointers to the start and one-past-the-end of any object's memory representation. Used throughout the hashing code to pass structs directly to SHA256: `Hash(BEGIN(header), END(header))` hashes the raw bytes of a block header without serialization overhead. `UBEGIN/UEND` are the unsigned-char variants. `ARRAYLEN` computes array length at compile time -- the classic `sizeof(array)/sizeof(array[0])` idiom.

```
#define BEGIN(a) ((char*)&(a))
#define END(a) ((char*)&((&(a))[1]))
#define UBEGIN(a) ((unsigned char*)&(a))
#define UEND(a) ((unsigned char*)&((&(a))[1]))
#define ARRAYLEN(array) (sizeof(array)/sizeof((array)[0]))

```

```
#ifdef _WINDOWS
```

`#define printf OutputDebugStringF` -- on Windows, Satoshi *redefines printf itself*. Every `printf()` call in the entire codebase is silently redirected to `OutputDebugStringF()` (defined on line 232), which -- in debug builds, the makefile's default -- writes to both the debug log file and the Windows debug output stream. This is why Bitcoin's source is littered with `printf()` calls that appear to print to the console but actually write to `debug.log`. It is an audacious macro that would horrify most C++ style guides, but it worked — and left a historical record: Hal Finney's January 10, 2009 crash post attached his `debug.log`, later used to show he joined near block 49 rather than at launch.[23]

```
#define printf OutputDebugStringF
#endif

#ifdef snprintf
#undef snprintf
#endif
```

`#define snprintf my_snprintf` -- replaces the standard `snprintf` with Satoshi's wrapper (implemented in `util.cpp`). The wrapper ensures consistent behavior across platforms where `snprintf` had different edge-case semantics, particularly around buffer overflow and null termination. Defensive programming: if the underlying function might behave differently on MSVC vs. GCC, wrap it and enforce uniform behavior.

```
#define snprintf my_snprintf

```

`PRId64`, `PRIu64`, `PRIx64` -- format specifiers for 64-bit integers in printf-style functions. MSVC uses `"I64d"`; POSIX systems use `"lld"`. Without these, printing a satoshi amount would produce garbage on one platform or the other. Used as: `printf("%" PRId64, nSatoshis)`. The C99 standard defined these in `<inttypes.h>`, but MSVC did not support C99 until 2013.

```
#ifndef PRId64
#if defined(_MSC_VER) || defined(__BORLANDC__) || defined(__MSVCRT__)
#define PRId64 "I64d"
#define PRIu64 "I64u"
#define PRIx64 "I64x"
#else
#define PRId64 "lld"
#define PRIu64 "llu"
#define PRIx64 "llx"
#endif
#endif

// This is needed because the foreach macro can't get over the comma in pair<t1, t2>
```

`PAIRTYPE(t1, t2)` -- a macro to work around a preprocessor limitation. The C preprocessor treats commas as argument separators, so `foreach(pair<int, string> item, myMap)` would be parsed as three arguments, not two. `PAIRTYPE` wraps the pair declaration in a single macro argument. This is one of those small annoyances of pre-C++11 template programming that C++11 range-for loops eliminated entirely.

```
#define PAIRTYPE(t1, t2)    pair<t1, t2>

// Used to bypass the rule against non-const reference to temporary
// where it makes sense with wrappers such as CFlatData or CTxDB
```

`REF()` -- a template function that casts away const from a reference. The comment explains: it bypasses the C++ rule against binding non-const references to temporaries. Used with wrapper types like `CFlatData` and `CTxDB` where Satoshi knows the temporary will outlive the reference. This is technically undefined behavior in strict C++, but every major compiler handles it as expected. Pragmatism over purity.

```
55 template<typename T>
   CALLED FROM  net.h net.h p. 350 · serialize.h serialize.h p. 429
56 inline T& REF(const T& val)
57 {
58 return (T&)val;
59 }
60
61
62
63
64
65
66
67
68
69 extern bool fDebug;
70
```

Utility function declarations. `RandAddSeed()` feeds entropy into OpenSSL's random number generator -- called at startup and periodically to strengthen key generation. `strprintf()` is a sprintf that returns a `std::string` (still varargs, so not type-checked). `FormatMoney()` converts satoshi integers to human-readable BTC strings with two decimal places. `GetRand()` returns a cryptographically secure random number. `GetAdjustedTime()` returns the current time adjusted by the median offset of all connected peers -- Bitcoin's network time.

```
71 void RandAddSeed(bool fPerfmon=false);
72 int my_snprintf(char* buffer, size_t limit, const char* format, ...);
73 string strprintf(const char* format, ...);
74 bool error(const char* format, ...);
75 void PrintException(std::exception* pex, const char* pszThread);
76 void ParseString(const string& str, char c, vector<string>& v);
77 string FormatMoney(int64 n, bool fPlus=false);
78 bool ParseMoney(const char* pszIn, int64& nRet);
79 bool FileExists(const char* psz);
80 int GetFilesize(FILE* file);
81 uint64 GetRand(uint64 nMax);
82 int64 GetTime();
83 int64 GetAdjustedTime();
```

`AddTimeData()` -- collects time offsets from peers to compute network-adjusted time. Each peer reports its local time during the version handshake; Bitcoin computes the median offset across all peers and adds it to local time. This prevents a node with a wrong clock from rejecting valid blocks or creating blocks with invalid timestamps. The median (not mean) is used because it resists manipulation by a minority of dishonest peers.

```
84 void AddTimeData(unsigned int ip, int64 nTime);
85
86
87
88
89
90
91
92
```

```
93
94
95
96
97 // Wrapper to automatically initialize critical section
98 // Could use wxCriticalSection for portability, but it doesn't support TryEnterCriticalSection
```

`CCriticalSection` -- Satoshi's mutex wrapper around the Windows `CRITICAL_SECTION` API. The comment on line 98 explains why he didn't use wxWidgets' `wxCriticalSection`: it lacks `TryEnterCriticalSection()`, needed for non-blocking lock attempts. The `pszFile` and `nLine` fields (lines 104-105) store the source location of whoever currently holds the lock -- invaluable for debugging deadlocks. When a thread holds a lock, you can inspect the object to see exactly which file and line acquired it.

```
99 class CCriticalSection
100 {
101 protected:
102     CRITICAL_SECTION cs;
103 public:
104 char* pszFile;
105 int nLine;
106 explicit CCriticalSection() { InitializeCriticalSection(&cs); }
107 ~CCriticalSection() { DeleteCriticalSection(&cs); }
108 void Enter() { EnterCriticalSection(&cs); }
109 void Leave() { LeaveCriticalSection(&cs); }
110 bool TryEnter() { return TryEnterCriticalSection(&cs); }
111     CRITICAL_SECTION* operator&() { return &cs; }
112 };
113
114 // Automatically leave critical section when leaving block, needed for exception safety
```

`CCriticalBlock` -- the RAII lock guard. The constructor acquires the lock; the destructor releases it. This guarantees the lock is released even if an exception is thrown inside the critical section -- the C++ stack unwinding mechanism calls the destructor automatically. Two constructors (lines 120-121) accept either a raw `CRITICAL_SECTION` or a `CCriticalSection` wrapper, making the class work with both Windows primitives and Satoshi's wrapper.

```
115 class CCriticalBlock
116 {
117 protected:
118     CRITICAL_SECTION* pcs;
119 public:
120 CCriticalBlock(CRITICAL_SECTION& csIn) { pcs = &csIn; EnterCriticalSection(pcs); }
121 CCriticalBlock(CCriticalSection& csIn) { pcs = &csIn; EnterCriticalSection(pcs); }
122 ~CCriticalBlock() { LeaveCriticalSection(pcs); }
123 };
124
125 // WARNING: This will catch continue and break!
126 // break is caught with an assertion, but there's no way to detect continue.
127 // I'd rather be careful than suffer the other more error prone syntax.
128 // The compiler will optimise away all this loop junk.
```

`CRITICAL_BLOCK` -- Satoshi's most-used synchronization primitive, a macro that creates a scoped lock. The double-for-loop construction is clever: the outer loop runs exactly once (controlled by `fcriticalblockonce`), and its exit condition includes an assertion that catches `break` statements -- breaking out of a `CRITICAL_BLOCK` would release the lock prematurely, a subtle and dangerous bug. The inner loop creates the `CCriticalBlock` RAII guard and records the file/line for debugging. The comment on line 125 warns: `continue` cannot be caught, so the programmer must be careful.

```
129 #define CRITICAL_BLOCK(cs)\
```

```
130 for (bool fcriticalblockonce=true; fcriticalblockonce; assert(("break caught by CRITICAL_BLOCK!",
    !fcriticalblockonce)), fcriticalblockonce=false)  \
131 for (CCriticalBlock criticalblock(cs); fcriticalblockonce && (cs.pszFile=__FILE__, cs.nLine=__LINE__,
    true); fcriticalblockonce=false, cs.pszFile=NULL, cs.nLine=0)
132
```

CTryCriticalBlock -- a non-blocking lock attempt. If the lock is already held by another thread, TryEnterCriticalSection() returns immediately with failure instead of waiting. Used by the network threads, which cannot stall on a lock in the middle of a select loop: `TRY_CRITICAL_BLOCK(pnode→cs_vSend) { ... }`. The miner never uses it — it takes the blocking `CRITICAL_BLOCK(cs_main)` instead. The Entered() method (line 141) lets the caller check whether the lock was acquired. This is why Satoshi couldn't use wxWidgets' critical section -- he needed this try-lock capability.

```
133 class CTryCriticalBlock
134 {
135 protected:
136     CRITICAL_SECTION* pcs;
137 public:
138 CTryCriticalBlock(CRITICAL_SECTION& csIn) { pcs = (TryEnterCriticalSection(&csIn) ? &csIn : NULL); }
139 CTryCriticalBlock(CCriticalSection& csIn) { pcs = (TryEnterCriticalSection(&csIn) ? &csIn : NULL); }
140 ~CTryCriticalBlock() { if (pcs) LeaveCriticalSection(pcs); }
141 bool Entered() { return pcs != NULL; }
142 };
143
144 #define TRY_CRITICAL_BLOCK(cs)\
145 for (bool fcriticalblockonce=true; fcriticalblockonce; assert(("break caught by TRY_CRITICAL_BLOCK!",
    !fcriticalblockonce)), fcriticalblockonce=false)  \
146 for (CTryCriticalBlock criticalblock(cs); fcriticalblockonce && (fcriticalblockonce =
    criticalblock.Entered()) && (cs.pszFile=__FILE__, cs.nLine=__LINE__, true); fcriticalblockonce=false,
    cs.pszFile=NULL, cs.nLine=0)
147
148
149
150
151
152
153
154
155
156
157
158
```

String conversion utilities. i64tostr(), itostr(), atoi64(), atoi() -- Satoshi wrapped every standard conversion function to handle platform differences. The atoi64() implementations use `_atoi64` on MSVC and `strtoll` on GCC. roundint() (line 192) rounds a double to the nearest integer using the 'add 0.5 and truncate' technique. Nothing in v0.1.0 actually calls it -- amounts are parsed as integers by ParseMoney(), never as floating point.

```
    CALLED FROM  CEditReviewDialog::GetReview ui.cpp p. 514
159 inline string i64tostr(int64 n)
160 {
161 return strprintf("%"PRId64, n);
162 }
163
    CALLED FROM  DateTimeStr ui.cpp p. 477 · CProductsDialog::OnButtonSearch ui.cpp p. 506 ·
    CViewProductDialog::UpdateProductDisplay ui.cpp p. 511 · CEditReviewDialog::GetReview ui.cpp p. 514
164 inline string itostr(int n)
165 {
166 return strprintf("%d", n);
167 }
168
```

```
inline int64 atoi64(const char* psz)
{
#ifdef _MSC_VER
return _atoi64(psz);
#else
return strtoll(psz, NULL, 10);
#endif
}

inline int64 atoi64(const string& str)
{
#ifdef _MSC_VER
return _atoi64(str.c_str());
#else
return strtoll(str.c_str(), NULL, 10);
#endif
}

```

CALLED FROM CAddress (inline) net.h p. 353 · CViewProductDialog::UpdateProductDisplay ui.cpp p. 511 · CMyApp::OnInit2 ui.cpp p. 516

```
inline int atoi(const string& str)
{
return atoi(str.c_str());
}

inline int roundint(double d)
{
return (int)(d > 0 ? d + 0.5 : d - 0.5);
}

```

`HexStr()` and `HexNumStr()` -- hexadecimal formatting templates. `HexStr()` outputs bytes left-to-right with optional spaces (used for debug logging of raw data). `HexNumStr()` outputs bytes in reverse order (big-endian display of little-endian integers) with an optional `0x` prefix. In v0.1.0 its only caller is `script.h`, which uses it to print data pushed onto a script; block hashes and transaction IDs get their hex from `uint256`'s own `GetHex()`. The distinction matters: a uint256 stored as `01 00 ... 00` in memory displays as `0x00...0001`.

```
template<typename T>
```

CALLED FROM CTxIn (inline) main.h p. 261 · CScript (inline) script.h p. 417 · PrintHex p. 693

```
string HexStr(const T itbegin, const T itend, bool fSpaces=true)
{
const unsigned char* pbegin = (const unsigned char*)&itbegin[0];
const unsigned char* pend = pbegin + (itend - itbegin) * sizeof(itbegin[0]);
    string str;
for (const unsigned char* p = pbegin; p != pend; p++)
        str += strprintf((fSpaces && p != pend-1 ? "%02x " : "%02x"), *p);
return str;
}

template<typename T>
```

CALLED FROM ValueString script.h p. 413

```
string HexNumStr(const T itbegin, const T itend, bool f0x=true)
{
const unsigned char* pbegin = (const unsigned char*)&itbegin[0];
const unsigned char* pend = pbegin + (itend - itbegin) * sizeof(itbegin[0]);
    string str = (f0x ? "0x" : "");
for (const unsigned char* p = pend-1; p >= pbegin; p--)
        str += strprintf("%02X", *p);
return str;
}

template<typename T>
void PrintHex(const T pbegin, const T pend, const char* pszFormat="%s", bool fSpaces=true)
{
printf(pszFormat, HexStr(pbegin, pend, fSpaces).c_str());
}
```

```
224
225
226
227
228
229
230
231
```

`OutputDebugStringF()` -- Bitcoin's actual logging implementation, silently called whenever any code writes `printf()` (due to the macro on line 30). In debug builds (`__WXDEBUG__`, which the makefile turns on by default), it appends each call straight to `debug.log`, then feeds the Windows debug output stream (visible in Visual Studio's Output window) through a static 50KB buffer (line 250) that is flushed only on newlines, so that stream gets whole lines. The mutex `cs_OutputDebugStringF` (line 247) protects the buffer. In builds without `__WXDEBUG__`, no `debug.log` is written at all: output goes to the console (line 290), and only when no wxWidgets app is running. Those logs are historical artifacts: Finney's attached `debug.log` still lets researchers reconstruct peer discovery and block sync from day two.[24]

```
232 inline int OutputDebugStringF(const char* pszFormat, ...)
233 {
234 #ifdef __WXDEBUG__
235 // log file
236     FILE* fileout = fopen("debug.log", "a");
237 if (fileout)
238 {
239         va_list arg_ptr;
240 va_start(arg_ptr, pszFormat);
241 vfprintf(fileout, pszFormat, arg_ptr);
242 va_end(arg_ptr);
243 fclose(fileout);
244 }
245
246 // accumulate a line at a time
247 static CCriticalSection cs_OutputDebugStringF;
248 CRITICAL_BLOCK(cs_OutputDebugStringF)
249 {
250 static char pszBuffer[50000];
251 static char* pend;
252 if (pend == NULL)
253             pend = pszBuffer;
254         va_list arg_ptr;
255 va_start(arg_ptr, pszFormat);
256 int limit = END(pszBuffer) - pend - 2;
257 int ret = _vsnprintf(pend, limit, pszFormat, arg_ptr);
258 va_end(arg_ptr);
259 if (ret < 0 || ret >= limit)
260 {
261             pend = END(pszBuffer) - 2;
262 *pend++ = '\n';
263 }
264 else
265             pend += ret;
266 *pend = '\0';
267 char* p1 = pszBuffer;
268 char* p2;
269 while (p2 = strchr(p1, '\n'))
270 {
271             p2++;
272 char c = *p2;
273 *p2 = '\0';
274 OutputDebugString(p1);
275 *p2 = c;
276             p1 = p2;
277 }
278 if (p1 != pszBuffer)
279 memmove(pszBuffer, p1, pend - p1 + 1);
```

```
        pend -= (p1 - pszBuffer);
return ret;
}
#endif

if (!wxTheApp)
{
// print to console
        va_list arg_ptr;
va_start(arg_ptr, pszFormat);
vprintf(pszFormat, arg_ptr);
va_end(arg_ptr);
}
return 0;
}









inline void heapchk()
{
if (_heapchk() != _HEAPOK)
DebugBreak();
}

```

`IMPLEMENT_RANDOMIZE_STACK` -- a macro that randomizes the stack depth by recursing a random number of times (up to 50) before executing the actual thread function. The comment explains: 'Randomize the stack to help protect against buffer overrun exploits.' By varying the stack pointer's position, an attacker who discovers a buffer overflow cannot predict exactly where the return address is stored. This is a defense-in-depth measure that complements OS-level ASLR (Address Space Layout Randomization). Used on the network threads.

```
// Randomize the stack to help protect against buffer overrun exploits
#define IMPLEMENT_RANDOMIZE_STACK(ThreadFn)\
{                                                       \
static char nLoops;                                 \
if (nLoops <= 0)                                    \
            nLoops = GetRand(50) + 1;                       \
if (nLoops-- > 1)                                   \
{                                                   \
            ThreadFn;                                       \
return;                                         \
}                                                   \
}

```

`CATCH_PRINT_EXCEPTION` -- a macro wrapping try/catch that logs both `std::exception` and unknown exceptions via `PrintException()`. Used at the top level of the network thread functions, it looks like a safety net -- but it does not stop a crash. `PrintException()` logs the error with the thread's name, shows a message box, and then ends with a bare `throw;` (util.cpp line 185), re-throwing the same exception. It escapes the thread anyway, so the 'sleep and restart' loops wrapped around these threads in net.cpp never actually get to restart anything.

```
#define CATCH_PRINT_EXCEPTION(pszFn)\
catch (std::exception& e) {          \
PrintException(&e, (pszFn));     \
} catch (...) {                      \
PrintException(NULL, (pszFn));   \

```

```
328 }
329
330
331
332
333
334
335
336
337
```

`Hash()` -- Bitcoin's fundamental cryptographic primitive: **double SHA-256**. Takes arbitrary input bytes, hashes them with SHA-256, then hashes the result again with SHA-256. Every block hash, every transaction ID, every merkle tree node uses this function. Why hash twice? Satoshi never said; the usual explanation is protection against length-extension attacks: SHA-256 is vulnerable to an attacker who, given `H(m)`, can compute `H(m || padding || suffix)` without knowing `m`. The second hash destroys this property. Three overloads handle 1, 2, and 3 input segments -- the multi-segment versions use SHA256_CTX to hash concatenated inputs without copying them into a single buffer.

```
338 template<typename T1>
339 inline uint256 Hash(const T1 pbegin, const T1 pend)
340 {
341     uint256 hash1;
342 SHA256((unsigned char*)&pbegin[0], (pend - pbegin) * sizeof(pbegin[0]), (unsigned char*)&hash1);
343     uint256 hash2;
344 SHA256((unsigned char*)&hash1, sizeof(hash1), (unsigned char*)&hash2);
345 return hash2;
346 }
347
348 template<typename T1, typename T2>
349 inline uint256 Hash(const T1 p1begin, const T1 p1end,
350 const T2 p2begin, const T2 p2end)
351 {
352     uint256 hash1;
353     SHA256_CTX ctx;
354 SHA256_Init(&ctx);
355 SHA256_Update(&ctx, (unsigned char*)&p1begin[0], (p1end - p1begin) * sizeof(p1begin[0]));
356 SHA256_Update(&ctx, (unsigned char*)&p2begin[0], (p2end - p2begin) * sizeof(p2begin[0]));
357 SHA256_Final((unsigned char*)&hash1, &ctx);
358     uint256 hash2;
359 SHA256((unsigned char*)&hash1, sizeof(hash1), (unsigned char*)&hash2);
360 return hash2;
361 }
362
363 template<typename T1, typename T2, typename T3>
364 inline uint256 Hash(const T1 p1begin, const T1 p1end,
365 const T2 p2begin, const T2 p2end,
366 const T3 p3begin, const T3 p3end)
367 {
368     uint256 hash1;
369     SHA256_CTX ctx;
370 SHA256_Init(&ctx);
371 SHA256_Update(&ctx, (unsigned char*)&p1begin[0], (p1end - p1begin) * sizeof(p1begin[0]));
372 SHA256_Update(&ctx, (unsigned char*)&p2begin[0], (p2end - p2begin) * sizeof(p2begin[0]));
373 SHA256_Update(&ctx, (unsigned char*)&p3begin[0], (p3end - p3begin) * sizeof(p3begin[0]));
374 SHA256_Final((unsigned char*)&hash1, &ctx);
375     uint256 hash2;
376 SHA256((unsigned char*)&hash1, sizeof(hash1), (unsigned char*)&hash2);
377 return hash2;
378 }
379
```

`SerializeHash()` -- serializes any Bitcoin object to a byte stream using the `IMPLEMENT_SERIALIZE` framework, then double-SHA256 hashes the result. This is how transaction IDs and block hashes are computed: serialize the struct, hash the bytes. The comment on line 383 notes a performance concern -- the `CDataStream` buffer allocation dominates the cost. Satoshi considered but did not implement a stack-allocated `CStaticStream` optimization.

```
380 template<typename T>
    CALLED FROM CUser (inline) market.h p. 315 · CReview (inline) market.h p. 316 ·
    CProduct (inline) market.h p. 318
381 uint256 SerializeHash(const T& obj, int nType=SER_GETHASH, int nVersion=VERSION)
382 {
383 // Most of the time is spent allocating and deallocating CDataStream's
384 // buffer.  If this ever needs to be optimized further, make a CStaticStream
385 // class with its buffer on the stack.
386     CDataStream ss(nType, nVersion);
387     ss.reserve(10000);
388     ss << obj;
389 return Hash(ss.begin(), ss.end());
390 }
391
```

`Hash160()` -- SHA-256 followed by RIPEMD-160, producing a 160-bit (20-byte) hash. This is the hash used in Bitcoin addresses: `pubKeyHash = RIPEMD160(SHA256(publicKey))`. The 160-bit output is shorter than SHA-256's 256 bits, making addresses more compact (a Base58Check-encoded 160-bit hash is 25-34 characters). Satoshi never explained the pairing. A common reading is defense in depth -- even if one function is partially broken, the other still protects the address -- with RIPEMD-160, an academic European design, offering independence from the NSA-designed SHA family.

```
    CALLED FROM PubKeyToAddress base58.h p. 97 · CWalletDB::LoadWallet db.cpp p. 129 ·
    AddKey main.cpp p. 202 · EvalScript script.cpp p. 394
392 inline uint160 Hash160(const vector<unsigned char>& vch)
393 {
394     uint256 hash1;
395 SHA256(&vch[0], vch.size(), (unsigned char*)&hash1);
396     uint160 hash2;
397 RIPEMD160((unsigned char*)&hash1, sizeof(hash1), (unsigned char*)&hash2);
398 return hash2;
399 }
400
```

SUMMARY

This utility header is the foundation layer of Bitcoin -- the code that every other file assumes exists. The platform abstractions, the threading primitives, the hashing functions, and the debug logging system defined here are called thousands of times per second during normal operation. None of it is glamorous. All of it is essential.

The `CRITICAL_BLOCK` macro deserves special recognition. Satoshi built his own scoped locking construct with break-detection, debug tracking, and exception safety. RAll lock guards already existed (Boost had `scoped_lock`), but his version is essentially what C++11 would later standardize as `std::lock_guard` plus `std::unique_lock`. That he also built a non-blocking variant (`TRY_CRITICAL_BLOCK`) shows a sophisticated understanding of concurrent programming. The socket handler cannot afford to stall on one peer's lock while every other peer waits; it tries the per-node locks and moves on if one is held.

The hashing functions at the end of the file are Bitcoin's cryptographic identity. Double-SHA256 and SHA256+RIPEMD160 are the two hash constructions that every block, every transaction, and every address in the system depends on. They have been computed trillions of times since January 2009,[1] by hardware ranging from Satoshi's desktop PC to purpose-built ASICs consuming megawatts of power. The implementations here -- thin wrappers around OpenSSL -- are among the most consequential hundred lines of code ever written.

KEY TERMS

int64

Satoshi's portable 64-bit signed integer type, mapped to __int64 on MSVC and Borland and to long long on GCC, used for amounts and timestamps.

foreach

A one-word alias for Boost's BOOST_FOREACH macro, letting the code loop over any STL container without spelling out iterator types.

CCriticalSection

A thin wrapper around the Windows CRITICAL_SECTION mutex that initializes and deletes it automatically and records the file and line of the current holder.

CRITICAL_BLOCK

A macro that locks a critical section for the duration of the following block and releases it on exit, even when an exception is thrown.

TRY_CRITICAL_BLOCK

The non-blocking variant of CRITICAL_BLOCK: it runs the following block only if the lock can be taken immediately, and otherwise skips it.

OutputDebugStringF

The logging function that every printf call is redirected to on Windows; in debug builds it appends to debug.log and the Windows debugger output.

IMPLEMENT_RANDOMIZE_STACK

A macro that makes a thread function call itself a random number of times, up to 50, so its working stack frame lands at an unpredictable depth.

SerializeHash

A helper that serializes any object into a byte stream and returns the double-SHA256 of those bytes, the standard way objects are given their hash.

SOURCES

1. Satoshi Nakamoto, *Bitcoin genesis block (block 0)* (January 3, 2009). Coinbase embeds The Times headline as timestamp proof. https://mempool.space/block/000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
2. Hal Finney, *[bitcoin-list] Crash in bitcoin 0.1.0 (with debug.log)* (January 10, 2009). SourceForge post attaching Finney's debug.log; forensic evidence he joined near block 49. https://web.archive.org/web/20141130200234/https://sourceforge.net/p/bitcoin/mailman/message/21295694/

3. Alex Waltz, *Who was the 1st Bitcoin Miner?* (2026). Forensic reconstruction of Bitcoin's first days from Finney's debug.log, emails, and ExtraNonce patterns. https://1stbitcoinminer.com/
4. Alex Waltz, *Copy-Paste & LLM-friendly transcriptions* (2026). Plain-text transcriptions of primary sources used in the 1st Bitcoin Miner investigation. https://1stbitcoinminer.com/transcriptions/

WHAT CAME AFTER

You've now studied Bitcoin at its origin—the pristine v0.1.0 code that launched a monetary revolution. But Bitcoin didn't stop evolving on January 9, 2009.[1][2] Over seventeen years, the codebase has grown from roughly 20,000 lines of C++ to more than 300,000 (counting tests and bundled libraries), with hundreds of contributors refining Satoshi's vision. Here's a chronological guide to what came next, so you know which versions to study for specific improvements.

THE SATOSHI ERA (2009–2011)

Satoshi personally maintained Bitcoin for its first two years, releasing incremental improvements while communicating with early adopters on forums and mailing lists.[1]

v0.1.0	Jan 9, 2009	First public release (this book). Windows only.[2] IRC-based peer discovery.[3] First transaction, Satoshi to Hal Finney, in block 170 (Jan 12).[4]
v0.1.5	Feb 4, 2009	Bug fixes.
v0.2.0	Dec 16, 2009	**Linux support**. Numerous optimizations.[1]
v0.3.0	Jul 6, 2010	**Mac OS X support**. JSON-RPC API and daemon mode begin.[1]
v0.3.1	Jul 2010	**1 MB block size limit** added quietly by Satoshi (widely understood as an anti-spam/DoS measure; no rationale was posted). Originally temporary, became contentious years later.[1]
v0.3.5	Jul 2010	**OP_RETURN bug fixed** after discovery of a critical script bug allowing anyone to spend any coins. scriptSig and scriptPubKey now run separately; OP_CAT and other risky opcodes disabled. Satoshi's emergency soft fork.[5][6]
v0.3.10	Aug 15, 2010	**Value overflow fix**. Block 74638 exploited integer overflow to create 184 billion BTC. The patched client shipped within five hours; the corrected chain overtook the bad one the next day.[7]
v0.3.14	Oct 21, 2010	**Testnet** introduced by Gavin Andresen for development testing without real value.[8]
v0.3.19	Dec 2010	Satoshi's final release (DoS protections). He stepped away from public development by April 2011, leaving Bitcoin to the community.[9][10][11]

v0.3.21	Apr 27, 2011	First major release under Gavin Andresen. UPnP, and **DNS seeding** for finding peers—the beginning of the end for v0.1.0's IRC bootstrap.[12]

Study these versions if: You want to trace Satoshi's thinking as he responded to real-world usage. The JSON-RPC API (v0.3.0+) enabled the first exchanges and services. The testnet idea (v0.3.14) lives on in its successors, testnet3 (v0.7) and testnet4 (v28).

EARLY COMMUNITY DEVELOPMENT (2011–2013)

After Satoshi's departure,[10] Gavin Andresen became lead maintainer. The project moved to GitHub, adopted a more formal release process, and began professionalizing.[1]

v0.4.0	Sep 23, 2011	**Encrypted wallets**. AES-256-CBC encryption protects private keys.[1]
v0.5.0	Nov 21, 2011	**New Qt GUI**. Better address book. "Bitcoin-Qt" naming begins.[113]
v0.6.0	Mar 30, 2012	**Compressed public keys** (33-byte) for smaller transactions. QR codes for addresses.[1]
v0.7.0	Sep 17, 2012	Reduced memory usage. Better fee handling. Implements BIPs 22, 34, 35 (block templates, height in coinbase).[1]
v0.8.0	Feb 18, 2013	**LevelDB** replaces Berkeley DB for UTXO set. Bloom filters (BIP 37) for SPV clients. Parallel signature checking.[1]
v0.8.1	Mar 18, 2013	**Chain split fix**. BerkeleyDB lock limit caused v0.7 nodes to reject valid blocks that v0.8 accepted. 6-hour fork resolved by emergency downgrade.[14]

Study these versions if: You're interested in wallet encryption (v0.4), SPV/light-client support via Bloom filters (v0.8), or the database architecture (v0.8). The March 2013 chain split[14] is a critical lesson in consensus-critical code changes—even "non-consensus" changes can break consensus.

MATURATION & REBRANDING (2014–2016)

The project rebranded to "Bitcoin Core" to distinguish it from the broader Bitcoin ecosystem.[1] Development became more rigorous, with extensive review processes and a focus on security.

v0.9.0	Mar 19, 2014	**Rebranded to "Bitcoin Core"**. OP_RETURN outputs allowed (40-byte data limit). Payment protocol (BIP 70).[1]
v0.10.0	Feb 16, 2015	**Headers-first sync**. Download block headers before blocks for faster IBD. Signing moves from OpenSSL to libsecp256k1.[115]
v0.11.0	Jul 12, 2015	**Block pruning**. OP_RETURN limit raised to 80 bytes.[1]
v0.12.0	Feb 23, 2016	**Opt-in RBF** (Replace-By-Fee). Memory pool limiting. Signature validation moves to **libsecp256k1**—over five times faster on x86_64.[116]
v0.13.0	Aug 23, 2016	**SegWit preparation** (BIPs 141, 143, 144). Compact blocks (BIP 152).[1]

Study these versions if: You want to understand modern sync strategies (v0.10), the libsecp256k1 library (v0.10–v0.12), or prepare for SegWit. The shift from OpenSSL to libsecp256k1 is a masterclass in security-focused refactoring.

SEGWIT & SCALING DEBATES (2017–2019)

Segregated Witness (SegWit) was the most significant protocol upgrade since the beginning.[1] It fixed transaction malleability, enabled the Lightning Network, and increased effective block capacity—all through a soft fork.

v0.14.0	Mar 8, 2017	Performance optimizations. Manual pruning. Improved fee estimation.[1]
v0.15.0	Sep 14, 2017	**Better fee estimation**. Multi-wallet support. Script caching.[1]
v0.16.0	Feb 26, 2018	**Full SegWit wallet support**. Native bech32 addresses (bc1...).[1]
v0.17.0	Oct 3, 2018	**Partial Spend Avoidance**. Branch and Bound coin selection. Scantxoutset RPC.[1]
v0.18.0	May 2, 2019	**Output descriptors**. Hardware wallet support via HWI.[1]
v0.19.0	Nov 24, 2019	**BIP 158 block filters** (Neutrino). CPFP carve-out for Lightning.[1]

Study these versions if: You want to understand SegWit implementation (v0.13–v0.16), coin selection algorithms (v0.17), or output descriptors (v0.18). The descriptor wallet model fundamentally changed how Bitcoin Core manages keys.

MODERN BITCOIN CORE (2020–PRESENT)

Recent development has focused on Taproot (the largest upgrade since SegWit),[1] privacy improvements, and hardening against sophisticated attacks.

v0.20.0	Jun 3, 2020	**ASMap** for BGP attack resistance. Remove BIP 70 payment protocol.[1]
v0.21.0	Jan 14, 2021	**Descriptor wallets (experimental)**. Tor V3 support. Signet test network.[117]
v22.0	Sep 13, 2021	**Taproot** (BIPs 340, 341, 342) activates on mainnet at block 709632 in November 2021; the activation logic shipped in 0.21.1. Version numbering change.[118]
v23.0	Apr 25, 2022	**Descriptor wallets become default**. Taproot wallet support (bech32m addresses). CJDNS network support. Tracepoints for debugging.[119]
v24.0	Nov 25, 2022	**Full RBF option** (mempoolfullrbf). Watch-only descriptor wallets.[1]
v25.0	May 26, 2023	**Miniscript** for complex spending conditions. Improved coin selection.[1]
v26.0	Dec 6, 2023	**V2 transport protocol** (BIP 324), opt-in and experimental. Encrypted P2P connections.[120]
v27.0	Apr 17, 2024	**V2 encrypted transport on by default**. CoinGrinder coin selection. libbitcoinconsensus deprecated.[121]
v28.0	Oct 2, 2024	Testnet4. Package relay groundwork (1P1C). Full TRUC/V3 transaction support.[1]
v29.0	Apr 14, 2025	**CMake migration**—Autotools build system retired.[1]
v30.0	Oct 10, 2025	Qt 6 GUI. New unified `bitcoin` command. Legacy BDB wallets removed. `-datacarriersize` default raised to 100,000 bytes, effectively uncapping OP_RETURN data.[122]

Study these versions if: You want to understand Schnorr signatures and Taproot (v0.21.1–v22), Miniscript (v25), or encrypted P2P (v26–v27). The libbitcoinkernel project (ongoing) is modularizing the consensus code for use by other implementations.

NOTABLE BUGS & INCIDENTS

Bitcoin's history includes several critical bugs that tested the network's resilience. Studying these incidents reveals how consensus systems fail and recover:

Jul 2010	OP_RETURN Bug	*Introduced by: Satoshi (v0.1.0)* — A flaw in script evaluation let anyone spend anyone's coins using OP_RETURN. Present since day one but undiscovered. Fixed in v0.3.5 by running scriptSig and scriptPubKey separately; Satoshi also disabled multiple opcodes in the same emergency soft fork.[5][6]
Aug 2010	Value Overflow	*Introduced by: Satoshi (v0.1.0)* — Integer overflow created 184 billion BTC in block 74638. Missing overflow check in transaction validation. A patched client (v0.3.10) was released within five hours; the corrected chain overtook the bad one the next day.[7]
Mar 2013	BDB Lock Crisis	*Root cause: Berkeley DB lock limits in all pre-0.8 versions* — Version 0.8's switch to LevelDB removed an undocumented limit that had silently become a consensus rule. A large block accepted by 0.8 was rejected by 0.7 nodes. 6-hour chain split resolved by miners downgrading.[14]
Jul 2015	BIP 66 Fork	*Introduced by: miners (SPV mining)* — Miners signaling BIP 66 (strict DER) weren't actually validating blocks. Brief fork when an invalid block was extended. Exposed risks of mining on headers without full validation.[1]
Sep 2018	CVE-2018-17144	*Introduced by: Core developers (v0.14)* — Optimization to skip duplicate input checking: in v0.14 it caused a crash; from v0.15 it allowed inflation via a duplicated input within a single block. Silently patched in v0.16.3. Most critical vulnerability since 2010.[23]

Key lessons: (1) Even Satoshi's original code contained critical bugs—peer review is essential. (2) Consensus code changes require extreme caution—even database backends can affect consensus. (3) Quick community response is crucial. (4) Silent patching of critical bugs is sometimes necessary to prevent exploitation before nodes upgrade.

KEY ARCHITECTURAL CHANGES

Looking at the big picture, these are the most significant architectural departures from the v0.1.0 code you just studied:

COMPONENT	V0.1.0	MODERN BITCOIN CORE
Cryptography	OpenSSL (ECDSA)	libsecp256k1, Schnorr signatures

Database	Berkeley DB	LevelDB (UTXO), SQLite (wallet)
Peer Discovery	IRC channel, plus addr gossip	DNS seeds, addr gossip
Script	Basic opcodes	P2SH, SegWit, Tapscript
Addresses	Raw public keys (P2PK, used by coinbase and pay-to-IP) and 1... (P2PKH)	3... (P2SH), bc1q... (SegWit), bc1p... (Taproot)
Wallet	Loose key collection	HD wallets, descriptors
GUI	wxWidgets[24]	Qt Framework[13]
Build System	Makefiles (hand-written)	CMake (as of v29)
Code Size	~20,000 lines	300,000+ lines (with tests and bundled libraries)

RECOMMENDED STUDY PATH

Having mastered v0.1.0, here's a suggested progression based on your interests:

- **Consensus Rules:** v0.3.10 (overflow fix) → v0.8.1 (DB fork) → v22.0 (Taproot)
- **Cryptography:** v0.12.0 (libsecp256k1 validation) → v22.0 (Schnorr/Taproot)
- **Wallet Development:** v0.4.0 (encryption) → v0.18.0 (descriptors) → v0.21.0 (descriptor wallets) → v23.0 (descriptor default)
- **Networking:** v0.3.0 (daemon) → v0.8.0 (Bloom filters) → v26.0 (encrypted P2P)
- **Script/Smart Contracts:** v0.6.0 (P2SH) → v0.13.0 (SegWit) → v22.0 (Tapscript) → v25.0 (Miniscript)
- **Lightning Prerequisites:** v0.13.0 (SegWit) → v0.19.0 (CPFP carve-out) → v24.0 (full RBF)
- **Security & Bugs:** v0.3.5 (OP_RETURN)[5] → v0.3.10 (overflow)[7] → v0.8.1 (BDB)[14] → v0.16.3 (CVE-2018-17144)[23]

RESOURCES FOR CONTINUED STUDY

- **Bitcoin Core GitHub:** github.com/bitcoin/bitcoin — Full commit history since 2009

- **Bitcoin Improvement Proposals:** github.com/bitcoin/bips — All protocol specifications
- **Bitcoin Optech:** bitcoinops.org — Weekly technical newsletter since 2018
- **Bitcoin Stack Exchange:** bitcoin.stackexchange.com — Q&A for technical questions
- **Chaincode Labs Seminars:** chaincode.com — Free courses on Bitcoin/Lightning development
- **Learning Bitcoin from the Command Line:** github.com/BlockchainCommons — Hands-on tutorials

The code you've studied in this book is where it all began. Every feature, every optimization, every security fix in modern Bitcoin Core traces back to these 20,000 lines. Satoshi wrote: "I'm sure that in 20 years there will either be very large transaction volume or no volume."[25] Seventeen years in, the code keeps evolving—and now you know where to look.

SOURCES

1. Bitcoin Core Contributors, *Bitcoin Core Release History* (2009–present). https://bitcoincore.org/en/releases/
2. Satoshi Nakamoto, *Bitcoin v0.1 released (Cryptography Mailing List / BitcoinTalk)* (January 9, 2009). Announced Windows-only for the first release. https://satoshi.nakamotoinstitute.org/emails/cryptography/15/
3. Satoshi Nakamoto, *BitcoinTalk — IRC channel for Bitcoin* (March 16 and June 14, 2010). IRC seeding described as a temporary solution. https://bitcointalk.org/index.php?topic=84.0
4. Hal Finney, *Bitcoin transaction — first received coins from Satoshi* (January 12, 2009). Block 170; Finney was the first person other than Satoshi to receive bitcoin. https://bitcointalk.org/index.php?topic=13.msg46#msg46
5. Bitcoin Wiki, *Common Vulnerabilities and Exposures — CVE-2010-5141 (OP_RETURN) and CVE-2010-5137* (July 28, 2010). Announced 2010-07-28; fixed in Bitcoin 0.3.5. https://en.bitcoin.it/wiki/Common_Vulnerabilities_and_Exposures
6. Satoshi Nakamoto / Bitcoin developers, *OP_RETURN and dangerous opcodes disabled* (July 2010). Response to early Script vulnerabilities. https://en.bitcoin.it/wiki/Script#Disabled_opcodes

7. Bitcoin community, *Value overflow incident (block 74638)* (August 15, 2010). Transaction created ~184 billion BTC; fixed in subsequent soft fork. https://en.bitcoin.it/wiki/Value_overflow_incident
8. Satoshi Nakamoto, *Version 0.3.14 (BitcoinTalk announcement)* (October 21, 2010). Key pool; TEST network mode with -testnet (Gavin Andresen); validateaddress. https://satoshi.nakamotoinstitute.org/posts/bitcointalk/491/
9. Bitcoin Wiki, *Original Bitcoin client*. 0.3.19 was the last release in which Satoshi Nakamoto was chief and main developer. https://en.bitcoin.it/wiki/Bitcoin_client
10. Satoshi Nakamoto, *Last known email to Mike Hearn ("I've moved on to other things")* (April 23, 2011). "I've moved on to other things. It's in good hands with Gavin and everyone.". https://cointelegraph.com/news/satoshi-nakamoto-final-email-revisited-after-13-years
11. Satoshi Nakamoto, *BitcoinTalk — Re: Bitcoin 0.3.21* (June 17, 2010). On the core design being set in stone for the rest of its lifetime. https://bitcointalk.org/index.php?topic=195.msg1611#msg1611
12. Gavin Andresen, *Bitcoin 0.3.21 release announcement (BitcoinTalk)* (April 27, 2011). UPnP support; DNS-based peer discovery; full-precision amounts. https://ic.unicamp.br/~stolfi/bitcoin/projects-bitcoin/bitcointalk/Gavin_Andresen/split/2011-04-27-172917.html
13. Qt Company / Nokia, *Qt licensed under LGPL* (January 2009). Enabled Bitcoin Core's later migration from wxWidgets to Qt. https://blog.qt.io/blog/2009/01/14/nokia-qt-lgpl/
14. Gavin Andresen, *BIP 50 — March 2013 Chain Fork Post-Mortem* (2013). Berkeley DB lock limit caused a consensus fork around block 225430. https://github.com/bitcoin/bips/blob/master/bip-0050.mediawiki
15. Bitcoin Core Contributors, *Bitcoin Core 0.10.0 release notes* (February 2015). Signing switched from OpenSSL to libsecp256k1. https://bitcoincore.org/en/releases/0.10.0/
16. Bitcoin Core Contributors, *Bitcoin Core 0.12.0 release notes* (February 2016). Signature validation using libsecp256k1; over five times faster on x86_64. https://bitcoincore.org/en/releases/0.12.0/
17. Bitcoin Core Contributors, *Bitcoin Core 0.21.0 release notes* (January 2021). Experimental descriptor wallets. https://bitcoincore.org/en/releases/0.21.0/
18. Bitcoin Core Contributors, *Bitcoin Core 0.21.1 release notes* (May 1, 2021). Taproot activation via Speedy Trial; active as of block 709632. https://bitcoincore.org/en/releases/0.21.1/
19. Bitcoin Core Contributors, *Bitcoin Core 23.0 release notes* (April 25, 2022). Descriptor wallets are now the default wallet type. https://bitcoincore.org/en/releases/23.0/

20. Bitcoin Core Contributors, *Bitcoin Core 26.0 release notes* (December 2023). Experimental support for the BIP 324 v2 transport protocol. https://bitcoincore.org/en/releases/26.0/
21. Bitcoin Core Contributors, *Bitcoin Core 27.0 release notes* (April 2024). BIP 324 v2 transport enabled by default; CoinGrinder; libbitcoinconsensus deprecated. https://bitcoincore.org/en/releases/27.0/
22. Bitcoin Core Contributors, *Bitcoin Core 30.0 release notes* (October 10, 2025). -datacarriersize default 100,000; legacy BDB wallets removed; Qt 6; bitcoin command. https://bitcoincore.org/en/releases/30.0/
23. Bitcoin Core, *CVE-2018-17144 — Inflation vulnerability* (September 2018). Fixed in Bitcoin Core 0.16.3 and 0.17.0rc4. https://bitcoincore.org/en/2018/09/20/notice/
24. wxWidgets Project, *wxWidgets Licence* (2005). https://www.wxwidgets.org/about/licence/
25. Satoshi Nakamoto, *Re: What's with this odd generation? (BitcoinTalk)* (February 14, 2010). Either very large transaction volume or no volume in 20 years. https://bitcointalk.org/index.php?topic=48.msg329#msg329

OPENSSL AS BITCOIN USED IT

APPENDIX

The Cryptography Basics primer stopped at a boundary: you won't find elliptic-curve math in Bitcoin's source code, because Satoshi delegated it to OpenSSL. This appendix crosses that boundary. It follows the calls in `key.h`, `util.cpp`, and `bignum.h` into the library that actually generated the keys, signed the transactions, and checked the signatures of the first Bitcoin network.

The version matters. v0.1.0's `readme.txt` names OpenSSL v0.9.8h, the release of May 28, 2008,[1] and the Windows download shipped it as a compiled `libeay32.dll` beside `bitcoin.exe`. Elliptic-curve signatures were still new to OpenSSL then: ECDSA had arrived three years earlier, in 0.9.8.[2] The excerpts below come from the official 0.9.8h tarball, byte for byte, and keep their original line numbers.

It is a selection, not a reprint. The 0.9.8h release holds about 362,000 lines of C and headers, and its `crypto/` library alone about 214,000, more than ten times the whole of Bitcoin v0.1.0. What follows is the few hundred lines Bitcoin's security actually rested on: choosing the curve, making a key, signing, verifying, parsing a signature, and seeding the random number generator. The readme's build notes only switch off ciphers Bitcoin never calls (RSA, DES, AES, and the rest); the elliptic-curve, hashing, and random-number code was built as released.

WHAT V0.1.0 CALLS

Every OpenSSL name the v0.1.0 source uses, grouped by job. The page numbers point to the first use in each file. `sha.cpp` is not here: the miner's SHA-256 is Wei Dai's Crypto++ code, not OpenSSL's.[3]

50 NAMES	USED FOR	FIRST USE
Key objects `EC_KEY` , `EC_KEY_new_by_curve_name` , `NID_secp256k1` , `EC_KEY_dup` , `EC_KEY_copy` , `EC_KEY_free` , `EC_KEY_generate_key`	Create a secp256k1 key, copy it, free it, and generate a fresh key pair.	`key.h` p. 183
Key encoding `d2i_ECPrivateKey` , `i2d_ECPrivateKey` , `o2i_ECPublicKey` , `i2o_ECPublicKey`	Private keys to and from DER for `wallet.dat` ; public keys to and from the 65-byte uncompressed point.	`key.h` p. 184
Signatures `ECDSA_sign` , `ECDSA_verify`	Sign a 32-byte hash and return a DER-encoded signature; parse and check one.	`key.h` p. 185
Hashing `SHA256` , `SHA256_CTX` , `SHA256_Init` , `SHA256_Update` , `SHA256_Final` , `RIPEMD160`	Double SHA-256 for transaction and block hashes, SHA-256 then RIPEMD-160 for addresses, and the Script hash opcodes.	`script.cpp` p. 393 `util.cpp` p. 676 `util.h` p. 696
Big numbers `BIGNUM` , `BN_CTX` , `BN_CTX_new` , `BN_CTX_free` , `BN_init` , `BN_clear_free` , `BN_copy` , `BN_set_word` , `BN_get_word` , `BN_is_negative` , `BN_set_negative` , `BN_is_zero` , `BN_cmp` , `BN_add` , `BN_sub` , `BN_mul` , `BN_div` , `BN_mod` , `BN_lshift` , `BN_rshift` , `BN_value_one` , `BN_bn2mpi` , `BN_mpi2bn`	`CBigNum` : difficulty targets, Base58, and Script arithmetic.	`base58.h` p. 93 `bignum.h` p. 102 `script.cpp` p. 392
Randomness `RAND_screen` , `RAND_add` , `RAND_bytes`	Seed OpenSSL's pool at startup and from mouse events; draw random bytes.	`net.h` p. 363 `ui.cpp` p. 482 `util.cpp` p. 674
Threads and memory `CRYPTO_num_locks` , `CRYPTO_set_locking_callback` , `CRYPTO_LOCK` , `OPENSSL_malloc` , `OPENSSL_free`	Give OpenSSL Windows mutexes so the network, miner, and UI threads can share it.	`util.cpp` p. 674

FROM A NAME TO A CURVE

`CKey` 's constructor makes one call: `EC_KEY_new_by_curve_name(NID_secp256k1)` . `NID_secp256k1` is just an integer, OpenSSL's numeric ID for the curve. Everything else comes out of OpenSSL's tables.

crypto/ec/ec_key.c lines 69–103

```
 69 EC_KEY *EC_KEY_new(void)
 70     {
 71     EC_KEY *ret;
 72
 73     ret=(EC_KEY *)OPENSSL_malloc(sizeof(EC_KEY));
 74     if (ret == NULL)
 75 {
 76 ECerr(EC_F_EC_KEY_NEW, ERR_R_MALLOC_FAILURE);
 77 return(NULL);
 78 }
 79
 80     ret→version = 1;
```

```
    ret->group   = NULL;
    ret->pub_key = NULL;
    ret->priv_key= NULL;
    ret->enc_flag= 0;
```

New keys default to *uncompressed* points: a 0×04 byte, then the 32-byte x and the 32-byte y. That default is why every v0.1.0 public key is 65 bytes, the `PUBLIC_KEY_SIZE` in `key.h`'s comments. Compressed 33-byte keys came to Bitcoin later; OpenSSL could already produce them.

```
    ret->conv_form = POINT_CONVERSION_UNCOMPRESSED;
    ret->references= 1;
    ret->method_data = NULL;
    return(ret);
    }

EC_KEY *EC_KEY_new_by_curve_name(int nid)
    {
    EC_KEY *ret = EC_KEY_new();
    if (ret == NULL)
return NULL;
```

The key holds a pointer to a *group*, OpenSSL's name for a curve together with its generator and order. The lookup by ID happens here.

```
    ret->group = EC_GROUP_new_by_curve_name(nid);
    if (ret->group == NULL)
{
EC_KEY_free(ret);
return NULL;
}
    return ret;
    }
```

crypto/ec/ec_curve.c lines 381–391

```
static const EC_CURVE_DATA _EC_SECG_PRIME_256K1 = {
    NID_X9_62_prime_field,
```

p, the field prime: $2^{256} - 2^{32} - 977$. All of secp256k1 lives in the integers modulo this number.[4]

```
    "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F",
```

a = 0 and b = 7: the curve $y^2 = x^3 + 7$.

```
    "0",
    "7",
```

The x and y coordinates of the generator point G. Every Bitcoin public key is G multiplied by a private key.

```
    "79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798",
    "483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8",
```

n, the order of G: the number of distinct multiples of G, and the range a private key must fall in. The 1 after it is the cofactor; every point on the curve is a multiple of G.

```
388     "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141",1,
```

`NULL` : no seed. NIST's curves were built from seeds run through SHA-1 so their constants are "verifiably random". SEC 2 calls secp256k1 a Koblitz curve instead, with constants chosen for an efficiently computable endomorphism rather than at random.[4] The primer's point about Dual_EC_DRBG is why that mattered to some: there's no hidden seed to wonder about.

```
389     NULL, 20,
390     "SECG curve over a 256 bit prime field"
391     };
```

crypto/ec/ec_curve.c lines 1040–1057

```
1040 static const ec_list_element curve_list[] = {
1041     /* prime field curves */
1042     /* secg curves */
1043     { NID_secp112r1, &_EC_SECG_PRIME_112R1},
1044     { NID_secp112r2, &_EC_SECG_PRIME_112R2},
1045     { NID_secp128r1, &_EC_SECG_PRIME_128R1},
1046     { NID_secp128r2, &_EC_SECG_PRIME_128R2},
```

secp160k1, secp192k1, secp224k1, secp256k1: the four Koblitz curves whose key and signature sizes Satoshi listed at the top of `key.h` . He compared them all, then picked the largest.

```
1047     { NID_secp160k1, &_EC_SECG_PRIME_160K1},
1048     { NID_secp160r1, &_EC_SECG_PRIME_160R1},
1049     { NID_secp160r2, &_EC_SECG_PRIME_160R2},
1050     /* SECG secp192r1 is the same as X9.62 prime192v1 and hence omitted */
1051     { NID_secp192k1, &_EC_SECG_PRIME_192K1},
1052     { NID_secp224k1, &_EC_SECG_PRIME_224K1},
1053     { NID_secp224r1, &_EC_NIST_PRIME_224},
```

The entry v0.1.0 actually uses.

```
1054     { NID_secp256k1, &_EC_SECG_PRIME_256K1},
1055     /* SECG secp256r1 is the same as X9.62 prime256v1 and hence omitted */
1056     { NID_secp384r1, &_EC_NIST_PRIME_384},
1057     { NID_secp521r1, &_EC_NIST_PRIME_521},
```

crypto/ec/ec_curve.c lines 1228–1252

```
1228 EC_GROUP *EC_GROUP_new_by_curve_name(int nid)
1229     {
1230     size_t i;
1231     EC_GROUP *ret = NULL;
1232
1233     if (nid <= 0)
1234 return NULL;
1235
```

A linear search of the table above. The match goes to `ec_group_new_from_data` , which turns the hex strings into BIGNUMs, builds the curve, and sets G, n, and the cofactor.

```
1236     for (i=0; i<curve_list_length; i++)
```

```
1237 if (curve_list[i].nid == nid)
1238 {
1239             ret = ec_group_new_from_data(curve_list[i].data);
1240 break;
1241 }
1242
1243     if (ret == NULL)
1244 {
1245 ECerr(EC_F_EC_GROUP_NEW_BY_CURVE_NAME, EC_R_UNKNOWN_GROUP);
1246 return NULL;
1247 }
1248
1249     EC_GROUP_set_curve_name(ret, nid);
1250
1251     return ret;
1252     }
```

Those hex strings are the whole definition of Bitcoin's curve. Satoshi never typed them; they came with the library, from the SEC 2 standard of September 2000.[4]

MAKING A KEY

`CKey::MakeNewKey` calls `EC_KEY_generate_key`. That function is short: pick a random number below n, multiply G by it, keep both.

crypto/ec/ec_key.c lines 233–293

```
233 int EC_KEY_generate_key(EC_KEY *eckey)
234     {
235     int ok = 0;
236     BN_CTX  *ctx = NULL;
237     BIGNUM  *priv_key = NULL, *order = NULL;
238     EC_POINT *pub_key = NULL;
239
240     if (!eckey || !eckey->group)
241 {
242 ECerr(EC_F_EC_KEY_GENERATE_KEY, ERR_R_PASSED_NULL_PARAMETER);
243 return 0;
244 }
245
246     if ((order = BN_new()) == NULL) goto err;
247     if ((ctx = BN_CTX_new()) == NULL) goto err;
248
249     if (eckey->priv_key == NULL)
250 {
251         priv_key = BN_new();
252 if (priv_key == NULL)
253 goto err;
254 }
255     else
256         priv_key = eckey->priv_key;
257
258     if (!EC_GROUP_get_order(eckey->group, order, ctx))
259 goto err;
260
261     do
```

The private key: a uniformly random integer in [0, n), drawn from OpenSSL's random pool by `BN_rand_range`. The loop throws away zero. Everything Bitcoin's startup code does to seed that pool exists for this line and for the signing nonce below.

```
262 if (!BN_rand_range(priv_key, order))
263 goto err;
264     while (BN_is_zero(priv_key));
265
266     if (eckey->pub_key == NULL)
267 {
268         pub_key = EC_POINT_new(eckey->group);
269 if (pub_key == NULL)
270 goto err;
271 }
272     else
273         pub_key = eckey->pub_key;
274
```

The public key: priv_key × G. `EC_POINT_mul` takes a scalar for G and a second point and scalar; here only the first is used. Undoing this multiplication is the discrete-logarithm problem.

```
275     if (!EC_POINT_mul(eckey->group, pub_key, priv_key, NULL, NULL, ctx))
276 goto err;
277
278     eckey->priv_key = priv_key;
279     eckey->pub_key  = pub_key;
280
281     ok=1;
282
283 err:
284     if (order)
285 BN_free(order);
286     if (pub_key  != NULL && eckey->pub_key  == NULL)
287 EC_POINT_free(pub_key);
288     if (priv_key != NULL && eckey->priv_key == NULL)
289 BN_free(priv_key);
290     if (ctx != NULL)
291 BN_CTX_free(ctx);
292     return(ok);
293     }
```

`i2d_ECPrivateKey` then serializes the result for `wallet.dat`. Its DER structure carries the curve parameters along with the 32-byte secret, which is why `key.h` lists a 279-byte private key.

SIGNING

`CKey::Sign` hands `ECDSA_sign` the 32-byte transaction hash and a 10,000-byte buffer. Two layers of wrapper later, the real work happens in `ecs_ossl.c`.

crypto/ecdsa/ecs_sign.c lines 75–95

The `type` argument is ignored; v0.1.0 passes 0.

```
int ECDSA_sign(int type, const unsigned char *dgst, int dlen, unsigned char
*sig, unsigned int *siglen, EC_KEY *eckey)
{
    return ECDSA_sign_ex(type, dgst, dlen, sig, siglen, NULL, NULL, eckey);
}

int ECDSA_sign_ex(int type, const unsigned char *dgst, int dlen, unsigned char
    *sig, unsigned int *siglen, const BIGNUM *kinv, const BIGNUM *r,
    EC_KEY *eckey)
{
    ECDSA_SIG *s;
    s = ECDSA_do_sign_ex(dgst, dlen, kinv, r, eckey);
    if (s == NULL)
    {
*siglen=0;
return 0;
    }
```

The signature (r, s) leaves OpenSSL DER-encoded: a SEQUENCE of two INTEGERs, at most 72 bytes for secp256k1. `key.h` 's 10,000-byte buffer is far larger than it needs to be.

```
    *siglen = i2d_ECDSA_SIG(s, &sig);
    ECDSA_SIG_free(s);
    return 1;
}
```

crypto/ecdsa/ecs_ossl.c lines 135–193

```
    do
    {
/* get random k */
do
```

k, the per-signature nonce, comes from the same random pool as the private key. If k ever repeats across two signatures, or can be predicted, the private key falls out of simple algebra; that is how the PlayStation 3 signing key was recovered.[5]

```
if (!BN_rand_range(k, order))
{
ECDSAerr(ECDSA_F_ECDSA_SIGN_SETUP,
                ECDSA_R_RANDOM_NUMBER_GENERATION_FAILED);
goto err;
}
while (BN_is_zero(k));

/* compute r the x-coordinate of generator * k */
```

R = k × G.

```
if (!EC_POINT_mul(group, tmp_point, k, NULL, NULL, ctx))
{
ECDSAerr(ECDSA_F_ECDSA_SIGN_SETUP, ERR_R_EC_LIB);
goto err;
}
if (EC_METHOD_get_field_type(EC_GROUP_method_of(group)) == NID_X9_62_prime_field)
{
if (!EC_POINT_get_affine_coordinates_GFp(group,
                tmp_point, X, NULL, ctx))
```

```
157 {
158 ECDSAerr(ECDSA_F_ECDSA_SIGN_SETUP,ERR_R_EC_LIB);
159 goto err;
160 }
161 }
162 else /* NID_X9_62_characteristic_two_field */
163 {
164 if (!EC_POINT_get_affine_coordinates_GF2m(group,
165                tmp_point, X, NULL, ctx))
166 {
167 ECDSAerr(ECDSA_F_ECDSA_SIGN_SETUP,ERR_R_EC_LIB);
168 goto err;
169 }
170 }
```

r is R's x-coordinate reduced mod n. A zero r would be useless, so the outer loop starts over.

```
171 if (!BN_nnmod(r, X, order, ctx))
172 {
173 ECDSAerr(ECDSA_F_ECDSA_SIGN_SETUP, ERR_R_BN_LIB);
174 goto err;
175 }
176     }
177     while (BN_is_zero(r));
178
179     /* compute the inverse of k */
```

k is replaced by its inverse mod n, the form the signing step needs.

```
180     if (!BN_mod_inverse(k, k, order, ctx))
181     {
182 ECDSAerr(ECDSA_F_ECDSA_SIGN_SETUP, ERR_R_BN_LIB);
183 goto err;
184     }
185     /* clear old values if necessary */
186     if (*rp != NULL)
187 BN_clear_free(*rp);
188     if (*kinvp != NULL)
189 BN_clear_free(*kinvp);
190     /* save the pre-computed values  */
191     *rp    = r;
192     *kinvp = k;
193     ret = 1;
```

crypto/ecdsa/ecs_ossl.c lines 269–324

The hash becomes the integer m. Bitcoin's 256-bit hash exactly fits the 256-bit order, so the length check just above never fires.

```
269     if (!BN_bin2bn(dgst, dgst_len, m))
270     {
271 ECDSAerr(ECDSA_F_ECDSA_DO_SIGN, ERR_R_BN_LIB);
272 goto err;
273     }
274     do
275     {
276 if (in_kinv == NULL || in_r == NULL)
277 {
278 if (!ECDSA_sign_setup(eckey, ctx, &kinv, &ret->r))
279 {
280 ECDSAerr(ECDSA_F_ECDSA_DO_SIGN,ERR_R_ECDSA_LIB);
281 goto err;
282 }
283             ckinv = kinv;
```

```
284 }
285 else
286 {
287             ckinv  = in_kinv;
288 if (BN_copy(ret→r, in_r) == NULL)
289 {
290 ECDSAerr(ECDSA_F_ECDSA_DO_SIGN, ERR_R_MALLOC_FAILURE);
291 goto err;
292 }
293 }
294
```

These three steps compute $s = k^{-1}(m + r \cdot d) \bmod n$, where d is the private key. This is the ECDSA equation from the primer, in OpenSSL's code.

```
295 if (!BN_mod_mul(tmp, priv_key, ret→r, order, ctx))
296 {
297 ECDSAerr(ECDSA_F_ECDSA_DO_SIGN, ERR_R_BN_LIB);
298 goto err;
299 }
300 if (!BN_mod_add_quick(s, tmp, m, order))
301 {
302 ECDSAerr(ECDSA_F_ECDSA_DO_SIGN, ERR_R_BN_LIB);
303 goto err;
304 }
305 if (!BN_mod_mul(s, s, ckinv, order, ctx))
306 {
307 ECDSAerr(ECDSA_F_ECDSA_DO_SIGN, ERR_R_BN_LIB);
308 goto err;
309 }
```

s = 0 is also invalid, so the loop repeats with a fresh k.

```
310 if (BN_is_zero(s))
311 {
312 /* if kinv and r have been supplied by the caller
313 * don't to generate new kinv and r values */
314 if (in_kinv != NULL && in_r != NULL)
315 {
316 ECDSAerr(ECDSA_F_ECDSA_DO_SIGN, ECDSA_R_NEED_NEW_SETUP_VALUES);
317 goto err;
318 }
319 }
320 else
321 /* s != 0 => we have a valid signature */
322 break;
323     }
324     while (1);
```

Nothing here forces s into the lower half of its range. If (r, s) is valid, so is (r, n − s), and anyone relaying the transaction can swap one for the other without the key. That changes the transaction's hash but not its meaning. Bitcoin lived with this malleability for years before the LOW_S rule addressed it.[6]

VERIFYING

Every input of every transaction reaches `CKey::Verify` through `OP_CHECKSIG`. `ECDSA_verify` first parses the DER bytes, then checks the math.

crypto/ecdsa/ecs_vrf.c lines 83–96

```
83 int ECDSA_verify(int type, const unsigned char *dgst, int dgst_len,
84 const unsigned char *sigbuf, int sig_len, EC_KEY *eckey)
85 {
86     ECDSA_SIG *s;
87     int ret=-1;
88
89     s = ECDSA_SIG_new();
90     if (s == NULL) return(ret);
```

The parse succeeds if the bytes can be *read* as a signature. Nothing checks that they were the one correct DER encoding of it, or that the parser used all `sig_len` bytes. Many byte strings could carry the same (r, s), and this version accepted them all.

```
91     if (d2i_ECDSA_SIG(&s, &sigbuf, sig_len) == NULL) goto err;
92     ret=ECDSA_do_verify(dgst, dgst_len, s, eckey);
93 err:
94     ECDSA_SIG_free(s);
95     return(ret);
96     }
```

crypto/ecdsa/ecs_ossl.c lines 403–471

r and s must both be in [1, n − 1]. That is the only range check; a high s passes.

```
403     if (BN_is_zero(sig->r) || BN_is_negative(sig->r) ||
404 BN_ucmp(sig->r, order) >= 0 || BN_is_zero(sig->s) ||
405 BN_is_negative(sig->s) || BN_ucmp(sig->s, order) >= 0)
406     {
407 ECDSAerr(ECDSA_F_ECDSA_DO_VERIFY, ECDSA_R_BAD_SIGNATURE);
408         ret = 0;     /* signature is invalid */
409 goto err;
410     }
411     /* calculate tmp1 = inv(S) mod order */
```

$w = s^{-1} \bmod n$ (kept in `u2`).

```
412     if (!BN_mod_inverse(u2, sig->s, order, ctx))
413     {
414 ECDSAerr(ECDSA_F_ECDSA_DO_VERIFY, ERR_R_BN_LIB);
415 goto err;
416     }
417     /* digest -> m */
418     if (!BN_bin2bn(dgst, dgst_len, m))
419     {
420 ECDSAerr(ECDSA_F_ECDSA_DO_VERIFY, ERR_R_BN_LIB);
421 goto err;
422     }
423     /* u1 = m * tmp mod order */
```

u1 = m·w and u2 = r·w, both mod n.

```
424     if (!BN_mod_mul(u1, m, u2, order, ctx))
425     {
426 ECDSAerr(ECDSA_F_ECDSA_DO_VERIFY, ERR_R_BN_LIB);
427 goto err;
428     }
429     /* u2 = r * w mod q */
430     if (!BN_mod_mul(u2, sig->r, u2, order, ctx))
431     {
432 ECDSAerr(ECDSA_F_ECDSA_DO_VERIFY, ERR_R_BN_LIB);
433 goto err;
434     }
435
436     if ((point = EC_POINT_new(group)) == NULL)
437     {
438 ECDSAerr(ECDSA_F_ECDSA_DO_VERIFY, ERR_R_MALLOC_FAILURE);
439 goto err;
440     }
```

One combined multiplication: u1 × G + u2 × Q, where Q is the public key. This is the costliest line in v0.1.0's validation; libsecp256k1 later did the same verification over five times faster on x86_64.[7]

```
441     if (!EC_POINT_mul(group, point, u1, pub_key, u2, ctx))
442     {
443 ECDSAerr(ECDSA_F_ECDSA_DO_VERIFY, ERR_R_EC_LIB);
444 goto err;
445     }
446     if (EC_METHOD_get_field_type(EC_GROUP_method_of(group)) == NID_X9_62_prime_field)
447     {
448 if (!EC_POINT_get_affine_coordinates_GFp(group,
449             point, X, NULL, ctx))
450 {
451 ECDSAerr(ECDSA_F_ECDSA_DO_VERIFY, ERR_R_EC_LIB);
452 goto err;
453 }
454     }
455     else /* NID_X9_62_characteristic_two_field */
456     {
457 if (!EC_POINT_get_affine_coordinates_GF2m(group,
458             point, X, NULL, ctx))
459 {
460 ECDSAerr(ECDSA_F_ECDSA_DO_VERIFY, ERR_R_EC_LIB);
461 goto err;
462 }
463     }
464
465     if (!BN_nnmod(u1, X, order, ctx))
466     {
467 ECDSAerr(ECDSA_F_ECDSA_DO_VERIFY, ERR_R_BN_LIB);
468 goto err;
469     }
470     /*  if the signature is correct u1 is equal to sig->r */
```

The signature is good if that point's x-coordinate, mod n, equals r. Returns 1; v0.1.0 treats anything else as failure.

```
471     ret = (BN_ucmp(u1, sig->r) == 0);
```

WHERE THE RULES LIVED

Because v0.1.0 calls `ECDSA_verify` directly, whatever OpenSSL accepted *was* Bitcoin's rule for a valid signature. That made OpenSSL's parser part of consensus. The parser was written for certificates, and it read a signature's length fields like this:

crypto/asn1/asn1_lib.c lines 154–191

```
154 static int asn1_get_length(const unsigned char **pp, int *inf, long *rl, int max)
155     {
156     const unsigned char *p= *pp;
157     unsigned long ret=0;
158     unsigned int i;
159
160     if (max-- < 1) return(0);
161     if (*p == 0x80)
162 {
163 *inf=1;
164         ret=0;
165         p++;
166 }
167     else
168 {
169 *inf=0;
170         i= *p&0x7f;
```

A length byte with the high bit set says "the length follows in the next i bytes." DER requires the shortest form; BER, which this parser accepts, does not. A 70-byte signature can legally announce its length in five bytes.

```
171 if (*(p++) & 0x80)
172 {
```

The limit is `sizeof(long)`, which depends on the platform: 4 bytes on Windows and on 32-bit systems, 8 on 64-bit Linux and OS X. Pieter Wuille showed in 2015 that a transaction using a 5-byte length could have split the network along that line.[8]

```
173 if (i > sizeof(long))
174 return 0;
175 if (max-- == 0) return(0);
176 while (i-- > 0)
177 {
178                 ret<<=8L;
179                 ret|= *(p++);
180 if (max-- == 0) return(0);
181 }
182 }
183 else
184             ret=i;
185 }
186     if (ret > LONG_MAX)
187 return 0;
188     *pp=p;
189     *rl=(long)ret;
190     return(1);
191     }
```

While Bitcoin ran only on Windows, as v0.1.0 did, every node had a 4-byte `long` and the split could not happen. Nobody planned it; the bug sat inside a general-purpose library that had never promised to behave identically everywhere. BIP 66 put it plainly: "OpenSSL is not designed for consensus-critical behaviour (it does not guarantee bug-for-bug compatibility between versions)."[9] The warning had already come true. In January 2015 OpenSSL addressed a certificate issue rooted in its acceptance of non-DER signature encodings,[10] and when the new 1.0.0p and 1.0.1k releases stopped accepting some of those deviations, some nodes rejected the chain.[9] BIP 66 responded by making strict DER a consensus rule, fully enforced from July 2015,[8] so Bitcoin itself, not its crypto library, defined what a valid signature looked like. Wuille revealed afterward that the change had also quietly closed the `sizeof(long)` fork.[8]

SEEDING THE POOL

Both random draws above, the private key and k, come from OpenSSL's pool. The first thing v0.1.0 does, in `util.cpp`'s `CInit` constructor before `main` runs, is call `RAND_screen()`. On Windows that function does two things:

crypto/rand/rand_win.c lines 625–629

```
625 void RAND_screen(void) /* function available for backward compatibility */
626 {
```

`RAND_poll` gathers Windows sources: the CryptoAPI generator (`CryptGenRandom`), the process, thread, and heap lists, memory status, the foreground window, the cursor position, and the message queue.

```
627     RAND_poll();
628     readscreen();
629 }
```

crypto/rand/rand_win.c lines 689–746

```
689 static void readscreen(void)
690 {
691 #if !defined(OPENSSL_SYS_WINCE) && !defined(OPENSSL_SYS_WIN32_CYGWIN)
692   HDC       hScrDC; /* screen DC */
693   HDC       hMemDC; /* memory DC */
694   HBITMAP   hBitmap;    /* handle for our bitmap */
695   HBITMAP   hOldBitmap; /* handle for previous bitmap */
696   BITMAP    bm; /* bitmap properties */
697 unsigned int    size; /* size of bitmap */
698 char *bmbits;   /* contents of bitmap */
699 int     w; /* screen width */
700 int     h; /* screen height */
701 int     y; /* y-coordinate of screen lines to grab */
702 int     n = 16; /* number of screen lines to grab at a time */
703
```

This test is backwards. `OPENSSL_isservice()` returns 0 for an ordinary desktop program, so `!OPENSSL_isservice()` is true and the function returns before it reads a single pixel. In 0.9.8h, the screen was scraped only for Windows services, the one kind of program meant to stay off the desktop. OpenSSL fixed it in March 2010 as a "logical bug in readscreen".[11]

```
704 if (GetVersion() >= 0x80000000 || !OPENSSL_isservice())
705 return;
706
707 /* Create a screen DC and a memory DC compatible to screen DC */
708   hScrDC = CreateDC(TEXT("DISPLAY"), NULL, NULL, NULL);
709   hMemDC = CreateCompatibleDC(hScrDC);
710
711 /* Get screen resolution */
712   w = GetDeviceCaps(hScrDC, HORZRES);
713   h = GetDeviceCaps(hScrDC, VERTRES);
714
715 /* Create a bitmap compatible with the screen DC */
716   hBitmap = CreateCompatibleBitmap(hScrDC, w, n);
717
718 /* Select new bitmap into memory DC */
719   hOldBitmap = SelectObject(hMemDC, hBitmap);
720
721 /* Get bitmap properties */
722 GetObject(hBitmap, sizeof(BITMAP), (LPSTR)&bm);
723   size = (unsigned int)bm.bmWidthBytes * bm.bmHeight * bm.bmPlanes;
724
725   bmbits = OPENSSL_malloc(size);
726 if (bmbits) {
727 /* Now go through the whole screen, repeatedly grabbing n lines */
728 for (y = 0; y < h-n; y += n)
729 {
730     unsigned char md[MD_DIGEST_LENGTH];
731
732     /* Bitblt screen DC to memory DC */
733     BitBlt(hMemDC, 0, 0, w, n, hScrDC, 0, y, SRCCOPY);
734
735     /* Copy bitmap bits from memory DC to bmbits */
736     GetBitmapBits(hBitmap, size, bmbits);
737
738     /* Get the hash of the bitmap */
739     MD(bmbits,size,md);
740
741     /* Seed the random generator with the hash value */
```

Even when the loop runs, each screen hash is credited with zero entropy: it is stirred into the pool, but OpenSSL does not count on it.

```
742     RAND_add(md, MD_DIGEST_LENGTH, 0);
743     }
744
745 OPENSSL_free(bmbits);
746 }
```

So the screen capture that `RAND_screen`'s name promises, and that `util.cpp`'s comment describes, never ran in v0.1.0 on a normal Windows desktop. `RAND_poll` did, and it includes the operating system's own cryptographic generator. Satoshi also added his own sources: `RandAddSeed` mixes in the CPU performance counter

and a hash of Windows performance data, and the main window feeds every mouse movement into `RAND_add`. Belt and suspenders turned out to be the right instinct. One of the belts was never fastened.

SUMMARY

Using OpenSSL was the right call in 2009. It gave one person a working, widely deployed implementation of secp256k1, along with hashing, big numbers, and a seeded random number generator. But it also put a general-purpose library inside Bitcoin's consensus rules. Script's `OP_MUL`, `OP_DIV`, and `OP_MOD` went straight to `BN_mul`, `BN_div`, and `BN_mod` until those opcodes were disabled in 2010.[12] Signature validity was whatever `d2i_ECDSA_SIG` parsed until 2015.[9] The same January 2015 advisory that changed signature parsing also fixed a bignum squaring bug reported by Bitcoin developer Pieter Wuille.[10]

The project's answer was to stop depending on that code. Bitcoin Core 0.10 moved signing to libsecp256k1, a library written for this one curve,[13] and 0.12 moved validation too.[7] The excerpts in this appendix are the code that was retired.

KEY TERMS

BER

Basic Encoding Rules: the permissive ASN.1 encoding that allows several byte strings for the same value, such as length fields longer than needed. OpenSSL's parser accepted BER signatures.

Koblitz curve

In SEC 2, a curve whose parameters were chosen for an efficiently computable endomorphism rather than generated from a random seed; secp256k1 is one.

readscreen

The OpenSSL 0.9.8 routine meant to hash the screen's contents into the random pool; in 0.9.8h a reversed test made it return early for ordinary desktop programs.

signature malleability

The ability of a third party to change a signature's bytes, for example by replacing s with n – s or re-encoding it, without invalidating it, which changes the transaction's hash.

SOURCES

1. OpenSSL Project, *OpenSSL 0.9.8h source release (openssl-0.9.8h.tar.gz)* (May 28, 2008). SHA-1 ced4f2da24a202e01ea22bef30ebc8aee274de86; the version named in v0.1.0's readme.txt. Files quoted in the OpenSSL appendix are pinned in src/openssl-manifest.json. https://www.openssl.org/source/old/0.9.x/openssl-0.9.8h.tar.gz
2. OpenSSL Project, *OpenSSL NEWS: Major changes between OpenSSL 0.9.7g and OpenSSL 0.9.8* (July 5, 2005). NEWS file in the release tarball: "Major work on Elliptic Crypto; ECDH and ECDSA added". https://www.openssl.org/source/old/0.9.x/openssl-0.9.8h.tar.gz
3. Wei Dai, *Crypto++ Library* (1995–). SHA-256 implementation lineage referenced in sha.cpp comments. https://www.cryptopp.com/
4. Certicom Research (Standards for Efficient Cryptography Group), *SEC 2: Recommended Elliptic Curve Domain Parameters, Version 1.0* (September 20, 2000). Defines secp256k1; Koblitz-curve parameters versus verifiably random ones. https://www.secg.org/SEC2-Ver-1.0.pdf
5. fail0verflow, *Console Hacking 2010 — PS3 ECDSA nonce reuse* (December 2010). Classic demonstration that reusing ECDSA nonces leaks private keys. https://events.ccc.de/congress/2010/Fahrplan/events/4087.en.html
6. Johnson Lau, Pieter Wuille, *BIP 146 — Dealing with signature encoding malleability* (2016). LOW_S: negating S modulo the curve order leaves an ECDSA signature valid. https://github.com/bitcoin/bips/blob/master/bip-0146.mediawiki
7. Bitcoin Core Contributors, *Bitcoin Core 0.12.0 release notes* (February 2016). Signature validation using libsecp256k1; over five times faster on x86_64. https://bitcoincore.org/en/releases/0.12.0/

8. Pieter Wuille, *Disclosure: consensus bug indirectly solved by BIP66 (bitcoin-dev)* (July 28, 2015). OpenSSL's BER length parsing depended on the size of a C long, so 32-bit/Windows and 64-bit Linux/OS X nodes could disagree. https://gnusha.org/url/https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2015-July/009697.html
9. Pieter Wuille, *BIP 66 — Strict DER signatures* (2015). "OpenSSL is not designed for consensus-critical behaviour"; activated by version-3 blocks in July 2015. https://github.com/bitcoin/bips/blob/master/bip-0066.mediawiki
10. OpenSSL Project, *OpenSSL Security Advisory [08 Jan 2015]* (January 8, 2015). CVE-2014-3570 (BN_sqr, reported by Pieter Wuille) and CVE-2014-8275 (non-DER signature encodings); fixed in 1.0.1k, 1.0.0p, 0.9.8zd. https://openssl-library.org/news/secadv/20150108.txt
11. Andy Polyakov (OpenSSL Project), *rand_win.c: fix logical bug in readscreen* (March 22, 2010). Reverses the service test so the screen is skipped only for Windows services; first released in OpenSSL 0.9.8n. https://github.com/openssl/openssl/commit/c3484e0268cfa7b7c6db81e01fc0f837bf375e52
12. Satoshi Nakamoto / Bitcoin developers, *OP_RETURN and dangerous opcodes disabled* (July 2010). Response to early Script vulnerabilities. https://en.bitcoin.it/wiki/Script#Disabled_opcodes
13. Bitcoin Core Contributors, *Bitcoin Core 0.10.0 release notes* (February 2015). Signing switched from OpenSSL to libsecp256k1. https://bitcoincore.org/en/releases/0.10.0/

OPENSSL LICENSE

The OpenSSL excerpts above are reproduced under the OpenSSL and original SSLeay licenses, whose full text follows as it appears in the 0.9.8h release. This product includes software developed by the OpenSSL Project for use in the OpenSSL Toolkit (http://www.openssl.org/).

LICENSE OpenSSL 0.9.8h

```

 LICENSE ISSUES
 ==============

 The OpenSSL toolkit stays under a dual license, i.e. both the conditions of
 the OpenSSL License and the original SSLeay license apply to the toolkit.
 See below for the actual license texts. Actually both licenses are BSD-style
 Open Source licenses. In case of any license issues related to OpenSSL
 please contact openssl-core@openssl.org.

 OpenSSL License
 ---------------

/* ====================================================================
 * Copyright (c) 1998-2008 The OpenSSL Project.  All rights reserved.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 *
 * 1. Redistributions of source code must retain the above copyright
 *    notice, this list of conditions and the following disclaimer.
 *
 * 2. Redistributions in binary form must reproduce the above copyright
 *    notice, this list of conditions and the following disclaimer in
 *    the documentation and/or other materials provided with the
 *    distribution.
 *
 * 3. All advertising materials mentioning features or use of this
 *    software must display the following acknowledgment:
 *    "This product includes software developed by the OpenSSL Project
 *    for use in the OpenSSL Toolkit. (http://www.openssl.org/)"
 *
 * 4. The names "OpenSSL Toolkit" and "OpenSSL Project" must not be used to
 *    endorse or promote products derived from this software without
 *    prior written permission. For written permission, please contact
 *    openssl-core@openssl.org.
 *
 * 5. Products derived from this software may not be called "OpenSSL"
 *    nor may "OpenSSL" appear in their names without prior written
 *    permission of the OpenSSL Project.
 *
 * 6. Redistributions of any form whatsoever must retain the following
 *    acknowledgment:
 *    "This product includes software developed by the OpenSSL Project
 *    for use in the OpenSSL Toolkit (http://www.openssl.org/)"
 *
 * THIS SOFTWARE IS PROVIDED BY THE OpenSSL PROJECT ``AS IS'' AND ANY
 * EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
 * PURPOSE ARE DISCLAIMED.  IN NO EVENT SHALL THE OpenSSL PROJECT OR
 * ITS CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
 * SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT
 * NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES;
 * LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
 * STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
 * ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED
```

```
 * OF THE POSSIBILITY OF SUCH DAMAGE.
 * ====================================================================
 *
 * This product includes cryptographic software written by Eric Young
 * (eay@cryptsoft.com).  This product includes software written by Tim
 * Hudson (tjh@cryptsoft.com).
 *
 */

Original SSLeay License
-----------------------

/* Copyright (C) 1995-1998 Eric Young (eay@cryptsoft.com)
 * All rights reserved.
 *
 * This package is an SSL implementation written
 * by Eric Young (eay@cryptsoft.com).
 * The implementation was written so as to conform with Netscapes SSL.
 *
 * This library is free for commercial and non-commercial use as long as
 * the following conditions are aheared to.  The following conditions
 * apply to all code found in this distribution, be it the RC4, RSA,
 * lhash, DES, etc., code; not just the SSL code.  The SSL documentation
 * included with this distribution is covered by the same copyright terms
 * except that the holder is Tim Hudson (tjh@cryptsoft.com).
 *
 * Copyright remains Eric Young's, and as such any Copyright notices in
 * the code are not to be removed.
 * If this package is used in a product, Eric Young should be given attribution
 * as the author of the parts of the library used.
 * This can be in the form of a textual message at program startup or
 * in documentation (online or textual) provided with the package.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the copyright
 *    notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 *    notice, this list of conditions and the following disclaimer in the
 *    documentation and/or other materials provided with the distribution.
 * 3. All advertising materials mentioning features or use of this software
 *    must display the following acknowledgement:
 *    "This product includes cryptographic software written by
 *     Eric Young (eay@cryptsoft.com)"
 *    The word 'cryptographic' can be left out if the rouines from the library
 *    being used are not cryptographic related :-).
 * 4. If you include any Windows specific code (or a derivative thereof) from
 *    the apps directory (application code) you must include an acknowledgement:
 *    "This product includes software written by Tim Hudson (tjh@cryptsoft.com)"
 *
 * THIS SOFTWARE IS PROVIDED BY ERIC YOUNG ``AS IS'' AND
 * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
 * ARE DISCLAIMED.  IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
 * FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
 * DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
 * OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
 * LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
 * OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
 * SUCH DAMAGE.
 *
 * The licence and distribution terms for any publically available version or
 * derivative of this code cannot be changed.  i.e. this code cannot simply be
 * copied and put under another distribution licence
 * [including the GNU Public Licence.]
 */

```

GLOSSARY

The Key Terms from every chapter, merged. Page numbers point to the chapter where each term is defined.

AcceptReview
The routine run on each incoming review message that verifies its signature, appends it to the recipient's stored reviews, records the reviewer's trust link, and propagates atoms.
market.cpp p. 309

addr gossip
The relaying of peer addresses in addr messages, letting the network discover new participants without any central directory.
net.h p. 366

address
The string you share to receive payment: a hash160 plus a version byte, Base58Check-encoded into something like 1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa.
base58.h p. 98

AddTimeData
The function that records each new peer's clock offset once per IP address and, after at least five samples, adopts their median as the node's time offset. util.cpp p. 683

AdvertInsert
The function that adds a product to mapProducts, or replaces an existing entry with the same hash only when the incoming listing carries a higher nSequence. market.cpp p. 309

advertising clause
A license term inherited from the original BSD license that obliges credit to the OpenSSL Project and Eric Young, which is why the README names them. readme.txt p. 376

alt stack
A secondary stack on which a script can park a value with OP_TOALTSTACK and retrieve it later with OP_FROMALTSTACK. script.cpp p. 403

arbitrary precision
Arithmetic on integers of any size instead of the fixed 32 or 64 bits a processor handles natively, required for 256-bit keys and targets.
bignum.h p. 112

ASIC
An application-specific integrated circuit; modern miners are chips that perform this file's Transform routine in silicon billions of times per second. sha.h p. 470

atom propagation
The wave-by-wave process in AddAtomsAndPropagate that adds atoms to a user, then passes any newly forwarded atoms to everyone that user reviewed, until no one forwards anything new. market.cpp p. 309

avalanche effect
The property that flipping a single input bit changes about half the output bits, which is why mining is a lottery with no partial progress.
sha.cpp p. 460

Base58
The binary-to-text encoding whose 58-character alphabet drops 0, O, I, and l, so addresses can be read aloud or typed without ambiguity.
base58.h p. 98

Base58Check
Base58 with a four-byte checksum appended, so a mistyped address fails validation with only about a one-in-four-billion chance of slipping through. base58.h p. 98

BER
Basic Encoding Rules: the permissive ASN.1 encoding that allows several byte strings for the same value, such as length fields longer than needed. OpenSSL's parser accepted BER signatures. Appendix: OpenSSL p. 726

Berkeley DB
The embedded, in-process key-value database library that holds all of v0.1.0's persistent data, providing transactions, logging, and crash recovery without a separate database server.
db.h p. 148 · readme.txt p. 376

bit shift
An operation sliding every bit left or right; right-shifting the subsidy by the era number divides the block reward by two at each halving.
uint256.h p. 605

#bitcoin
The IRC channel every v0.1.0 node joins at startup; its member list doubles as a live directory of nodes, since each reachable node's nickname encodes its IP address and port.
irc.h p. 177

block header
The 80 bytes that are hashed for proof-of-work: version, previous block hash, merkle root, timestamp, nBits, and nonce. main.h p. 280

BN_CTX
OpenSSL's scratch context holding temporary values during multi-step calculations, managed here by the RAII wrapper CAutoBN_CTX.
bignum.h p. 112

Boost
A widely used collection of C++ libraries; headers.h pulls in its foreach, lexical_cast, tuple, and array components. headers.h p. 156

bootstrap problem
The puzzle every new peer-to-peer node faces on first launch: it must connect to existing peers, yet it knows no peer addresses until some peer tells it. irc.h p. 177

burn address
A syntactically valid address with no known private key, so anything sent to it can never be spent. base58.h p. 98

CAddrDB
The CDB subclass for addr.dat, which saves every known peer address so a restarted node can reconnect without rediscovering the network from scratch. db.h p. 148

CAddressBookDialog
The window listing other people's addresses with names; opened standalone it manages contacts, and opened from Send Coins it returns the chosen address. ui.h p. 532

carry
The overflow from one 32-bit word into the next during addition, propagated word by word like grade-school column addition. uint256.h p. 605

CBigNum
Bitcoin's C++ wrapper around OpenSSL's BIGNUM, letting the code write a + b on numbers far larger than any CPU register holds.
bignum.h p. 112

CBlockIndex
The in-memory record holding a known block's metadata and links to its predecessor and successor, so the node can walk the chain without loading full blocks from disk. main.h p. 280

CCriticalSection
A thin wrapper around the Windows CRITICAL_SECTION mutex that initializes and deletes it automatically and records the file and line of the current holder. util.h p. 698

CDataStream
Bitcoin's byte buffer with stream operators that serialize objects as they pass through; every network message flows through one.
serialize.h p. 444

CDB
The base class wrapping one Berkeley DB file, offering templated Read, Write, Erase, and Exists methods that serialize any key and value through a CDataStream before touching disk. db.h p. 148

chain reorganization
The switch to a competing longer valid chain, in which the old tip's blocks are disconnected and their transactions return to the mempool.
main.cpp p. 250

checkpoint
A txn_checkpoint call that flushes logged changes into the data files; v0.1.0 checkpoints whenever a database closes and in DBFlush, which also resets the log sequence numbers of idle files. db.cpp p. 132

class C

A block of IPv4 addresses sharing their first three bytes (a /24); ThreadOpenConnections picks a random class C before picking an IP inside it, blunting address flooding from a single block. net.cpp p. 344

CMainFrame

The main application window, derived from CMainFrameBase, which shows the balance, receiving address, and transaction list and overrides 21 of the generated event handlers. ui.h p. 532

CMyApp

The wxApp subclass that serves as the program's entry point; IMPLEMENT_APP generates the real main function, and OnInit2 loads the wallet, opens the window, and starts the node. ui.cpp p. 521

CNode

The class representing one live connection to a peer, holding its buffers, negotiated version, and records of what that peer has and knows. net.h p. 366

coinbase

The special first transaction of a block whose input references nothing (a null prevout) and whose outputs pay the miner the subsidy plus fees. main.cpp p. 250

coinbase maturity

The rule (COINBASE_MATURITY, 100 blocks) that freshly mined coins may not be spent until their block is buried 100 deep. main.h p. 280

collision resistance

The property that no one can find two different inputs with the same hash, keeping transaction IDs and block hashes unique. sha.h p. 470

compact target

The 4-byte nBits encoding of the 256-bit proof-of-work threshold, one exponent byte plus three mantissa bytes, decoded by SetCompact. bignum.h p. 112

CompactSize

The variable-length integer encoding that stores a count in one byte when it is under 253, growing to 3, 5, or 9 bytes only for larger values. serialize.h p. 444

compression function

The core routine SHA256::Transform that mixes one 512-bit block into the running hash state, executed trillions of times per second across the mining network. sha.cpp p. 460

Connect()

The wxWidgets call that binds an event on a widget to a handler method at run time; uibase.cpp makes 200 such calls, each undone by a matching Disconnect() in a destructor. uibase.cpp p. 569

copyleft

A licensing approach, used by the GPL, that requires derived works to be distributed under the same open terms; the MIT License deliberately has none. license.txt p. 195

COutPoint

The pair of a transaction hash and an output index that uniquely names one coin, and the pointer every input uses to say what it spends. main.h p. 280

CProduct

A signed marketplace listing carrying the seller's network address, summary fields, optional details and order form, a sequence number, and the seller's public key and signature. market.h p. 319

CReview

A signed marketplace review: the recipient's hash, free-form key-value fields, the reviewer's public key, and an ECDSA signature over everything except the signature itself. market.h p. 319

CReviewDB

The CDB subclass for reviews.dat, storing each user's atom state under a "user" key and the reviews they have received under a "reviews" key. market.cpp p. 309

CRITICAL_BLOCK

A macro that locks a critical section for the duration of the following block and releases it on exit, even when an exception is thrown. util.h p. 698

cross-thread call

Posting a wxCommandEvent to the main window's queue with AddPendingEvent, so that the UI thread rather than a worker thread performs the actual screen update. ui.cpp p. 521

CScript
The class that holds a script as a vector of bytes; a script is inert data until EvalScript runs it. script.h p. 419

CSendDialog
The Send Coins window, which accepts a Bitcoin address or an IP address plus an amount, checks the address as it is typed, and starts the payment. ui.h p. 532

CSendingDialog
The progress window for a payment to an IP address, tracking the live exchange with the recipient's node through flags such as fCanCancel and fWorkDone. ui.h p. 532

CTxDB
The CDB subclass for blkindex.dat, holding the transaction index, the block index, and the hash of the best chain; it opens no file at all when fClient is set. db.h p. 148

CTxIndex
The record CTxDB stores under each confirmed transaction's hash: the transaction's position on disk plus one vSpent entry per output, noting where that output was later spent. db.cpp p. 132

cursor
A Berkeley DB iterator obtained with GetCursor and advanced with ReadAtCursor, used to walk records in key order, such as every block-index entry or every wallet record at startup. db.h p. 148

CUser
A participant in the marketplace's reputation graph, stored in reviews.dat under a user hash, holding received, pending, and forwarded atoms plus links to the users it has reviewed. market.h p. 319

custom event type
An application-defined wxWidgets event declared with DECLARE_EVENT_TYPE, such as wxEVT_CROSSTHREADCALL or wxEVT_REPLY1, used to carry messages from worker threads to the UI thread. ui.h p. 532

CWalletDB
The CDB subclass for wallet.dat, storing address-book names, wallet transactions, unencrypted key pairs, the default key, and settings: the one file whose loss means lost coins. db.h p. 148

CYourAddressDialog
The window listing the user's own receiving addresses, where each can be labeled, copied to the clipboard, or joined by a freshly generated one. ui.h p. 532

database environment
The single global DbEnv, opened lazily by the first CDB constructor in the application directory, that every database file shares for locking, logging, caching, and transactions. db.cpp p. 132

database transaction
A Berkeley DB unit of work opened with TxnBegin and finished with TxnCommit or TxnAbort, so its writes either all take effect or none do; the vTxn stack lets them nest. db.h p. 148

DB_NOOVERWRITE
The Berkeley DB flag Write passes when fOverwrite is false, making the write fail instead of replacing an existing key; WriteKey uses it so a stored private key is never overwritten. db.h p. 148

DB_RECOVER
The flag passed when opening the database environment that makes Berkeley DB run recovery from its log first, restoring the files to a consistent state after an unclean shutdown. db.cpp p. 132

DER
Distinguished Encoding Rules, the binary ASN.1 format wrapping private keys (279 bytes) and signatures (up to 72 bytes) for storage and transmission. key.h p. 187

derived window class
A hand-written class such as CSendDialog that inherits a generated base class from uibase.h and overrides its event handlers to add Bitcoin's behavior. ui.h p. 532

deterministic
The property that the same object always serializes to the same bytes; without it two nodes would hash the same transaction differently and fork. serialize.h p. 444

difficulty target
The 256-bit threshold a block's hash must not exceed, stored compactly in the header as nBits and recalculated every 2016 blocks to hold the ten-minute average. main.cpp p. 250

digest
The fixed-size output a hash function produces, which changes completely if even one input bit flips. sha.cpp p. 460

disabled opcode
An instruction such as OP_CAT or OP_MUL that stays in the enum but was turned off in August 2010 after denial-of-service bugs; any script using it fails. script.h p. 419

double-SHA256
Bitcoin's standard construction SHA256(SHA256(data)), used for block hashes and transaction IDs as a safety margin against length-extension attacks. sha.h p. 470

ECDSA
The Elliptic Curve Digital Signature Algorithm, by which a private key signs a hash and anyone holding the public key can verify it without learning the key. key.h p. 187

encoded nickname
A nickname made of the letter u followed by the Base58Check encoding of an ircaddr, about 15 characters in all, from which any channel member can recover a node's IP address and port. irc.cpp p. 170

endianness
The byte-order convention of a machine; SHA-256 is defined big-endian while most CPUs are little-endian, so ByteReverse swaps bytes before hashing. sha.h p. 470

entropy
Unpredictable input, such as timing jitter or system statistics, fed into a random number generator so its output cannot be guessed by an attacker. util.cpp p. 683

error()
A logging helper that prints its formatted message prefixed with "ERROR:" and always returns false, so callers can write return error(...) in one line. util.cpp p. 683

EvalScript
The interpreter function that runs a whole script and returns true only if execution finishes with a truthy value on top of the stack. script.cpp p. 403

.fbp project file
wxFormBuilder's XML save format, recording each window, widget, and setting as nested object and property elements; this one describes sixteen top-level windows in 11,825 lines. uiproject.fbp p. 669

FLATDATA
The macro serializing a fixed-layout struct as raw bytes with no length prefix, for fields whose wire format is exactly their memory layout. serialize.h p. 444

foreach
A one-word alias for Boost's BOOST_FOREACH macro, letting the code loop over any STL container without spelling out iterator types. util.h p. 698

FormatMoney
The function that turns an integer amount of satoshis into display text with two decimal places, comma thousands separators, and an optional plus sign. util.cpp p. 683

fRestartIRCSeed
A global flag that, when set, makes ThreadIRCSeed drop its IRC connection and reconnect from the top; nothing in v0.1.0 ever sets it to true. irc.h p. 177

generated base class
A class such as CMainFrameBase that wxFormBuilder writes out, holding the widgets and empty event handlers; ui.h derives the working window classes from it. uibase.h p. 585

generated code
Source written by a tool rather than a person; uibase.cpp opens with PLEASE DO "NOT" EDIT THIS FILE! because wxFormBuilder overwrites it whenever the design is regenerated. uibase.cpp p. 569

GetAdjustedTime
The node's notion of network time: the local system clock plus the median offset reported by peers, used for timestamps and the two-hour future-block limit. util.cpp p. 683

getdata
The message a node sends to pull the actual transaction or block after learning of it from a peer's inv announcement. net.h p. 366

GNU Make
The build tool that reads this makefile, compares file timestamps against each rule's dependencies, and reruns only the commands whose outputs are out of date. makefile p. 290

halving
The event every 210,000 blocks that right-shifts the subsidy in GetBlockValue, cutting the block reward in half and converging the total supply toward 21 million. main.cpp p. 250

hash function
A function mapping any input to a fixed-size fingerprint that is easy to compute but infeasible to reverse. sha.h p. 470

hash160
The 160-bit RIPEMD160(SHA256(pubkey)) digest of a public key at the heart of every address. base58.h p. 98 · uint256.h p. 605

hashBestChain
The blkindex.dat record holding the hash of the current best-chain tip, rewritten as the chain advances and read by LoadBlockIndex at startup to restore pindexBest and nBestHeight. db.cpp p. 132

headers.h.gch
GCC's precompiled-header file, built from headers.h by its own makefile rule and picked up automatically whenever a source file includes headers.h. makefile p. 290

hex
Base-16 notation using digits 0-9 and a-f, the 64-character form in which hashes appear in debug output and block explorers. uint256.h p. 605

hidden widget
A control the generated code creates and then immediately hides with Hide(), as with the main window's transaction filter and the four marketplace list panels. uibase.cpp p. 569

ICON and BITMAP
The two resource types declared in ui.rc: ICON for Windows icons such as the application coin and the check mark, BITMAP for the toolbar pictures and their masks. ui.rc p. 538

idle-time refresh
The pattern in which RefreshListCtrl only sets a flag and wakes the idle loop, leaving OnIdle to rebuild the transaction list when the interface has nothing else to do. ui.cpp p. 521

IMPLEMENT_RANDOMIZE_STACK
A macro that makes a thread function call itself a random number of times, up to 50, so its working stack frame lands at an unpredictable depth. util.h p. 698

IMPLEMENT_SERIALIZE
The macro that expands one block of READWRITE statements into three functions, size, write, and read, so encoding and decoding can never drift apart. serialize.h p. 444

int64
Satoshi's portable 64-bit signed integer type, mapped to __int64 on MSVC and Borland and to long long on GCC, used for amounts and timestamps. util.h p. 698

inv
The inventory message advertising a list of (type, hash) pairs, so peers can request only the transactions or blocks they lack. net.h p. 366

IRC
Internet Relay Chat, the line-based text chat protocol whose public servers v0.1.0 borrows as a meeting point where Bitcoin nodes announce their addresses and find one another. irc.h p. 177

ircaddr
The packed 6-byte struct, a 4-byte IPv4 address followed by a 2-byte port, whose Base58Check encoding becomes a node's IRC nickname. irc.cpp p. 170

JOIN
The IRC command for entering a channel; the server also relays each newcomer's JOIN to existing members, so a running node learns of newly started peers in real time. irc.cpp p. 170

Koblitz curve
In SEC 2, a curve whose parameters were chosen for an efficiently computable endomorphism rather than generated from a random seed; secp256k1 is one. Appendix: OpenSSL p. 726

LGPL

The GNU Lesser General Public License, a weaker form of copyleft that permits linking a library into programs under other licenses; wxWidgets used version 2.1. readme.txt p. 376

libeay32

The Windows build of OpenSSL's general cryptography library, the only part of OpenSSL that Bitcoin needs; the SSL protocol library is never used. readme.txt p. 376

little-endian

The byte order storing the least significant word first in the pn[] array, which makes carries ripple naturally through the arithmetic. uint256.h p. 605

Load From Resource

The bitmap source chosen for the toolbar buttons, which makes the generated code fetch images such as send20 by name from resources compiled into the executable rather than from files on disk. uiproject.fbp p. 669

lock limit

The ceiling of 10,000 Berkeley DB locks and lock objects set with set_lk_max_locks and set_lk_max_objects at startup, a tuning value later implicated in the chain split described in BIP 50. db.cpp p. 132

magic bytes

The fixed 4-byte marker 0xf9beb4d9 that starts every message, chosen to be unlikely in ordinary data so message boundaries are easy to find. net.h p. 366

mapAddressBook

The map from Bitcoin address strings to contact names that labels rows in the transaction list; it is saved in wallet.dat as "name" records. ui.cpp p. 521

mapAddresses

The in-memory table of known peer addresses, keyed by serialized IP and port, loaded from addr.dat at startup and consulted whenever the node chooses whom to dial next. net.cpp p. 344

mapProducts

The global, mutex-guarded in-memory catalog of product listings keyed by product hash; its sibling mapMyProducts holds the user's own listings. market.h p. 319

mempool

The node's in-memory pool (mapTransactions) of valid but unconfirmed transactions, where they wait for a miner to include them in a block. main.cpp p. 250

merkle root

The single hash stored in a block header that commits to every transaction in the block, built by repeatedly double-hashing pairs of transaction IDs. main.cpp p. 250

Merkle-Damgård

The construction that chains input blocks together by adding each block's output into the next block's starting state, so one input bit affects the entire final hash. sha.cpp p. 460

message header

The 20-byte prefix on every network message: four magic bytes, a 12-byte command name, and the payload size. net.h p. 366

message schedule

The expansion that stretches 16 words of input into the 64 words the rounds consume, so every input bit influences many rounds. sha.cpp p. 460

MinGW

Minimalist GNU for Windows, a port of the GCC compiler and tools that builds native Windows programs; the README names version 3.4.5. makefile p. 290

MIT License

A short permissive open-source license, also called the X11 license, that lets anyone use, modify, and sell the software if the notice is kept. license.txt p. 195

modular arithmetic

Math in which results wrap around at a fixed modulus; ECDSA signing and verification are built from modular multiplication and inversion. bignum.h p. 112

MPI

Multi-Precision Integer, OpenSSL's big-endian byte format with a length prefix and a sign bit, used to store and transmit big numbers. bignum.h p. 112

MSYS
A minimal Unix-style shell environment for Windows that accompanies MinGW, used here to run Berkeley DB's configure script and make. readme.txt p. 376

multisig
A spending condition requiring M of N possible signatures, verified by OP_CHECKMULTISIG and used for shared custody such as escrows and treasuries. script.h p. 419

nFlowthroughRate
The constant 2 governing atom forwarding: once a user has collected this many new atoms, or has never forwarded one, a randomly chosen new atom flows on to the users they reviewed. market.h p. 319

nLockTime
The transaction field that, in v0.1.0, gives the block height the chain must pass before the transaction is final; zero means it is immediately final. Later versions also let it hold a timestamp. main.h p. 280

NMAKE
Microsoft's make utility, which reads makefile.vc and uses its own syntax, such as !IF and !ENDIF directives, instead of GNU Make's ifeq and endif. makefile.vc p. 297

no-everything build
Satoshi's name for an OpenSSL build configured with options like no-rsa and no-aes, leaving out the encryption routines Bitcoin never calls. readme.txt p. 376

NODE_NETWORK
The service flag a node sets to declare itself a full node willing to serve historical blocks to its peers. net.h p. 366

nonce
The fresh random value k that each ECDSA signature consumes; reusing one across two signatures lets anyone algebraically recover the private key. key.h p. 187 · main.cpp p. 250

nothing-up-my-sleeve number
A constant derived from a simple verifiable formula such as square roots of primes, proving the designers had no freedom to hide a weakness. sha.cpp p. 460

OP_CHECKMULTISIG
The opcode implementing M-of-N multi-signature, succeeding when at least M of the supplied signatures verify against the N listed public keys. script.cpp p. 403

OP_CHECKSIG
The opcode that pops a public key and a signature and verifies them with ECDSA, pushing 1 on success; the lock on virtually every standard payment. script.cpp p. 403

OP_HASH160
The opcode computing RIPEMD160(SHA256(x)), the 20-byte hash of a public key that a Bitcoin address encodes. script.h p. 419

OP_RETURN
The opcode that in v0.1.0 merely ended script execution, letting anyone spend any coin; v0.3.5 (July 2010) made it fail immediately, which later gave a clean way to embed data in provably unspendable outputs. script.h p. 419

opcode
A single-byte instruction such as OP_DUP or OP_CHECKSIG that the interpreter executes one after another. script.cpp p. 403 · script.h p. 419

OpenSSL
The open-source cryptography library that supplies Bitcoin v0.1.0's elliptic-curve signatures, SHA-256, RIPEMD-160, and secure random numbers. headers.h p. 156

operator overloading
The C++ feature letting a class redefine arithmetic and comparison symbols, so big-number code reads like ordinary math. bignum.h p. 112

optimization level
A compiler setting trading build speed and debuggability for faster code; this makefile uses -O0 (none) everywhere except sha.cpp, which gets -O3. makefile p. 290

origin atom
An atom added with fOrigin set, meaning the user created it; it is recorded and queued for forwarding at once, a path only the commented-out mining-reward code in main.cpp would have used. market.cpp p. 309

orphan transaction
A transaction that arrives before the parent whose output it spends, parked in mapOrphanTransactions until the parent shows up. main.cpp p. 250

OutputDebugStringF
The logging function that every printf call is redirected to on Windows; in debug builds it appends to debug.log and the Windows debugger output. util.h p. 698

P2PKH
Pay-to-public-key-hash, the standard script pattern OP_DUP OP_HASH160 hash OP_EQUALVERIFY OP_CHECKSIG that pays whoever can sign with the key behind an address. script.cpp p. 403

pay-to-IP-address
v0.1.0's online payment mode, in which the sender connects to the recipient's node, obtains a public key with checkorder, and delivers the signed transaction with submitorder. ui.cpp p. 521

permissive license
An open-source license that imposes almost no conditions on reuse, allowing derived works to be distributed under any terms, including closed-source ones. license.txt p. 195

PING/PONG
The IRC keepalive exchange: the server sends PING and expects PONG in reply, which RecvLineIRC produces by overwriting the I in the received line with an O and echoing it back. irc.cpp p. 170

point multiplication
The one-way operation that derives a public key by adding the curve's generator point to itself private-key times; reversing it is the discrete logarithm problem. key.h p. 187

poker dialogs
CPokerLobbyDialogBase and CPokerDialogBase, fully laid-out windows for a card game with Deal Hand, Fold, Call, and Raise controls; no class in ui.h ever derives from them. uibase.h p. 585

#pragma hdrstop
A compiler directive marking where the precompiled portion of a header ends; everything included above it can be compiled once and cached. headers.h p. 156

precompiled header
A header whose parsed contents the compiler saves to disk once and reuses for every source file, sparing it from re-reading large library headers each time. headers.h p. 156

preimage resistance
The property that given a hash output you cannot find an input producing it, which makes proof-of-work a lottery rather than a puzzle to solve. sha.h p. 470

private key
A 256-bit random number that controls coins; anyone who learns it can spend the funds, so CKey zeroes it from memory on destruction. key.h p. 187

program database
The .pdb file MSVC writes when building with /Zi or linking with /DEBUG, holding the symbols a debugger needs to map machine code back to source. makefile.vc p. 297

protocol version
The VERSION constant (101 for v0.1.0) carried in every stream so newer software can still parse data written by older releases. serialize.h p. 444

public key
The 65-byte curve point (a 0×04 prefix plus x and y coordinates) derived from a private key, safe to publish and used to verify signatures. key.h p. 187

RAII
The C++ pattern in which a constructor acquires a resource and the destructor releases it, guaranteeing cleanup even when an exception is thrown. bignum.h p. 112

RandAddSeed
The function that mixes the CPU performance counter, and at most every five minutes a hash of Windows performance data, into OpenSSL's random pool. util.cpp p. 683

rc
Microsoft's Resource Compiler, which turns ui.rc and its icon and bitmap files into a compiled .res file that the linker embeds in bitcoin.exe. makefile.vc p. 297

readscreen
The OpenSSL 0.9.8 routine meant to hash the screen's contents into the random pool; in 0.9.8h a reversed test made it return early for ordinary desktop programs. Appendix: OpenSSL p. 726

READWRITE
The macro applied to each field that counts, writes, or reads it depending on which of the three operations is in progress. serialize.h p. 444

record type tag
The short string, such as "tx", "blockindex", "name", or "key", that begins every database key, grouping records by kind so cursors can range-scan them and LoadWallet can dispatch on them. db.cpp p. 132

RecvLine
The helper that reads one line from a socket a byte at a time, stopping at the carriage return; exported here so net.cpp can reuse it for the HTTP reply in GetMyExternalIP. irc.h p. 177

reference count
The per-node count raised by AddRef and lowered by Release, plus an optional timed hold in nReleaseTime, that keeps a disconnected node in memory until no thread still uses it. net.cpp p. 344

rejection sampling
Discarding random draws that fall beyond the largest multiple of the desired range, so that taking the remainder gives every result an equal chance. util.cpp p. 683

relay
The forwarding of transactions and blocks to every connected peer except those already known to have them, flooding new data across the mesh. net.h p. 366

report mode
The wxLC_REPORT style that makes a wxListCtrl show rows under column headings, like a spreadsheet; the main transaction list combines it with descending sort order. uibase.cpp p. 569

reputation atom
A small unsigned number serving as one unit of reputation; atoms flow from reviewer to reviewed, and a user's score is simply how many distinct atoms have reached them. market.h p. 319

resource compiler
The build tool that turns a resource script into binary data linked into the executable: windres in the MinGW makefile, Microsoft's rc in makefile.vc. ui.rc p. 538

resource script
A plain-text .rc file listing the icons, bitmaps, and other assets a Windows program needs, each under a symbolic name the code can ask for at run time. ui.rc p. 538

reverse Polish notation
The Forth-style syntax Script uses, where operators follow their operands (3 4 + instead of 3 + 4), so no parsing or precedence rules are needed. script.cpp p. 403

RIPEMD-160
The 160-bit hash applied after SHA-256 when deriving addresses, an independent second design so addresses never depend on one algorithm alone. sha.h p. 470

rotation
A bit shift in which bits pushed off one end reappear at the other; SHA-256's mixing functions are XORs of rotations such as rotr(x,2), rotr(x,13), and rotr(x,22). sha.h p. 470

round constant
One of the 64 fixed 32-bit values added per round, each taken from the fractional cube root of a prime to keep the rounds from being self-similar. sha.cpp p. 460

RPL_MYINFO
IRC numeric reply 004, the last of the server's welcome messages; ThreadIRCSeed waits for it after sending NICK and USER before joining #bitcoin. irc.cpp p. 170

satoshi
The smallest money unit in this code, one hundred-millionth of a bitcoin (COIN = 100,000,000), used as the integer unit for all amounts. main.h p. 280

script
Bitcoin's stack-based language, defined by this header, that encodes the conditions under which coins can be spent. script.h p. 419

scriptPubKey
The small program stored in each output that states the conditions for spending it, typically a signature matching a given public key hash.
main.h p. 280

scriptSig
The data packed into an input to satisfy the previous output's scriptPubKey, usually a signature plus the public key it was made with.
main.h p. 280

secp256k1
The particular elliptic curve Bitcoin signs on, chosen because its constants are verifiably non-random rather than taken from NIST. key.h p. 187

secure_allocator
The memory allocator used for key material that overwrites buffers with zeros before freeing them, so secrets do not linger in freed memory.
key.h p. 187 · serialize.h p. 444

serialization
The conversion of in-memory objects to bytes for disk, network, or hashing, and back again.
serialize.h p. 444

SerializeHash
A helper that serializes any object into a byte stream and returns the double-SHA256 of those bytes, the standard way objects are given their hash. util.h p. 698

setCallbackAvailable
The set of dialog pointers still open, checked before a network reply is posted, so a response that arrives after its dialog has closed is quietly dropped. ui.cpp p. 521

SHA-1
The older 160-bit hash implemented in this file's first half; Bitcoin never uses it, and it shipped only because the Crypto++ source came as a bundle.
sha.cpp p. 460

SHA-256
The NIST hash standard (FIPS 180-2) that maps any input to 256 bits through 64 rounds of mixing, and underlies every block hash and transaction ID. sha.cpp p. 460

sighash
The flag byte appended to a signature (SIGHASH_ALL, NONE, or SINGLE, optionally with ANYONECANPAY) that selects which inputs and outputs the signature commits to.
script.cpp p. 403

signature malleability
The ability of a third party to change a signature's bytes, for example by replacing s with n − s or re-encoding it, without invalidating it, which changes the transaction's hash.
Appendix: OpenSSL p. 726

SignatureHash
The function that builds the exact byte string a signature signs: a modified copy of the transaction shaped by the sighash flags.
script.h p. 419

single-instance check
The startup guard built on wxSingleInstanceChecker that keeps a second copy of Bitcoin from running and brings the existing window forward instead, protecting the database files. ui.cpp p. 521

sizer
A wxWidgets layout manager that arranges child widgets in rows, columns, or grids and resizes them with the window; this project nests 58 box sizers and 3 flex-grid sizers. uiproject.fbp p. 669

SOCKS4 proxy
An optional relay configured in addrProxy; when set, ConnectSocket dials the proxy and sends a SOCKS4 CONNECT request instead of reaching routable peers directly. net.cpp p. 344

stack
The last-in-first-out list of byte arrays scripts operate on: pushes add data, and each opcode pops its arguments and pushes its result.
script.cpp p. 403

strprintf
A printf-style formatter that returns a std::string, trying a 50,000-byte stack buffer first and doubling a heap buffer until the output fits. util.cpp p. 683

sublicense
The right to pass the software on to others under terms of one's own choosing, which the MIT License grants explicitly alongside copying and selling. license.txt p. 195

summary product
A product listing with no details or order form, the only kind CheckProduct accepts; the check also stamps it with the seller's current atom count from the review database. market.cpp p. 309

symbolic resource name
The identifier opening each line, such as send20 or check, which the code passes to wxBitmap with wxBITMAP_TYPE_RESOURCE, or to the wxICON macro, to fetch the embedded image. ui.rc p. 538

Template Method pattern
The design pattern in which a base class fixes the overall structure and calls overridable hooks, leaving subclasses to supply the specific steps, here the event handlers. uibase.h p. 585

ThreadIRCSeed
The background thread StartNode launches to connect to chat.freenode.net on port 6667, join #bitcoin, advertise this node in its nickname, and harvest peer addresses from other nicknames. irc.h p. 177

ThreadMessageHandler
The below-normal-priority thread that, roughly every 100 milliseconds, runs ProcessMessages and SendMessages for each connected node. net.cpp p. 344

ThreadOpenConnections
The thread that, while fewer than 15 peers are connected, dials a known address, advertises the node's own address if routable, and sends getaddr to learn more. net.cpp p. 344

ThreadSocketHandler
The network I/O thread that uses select() to accept inbound connections, read incoming bytes into each node's vRecv buffer, flush its vSend buffer, and delete disconnected nodes. net.cpp p. 344

trust link
An entry in a user's vLinksOut, added by AcceptReview when that user reviews someone, forming the directed edges along which reputation atoms flow. market.cpp p. 309

TRY_CRITICAL_BLOCK
The non-blocking variant of CRITICAL_BLOCK: it runs the following block only if the lock can be taken immediately, and otherwise skips it. util.h p. 698

Turing-complete
Describes a language that can loop forever; Script deliberately has no loops or recursion, so every script's running time is provably bounded. script.h p. 419

two's complement
The standard way to negate a binary number, flip every bit and add one, used by operator-() to compute additive inverses. uint256.h p. 605

txid
A transaction's unique identifier, the double-SHA256 of its serialized bytes as returned by CTransaction::GetHash(). main.h p. 280

uint160
The 160-bit sibling type used for address hashes, short enough to keep addresses at a manageable length. uint256.h p. 605

uint256
A 256-bit unsigned integer stored as eight 32-bit words; the type of every block hash, transaction ID, and merkle root. uint256.h p. 605

user hash
A marketplace participant's identity, computed by GetUserHash as the double-SHA256 hash of their public key, so reputation attaches to a key rather than to a name. market.h p. 319

UTXO

An unspent transaction output: a coin that has been created but not yet spent. In v0.1.0, spends are recorded on disk in each transaction's CTxIndex (vSpent) when a block connects. main.cpp p. 250 · main.h p. 280

version byte

The leading byte (0×00 on mainnet) prepended before encoding that names the address type and produces the familiar leading 1. base58.h p. 98

version handshake

The exchange of version messages when two nodes connect, negotiating protocol version, capabilities, and time before any other traffic flows. net.h p. 366

virtual event handler

A protected virtual method such as OnButtonSend whose generated body only calls event.Skip(), waiting for a derived class to override it with real logic. uibase.h p. 585

vNodes

The global vector of live peer connections, guarded by cs_vNodes and shared by the socket-handling, connection-opening, and message-handling threads. net.cpp p. 344

warranty disclaimer

The capitalized clause stating the software is provided "AS IS" and that its authors bear no liability for any damage arising from its use. license.txt p. 195

WHO

The IRC command requesting a channel's member list; each answer arrives as numeric reply 352 (RPL_WHOREPLY), whose eighth word, index 7, carries a member's nickname. irc.cpp p. 170

widget ID

An integer constant such as wxID_BUTTONSEND (1002) that identifies a control so events can be routed to it; uibase.h defines 52 of them, numbered from 1000. uibase.h p. 585

_WIN32_WINNT

A macro telling the Windows headers which minimum Windows version to target; headers.h sets it to 0×0400, the value for Windows NT 4.0. headers.h p. 156

windres

The GNU resource compiler that turns the ui.rc resource script and its icons and bitmaps into an object file linked into bitcoin.exe. makefile p. 290

Winsock

Microsoft's implementation of the Berkeley sockets interface, included through winsock2.h and started with WSAStartup before Bitcoin opens any network connection. headers.h p. 156

write-ahead log

Berkeley DB's record of every change, kept in the database subdirectory and written before the data files are updated, so interrupted work can be replayed or rolled back after a crash. db.cpp p. 132

wx.rc

The standard wxWidgets resource script, pulled in on line 3, that supplies the cursors and stock images the toolkit itself expects to find inside every Windows program. ui.rc p. 538

__WXDEBUG__

A preprocessor symbol defined only in debug builds that turns on wxWidgets' debug checks and, through util.h, Bitcoin's debug.log file. makefile p. 290

wxDialog

The wxWidgets class for a secondary window that can run modally, blocking its parent until closed; Send Coins, Options, About, and the address windows derive from it. uibase.h p. 585

wxFormBuilder

A free visual designer for wxWidgets interfaces; Satoshi laid out every window in it, and its April 16, 2008 build generated uibase.cpp and uibase.h from this project file. uiproject.fbp p. 669

wxFrame

The wxWidgets class for an independent top-level window with its own title bar; the main window and the marketplace and poker windows all derive from it. uibase.h p. 585

wxHtmlWindow

A wxWidgets widget that renders simple HTML; the Transaction Details dialog uses one so ui.cpp can lay out a transaction's status, amounts, and addresses as markup. uibase.cpp p. 569

wxNotebook

A tabbed container widget; in v0.1.0 the main window's notebook holds a single All Transactions page containing the transaction list. uibase.cpp p. 569

wxStaticBitmap

A non-interactive widget that simply displays an image; the Send Coins dialog uses a 16-pixel one to show the check icon beside a valid address. uibase.cpp p. 569

wxWidgets

The C++ GUI toolkit that draws Bitcoin's windows, dialogs, and controls; the makefiles link its 2.8 libraries, named wxmsw28 and wxbase28. headers.h p. 156 · ui.h p. 532

wxYield

A wxWidgets call that briefly processes pending events from inside a long loop, keeping the window responsive while ui.cpp reloads a large wallet's transactions. ui.cpp p. 521

x nickname

The random name, x followed by a number below one billion, that a node uses when its own address is not routable, so it can listen in the channel without advertising an unreachable address. irc.cpp p. 170

/YX

An MSVC option that creates and reuses a precompiled header automatically; paired here with /Fp, which stores it as obj/headers.pch. makefile.vc p. 297

zero atom

The atom numbered 0, sent by a reviewer who has no outgoing atoms of their own; its recipient records it but never forwards it further. market.cpp p. 309

INDEX

A

AcceptBlock, 81, 222, 255, 301, 349
AcceptTransaction, 115, 199, 255, 475, 673
Adam Back, 19, 199
addr, 63, 115, 137, 161, 199, 313, 323, 349, 475, 523
AddToBlockIndex, 115, 220, 255, 475

B

b-money, 19, 199, 371, 465
base58.h, 91, 151, 285, 293, 687, 711
Berkeley DB, 701, 115, 137, 151, 199, 255, 285, 293, 301, 313, 371, 407, 475
Bit Gold, 19, 199
BitcoinMiner, 81, 63, 240, 255, 323, 447, 465, 475, 673
blk0001.dat, 115, 255
blkindex.dat, 115, 137, 475
block, 81, 63, 101, 115, 137, 151, 161, 199, 255, 285, 301, 323, 349, 371, 381, 407, 423, 447, 465, 475, 523, 589, 609, 673, 687, 711
blockchain, 115, 137, 199, 255, 323, 423, 475, 523

C

CAddress, 115, 137, 161, 199, 255, 313, 323, 352, 475, 523, 673, 687
CBigNum, 51, 91, 103, 199, 255, 381, 407, 589, 711
CBlock, 51, 115, 199, 270, 313, 349, 423, 475, 673
CBlockIndex, 115, 199, 274, 475, 523
CDataStream, 115, 137, 199, 255, 323, 349, 381, 436, 475, 523, 687
CheckBlock, 222, 255, 673
CheckTransaction, 199, 255
CInv, 199, 323, 356, 475
CKey, 51, 183, 199, 255, 301, 381, 711
CNode, 199, 255, 323, 358, 475, 673
coinbase, 63, 137, 191, 199, 255, 349, 407, 475, 523, 589, 687
ConnectBlock, 115, 218, 255, 423
consensus, 15, 115, 151, 199, 255, 285, 313, 323, 371, 407, 423, 447, 465, 475, 573, 589, 673, 711
COutPoint, 115, 137, 199, 259, 423, 475
CreateTransaction, 81, 247, 255, 381, 475
CScript, 199, 255, 381, 414, 423, 475, 687
CTransaction, 51, 115, 137, 199, 263, 313, 381, 407, 673, 687
CTxIn, 199, 260, 381, 423, 475, 673, 687
CTxOut, 199, 262, 381, 475
cypherpunk, 19, 199, 371

D

David Chaum, 19, 199, 371
db.cpp, 91, 115, 137, 199, 285, 293, 323, 371, 673, 687
db.h, 137, 151, 285, 293
DecodeBase58, 91
difficulty, 19, 63, 101, 115, 137, 199, 255, 323, 589, 687, 711
DigiCash, 19, 199, 371
digital signature, 19, 35, 181, 199, 255, 301, 313, 381, 407, 687, 711
discrete logarithm, 35, 181
double-spend, 15, 199, 255, 447, 475

M

main.cpp, 15, 101, 115, 199, 255, 285, 293, 301, 313, 323, 349, 381, 423, 447, 475, 673, 687
main.h, 101, 151, 199, 255, 285, 293, 313, 475, 589, 673, 687
mempool, 115, 137, 151, 199, 255, 301, 313, 349, 475, 589, 687
merkle tree, 19, 35, 115, 199, 255, 447, 589, 687
message, 115, 151, 161, 181, 199, 301, 323, 349, 423, 447, 475, 523, 541, 573, 609, 673, 687, 711

N

net.cpp, 15, 115, 161, 175, 199, 285, 293, 301, 323, 349, 475, 673, 687
net.h, 151, 285, 293, 301, 323, 349, 673, 687, 711
Nick Szabo, 19, 199
node, 15, 115, 137, 161, 175, 181, 191, 199, 255, 313, 323, 349, 371, 381, 407, 447, 475, 523, 541, 673, 687, 711
nonce, 115, 181, 199, 255, 447, 465, 589, 673, 711

O

OP_CHECKSIG, 63, 199, 381, 407, 475, 711
OP_DUP, 63, 381, 407, 475
OP_EQUALVERIFY, 63, 381, 407, 475
OP_HASH160, 63, 381, 407, 475
OpenSSL, 35, 101, 151, 181, 199, 285, 293, 371, 381, 447, 465, 475, 535, 541, 609, 673, 687, 711
orphan block, 199

P

peer-to-peer, 15, 101, 151, 161, 175, 199, 255, 301, 313, 323, 349, 371, 465, 475, 541, 573, 609
private key, 29, 35, 91, 115, 137, 181, 199, 381, 407, 465, 589, 673, 711
ProcessBlock, 81, 223
proof-of-work, 15, 19, 35, 81, 63, 101, 115, 191, 199, 255, 285, 293, 301, 323, 371, 447, 465, 475, 523, 541, 573, 589, 609, 673, 687
public key, 29, 35, 91, 115, 137, 181, 199, 255, 301, 313, 371, 381, 407, 465, 475, 711

R

RIPEMD-160, 19, 35, 91, 151, 371, 381, 407, 465, 589, 687, 711
RPOW, 19, 199

S

satoshi, 15, 701, 91, 101, 115, 137, 151, 161, 175, 181, 191, 199, 255, 285, 293, 301, 313, 323, 349, 371, 381, 407, 423, 447, 465, 475, 523, 535, 541, 573, 589, 609, 673, 687, 711
Satoshi Nakamoto, 15, 91, 101, 115, 137, 151, 161, 175, 181, 191, 199, 255, 285, 293, 301, 313, 323, 349, 371, 381, 407, 423, 465, 475, 523, 535, 541, 573, 589, 609, 673, 687, 711
script.cpp, 15, 101, 285, 293, 381, 407, 465, 687, 711
script.h, 151, 285, 293, 407, 687
scriptPubKey, 63, 91, 199, 255, 381, 407, 475
scriptSig, 63, 199, 255, 381, 407
secp256k1, 35, 181, 199, 447, 711
SendMoney, 81, 249, 255, 475, 673
serialize.h, 101, 151, 181, 285, 293, 349, 423, 589, 687
SHA-256, 19, 29, 35, 91, 151, 199, 255, 285, 293, 313, 371, 381, 407, 447, 465, 589, 673, 687, 711
Sign, 91, 101, 181, 199, 255, 301, 313, 381, 407, 475, 673, 711
socket, 151, 161, 175, 199, 323, 349, 687
Solver, 63, 381

T

ThreadBitcoinMiner, 199, 339, 349, 475
timestamp, 115, 137, 199, 255, 301, 349, 475, 589, 673, 687
transaction, 81, 101, 115, 137, 151, 181, 199, 255, 285, 301, 313, 323, 349, 371, 381, 407, 423, 447, 465, 475, 523, 541, 573, 589, 609, 673, 687, 711

U

ui.cpp, 91, 115, 137, 199, 285, 293, 301, 323, 349, 381, 475, 535, 541, 573, 609, 673, 687, 711
uint160, 91, 115, 137, 199, 255, 381, 407, 475, 598, 687
uint256, 51, 91, 101, 115, 137, 151, 181, 199, 255, 285, 293, 301, 313, 323, 349, 381, 407, 475, 523, 600, 673, 687
util.cpp, 285, 293, 323, 475, 673, 687, 711
util.h, 151, 285, 293, 447, 687, 711
UTXO, 63, 137, 151, 191, 199, 255, 407

COLOPHON

Book edition: v0.3.9.

About this edition. *Bitcoin v0.1.0 — Annotated Edition* presents the first public Bitcoin release (January 2009) as a single volume: original C++ source and headers with inline commentary, introductory primers, and reference material. The line-by-line annotations are designed for continuous reading alongside the code, not as a bare source listing.

Source material. Program text is reproduced from Satoshi Nakamoto's v0.1.0 tree. Source code is copyright © 2009 Satoshi Nakamoto and licensed under the MIT/X11 License unless noted in individual files. Commentary, selection, and book design are additional material to that release.

Source verification. The 43 files of the source tree reproduced here (19,820 lines of C++) were checked byte for byte against the Nakamoto Institute archive of the January 2009 release before this edition was typeset. Their combined SHA-256 tree hash is `d1af86797f725470 8bacfc1dde657c55 df2a5c2ab1d37ff1 452ae3b5590d5fba`. Per-file hashes and the recipe are in `src/source-manifest.json` in the project repository; `node scripts/source-manifest.mjs` recomputes them.

The OpenSSL 0.9.8h excerpts in the appendix are reproduced from the OpenSSL Project's release tarball, whose published SHA-1 is `ced4f2da24a202e01ea22bef30ebc8aee274de86`. Their per-file hashes are in `src/openssl-manifest.json`; `node scripts/openssl-source.mjs --tarball openssl-0.9.8h.tar.gz` checks them against a downloaded copy.

Copyright and license. The annotations, introductory primers, selection, and book design are © 2026 PirateHash and, like the code they accompany, are released under the MIT License. The commentary was drafted with AI assistance and revised through human and machine review (see *Credits* below).

Typography. Body and annotations are set in **Inter**. Code listings, headings, and UI labels use **Basis Grotesque Mono Pro** (with a monospaced fallback stack for print).

Production. This volume was generated from a structured HTML manuscript: syntax highlighting via Prism, pagination via Paged.js, and PDF output via headless Chromium (Puppeteer). The toolchain is Node.js-based and open source.

Intended print specification. Trim **7″ × 10″** (executive), black text, perfect-bound paperback; body stock typically **60#** uncoated white or equivalent; cover per printer profile (e.g. gloss or matte laminate). Exact stock and binding may vary by print vendor.

Credits. Annotations by Claude Sonnet 4.6 (Revised by Opus 4.6, GPT 5.6 Sol, Grok 4.5, Kimi K3, Opus 5.5). Publisher mark: **PirateHash**.

Errata and updates. This edition is version-controlled: corrections are recorded in the project's public repository (github.com/pedromvpg/bitcoin-alpha-book) and released as incremented editions. The edition string on the title page and this page (v0.3.9) identifies the content of a given printing; the repository's tags and release history provide the errata trail between printings.